

Every question, solved and explained

How this is arranged

All **1245 questions** from the lecture slides, the reference sheets and both real tests. Grouped by topic, and within each topic ordered **easy first**, so the definitions come before the two-step calculations that assume them.

Every question carries the source it came from. Where the same question was set twice, it appears once and says where else it was asked. The answer is marked, and every other option has a line saying why it fails.

1 Architecture versus Organization

1.1**EASY**

The real test 1, question 33

A cloud computing provider improves application response time by introducing multiple levels of cache memory. This improvement is mainly related to:

- a. Data representation
- b. Character encoding
- c. ISA design
- d. Memory organization

X Data representation

Wrong. Data representation is how numbers are encoded in bits. No number changed here.

X Character encoding

Wrong. Character encoding is ASCII and Unicode, which have nothing to do with cache levels.

X ISA design

Wrong. The instruction set is untouched. Programs run unmodified, they just run faster.

✓ Memory organization

Correct. Levels of cache are an arrangement of memory, which is precisely what memory organization means.

Adding cache levels changes nothing a programmer can see in the instruction set. It changes how memory is arranged behind the scenes.

1.2

EASY

Intro S4

According to the course description, which of these is NOT one of the areas this course covers?

- a. The organization of the CPU
- b. Memory hierarchy
- c. Database normalisation
- d. Performance measurement

✗ The organization of the CPU

Wrong, it is covered. The organization of the CPU is named in the course description.

✗ Memory hierarchy

Wrong, it is covered. Memory hierarchy is named in the course description and gets a whole deck of its own.

✓ Database normalisation

Correct. Database normalisation appears nowhere in the course description; this course is about hardware structure and how programs execute on it.

✗ Performance measurement

Wrong, it is covered. Performance measurement closes the list in the course description.

The course description lists the relationship between hardware and software, the organization of the CPU, memory hierarchy, input/output mechanisms, instruction execution, and performance measurement. Databases belong to a different course.

1.3

EASY

Intro S4

The course description says students will learn how computers are designed, how they execute programs, and how architectural choices affect system performance.

Answer: **true**

That is the closing sentence of the course description, and it is a good statement of what the whole course is for.

1.4

EASY

Intro S9

The introduction says that increased efficiency, through automation and organization, only scratches the surface of what computing technology has afforded humanity.

Answer: **true**

Word for word from the unit introduction. The point is that the effect of computing runs wider than efficiency alone.

1.5

EASY

Intro S9

According to the introduction, what is the position of people who do not use computing technology in their businesses?

- a. They are at a disadvantage
- b. They save money on maintenance
- c. They are unaffected either way
- d. They are legally required to adopt it

✓ They are at a disadvantage

Correct. The word the slide uses is disadvantage.

✗ They save money on maintenance

Wrong. Cost savings are not mentioned anywhere in the introduction.

✗ They are unaffected either way

Wrong. The slide's whole point is that they are affected, and negatively.

✗ They are legally required to adopt it

Wrong. Nothing in the course speaks of a legal requirement.

The introduction says the technology has progressed so much that people not utilizing computing technology in their businesses are at a disadvantage.

1.6

EASY

Intro S11

Definition 1 in the deck says a computer is a programmable device that does what three things?

- a. Stores, retrieves and processes data
- b. Accepts, displays and prints data
- c. Reads, writes and deletes data
- d. Encrypts, transmits and receives data

✓ Stores, retrieves and processes data

Correct. Stores, retrieves, processes, and the word programmable in front of them.

✗ Accepts, displays and prints data

Wrong. Accept and produce output belong to Definition 3, and display and print appear in neither.

✗ Reads, writes and deletes data

Wrong. These are file operations, not the deck's wording.

✗ Encrypts, transmits and receives data

Wrong. Nothing about encryption or transmission appears in any of the three definitions.

Definition 1: a computer is a programmable device that stores, retrieves, and processes data. Three verbs, in that order.

1.7

EASY

Intro S15

Almost every digital device uses the same set of basic components, so examining the personal computer gives insight into the structure of a variety of digital devices.

Answer: **true**

This is why the deck teaches from the personal computer: the parts generalise. The same idea returns later, when microcontrollers in washing machines and cars turn out to use Harvard architecture.

According to the deck, what does thinking about computers in terms of their basic components, and the functionality associated with each, reveal?

- a. The primary functions of the computer in general
- b. The cost of building a computer
- c. The instruction set of the processor
- d. The best operating system for the machine

✓ The primary functions of the computer in general

Correct. Primary functions of the computer in general is the slide's phrase.

✗ The cost of building a computer

Wrong. Cost appears later, as a memory design constraint, not here.

✗ The instruction set of the processor

Wrong. The instruction set is an architectural attribute, taught further down the deck.

✗ The best operating system for the machine

Wrong. Operating systems appear only in the ISA compatibility example.

| The slide's own conclusion: it reveals the primary functions of the computer in general.

1.9

EASY

Intro S17

Computer systems are designed using which two closely related but distinct concepts?

- a. Computer architecture and computer organization
- b. Hardware design and software design
- c. Von Neumann design and Harvard design
- d. The instruction set and the microarchitecture

✓ Computer architecture and computer organization

Correct. These are the two concepts the slide names.

✗ Hardware design and software design

Wrong. Hardware and software is a different division; both concepts here are about the machine.

✗ Von Neumann design and Harvard design

Wrong. Von Neumann and Harvard are two system designs, and they sit inside architecture as one component of it.

✗ The instruction set and the microarchitecture

Wrong. This is a real distinction but not the one the slide draws; the microarchitecture is not named in this deck.

Architecture and organization. They are the two halves the whole first unit is built on, and the deck is careful to call them closely related but distinct.

1.10

EASY

Intro S17

Computer architecture and computer organization define how a computer system _____ and operates internally.

- a. functions
- b. performs
- c. behaves
- d. executes

Answer: **functions**

Functions and operates internally. The pairing is deliberate: functions points at architecture, operates internally points at organization.

1.11**EASY**

Intro S20

Computer architecture answers which question?

- a. What does the computer do?
- b. How does the computer do it?
- c. How fast does the computer do it?
- d. How much does the computer cost?

✓ What does the computer do?

Correct. Architecture answers what, and therefore serves as a blueprint for system design.

✗ How does the computer do it?

Wrong. How does the computer do it is the question organization answers.

✗ How fast does the computer do it?

Wrong. Speed is a result of organization, not the question architecture is said to answer.

✗ How much does the computer cost?

Wrong. Cost is never given as the question either concept answers.

What does the computer do. That is the single cleanest handle on the whole distinction, and the exam has used it.

1.12**EASY**

Intro S20

Because it answers what the computer does, computer architecture serves as a _____ for system design.

- a. blueprint
- b. benchmark
- c. controller
- d. compiler

Answer: blueprint

Blueprint. It pairs with the building analogy two slides later: architecture is the design, organization is the construction.

The in-class question that follows the architecture slides asks:

- a. What is the primary focus of computer architecture?
- b. What does computer organization describe?
- c. Why were CISC architectures initially popular?
- d. Why do we not use only cache memory?

✓ What is the primary focus of computer architecture?

Correct. This is the in-class question on the architecture side.

✗ What does computer organization describe?

Wrong. That is the in-class question that closes the organization section, four slides later.

✗ Why were CISC architectures initially popular?

Wrong. That question belongs to the RISC and CISC deck.

✗ Why do we not use only cache memory?

Wrong. That question belongs to the memory hierarchy deck.

The architecture section closes with this in-class question, and the answer slide follows it immediately.

Computer organization answers which question?

- a. How does the computer do it?
- b. What does the computer do?
- c. Who designed the computer?
- d. When does the computer do it?

✓ How does the computer do it?

Correct. Organization answers how.

✗ What does the computer do?

Wrong. What does the computer do is the question architecture answers.

✗ Who designed the computer?

Wrong. Neither concept is defined by who designed the machine.

✗ When does the computer do it?

Wrong. Timing is not the question either concept is said to answer.

| How does the computer do it. It ensures the architecture is translated into working hardware.

1.15

EASY

Intro S24

In the deck's analogy, computer organization is likened to:

- a. The actual construction of the building
- b. The building design
- c. The building regulations
- d. The architect's fee

✓ The actual construction of the building

Correct. Organization is the construction, the part that makes the design real.

✗ The building design

Wrong. The building design is the analogy for architecture.

✗ The building regulations

Wrong. Regulations are not part of the analogy.

✗ The architect's fee

Wrong. Cost is not part of the analogy.

Architecture is the building design, organization is the actual construction or implementation of the building.

1.16

EASY

Intro S24

Match each concept to the half of the building analogy it stands for.

Computer architecture

The building design

Computer organization

The actual construction of the building

The design against the construction. It is the same shape as the blueprint sentence and the what against how pair, so all three can be learned as one idea.

1.17

EASY

Intro S25

The in-class question that closes the organization section asks:

- a. What does computer organization describe?
- b. What is the primary focus of computer architecture?
- c. How does the computer do it?
- d. Which components are visible to the programmer?

✓ What does computer organization describe?

Correct. This is the second of the two in-class questions in this deck.

✗ What is the primary focus of computer architecture?

Wrong. That is the first in-class question, on the architecture side.

✗ How does the computer do it?

Wrong. That is the question organization is said to answer, not the in-class question asked about it.

✗ Which components are visible to the programmer?

Wrong. No in-class question in this deck is phrased that way.

The organization section closes with this question, mirroring the one that closed the architecture section.

1.18

EASY

Intro S26

Computer organization describes the physical arrangement and implementation of hardware components.

Answer: **true**

This is the answer slide to the second in-class question, and it is the shortest usable definition of organization in the whole deck.

1.19

EASY

Intro S28

The deck says a computer system is not just a collection of hardware parts but a carefully designed system governed by computer architecture and computer organization.

Answer: **true**

That is the opening sentence of the components section, and it sets up why both concepts have to be studied together.

1.20

EASY

Intro S28

Understanding the components of architecture and organization is said to be crucial for all of these EXCEPT:

- a. Designing efficient hardware systems
- b. Writing optimized software
- c. Improving system performance
- d. Choosing a programming language

✗ Designing efficient hardware systems

Wrong, it is on the list, and it heads it.

✗ Writing optimized software

Wrong, it is on the list, phrased as writing optimized software.

✗ Improving system performance

Wrong, it is on the list and closes it.

✓ Choosing a programming language

Correct. Choosing a programming language appears nowhere in this deck.

Three reasons are given: designing efficient hardware systems, writing optimized software, improving system performance. Language choice is not one of them.

1.21

EASY

Intro S29

Computer architecture defines the logical structure and functional behavior of a computer system.

Answer: **true**

The same pair of words as the in-class answer slide, in the reverse order: logical structure and functional behavior.

Understanding both architecture and organization is said to allow all of these EXCEPT:

- a. Engineers to design better systems
- b. Programmers to write optimized code
- c. Organizations to build high-performance computing solutions
- d. Users to repair their own hardware

✗ Engineers to design better systems

Wrong, engineers are named first.

✗ Programmers to write optimized code

Wrong, programmers are named second.

✗ Organizations to build high-performance computing solutions

Wrong, organizations building high-performance computing solutions closes the list.

✓ Users to repair their own hardware

Correct. This appears nowhere in the conclusion.

Three groups are named: engineers, programmers and organizations. Users repairing hardware is not among them.

An engineer describes computer architecture as the "blueprint" of a system. Which analogy best represents computer organization?

- a. The owner of the building
- b. The cost estimate
- c. The construction process implementing the design
- d. The building design

✗ The owner of the building

Wrong. The owner commissions the building. That is neither the design nor the act of building, so it maps to nothing in this pair.

✗ The cost estimate

Wrong. A cost estimate is a budget. Organization is about how the design is realised in hardware, not what it costs.

✓ The construction process implementing the design

Correct. Organization is the construction: the buses, the cache, the control paths that turn the blueprint into a working machine.

✗ The building design

Wrong. The building design IS the blueprint, and the stem has already given the blueprint to architecture. Choosing the building design says architecture and organization are the same thing.

Architecture is what the machine looks like to the programmer, the blueprint. Organization is how that blueprint is actually built and wired up, the construction process.

Which of the following is NOT an example of a computer architecture component?

- a. Memory technology used
- b. Addressing modes
- c. Register organization
- d. Instruction set

✓ Memory technology used

Correct, and note the question asks which is NOT. Whether the machine uses SRAM, DRAM or flash cannot be seen from any program, so it is an organization decision.

✗ Addressing modes

Wrong choice, because addressing modes ARE architecture. They are the ways an instruction may name its operand, written into the ISA.

✗ Register organization

Wrong choice, because register organization IS architecture. The programmer-visible registers are named directly in instructions.

✗ Instruction set

Wrong choice, because the instruction set IS the architecture, the very heart of it.

Architecture is the programmer-visible contract: instruction set, addressing modes, registers. Which memory technology is fitted is an implementation choice, so it belongs to organization.

A laptop and a desktop both use x86 processors. The desktop performs significantly better due to more CPU cores and larger caches. This performance difference arises mainly from:

- a. Organizational differences
- b. Different programming languages
- c. Different operating systems
- d. Architectural incompatibility

✓ Organizational differences

Correct. The stem names cores and caches, and both are organization.

✗ Different programming languages

Wrong. No language is mentioned, and the same program is assumed on both machines.

✗ Different operating systems

Wrong. No operating system is mentioned, and an OS would not account for a hardware gap of this kind.

✗ Architectural incompatibility

Wrong, and impossible. Both are x86, so the architectures are identical, not incompatible.

Both run the same x86 instruction set, so the architecture is identical. Cores and cache size are organization, and organization is where performance comes from.

A bank upgrades its servers to processors with larger caches and faster RAM while retaining the same x86 instruction set. What has primarily changed?

- a. Architecture only
- b. Organization only
- c. Both architecture and organization
- d. Neither architecture nor organization

X Architecture only

Wrong, and backwards. The one thing explicitly UNCHANGED is the instruction set, which is the architecture.

✓ Organization only

Correct. Cache size and RAM speed are implementation, so only the organization moved.

X Both architecture and organization

Wrong. It would need the ISA to have changed too, and the stem rules that out.

X Neither architecture nor organization

Wrong. Something did change: the caches and the memory. Only its category is in question.

The instruction set is untouched, so the architecture is untouched. Bigger caches and faster RAM are pure organization. The giveaway phrase is "retaining the same instruction set".

A computer scientist states that architecture and organization must be studied together because:

- a. Organization alone determines functionality.
- b. Architecture alone determines performance.
- c. Both concepts refer to the same thing.
- d. Architecture defines capabilities, while organization determines efficiency and performance.

✗ Organization alone determines functionality.

Wrong, the two are swapped. Functionality, what the machine can do at all, comes from the architecture.

✗ Architecture alone determines performance.

Wrong, swapped the other way. Performance comes from the organization, which is why two x86 chips differ in speed.

✗ Both concepts refer to the same thing.

Wrong. If they were the same thing there would be nothing to study together, and the whole distinction would collapse.

✓ Architecture defines capabilities, while organization determines efficiency and performance.

Correct, and this is the sentence to memorise for every question in this topic.

The one-line rule for this whole topic: architecture says **WHAT** the machine can do, organization says **HOW WELL** it does it.

A research laboratory designs a custom processor specifically optimized for matrix multiplication in artificial intelligence workloads. This design illustrates which architectural principle?

- a. General-purpose optimization always outperforms specialization.
- b. Matrix operations require no architectural optimization.
- c. Specialized architectures eliminate software dependence.
- d. Domain-specific architectures can significantly improve efficiency for targeted applications.

✗ General-purpose optimization always outperforms specialization.

Wrong, and contradicted by the stem. If general-purpose always won, nobody would build the custom chip described.

✗ Matrix operations require no architectural optimization.

Wrong. The entire scenario exists because matrix work rewards architectural optimisation.

✗ Specialized architectures eliminate software dependence.

Wrong. A specialised chip still needs drivers, libraries and compilers. Hardware never removes the software.

✓ Domain-specific architectures can significantly improve efficiency for targeted applications.

Correct. Narrow the workload, and you may spend the transistors where that workload actually needs them.

Building the silicon around one workload is the definition of a domain-specific architecture, and the payoff is efficiency on that workload.

Which statement best characterizes the distinction between computer architecture and computer engineering?

- a. Computer engineering excludes hardware design.
- b. Both disciplines are identical.
- c. Computer architecture concerns only software.
- d. Computer architecture focuses primarily on the conceptual design and operational behavior of computing systems, whereas computer engineering encompasses the broader design, implementation, and integration of hardware systems.

X Computer engineering excludes hardware design.

Wrong, and the reverse of the truth. Hardware design is the core of computer engineering.

X Both disciplines are identical.

Wrong. Architecture is one concern inside the broader engineering discipline, not a synonym for it.

X Computer architecture concerns only software.

Wrong. Architecture is the interface BETWEEN hardware and software, so it can never be software only.

✓ Computer architecture focuses primarily on the conceptual design and operational behavior of computing systems, whereas computer engineering encompasses the broader design, implementation, and integration of hardware systems.

Correct. Conceptual and operational design on one side, the broader build and integration of hardware on the other.

Architecture is the conceptual and operational design. Engineering is the wider job of designing, implementing and integrating real hardware.

Which statement most accurately reflects the role of abstraction in computer architecture?

- a. Abstraction eliminates hardware complexity entirely.
- b. Abstraction reduces processor speed.
- c. Abstraction only applies to programming languages.
- d. Abstraction enables designers to separate concerns across different layers, allowing software developers and hardware engineers to work independently while maintaining compatibility.

✗ Abstraction eliminates hardware complexity entirely.

Wrong. The complexity is still there, it is merely hidden behind the interface. "Entirely" gives this away.

✗ Abstraction reduces processor speed.

Wrong. Abstraction is a design principle, not a runtime cost imposed on the processor.

✗ Abstraction only applies to programming languages.

Wrong. The layering runs from transistors up through logic, microarchitecture, ISA, operating system and applications.

✓ Abstraction enables designers to separate concerns across different layers, allowing software developers and hardware engineers to work independently while maintaining compatibility.

Correct. Separation of concerns across layers, with compatibility preserved at each interface.

Abstraction hides complexity behind a stable interface, it does not delete it. That separation of layers is what lets the two sides of the industry work independently and still interoperate.

Which book heads the recommended textbook list, described as one of the most widely used undergraduate textbooks?

- a. Structured Computer Organization by Andrew S. Tanenbaum
- b. Computer Organization and Architecture by William Stallings
- c. Computer Organization and Design by Patterson and Hennessy
- d. Essentials of Computer Organization and Architecture by Null and Lobur

✗ Structured Computer Organization by Andrew S. Tanenbaum

Wrong. Tanenbaum is number 3 on the list, 6th Edition, 2013.

✓ Computer Organization and Architecture by William Stallings

Correct. Stallings is number 1, and the widely used note is attached to it.

✗ Computer Organization and Design by Patterson and Hennessy

Wrong. Patterson and Hennessy is number 2, with ARM and MIPS versions available.

✗ Essentials of Computer Organization and Architecture by Null and Lobur

Wrong. Null and Lobur is number 4 on the list.

Stallings is number 1 on the list, 11th Edition, Pearson Education, 2019, and it carries the note about being one of the most widely used undergraduate textbooks.

1.32

MEDIUM

Intro S7

The recommended Stallings text is the _____ Edition, published by Pearson Education in _____ .

BLANK 1

- a. 11th
- b. 9th
- c. 10th
- d. 12th

BLANK 2

- a. 2019
- b. 2013
- c. 2017
- d. 2025

Answer: **11th, 2019**

Stallings, 11th Edition, Pearson Education, 2019. The other years on the list belong to other books: 2013 is Tanenbaum, 2017 and later is Patterson and Hennessy, 2025 is Shiva.

1.33

MEDIUM

Intro S7

Which recommended textbook is available in ARM and MIPS versions?

- a. Computer Organization and Design: The Hardware/Software Interface
- b. Structured Computer Organization
- c. Computer Organization and Architecture
- d. Computer Organization, Design, and Architecture

✓ Computer Organization and Design: The Hardware/Software Interface

Correct. The list notes 5th and 6th Edition, ARM or MIPS versions available, against Patterson and Hennessy.

✗ Structured Computer Organization

Wrong. Tanenbaum is listed as one edition, the 6th, with no instruction set variants.

✗ Computer Organization and Architecture

Wrong. Stallings is listed as the 11th Edition only.

✗ Computer Organization, Design, and Architecture

Wrong. Shiva is listed as the 6th Edition, CRC Press, with no instruction set variants.

The Patterson and Hennessy entry is the one whose editions are noted as ARM or MIPS versions, which fits a book that teaches through a concrete instruction set.

1.34

MEDIUM

Intro S11

Definition 1: a computer is a _____ device that stores, retrieves, and processes data.

- a. programmable
- b. electronic
- c. digital
- d. mechanical

Answer: **programmable**

Programmable is the load-bearing word in Definition 1. Electronic belongs to Definition 3, sophisticated electronic calculating machine.

Definition 2 says a computer performs processes, calculations and operations based on instructions provided by what?

- a. A software or hardware program
- b. The operating system only
- c. The user at a keyboard
- d. An internally stored list only

✓ A software or hardware program

Correct. Software or hardware program is the deck's phrase.

✗ The operating system only

Wrong. The operating system is not mentioned in any of the three definitions.

✗ The user at a keyboard

Wrong. The user is nowhere in Definition 2.

✗ An internally stored list only

Wrong. The internally stored list of instructions is Definition 3, not Definition 2.

Definition 2 says the instructions are provided by a software or hardware program. Both words matter, because instructions can be wired as well as loaded.

Definition 2: a computer is a _____ that performs processes, calculations and operations based on instructions provided by a _____ .

BLANK 1

- a. machine or device
- b. programmable device
- c. calculating machine
- d. digital system

BLANK 2

- a. software or hardware program
- b. list of internally stored instructions
- c. compiler or assembler
- d. sequence of control signals

Answer: machine or device, software or hardware program

Definition 2 in its own words. Programmable device is Definition 1 and calculating machine is Definition 3, so the three definitions are told apart by their opening noun.

In Definition 3, according to what does the computer process the data it accepts?

- a. A list of internally stored instructions
- b. A software or hardware program
- c. The control signals of the peripherals
- d. The order in which the data arrived

✓ A list of internally stored instructions

Correct. A list of internally stored instructions is the phrase, and it is the stored program idea in plain words.

✗ A software or hardware program

Wrong. That phrasing belongs to Definition 2.

✗ The control signals of the peripherals

Wrong. Control signals are an organizational attribute, and they appear later in the deck.

✗ The order in which the data arrived

Wrong. Arrival order is nowhere in the definition.

Definition 3: a sophisticated electronic calculating machine that accepts data, processes the data according to a list of internally stored instructions and produces the resulting output information.

Select the basic components that, according to the deck, all personal computers consist of.

- a. Central Processing Unit
- b. Memory
- c. Circuit board
- d. Storage
- e. Input/output devices
- f. Operating system

✓ Central Processing Unit

Correct, it is on the list and heads it.

✓ Memory

Correct, memory is second on the list.

✓ Circuit board

Correct, and it is the one most often forgotten, because it is the board rather than a part mounted on it.

✓ Storage

Correct, storage is listed separately from memory.

✓ Input/output devices

Correct, input/output devices close the list.

✗ Operating system

Wrong. The operating system is software; the slide lists the basic hardware components only.

Five components: CPU, memory, circuit board, storage, and input/output devices. The list is entirely hardware, which is why the operating system is not on it.

1.39

MEDIUM

Intro S13

The deck says all personal computers consist of the same ____ basic components.

- a. 5
- b. 3
- c. 4
- d. 6

Answer: **5**

Five. Losing count usually means folding storage into memory, or dropping the circuit board, which is the easiest one to overlook.

1.40

MEDIUM

Intro S18

Computer architecture refers simply to the attributes of a computer that are:

- a. Visible to a programmer
- b. Transparent to the programmer
- c. Visible only to the circuit designer
- d. Measured by benchmarks

✓ Visible to a programmer

Correct. Visible to a programmer is the definition, and the test of visibility is impact on the logical execution of a program.

✗ Transparent to the programmer

Wrong, and dangerously close. Transparent to the programmer is the phrase used for organizational attributes such as control signals.

✗ Visible only to the circuit designer

Wrong. Nothing in the definition restricts architecture to the circuit designer.

✗ Measured by benchmarks

Wrong. Benchmarks measure performance, which is an outcome of organization.

Architecture is the attributes visible to a programmer, that is, those with a direct impact on the logical execution of a program. Transparent to the programmer is the opposite phrase, and it defines organization.

1.41

MEDIUM

Intro S18

Architectural attributes are those that have a direct impact on the logical execution of a computer program.

Answer: **true**

That is the test the deck gives for whether an attribute is architectural. If a program can tell the difference, it is architecture.

1.42

MEDIUM

Intro S18

Whether a computer will have a multiply instruction at all is an architectural design issue.

Answer: **true**

Yes. The presence of the instruction changes what a programmer can write, so it is architecture. Only its implementation is organization.

Select the aspects of a computer that computer architecture describes, according to the deck.

- a. Instruction set
- b. Data types
- c. Addressing modes
- d. Register organization
- e. Control signals
- f. Memory technology used

✓ Instruction set

Correct, and it heads the list.

✓ Data types

Correct, data types is second.

✓ Addressing modes

Correct, addressing modes is third.

✓ Register organization

Correct, register organization closes the list of four.

✗ Control signals

Wrong. Control signals are named as an organizational attribute, transparent to the programmer.

✗ Memory technology used

Wrong. The memory technology used is also named as an organizational attribute.

Four items: instruction set, data types, addressing modes, register organization. Control signals and memory technology are the deck's own examples of ORGANIZATIONAL attributes, listed a few slides later.

1.44

MEDIUM

Intro S19

Computer architecture refers to the _____ of a computer system.

- a. functional design and logical structure
- b. operational units and their interconnections
- c. physical arrangement of hardware components
- d. control signals and memory technology

Answer: **functional design and logical structure**

Functional design and logical structure. The three wrong choices are all real phrases from the deck, but every one of them defines organization instead.

1.45

MEDIUM

Intro S20

Select the examples of architectural attributes the deck gives.

- a. The instruction set
- b. The number of bits used to represent various data types
- c. I/O mechanisms
- d. Techniques for addressing memory
- e. The clock speed of the processor

✓ The instruction set

Correct, the instruction set is the first example.

✓ The number of bits used to represent various data types

Correct, and the slide spells out the data types as numbers and characters.

✓ I/O mechanisms

Correct, I/O mechanisms is the third example.

✓ Techniques for addressing memory

Correct, techniques for addressing memory closes the list.

✗ The clock speed of the processor

Wrong. Clock speed is never listed as architectural; two machines of the same architecture can differ in clock speed, which is the point of the laptop example later in the deck.

Four examples: the instruction set, the number of bits used for data types such as numbers and characters, I/O mechanisms, and techniques for addressing memory.

When the deck lists the number of bits used to represent various data types as an architectural attribute, which examples of data types does it give?

- a. Numbers and characters
- b. Integers and pointers
- c. Images and sound
- d. Signed and unsigned

✓ Numbers and characters

Correct. The slide's bracket reads numbers, characters.

✗ Integers and pointers

Wrong. Pointers are never named as a data type in this deck.

✗ Images and sound

Wrong. Graphics come much later, in the data representation deck, and are not the example here.

✗ Signed and unsigned

Wrong. Signed and unsigned are two ways of representing numbers, not the two examples given here.

Numbers and characters, given in brackets on the slide. Both get whole sections of the data representation deck later.

1.47

MEDIUM

Intro S22

The answer slide says computer architecture focuses on:

- a. The functional behavior and logical design of the computer system
- b. The physical arrangement and implementation of hardware components
- c. The speed and battery life of the system
- d. The control signals passing between components

✓ The functional behavior and logical design of the computer system

Correct. This is the answer slide word for word.

✗ The physical arrangement and implementation of hardware components

Wrong. That is the answer to the other in-class question, the one about organization.

✗ The speed and battery life of the system

Wrong. Speed and battery life are what organization determines in the smartphone example.

✗ The control signals passing between components

Wrong. Control signals are an organizational attribute.

Functional behavior and logical design. Set it beside the organization answer slide, physical arrangement and implementation, and the pair is easy to hold.

1.48

MEDIUM

Intro S22

According to the answer slide, the primary focus of computer architecture is the physical arrangement of hardware components.

Answer: **false**

That sentence describes organization. Architecture focuses on the functional behavior and logical design of the system.

1.49

MEDIUM

Intro S23

Computer organization refers to the operational units and their interconnections that:

- a. Realize the architectural specifications
- b. Define the instruction set
- c. Are visible to a programmer
- d. Serve as a blueprint for system design

✓ Realize the architectural specifications

Correct. Operational units and their interconnections that realize the architectural specifications is the definition.

✗ Define the instruction set

Wrong. The instruction set is defined by architecture, not organization.

✗ Are visible to a programmer

Wrong. Visible to a programmer defines architecture; organizational details are transparent to the programmer.

✗ Serve as a blueprint for system design

Wrong. The blueprint is architecture. Organization is the construction.

Realize the architectural specifications. The word realize is what makes organization second in order: architecture specifies, organization realizes.

1.50

MEDIUM

Intro S23

Computer organization refers to the operational units and their interconnections that _____ the architectural specifications.

- a. realize
- b. define
- c. describe
- d. replace

Answer: **realize**

Realize. Architecture defines and describes; organization realizes what has been defined.

Select the examples of organizational attributes the deck gives.

- a. Control signals
- b. Interfaces between the computer and peripherals
- c. The memory technology used
- d. Addressing modes

✓ Control signals

Correct, control signals is the first example given.

✓ Interfaces between the computer and peripherals

Correct, interfaces between the computer and peripherals is the second.

✓ The memory technology used

Correct, the memory technology used is the third.

✗ Addressing modes

Wrong. Addressing modes are listed under architecture, in the group with instruction set, data types and register organization.

Three examples: control signals, interfaces between the computer and peripherals, and the memory technology used.

Computer organization refers to the physical implementation of the system. Select what the deck says that includes.

- a. Hardware components
- b. Interconnections
- c. Control signals
- d. Memory structure
- e. Register organization

✓ **Hardware components**

Correct, hardware components heads the list.

✓ **Interconnections**

Correct, interconnections is second.

✓ **Control signals**

Correct, control signals appears here as well as in the transparent to the programmer sentence.

✓ **Memory structure**

Correct, memory structure closes the list.

✗ **Register organization**

Wrong. Register organization belongs to architecture, alongside instruction set, data types and addressing modes.

Four items: hardware components, interconnections, control signals, memory structure. Register organization is on the architecture list, which is the trap here.

1.53

MEDIUM

Intro S23

Computer organization refers to the _____ of the system, including hardware components, interconnections, control signals and _____.

BLANK 1

- a. physical implementation
- b. logical structure
- c. functional design
- d. instruction set

BLANK 2

- a. memory structure
- b. register organization
- c. addressing modes
- d. data types

Answer: **physical implementation, memory structure**

Physical implementation, and memory structure. The three wrong choices in the second blank are all on the architecture list, so this blank is really asking you which side each item sits on.

1.54

MEDIUM

Intro S24

Computer organization ensures the architecture is _____ into working hardware.

- a. translated
- b. compiled
- c. compared
- d. simplified

Answer: **translated**

Translated into working hardware. Organization is the step that turns a specification into a machine.

The answer slide says computer organization describes:

- a. The physical arrangement and implementation of hardware components
- b. The functional behavior and logical design of the computer system
- c. The instruction set and the addressing modes
- d. The attributes visible to a programmer

✓ The physical arrangement and implementation of hardware components

Correct. This is the answer slide word for word.

✗ The functional behavior and logical design of the computer system

Wrong. That is the answer given for architecture.

✗ The instruction set and the addressing modes

Wrong. Both of those are architectural attributes.

✗ The attributes visible to a programmer

Wrong. Visible to a programmer is the definition of architecture.

Physical arrangement and implementation of hardware components. Its twin, on the architecture side, is functional behavior and logical design.

1.56

MEDIUM

Intro S29

The components of computer architecture are described as typically visible to programmers and as determining:

- a. How software interacts with hardware
- b. How fast the hardware runs
- c. How much power the hardware uses
- d. How the hardware is manufactured

✓ How software interacts with hardware

Correct. How software interacts with hardware is the slide's phrase.

✗ How fast the hardware runs

Wrong. Speed is determined by organization, as the laptop example shows.

✗ How much power the hardware uses

Wrong. Power and battery life are organizational outcomes in the smartphone example.

✗ How the hardware is manufactured

Wrong. Manufacture is not discussed in this deck.

They determine how software interacts with hardware, which is the same idea as the ISA being the interface between hardware and software.

1.57

MEDIUM

Intro S54

In the smartphone example, match each item to the concept that determines it.

Instruction set	Architecture
Data types	Architecture
Speed	Organization
Battery life	Organization
Multitasking capability	Organization

Architecture defines two things here, organization determines three. Notice that everything a user would notice about the phone falls on the organization side.

In the smartphone example, battery life is determined by:

- a. Organization
- b. Architecture
- c. The instruction set
- d. The data types supported

✓ Organization

Correct. Battery life is one of the three organization determines.

✗ Architecture

Wrong. Architecture defines the instruction set and data types in this example, nothing else.

✗ The instruction set

Wrong. The instruction set is on the architecture side of this very example.

✗ The data types supported

Wrong. Data types are also on the architecture side.

Organization determines speed, battery life and multitasking capability. Architecture defines only the instruction set and the data types.

1.59

MEDIUM

Intro S55

Laptops with the same architecture, for example _____, but different organizations in cache size, RAM speed and CPU cores, end up at different _____ levels.

BLANK 1

- a. x86
- b. ARM
- c. MIPS
- d. s390

BLANK 2

- a. performance
- b. compatibility
- c. precision
- d. power

Answer: **x86, performance**

x86, and performance. The example is the cleanest proof that organization is what you feel and architecture is what you can run.

The conclusion states that while architecture provides the blueprint and capabilities, organization determines:

- a. Efficiency and performance
- b. Portability and compatibility
- c. Correctness and precision
- d. Cost and availability

✓ Efficiency and performance

Correct. Organization determines efficiency and performance.

✗ Portability and compatibility

Wrong. Compatibility follows from sharing an architecture, and it is not the word used here.

✗ Correctness and precision

Wrong. Correctness and precision belong to the data representation deck.

✗ Cost and availability

Wrong. Cost is discussed as a memory design constraint, not here.

Efficiency and performance. The whole unit reduces to this sentence: capabilities from architecture, efficiency from organization.

1.61

MEDIUM

Intro S56

Architecture provides the _____ and capabilities, while organization determines _____ and performance.

BLANK 1

- a. blueprint
- b. hardware
- c. interconnection
- d. control

BLANK 2

- a. efficiency
- b. compatibility
- c. capability
- d. complexity

Answer: **blueprint, efficiency**

Blueprint and capabilities on one side, efficiency and performance on the other. Blueprint reappears here from the earlier what does the computer do slide.

1.62

MEDIUM

Rev S2

From the revision class, match each description to the concept it defines.

The functional design and logical structure of a computer system	Computer architecture
The operational units and their interconnections that realize the architectural specifications	Computer organization
Answers: what does the computer do?	Computer architecture
Answers: how does the computer do it?	Computer organization
Hardware details transparent to the programmer, such as control signals	Computer organization

The revision slide puts both definitions side by side. Every phrase here has appeared in an exam question at some point, so the mapping is worth being fluent in.

In the revision summary, which of these does computer architecture describe?

- a. Instruction set, data types, addressing modes, register organization
- b. Control signals, peripheral interfaces, memory technology
- c. Cache size, RAM speed, CPU cores
- d. Speed, battery life, multitasking

✓ Instruction set, data types, addressing modes, register organization

Correct. These are the four aspects architecture describes.

✗ Control signals, peripheral interfaces, memory technology

Wrong. These are the organizational attributes transparent to the programmer.

✗ Cache size, RAM speed, CPU cores

Wrong. These are the organizational differences from the laptop example.

✗ Speed, battery life, multitasking

Wrong. These are what organization determines in the smartphone example.

The revision slide repeats the four aspects from the introduction deck: instruction set, data types, addressing modes, register organization.

From the revision class, select the components of COMPUTER ORGANIZATION.

- a. Control unit
- b. ALU unit
- c. Memory organization
- d. Input/output systems
- e. System buses and interconnections
- f. Pipelining and parallelism
- g. Addressing modes

✓ Control unit

Correct, the control unit heads the list.

✓ ALU unit

Correct, the ALU unit is second.

✓ Memory organization

Correct, memory organization is third.

✓ Input/output systems

Correct, input/output systems is fourth.

✓ System buses and interconnections

Correct, system buses and interconnections is fifth.

✓ Pipelining and parallelism

Correct, pipelining and parallelism closes the list.

✗ Addressing modes

Wrong. Addressing modes is fourth on the ARCHITECTURE list.

Six components of organization: control unit, ALU unit, memory organization, input/output systems, system buses and interconnections, pipelining and parallelism.

A national supercomputing center is evaluating two competing processor designs. Processor X achieves higher benchmark performance but consumes significantly more power. Processor Y delivers slightly lower performance while offering substantially better energy efficiency and scalability. Considering contemporary computer architecture principles for high-performance computing, which evaluation is the most appropriate?

- a. Processor Y may represent the superior architectural choice because modern processor evaluation considers performance/scalability, power efficiency, thermal constraints, and total cost of ownership rather than peak speed alone.
- b. Both processors are architecturally equivalent.
- c. Processor X is always preferable because peak performance is the sole design objective.
- d. Energy efficiency has no relevance in processor architecture.

✓ Processor Y may represent the superior architectural choice because modern processor evaluation considers performance/scalability, power efficiency, thermal constraints, and total cost of ownership rather than peak speed alone.

Correct, and note the hedged verb "may represent". It weighs several criteria instead of crowning one.

✗ Both processors are architecturally equivalent.

Wrong. The stem itself lists differences in performance, power and scalability, so they cannot be equivalent.

✗ Processor X is always preferable because peak performance is the sole design objective.

Wrong on two counts: "always" admits no trade-off, and peak performance is not the sole objective in a facility that must be powered and cooled.

✗ Energy efficiency has no relevance in processor architecture.

Wrong, and the strongest claim of the four. Power is a first-order constraint in high-performance computing, since a machine that cannot be cooled cannot be run.

Modern evaluation weighs performance per watt, thermal limits, scalability and total cost of ownership, not peak speed alone. On this paper, any option containing an absolute such as "always", "sole" or "no relevance" is the wrong one.

Two processors implement exactly the same Instruction Set Architecture but exhibit significantly different execution times when running identical benchmark applications.

Which explanation best accounts for this observation?

- a. Different ISAs always produce identical performance.
- b. The compiler determines all performance differences regardless of hardware.
- c. Identical ISAs require identical transistor layouts.
- d. Variations in clock frequency, cache hierarchy, execution units, and pipeline organization influence performance despite identical ISAs.

X Different ISAs always produce identical performance.

Wrong, and it answers a question that was not asked. The stem says the ISAs are the SAME, and the claim itself is false anyway.

X The compiler determines all performance differences regardless of hardware.

Wrong. The benchmark is identical on both machines, so the compiler is a constant here, not a variable.

X Identical ISAs require identical transistor layouts.

Wrong, and it is the exact misconception being tested. An ISA fixes the interface, never the transistors under it.

✓ Variations in clock frequency, cache hierarchy, execution units, and pipeline organization influence performance despite identical ISAs.

Correct. This is the microarchitecture, and it is where identical-ISA machines diverge.

A shared ISA guarantees the same instructions, never the same speed. Clock, cache, execution units and pipeline depth are microarchitecture, and they are free to differ.

A processor supports backward compatibility with software written twenty years ago despite significant improvements in hardware implementation. Which architectural philosophy is primarily responsible for this capability?

- a. Eliminating compiler optimizations.
- b. Maintaining a stable ISA while evolving the underlying organization and microarchitecture.
- c. Replacing the ISA every processor generation.
- d. Increasing clock frequency only.

X Eliminating compiler optimizations.

Wrong. Twenty-year-old binaries were compiled long ago, so nothing the compiler does now affects whether they run.

✓ Maintaining a stable ISA while evolving the underlying organization and microarchitecture.

Correct. Freeze the interface, evolve the implementation. That is the whole trick of backward compatibility.

X Replacing the ISA every processor generation.

Wrong, and it is the opposite policy. A new ISA every generation would break every old program.

X Increasing clock frequency only.

Wrong. A faster clock makes old software run faster, it does not make it run at all.

Old binaries keep running because the programmer-visible contract, the ISA, is frozen. Everything under it is free to be rebuilt generation after generation.

1.68

HARD

Intro S7

Match each recommended textbook to its author or authors.

Computer Organization and Architecture

William Stallings

Computer Organization and Design: The Hardware/Software Interface

David A. Patterson and John L. Hennessy

Structured Computer Organization

Andrew S. Tanenbaum

Essentials of Computer Organization and Architecture

Linda Null and Julia Lobur

Computer Organization, Design, and Architecture

Sajjan G. Shiva

All five recommended texts in one pass. Note how close the titles are to each other: three of the five contain both the words organization and architecture, so the author is the reliable handle.

1.69

HARD

Intro S12

Definition 3 calls a computer a sophisticated electronic _____ machine that accepts data and produces the resulting output information.

- a. calculating
- b. programmable
- c. processing
- d. computing

Answer: **calculating**

Calculating. Definition 3 is the longest of the three and the only one that names input, processing and output as three separate stages.

In the deck's worked example, deciding whether a multiply instruction is implemented by a special multiply unit or by repeated use of the add unit is:

- a. An organizational issue
- b. An architectural issue
- c. Neither, it is a compiler issue
- d. Both equally

✓ An organizational issue

Correct. The implementation choice is invisible to the program, so it is organizational.

✗ An architectural issue

Wrong. The architectural half of this example is whether the computer will have a multiply instruction at all.

✗ Neither, it is a compiler issue

Wrong. The compiler is not part of this example; both halves are machine design decisions.

✗ Both equally

Wrong. The example exists precisely to separate the two, so it cannot be both.

Whether the machine HAS a multiply instruction is architectural, because a program can see it.
HOW that instruction is built is organizational, because the program cannot tell.

Organizational attributes are described as hardware details that are:

- a. Transparent to the programmer
- b. Visible to the programmer
- c. Defined by the instruction set
- d. Independent of the architecture

✓ Transparent to the programmer

Correct, and the word transparent is worth pausing on: it means unseen, not obvious.

✗ Visible to the programmer

Wrong. Visible to the programmer is the definition of architectural attributes.

✗ Defined by the instruction set

Wrong. The instruction set is itself an architectural attribute.

✗ Independent of the architecture

Wrong. Organization exists precisely to realize the architecture, so it is not independent of it.

Transparent to the programmer. In this sense transparent means the programmer sees straight through them and cannot tell they are there, which is the opposite of visible.

Two laptops share the x86 architecture but reach different performance levels. According to the deck, what differs between them?

- a. Cache size, RAM speed and CPU cores
- b. Instruction set and addressing modes
- c. Data types and register set
- d. The programs they can run

✓ Cache size, RAM speed and CPU cores

Correct. These three are the organizational differences the slide names.

✗ Instruction set and addressing modes

Wrong. Instruction set and addressing modes are architectural, and they are the same on both machines by assumption.

✗ Data types and register set

Wrong. Data types and register set are architectural too, so sharing x86 means sharing them.

✗ The programs they can run

Wrong. Sharing an architecture is precisely what makes the same programs run on both.

Same architecture, different organizations: cache size, RAM speed, CPU cores. The result is different performance levels, with no change to what the machines can be asked to do.

From the revision class, select the components of COMPUTER ARCHITECTURE.

- a. Instruction Set Architecture
- b. Data types and representation
- c. Instruction set and format
- d. Addressing modes
- e. Register set
- f. System design, Von Neumann and Harvard
- g. Control unit
- h. System buses and interconnections

✓ Instruction Set Architecture

Correct, the ISA heads the architecture list.

✓ Data types and representation

Correct, data types and representation is second.

✓ Instruction set and format

Correct, instruction set and format is third.

✓ Addressing modes

Correct, addressing modes is fourth.

✓ Register set

Correct, register set is fifth.

✓ System design, Von Neumann and Harvard

Correct, system design, meaning Von Neumann and Harvard, closes the architecture list.

✗ Control unit

Wrong. The control unit heads the ORGANIZATION list.

✗ System buses and interconnections

Wrong. System buses and interconnections is fifth on the ORGANIZATION list.

Six components of architecture. The control unit and the system buses are on the organization list, which has six of its own.

Match each component to the side it belongs on in the revision class listing.

Instruction Set Architecture	Computer architecture
Register set	Computer architecture
System design, Von Neumann and Harvard	Computer architecture
Control unit	Computer organization
ALU unit	Computer organization
Memory organization	Computer organization
Pipelining and parallelism	Computer organization

Six a side. The reliable test is the one from the definitions: if a program can see it, it is architecture; if it is a physical unit doing the work, it is organization.

2 ISA and Microarchitecture

2.1

EASY

The real test 2, question 10

Which instruction is typically executed immediately after a successful comparison operation?

- a. Divide
- b. Conditional Branch
- c. STORE
- d. LOAD

✗ Divide

Wrong. Divide is arithmetic. It neither reads nor needs the condition flags.

✓ Conditional Branch

Correct. Compare then branch is the standard pair, which is how an if statement is built.

✗ STORE

Wrong. STORE writes a register to memory and ignores the flags.

✗ LOAD

Wrong. LOAD reads memory into a register and also ignores the flags.

A compare sets the condition flags. The instruction that reads those flags and decides where to go next is the conditional branch.

Which instruction category is responsible for moving data between CPU registers and memory?

- a. Floating-point instructions
- b. Data transfer instructions
- c. Control instructions
- d. Logical instructions

✗ Floating-point instructions

Wrong. Floating-point instructions compute on real numbers, they do not exist to move data.

✓ Data transfer instructions

Correct. LOAD, STORE and MOVE are its members.

✗ Control instructions

Wrong. Control instructions change which instruction runs next, not where data sits.

✗ Logical instructions

Wrong. Logical instructions such as AND, OR and NOT compute on values already held.

Data transfer is the category that only moves values around: LOAD, STORE, MOVE. It changes location, never value.

Which instruction would most likely access the main memory?

- a. XOR
- b. LOAD
- c. SHIFT
- d. ADD

✗ XOR

Wrong. XOR is a logical operation between register values.

✓ LOAD

Correct. Reaching into memory for an operand is the entire job of a LOAD.

✗ SHIFT

Wrong. SHIFT moves bits within a single register.

✗ ADD

Wrong. ADD sums register values, which is why compilers work so hard to keep operands in registers.

LOAD fetches an operand from main memory into a register. The others work on values already held in registers.

Which instruction category primarily changes the sequence of program execution?

- a. Arithmetic
- b. Control Transfer
- c. Data Transfer
- d. Logical

✗ Arithmetic

Wrong. Arithmetic computes values, then execution simply falls through to the next instruction.

✓ Control Transfer

Correct. Jump, branch, call and return are its members.

✗ Data Transfer

Wrong. Data transfer moves values between registers and memory, leaving the flow untouched.

✗ Logical

Wrong. Logical instructions compute a result. Only a following branch changes the sequence.

Control transfer instructions, the jumps, branches, calls and returns, write a new value into the program counter and so redirect the flow.

A processor supports arithmetic, logical, branch, load/store, and input/output instructions. These collectively constitute its:

- a. Instruction set
- b. Address bus
- c. Register file
- d. Pipeline stages

✓ Instruction set

Correct. The word "collectively" is the clue: all the instructions together are the set.

✗ Address bus

Wrong. The address bus is a set of wires carrying a location, not a collection of instructions.

✗ Register file

Wrong. The register file is storage inside the CPU, not the operations it can perform.

✗ Pipeline stages

Wrong. Pipeline stages are how one instruction is processed in steps, unrelated to how many instructions exist.

Every instruction a processor understands, taken together, is its instruction set. The five categories listed are the standard grouping of that set.

2.6

EASY

ISA S3

The Instruction Set Architecture defines the interface between:

- a. Hardware and software
- b. The CPU and the memory
- c. The user and the operating system
- d. The compiler and the assembler

✓ Hardware and software

Correct. Hardware and software, and the deck calls it a bridge.

✗ The CPU and the memory

Wrong. The CPU to memory link is carried by the buses, which are an organizational component.

✗ The user and the operating system

Wrong. That interface is the user interface, which this course does not deal with.

✗ The compiler and the assembler

Wrong. Both compiler and assembler are software, and both target the ISA rather than sitting either side of it.

The ISA is the interface between hardware and software, a bridge that allows software programs to communicate with the processor. That one sentence is the reason the ISA sits at the centre of the whole course.

2.7

EASY

ISA S3

The ISA acts as a _____ that allows software programs to communicate with the processor.

- a. bridge
- b. buffer
- c. barrier
- d. bus

Answer: **bridge**

Bridge. It is the deck's own metaphor, and it is worth keeping because a bridge carries traffic both ways: the compiler targets the ISA, and the hardware implements it.

2.8

EASY

ISA S5

The instruction set is defined as:

- a. The complete set of instructions a processor can execute
- b. The instructions a particular program happens to use
- c. The instructions stored in the instruction register
- d. The set of instructions the compiler can generate

✓ The complete set of instructions a processor can execute

Correct. Complete set of instructions a processor can execute.

✗ The instructions a particular program happens to use

Wrong, and this is the RISC observation in disguise: most programs use only a small subset of the available instructions, but the set is still the whole of it.

✗ The instructions stored in the instruction register

Wrong. The instruction register holds one instruction at a time during execution.

✗ The set of instructions the compiler can generate

Wrong. What the compiler emits is a choice; the instruction set is what the hardware supports.

The complete set a processor can execute. The word complete matters: it is the whole vocabulary of the machine, not the subset any one program uses.

2.9

EASY

ISA S5

The instruction set is the _____ set of instructions a processor can execute.

- a. complete
- b. reduced
- c. common
- d. internal

Answer: **complete**

Complete. Reduced is the R in RISC and describes one design philosophy, not the definition of an instruction set.

2.10

EASY

ISA S5

Match each pair of example instructions to the category the deck puts them in.

ADD, SUB

Arithmetic

AND, OR

Logical

JUMP, CALL

Control

LOAD, STORE

Data transfer

Four categories with two examples each. LOAD and STORE being data transfer rather than memory instructions is the distinction RISC leans on hardest.

2.11

EASY

ISA S5

JUMP and CALL are given as examples of which category of instruction?

- a. Control
- b. Arithmetic
- c. Logical
- d. Data transfer

✓ Control

Correct. Control instructions decide what executes next.

✗ Arithmetic

Wrong. Arithmetic is ADD and SUB.

✗ Logical

Wrong. Logical is AND and OR.

✗ Data transfer

Wrong. Data transfer is LOAD and STORE.

Control. Both change which instruction runs next rather than computing a value.

2.12**EASY**

ISA S6

Select the data types the ISA slide says are supported.

- a. Integer
- b. Floating point
- c. Character
- d. Boolean
- e. Pointer

✓ Integer

Correct, integer is first.

✓ Floating point

Correct, floating point is second and gets a deck of its own.

✓ Character

Correct, character is third and leads to ASCII, EBCDIC and Unicode.

✓ Boolean

Correct, Boolean closes the list.

✗ Pointer

Wrong. Pointer is not listed as a data type anywhere in this course, although indirect addressing uses the idea.

Four: integer, floating point, character, Boolean.

2.13**EASY**

ISA S6

The ISA data types component lists _____ supported data types.

- a. 4
- b. 3
- c. 5
- d. 6

Answer: **4**

Four. Boolean is the one most often dropped when the list is recalled from memory.

2.14

EASY

ISA S7

Addressing modes are defined as the methods used to:

- a. Access operands at runtime
- b. Decode the opcode
- c. Allocate registers to variables
- d. Move blocks between cache and memory

✓ Access operands at runtime

Correct. Access operands at runtime.

✗ Decode the opcode

Wrong. Decoding the opcode is the ID stage of the pipeline, and the mode field is separate from the opcode field.

✗ Allocate registers to variables

Wrong. Register allocation is a compiler task.

✗ Move blocks between cache and memory

Wrong. That is cache mapping, a different topic entirely.

Methods used to access operands at runtime. Runtime is the key word: the mode decides where the value is found while the instruction executes.

2.15

EASY

ISA S8

The instruction format is the _____ of an instruction: opcode, operands and addressing mode.

- a. structure
- b. length
- c. meaning
- d. sequence

Answer: **structure**

Structure. In the instruction format deck the same idea is put as the layout of bits in an instruction.

In the example `ADD R1, R2`, which adds R1 and R2 and stores the result in R1, what are the operands?

- a. R1 and R2
- b. ADD only
- c. ADD and R1
- d. R1 only, because the result goes there

✓ R1 and R2

Correct. Both registers are operands.

✗ ADD only

Wrong. ADD is the opcode, not an operand.

✗ ADD and R1

Wrong. ADD is the opcode; mixing it into the operands confuses the two fields.

✗ R1 only, because the result goes there

Wrong. R2 is an operand too, even though it is only read.

Opcode is ADD, operands are R1 and R2. R1 happens to be both a source and the destination, which is exactly the two-address convention.

2.17

EASY

ISA S8

In ADD R1, R2 the opcode is _____ and the operands are _____.

BLANK 1

- a. ADD
- b. R1
- c. R2
- d. R1, R2

BLANK 2

- a. R1 and R2
- b. R1 only
- c. R2 only
- d. ADD and R1

Answer: **ADD, R1 and R2**

Opcode ADD, operands R1 and R2. The instruction stores its result in R1, so R1 serves twice.

2.18

EASY

ISA S9

Within the ISA, types of instructions define:

- a. The various operations a processor can perform on data
- b. The number of registers the processor has
- c. The width of the data bus
- d. The size of the cache

✓ The various operations a processor can perform on data

Correct, straight from the slide.

✗ The number of registers the processor has

Wrong. The register set is a separate ISA component.

✗ The width of the data bus

Wrong. Bus width is organizational.

✗ The size of the cache

Wrong. Cache size is organizational, and the memory deck makes that explicit.

The various operations a processor can perform on data. Instruction types are a way of grouping those operations by the job they do.

2.19 EASY ISA S10

Each instruction type handles a specific category of tasks, and together they form the complete functional capability of a CPU.

Answer: **true**

This is the sentence that justifies teaching instructions by type rather than one at a time.

2.20 EASY ISA S10

Understanding instruction types is said to be essential for all of these EXCEPT:

- a. Writing efficient assembly programs
- b. Designing compilers
- c. Optimizing system performance
- d. Installing device drivers

✗ Writing efficient assembly programs

Wrong, it is on the list and heads it.

✗ Designing compilers

Wrong, designing compilers is second.

✗ Optimizing system performance

Wrong, optimizing system performance closes the list.

✓ Installing device drivers

Correct. Device drivers are not mentioned in this deck.

Three reasons are given: writing efficient assembly programs, designing compilers, optimizing system performance.

2.21 EASY ISA S11

The deck warns that the two terms load and store are often confused and should be carefully distinguished from each other.

Answer: **true**

It does, and the distinguishing rule follows three slides later: load copies into a register, store copies out of one.

2.22

EASY

ISA S14

In a store operation the processor copies data:

- a. From a register into a memory location
- b. From a memory location into a register
- c. From one memory location into another
- d. From the cache into a register

✓ From a register into a memory location

Correct. Store writes memory from a register.

✗ From a memory location into a register

Wrong. That is a load.

✗ From one memory location into another

Wrong. Memory to memory is neither, and RISC forbids it outright.

✗ From the cache into a register

Wrong. The cache is transparent here; the instruction names a memory address and the cache decides whether it can serve it.

Store goes out of the register into memory. Load is the other direction: the processor copies data into a register.

2.23

EASY

ISA S14

Match each operation to what the processor does in it.

Load Copies data into a register

Store Copies data from a register into a memory location

The two directions, side by side. Read them from the register's point of view and they never get mixed up: load fills the register, store empties it into memory.

2.24

EASY

ISA S15

Data movement instructions must include information identifying the source and destination.

Answer: **true**

They must. A copy with only one end named would have nowhere to put the data.

2.25

EASY

ISA S16

Arithmetic and logic operations provide the primary _____ capabilities of the computer.

- a. data-processing
- b. decision-making
- c. data movement
- d. control

Answer: **data-processing**

Data-processing. Decision-making belongs to test and branch, and data movement belongs to load and store, so each instruction type has its own capability word.

2.26

EASY

ISA S19

Test and branch operations provide which capability in computer programs?

- a. Decision-making
- b. Data-processing
- c. Data movement
- d. Input and output

✓ Decision-making

Correct. Decision-making capabilities is the deck's phrase.

✗ Data-processing

Wrong. Data-processing belongs to arithmetic and logic instructions.

✗ Data movement

Wrong. Data movement belongs to load and store.

✗ Input and output

Wrong. Input and output is an organizational component, not one of these three instruction types.

Decision-making. Each instruction type gets its own capability: movement, processing, decisions.

2.27

EASY

ISA S20

Depending on the result of the test, the processor may or may not branch to a remote location for the next instruction.

Answer: **true**

May or may not is the point: the branch is conditional, which is precisely what makes it a control hazard for a pipeline.

2.28

EASY

Rev S9

The revision class repeats that instruction types determine how a program does all of these EXCEPT:

- a. Schedules its own threads
- b. Moves data
- c. Makes decisions
- d. Interacts with memory and I/O devices

✓ Schedules its own threads

Correct. Thread scheduling is nowhere in this course.

✗ Moves data

Wrong, moves data is on the list.

✗ Makes decisions

Wrong, makes decisions is on the list.

✗ Interacts with memory and I/O devices

Wrong, interacts with memory and I/O devices closes the list.

The revision list is the same four as the ISA deck: moves data, performs computations, makes decisions, interacts with memory and I/O devices.

2.29

EASY

Rev S11

The revision class repeats that test and branch operations provide:

- a. The decision-making capabilities in computer programs
- b. The primary data-processing capabilities
- c. The data movement capabilities
- d. The interrupt handling capabilities

✓ The decision-making capabilities in computer programs

Correct, unchanged from the ISA deck.

✗ The primary data-processing capabilities

Wrong. That belongs to arithmetic and logic.

✗ The data movement capabilities

Wrong. That belongs to load and store.

✗ The interrupt handling capabilities

Wrong. Interrupts are not covered in this course.

Decision-making, and the revision slide repeats the same four examples, BCC, BEQ, BMI and BRA.

2.30

EASY

Rev S12

The revision class lists BCC, BEQ, BMI and BRA again on its slide about branching to a remote location.

Answer: **true**

It does. The four examples appear twice in the revision deck, on consecutive slides, which is a fair signal that they are worth knowing.

2.31

EASY

Intro S30

The introduction deck calls the Instruction Set Architecture:

- a. The core of computer architecture
- b. The core of computer organization
- c. One organizational attribute among many
- d. A property of the compiler

✓ The core of computer architecture

Correct. The ISA is the core of computer architecture.

✗ The core of computer organization

Wrong. Organization's components are the control unit, ALU, memory organization, I/O, buses, and pipelining and parallelism.

✗ One organizational attribute among many

Wrong. It is architectural, and it is first among architecture's components rather than one among many organizational ones.

✗ A property of the compiler

Wrong. The compiler targets the ISA; it does not own it.

The core of computer architecture, and it acts as the interface between hardware and software. It heads the list of architecture's six components.

2.32

EASY

Intro S30

The ISA acts as the interface between hardware and software.

Answer: **true**

Both the introduction deck and the ISA deck say so, in the same words. It is the single most repeated sentence in the course.

2.33

EASY

Intro S31

Mobile apps are built for the _____-based processors in smartphones.

- a. ARM
- b. x86
- c. MIPS
- d. s390

Answer: **ARM**

ARM. It pairs with the other example on the slide: Windows is optimized for x86.

2.34

EASY

Intro S31

The deck's example of an operating system designed for a specific ISA is:

- a. Windows, optimized for x86 architecture systems
- b. Android, optimized for s390 systems
- c. Linux, optimized for MIPS systems
- d. Windows, optimized for ARM systems

✓ Windows, optimized for x86 architecture systems

Correct. Windows OS is optimized for x86 architecture systems.

✗ Android, optimized for s390 systems

Wrong. Android is not named in this deck, and s390 appears only in the CISC examples.

✗ Linux, optimized for MIPS systems

Wrong. Linux is not named in this deck.

✗ Windows, optimized for ARM systems

Wrong pairing. ARM is the smartphone example on this slide, and it is paired with mobile apps, not with Windows.

Windows on x86. An operating system is software too, so it is bound to an ISA in exactly the same way an application is.

2.35

EASY

Intro S31

An operating system is designed for a specific ISA.

Answer: **true**

It is, and the deck gives Windows on x86 as the example.

2.36

EASY

Intro S32

The introduction deck repeats its real-life applications slide on software compatibility, operating systems and differing organizations of the same ISA.

Answer: **true**

Slides 31 and 32 of the introduction deck carry the same three points. Repetition in a lecture deck is usually a signal about what will be asked.

2.37

MEDIUM

The real test 1, question 22

A game console experiences improved frame rates because the compiler uses efficient memory-access techniques. Which architectural component is most directly involved?

- a. Control signals
- b. System buses
- c. Addressing modes
- d. Memory chips

X Control signals

Wrong. Control signals are generated by the hardware itself. A compiler cannot select them.

X System buses

Wrong. Buses are physical organization. The compiler cannot choose a different bus.

✓ Addressing modes

Correct. The stem says the COMPILER did it, and addressing modes are the memory-access choice available to a compiler.

X Memory chips

Wrong. Memory chips are hardware fitted to the board, invisible to and unselectable by the compiler.

Addressing modes are the ways an instruction may name its operand, and they are the part of the ISA a compiler actually chooses between when it optimises memory access.

Which of the following would most likely be visible to a programmer?

- a. Memory chip technology
- b. Cache implementation details
- c. Control signal generation circuitry
- d. Instruction formats

X Memory chip technology

Wrong. A program behaves the same whichever memory technology is fitted, so the technology is invisible to it.

X Cache implementation details

Wrong. Cache is deliberately transparent: it changes timing, never results.

X Control signal generation circuitry

Wrong. Control circuitry is deep microarchitecture, generated inside the processor and never named by a program.

✓ Instruction formats

Correct. A programmer or compiler must know the exact bit layout to encode an instruction at all.

Instruction formats are part of the ISA, which is by definition the programmer-visible interface. Everything else listed sits below that line, in the organization.

A programmer wants to know whether a processor supports floating-point instructions. This concern relates to:

- a. Control signals
- b. Memory hierarchy
- c. Bus organization
- d. Computer architecture

✗ Control signals

Wrong. Control signals implement instructions that already exist, they do not decide which exist.

✗ Memory hierarchy

Wrong. The memory hierarchy affects how quickly operands arrive, not whether an instruction is supported.

✗ Bus organization

Wrong. Bus organization is about moving data between units, again a speed matter.

✓ Computer architecture

Correct. The set of supported instructions is exactly what the architecture defines.

Which instructions EXIST is an architecture question, settled by the ISA. How fast they run would be an organization question.

A compiler attempts to minimize memory accesses by keeping frequently used variables inside processor registers. Which instruction category benefits the most?

- a. Register-register arithmetic
- b. Privileged instructions
- c. Input/output instructions
- d. Interrupt instructions

✓ Register-register arithmetic

Correct. Operands already in registers is precisely the condition this category needs.

✗ Privileged instructions

Wrong. Privileged instructions are about protection and operating-system mode, not operand location.

✗ Input/output instructions

Wrong. Input/output instructions talk to devices, so keeping variables in registers does not help them.

✗ Interrupt instructions

Wrong. Interrupt instructions handle external events. They are unaffected by where a variable lives.

Register-register arithmetic takes both operands from registers and writes the result back to a register, so it never touches memory at all.

Which statement best distinguishes ISA from microarchitecture?

- a. ISA specifies transistor layout.
- b. ISA determines cache replacement policies.
- c. ISA specifies the programmer-visible interface.
- d. ISA defines how instructions are executed internally.

✗ ISA specifies transistor layout.

Wrong. Transistor layout is physical implementation, the far end of microarchitecture.

✗ ISA determines cache replacement policies.

Wrong. Cache replacement, whether LRU or random, is invisible to a program and is a microarchitecture policy.

✓ ISA specifies the programmer-visible interface.

Correct, and this is the definition to memorise.

✗ ISA defines how instructions are executed internally.

Wrong. "How instructions are executed internally" is the definition of MICROARCHITECTURE, offered here as the near miss.

The ISA is WHAT the machine does, the interface a programmer codes against. The microarchitecture is HOW it is done inside: transistors, caches, pipelines.

Why are specialized instructions such as SIMD instructions introduced into modern processors?

- a. To increase instruction fetch time
- b. To simplify cache design
- c. To eliminate registers
- d. To accelerate data-parallel computations

✗ To increase instruction fetch time

Wrong, and nobody adds a feature to make fetching SLOWER.

✗ To simplify cache design

Wrong. SIMD makes the datapath wider and the design more complex, not the cache simpler.

✗ To eliminate registers

Wrong, and backwards. SIMD needs wide vector registers, so it adds register state rather than removing it.

✓ To accelerate data-parallel computations

Correct. One instruction, many data elements, which is where the name comes from.

SIMD is Single Instruction, Multiple Data: one instruction applies the same operation to many values at once, which is exactly what graphics and matrix work need.

A software developer successfully runs the same compiled application on two processors from different manufacturers. This is possible because both processors implement the same:

- a. Instruction Set Architecture
- b. Microarchitecture
- c. Control unit design
- d. Cache hierarchy

✓ Instruction Set Architecture

Correct. The ISA is the contract a binary is written against.

✗ Microarchitecture

Wrong, and the point of the question is that the two microarchitectures are almost certainly different.

✗ Control unit design

Wrong. Control unit design is internal and varies freely between manufacturers.

✗ Cache hierarchy

Wrong. Cache sizes and levels differ between vendors and are invisible to the binary anyway.

A compiled binary is a stream of ISA instructions. Any processor implementing that ISA can execute it, whoever built it and however differently it is built inside.

2.44

MEDIUM

ISA S3

In simple terms, what does the ISA specify?

- a. What operations a processor can perform
- b. How instructions are represented
- c. How data is accessed and manipulated
- d. How many clock cycles each instruction takes

✓ What operations a processor can perform

Correct, and the mechanism is the instruction set.

✓ How instructions are represented

Correct, and the mechanism is the instruction format.

✓ How data is accessed and manipulated

Correct, and the mechanism is the addressing modes.

✗ How many clock cycles each instruction takes

Wrong. Cycle counts are an implementation matter; two machines with the same ISA can differ in them, which is the whole point of the same ISA, different speed idea.

Three things, and the deck names the mechanism for each: the instruction set, the instruction format, and the addressing modes.

2.45

MEDIUM

ISA S3

Match each thing the ISA specifies to the mechanism that carries it.

What operations a processor can perform

Instruction set

How instructions are represented

Instruction format

How data is accessed and manipulated

Addressing modes

Three questions, three mechanisms. Each of the three becomes a topic of its own later in the deck.

The deck lists the special registers of the ISA as:

- a. PC, IR, MAR, MDR
- b. PC, SP, AC, R1
- c. MAR, MDR, ALU, CU
- d. R1, R2, R3, R4

✓ PC, IR, MAR, MDR

Correct. These four are the special registers named on the slide.

✗ PC, SP, AC, R1

Wrong. The Stack Pointer and the accumulator appear elsewhere in the course, but this slide's four are PC, IR, MAR and MDR.

✗ MAR, MDR, ALU, CU

Wrong. The ALU and the control unit are units, not registers.

✗ R1, R2, R3, R4

Wrong. R1 to R4 are general purpose registers, the other category.

Program Counter, Instruction Register, Memory Address Register, Memory Data Register. The other kind is the General Purpose Register.

2.47

MEDIUM

ISA S6

Registers are small, fast storage locations inside the _____, of two types: General Purpose Registers and _____ Registers.

BLANK 1

- a. CPU
- b. cache
- c. main memory
- d. ALU

BLANK 2

- a. Special
- b. Shadow
- c. Segment
- d. Status

Answer: **CPU, Special**

Inside the CPU, and the second type is Special Registers. Inside the CPU is the phrase that separates a register from cache, which is close to the CPU but not inside it.

Select the common addressing modes listed as a key component of the ISA.

- a. Immediate
- b. Direct
- c. Indirect
- d. Register
- e. Indexed
- f. Relative

✓ Immediate

Correct, immediate heads the list.

✓ Direct

Correct, direct is second.

✓ Indirect

Correct, indirect is third.

✓ Register

Correct, register is fourth.

✓ Indexed

Correct, indexed closes the list of five.

✗ Relative

Wrong on this slide. Relative addressing is taught in full later, with $EA = PC + \text{Offset}$, but the key components slide names only five.

Five common types on this slide: immediate, direct, indirect, register, indexed. Relative is taught later in the same deck but is not on this particular list.

The instruction format component of the ISA gives the structure of an instruction. Select its parts.

- a. Opcode
- b. Operand or operands
- c. Addressing mode
- d. Clock cycle count

✓ Opcode

Correct, the opcode is the operation.

✓ Operand or operands

Correct, the operands are what it operates on.

✓ Addressing mode

Correct, the addressing mode says how the operands are reached.

✗ Clock cycle count

Wrong. No instruction carries a cycle count in its own bits.

Opcode, operands, addressing mode. The same three parts reappear in the instruction format deck as opcode, mode and address fields.

Types of instructions determine how a program does what, according to the deck?

- a. Moves data
- b. Performs computations
- c. Makes decisions
- d. Interacts with memory and I/O devices
- e. Allocates its own memory

✓ Moves data

Correct, and it maps onto load and store instructions.

✓ Performs computations

Correct, and it maps onto arithmetic and logic instructions.

✓ Makes decisions

Correct, and it maps onto test and branch instructions.

✓ Interacts with memory and I/O devices

Correct, and it closes the list.

✗ Allocates its own memory

Wrong. Memory allocation is an operating system job, not an instruction type.

Four: moves data, performs computations, makes decisions, interacts with memory and I/O devices. Each maps onto one instruction type taught next.

2.51

MEDIUM

ISA S11

Load and store operations move data between:

- a. A register in the processor and a memory location, or another register
- b. Two memory locations only
- c. The cache and main memory only
- d. The ALU and the control unit

✓ A register in the processor and a memory location, or another register

Correct, and note the or another register, which allows register to register moves.

✗ Two memory locations only

Wrong. Memory to memory movement is the CISC pattern, and even there it is not what load and store mean.

✗ The cache and main memory only

Wrong. Cache to memory traffic is handled by the cache hardware, not by load and store instructions the programmer writes.

✗ The ALU and the control unit

Wrong. No instruction moves data between the ALU and the control unit.

Between a register and a memory location, or another register. They are often collectively called data movement instructions.

2.52

MEDIUM

ISA S11

Load and store operations are often collectively referred to as _____ instructions.

- a. data movement
- b. data transfer
- c. memory access
- d. register transfer

Answer: **data movement**

Data movement instructions. Note that the key components slide earlier used data transfer for the same category, so both phrases appear in this course; on this slide the phrase is data movement.

2.53

MEDIUM

ISA S14

During instruction execution the source operand is modified along with the destination.

Answer: **false**

The source does not get changed, only the destination. Data movement copies rather than moves, despite the name.

2.54

MEDIUM

ISA S14

Which set of instructions is given as MC68009 load and store examples?

- a. LDA, STA, LDB, STB, LDD, STD
- b. MOVE, MOVEM, EXG
- c. ADDDB, SUBA, SUBD, ANDA, EORA
- d. BCC, BEQ, BMI, BRA

✓ LDA, STA, LDB, STB, LDD, STD

Correct. These are the MC68009 load and store examples.

✗ MOVE, MOVEM, EXG

Wrong. MOVE, MOVEM and EXG are the MC68000 data movement examples.

✗ ADDDB, SUBA, SUBD, ANDA, EORA

Wrong. These are MC6809 arithmetic and logic instructions.

✗ BCC, BEQ, BMI, BRA

Wrong. These are test and branch instructions.

The LD and ST pairs belong to the MC68009. Read them as load A, store A, load B, store B, load D, store D.

2.55

MEDIUM

ISA S14

In the MC68009 the instruction LDA loads accumulator A, and the instruction _____ stores it.

- a. STA
- b. STB
- c. STD
- d. EXG

Answer: **STA**

STA. The six examples pair up: LDA with STA, LDB with STB, LDD with STD.

2.56

MEDIUM

ISA S15

Which MC68000 instruction moves the content of several registers into or out of memory with a single instruction?

- a. MOVEM
- b. MOVE
- c. EXG
- d. LDD

✓ MOVEM

Correct. MOVEM is the multiple register move.

✗ MOVE

Wrong. MOVE handles either direction, but not several registers at once.

✗ EXG

Wrong. EXG exchanges register contents.

✗ LDD

Wrong. LDD is an MC68009 load, not an MC68000 instruction.

MOVEM, where the M is for multiple. MOVE handles one item in either direction and EXG swaps two registers.

What are the minimum arithmetic operations required of a processor?

- a. Addition and subtraction
- b. Addition, subtraction, multiplication and division
- c. Addition only
- d. Addition and multiplication

✓ Addition and subtraction

Correct. Those two are the stated minimum.

✗ Addition, subtraction, multiplication and division

Wrong. Multiplication and division can be built from the minimum, which is the RISC example of multiplying by repeated addition.

✗ Addition only

Wrong. Subtraction is included in the minimum, even though it can be done with complement arithmetic.

✗ Addition and multiplication

Wrong. Subtraction, not multiplication, joins addition in the minimum.

Addition and subtraction. The deck even shows why subtraction is not a separate problem: 5 minus 3 is 5 plus minus 3, which is why two's complement matters so much.

2.58

MEDIUM

ISA S16

Which basic logic operations do all processors implement on a bit-by-bit basis between two operands?

- a. AND
- b. OR
- c. Exclusive-OR
- d. NAND

✓ AND

Correct, AND is first.

✓ OR

Correct, OR is second.

✓ Exclusive-OR

Correct, exclusive-OR closes the list, and it appears again as the EOR and EORA instructions.

✗ NAND

Wrong. NAND is not named on this slide.

AND, OR and exclusive-OR, all bit by bit between two operands. NAND is a gate this course uses in logic design, but it is not on the deck's list of basic operations all processors implement.

2.59

MEDIUM

ISA S16

The basic logic operations are applied on a bit-by-bit basis between two operands.

Answer: **true**

Bit by bit is what makes them logic operations rather than arithmetic ones: no carry crosses from one bit position to the next.

2.60

MEDIUM

ISA S16

With operations requiring two operands, smaller microprocessors usually require that one operand must be in a _____ register initially.

- a. destination
- b. source
- c. index
- d. general purpose

Answer: **destination**

Destination. The result lands where one of the inputs already was, which is the two-address convention in hardware terms.

2.61

MEDIUM

ISA S17

Smaller microprocessors support operations such as ADDing or ANDing to:

- a. Processor registers, but not to memory locations
- b. Memory locations, but not to processor registers
- c. Both equally
- d. Neither, only to the accumulator

✓ Processor registers, but not to memory locations

Correct. The deck says as illustrated but not to memory locations.

✗ Memory locations, but not to processor registers

Wrong, and it inverts the restriction.

✗ Both equally

Wrong. The whole point of the slide is that the two are not equal.

✗ Neither, only to the accumulator

Wrong. The accumulator restriction belongs to the MC6809 examples specifically, and even there the target is a register.

To processor registers, not to memory locations, and their instructions may identify at most a single operand in memory.

2.62

MEDIUM

ISA S17

The instructions of smaller microprocessors may identify at most a single operand in memory.

Answer: **true**

At most one. That single memory operand limit is what keeps the instruction short and the hardware simple.

2.63

MEDIUM

ISA S18

In the MC6809 examples ADDB, SUBA, SUBD, ANDA and EORA, what does each of these require?

- a. One operand to be in an accumulator
- b. Both operands to be in memory
- c. An explicit source and destination
- d. An immediate operand

✓ One operand to be in an accumulator

Correct. One operand in an accumulator is the MC6809 requirement.

✗ Both operands to be in memory

Wrong. That is not permitted here.

✗ An explicit source and destination

Wrong, and this is the MC68000 rule instead: all its two-operand instructions must specify source and destination.

✗ An immediate operand

Wrong. Immediate operands are not required by these instructions.

Each requires one operand in an accumulator, which is why the instruction names end in A, B or D: the letter says which accumulator.

2.64

MEDIUM

ISA S18

Among the MC6809 two-operand examples, the instruction _____ performs exclusive-OR.

- a. EORA
- b. ANDA
- c. SUBD
- d. ADDB

Answer: **EORA**

EORA. In the MC68000 list the same operation appears without the accumulator letter, as EOR.

Select the two-operand arithmetic and logic instructions listed for the MC68000.

- a. ADD
- b. AND
- c. OR
- d. SUB
- e. EOR
- f. ANDA

✓ ADD

Correct, ADD is on the MC68000 list.

✓ AND

Correct, AND is on the list.

✓ OR

Correct, OR is on the list, and note it has no MC6809 counterpart in the deck.

✓ SUB

Correct, SUB is on the list.

✓ EOR

Correct, EOR closes the list.

✗ ANDA

Wrong. ANDA, with the accumulator letter, is an MC6809 instruction.

Five: ADD, AND, OR, SUB, EOR. The MC6809 versions carry an accumulator letter on the end, which is how the two lists are told apart.

2.66

MEDIUM

ISA S19

The test in a test and branch instruction is often performed on the result of some prior _____ operation.

- a. arithmetic/logic
- b. load/store
- c. input/output
- d. control transfer

Answer: **arithmetic/logic**

A prior arithmetic or logic operation. This is why the two instruction types sit next to each other: one computes and sets condition codes, the other reads them.

2.67

MEDIUM

ISA S19

During the execution of a test and branch instruction, what does the processor examine?

- a. The appropriate condition code bits
- b. The opcode of the previous instruction
- c. The contents of the program counter
- d. The tag field of the cache line

✓ The appropriate condition code bits

Correct. The condition code register is what the test reads.

✗ The opcode of the previous instruction

Wrong. The previous opcode is gone by then; what it left behind is the condition codes.

✗ The contents of the program counter

Wrong. The PC says where to fetch next, and relative addressing uses it, but the test does not examine it.

✗ The tag field of the cache line

Wrong. Cache tags belong to the memory deck and have nothing to do with branching.

The appropriate condition code bits, which record whether the previous result was zero or non-zero, positive or negative, or some other binary choice.

What does the test in a test and branch instruction determine?

- a. Whether the result was zero or non-zero
- b. Whether the result was positive or negative
- c. Some other binary choice involving bits in the condition code register
- d. How many cycles the previous instruction took

✓ Whether the result was zero or non-zero

Correct, zero or non-zero is the first.

✓ Whether the result was positive or negative

Correct, positive or negative is the second.

✓ Some other binary choice involving bits in the condition code register

Correct, and it leaves room for carry, overflow and the rest.

✗ How many cycles the previous instruction took

Wrong. Cycle counts are not recorded in condition codes.

| All three are on the slide, and each of them is binary, which is why a single bit can record it.

Which of the listed test and branch instructions branches unconditionally?

- a. BRA
- b. BCC
- c. BEQ
- d. BMI

✓ BRA

Correct. Branch always, so no condition is examined.

✗ BCC

Wrong. BCC branches only if the carry is cleared.

✗ BEQ

Wrong. BEQ branches only if the result was equal to 0.

✗ BMI

Wrong. BMI branches only if the result was minus.

BRA, branch always. The other three each read a condition code before deciding.

In the revision class, which processor is credited with the load and store examples LDA, STA, LDB, STB, LDD and STD?

- a. The MC68009
- b. The MC68000
- c. The MC6809
- d. The IBM 801

✓ The MC68009

Correct. The load and store examples are the MC68009's.

✗ The MC68000

Wrong. The MC68000 contributes MOVE, MOVEM and EXG.

✗ The MC6809

Wrong. The MC6809 contributes ADDDB, SUBA, SUBD, ANDA and EORA.

✗ The IBM 801

Wrong. The IBM 801 is an early RISC machine and has no instruction examples in this course.

The MC68009, exactly as in the ISA deck. The MC6809 supplies the arithmetic examples on the same revision slide and the MC68000 supplies MOVE, MOVEM and EXG in the ISA deck.

Select what the introduction deck says the ISA defines.

- a. The set of instructions a processor can execute
- b. Instruction formats, for example ADD, LOAD, STORE
- c. Data types, integer and floating point
- d. Registers available
- e. Addressing modes
- f. Cache size

✓ The set of instructions a processor can execute

Correct, and it heads the list.

✓ Instruction formats, for example ADD, LOAD, STORE

Correct, with ADD, LOAD and STORE given as the examples.

✓ Data types, integer and floating point

Correct, with integer and floating point given as the examples.

✓ Registers available

Correct, registers available is fourth.

✓ Addressing modes

Correct, addressing modes closes the list.

✗ Cache size

Wrong. Cache size is an organizational difference, named in the laptop example.

Five items. Cache size is organizational, which is exactly why two machines with the same ISA can differ in performance.

2.72

MEDIUM

Intro S30

The introduction deck gives ADD, LOAD and STORE as examples of instruction ____ .

- a. formats
- b. types
- c. modes
- d. sets

Answer: **formats**

Formats, as the slide has it. The ISA deck itself treats ADD, LOAD and STORE as instruction categories, so hold the wording of this slide separately.

2.73

MEDIUM

Intro S31

Applications compiled for a specific ISA, for example x86 or ARM:

- a. Will only run on processors supporting that ISA
- b. Will run on any processor with enough memory
- c. Will run on any processor of the same speed
- d. Will run anywhere once recompiled by the operating system at load time

✓ Will only run on processors supporting that ISA

Correct. Compatibility follows the ISA, not the machine.

✗ Will run on any processor with enough memory

Wrong. Memory size has nothing to do with which instructions the processor understands.

✗ Will run on any processor of the same speed

Wrong. Speed is organizational and irrelevant to compatibility.

✗ Will run anywhere once recompiled by the operating system at load time

Wrong. Nothing in this course describes load-time recompilation.

Only on processors supporting that ISA. That is why mobile apps are built for the ARM-based processors in smartphones.

2.74

MEDIUM

Intro S31

Different _____, that is different hardware implementations, can support the same ISA but vary in _____.

BLANK 1

- a. organizations
- b. architectures
- c. instruction sets
- d. compilers

BLANK 2

- a. performance
- b. compatibility
- c. capability
- d. correctness

Answer: **organizations, performance**

Organizations, and performance. Say it the other way round to check yourself: if two machines run the same programs at different speeds, the ISA is shared and the organization is not.

Which of the following is NOT defined by an Instruction Set Architecture?

- a. Internal ALU implementation
- b. Addressing modes
- c. Instruction formats
- d. Programmer-visible registers

✓ Internal ALU implementation

Correct, and remember the question asks which is NOT. Two chips with the same ISA may use completely different adder designs, and no program can tell.

✗ Addressing modes

Wrong choice. Addressing modes ARE in the ISA, since an instruction must state how it names its operand.

✗ Instruction formats

Wrong choice. Instruction formats ARE in the ISA, since the bit layout must be fixed for code to be encoded.

✗ Programmer-visible registers

Wrong choice. Programmer-visible registers ARE in the ISA, since instructions name them directly.

The ISA defines the instructions, their formats, their addressing modes and the registers you can name. How the ALU is built to carry out an ADD is microarchitecture.

2.76

HARD

ISA S14

One of the universal characteristics of instruction execution is that:

- a. The source does not get changed, only the destination
- b. The destination does not get changed, only the source
- c. Both source and destination change
- d. Neither changes until the write back stage

✓ The source does not get changed, only the destination

Correct, and the deck calls it universal.

✗ The destination does not get changed, only the source

Wrong, and it is exactly backwards.

✗ Both source and destination change

Wrong. If the source changed too, copying would be destructive, which it is not.

✗ Neither changes until the write back stage

Wrong. Write back is a pipeline stage, and this characteristic is about which operand changes, not when.

The source is read, the destination is written. This is why a load leaves memory untouched and a store leaves the register untouched.

2.77

HARD

ISA S15

Match each MC68000 data movement instruction to what it does.

MOVE Either direction of data movement

MOVEM Moves the content of several registers into or out of memory with a single instruction

EXG Exchanges register contents

Three instructions, three jobs. MOVEM is the interesting one: one instruction moving several registers is exactly the kind of complex instruction CISC is built on.

For two-operand operations, what do smaller microprocessors usually require?

- a. One operand must be in a destination register initially, while the other may be in another register or in memory
- b. Both operands must be in registers
- c. Both operands must be in memory
- d. One operand must be immediate

✓ One operand must be in a destination register initially, while the other may be in another register or in memory

Correct, straight from the slide.

✗ Both operands must be in registers

Wrong, and that is the RISC rule, register to register only, not the rule for smaller microprocessors here.

✗ Both operands must be in memory

Wrong. Memory to memory is the CISC pattern and is not what this slide requires.

✗ One operand must be immediate

Wrong. Immediate operands are one addressing mode among several, not a requirement.

One in a destination register, the other in a register or in memory. It is the accumulator idea generalised, and it is why the MC6809 examples all need an operand in an accumulator.

2.79

HARD

ISA S18

All two-operand instructions in the MC68000 instruction set must specify the source and the destination. With a few exceptions, these may be:

- a. Both registers, or both memory locations, or one of each
- b. Both registers only
- c. One register and one immediate value only
- d. Both memory locations only

✓ Both registers, or both memory locations, or one of each

Correct. The deck lists all three possibilities, with a few exceptions.

✗ Both registers only

Wrong. Register to register only is the RISC restriction, not the MC68000's.

✗ One register and one immediate value only

Wrong. Immediate values are not what the slide is contrasting here.

✗ Both memory locations only

Wrong. Memory to memory is allowed but is not the only option.

Both registers, both memory locations, or one of each. That freedom, memory to memory included, is a distinctly CISC trait.

2.80

HARD

ISA S20

Match each test and branch instruction to its meaning.

BCC	Branch if carry is cleared
------------	----------------------------

BEQ	Branch if equal to 0
------------	----------------------

BMI	Branch if minus
------------	-----------------

BRA	Branch always
------------	---------------

Four examples. BRA is the odd one: it always branches, so it tests nothing and is an unconditional jump.

Different organizations, that is different hardware implementations, can support the same ISA but:

- a. Vary in performance
- b. Run different software
- c. Use different instruction formats
- d. Support different addressing modes

✓ Vary in performance

Correct. Same ISA, same software, different speed.

✗ Run different software

Wrong. Sharing an ISA is precisely what makes the same software run on both.

✗ Use different instruction formats

Wrong. Instruction formats are part of the ISA, so they are shared.

✗ Support different addressing modes

Wrong. Addressing modes are part of the ISA, so they are shared too.

They vary in performance. This is the single most examined idea in the whole architecture against organization topic.

3 Number Systems and Codes

3.1

EASY

The real test 1, question 1

BCD stands for:

- a. Binary Control Digit
- b. Base Conversion Decimal
- c. Binary Coded Decimal
- d. Binary Character Decimal

✗ Binary Control Digit

Wrong. No such term. "Control" belongs to the control unit, not to a numeric code.

✗ Base Conversion Decimal

Wrong. Base conversion is an operation you perform, not the name of a code.

✓ Binary Coded Decimal

Correct. Binary Coded Decimal: decimal digits, each coded in binary.

✗ Binary Character Decimal

Wrong. "Character" points at text codes such as ASCII, while BCD codes numbers.

Binary Coded Decimal. Each decimal digit is coded separately in four bits, which is why 59 becomes 0101 1001 rather than the pure binary 111011.

The decimal value 255 represented in base 16 is:

- a. FE
- b. FF
- c. AE
- d. F0

✗ FE

Wrong. FE is $15 \times 16 + 14 = 254$, one short.

✓ FF

Correct. Both digits are F, that is $15 \times 16 + 15 = 255$.

✗ AE

Wrong. AE is $10 \times 16 + 14 = 174$.

✗ F0

Wrong. F0 is $15 \times 16 + 0 = 240$.

255 = $15 \times 16 + 15$, and 15 is F, so the answer is FF. Worth memorising: 255 is FF, is 11111111, and is the largest value eight unsigned bits can hold.

Why is hexadecimal preferred for displaying binary information?

- a. It increases precision
- b. It groups bits conveniently into sets of four
- c. It reduces storage requirements
- d. It speeds processor execution

✗ It increases precision

Wrong. Changing the display base cannot change the value or its precision.

✓ It groups bits conveniently into sets of four

Correct. Four bits per digit means a 32-bit word reads as eight tidy characters.

✗ It reduces storage requirements

Wrong. The stored bits are identical. Only the way they are printed changes.

✗ It speeds processor execution

Wrong. Hexadecimal is a notation for humans. The processor works in binary either way.

One hex digit is exactly four bits, because 16 is 2 to the power 4. The bits are simply easier for a human to read.

Unicode was introduced primarily to:

- a. Replace binary arithmetic
- b. Support multiple world languages
- c. Improve floating-point performance
- d. Reduce storage

✗ Replace binary arithmetic

Wrong. Unicode is a character code. It has nothing to do with arithmetic.

✓ Support multiple world languages

Correct. One code space covering the world's writing systems.

✗ Improve floating-point performance

Wrong. Floating point is a numeric format, untouched by a text standard.

✗ Reduce storage

Wrong, and backwards. Unicode uses more bits per character than ASCII, so it costs storage rather than saving it.

ASCII in seven bits covers 128 characters, enough for English alone. Unicode was created to give every script in the world a code point.

A company logo needs to be enlarged to billboard size without losing quality. Which format is most appropriate?

- a. ASCII
- b. Bitmap
- c. Vector
- d. BCD

✗ ASCII

Wrong. ASCII encodes text characters, not images.

✗ Bitmap

Wrong, and this is the trap. A bitmap has a fixed pixel grid, so blowing it up to billboard size shows the blocks.

✓ Vector

Correct. Shapes are stored as maths and redrawn cleanly at any scale.

✗ BCD

Wrong. BCD is a numeric code for decimal digits, with no image capability.

A vector image stores shapes as mathematical descriptions, so it is redrawn at any size. A bitmap stores a fixed grid of pixels, so enlarging it enlarges the pixels.

3.6

EASY

Data S2

Inside a computer, information is represented by currents, voltage differences, or, for permanent storage:

- a. The orientation of magnetic material
- b. The temperature of the circuit
- c. The resistance of the wire
- d. The frequency of the clock

✓ The orientation of magnetic material

Correct. Magnetic orientation is the deck's example of permanent storage.

✗ The temperature of the circuit

Wrong. Temperature is never used to carry information here.

✗ The resistance of the wire

Wrong. Resistance is not one of the three the slide names.

✗ The frequency of the clock

Wrong. The clock paces the machine; it does not store its data.

Currents and voltage differences while the machine runs, orientation of magnetic material when it has to survive a power cut.

3.7

EASY

Data S2

A single electronic circuit or piece of magnetic material is often regarded as being in one of two states, so it can be used to represent a binary digit.

Answer: **true**

This is the sentence that connects the physics to the arithmetic, and everything else in the deck rests on it.

3.8**EASY**

Data S3

Although data is stored in binary, it is often displayed in hexadecimal because:

- a. It takes up less space on the page and is easier to remember
- b. The hardware works in base 16
- c. Hexadecimal arithmetic is faster
- d. Binary cannot represent large numbers

✓ It takes up less space on the page and is easier to remember

Correct. Both reasons are on the slide.

✗ The hardware works in base 16

Wrong. The hardware has two states, so it works in base 2.

✗ Hexadecimal arithmetic is faster

Wrong. The arithmetic is the same numbers either way; only the notation changes.

✗ Binary cannot represent large numbers

Wrong. Binary represents any number hexadecimal can, just with four times as many digits.

Less space, easier to remember. Hexadecimal is a convenience for people; the machine is still in binary.

3.9**EASY**

Data S3

Hexadecimal is base ____ .

- a. 16
- b. 8
- c. 10
- d. 32

Answer: **16**

Base 16, which is 2 to the power 4, and that is exactly why four bits map onto one hex digit.

3.10

EASY

Data S5

To convert an integer into a digit string in a base, repeatedly _____ the number by the base.

- a. divide
- b. multiply
- c. subtract
- d. shift

Answer: **divide**

Divide. Going the other way, from a base into decimal, you multiply instead.

3.11

EASY

Data S10

The arithmetic in a base conversion computation can be performed in any base you like.

Answer: **true**

The deck says so. The method does not care what base you do the sums in; only the answer's notation changes.

Conversion between binary and hexadecimal amounts to grouping or ungrouping bits into sets of:

- a. Four
- b. Three
- c. Eight
- d. Two

✓ Four

Correct. One hex digit is exactly four bits.

✗ Three

Wrong. Three bits is one octal digit.

✗ Eight

Wrong. Eight bits is a byte, which is two hex digits.

✗ Two

Wrong. Two bits would give base 4, which this course does not use.

Four, because 16 is 2 to the power 4. Sets of three would give octal, and sets of eight give a byte.

3.13**EASY**

Data S13

To divide a binary number by 2, you:

- a. Shift the digits right by one place
- b. Shift the digits left by one place
- c. Take the one's complement
- d. Subtract the number from itself

✓ Shift the digits right by one place

Correct. Every bit moves one place towards the low end, so each is worth half what it was.

✗ Shift the digits left by one place

Wrong. Shifting left multiplies by 2.

✗ Take the one's complement

Wrong. The one's complement negates; it does not divide.

✗ Subtract the number from itself

Wrong. That gives zero.

Right shift halves, left shift doubles. In base 2 the shift is the arithmetic.

3.14**EASY**

Data S13

Shifting the digits of a binary number left by one place multiplies it by 2.

Answer: **true**

It does, and the deck gives it as the companion to the right shift. It is the same reason that adding a zero to the end of a decimal number multiplies it by ten.

3.15

EASY

Data S41

Binary Coded Decimal is:

- a. A type of binary code used to represent a given decimal number in an equivalent binary form
- b. The pure binary value of a decimal number
- c. A method of representing negative numbers
- d. A character code for letters and symbols

✓ A type of binary code used to represent a given decimal number in an equivalent binary form

Correct. It is the deck's definition word for word.

✗ The pure binary value of a decimal number

Wrong, and that is the distinction that matters: pure binary converts the whole number, BCD codes each digit separately.

✗ A method of representing negative numbers

Wrong. Negative numbers use sign magnitude, one's complement or two's complement.

✗ A character code for letters and symbols

Wrong. Character codes are ASCII, EBCDIC and Unicode.

BCD codes a decimal number digit by digit, which is why 59 becomes 0101 1001 rather than the pure binary 111011.

3.16

EASY

Data S41

BCD stands for Binary _____ Decimal.

- a. Coded
- b. Control
- c. Character
- d. Converted

Answer: **Coded**

Coded. Test 1 asked for this expansion directly, with Binary Control Digit and Binary Character Decimal among the distractors.

3.17**EASY**

Data S41

The deck names _____ different types of BCD.

- a. 3
- b. 2
- c. 4
- d. 5

Answer: **3**

Three: 8421, 5421, 4221.

3.18**EASY**

Data S43

BCD code is described as useful for outputting to displays that are:

- a. Always numeric, 0 to 9, such as digital clocks or digital voltmeters
- b. Always textual, such as terminals
- c. Graphical, such as monitors
- d. Printed, such as line printers

✓ Always numeric, 0 to 9, such as digital clocks or digital voltmeters

Correct, with both examples as given.

✗ Always textual, such as terminals

Wrong. Text needs an alphanumeric code such as ASCII.

✗ Graphical, such as monitors

Wrong. Graphics are covered separately, as bitmap and vector.

✗ Printed, such as line printers

Wrong. Printers are mentioned as devices interfaced using alphanumeric codes.

Always numeric, and the deck names digital clocks and digital voltmeters. If a display can only show digits, coding digit by digit costs nothing and saves the conversion.

3.19**EASY**

Data S43

The deck's two examples of always numeric displays are digital clocks and digital ____ .

- a. voltmeters
- b. cameras
- c. watches
- d. calculators

Answer: **voltmeters**

Voltmeters. Both examples show only digits, which is the condition that makes BCD worthwhile.

3.20**EASY**

Data S45

The Excess-3 code is determined by adding ____ to each decimal digit and then replacing each digit by its four bit binary equivalent.

- a. 3
- b. 1
- c. 2
- d. 9

Answer: **3**

3, which is where the name comes from: the code is in excess of the true value by 3.

3.21**EASY**

Data S56

Earlier computers were used only for the purpose of calculations, that is, only as a calculating device.

Answer: **true**

The deck opens the alphanumeric codes section this way, to explain why letters and symbols needed encoding at all.

3.22

EASY

Data S56

A computer is a digital system and can only deal with _____ and 0's, so it uses alphanumeric codes for letters and symbols.

- a. 1's
- b. digits
- c. words
- d. bytes

Answer: **1's**

1's and 0's. Two symbols only, so everything else has to be coded into them.

3.23

EASY

Data S57

Using alphanumeric codes, which devices can be interfaced with the computer?

- a. Keyboards, monitors and printers
- b. Only keyboards
- c. Only storage devices
- d. Only network cards

✓ Keyboards, monitors and printers

Correct, all three are named.

✗ Only keyboards

Wrong. Monitors and printers are named too.

✗ Only storage devices

Wrong. Storage is not the example given.

✗ Only network cards

Wrong. Network cards are not mentioned in this course.

Input-output devices such as keyboards, monitors and printers. Each of them deals in characters, so each needs the same code.

3.24

EASY

Data S59

ASCII stands for:

- a. American Standard Code for Information Interchange
- b. American Symbolic Code for Internal Interchange
- c. Advanced Standard Code for Information Interchange
- d. American Standard Character Interchange Index

✓ American Standard Code for Information Interchange

Correct. American Standard Code for Information Interchange, a 7-bit character code.

✗ American Symbolic Code for Internal Interchange

Wrong. Standard, not Symbolic, and Information, not Internal.

✗ Advanced Standard Code for Information Interchange

Wrong. American, not Advanced.

✗ American Standard Character Interchange Index

Wrong. Code for Information Interchange, not Character Interchange Index.

American Standard Code for Information Interchange, a 7-bit character code.

3.25

EASY

Intro S33

Data types and representation define how data is _____ and interpreted.

- a. stored
- b. moved
- c. encrypted
- d. compressed

Answer: **stored**

Stored and interpreted. Two machines can store the same bits and interpret them differently, which is exactly why the representation has to be specified.

3.26

EASY

Intro S34

For which kind of system does the deck say accurate floating-point representation is critical?

- a. Banking software in financial systems
- b. Gaming systems
- c. Embedded systems
- d. Smart homes

✓ Banking software in financial systems

Correct. Financial systems: accurate floating-point representation is critical for banking software.

✗ Gaming systems

Wrong. Gaming systems appear under addressing modes and under the ALU, not here.

✗ Embedded systems

Wrong. Embedded systems appear under addressing modes.

✗ Smart homes

Wrong. Smart homes appear under input/output systems.

Financial systems: accurate floating-point representation is critical for banking software. The Vancouver Stock Exchange story later in the course is what happens when it is not.

3.27

EASY

Intro S34

The deck says accurate floating-point representation is critical for banking software.

Answer: **true**

It does, under Financial Systems in the real-life applications of data types.

If a hexadecimal value is shifted right by one bit, the operation is equivalent to:

- a. Multiplication by 2
- b. Division by 16
- c. Division by 2
- d. Multiplication by 16

✗ Multiplication by 2

Wrong direction. Multiplying by 2 is a shift LEFT.

✗ Division by 16

Wrong. Dividing by 16 needs a shift of four bits, or one whole hex digit. The stem says one BIT.

✓ Division by 2

Correct. Each bit position right is one halving.

✗ Multiplication by 16

Wrong on both counts, wrong direction and wrong factor.

Shifting is a BIT operation, so the base you display the number in is irrelevant. One bit right halves the value.

3.29

MEDIUM

The real test 1, question 17

The 8421 BCD representation of decimal 59 is:

- a. 00110101
- b. 00111011
- c. 01011100
- d. 01011001

✗ 00110101

Wrong. 0011 0101 is BCD for 35, the digits reversed.

✗ 00111011

Wrong, and this is the trap. 00111011 is the PURE BINARY 59, not its BCD coding.

✗ 01011100

Wrong. The first nibble 0101 is right for 5, but 1100 is 12, which is not a valid BCD digit at all.

✓ 01011001

Correct. 0101 for 5 and 1001 for 9.

Code each digit on its own in four bits: 5 is 0101 and 9 is 1001, so 59 is 0101 1001.

A binary number occupies 20 bits. How many hexadecimal digits are needed to represent it exactly?

- a. 4
- b. 6
- c. 8
- d. 5

✗ 4

Wrong. 4 hex digits hold only 16 bits.

✗ 6

Wrong. 6 hex digits hold 24 bits, more than asked for, and the question says exactly.

✗ 8

Wrong. 8 hex digits hold 32 bits, the size of a full word.

✓ 5

Correct. 20 divided by 4 is 5.

Four bits per hex digit, so 20 divided by 4 is 5. Run the rule backwards and a 5-digit hex value needs 20 bits.

Which code was specifically designed to simplify decimal arithmetic operations?

- a. Unicode
- b. ASCII
- c. EBCDIC
- d. Excess-3

✗ Unicode

Wrong. Unicode is a character code for the world's languages.

✗ ASCII

Wrong. ASCII is a character code for text.

✗ EBCDIC

Wrong. EBCDIC is another character code, IBM's rival to ASCII.

✓ Excess-3

Correct, and it is the only NUMERIC code in the list. The other three all encode text.

Excess-3 is a numeric code, built self-complementing so decimal subtraction can be done by complementing and adding.

The Excess-3 code for decimal 9 is:

- a. 1001
- b. 1010
- c. 1100
- d. 1111

✗ 1001

Wrong, and the classic trap. 1001 is plain BCD for 9, with the excess not yet added.

✗ 1010

Wrong. 1010 is 10, which would be the Excess-3 code for 7.

✓ 1100

Correct. $9 + 3 = 12 = 1100$.

✗ 1111

Wrong. 1111 is 15, which would encode the digit 12, and no such decimal digit exists.

| Add the excess of 3 first: $9 + 3 = 12$, and 12 in four bits is 1100.

The hexadecimal value 2A7F corresponds to which binary representation?

- a. 0010101001111111
- b. 1010101111111111
- c. 1010100011111111
- d. 0010101001111110

✓ 0010101001111111

Correct. 0010 1010 0111 1111, read straight off the four digits.

✗ 1010101111111111

Wrong. It starts 1010, which is A, so this decodes as ABFF and not 2A7F.

✗ 1010100011111111

Wrong. Its first two groups give A8, not 2A.

✗ 0010101001111110

Wrong by exactly one bit. It ends 1110, which is E, so the final F is lost. Always check the last nibble.

Expand each digit into four bits: 2 is 0010, A is 1010, 7 is 0111, F is 1111.

Excess-3 code is:

- a. Non-weighted
- b. Weighted
- c. ASCII-based
- d. Floating-point

✓ **Non-weighted**

Correct. No set of positional weights reproduces the digit, because of the added 3.

✗ **Weighted**

Wrong. That describes 8421 BCD, where the bits are worth 8, 4, 2 and 1.

✗ **ASCII-based**

Wrong. ASCII is a character code and has no bearing on Excess-3.

✗ **Floating-point**

Wrong. Excess-3 codes a single decimal digit, and has no sign, exponent or mantissa.

In a weighted code such as 8421 BCD each bit position carries a fixed value you can add up. Excess-3 shifts every code by 3 first, so the positions no longer carry those weights.

The Excess-3 code 0110 corresponds to decimal:

- a. 6
- b. 9
- c. 12
- d. 3

✗ 6

Wrong, and the trap. 6 is the raw binary value before the excess of 3 is removed.

✗ 9

Wrong. That is 6 plus 3, so it adds the excess when decoding instead of subtracting it.

✗ 12

Wrong. 12 is the code you would WRITE for the digit 9, not a digit itself.

✓ 3

Correct. 6 minus 3 is 3.

Decode by subtracting the excess: 0110 is 6, and 6 minus 3 is 3.

Why are numbers usually represented within a computer in base 2?

- a. Because a single circuit or piece of magnetic material is often regarded as being in one of two states
- b. Because base 2 arithmetic is more accurate than base 10
- c. Because base 2 takes up less space on the page
- d. Because programmers find binary easier to read

✓ Because a single circuit or piece of magnetic material is often regarded as being in one of two states

Correct. Two states means it can represent a binary digit, and base 2 follows from that.

✗ Because base 2 arithmetic is more accurate than base 10

Wrong. Accuracy is a matter of how many bits you use, not which base the hardware works in.

✗ Because base 2 takes up less space on the page

Wrong, and reversed: taking up less space on the page is the deck's reason for displaying in hexadecimal.

✗ Because programmers find binary easier to read

Wrong. The deck says the opposite: hexadecimal is used for display because it is easier to remember.

Two physical states, so one binary digit. The representation follows the hardware, not the other way round.

3.37

MEDIUM

Data S3

In the digit sequence formula, what is the last term, the one for the rightmost digit?

- a. a nought times base to the power 0
- b. a nought times base to the power 1
- c. a nought times base
- d. a nought only, with no base at all

✓ a nought times base to the power 0

Correct. The exponent counts up from 0 at the right.

✗ a nought times base to the power 1

Wrong. Power 1 belongs to the digit one place to the left.

✗ a nought times base

Wrong. Multiplying by the base itself is the same mistake as power 1.

✗ a nought only, with no base at all

Wrong in form, even though the value works out the same, because the formula is written with the base to the power 0.

The rightmost digit is multiplied by base to the power 0, which is 1. That is why it contributes its own face value.

3.38

MEDIUM

Data S3

In a digit sequence of n digits, the leftmost digit is multiplied by base to the power

_____ .

- a. $n-1$
- b. n
- c. $n+1$
- d. 0

Answer: **$n-1$**

n minus 1. With n digits the exponents run from n minus 1 down to 0, which is n values in all.

To convert an integer into a digit string in a base, you repeatedly divide by the base. The digits are:

- a. The remainders, generated from right to left
- b. The remainders, generated from left to right
- c. The quotients, generated from right to left
- d. The quotients, generated from left to right

✓ The remainders, generated from right to left

Correct. Remainders, and they are generated from right to left.

✗ The remainders, generated from left to right

Wrong direction. Writing them left to right reverses the number.

✗ The quotients, generated from right to left

Wrong. The quotients are what you keep dividing; they are not the digits.

✗ The quotients, generated from left to right

Wrong on both counts.

Remainders, right to left. The first remainder you get is the last digit you write, which is why the answer is read bottom to top.

In the toBase program, what happens when n is negative?

- a. It returns a minus sign followed by toBase of the positive value
- b. It returns the string 0
- c. It throws an error
- d. It takes the two's complement first

✓ It returns a minus sign followed by toBase of the positive value

Correct. It returns minus, joined to toBase of minus n.

✗ It returns the string 0

Wrong. That is the branch for n equal to 0.

✗ It throws an error

Wrong. Throwing an Invalid digit error belongs to fromDigit, in the other program.

✗ It takes the two's complement first

Wrong. The two's complement is a hardware representation, and this program produces a printable digit string.

The program handles the sign separately, prefixing a minus and recursing on the positive value. Returning 0 is the case for n equal to zero.

Converting decimal 13 to binary by repeated division gives:

- a. 1101
- b. 1011
- c. 1110
- d. 0111

✓ 1101

Correct. 8 plus 4 plus 1 is 13.

✗ 1011

Wrong. 1011 is 11, and it is what you get by reading the remainders in the order they came out.

✗ 1110

Wrong. 1110 is 14.

✗ 0111

Wrong. 0111 is 7.

13 over 2 is 6 remainder 1, 6 over 2 is 3 remainder 0, 3 over 2 is 1 remainder 1, 1 over 2 is 0 remainder 1. Read bottom to top: 1101.

3.42

MEDIUM

Data S7

Dividing 13 by 2 gives quotient 6 remainder _____, and dividing 6 by 2 gives quotient 3 remainder _____.

BLANK 1

- a. 1
- b. 0
- c. 2
- d. 3

BLANK 2

- a. 0
- b. 1
- c. 2
- d. 3

Answer: **1, 0**

Remainder 1 then remainder 0. The remainders in order are 1, 0, 1, 1, and reading them bottom to top gives 1101.

3.43

MEDIUM

Data S8

Converting from a digit string in a base to an integer is described as being like:

- a. Evaluating a polynomial
- b. Solving an equation
- c. Taking a complement
- d. Shifting the bits left

✓ Evaluating a polynomial

Correct, and the method is Horner's scheme in all but name.

✗ Solving an equation

Wrong. Nothing is being solved for.

✗ Taking a complement

Wrong. Complements belong to signed number representation.

✗ Shifting the bits left

Wrong. Shifting left multiplies by 2, which is a different operation covered two slides later.

Like evaluating a polynomial: process the digits left to right, multiply what you have so far by the base, add the next digit.

3.44

MEDIUM

Data S8

Process the digits from left to right, multiply what you have so far by the base, and _____ the next digit.

- a. add
- b. subtract
- c. divide by
- d. multiply by

Answer: **add**

Add. Multiply then add, once per digit, is the whole method.

In the `fromBase` routine, what does the accumulation step do on each character?

- a. result equals result times base, plus `fromDigit` of the character
- b. result equals result plus base, times `fromDigit` of the character
- c. result equals result divided by base, plus the character
- d. result equals `fromDigit` of the character only

✓ result equals result times base, plus `fromDigit` of the character

Correct, and it matches the worked example: 0 times 2 plus 1, then 1 times 2 plus 1, and so on.

✗ result equals result plus base, times `fromDigit` of the character

Wrong. The operations are the wrong way round.

✗ result equals result divided by base, plus the character

Wrong. Dividing belongs to the other direction, decimal into a base.

✗ result equals `fromDigit` of the character only

Wrong. That would discard every digit but the last.

Multiply the running total by the base, then add the new digit. It is the polynomial evaluation from the previous slide written as code.

3.46

MEDIUM

Data S10

Evaluating binary 1101 left to right gives 1, then 3, then 6, then:

- a. 13
- b. 12
- c. 11
- d. 14

✓ 13

Correct, and it agrees with the other direction, where 13 converted to binary gave 1101.

✗ 12

Wrong. 12 would need a final digit of 0.

✗ 11

Wrong. 11 is 1011, a different bit pattern.

✗ 14

Wrong. 14 is 1110.

6 times 2 plus 1 is 13. The four steps are 0 times 2 plus 1 is 1, 1 times 2 plus 1 is 3, 3 times 2 plus 0 is 6, 6 times 2 plus 1 is 13.

3.47

MEDIUM

Data S10

The third step of evaluating 1101 is _____ times 2 plus 0, which is _____ .

BLANK 1

- a. 3
- b. 1
- c. 6
- d. 13

BLANK 2

- a. 6
- b. 3
- c. 7
- d. 12

Answer: **3, 6**

3 times 2 plus 0 is 6. The running total doubles at every step and picks up the next bit.

3.48

MEDIUM

Data S11

Using the conversion table, hexadecimal 3fc is _____ in binary.

- a. 0011 1111 1100
- b. 0011 1111 1010
- c. 0111 1111 1100
- d. 0011 1110 1100

Answer: **0011 1111 1100**

3 is 0011, f is 1111, c is 1100. This is the number the deck then halves two slides later to get 1fe.

3.49

MEDIUM

Data S12

Expanding A34E, the digit A is worth _____ in decimal and the digit E is worth _____ .

BLANK 1

- a. 10
- b. 11
- c. 14
- d. 15

BLANK 2

- a. 14
- b. 15
- c. 13
- d. 11

Answer: **10, 14**

A is 10 and E is 14. Counting up from A gives A 10, B 11, C 12, D 13, E 14, F 15.

3.50

MEDIUM

Data S12

What is hexadecimal A34E in binary?

- a. 1010 0011 0100 1110
- b. 1010 0011 0100 1010
- c. 1011 0011 0100 1110
- d. 1010 0100 0011 1110

✓ 1010 0011 0100 1110

Correct. A is 1010, 3 is 0011, 4 is 0100, E is 1110.

✗ 1010 0011 0100 1010

Wrong. The last group is E, which is 1110, not 1010.

✗ 1011 0011 0100 1110

Wrong. The first group is A, which is 1010, not 1011.

✗ 1010 0100 0011 1110

Wrong. The middle two digits are 3 then 4, so 0011 then 0100.

A is 1010, 3 is 0011, 4 is 0100, E is 1110. Four digits, four groups of four bits.

3.51

MEDIUM

Data S12

In A34E, the hexadecimal digit E is _____ in binary.

- a. 1110
- b. 1111
- c. 1101
- d. 1010

Answer: **1110**

1110, which is 14. 1111 would be F and 1101 would be D.

3.52

MEDIUM

Data S41

The main advantage of BCD is that it allows:

- a. Easy conversion to decimal digits for printing or display, and faster calculations
- b. More numbers to be stored in the same number of bits
- c. Negative numbers to be represented without a sign bit
- d. Letters and digits to share one encoding

✓ Easy conversion to decimal digits for printing or display, and faster calculations

Correct, both halves are on the slide.

✗ More numbers to be stored in the same number of bits

Wrong, and it is the opposite: BCD wastes six of the sixteen patterns in every four bits.

✗ Negative numbers to be represented without a sign bit

Wrong. BCD says nothing about sign.

✗ Letters and digits to share one encoding

Wrong. That would be an alphanumeric code.

Easy conversion for printing or display, and faster calculations. BCD trades density for convenience, which is why it suits displays.

3.53

MEDIUM

Data S41

BCD is chosen because it stores decimal numbers in fewer bits than pure binary does.

Answer: **false**

The advantage claimed is easy conversion for printing or display and faster calculations, not compactness. Coding each digit in four bits uses more space, not less.

Select the three types of BCD the deck names.

- a. 8 4 2 1
- b. 5 4 2 1
- c. 4 2 2 1
- d. 2 4 2 1
- e. Excess-3

✓ 8 4 2 1

Correct, 8421 is the familiar one.

✓ 5 4 2 1

Correct, 5421 is second.

✓ 4 2 2 1

Correct, 4221 is third.

✗ 2 4 2 1

Wrong. 2421 is a real BCD variant in the wider literature, but it is not one of the three this deck names.

✗ Excess-3

Wrong. Excess-3 is a non-weighted code, and the deck introduces it as a separate topic.

8421, 5421 and 4221. Excess-3 is a code in this deck too, but it is non-weighted and is taught separately from BCD.

In the names 8421, 5421 and 4221, what do the four numbers represent?

- a. The weights of the relevant bits
- b. The decimal digits the code can represent
- c. The order in which the bits are transmitted
- d. The number of bits in each group

✓ The weights of the relevant bits

Correct, and that is what makes BCD a weighted code.

✗ The decimal digits the code can represent

Wrong. Every one of the three codes represents the digits 0 to 9.

✗ The order in which the bits are transmitted

Wrong. Transmission order is not discussed anywhere in this deck.

✗ The number of bits in each group

Wrong. All three use four bits per digit, which is why all four names have four numbers.

Weights. Each of the four bit positions carries the weight its name gives, and the digit is the sum of the weights whose bit is 1.

3.56

MEDIUM

Data S43

The weights of 8421 BCD are 8, 4, _____ and _____ .

BLANK 1

- a. 2
- b. 1
- c. 3
- d. 5

BLANK 2

- a. 1
- b. 2
- c. 0
- d. 4

Answer: **2, 1**

8, 4, 2, 1, which are the ordinary powers of two, and that is why 8421 BCD looks like plain binary for a single digit.

BCD is a weighted code, meaning that:

- a. The sum of the weights of the binary digits whose value is 1 equals the decimal digit they represent
- b. Some digits are more important than others and are stored first
- c. The code adds a fixed amount to each decimal digit
- d. The code is self-complementing

✓ The sum of the weights of the binary digits whose value is 1 equals the decimal digit they represent

Correct. It is the definition of a weighted code.

✗ Some digits are more important than others and are stored first

Wrong. Weight here is a numeric value, not a priority.

✗ The code adds a fixed amount to each decimal digit

Wrong, and that is Excess-3, which adds 3 to each digit and is explicitly non-weighted.

✗ The code is self-complementing

Wrong. Self-complementing is a property of Excess-3.

| Add up the weights where the bit is 1. That single rule decodes 8421, 5421 and 4221 alike.

3.58

MEDIUM

Data S44

Excess-3, also called XS3, is:

- a. A non-weighted code used to express decimal numbers
- b. A weighted code used to express decimal numbers
- c. A character code
- d. A method of representing negative numbers

✓ A non-weighted code used to express decimal numbers

Correct, and non-weighted is the word the exam has used.

✗ A weighted code used to express decimal numbers

Wrong, and this is the trap. BCD is weighted; Excess-3 is not.

✗ A character code

Wrong. Character codes are ASCII, EBCDIC and Unicode.

✗ A method of representing negative numbers

Wrong. Excess-3 codes decimal digits, not signs.

Non-weighted. It has no per-bit weights at all, because the value is shifted by 3 before being coded.

3.59

MEDIUM

Data S44

Excess-3 is a weighted code, like 8421 BCD.

Answer: **false**

Excess-3 is explicitly non-weighted. In a weighted code you decode by adding the weights of the 1 bits; in Excess-3 you decode by subtracting 3.

3.60

MEDIUM

Data S44

Excess-3 overcomes the shortcomings of the 8421 BCD code when adding two decimal digits whose sum exceeds ____ .

- a. 9
- b. 10
- c. 8
- d. 15

Answer: **9**

9. Above 9 a single decimal digit will not hold the answer, and 8421 needs a correction step that Excess-3 does not.

3.61

MEDIUM

Data S45

How is the Excess-3 code for a decimal number formed?

- a. Add 3 to each decimal digit, then replace each new digit by its four bit binary equivalent
- b. Convert to binary, then add 3 to the result
- c. Add 3 to the whole number, then convert to binary
- d. Subtract 3 from each decimal digit, then convert

✓ Add 3 to each decimal digit, then replace each new digit by its four bit binary equivalent

Correct, and the order matters: add first, then code.

✗ Convert to binary, then add 3 to the result

Wrong. Adding after conversion would change the value, not the code.

✗ Add 3 to the whole number, then convert to binary

Wrong. The 3 is added to each digit separately, not to the number as a whole.

✗ Subtract 3 from each decimal digit, then convert

Wrong. Subtracting 3 is how you decode Excess-3, not how you form it.

Add 3 per digit first, then code each digit in four bits. Doing it to the whole number instead of digit by digit is the usual mistake.

3.62

MEDIUM

Data S45

What is the XS3 code of 24?

- a. 0101 0111
- b. 0010 0100
- c. 0111 0101
- d. 0101 0100

✓ 0101 0111

Correct. 2 plus 3 is 5, which is 0101, and 4 plus 3 is 7, which is 0111.

✗ 0010 0100

Wrong. That is plain 8421 BCD for 24, with no 3 added.

✗ 0111 0101

Wrong. The digits are the right ones but in the wrong order.

✗ 0101 0100

Wrong. The second digit should be 7, which is 0111, not 4.

| 2 plus 3 is 5, which is 0101, and 4 plus 3 is 7, which is 0111. So 24 becomes 0101 0111.

3.63

MEDIUM

Data S45

For the XS3 code of 24, the digit 2 becomes _____ and the digit 4 becomes _____ before being written in binary.

BLANK 1

- a. 5
- b. 7
- c. 2
- d. 4

BLANK 2

- a. 7
- b. 5
- c. 4
- d. 1

Answer: **5, 7**

| 5 and 7. Then 0101 and 0111.

3.64

MEDIUM

Data S46

The Excess-3 code for decimal 6 is _____, and its 1's complement is _____.

BLANK 1

- a. 1001
- b. 0110
- c. 0011
- d. 1100

BLANK 2

- a. 0110
- b. 1001
- c. 1010
- d. 0101

Answer: **1001, 0110**

6 plus 3 is 9, which is 1001. Flipping every bit gives 0110.

3.65

MEDIUM

Data S47

To convert an Excess-3 code back to decimal, after grouping into fours from the radix point you:

- a. Subtract 0011 from each group
- b. Add 0011 to each group
- c. Take the 1's complement of each group
- d. Shift each group right by three places

✓ Subtract 0011 from each group

Correct. Subtract 3 per group.

✗ Add 0011 to each group

Wrong. Adding is the encoding direction.

✗ Take the 1's complement of each group

Wrong. The 1's complement gives the 9's complement, which is a different property.

✗ Shift each group right by three places

Wrong. Shifting divides by powers of 2 and has nothing to do with the excess.

Subtract 0011, which is 3, from every group. Encoding adds 3, so decoding takes it back off.

3.66

MEDIUM

Data S47

To decode Excess-3, first make groups of ____ bits starting from the radix point.

- a. 4
- b. 3
- c. 8
- d. 2

Answer: **4**

Four bits per group, since each group codes one decimal digit. Starting from the radix point matters, because that is what makes a short leading group pad correctly.

3.67

MEDIUM

Data S56

Why does a computer need alphanumeric codes to deal with letters and symbols?

- a. Because it is a digital system and can only deal with 1's and 0's
- b. Because letters take up more space than numbers
- c. Because keyboards send letters in a different order
- d. Because the ALU cannot perform logic operations on letters

✓ Because it is a digital system and can only deal with 1's and 0's

Correct, and it is the same argument that gave base 2 in the first place.

✗ Because letters take up more space than numbers

Wrong. Space is not the reason given.

✗ Because keyboards send letters in a different order

Wrong. Keyboard order is not discussed.

✗ Because the ALU cannot perform logic operations on letters

Wrong. Once a letter is a bit pattern the ALU can operate on it perfectly well.

The machine has only two symbols, so every letter and mark has to be given a binary pattern. That is what an alphanumeric code is.

Alphanumeric codes are also called:

- a. Character codes
- b. Weighted codes
- c. Self-complementing codes
- d. Binary coded decimals

✓ Character codes

Correct, and the deck gives both names on the same line.

✗ Weighted codes

Wrong. Weighted describes BCD.

✗ Self-complementing codes

Wrong. Self-complementing describes Excess-3.

✗ Binary coded decimals

Wrong. BCD codes digits, not letters and punctuation.

| Character codes. They are binary codes used to represent alphanumeric data.

Alphanumeric codes write which kinds of data in a form a computer can process?

- a. Letters of the alphabet
- b. Numbers
- c. Mathematical symbols
- d. Punctuation marks
- e. Images

✓ Letters of the alphabet

Correct, letters of the alphabet head the list.

✓ Numbers

Correct, numbers are included, which is why a character code can print a digit.

✓ Mathematical symbols

Correct, mathematical symbols are third.

✓ Punctuation marks

Correct, punctuation marks close the list.

✗ Images

Wrong. Images are represented as vector or bitmap graphics.

Four kinds. Images are handled by the graphics section, as bitmap or vector, not by a character code.

Select the alphanumeric codes the deck gives as examples.

- a. Morse code
- b. Hollerith code
- c. ASCII code
- d. EBCDIC code
- e. Unicode
- f. Excess-3

✓ Morse code

Correct, Morse heads the list.

✓ Hollerith code

Correct, Hollerith is second, and it is the punched card code.

✓ ASCII code

Correct, ASCII is third.

✓ EBCDIC code

Correct, EBCDIC is fourth.

✓ Unicode

Correct, Unicode closes the list.

✗ Excess-3

Wrong. Excess-3 expresses decimal numbers, not letters and punctuation.

Five: Morse, Hollerith, ASCII, EBCDIC and Unicode. Excess-3 codes decimal digits, so it is a numeric code, not an alphanumeric one.

3.71

MEDIUM

Data S57

Besides Morse, ASCII, EBCDIC and Unicode, the deck names the _____ code as an alphanumeric code.

- a. Hollerith
- b. Huffman
- c. Hamming
- d. Gray

Answer: **Hollerith**

Hollerith, the punched card code. Hamming and Gray codes are real codes but neither is named in this deck.

3.72

MEDIUM

Data S59

ASCII is a _____ -bit character code.

- a. 7
- b. 8
- c. 16
- d. 6

Answer: **7**

7 bits, which is why the codes run 0 to 127 and why anything above 127 belongs to an 8-bit extension.

Which ASCII character codes are called the printable characters?

- a. 32 to 127
- b. 0 to 31
- c. 128 to 255
- d. 129 to 159

✓ 32 to 127

Correct. Codes 32 to 127 are common to all variations of the ASCII table and are called printable characters: letters, digits, punctuation marks and a few miscellaneous symbols.

✗ 0 to 31

Wrong. Below 32 are the control codes, which the deck does not list here.

✗ 128 to 255

Wrong. 128 to 255 are the extended ASCII codes.

✗ 129 to 159

Wrong. 129 to 159 hold the Microsoft Windows Latin-1 extended characters.

Codes 32 to 127 are common to all variations of the ASCII table and are called printable characters: letters, digits, punctuation marks and a few miscellaneous symbols.

ASCII codes _____ to 127 are the printable characters, and character _____ represents the command DEL.

BLANK 1

- a. 32
- b. 0
- c. 33
- d. 31

BLANK 2

- a. 127
- b. 126
- c. 128
- d. 255

Answer: **32, 127**

32 to 127, and 127 is DEL. So the last of the printable range is not, in fact, printable, which is a fair thing to be caught out by.

3.75

MEDIUM

Data S61

EBCDIC stands for:

- a. Extended Binary Coded Decimal Interchange Code
- b. Extended Byte Coded Decimal Interchange Code
- c. Enhanced Binary Character Data Interchange Code
- d. Extended Binary Character Decimal Information Code

✓ Extended Binary Coded Decimal Interchange Code

Correct. Extended Binary Coded Decimal Interchange Code.

✗ Extended Byte Coded Decimal Interchange Code

Wrong. Binary, not Byte.

✗ Enhanced Binary Character Data Interchange Code

Wrong. Extended, not Enhanced, and Coded Decimal, not Character Data.

✗ Extended Binary Character Decimal Information Code

Wrong. Coded Decimal Interchange, not Character Decimal Information.

Extended Binary Coded Decimal Interchange Code. Its name contains BCD because it descended from a 6-bit binary-coded decimal code.

3.76

MEDIUM

Data S61

EBCDIC is an _____ -bit character encoding used mainly on IBM mainframe and IBM midrange computer operating systems.

- a. 8
- b. 7
- c. 6
- d. 16

Answer: 8

8 bits, against ASCII's 7. That difference is worth holding: ASCII 7, EBCDIC 8.

3.77

MEDIUM

Intro S33

The data types and representation component of architecture defines how data is stored and interpreted. Select what it covers.

- a. Integer formats, signed and unsigned
- b. Floating-point representation
- c. Character encoding such as ASCII and Unicode
- d. The number of registers available

✓ Integer formats, signed and unsigned

Correct, and signed and unsigned are named explicitly.

✓ Floating-point representation

Correct, floating-point representation is second.

✓ Character encoding such as ASCII and Unicode

Correct, with ASCII and Unicode as the examples.

✗ The number of registers available

Wrong. Registers available belong to the register set, a separate architecture component.

Three areas, and each becomes a section of the data representation deck: signed numbers, floating point, character codes.

3.78

MEDIUM

Intro S34

Match each system to what the deck says it depends on.

Financial systems Accurate floating-point representation for banking software

Multimedia systems Data formats such as RGB encoding for video and image processing

Scientific computing Precision in floating-point calculations for simulations

Three systems, three demands. Two of the three are about floating point, which is a fair signal of how much weight that topic carries.

Which data format does the deck name for video and image processing in multimedia systems?

- a. RGB encoding
- b. BCD encoding
- c. EBCDIC encoding
- d. Excess-3 encoding

✓ RGB encoding

Correct. RGB encoding, given as the example under Multimedia Systems.

✗ BCD encoding

Wrong. BCD codes decimal digits for numeric displays.

✗ EBCDIC encoding

Wrong. EBCDIC is a character code for IBM systems.

✗ Excess-3 encoding

Wrong. Excess-3 is a non-weighted decimal code.

RGB encoding, given as the example under Multimedia Systems.

Why is Excess-3 considered self-complementing?

- a. Its 1's complement represents the 9's complement of the original digit
- b. It eliminates overflow
- c. It uses 3-bit encoding
- d. It automatically detects errors

✓ Its 1's complement represents the 9's complement of the original digit

Correct. Bit inversion in the code equals 9's complement in decimal, which is what self-complementing means.

✗ It eliminates overflow

Wrong. No code eliminates overflow. Overflow is a range problem, not an encoding problem.

✗ It uses 3-bit encoding

Wrong. Excess-3 uses FOUR bits per digit. The 3 in its name is the excess added to the digit, not a bit count.

✗ It automatically detects errors

Wrong. Error detection needs a parity or checksum scheme. Excess-3 carries no check bit.

Take 3, whose Excess-3 code is 0110. Invert every bit and you get 1001, the Excess-3 code for 6, and 6 is the 9's complement of 3. Inverting the bits performs the decimal 9's complement for free.

A processor receives the hexadecimal value 7D3A. What is its decimal equivalent?

- a. 32090
- b. 32058
- c. 32042
- d. 32122

✗ 32090

Wrong, 32 too high. It is what you get by mis-weighting the third digit.

✓ 32058

Correct. $28672 + 3328 + 48 + 10 = 32058$.

✗ 32042

Wrong, 16 too low, the result of reading the 3 as a 2.

✗ 32122

Wrong, 64 too high. Check the $3 \times 16 = 48$ term.

Weight each digit by its power of 16: $7 \times 4096 = 28672$, D is 13 so $13 \times 256 = 3328$, $3 \times 16 = 48$, and A is 10. Adding gives $28672 + 3328 + 48 + 10 = 32058$.

A forensic analyst discovers that a malware author stored the value 111010111011 (binary). What is its hexadecimal equivalent?

- a. EBB
- b. EBB16
- c. EAB
- d. EBB and EBB16 are equivalent

✗ EBB

Right value, but incomplete. The paper also offers the same value written with its base, so a is not the fullest answer.

✗ EBB16

Also right in value. EBB16 is EBB with the base 16 written as a subscript.

✗ EAB

Wrong. EAB would need the middle group to be 1010, but it is 1011.

✓ EBB and EBB16 are equivalent

Correct, and this is why the paper set the question. The subscript states the base, it does not change the value, so a and b name the same number.

Group the bits in fours from the right: 1110 1011 1011, which is E, B, B.

In the `toDigit` routine, what is returned when `n` is 10 or more?

- a. `n` plus the character `a`, minus 10
- b. `n` plus the character `0`
- c. `n` minus 10 only
- d. The character `0` repeated `n` times

✓ `n` plus the character `a`, minus 10

Correct. That maps 10 to `a`, 11 to `b`, and so on.

✗ `n` plus the character `0`

Wrong. That is the branch for `n` less than 10.

✗ `n` minus 10 only

Wrong. Subtracting 10 alone gives a number, not a character.

✗ The character `0` repeated `n` times

Wrong. Nothing in the routine repeats a character.

Digits 0 to 9 come from adding the character `0`. Values from 10 upward become letters, by adding the character `a` and subtracting 10 so that 10 lands on `a`.

The fromDigit routine accepts letters a to z as digits. What is the highest base it therefore allows?

- a. 36
- b. 26
- c. 16
- d. 10

✓ 36

Correct, and the slide comments it in the code.

✗ 26

Wrong. That counts only the letters and forgets the ten digits.

✗ 16

Wrong. Base 16 uses only a to f.

✗ 10

Wrong. That is the digits alone, with no letters.

Ten digits plus twenty six letters is thirty six symbols, and the deck's comment says so: allow up to base 36.

3.85

HARD

Data S11

The deck groups 110101101011101 as 110, 1011, 0101, 1101 and prints the answer 6B5C. Working it yourself, what is the correct hexadecimal?

- a. 6B5D
- b. 6B5C, the slide is right
- c. 6B5B
- d. 6C5D

✓ 6B5D

Correct. 6, B, 5, D. Take the slide's grouping but finish the last digit yourself.

✗ 6B5C, the slide is right

Wrong, and this is the answer as printed. 13 decimal is D, not C, so the final digit is wrong on the slide.

✗ 6B5B

Wrong. B is 11, so 6B5B would need a final group of 1011.

✗ 6C5D

Wrong. The second group 1011 is B, not C.

The last group is 1101, which is 13 decimal, and 13 in hexadecimal is D. The slide's own working line reads 13D=CH, which is where the slip enters, and the printed answer 6B5C carries it through. Everything else on the slide is right: 110 is 6, 1011 is B, 0101 is 5.

3.86

HARD

Data S11

The binary group 1101 converts to the hexadecimal digit C.

Answer: **false**

1101 is 13 decimal, and 13 is hexadecimal D. C is 12, which is 1100. The deck's conversion slide prints this the wrong way round, so trust the arithmetic rather than the slide.

What is hexadecimal A34E in decimal?

- a. 41806
- b. 41802
- c. 40960
- d. 4180

✓ 41806

Correct. 41806. 10 times 4096 is 40960, plus 3 times 256 is 768, plus 4 times 16 is 64, plus 14.

✗ 41802

Wrong. That takes E as 10 rather than 14.

✗ 40960

Wrong. That is the A term alone, with the other three digits dropped.

✗ 4180

Wrong by a factor of ten, a digit lost in transcription.

10 times 4096 is 40960, plus 3 times 256 is 768, plus 4 times 16 is 64, plus 14. That is 40960 plus 768 plus 64 plus 14, giving 41806.

Computing 3fc divided by 2 in hexadecimal gives:

- a. 1fe
- b. 1f6
- c. 7f8
- d. 3f6

✓ 1fe

Correct. 1fe, and you can check it: 1fe is 510, and 3fc is 1020.

✗ 1f6

Wrong. The last group after the shift is 1110, which is e, not 6.

✗ 7f8

Wrong, and that is 3fc shifted the other way, so it is 3fc times 2.

✗ 3f6

Wrong. The leading digit must halve too.

Decode 3fc as 0011 1111 1100, shift the bits right by 1 to get 0001 1111 1110, and encode again to get 1fe.

3.89

HARD

Data S13

To halve a hexadecimal number, decode it into binary, perform the _____, then _____ the digits again.

BLANK 1

- a. shift
- b. complement
- c. carry
- d. division

BLANK 2

- a. encode
- b. decode
- c. invert
- d. group

Answer: **shift, encode**

Decode, shift, encode. Hexadecimal has no shift of its own, so you borrow binary's.

3.90

HARD

Data S43

In 4221 BCD the bit pattern 1111 has weights that sum to _____, which is the decimal digit it represents.

- a. 9
- b. 15
- c. 8
- d. 10

Answer: **9**

4 plus 2 plus 2 plus 1 is 9, which is why the deck gives 1111 as the 4221 code for 9. In 8421 the same pattern would sum to 15, which is not a decimal digit at all, and that is the difference a weighted code makes.

3.91

HARD

Data S43

Match each BCD code to its representation of 9.2.

8421 BCD

1001.0010

4221 BCD

1111.0010

5421 BCD

1100.0010

The digit 2 codes the same in all three, as 0010, because 2 is reachable by the same single weight in each. It is the 9 that separates them: 8 plus 1, then 4 plus 2 plus 2 plus 1, then 5 plus 4.

3.92

HARD

Data S43

What is the 5421 BCD code for 9.2?

- a. 1100.0010
- b. 1001.0010
- c. 1111.0010
- d. 1100.0100

✓ 1100.0010

Correct. 1100 for the 9, 0010 for the 2.

✗ 1001.0010

Wrong. 1001 is the 8421 code for 9.

✗ 1111.0010

Wrong. 1111 is the 4221 code for 9.

✗ 1100.0100

Wrong on the second digit: 0100 would be 4 in 5421, not 2.

In 5421 the weights are 5, 4, 2, 1, so 9 is 5 plus 4, which is 1100. The 2 is 0010 as usual.

Excess-3 is significant for arithmetic because it overcomes a shortcoming of the 8421 BCD code when:

- a. Adding two decimal digits whose sum exceeds 9
- b. Adding two decimal digits whose sum is less than 9
- c. Subtracting a larger digit from a smaller one
- d. Multiplying two decimal digits

✓ Adding two decimal digits whose sum exceeds 9
Correct, and the deck says exceeds 9 in exactly those words.

✗ Adding two decimal digits whose sum is less than 9
Wrong. Sums of 9 or less cause no trouble in 8421.

✗ Subtracting a larger digit from a smaller one
Wrong. Subtraction is not the case the slide names, though self-complementing does help there.

✗ Multiplying two decimal digits
Wrong. Multiplication is not mentioned.

When two 8421 digits sum past 9 the result runs into the six unused bit patterns and has to be corrected. Excess-3 avoids that, which is why it is used in some old computers.

3.94

HARD

Data S46

Excess-3 is self-complementing, which means:

- a. The 1's complement of an Excess-3 number is the Excess-3 code for the 9's complement of the corresponding decimal number
- b. The 1's complement of an Excess-3 number is the Excess-3 code for the same decimal number
- c. The 2's complement of an Excess-3 number is its own negative
- d. The code needs no complement operation at all

✓ The 1's complement of an Excess-3 number is the Excess-3 code for the 9's complement of the corresponding decimal number

Correct, and the deck's worked case is 6 giving 3.

✗ The 1's complement of an Excess-3 number is the Excess-3 code for the same decimal number

Wrong. If it gave the same number back the property would be useless.

✗ The 2's complement of an Excess-3 number is its own negative

Wrong. The property is about the 1's complement, and about the 9's complement in decimal.

✗ The code needs no complement operation at all

Wrong. The complement is exactly what is being taken.

Flip the bits and you get the 9's complement for free. That is what makes Excess-3 useful in some arithmetic operations.

3.95

HARD

Data S46

The 1's complement of an Excess-3 number is the Excess-3 code for the ____ 's complement of the corresponding decimal number.

- a. 9
- b. 10
- c. 1
- d. 2

Answer: **9**

The 9's complement, since the digits run 0 to 9 and the 9's complement of a digit d is 9 minus d .

The Excess-3 code for decimal 6 is 1001. Taking its 1's complement gives 0110. Which decimal digit does 0110 stand for in Excess-3?

- a. 3
- b. 6
- c. 9
- d. 0

✓ 3

Correct. 6 minus 3 is 3, the 9's complement of the original 6.

✗ 6

Wrong. 6 is the number we started from; its code is 1001.

✗ 9

Wrong. 9 in Excess-3 would be 12, which is 1100.

✗ 0

Wrong. 0 in Excess-3 is 3, which is 0011.

0110 is 6, and subtracting the excess of 3 gives 3. And 3 is the 9's complement of 6, since 9 minus 6 is 3, which is exactly the property being demonstrated.

3.97

HARD

Data S46

What is the Excess-3 code for 27?

- a. 01011010
- b. 00100111
- c. 01010111
- d. 10100101

✓ 01011010

Correct. 2 plus 3 is 5, which is 0101, and 7 plus 3 is 10, which is 1010.

✗ 00100111

Wrong. That is plain 8421 BCD for 27.

✗ 01010111

Wrong. That is the Excess-3 code for 24, from the earlier example.

✗ 10100101

Wrong. The two nibbles are the right ones but reversed.

| 2 plus 3 is 5, which is 0101, and 7 plus 3 is 10, which is 1010. Together, 01011010.

3.98

HARD

Data S46

The Excess-3 code for 597 is ____ .

- a. 100011001010
- b. 010110011010
- c. 100011000111
- d. 010110010111

Answer: **100011001010**

| 5 plus 3 is 8 which is 1000, 9 plus 3 is 12 which is 1100, 7 plus 3 is 10 which is 1010. So 100011001010, exactly as the deck prints it.

The Excess-3 code 1000110 decodes to which decimal number?

- a. 13
- b. 70
- c. 46
- d. 16

✓ 13

Correct. 13, as the deck's worked example gives it.

✗ 70

Wrong. That reads the groups as plain BCD without subtracting the excess.

✗ 46

Wrong. That reads the padded groups as BCD, again with no subtraction.

✗ 16

Wrong. Subtracting 3 once from the whole number rather than from each group gives this kind of error.

Grouped from the radix point and padded, 1000110 becomes 0100 0110. Subtracting 0011 from each group gives 0001 0011, which is 13.

3.100**HARD**

Data S47

Subtracting 0011 from each group of the Excess-3 code 1000110 gives the new number _____, whose decimal equivalent is _____.

BLANK 1

- a. 00010011
- b. 01000110
- c. 00010001
- d. 00110011

BLANK 2

- a. 13
- b. 11
- c. 46
- d. 70

Answer: **00010011, 13**

00010011, which reads as BCD 1 then 3, so 13.

3.101**HARD**

Data S59

Which variation of the 8-bit ASCII table is also called ISO Latin-1?

- a. ISO 8859-1
- b. ISO 8859-15
- c. ISO 646
- d. ISO 10646

✓ ISO 8859-1

Correct, and it is the only ISO number this deck gives.

✗ ISO 8859-15

Wrong. Not named in this deck.

✗ ISO 646

Wrong. Not named in this deck.

✗ ISO 10646

Wrong. Not named in this deck.

ISO 8859-1, also called ISO Latin-1. Its codes 129 to 159 hold the Microsoft Windows Latin-1 extended characters.

3.102**HARD**

Data S59

The extended ASCII codes run from _____ to 255, and codes 129 to 159 contain the Microsoft Windows Latin-1 extended characters.

- a. 128
- b. 127
- c. 129
- d. 160

Answer: **128**

128, picking up exactly where the 7-bit range stops. The Windows block starts one further on, at 129.

EBCDIC descended from the code used with punched cards and from:

- a. A 6-bit binary-coded decimal code used with IBM's computer peripherals of the late 1950s and early 1960s
- b. A 7-bit code used with IBM's terminals of the 1970s
- c. Morse code
- d. Unicode

✓ A 6-bit binary-coded decimal code used with IBM's computer peripherals of the late 1950s and early 1960s

Correct, with the width, the company and the period all as the deck gives them.

✗ A 7-bit code used with IBM's terminals of the 1970s

Wrong. 7 bits is ASCII, and the period is wrong.

✗ Morse code

Wrong. Morse is a separate alphanumeric code with no descent relationship here.

✗ Unicode

Wrong. Unicode came long after EBCDIC.

A 6-bit binary-coded decimal code, used with most of IBM's computer peripherals of the late 1950s and early 1960s. That ancestry is why BCD sits inside the name.

3.104**HARD**

Data S61

EBCDIC descended from the code used with _____ cards and the corresponding _____ - bit binary-coded decimal code used with most of IBM's computer peripherals of the late 1950s and early 1960s.

BLANK 1

- a. punched
- b. magnetic
- c. paper tape
- d. optical

BLANK 2

- a. 6
- b. 8
- c. 7
- d. 4

Answer: **punched, 6**

Punched cards, and a 6-bit code. EBCDIC itself is 8-bit; the 6-bit figure is its ancestor.

4 Signed Numbers and Complements

4.1

EASY

The real test 1, question 2

What is the range of values representable by a 10-bit unsigned integer?

- a. -1023 to 1023
- b. 0 to 1023
- c. -512 to 511
- d. 0 to 1024

✗ -1023 to 1023

Wrong. UNSIGNED means no negative values at all, so the range cannot start below zero.

✓ 0 to 1023

Correct. 1024 distinct patterns, counting from 0, so the top value is 1023.

✗ -512 to 511

Wrong. That is the 10-bit TWO'S COMPLEMENT range, offered to catch a reader who skimmed past the word unsigned.

✗ 0 to 1024

Wrong by one. There are 1024 patterns, but they run 0 to 1023, because 0 uses one of them.

An n-bit unsigned range is 0 to 2 to the power n, minus 1. Here 2 to the power 10 is 1024, so the range is 0 to 1023.

In one's complement representation, the additive inverse of a number is obtained by:

- a. Shifting right
- b. Flipping all bits
- c. Adding 1
- d. Shifting left

✗ Shifting right

Wrong. Shifting right divides by 2, it does not change the sign.

✓ Flipping all bits

Correct. Invert every bit and you have the additive inverse.

✗ Adding 1

Wrong on its own, and this is the near miss. Invert AND add 1 gives the TWO'S complement. Adding 1 by itself simply counts up.

✗ Shifting left

Wrong. Shifting left multiplies by 2.

| One's complement negation is a straight bit inversion, nothing more.

An 8-bit unsigned register stores 11111111. What decimal value does it represent?

- a. 127
- b. 255
- c. 254
- d. 128

✗ 127

Wrong. 127 is 01111111, the largest value in SEVEN bits, which is what you get if you wrongly reserve a sign bit.

✓ 255

Correct. $128 + 64 + 32 + 16 + 8 + 4 + 2 + 1 = 255$.

✗ 254

Wrong by one. 254 is 11111110.

✗ 128

Wrong. 128 is 10000000, just the top bit.

Eight ones is 2 to the power 8, minus 1, which is 255. The word UNSIGNED means there is no sign bit to set aside.

How many representations of zero exist in sign-magnitude representation?

- a. 2
- b. Depends on word length
- c. 1
- d. 4

✓ 2

Correct. One sign bit, two settings, both with an all-zero magnitude.

✗ Depends on word length

Wrong. A longer word adds more magnitude bits, but there is still only ever one sign bit, so the count stays at 2.

✗ 1

Wrong. A single zero is the TWO'S complement property.

✗ 4

Wrong. Four would need two sign bits, which no scheme uses.

There are always exactly two, +0 and -0, because the sign bit is free to be either value while the magnitude is zero.

4.5

EASY

The real test 1, question 49

Overflow occurs when:

- a. Result exceeds representable range
- b. Division by zero occurs
- c. Sign changes unexpectedly
- d. Bits are shifted

✓ Result exceeds representable range

Correct, and this is the definition. The word has a fixed width, and the answer did not fit.

✗ Division by zero occurs

Wrong. Division by zero is a separate arithmetic exception with its own trap.

✗ Sign changes unexpectedly

Wrong, but tempting: an unexpected sign flip is how you DETECT two's complement overflow. It is the symptom, not the definition.

✗ Bits are shifted

Wrong. Shifting is a normal operation, though shifting bits out of the top can itself cause a loss of value.

| Overflow is a result too large for the word to hold.

4.6

EASY

Data S15

Select the four commonly known integer representations the deck names.

- a. Unsigned
- b. Sign magnitude
- c. One's complement
- d. Two's complement
- e. Excess-3

✓ Unsigned

Correct, unsigned is first and is the only one that cannot hold a negative value.

✓ Sign magnitude

Correct, sign magnitude is second.

✓ One's complement

Correct, one's complement is third.

✓ Two's complement

Correct, two's complement is fourth and is what modern machines use.

✗ Excess-3

Wrong. Excess-3 codes decimal digits and belongs with BCD.

Four, in that order, and the deck adds that all have been used at various times for various reasons. Excess-3 is a decimal code, not an integer representation.

4.7

EASY

Data S15

The deck says there are _____ commonly known integer representations.

- a. 4
- b. 3
- c. 5
- d. 2

Answer: 4

Four: unsigned, sign magnitude, one's complement, two's complement.

4.8

EASY

Data S16

Virtually all modern computers operate based on two's complement representation.

Answer: **true**

They do, for the two reasons the deck gives: faster hardware and simpler hardware.

4.9

EASY

Data S17

The standard binary encoding, unsigned, gives only _____ values.

- a. positive
- b. negative
- c. even
- d. signed

Answer: **positive**

Positive only, since there is no bit set aside to record a sign. Strictly the range includes 0, which is neither, but positive is the deck's word.

4.10

EASY

Data S17

With 4 bits, what unsigned values can be represented?

- a. 0 to 15
- b. 0 to 16
- c. minus 7 to plus 7
- d. minus 8 to plus 7

✓ 0 to 15

Correct, sixteen values from 0 to 15.

✗ 0 to 16

Wrong by one, and it is the commonest slip in this topic.

✗ minus 7 to plus 7

Wrong. That is the 4-bit sign magnitude range.

✗ minus 8 to plus 7

Wrong. That is the 4-bit two's complement range.

2 to the power 4 minus 1 is 15, so 0 to 15. The other two ranges belong to sign magnitude and two's complement on 4 bits.

4.11

EASY

Data S17

In the 4-bit table, binary 1100 is:

- a. decimal 12, hex c
- b. decimal 12, hex d
- c. decimal 13, hex c
- d. decimal 11, hex b

✓ decimal 12, hex c

Correct. 8 plus 4 is 12, and 12 is hex c.

✗ decimal 12, hex d

Wrong. Hex d is 13, whose pattern is 1101.

✗ decimal 13, hex c

Wrong. Decimal 13 is 1101.

✗ decimal 11, hex b

Wrong. Decimal 11 is 1011.

| 8 plus 4 is 12, and 12 is hex c.

4.12

EASY

Data S18

The deck says any computer not capable of dealing with negative numbers would not be particularly useful.

Answer: **true**

| That is how the negative number section opens, and it is the justification for spending three slides on three competing schemes.

4.13

EASY

Data S18

Select the methods the deck lists for representing negative numbers in binary.

- a. Sign-Magnitude method
- b. One's complement
- c. Two's complement
- d. Unsigned

✓ Sign-Magnitude method

Correct, sign-magnitude is first.

✓ One's complement

Correct, one's complement is second.

✓ Two's complement

Correct, two's complement is third.

✗ Unsigned

Wrong. Unsigned gives only positive values, which is why negatives need a scheme of their own.

Three methods. Unsigned is on the earlier list of four integer representations, but it cannot hold a negative value at all, so it is not one of these three.

4.14

EASY

Data S18

The deck lists _____ methods for representing negative numbers in binary.

- a. 3
- b. 2
- c. 4
- d. 5

Answer: **3**

Three. The list of four earlier included unsigned, which is not a way of writing a negative number.

In sign magnitude, what does a sign bit of 1 mean?

- a. The number is negative
- b. The number is positive
- c. The number is zero
- d. The number has overflowed

✓ The number is negative

Correct. 0 for positive, 1 for negative.

✗ The number is positive

Wrong, and reversed: 0 is positive.

✗ The number is zero

Wrong. Zero has two representations in sign magnitude, 0000 and 1000, so the sign bit alone does not mark it.

✗ The number has overflowed

Wrong. Overflow is not signalled by the sign bit.

0 for positive, 1 for negative. The remaining bits are a magnitude, encoded exactly as an unsigned integer would be.

4.16

EASY

Data S21

In 4-bit sign magnitude, what does 1101 represent?

- a. minus 5
- b. minus 3
- c. 13
- d. minus 13

✓ minus 5

Correct. The sign bit is 1, so it is negative, and the magnitude 101 is 5.

✗ minus 3

Wrong. Minus 3 would be 1011.

✗ 13

Wrong. 13 is the unsigned reading, which ignores the sign bit.

✗ minus 13

Wrong. The sign bit does not count towards the magnitude.

The sign bit is 1, so it is negative, and the magnitude 101 is 5. So minus 5, matching 0101 for plus 5.

4.17

EASY

Data S21

To get the additive inverse of a sign magnitude number, just flip the _____ bit.

- a. sign
- b. last
- c. first
- d. least significant

Answer: **sign**

The sign bit. The deck also calls flipping not, invert, complement or negate, so all four words mean the same operation here.

4.18

EASY

Data S23

The ones' complement form of a negative binary number is:

- a. The bitwise NOT applied to its positive counterpart
- b. The positive counterpart with the sign bit set
- c. One more than the bitwise NOT of the positive counterpart
- d. The positive counterpart shifted right by one

✓ The bitwise NOT applied to its positive counterpart

Correct. Negation is a bitwise complement of the positive representation.

✗ The positive counterpart with the sign bit set

Wrong. That is sign magnitude.

✗ One more than the bitwise NOT of the positive counterpart

Wrong. That extra 1 makes it the two's complement.

✗ The positive counterpart shifted right by one

Wrong. Shifting right divides by 2.

| Bitwise NOT, nothing more. Adding one afterwards is what makes it a two's complement instead.

4.19

EASY

Data S23

In one's complement, negation is done by taking a bitwise _____ of the positive representation.

- a. complement
- b. shift
- c. sum
- d. carry

Answer: **complement**

| Complement, which the deck also writes as bitwise NOT. Flip every bit, sign bit included.

4.20

EASY

Data S29

The two's complement of a value is its one's complement plus one.

Answer: **true**

| That is the definition, and it is also the slower of the two methods the deck gives; the quick method copies digits up to the first 1 and inverts the rest.

4.21

EASY

Data S34

Overflow occurs when doing _____ operations.

- a. arithmetic
- b. logic
- c. load and store
- d. branch

Answer: **arithmetic**

Arithmetic. A logic operation works bit by bit and produces no carry, so it cannot overflow.

4.22

EASY

Rev S4

Which representation does the revision class attach the phrase end-around carry to?

- a. 1's complement
- b. 2's complement
- c. Sign-magnitude
- d. Unsigned

✓ 1's complement

Correct. One's complement.

✗ 2's complement

Wrong, and two's complement exists partly to remove the need for it.

✗ Sign-magnitude

Wrong. Sign magnitude needs separate add and subtract operations instead.

✗ Unsigned

Wrong. Unsigned arithmetic has no complement step at all.

One's complement. The revision slide writes it as a parenthetical reminder, because it is the step that scheme cannot do without.

The spacecraft telemetry system uses a processor of what width?

- a. 16-bit
- b. 8-bit
- c. 32-bit
- d. 64-bit

✓ 16-bit

Correct. 16-bit, using 2's complement for signed integers.

✗ 8-bit

Wrong. 8 bits could not hold minus 1023 at all.

✗ 32-bit

Wrong. The question specifies 16 bits, and all the workings use 16-bit patterns.

✗ 64-bit

Wrong. 64 bits appears in the Ariane 5 story, not here.

16-bit, using 2's complement for signed integers. The width is what fixes the range at minus 32768 to 32767.

Select what the telemetry question asks for.

- a. The range of values representable in a 16-bit two's complement system
- b. Whether the given numbers fall within that range
- c. Each value in binary and hexadecimal
- d. A demonstration of minus 182 plus 487
- e. Each value in IEEE single precision

✓ The range of values representable in a 16-bit two's complement system

Correct, part i asks for the range.

✓ Whether the given numbers fall within that range

Correct, part i also asks whether the values fall inside it.

✓ Each value in binary and hexadecimal

Correct, part ii asks for binary and hexadecimal.

✓ A demonstration of minus 182 plus 487

Correct, part iii asks for the addition.

✗ Each value in IEEE single precision

Wrong. This question is about integers in two's complement.

Four parts. IEEE single precision belongs to the floating point topic and is not asked for here.

Which statement about one's complement is TRUE?

- a. It has one representation of zero
- b. Overflow never occurs
- c. It has positive and negative zero
- d. It has no negative numbers

X It has one representation of zero

Wrong. A single zero is the TWO'S complement property, and it is one of the reasons two's complement won.

X Overflow never occurs

Wrong. Overflow depends on the width of the word, not on the negation scheme, so it can occur in any of them.

✓ It has positive and negative zero

Correct. 00000000 and 11111111 are both zero, which is the scheme's central weakness.

X It has no negative numbers

Wrong. Representing negatives is the entire purpose of a complement scheme.

00000000 is +0 and 11111111 is -0, so one's complement carries two zeros, just as sign-magnitude does. Only two's complement has a single zero.

4.26

MEDIUM

The real test 1, question 21

For an 8-bit sign-magnitude system, the decimal value -0 is represented by:

- a. 11111111
- b. 00000000
- c. 01111111
- d. 10000000

X 11111111

Wrong. 11111111 is negative zero in ONE'S complement, and in sign-magnitude it reads as -127.

X 00000000

Wrong. That is POSITIVE zero, sign bit 0 with zero magnitude.

X 01111111

Wrong. Sign bit 0 with magnitude 1111111 is +127.

✓ 10000000

Correct. Sign bit 1, magnitude 0000000, so minus nothing.

Sign-magnitude splits the word into a sign bit and a magnitude. Negative zero is sign bit 1 with a magnitude of zero.

Which representation requires an end-around carry during addition?

- a. Excess-3
- b. Sign Magnitude
- c. One's Complement
- d. Two's Complement

✗ Excess-3

Wrong. Excess-3 is a digit code, not a signed number scheme, so end-around carry does not arise.

✗ Sign Magnitude

Wrong. Sign-magnitude addition compares magnitudes and decides the sign. There is no carry to wrap.

✓ One's Complement

Correct. The carry out is fed back into the low end, and the fix exists because of the two zeros.

✗ Two's Complement

Wrong. Two's complement simply DISCARDS the carry out, which is one of the reasons its hardware is simpler.

In one's complement, a carry out of the most significant bit must be added back into the least significant bit.

In sign-magnitude representation, the 8-bit value 10101010 represents:

- a. +170
- b. -170
- c. +42
- d. -42

X +170

Wrong twice. It reads all eight bits as the magnitude, giving 170, and then ignores the sign bit.

X -170

Wrong once. The sign is right, but the magnitude wrongly includes the sign bit, giving 170 instead of 42.

X +42

Wrong once. The magnitude is right, but the leading 1 was ignored, so the sign is lost.

✓ -42

Correct. Negative, magnitude 42.

| Split off the sign bit: 1 means negative, and the remaining seven bits 0101010 are $32 + 8 + 2 = 42$.

Why is two's complement widely used in modern computers?

- a. Easier decimal conversion
- b. Eliminates overflow
- c. Greater storage efficiency
- d. Faster and simpler hardware implementation

✗ Easier decimal conversion

Wrong. Converting a two's complement number to decimal by hand is arguably the fiddliest of the three schemes.

✗ Eliminates overflow

Wrong, and false on its face. Overflow is a consequence of a fixed word width and no scheme removes it.

✗ Greater storage efficiency

Wrong. All three schemes use exactly the same number of bits for the same width.

✓ Faster and simpler hardware implementation

Correct. One adder for both operations, one zero, no carry to wrap around.

One adder handles both addition and subtraction, there is a single zero, and no end-around carry is needed.

The major hardware disadvantage of sign-magnitude representation is:

- a. Inability to represent positive numbers
- b. Slow and complex arithmetic circuitry
- c. Large memory usage
- d. Limited range

X Inability to represent positive numbers

Wrong, and plainly false. Sign bit 0 gives every positive value.

✓ Slow and complex arithmetic circuitry

Correct. Signs must be compared, magnitudes ordered and the result's sign derived, all in hardware.

X Large memory usage

Wrong. An 8-bit sign-magnitude value occupies 8 bits, the same as any other scheme.

X Limited range

Wrong. Its range, -127 to +127, is essentially the same as two's complement, losing only the extra negative value.

The circuit must inspect both sign bits, decide whether to add or subtract, compare magnitudes and work out the sign of the result. Two's complement just adds.

Virtually all modern computers operate based on 2's complement representation. Why?

- a. The hardware is faster
- b. The hardware is simpler, which makes it faster
- c. It can represent larger numbers than any other scheme
- d. It is easier for programmers to read

✓ The hardware is faster

Correct, reason 1.

✓ The hardware is simpler, which makes it faster

Correct, reason 2, and the deck adds the bracket which makes it faster.

✗ It can represent larger numbers than any other scheme

Wrong. For n bits, two's complement holds one more value than one's complement or sign magnitude, but capacity is not the reason the deck gives.

✗ It is easier for programmers to read

Wrong. Readability is never offered as a reason; sign magnitude is the easiest of the three to read.

Two reasons, and the deck numbers them: hardware is faster, hardware is simpler. Notice that the second is given as the cause of the first.

4.32

MEDIUM

Data S17

For n bits, the unsigned range is:

- a. 0 to 2 to the power n , minus 1
- b. 0 to 2 to the power n
- c. minus 2 to the power n minus 1, to 2 to the power n minus 1, minus 1
- d. 1 to 2 to the power n

✓ 0 to 2 to the power n , minus 1

Correct. 0 up to 2 to the power n minus 1.

✗ 0 to 2 to the power n

Wrong by one. 2 to the power n is the number of patterns, not the largest value.

✗ minus 2 to the power n minus 1, to 2 to the power n minus 1, minus 1

Wrong. That is the two's complement range.

✗ 1 to 2 to the power n

Wrong. Unsigned starts at 0, not 1.

0 up to 2 to the power n minus 1. There are 2 to the power n patterns and the count starts at 0, so the largest is one less than that.

4.33

MEDIUM

Data S17

With n equal to 4, 2 to the power n minus 1 is ____ .

- a. 15
- b. 16
- c. 8
- d. 7

Answer: **15**

15. Read the expression as 2 to the power 4, then subtract 1, giving 16 minus 1.

4.34

MEDIUM

Data S17

From the deck's 4-bit table, match each binary pattern to its hexadecimal digit.

1010	a
1011	b
1100	c
1101	d
1110	e
1111	f

The six letter digits. This is the table that settles the slip on the binary to hexadecimal slide, where 1101 was labelled C: here it is plainly d.

4.35

MEDIUM

Data S19

Sign magnitude uses _____ bit of the integer for the sign, and the rest of the integer is a magnitude using the same encoding as _____ integers.

BLANK 1

- a. 1
- b. 2
- c. 4
- d. 8

BLANK 2

- a. unsigned
- b. two's complement
- c. one's complement
- d. floating point

Answer: **1, unsigned**

One bit for the sign, and the magnitude is encoded just as an unsigned integer. That is why 0101 is 5 and 1101 is minus 5: the low three bits read the same either way.

4.36

MEDIUM

Data S19

How does the deck describe the hardware that does arithmetic on sign magnitude integers?

- a. Not fast, and more complex than the hardware for 1's and 2's complement
- b. Fast, and simpler than the hardware for 1's and 2's complement
- c. The same speed as 2's complement hardware
- d. Impossible to build without a floating point unit

✓ Not fast, and more complex than the hardware for 1's and 2's complement

Correct, both halves are on the slide.

✗ Fast, and simpler than the hardware for 1's and 2's complement

Wrong on both counts.

✗ The same speed as 2's complement hardware

Wrong. The deck ranks it as worse than both complement schemes.

✗ Impossible to build without a floating point unit

Wrong. Floating point is a separate topic and not required here.

Not fast, and more complex than the alternatives. It is the same argument that made two's complement the modern default.

4.37

MEDIUM

Data S19

Sign magnitude arithmetic hardware is simpler than one's complement and two's complement hardware.

Answer: **false**

It is more complex, and not fast. That is the reason it lost, despite being the easiest of the three for a person to read.

4.38

MEDIUM

Data S20

The worked line of the class exercise gives minus 38 in an 8-bit sign-magnitude register as:

- a. 10100110
- b. 00100110
- c. 11011010
- d. 11011001

✓ 10100110

Correct. 38 is 00100110, and flipping the sign bit gives 10100110.

✗ 00100110

Wrong. That is plus 38; the sign bit has not been set.

✗ 11011010

Wrong. That is the two's complement of 38, not its sign magnitude.

✗ 11011001

Wrong. That is the two's complement of 39, from the later example.

38 is 00100110, and flipping the sign bit gives 10100110. The method the slide gives is exactly that: find the positive value first, then flip the last bit, meaning the sign bit.

4.39

MEDIUM

Data S21

With 4 bits, sign magnitude runs from minus 7 to plus ____ .

- a. 7
- b. 8
- c. 15
- d. 6

Answer: **7**

Plus 7. Working it through: minus 2 to the power 3, plus 1, is minus 8 plus 1, and 2 to the power 3 minus 1 is 8 minus 1.

4.40

MEDIUM

Data S22

In 4-bit sign magnitude, which two patterns both represent zero?

- a. 0000 and 1000
- b. 0000 and 1111
- c. 0000 and 0001
- d. 1000 and 1111

✓ 0000 and 1000

Correct. 0000 is zero and 1000 is zero with the sign bit set.

✗ 0000 and 1111

Wrong. 0000 and 1111 are the two zeros of ONE'S complement, not sign magnitude.

✗ 0000 and 0001

Wrong. 0001 is 1.

✗ 1000 and 1111

Wrong. 1111 in sign magnitude is minus 7.

0000 is zero and 1000 is zero with the sign bit set. Two patterns for one value, which the deck calls a problem for hardware.

4.41

MEDIUM

Data S22

Having two representations for zero is a problem because the computer must make all calculations come out correctly and the same whichever representation is used.

Answer: **true**

That is the deck's reason. Every comparison against zero has to check both patterns, which costs hardware.

4.42

MEDIUM

Data S22

Which operations are easy using sign-magnitude representation?

- a. Sign reversal and absolute value
- b. Addition and subtraction
- c. Multiplication and division
- d. Comparison against zero

✓ Sign reversal and absolute value

Correct, and both are named on the slide.

✗ Addition and subtraction

Wrong, and it is the opposite: different operations must be defined for addition and subtraction.

✗ Multiplication and division

Wrong. Multiplication and division are not discussed on this slide.

✗ Comparison against zero

Wrong. Comparison against zero is made harder by there being two zeros.

Sign reversal is one bit flip and absolute value is one bit clear. It is addition and subtraction that sign magnitude makes awkward.

4.43

MEDIUM

Data S22

Sign reversal and _____ value operations are easy using sign-magnitude representation.

- a. absolute
- b. relative
- c. maximum
- d. average

Answer: **absolute**

Absolute value, which is just clearing the sign bit and leaving the magnitude alone.

4.44

MEDIUM

Data S22

In sign-magnitude, adding the negative of a number gives the same result as subtracting it.

Answer: **false**

It does not, and that is precisely the deck's complaint. The identity holds in two's complement, which is one reason that scheme won.

4.45

MEDIUM

Data S23

In 8-bit one's complement, the two representations of zero are:

- a. 00000000 for plus 0 and 11111111 for minus 0
- b. 00000000 for plus 0 and 10000000 for minus 0
- c. 00000000 only
- d. 11111111 only

✓ 00000000 for plus 0 and 11111111 for minus 0

Correct. All zeros and all ones.

✗ 00000000 for plus 0 and 10000000 for minus 0

Wrong. 10000000 is the sign magnitude minus zero, not the one's complement one.

✗ 00000000 only

Wrong. One's complement has two zeros, like sign magnitude.

✗ 11111111 only

Wrong. 00000000 is a zero too.

All zeros and all ones. Take the bitwise NOT of 00000000 and you get 11111111, which is therefore also zero.

4.46

MEDIUM

Data S23

In one's complement, positive integers use the same representation as unsigned.

Answer: **true**

They do, and the same is true in sign magnitude and two's complement. Only the negatives differ between the three schemes.

4.47

MEDIUM

Data S24

The ones' complement form of 00101011, which is 43, is:

- a. 11010100
- b. 11010101
- c. 10101011
- d. 10101100

✓ 11010100

Correct. Flip every bit of 00101011 and you get 11010100, which is minus 43.

✗ 11010101

Wrong. That is the two's complement of 43, one more than the one's complement.

✗ 10101011

Wrong. That is sign magnitude minus 43, with only the sign bit changed.

✗ 10101100

Wrong. That is neither; check each bit against the original.

Flip every bit of 00101011 and you get 11010100, which is minus 43.

4.48

MEDIUM

Data S24

The bit pattern 00101011 is decimal ____ .

- a. 43
- b. 45
- c. 41
- d. 53

Answer: **43**

32 plus 8 plus 2 plus 1 is 43.

4.49

MEDIUM

Data S24

To read the one's complement value 11100, note that the leading bit makes it _____, then take the additive inverse, _____, which is plus 3.

BLANK 1

- a. negative
- b. positive
- c. zero
- d. overflowed

BLANK 2

- a. 00011
- b. 00100
- c. 11101
- d. 10011

Answer: **negative, 00011**

Negative, and the inverse is 00011. Flip, read, then put the minus sign back.

What is the conventional 8-bit range of one's complement?

- a. minus 127 to plus 127
- b. minus 128 to plus 127
- c. 0 to 255
- d. minus 255 to plus 255

✓ minus 127 to plus 127

Correct. Minus 127 to plus 127, plus and minus zero.

✗ minus 128 to plus 127

Wrong. That is the two's complement range, which reaches one further down because it has only one zero.

✗ 0 to 255

Wrong. That is the 8-bit unsigned range.

✗ minus 255 to plus 255

Wrong. 8 bits cannot reach 255 in either direction once a sign is carried.

Minus 127 to plus 127, plus and minus zero. The range is symmetric because two of the 256 patterns are spent on zero.

4.51

MEDIUM

Data S25

To add two numbers in one's complement, after the conventional binary addition it is necessary to:

- a. Do an end-around carry, adding any resulting carry back into the sum
- b. Discard any carry out of the top bit
- c. Take the complement of the result
- d. Set the sign bit to match the larger operand

✓ Do an end-around carry, adding any resulting carry back into the sum

Correct, and the name is worth knowing exactly.

✗ Discard any carry out of the top bit

Wrong. Discarding it is what leaves the answer one too low, as the worked example shows.

✗ Take the complement of the result

Wrong. No complement is taken after adding.

✗ Set the sign bit to match the larger operand

Wrong. Nothing in one's complement addition inspects operand size.

The end-around carry: the carry that falls off the top is added back at the bottom. Two's complement was invented partly to remove this step.

4.52

MEDIUM

Data S25

In one's complement addition it is necessary to do an ____ carry.

- a. end-around
- b. look-ahead
- c. ripple
- d. borrow

Answer: **end-around**

End-around. The revision deck names it too, in its summary of the signed number schemes.

4.53

MEDIUM

Data S25

Which addition does the deck use to demonstrate the end-around carry?

- a. minus 1, which is 11111110, plus plus 2, which is 00000010
- b. minus 2, which is 11111101, plus plus 1, which is 00000001
- c. plus 1 plus plus 2
- d. minus 1 plus minus 2

✓ minus 1, which is 11111110, plus plus 2, which is 00000010

Correct, and note that minus 1 in one's complement is 11111110, not 11111111, which is minus zero.

✗ minus 2, which is 11111101, plus plus 1, which is 00000001

Wrong. Those are not the operands the deck uses.

✗ plus 1 plus plus 2

Wrong. Two positives would never produce the carry that makes the point.

✗ minus 1 plus minus 2

Wrong. The example mixes a negative and a positive.

Minus 1 as 11111110 added to plus 2 as 00000010. The true answer is plus 1, and only the end-around carry produces it.

4.54

MEDIUM

Data S26

In the worked one's complement example, the binary addition alone gives the correct answer without the end-around carry.

Answer: **false**

The addition alone gives 00000000, which is wrong. Only when the carry is added back does the correct result 00000001 appear.

4.55

MEDIUM

Data S28

The ones' complement representation of zero used in that subtraction is a long sequence of ____ .

- a. ones
- b. zeros
- c. alternating bits
- d. sign bits

Answer: **ones**

Ones, that is minus zero. The scheme is named after the pattern you subtract from.

4.56

MEDIUM

Data S28

To convert a negative sign-magnitude number to one's complement you must complement the sign bit as well as the magnitude.

Answer: **false**

Only the magnitude is complemented. The sign bit already says the number is negative, and flipping it would undo that.

4.57

MEDIUM

Data S29

Which problems does two's complement solve?

- a. Multiple representations of 0
- b. The need for the end-around carry
- c. Overflow on addition
- d. The need for a sign bit

✓ Multiple representations of 0

Correct. Two's complement has exactly one zero.

✓ The need for the end-around carry

Correct. No carry has to be fed back round.

✗ Overflow on addition

Wrong. Overflow happens in every fixed-width scheme, as the 3-bit example shows.

✗ The need for a sign bit

Wrong. The most significant bit still tells you the number is negative.

Two problems, both inherited from one's complement. Overflow remains possible in any fixed width, and the top bit still carries the sign.

4.58

MEDIUM

Data S29

Two's complement solves the problems of multiple representations of 0 and the need for the _____ carry.

- a. end-around
- b. ripple
- c. look-ahead
- d. borrow

Answer: **end-around**

End-around carry. Those two fixes are the whole case for two's complement, and they are what the deck means by simpler and faster hardware.

In 2's complement, negative numbers are represented by the bit pattern which is:

- a. One greater, in an unsigned sense, than the ones' complement of the positive value
- b. One less than the ones' complement of the positive value
- c. The same as the ones' complement of the positive value
- d. The positive value with its sign bit set

✓ One greater, in an unsigned sense, than the ones' complement of the positive value

Correct, and in an unsigned sense means treat the pattern as a plain binary number when adding the 1.

✗ One less than the ones' complement of the positive value

Wrong direction.

✗ The same as the ones' complement of the positive value

Wrong. That would be the one's complement itself.

✗ The positive value with its sign bit set

Wrong. That is sign magnitude.

One greater than the one's complement. That single extra 1 removes the second zero and the end-around carry together.

4.60

MEDIUM

Data S31

Taking the additive inverse of 110011: the 1's complement is _____, and adding 1 gives _____, which is plus 13.

BLANK 1

- a. 001100
- b. 001101
- c. 110100
- d. 001110

BLANK 2

- a. 001101
- b. 001100
- c. 001110
- d. 010011

Answer: **001100, 001101**

001100 then 001101. The same two steps in reverse recover the original from plus 13.

In the mid-lecture quiz, which unsigned operation are students asked to perform in 2's complement using 6 bits?

- a. 47 plus 35
- b. 47 minus 35
- c. 35 plus 47, then negated
- d. 47 plus 35 in 8 bits

✓ 47 plus 35

Correct, and the second half of the question, what value will be stored, is the interesting part.

✗ 47 minus 35

Wrong. The quiz asks for an addition.

✗ 35 plus 47, then negated

Wrong. No negation is asked for.

✗ 47 plus 35 in 8 bits

Wrong. The quiz specifies 6 bits, which is what makes the answer overflow.

47 plus 35, in 2's complement using 6 bits, with a follow-up asking what value will be stored in the register. The sum is 82, which does not fit in 6 bits, so the question is really about overflow.

Overflow is said to occur when:

- a. A value cannot be represented in the number of bits allowed or specified
- b. A value is too small to be represented in the exponent field
- c. Two values have the same representation
- d. A carry is produced by an addition

✓ A value cannot be represented in the number of bits allowed or specified

Correct. A value that will not fit in the bits allowed.

✗ A value is too small to be represented in the exponent field

Wrong, and that is underflow, defined on the floating point slide.

✗ Two values have the same representation

Wrong. That is the two zeros problem.

✗ A carry is produced by an addition

Wrong. A carry can appear without overflow; it is whether the result fits that matters.

A value that will not fit in the bits allowed. The deck adds that overflow occurs when doing arithmetic operations.

4.63

MEDIUM

Data S34

For unsigned in 3 bits, the value 8 would require at least _____ bits, written _____ .

BLANK 1

- a. 4
- b. 3
- c. 5
- d. 6

BLANK 2

- a. 1000
- b. 0100
- c. 01000
- d. 111

Answer: **4, 1000**

Four bits, as 1000. Compare it with the sign magnitude case on the same slide, where 8 needs five bits because one of them is spent on the sign.

4.64

MEDIUM

Data S35

In 3-bit unsigned, adding 011 and 110 gives 1001. Why is this an overflow?

- a. Because 9 would require 4 bits, and only 3 are available
- b. Because the sign bit changed
- c. Because the two operands had different signs
- d. Because the result is negative

✓ Because 9 would require 4 bits, and only 3 are available

Correct. 3 plus 6 is 9, and 9 needs four bits.

✗ Because the sign bit changed

Wrong. Unsigned numbers have no sign bit.

✗ Because the two operands had different signs

Wrong. Both operands are positive, since the representation is unsigned.

✗ Because the result is negative

Wrong. An unsigned result cannot be negative.

3 plus 6 is 9, and 9 needs four bits. In three bits the answer will not fit, which is the definition of overflow.

4.65

MEDIUM

Data S35

In the 3-bit overflow example, 011 is 3 and 110 is _____, and their sum is _____.

BLANK 1

- a. 6
- b. 3
- c. 7
- d. 5

BLANK 2

- a. 9
- b. 8
- c. 10
- d. 7

Answer: **6, 9**

6, and the sum is 9. Written in binary the sum is 1001, four digits where only three were available.

Select the tasks the homework slide sets.

- a. Write minus 89 in sign-magnitude, 1's and 2's complement, using an 8-bit register
- b. Give the range of a 16-bit register using the sign-magnitude method
- c. Give the value of 10111011 in sign-magnitude, 1's and 2's complement
- d. Represent 23.5 in IEEE Single Precision, in hexadecimal
- e. Convert 597 to Excess-3

✓ Write minus 89 in sign-magnitude, 1's and 2's complement, using an 8-bit register

Correct, and it is the first task.

✓ Give the range of a 16-bit register using the sign-magnitude method

Correct, the 16-bit sign-magnitude range is asked for as from and to.

✓ Give the value of 10111011 in sign-magnitude, 1's and 2's complement

Correct, one bit pattern read three ways.

✓ Represent 23.5 in IEEE Single Precision, in hexadecimal

Correct, and it crosses into the floating point topic.

✗ Convert 597 to Excess-3

Wrong. 597 in Excess-3 is worked through on the Excess-3 slide, not set as homework.

Four tasks. The Excess-3 conversion of 597 is a worked example from the codes section, not part of the homework.

4.67

MEDIUM

Rev S4

In the revision class grouping of data representation, which items are listed under signed numbers?

- a. Sign-magnitude representation
- b. 1's complement representation, with end-around carry
- c. 2's complement representation
- d. Floating point numbers
- e. Graphics and objects

✓ Sign-magnitude representation

Correct, sign-magnitude heads the signed list.

✓ 1's complement representation, with end-around carry

Correct, and the revision slide attaches end-around carry to it as a reminder.

✓ 2's complement representation

Correct, 2's complement closes the signed list.

✗ Floating point numbers

Wrong. Floating point is under other representations.

✗ Graphics and objects

Wrong. Graphics and objects is the second item under other representations.

Three under signed numbers. Floating point and graphics are grouped separately, as other representations.

4.68

MEDIUM

Rev S5

In the spacecraft telemetry question, match each measurement to its value.

Temperature

minus 182

Pressure

487

Altitude

minus 1023

Three values on a 16-bit processor using 2's complement. Two are negative, which is what makes the question worth setting.

4.69

MEDIUM

Rev S6

What is the range of a 16-bit two's complement system?

- a. minus 32768 to 32767
- b. minus 32767 to 32767
- c. 0 to 65535
- d. minus 65536 to 65535

✓ minus 32768 to 32767

Correct, and the asymmetry is the giveaway that it is two's complement.

✗ minus 32767 to 32767

Wrong. A symmetric range points to sign magnitude or one's complement.

✗ 0 to 65535

Wrong. That is the 16-bit unsigned range.

✗ minus 65536 to 65535

Wrong on both ends.

Minus 2 to the power 15 up to 2 to the power 15 minus 1, that is minus 32768 to 32767. All three telemetry values fall within it.

4.70

MEDIUM

Rev S6

For a 16-bit two's complement system the range is minus 2 to the power _____ up to 2 to the power 15 minus 1.

- a. 15
- b. 16
- c. 14
- d. 8

Answer: **15**

15, that is one less than the width, because the top bit carries the sign.

4.71

MEDIUM

Rev S7

Pressure 487 in 16-bit binary is 0000000111100111. In hexadecimal that is:

- a. 01E7
- b. 01F7
- c. FE19
- d. 0187

✓ 01E7

Correct. Group the bits in fours: 0000, 0001, 1110, 0111, giving 0, 1, E, 7.

✗ 01F7

Wrong. The third group is 1110, which is E, not F.

✗ FE19

Wrong. That is roughly the complement, and 487 is positive.

✗ 0187

Wrong. Check the third group: 1110 is E.

Group the bits in fours: 0000, 0001, 1110, 0111, giving 0, 1, E, 7. Being positive, it needs no complement at all.

4.72

MEDIUM

Rev S7

Pressure 487 in 16-bit binary is ____ .

- a. 0000000111100111
- b. 0000000111100110
- c. 0000001111100111
- d. 0000000011100111

Answer: **0000000111100111**

0000000111100111. Check it: 256 plus 128 plus 64 plus 32 plus 4 plus 2 plus 1 is 487.

4.73

HARD

The real test 1, question 3

The 1's complement representation 11100011 corresponds to:

- a. -29
- b. -28
- c. -31
- d. -30

✗ -29

Wrong, and the trap. -29 is what this pattern means in TWO'S complement, where you invert and add 1.

✓ -28

Correct. Invert only, giving 28, and apply the negative sign for -28.

✗ -31

Wrong. 31 would be 00011111 after inversion, so the original would have been 11100000.

✗ -30

Wrong. 30 would be 00011110 after inversion, so the original would have been 11100001.

The leading bit is 1, so the value is negative. Invert every bit to recover the magnitude: 11100011 becomes 00011100, which is $16 + 8 + 4 = 28$.

4.74

HARD

The real test 1, question 32

An 8-bit register contains 10000000 in two's complement. What decimal value does it represent?

- a. -127
- b. +128
- c. -128
- d. 0

✗ -127

Wrong. -127 is 10000001. Invert and add 1 to check: 10000001 gives 01111111, which is 127.

✗ +128

Wrong, and impossible. +128 does not fit in 8-bit two's complement, where the largest positive is +127.

✓ -128

Correct. The one negative value with no positive partner.

✗ 0

Wrong. Zero is 00000000, and two's complement has only that one zero. Reading this pattern as -0 is the sign-magnitude habit.

Two's complement is asymmetric: 8 bits run from -128 to +127, and 10000000 is that most negative value. Weight the sign bit negatively and you get $-128 + 0 = -128$.

4.75

HARD

The real test 1, question 47

The two's complement of 00101100 is:

- a. 10110100
- b. 11110011
- c. 11010011
- d. 11010100

✗ 10110100

Wrong. This matches neither step. Inverting gives 11010011, and adding 1 gives 11010100.

✗ 11110011

Wrong. 11110011 is not the inversion of 00101100. Compare them bit by bit.

✗ 11010011

Wrong, and the classic near miss. 11010011 is the ONE'S complement, the result of stopping before you add 1.

✓ 11010100

Correct. Inverted, then incremented.

Two steps, in order. Invert 00101100 to get 11010011, then add 1 to get 11010100. Check it: the original is 44, and the answer should read as -44.

The sign magnitude range for n bits is:

- a. minus 2 to the power n minus 1, plus 1, up to 2 to the power n minus 1, minus 1
- b. minus 2 to the power n minus 1, up to 2 to the power n minus 1, minus 1
- c. 0 up to 2 to the power n , minus 1
- d. minus 2 to the power n , up to 2 to the power n

✓ minus 2 to the power n minus 1, plus 1, up to 2 to the power n minus 1, minus 1
Correct, and on 4 bits it gives minus 7 to plus 7.

✗ minus 2 to the power n minus 1, up to 2 to the power n minus 1, minus 1
Wrong. That is the two's complement range, which reaches one further down.

✗ 0 up to 2 to the power n , minus 1
Wrong. That is the unsigned range.

✗ minus 2 to the power n , up to 2 to the power n
Wrong on both ends.

The plus 1 on the negative end is what makes the range symmetric, and it is there because two of the patterns are spent on zero.

Why must different operations be defined for addition and subtraction with sign-magnitude numbers?

- a. Because adding the negative of a number is not the same as subtraction in sign-magnitude
- b. Because the sign bit is not part of the magnitude
- c. Because there is only one representation of zero
- d. Because the range is not symmetric

✓ Because adding the negative of a number is not the same as subtraction in sign-magnitude

Correct, and it is the deck's stated reason.

✗ Because the sign bit is not part of the magnitude

Wrong. That is true, but it is not the reason given for needing two operations.

✗ Because there is only one representation of zero

Wrong, and factually reversed: sign magnitude has two representations of zero.

✗ Because the range is not symmetric

Wrong. The sign magnitude range is symmetric, minus 7 to plus 7 on 4 bits.

In two's complement, subtracting is adding the negative, so one adder does both. In sign magnitude that identity fails, so two separate operations are needed.

4.78

HARD

Data S24

What number in 1's complement is 11100?

- a. minus 3
- b. minus 4
- c. 28
- d. minus 28

✓ minus 3

Correct. The leading bit is 1, so it is negative.

✗ minus 4

Wrong. Minus 4 in one's complement on 5 bits would be 11011.

✗ 28

Wrong. 28 is the unsigned reading of the pattern.

✗ minus 28

Wrong. The magnitude comes from the complement, not from reading the pattern directly.

The leading bit is 1, so it is negative. Take the additive inverse by flipping every bit: 00011, which is plus 3 by sight. So 11100 is minus 3.

4.79

HARD

Data S24

The range of signed numbers using ones' complement is minus 2^{N-1} , minus 1, up to 2^{N-1} , minus 1, and ____ zero.

- a. plus and minus
- b. positive only
- c. negative only
- d. unsigned

Answer: **plus and minus**

Plus and minus zero, which the deck writes as the plus or minus sign in front of 0. Those two patterns are what cost the range its extra negative value.

4.80

HARD

Data S25

In the end-around carry example, minus 1 is written as _____ in 8-bit one's complement.

- a. 11111110
- b. 11111111
- c. 10000001
- d. 00000001

Answer: **11111110**

11111110, which is the bitwise NOT of 00000001. The all-ones pattern 11111111 is minus zero, which is a distinct value in this scheme.

4.81

HARD

Data S26

In the worked example the binary addition alone gives 00000000. What appears once the carry is added back in?

- a. 00000001
- b. 11111111
- c. 00000010
- d. 10000000

✓ 00000001

Correct. 00000001, which is plus 1, and that is the correct answer to minus 1 plus 2.

✗ 11111111

Wrong. That is minus zero, not the corrected sum.

✗ 00000010

Wrong. That would be plus 2, the second operand unchanged.

✗ 10000000

Wrong. That pattern has no role in this example.

00000001, which is plus 1, and that is the correct answer to minus 1 plus 2. Without the end-around carry the machine would report zero.

4.82

HARD

Data S26

Select the older computers the deck names as using the one's complement representation.

- a. PDP-1
- b. CDC 160 series
- c. UNIVAC 1100/2200 series
- d. IBM 801

✓ PDP-1

Correct, the PDP-1 is named first.

✓ CDC 160 series

Correct, the CDC 160 series is second.

✓ UNIVAC 1100/2200 series

Correct, the UNIVAC 1100/2200 series is third.

✗ IBM 801

Wrong. The IBM 801 is one of the first RISC projects, from the RISC deck.

Three machines, and the deck adds among many others. The IBM 801 belongs to the RISC story, not to this one.

4.83

HARD

Data S26

Alongside the PDP-1 and the CDC 160 series, the deck names the _____ 1100/2200 series as a one's complement machine.

- a. UNIVAC
- b. IBM
- c. DEC
- d. CDC

Answer: **UNIVAC**

UNIVAC. All three names are worth holding, because a question asking which machine used one's complement has no other handle.

Why is the system referred to as ones' complement?

- a. Because the negation of x can also be formed by subtracting x from the ones' complement representation of zero, a long sequence of ones
- b. Because every bit is set to one
- c. Because one is added to the complement
- d. Because it can represent only one zero

✓ Because the negation of x can also be formed by subtracting x from the ones' complement representation of zero, a long sequence of ones

Correct, and it is a second, equivalent way to compute the same thing as the bitwise NOT.

✗ Because every bit is set to one

Wrong. Only minus zero has every bit set.

✗ Because one is added to the complement

Wrong. Adding one to the complement gives the two's complement.

✗ Because it can represent only one zero

Wrong, and reversed: one's complement has two zeros.

Subtract from the all-ones pattern, which is minus zero in this scheme, and you get the negation.
The name records the method.

How can the ones' complement representation of a negative number be obtained from its sign-magnitude representation?

- a. By bitwise complementing the magnitude
- b. By bitwise complementing the whole number including the sign bit
- c. By adding one to the magnitude
- d. By clearing the sign bit

✓ By bitwise complementing the magnitude

Correct, and merely is the deck's word: nothing else changes.

✗ By bitwise complementing the whole number including the sign bit

Wrong. Complementing the sign bit too would turn the number positive.

✗ By adding one to the magnitude

Wrong. Adding one belongs to the two's complement.

✗ By clearing the sign bit

Wrong. Clearing the sign bit gives the positive value.

Complement the magnitude and leave the sign bit alone. Check it: sign magnitude minus 3 on 5 bits is 10011, and complementing just the magnitude 0011 gives 1100, so 11100, which is one's complement minus 3.

Using the quick method on 001110, which is 14, what is the 6-bit two's complement of minus 14?

- a. 110010
- b. 110001
- c. 110011
- d. 101110

✓ 110010

Correct. Write down the positive value 001110, copy the digits from the right up to and including the first 1, which is just 10, then invert the rest: 0011 becomes 1100.

✗ 110001

Wrong. The copied part is 10, not 01; the first 1 from the right is in the second position.

✗ 110011

Wrong. That is one more than the answer.

✗ 101110

Wrong. Only the bits above the first 1 are inverted.

Write down the positive value 001110, copy the digits from the right up to and including the first 1, which is just 10, then invert the rest: 0011 becomes 1100. The answer is 110010.

4.87

HARD

Data S30

The quick method: write the positive binary number, copy the digits from the right hand side up to and including the first _____, then _____ the rest of the digits.

BLANK 1

- a. 1
- b. 0
- c. 2
- d. sign bit

BLANK 2

- a. invert
- b. copy
- c. shift
- d. add to

Answer: **1, invert**

Up to and including the first 1, then invert the rest. It gives the same answer as complement then add one, without the addition.

4.88

HARD

Data S30

Match each decimal value to its 8-bit two's complement representation.

plus 24	00011000
plus 127	01111111
minus 39	11011001
minus 92	10100100
minus 128	10000000

All five worked examples. Minus 128 is the interesting one: it has no positive counterpart in 8 bits, which is why the two's complement range is asymmetric.

4.89

HARD

Data S30

What is minus 92 in 8-bit two's complement?

- a. 10100100
- b. 10100011
- c. 11011100
- d. 01011100

✓ 10100100

Correct. 92 is 01011100.

✗ 10100011

Wrong. That is the one's complement of 92, one short of the answer.

✗ 11011100

Wrong. Check the bits against 01011100.

✗ 01011100

Wrong. That is plus 92.

| 92 is 01011100. Its one's complement is 10100011, and adding one gives 10100100.

What decimal value does the two's complement 110011 represent?

- a. minus 13
- b. minus 12
- c. 51
- d. minus 51

✓ minus 13

Correct. The most significant bit is 1, so it is negative.

✗ minus 12

Wrong. That would come from stopping at the one's complement, 001100, and forgetting to add the 1.

✗ 51

Wrong. 51 is the unsigned reading of the pattern.

✗ minus 51

Wrong. Reading the pattern directly and adding a minus sign skips the complement step.

The most significant bit is 1, so it is negative. Take the one's complement, 001100, add 1 to get 001101, which is plus 13. So 110011 is minus 13.

For sign-magnitude in 4 bits, how many bits would the value 8 require?

- a. At least 5, as 01000
- b. At least 4, as 1000
- c. At least 6
- d. It fits in 4 bits

✓ At least 5, as 01000

Correct, and the deck gives 01000 exactly.

✗ At least 4, as 1000

Wrong, and it is the unsigned answer: 1000 is 8 only if there is no sign bit.

✗ At least 6

Wrong. Five is enough.

✗ It fits in 4 bits

Wrong. In 4-bit sign magnitude the largest value is 7.

At least 5 bits, written 01000. The extra bit is the sign, so the magnitude 1000 needs four bits of its own and the sign needs a fifth.

In the telemetry solution, what is minus 182 in 16-bit two's complement, in hexadecimal?

- a. FF4A
- b. FF49
- c. 00B6
- d. FC01

✓ FF4A

Correct. 182 is 0000000010110110, inverting gives 1111111101001001, adding 1 gives 1111111101001010, which is FF4A.

✗ FF49

Wrong. FF49 is the one's complement, one short of the answer.

✗ 00B6

Wrong. 00B6 is plus 182.

✗ FC01

Wrong. FC01 is minus 1023, the altitude.

182 is 0000000010110110, inverting gives 1111111101001001, adding 1 gives 1111111101001010, which is FF4A.

4.93

HARD

Rev S6

Working minus 182: plus 182 in 16 bits is _____, and after inverting and adding 1 the two's complement is _____.

BLANK 1

- a. 0000000010110110
- b. 0000000010110111
- c. 1111111101001001
- d. 0000000101110110

BLANK 2

- a. 1111111101001010
- b. 1111111101001001
- c. 1111111101001011
- d. 0000000010110110

Answer: **0000000010110110, 1111111101001010**

0000000010110110 then 1111111101001010. The middle step, the one's complement 1111111101001001, is the one the slide shows separately.

4.94

HARD

Rev S7

Altitude minus 1023 in 16-bit two's complement is, in hexadecimal:

- a. FC01
- b. FC00
- c. 03FF
- d. FF4A

✓ FC01

Correct. 1023 is 0000001111111111, inverting gives 1111110000000000, and adding 1 gives 1111110000000001, which is FC01.

✗ FC00

Wrong. FC00 is the one's complement, before the 1 is added.

✗ 03FF

Wrong. 03FF is plus 1023.

✗ FF4A

Wrong. FF4A is minus 182, the temperature.

1023 is 0000001111111111, inverting gives 1111110000000000, and adding 1 gives 1111110000000001, which is FC01. Note that the slide prints this sum with one digit too many; the hexadecimal it gives, FC01, is right.

4.95

HARD

Rev S7

On the altitude slide, the printed two's complement of minus 1023 has seventeen binary digits, one more than a 16-bit register can hold.

Answer: **true**

It does. The slide prints 11111100000000001 where the 16-bit answer is 1111110000000001. The hexadecimal beside it, FC01, is correct, so the intended value is not in doubt.

4.96

HARD

Rev S8

Adding 111111101001010, which is minus 182, to 0000000111100111, which is 487, gives which decimal result?

- a. 305
- b. 669
- c. minus 305
- d. 182

✓ 305

Correct. The sum is 0000000100110001, which is 305.

✗ 669

Wrong. 669 is 487 plus 182, treating the negative operand as positive.

✗ minus 305

Wrong sign. The positive operand is the larger of the two.

✗ 182

Wrong. 182 is one of the inputs, not the answer.

The sum is 0000000100110001, which is 305. And that is the arithmetic check: 487 minus 182 is 305, so two's complement subtraction really is just addition.

4.97

HARD

Rev S8

The 16-bit result of the telemetry addition is _____, which is 305.

- a. 0000000100110001
- b. 0000001010011101
- c. 0000000100110000
- d. 1111111100110001

Answer: **0000000100110001**

0000000100110001. In hexadecimal that is 0131, and 0x131 is 305.

5 Floating Point and IEEE 754

5.1

EASY

The real test 1, question 15

IEEE Double Precision uses:

- a. 64 bits
- b. 32 bits
- c. 48 bits
- d. 128 bits

✓ 64 bits

Correct. $1 + 11 + 52 = 64$.

✗ 32 bits

Wrong. 32 bits is SINGLE precision, and the name double says twice that.

✗ 48 bits

Wrong. 48 bits is not an IEEE 754 basic format.

✗ 128 bits

Wrong. 128 bits is quadruple precision, which the course does not set.

Double precision is 64 bits: 1 sign, 11 exponent, 52 mantissa. Single precision is 32 bits: 1 sign, 8 exponent, 23 mantissa. Those two lines answer most floating-point questions on this paper.

What is the principal advantage of floating-point representation?

- a. Exact decimal storage
- b. Faster arithmetic
- c. Elimination of overflow
- d. Representation of very large and very small values

X Exact decimal storage

Wrong, and the opposite of the truth. Floating point is approximate: 0.1 cannot be stored exactly in binary.

X Faster arithmetic

Wrong. Floating-point arithmetic is generally SLOWER than integer arithmetic, since it must align exponents and normalise.

X Elimination of overflow

Wrong. The range is larger, not infinite, so overflow still occurs at the ends.

✓ Representation of very large and very small values

Correct. Huge and tiny magnitudes in the same fixed-width word, which is what dynamic range means.

| A movable binary point buys enormous dynamic range in a fixed number of bits.

5.3

EASY

The real test 1, question 29

A weather simulation program requires highly accurate numerical computations. Which architectural component is most critical?

- a. Floating-point representation
- b. Cache replacement policy
- c. Character encoding
- d. Control bus design

✓ Floating-point representation

Correct. Precision comes from the number format, and simulations accumulate error over millions of steps.

✗ Cache replacement policy

Wrong. A replacement policy affects hit rate and therefore speed, not the computed values.

✗ Character encoding

Wrong. Character encoding is for text, and a simulation computes on numbers.

✗ Control bus design

Wrong. The control bus carries command signals, which has no bearing on numerical accuracy.

Same rule as the banking question: when a stem says ACCURATE or PRECISE numerical work, the answer is floating-point representation. When it says FASTER, look to cache and organization instead.

5.4

EASY

Data S36

Match each condition to its definition in the deck.

Overflow The exponent is too large to be represented in the Exponent field

Underflow The number is too small to be represented in the Exponent field

Both definitions come from one sentence on the same slide, which is why they are so easily swapped under exam pressure.

5.5

EASY

Data S37

To reduce the chances of underflow or overflow, use _____ -bit double-precision arithmetic.

- a. 64
- b. 32
- c. 16
- d. 128

Answer: **64**

64 bits. The deck then shows two cases of what happens when arithmetic goes wrong: the Ariane 5 and the Vancouver Stock Exchange.

5.6

EASY

Data S49

Which standards does the deck name for representing floating point?

- a. IEEE-754 and 854
- b. IEEE-754 only
- c. ISO 8859-1
- d. IEEE-802

✓ IEEE-754 and 854

Correct, both are on the slide.

✗ IEEE-754 only

Wrong, incomplete: 854 is named alongside it.

✗ ISO 8859-1

Wrong. ISO 8859-1 is the extended ASCII table.

✗ IEEE-802

Wrong. Not mentioned in this course.

IEEE-754, 854. The second number is easy to miss, since 754 is the one usually quoted.

5.7

EASY

Data S49

It is called floating point because the decimal point can ____ .

- a. move
- b. vanish
- c. double
- d. round

Answer: **move**

Move. The exponent says where the point sits, so the same bits can express a huge number or a tiny fraction.

5.8

EASY

Data S49

Floating point is useful for scientific calculations. What can it represent?

- a. Very large integers
- b. Very small fractions
- c. Character strings
- d. Memory addresses only

✓ Very large integers

Correct, very large integers is the first item.

✓ Very small fractions

Correct, very small fractions is the second.

✗ Character strings

Wrong. Character strings need an alphanumeric code such as ASCII.

✗ Memory addresses only

Wrong. Addresses are integers and are not stored in floating point.

Both ends of the scale, which is the whole reason for the format.

5.9**EASY**

Data S50

In single precision, a sign bit of 0 signifies:

- a. Plus
- b. Minus
- c. Zero
- d. An underflow

✓ Plus

Correct. 0 is plus and 1 is minus, the same convention as sign magnitude for integers.

✗ Minus

Wrong, that is a sign bit of 1.

✗ Zero

Wrong. Zero is signalled by the whole pattern, not by the sign bit alone.

✗ An underflow

Wrong. Underflow is an exponent condition.

| 0 is plus and 1 is minus, the same convention as sign magnitude for integers.

5.10**EASY**

Data S63

Quiz 3 also asks students to represent minus 82 using sign-magnitude, 1's and 2's complement.

Answer: **true**

| It does. The quiz has two questions and they cross both topics: one floating point, one signed integers.

Which strategy helps reduce overflow/underflow occurrences?

- a. Using ASCII
- b. Using double precision arithmetic
- c. Using fewer bits
- d. Using BCD

✗ Using ASCII

Wrong. ASCII is a character code and performs no arithmetic at all.

✓ Using double precision arithmetic

Correct. More exponent bits means more range, so fewer results fall off either end.

✗ Using fewer bits

Wrong, and exactly backwards. Fewer bits shrink the range and cause MORE overflow.

✗ Using BCD

Wrong. BCD codes decimal digits and is less compact than binary, so it does not extend numeric range.

Double precision widens the exponent from 8 bits to 11, which stretches the range from about 10 to the power plus or minus 38 out to about 10 to the power plus or minus 308.

A banking application depends heavily on accurate decimal calculations. Which architectural feature directly influences calculation accuracy?

- a. Floating-point representation
- b. Cache hierarchy
- c. Control signals
- d. Bus width

✓ Floating-point representation

Correct. The mantissa width sets how many significant digits survive, which is precision itself.

✗ Cache hierarchy

Wrong. Cache changes how quickly data arrives, never what the computed value is.

✗ Control signals

Wrong. Control signals sequence the hardware. They do not alter numeric precision.

✗ Bus width

Wrong. A wider bus moves more bits per transfer, a speed matter and not an accuracy one.

Accuracy is decided by how a number is REPRESENTED: how many mantissa bits hold the significant digits.

Approximately what numeric range can IEEE Double Precision represent?

- a. $10^{\pm 38}$
- b. $10^{\pm 128}$
- c. $10^{\pm 308}$
- d. $10^{\pm 256}$

✗ $10^{\pm 38}$

Wrong. That is the SINGLE precision figure, the other half of the pair.

✗ $10^{\pm 128}$

Wrong. 128 is close to the single precision exponent bias of 127, not a decimal range.

✓ $10^{\pm 308}$

Correct. About 10 to the power plus or minus 308.

✗ $10^{\pm 256}$

Wrong. 256 is a familiar power of two, put here to look plausible. It is not the figure.

Double precision reaches roughly 10 to the power plus or minus 308. Learn it as a pair with single precision at roughly 10 to the power plus or minus 38, because this paper sets both.

Approximately what numeric range can IEEE Single Precision represent?

- a. $10^{\pm 308}$
- b. $10^{\pm 38}$
- c. $10^{\pm 20}$
- d. $10^{\pm 10}$

✗ $10^{\pm 308}$

Wrong. That is the DOUBLE precision figure, offered to catch a reader who learned only one of the pair.

✓ $10^{\pm 38}$

Correct. About 10 to the power plus or minus 38 from 32 bits with an 8-bit exponent.

✗ $10^{\pm 20}$

Wrong, far too small. 8 exponent bits reach well past 10 to the power 20.

✗ $10^{\pm 10}$

Wrong, smaller still. That is roughly the range of a 32-bit integer, not of a float.

| Single precision reaches roughly 10 to the power plus or minus 38.

5.15

MEDIUM

The real test 1, question 54

IEEE Single Precision allocates how many bits to the exponent?

- a. 8
- b. 23
- c. 7
- d. 11

✓ 8

Correct. 8 exponent bits, which is where the plus or minus 38 range comes from.

✗ 23

Wrong. 23 is the MANTISSA width of the same format.

✗ 7

Wrong. 7 bits is the width of an ASCII character, not of this exponent.

✗ 11

Wrong. 11 is the exponent width of DOUBLE precision.

| Single precision is $1 + 8 + 23 = 32$ bits, so the exponent takes 8.

The exponent bias used in IEEE Double Precision is:

- a. 127
- b. 255
- c. 511
- d. 1023

✗ 127

Wrong. 127 is the SINGLE precision bias, from 8 exponent bits.

✗ 255

Wrong. 255 is the largest value in 8 bits, not a bias.

✗ 511

Wrong. 511 would be the bias for a 10-bit exponent, and double uses 11.

✓ 1023

Correct. 2 to the power 10, which is 1024, minus 1.

The bias is 2 to the power (e minus 1), minus 1, where e is the exponent width. Double has 11 exponent bits, so the bias is $2^{11} - 1 = 1023$.

5.17

MEDIUM

Data S36

In the floating point sense, underflow is a situation in which:

- a. The number is too small to be represented in the Exponent field
- b. The exponent is too large to be represented in the Exponent field
- c. The mantissa has too many digits
- d. The sign bit is set on a positive number

✓ The number is too small to be represented in the Exponent field

Correct. Too small is underflow, too large is overflow.

✗ The exponent is too large to be represented in the Exponent field

Wrong, that is overflow, the other half of the same sentence.

✗ The mantissa has too many digits

Wrong. Too many mantissa digits costs precision, not range, and the deck calls that inexact.

✗ The sign bit is set on a positive number

Wrong. The sign bit has nothing to do with either condition.

Too small is underflow, too large is overflow. Both are stated against the Exponent field, which is the deck's own framing.

5.18

MEDIUM

Data S36

Underflow can in part be regarded as _____ overflow of the exponent of the floating point value.

- a. negative
- b. positive
- c. silent
- d. partial

Answer: **negative**

Negative overflow. It is a useful way to hold the pair: both conditions are the exponent running off an end of its range, just different ends.

To reduce the chances of underflow or overflow, the deck says you can use:

- a. 64-bit double-precision arithmetic
- b. 32-bit single-precision arithmetic
- c. 16-bit integer arithmetic
- d. BCD arithmetic

✓ 64-bit double-precision arithmetic

Correct. 64-bit double precision.

✗ 32-bit single-precision arithmetic

Wrong. Single precision is the narrower option, so it makes the problem more likely, not less.

✗ 16-bit integer arithmetic

Wrong, and a 16-bit signed integer is precisely what destroyed the Ariane 5 on the next slide.

✗ BCD arithmetic

Wrong. BCD is a decimal digit code, not a floating point format.

64-bit double precision. Its 11-bit exponent reaches about 10 to the power plus or minus 308, against single precision's 38.

5.20

MEDIUM

Data S39

How long after lift-off did the unmanned Ariane 5 rocket explode?

- a. Forty seconds
- b. Four seconds
- c. Forty minutes
- d. Four minutes

✓ Forty seconds

Correct. Just forty seconds after lift-off, on 4 June 1996, on its first voyage, after a decade of development costing 7 billion dollars.

✗ Four seconds

Wrong. The deck says forty seconds.

✗ Forty minutes

Wrong unit. Just forty seconds after lift-off, on 4 June 1996, on its first voyage, after a decade of development costing 7 billion dollars.

✗ Four minutes

Wrong on both count and unit.

Just forty seconds after lift-off, on 4 June 1996, on its first voyage, after a decade of development costing 7 billion dollars.

5.21

MEDIUM

Data S39

The 64 bit floating point number that failed to convert related to the _____ velocity of the rocket with respect to the platform.

- a. horizontal
- b. vertical
- c. angular
- d. relative

Answer: **horizontal**

Horizontal velocity. The Ariane 5 flew a different trajectory from its predecessor, so a value that had always been small on the old rocket was not small on the new one.

5.22

MEDIUM

Data S40

The Vancouver stock exchange index was instituted in 1982 at a value of 1000.000. What had it fallen to twenty two months later?

- a. 520
- b. 1098.892
- c. 500
- d. 820

✓ 520

Correct. It had fallen to 520, while the correctly rounded calculation gave 1098.

✗ 1098.892

Wrong. 1098.892 is what the rounded calculation gave, that is the value it should have shown.

✗ 500

Wrong. The deck says 520.

✗ 820

Wrong. That figure appears nowhere.

It had fallen to 520, while the correctly rounded calculation gave 1098.892. The index had lost almost half its value to arithmetic alone.

5.23

MEDIUM

Data S40

In _____ the Vancouver stock exchange instituted a new index initialized to a value of _____ , updated after each transaction.

BLANK 1

- a. 1982
- b. 1996
- c. 1984
- d. 1992

BLANK 2

- a. 1000.000
- b. 1098.892
- c. 520
- d. 100.000

Answer: **1982, 1000.000**

1982, starting at 1000.000. Updating after each transaction is what let a tiny error per update compound into a huge one.

What caused the Vancouver index to drift so far from its true value?

- a. The updated value was truncated rather than rounded
- b. The updated value was rounded rather than truncated
- c. A 64 bit float was narrowed to a 16 bit integer
- d. The exponent underflowed

✓ The updated value was truncated rather than rounded

Correct, and the direction of the bias is why the error only ever grew.

✗ The updated value was rounded rather than truncated

Wrong, and reversed: rounding is what should have been done, and it gave 1098.892.

✗ A 64 bit float was narrowed to a 16 bit integer

Wrong. That is the Ariane 5.

✗ The exponent underflowed

Wrong. No exponent condition is involved.

Truncated rather than rounded. Truncation always loses in the same direction, so thousands of updates pushed the index steadily down.

5.25

MEDIUM

Data S48

In the deck's example, 976,000,000,000,000 is written in scientific notation as:

- a. 9.76 times 10 to the power 14
- b. 9.76 times 10 to the power 15
- c. 9.76 times 10 to the power minus 14
- d. 97.6 times 10 to the power 13

✓ 9.76 times 10 to the power 14

Correct. 9. 9.76 times 10 to the power 14. Its companion on the slide is 0.0000000000000976, which is 9.76 times 10 to the power minus 14.

✗ 9.76 times 10 to the power 15

Wrong by one in the exponent.

✗ 9.76 times 10 to the power minus 14

Wrong sign, and that is the small companion value.

✗ 97.6 times 10 to the power 13

Wrong form. Scientific notation puts a single digit before the point.

9.76 times 10 to the power 14. Its companion on the slide is 0.0000000000000976, which is 9.76 times 10 to the power minus 14.

5.26

MEDIUM

Data S48

The deck writes 0.0000000000000976 as 9.76 times 10 to the power ____ .

- a. -14
- b. 14
- c. -13
- d. -15

Answer: **-14**

Minus 14. The pair of examples makes the point that one notation covers both the very large and the very small.

5.27

MEDIUM

Data S48

A number represented by M times B to the power plus or minus E is stored in a binary word with which fields?

- a. The mantissa
- b. The exponent E
- c. The sign
- d. The base B

✓ The mantissa

Correct, the mantissa is stored.

✓ The exponent E

Correct, the exponent E is stored.

✓ The sign

Correct, and it is the one bit at the front of both single and double precision.

✗ The base B

Wrong. The deck says the base B is implicit and need not be stored.

The base B is implicit and need not be stored, which is why it costs no bits. The sign, exponent and mantissa are the three that do.

5.28

MEDIUM

Data S48

In the form M times B to the power plus or minus E , the base B is _____ and need not be stored.

- a. implicit
- b. explicit
- c. variable
- d. signed

Answer: **implicit**

Implicit. Every number in the format shares the same base, so recording it in each one would be a waste of bits.

Single precision floating point can represent numbers of roughly what order?

- a. 10 to the power plus or minus 38
- b. 10 to the power plus or minus 308
- c. 10 to the power plus or minus 127
- d. 10 to the power plus or minus 23

✓ 10 to the power plus or minus 38

Correct. About 10 to the power plus or minus 38 for single precision, against 10 to the power plus or minus 308 for double.

✗ 10 to the power plus or minus 308

Wrong. That is double precision.

✗ 10 to the power plus or minus 127

Wrong. 127 is the single precision exponent bias, not the decimal range.

✗ 10 to the power plus or minus 23

Wrong. 23 is the number of mantissa bits in single precision.

About 10 to the power plus or minus 38 for single precision, against 10 to the power plus or minus 308 for double.

How is a 32-bit single precision floating point number laid out?

- a. 1 sign bit, an 8-bit excess-127 exponent, and a 23-bit mantissa fraction
- b. 1 sign bit, a 23-bit exponent, and an 8-bit mantissa fraction
- c. 1 sign bit, an 11-bit exponent, and a 20-bit mantissa fraction
- d. 8 sign bits, a 1-bit exponent, and a 23-bit mantissa

✓ 1 sign bit, an 8-bit excess-127 exponent, and a 23-bit mantissa fraction

Correct. 1 plus 8 plus 23 is 32.

✗ 1 sign bit, a 23-bit exponent, and an 8-bit mantissa fraction

Wrong, the two field widths are swapped.

✗ 1 sign bit, an 11-bit exponent, and a 20-bit mantissa fraction

Wrong. 11 exponent bits belong to double precision.

✗ 8 sign bits, a 1-bit exponent, and a 23-bit mantissa

Wrong. There is exactly one sign bit.

1 plus 8 plus 23 is 32. The exponent is stored in excess-127 form, which is what the bias of 127 means.

5.31

MEDIUM

Data S50

Single precision is _____ bits in total, with a _____ -bit mantissa fraction.

BLANK 1

- a. 32
- b. 64
- c. 16
- d. 23

BLANK 2

- a. 23
- b. 8
- c. 52
- d. 24

Answer: **32, 23**

32 bits, of which 23 are the mantissa fraction. Add the 8-bit exponent and the 1 sign bit and the 32 is accounted for.

5.32

MEDIUM

Data S51

How is a double precision floating point number laid out?

- a. 64 bits: 1 sign bit, an 11-bit excess-1023 exponent, and a 52-bit mantissa fraction
- b. 64 bits: 1 sign bit, an 8-bit excess-127 exponent, and a 55-bit mantissa fraction
- c. 32 bits: 1 sign bit, an 11-bit exponent, and a 20-bit mantissa fraction
- d. 64 bits: 2 sign bits, a 10-bit exponent, and a 52-bit mantissa fraction

✓ 64 bits: 1 sign bit, an 11-bit excess-1023 exponent, and a 52-bit mantissa fraction

Correct. 1 plus 11 plus 52 is 64, and the exponent is stored in excess-1023 form.

✗ 64 bits: 1 sign bit, an 8-bit excess-127 exponent, and a 55-bit mantissa fraction

Wrong. 8 bits and a bias of 127 are single precision figures.

✗ 32 bits: 1 sign bit, an 11-bit exponent, and a 20-bit mantissa fraction

Wrong total. Double precision is 64 bits.

✗ 64 bits: 2 sign bits, a 10-bit exponent, and a 52-bit mantissa fraction

Wrong. There is one sign bit, and the exponent is 11 bits.

1 plus 11 plus 52 is 64, and the exponent is stored in excess-1023 form.

The value represented by a double precision number is plus or minus 1.M times 2 to the power:

- a. E prime minus 1023
- b. E prime minus 127
- c. E prime minus 308
- d. E prime plus 1023

✓ E prime minus 1023

Correct. Minus 1023, the double precision bias.

✗ E prime minus 127

Wrong, that is the single precision bias.

✗ E prime minus 308

Wrong. 308 is the decimal order of magnitude, not the bias.

✗ E prime plus 1023

Wrong sign. The bias is subtracted from the stored exponent.

Minus 1023, the double precision bias. The single precision formula is the same shape with 127 in its place.

Select the operations the IEEE Standard requires at a minimum.

- a. Add
- b. Subtract
- c. Multiply
- d. Divide
- e. Remainder
- f. Square Root
- g. Decimal and binary conversion
- h. Logarithm

✓ Add

Correct, add heads the list.

✓ Subtract

Correct, subtract is second.

✓ Multiply

Correct, multiply is third.

✓ Divide

Correct, divide is fourth.

✓ Remainder

Correct, remainder is fifth and easily missed.

✓ Square Root

Correct, square root is sixth and is a genuine surprise on a minimum list.

✓ Decimal and binary conversion

Correct, decimal and binary conversion closes the list.

✗ Logarithm

Wrong. Logarithm is not required by the standard as the deck presents it.

Seven required operations. Remainder and square root are the two most often forgotten, and logarithm is not among them.

5.35

MEDIUM

Data S52

The IEEE Standard requires _____ operations at a minimum.

- a. 7
- b. 5
- c. 6
- d. 8

Answer: **7**

| Seven. Counting only the four familiar arithmetic operations is the usual undercount.

Select the IEEE floating point exceptions the deck lists.

- a. Underflow
- b. Overflow
- c. Divide by 0
- d. Inexact
- e. Invalid
- f. Truncated

✓ Underflow

Correct, underflow is first.

✓ Overflow

Correct, overflow is second.

✓ Divide by 0

Correct, divide by 0 is third.

✓ Inexact

Correct, inexact is fourth and is the one raised by ordinary rounding.

✓ Invalid

Correct, invalid closes the list.

✗ Truncated

Wrong. Truncated is not one of the five, although the Vancouver case shows how much damage truncation can do.

Five exceptions: underflow, overflow, divide by 0, inexact, invalid. Truncation is what went wrong in Vancouver, but it is not an IEEE exception.

5.37

MEDIUM

Data S53

In step 1 of putting 64.2 into IEEE single precision, what is 64 in binary?

- a. 1000000
- b. 100000
- c. 10000000
- d. 1000001

✓ 1000000

Correct, seven digits in all.

✗ 100000

Wrong. That is 32.

✗ 10000000

Wrong. That is 128.

✗ 1000001

Wrong. That is 65.

64 is 2 to the power 6, so it is a 1 followed by six zeros: 1000000.

5.38

MEDIUM

Data S54

Normalising 1000000.00110011 and on gives 1.00000000110011 and on, times 2 to the power:

- a. 6
- b. 7
- c. 133
- d. 127

✓ 6

Correct, and the deck calls 6 the true exponent.

✗ 7

Wrong by one. Count the places the point moves, not the digits before it.

✗ 133

Wrong. 133 is the biased exponent, 6 plus 127.

✗ 127

Wrong. 127 is the bias.

The point moves six places left, so the true exponent is 6. 133 is what it becomes after the bias is added, and 127 is the bias itself.

5.39

MEDIUM

Data S54

Step 2 is to _____ the binary representation, that is, make it look like scientific notation.

- a. normalize
- b. truncate
- c. invert
- d. shift

Answer: **normalize**

Normalize. The result always has exactly one 1 before the point, which is why that 1 can then be left implicit.

5.40

MEDIUM

Data S54

For the standard form the exponent needs to be in biased-127 form, so 6 plus 127 is _____, whose 8-bit unsigned representation is _____.

BLANK 1

- a. 133
- b. 121
- c. 134
- d. 127

BLANK 2

- a. 10000101
- b. 10000110
- c. 01111111
- d. 10000100

Answer: **133, 10000101**

133, which is 10000101. Adding the bias is what lets a signed exponent be stored in an unsigned field.

5.41

MEDIUM

Data S54

The mantissa stored, F, is taken from where in the normalised form, and how much of it?

- a. The 23 bits to the right of the radix point
- b. The 23 bits to the left of the radix point
- c. All 32 bits
- d. The 8 bits to the right of the radix point

✓ The 23 bits to the right of the radix point

Correct. 23 bits, all of them to the right of the point.

✗ The 23 bits to the left of the radix point

Wrong. After normalising there is only one digit to the left of the point.

✗ All 32 bits

Wrong. 32 is the whole word, which also holds the sign and the exponent.

✗ The 8 bits to the right of the radix point

Wrong. 8 is the exponent width.

| 23 bits, all of them to the right of the point. The single 1 to the left is implicit and is not stored.

5.42

MEDIUM

Data S54

Step 4 of the worked example also requires including the correct sign bit.

Answer: **true**

| It does. For 64.2 the sign bit is 0, since the number is positive, and the finished pattern begins with that 0.

Select the values the exercise slide asks students to represent in IEEE format.

- a. 10.5
- b. 9.999 times 10 to the 1, plus 1.610 times 10 to the minus 1
- c. .510 times minus 0.4375
- d. 1.10 times 10 to the 10, times 9.2 times 10 to the minus 5
- e. 64.2

✓ 10.5

Correct, 10.5 is the first exercise.

✓ 9.999 times 10 to the 1, plus 1.610 times 10 to the minus 1

Correct, and it asks for an addition before the conversion.

✓ .510 times minus 0.4375

Correct, and it asks for a multiplication first.

✓ 1.10 times 10 to the 10, times 9.2 times 10 to the minus 5

Correct, another multiplication.

✗ 64.2

Wrong. 64.2 is the worked example immediately above the exercise.

Four exercise items. 64.2 is the worked example the exercise follows, not one of the questions set.

Quiz 3 asks for the decimal value of which hexadecimal number, represented in IEEE single precision format?

- a. BE7BAC10h
- b. 42806666h
- c. A34Eh
- d. FF4Ah

✓ BE7BAC10h

Correct. BE7BAC10h.

✗ 42806666h

Wrong. 42806666h is the answer to the worked example, 64.2.

✗ A34Eh

Wrong. A34E is from the hexadecimal conversion section.

✗ FF4Ah

Wrong. FF4A is minus 182 from the revision telemetry question.

BE7BAC10h. Its leading hex digit B is 1011 in binary, so the sign bit is 1 and the value is negative.

A floating-point number is generally stored as:

- a. Exponent, Sign, Mantissa
- b. Mantissa, Exponent, Sign
- c. ASCII digits
- d. Sign, Mantissa, Exponent

X Exponent, Sign, Mantissa

Wrong. The sign is always the leading bit, so it cannot come second.

X Mantissa, Exponent, Sign

Wrong. This puts the sign last, when it is in fact the first bit of the word.

X ASCII digits

Wrong. ASCII digits would store the number as text, which is not a floating-point format at all.

✓ Sign, Mantissa, Exponent

Correct as the paper keyed it, and the only answer offered that puts the sign first. Be aware that the true IEEE 754 order is Sign, Exponent, Mantissa, which this paper does not list.

Sign, Mantissa, Exponent is what the exam keyed, and it is the only option offered that puts the sign first. Know the truth as well: real IEEE 754 hardware lays the fields out as Sign, EXPONENT, Mantissa, and that ordering is not offered here. Take the keyed answer in this test, and write Sign, Exponent, Mantissa in a theory answer.

5.46

HARD

Data S36

Arithmetic underflow can occur when the true result of a floating point operation is:

- a. Smaller in magnitude, that is closer to zero, than the smallest value representable as a normal floating point number in the target datatype
- b. Larger in magnitude than the largest representable value
- c. Exactly zero
- d. Negative

✓ Smaller in magnitude, that is closer to zero, than the smallest value representable as a normal floating point number in the target datatype

Correct, and the deck spells out that smaller in magnitude means closer to zero.

✗ Larger in magnitude than the largest representable value

Wrong, that is overflow.

✗ Exactly zero

Wrong. Zero is representable exactly.

✗ Negative

Wrong. Sign is irrelevant; magnitude is what matters.

Closer to zero than the smallest normal value. The words closer to zero matter, because a very large negative number is not an underflow.

5.47

HARD

Data S36

Arithmetic underflow occurs when the true result is smaller in magnitude than the smallest value representable as a _____ floating point number in the target datatype.

- a. normal
- b. signed
- c. rounded
- d. exact

Answer: **normal**

Normal. The word is doing real work: values below the smallest normal number are not all lost at once, but they are no longer normal floating point numbers.

If the exponent part can represent values from minus 127 to 127, a result with a value less than which figure may cause underflow?

- a. minus 127
- b. 127
- c. 0
- d. minus 1023

✓ minus 127

Correct, and the figure is the bottom of the stated exponent range.

✗ 127

Wrong. Exceeding plus 127 is overflow.

✗ 0

Wrong. An exponent of 0 is perfectly ordinary.

✗ minus 1023

Wrong. Minus 1023 is the double precision bias, not the single precision exponent limit.

Less than minus 127. Underflow can in part be regarded as negative overflow of the exponent, so it is the bottom of the exponent range that is breached.

5.49

HARD

Data S39

On June 4, _____ an unmanned Ariane 5 rocket launched by the European Space Agency exploded, after a decade of development costing _____ billion dollars.

BLANK 1

- a. 1996
- b. 1986
- c. 1982
- d. 2006

BLANK 2

- a. 7
- b. 5
- c. 10
- d. 3

Answer: **1996, 7**

1996, and 7 billion dollars. 1982 is the other date in this section, and it belongs to the Vancouver Stock Exchange.

5.50

HARD

Data S39

What was the software error that destroyed the Ariane 5?

- a. A 64 bit floating point number was converted to a 16 bit signed integer, and the value was too large for it
- b. A 16 bit integer was converted to a 64 bit floating point number
- c. A value was truncated rather than rounded
- d. An exponent underflowed

✓ A 64 bit floating point number was converted to a 16 bit signed integer, and the value was too large for it

Correct, and it is an overflow on conversion, not in the arithmetic itself.

✗ A 16 bit integer was converted to a 64 bit floating point number

Wrong direction. Widening a value is safe; it was the narrowing that failed.

✗ A value was truncated rather than rounded

Wrong. Truncation rather than rounding is the Vancouver Stock Exchange case on the next slide.

✗ An exponent underflowed

Wrong. The value was too large, not too close to zero.

A 64 bit floating point number relating to the horizontal velocity of the rocket with respect to the platform was converted to a 16 bit signed integer, and the conversion failed. Note that the slide gives the limit as 32678; the largest 16-bit signed integer is in fact 32767.

5.51

HARD

Data S40

The correctly rounded calculation gave the Vancouver index a value of ____ .

- a. 1098.892
- b. 1000.000
- c. 520
- d. 1089.892

Answer: **1098.892**

1098.892 against the 520 actually displayed, a gap of well over five hundred points from a difference in rounding alone.

5.52

HARD

Data S50

The value represented by a single precision number is:

- a. plus or minus $1.M$ times 2 to the power E prime minus 127
- b. plus or minus M times 2 to the power E prime minus 127
- c. plus or minus $1.M$ times 2 to the power E prime minus 1023
- d. plus or minus $1.M$ times 10 to the power E prime minus 127

✓ plus or minus $1.M$ times 2 to the power E prime minus 127

Correct. The leading 1 is implied, the stored exponent has 127 subtracted from it, and the base is 2.

✗ plus or minus M times 2 to the power E prime minus 127

Wrong. The 1 in front of the point is implicit and must be restored.

✗ plus or minus $1.M$ times 2 to the power E prime minus 1023

Wrong bias. 1023 belongs to double precision.

✗ plus or minus $1.M$ times 10 to the power E prime minus 127

Wrong base. The base is 2, not 10.

| The leading 1 is implied, the stored exponent has 127 subtracted from it, and the base is 2.

5.53

HARD

Data S51

Double precision uses an _____ -bit excess-1023 exponent and a _____ -bit mantissa fraction, and can represent numbers of the order of 10 to the power plus or minus 308.

BLANK 1

- a. 11
- b. 8
- c. 12
- d. 10

BLANK 2

- a. 52
- b. 23
- c. 53
- d. 51

Answer: **11, 52**

| 11 and 52. Set them against single precision's 8 and 23 and the whole comparison is in hand.

5.54

HARD

Data S50, Data S51

Match each figure to the precision it belongs to.

32 bits in total	Single precision
8-bit exponent	Single precision
23-bit mantissa fraction	Single precision
Bias of 127	Single precision
64 bits in total	Double precision
11-bit exponent	Double precision
52-bit mantissa fraction	Double precision
Bias of 1023	Double precision

Eight figures, four a side. Every one of them has been asked about on a past paper, so this mapping is worth being fluent in.

5.55

HARD

Data S52

Besides underflow, overflow, divide by 0 and invalid, the fifth IEEE exception is ____ .

- a. inexact
- b. truncated
- c. unstable
- d. denormal

Answer: **inexact**

Inexact. It fires whenever a result has to be rounded, which for a value like 0.2 in binary is almost always.

5.56

HARD

Data S53

The fractional part .2 is found by repeated doubling, and gives the repeating pattern

_____ .

- a. 0011
- b. 1100
- c. 0101
- d. 1010

Answer: **0011**

0011, repeating for ever. That is why 0.2 can never be stored exactly in binary, however many bits you use.

5.57

HARD

Data S53

In the repeated doubling for .2, the third step is .8 times 2, which is 1.6. What digit does it contribute?

- a. 1
- b. 0
- c. 6
- d. 8

✓ 1

Correct, the whole part of 1.6.

✗ 0

Wrong. 0 is contributed when the product is less than 1, as in the first two steps.

✗ 6

Wrong. The 6 is the fractional part carried into the next step, not the digit emitted.

✗ 8

Wrong. 8 is the input to this step, not its output.

The digit is the whole part of the product, so 1.6 contributes 1 and leaves .6 to carry forward. The four steps give 0, 0, 1, 1, and then the pattern repeats.

5.58

HARD

Data S53

Putting the halves back together, 64.2 in binary is ____ .0011001100110011 and on.

- a. 1000000
- b. 0100000
- c. 1000001
- d. 10000000

Answer: **1000000**

1000000.0011001100110011 and on. The whole part terminates; the fraction never does.

5.59

HARD

Data S54

With a true exponent of 6, what is the biased exponent stored for E, in binary?

- a. 10000101
- b. 00000110
- c. 01111111
- d. 10000110

✓ 10000101

Correct. 6 plus 127 is 133, and 133 in 8-bit unsigned is 1000 0101.

✗ 00000110

Wrong. That is 6 itself, with no bias added.

✗ 01111111

Wrong. That is 127, the bias alone.

✗ 10000110

Wrong. That is 134, one too many.

6 plus 127 is 133, and 133 in 8-bit unsigned is 1000 0101.

5.60

HARD

Data S55

In the finished single precision representation of 64.2, what are S and E?

- a. S is 0 and E is 10000101
- b. S is 1 and E is 10000101
- c. S is 0 and E is 01111111
- d. S is 0 and E is 00000110

✓ S is 0 and E is 10000101

Correct. S is 0 because 64.

✗ S is 1 and E is 10000101

Wrong. A sign bit of 1 would make the number negative.

✗ S is 0 and E is 01111111

Wrong. 01111111 is 127, the bias, not the biased exponent.

✗ S is 0 and E is 00000110

Wrong. 00000110 is the true exponent 6, before the bias is added.

S is 0 because 64.2 is positive, and E is 10000101, which is the biased exponent 133.

5.61

HARD

Data S55

The stored mantissa F for 64.2 is _____ .

- a. 00000000110011001100110
- b. 00000000110011001100111
- c. 10000000110011001100110
- d. 00000001100110011001100

Answer: **00000000110011001100110**

00000000110011001100110, twenty three bits. The repeating 0011 pattern is simply cut off when the field runs out, which is where the inexactness comes from.

What is the IEEE single precision representation of 64.2 in hexadecimal?

- a. 42806666h
- b. 42800666h
- c. BE7BAC10h
- d. 40806666h

✓ 42806666h

Correct. The 32 bits group into 0100 0010 1000 0000 0110 0110 0110 0110, which is 42806666h.

✗ 42800666h

Wrong. Check the third byte: the repeating pattern gives 66, not 06.

✗ BE7BAC10h

Wrong. BE7BAC10h is the value set in Quiz 3, a different number.

✗ 40806666h

Wrong. The second hex digit comes from the exponent and is 2, not 0.

| The 32 bits group into 0100 0010 1000 0000 0110 0110 0110 0110, which is 42806666h.

5.63

HARD

Data S55

Grouped into fours, the representation of 64.2 begins _____ 0010 1000 0000 and ends in the repeating hexadecimal digits _____ .

BLANK 1

- a. 0100
- b. 1010
- c. 0010
- d. 1100

BLANK 2

- a. 6666
- b. 3333
- c. 9999
- d. 0666

Answer: **0100, 6666**

0100 at the front, and 6666 at the back. The repeating 6 in hexadecimal is the repeating 0110 in binary, which is the tail of the endless 0011 pattern of 0.2.

6 Instruction Types and Formats

6.1

EASY

The real test 2, question 17

An Instruction format defines:

- a. Layout of bits within an instruction
- b. Cache organization
- c. Internal processor wiring
- d. Operating system memory allocation

✓ Layout of bits within an instruction

Correct. It is the bit layout, and the decoder reads the word according to it.

✗ Cache organization

Wrong. Cache organization is about memory levels and mapping, not the shape of an instruction.

✗ Internal processor wiring

Wrong. Internal wiring is microarchitecture, hidden below the format.

✗ Operating system memory allocation

Wrong. Memory allocation is a software concern, handled by the operating system at run time.

A format is a map of the instruction word: which bits are the opcode, which name registers, which carry an address or a constant.

In a typical instruction format, the opcode field specifies:

- a. Memory address
- b. Operation to perform
- c. Register contents
- d. Interrupt priority

✗ Memory address

Wrong. An address lives in the ADDRESS field, a separate part of the word.

✓ Operation to perform

Correct. ADD, LOAD, JUMP and the rest are named by the opcode.

✗ Register contents

Wrong. Register CONTENTS are data held in the register file. The instruction can only name the register, never carry its contents.

✗ Interrupt priority

Wrong. Interrupt priority is set by the interrupt controller, not encoded in every instruction.

Opcode is short for operation code. It answers WHAT to do, while the operand fields answer WHAT TO DO IT TO.

Which instruction format is most commonly associated with stack-based architectures?

- a. Three-address
- b. Zero-address
- c. Two-address
- d. One-address

✗ Three-address

Wrong. Three-address names every operand explicitly, the opposite of the stack idea.

✓ Zero-address

Correct. Zero addresses, because the top of the stack is implied. Note that PUSH and POP still carry one address.

✗ Two-address

Wrong. Two-address is the classic register or memory format, as in ADD A, B.

✗ One-address

Wrong. One-address is the ACCUMULATOR machine, where the accumulator is implied but the operand is still named.

Stack machines need no address fields at all because the operands are implied by the stack, so their arithmetic instructions are zero-address.

6.4

EASY

ISA S22

An instruction is usually made up of a combination of an operation code and:

- a. Some way of specifying an operand, most commonly by its location or address in memory
- b. The number of cycles it will take
- c. A checksum
- d. The address of the next instruction

✓ Some way of specifying an operand, most commonly by its location or address in memory

Correct. Opcode plus a way of specifying an operand, usually by its location or address in memory.

✗ The number of cycles it will take

Wrong. Cycle counts are never encoded in an instruction.

✗ A checksum

Wrong. No checksum appears in any instruction format in this course.

✗ The address of the next instruction

Wrong. The program counter supplies the next address, except when a branch overrides it.

Opcode plus a way of specifying an operand, usually by its location or address in memory. That is the whole shape of an instruction.

6.5

EASY

ISA S22

An operand is most commonly specified by its location or _____ in memory.

- a. address
- b. value
- c. size
- d. type

Answer: **address**

Address. Immediate addressing is the exception that proves the rule: there the value itself sits inside the instruction.

6.6

EASY

ISA S22

Instruction Format is defined as:

- a. The layout of bits in an instruction
- b. The order in which instructions execute
- c. The set of instructions a processor supports
- d. The way operands are accessed at runtime

✓ The layout of bits in an instruction

Correct. The layout of bits in an instruction, for example the pattern 1110001111110000001110000 standing for ADD R1,R2.

✗ The order in which instructions execute

Wrong. Execution order is program flow, not format.

✗ The set of instructions a processor supports

Wrong. That is the instruction set.

✗ The way operands are accessed at runtime

Wrong. That is the addressing modes.

The layout of bits in an instruction, for example the pattern 1110001111110000001110000 standing for ADD R1,R2.

6.7

EASY

ISA S22

The deck gives 1110001111110000001110000 as an example of the bit layout for ADD R1,R2.

Answer: **true**

It does, and the point of showing it is that an assembly mnemonic is only a readable name for a pattern of bits.

6.8

EASY

ISA S22

There is usually more than one instruction format in an instruction set.

Answer: **true**

There usually is. Different instructions need different fields, so one fixed layout would waste bits on most of them.

6.9

EASY

ISA S23

The bits of an instruction are divided into groups called:

- a. Fields
- b. Blocks
- c. Words
- d. Segments

✓ Fields

Correct. Fields.

✗ Blocks

Wrong. A block is a group of words in memory, from the cache topic.

✗ Words

Wrong. A word is one or more complete rows of memory.

✗ Segments

Wrong. Segment is the word the pipelining deck uses for a stage.

Fields. Blocks and words are memory units, and segments are pipeline stages in this course, so the word matters.

6.10

EASY

ISA S23

Match each of the three common instruction fields to its position in the deck's general structure.

First field	Opcode
Second field	Mode
Third field	Address or Operand

Opcode, Mode, Address. The order is worth holding because the address is always last, which is what lets an address field be widened without disturbing the opcode.

6.11**EASY**

ISA S25

In the 16-bit format example, what does the 4-bit Register field specify?

- a. A register, for example R1 or R2
- b. The operation, for example ADD or SUB
- c. A memory location
- d. The addressing mode

✓ A register, for example R1 or R2

Correct. A register, and with four bits it can name sixteen of them.

✗ The operation, for example ADD or SUB

Wrong. The operation comes from the opcode field, which is also 4 bits.

✗ A memory location

Wrong. That is the address field.

✗ The addressing mode

Wrong. This particular example has no separate mode field.

| A register, and with four bits it can name sixteen of them.

6.12**EASY**

ISA S33

The same program is written in _____ different address formats to compare them.

- a. 4
- b. 3
- c. 2
- d. 5

Answer: **4**

| Four: three-address, two-address, one-address and zero-address.

The MOV instruction moves or _____ the operands to and from memory and processor registers.

- a. transfers
- b. compares
- c. complements
- d. shifts

Answer: **transfers**

Transfers. Like all data movement, it copies: the source is unchanged, only the destination is written.

The revision class repeats that an instruction is divided into:

- a. Opcode, operands and addressing mode
- b. Opcode and operand only
- c. Opcode, mode and condition code
- d. Registers, memory and addresses

✓ Opcode, operands and addressing mode

Correct. The same three parts as the ISA deck, along with the reminder that instructions typically have 0, 1, 2 or 3 operands.

✗ Opcode and operand only

Wrong. The addressing mode is a third part.

✗ Opcode, mode and condition code

Wrong. The condition code register is not part of an instruction.

✗ Registers, memory and addresses

Wrong. These are places operands can live, not the parts of an instruction.

The same three parts as the ISA deck, along with the reminder that instructions typically have 0, 1, 2 or 3 operands.

The revision class repeats that the most common fields found in instruction formats are:

- a. Opcode, Mode, and Address or Operand
- b. Opcode, Register, and Address
- c. Tag, Index and Offset
- d. Sign, Exponent and Mantissa

✓ Opcode, Mode, and Address or Operand

Correct. The same three fields as the ISA deck.

✗ Opcode, Register, and Address

Wrong. Register is a field in one worked example, not one of the three general fields.

✗ Tag, Index and Offset

Wrong. Tag, Index and Offset is the direct mapping cache address format.

✗ Sign, Exponent and Mantissa

Wrong. Sign, Exponent and Mantissa is the IEEE 754 layout.

The same three fields as the ISA deck. The other two options are real three-part layouts from elsewhere in the course, which is what makes them tempting.

A stack machine executes arithmetic operations without explicitly specifying operand addresses because operands are:

- a. Stored in cache
- b. Stored in virtual memory
- c. Stored in ROM
- d. Located at the top of the stack

✗ Stored in cache

Wrong. Cache may well hold the stack, but cache is transparent and never removes the need for an address.

✗ Stored in virtual memory

Wrong. Virtual memory is an address translation scheme, so it still requires addresses.

✗ Stored in ROM

Wrong. ROM holds firmware, and operands there could not be pushed and popped.

✓ Located at the top of the stack

Correct. The top of the stack is an implied location, which is exactly why the format is zero-address.

The operands are implied: an ADD pops the top two items, adds them and pushes the result back. Because their location is fixed by the rule, no address field is needed.

A processor employs fixed-length 32-bit instructions. Which advantage is most likely achieved?

- a. Faster instruction decoding
- b. Elimination of registers
- c. Unlimited opcode space
- d. Smaller program size

✓ Faster instruction decoding

Correct. No need to decode part of an instruction to learn where the next one begins.

✗ Elimination of registers

Wrong. Instruction length has nothing to do with whether registers exist, and RISC machines with fixed lengths have many.

✗ Unlimited opcode space

Wrong, and impossible. A fixed word actually LIMITS opcode space, since the field cannot grow.

✗ Smaller program size

Wrong, and this is the trap. Smaller programs are the VARIABLE-length advantage, which is why CISC uses variable length.

Every instruction starts at a predictable boundary and every field sits in a known place, so decoding is simple and pipelining is easy.

6.18

MEDIUM

The real test 2, question 44

An instruction contains the following fields: Opcode 8 bits, Register field 6 bits, Address field 18 bits. What is the total instruction length?

- a. 40 bits
- b. 32 bits
- c. 36 bits
- d. 30 bits

✗ 40 bits

Wrong, 8 too many. Check that the opcode was not counted twice.

✓ 32 bits

Correct. $8 + 6 + 18 = 32$, which is also the usual word size, a useful sanity check.

✗ 36 bits

Wrong, 4 too many.

✗ 30 bits

Wrong, 2 too few. Re-add the register field of 6.

| The fields simply add up: $8 + 6 + 18 = 32$ bits.

6.19

MEDIUM

ISA S22

An instruction is divided into which parts?

- a. Opcode, the function
- b. Operands, implicit or explicit
- c. Addressing mode
- d. Condition code

✓ Opcode, the function

Correct, and function is the deck's gloss on opcode.

✓ Operands, implicit or explicit

Correct, and implicit operands are what make one-address and zero-address formats possible.

✓ Addressing mode

Correct, the addressing mode is the third part.

✗ Condition code

Wrong. The condition code register is read by branch instructions but is not a field of an instruction.

Three parts. Note implicit or explicit against the operands: a zero-address instruction has operands, they are just implied by the stack.

6.20

MEDIUM

ISA S22

An instruction is divided into opcode, operands, which may be _____ or explicit, and addressing mode.

- a. implicit
- b. internal
- c. indirect
- d. indexed

Answer: **implicit**

Implicit. The accumulator in a one-address instruction is the clearest case: it is an operand nobody writes down.

6.21

MEDIUM

ISA S22

Instructions typically have how many operands?

- a. 0, 1, 2 or 3
- b. 1, 2 or 3
- c. Exactly 2
- d. Between 1 and 5

✓ 0, 1, 2 or 3

Correct, and zero is on the list because of stack machines.

✗ 1, 2 or 3

Wrong. Zero-address instructions exist, and the deck devotes three slides to them.

✗ Exactly 2

Wrong, though two-address instructions are the most common in commercial computers.

✗ Between 1 and 5

Wrong. The deck stops at 3.

0, 1, 2 or 3, which is exactly why the deck goes on to work the same expression through four address formats.

6.22

MEDIUM

ISA S23

The format of an instruction is usually depicted in a _____ box symbolizing the bits of the instruction as they appear in memory words or in a control register.

- a. rectangular
- b. circular
- c. triangular
- d. layered

Answer: **rectangular**

Rectangular, with each field drawn as a slice of it. That drawing convention is why exam questions can ask for field widths in bits.

6.23

MEDIUM

ISA S23

Select the most common fields found in instruction formats.

- a. Opcode
- b. Mode
- c. Address or Operand
- d. Length

✓ Opcode

Correct, the opcode field specifies the operation.

✓ Mode

Correct, the mode field says how the effective address is determined.

✓ Address or Operand

Correct, and the deck writes it as one field with two names.

✗ Length

Wrong. Length is not a field; in a variable-length format the opcode implies it.

Three fields: Opcode, Mode, Address or Operand. The same three appear on the revision slide.

6.24

MEDIUM

ISA S24

Match each instruction field to what it specifies.

Operation code field	The operation to be performed
Address field	A memory address or a processor register
Mode field	The way the operand or the effective address is determined

Three fields, three jobs. Note that the address field can designate a register, not only memory, which is what makes register addressing fit the same layout.

The mode field of an instruction specifies:

- a. The way the operand or the effective address is determined
- b. The operation to be performed
- c. A memory address or a processor register
- d. Whether the instruction is privileged

✓ The way the operand or the effective address is determined

Correct. How the operand is reached.

✗ The operation to be performed

Wrong. That is the operation code field.

✗ A memory address or a processor register

Wrong. That is the address field.

✗ Whether the instruction is privileged

Wrong. Privilege levels are not covered in this course.

| How the operand is reached. It is the addressing mode written into the instruction itself.

6.26

MEDIUM

ISA S25

In the simple 16-bit instruction format example, what are the three field widths?

- a. Opcode 4 bits, Register 4 bits, Address 8 bits
- b. Opcode 8 bits, Register 4 bits, Address 4 bits
- c. Opcode 4 bits, Register 8 bits, Address 4 bits
- d. Opcode 6 bits, Register 4 bits, Address 6 bits

✓ Opcode 4 bits, Register 4 bits, Address 8 bits

Correct. 4 plus 4 plus 8 is 16.

✗ Opcode 8 bits, Register 4 bits, Address 4 bits

Wrong, and it reverses opcode and address.

✗ Opcode 4 bits, Register 8 bits, Address 4 bits

Wrong. A register field of 8 bits would allow 256 registers, far more than the format implies.

✗ Opcode 6 bits, Register 4 bits, Address 6 bits

Wrong. The widths given are 4, 4 and 8.

4 plus 4 plus 8 is 16. The address takes half the instruction, which is typical: addresses need more bits than opcodes.

6.27

MEDIUM

ISA S25

In the 16-bit format example the Address field is ____ bits wide.

- a. 8
- b. 4
- c. 6
- d. 16

Answer: **8**

8 bits, so it can name 256 memory locations. That is what makes 11001010 read as location 202 in the binary example.

6.28

MEDIUM

ISA S25

In the 16-bit format, the _____ -bit opcode defines the operation and the 8-bit field gives the _____ location.

BLANK 1

- a. 4
- b. 8
- c. 6
- d. 2

BLANK 2

- a. memory
- b. register
- c. stack
- d. cache

Answer: **4, memory**

A 4-bit opcode and an 8-bit memory address. Four bits of opcode allow sixteen distinct operations, which is the opcode-space trade-off in miniature.

Which expression does the deck use to compare the 3, 2, 1 and 0 address instruction formats?

- a. X equals A plus B, times C plus D
- b. X equals A times B, plus C times D
- c. X equals A plus B plus C plus D
- d. X equals A minus B, times C minus D

✓ X equals A plus B, times C plus D

Correct. X equals A plus B, all times C plus D.

✗ X equals A times B, plus C times D

Wrong, and it swaps the operations round.

✗ X equals A plus B plus C plus D

Wrong. There is a multiplication in the expression.

✗ X equals A minus B, times C minus D

Wrong. Both inner operations are additions.

X equals A plus B, all times C plus D. Two additions and one multiplication, which is enough to show the difference between the formats without becoming long.

6.30

MEDIUM

ISA S34

How many instructions does the three-address program for X equals A plus B , times C plus D take?

- a. 3
- b. 6
- c. 7
- d. 8

✓ 3

Correct. Three: `ADD R1, A, B` then `ADD R2, C, D` then `MUL X, R1, R2`.

✗ 6

Wrong. Six is the two-address program.

✗ 7

Wrong. Seven is the one-address program.

✗ 8

Wrong. Eight is the zero-address program.

Three: `ADD R1, A, B` then `ADD R2, C, D` then `MUL X, R1, R2`. That shortness is the format's stated advantage.

6.31

MEDIUM

ISA S34

In the three-address program the symbol $M[A]$ denotes the _____ at the memory address symbolized by A .

- a. operand
- b. opcode
- c. register
- d. offset

Answer: **operand**

Operand. The notation matters, because $M[A]$ is the value stored at A , not A itself.

6.32

MEDIUM

ISA S34

What is the disadvantage of the three-address format?

- a. The binary-coded instructions require too many bits to specify three addresses
- b. The programs are longer than in any other format
- c. It needs a stack
- d. It cannot use registers

✓ The binary-coded instructions require too many bits to specify three addresses

Correct. Too many bits per instruction.

✗ The programs are longer than in any other format

Wrong, and reversed: the advantage is that it results in short programs.

✗ It needs a stack

Wrong. The stack belongs to the zero-address format.

✗ It cannot use registers

Wrong. The worked program uses R1 and R2 throughout.

Too many bits per instruction. It is the classic trade: fewer instructions, but each one is larger.

6.33

MEDIUM

ISA S34

The advantage of the three-address format is that it results in short programs when evaluating arithmetic expressions.

Answer: **true**

It does, and the worked example proves it: three instructions against eight for the stack machine.

6.34

MEDIUM

ISA S35

Two-address instructions are described as:

- a. The most common in commercial computers
- b. The rarest of the four formats
- c. Used only in stack machines
- d. Used only where there is no accumulator

✓ The most common in commercial computers

Correct. The most common in commercial computers, and each address field can specify either a processor register or a memory word.

✗ The rarest of the four formats

Wrong. It is the most common, not the rarest.

✗ Used only in stack machines

Wrong. Stack machines use the zero-address format.

✗ Used only where there is no accumulator

Wrong. The accumulator belongs to the one-address format.

The most common in commercial computers, and each address field can specify either a processor register or a memory word.

6.35

MEDIUM

ISA S35

In the two-address format each address field can specify either a processor register or a memory ____ .

- a. word
- b. block
- c. page
- d. line

Answer: **word**

Memory word. Blocks, pages and lines all belong to the memory hierarchy topic, not to instruction formats.

6.36

MEDIUM

ISA S37

One-address instructions use an implied register for all data manipulation. Which one?

- a. The accumulator, AC
- b. The program counter, PC
- c. The instruction register, IR
- d. The memory address register, MAR

✓ The accumulator, AC

Correct. The accumulator.

✗ The program counter, PC

Wrong. The PC holds the address of the next instruction.

✗ The instruction register, IR

Wrong. The IR holds the instruction being executed.

✗ The memory address register, MAR

Wrong. The MAR holds the address of the memory location being accessed.

The accumulator. Because it is implied, it never has to be named in the instruction, which is what leaves room for only one address.

6.37

MEDIUM

ISA S37

For multiplication and division in a one-address machine there is a need for a second register.

Answer: **true**

There is, and the deck then says it will neglect the second register and assume the AC contains the result of all operations, to keep the worked program simple.

6.38

MEDIUM

ISA S38

The one-address program begins ____ A, then ADD B, then STORE T.

- a. LOAD
- b. MOV
- c. PUSH
- d. MUL

Answer: **LOAD**

LOAD. In a one-address machine the accumulator has to be filled before anything can be done to it.

6.39

MEDIUM

ISA S38

In the one-address format all operations are done between the AC register and a memory operand.

Answer: **true**

They are, and that is the defining constraint: one operand is always the accumulator, so only the other needs an address.

6.40

MEDIUM

ISA S39

In a stack-organized computer, which instructions still need an address field?

- a. PUSH and POP
- b. ADD and MUL
- c. All of them
- d. None of them

✓ PUSH and POP

Correct. PUSH and POP need an address to say which operand communicates with the stack.

✗ ADD and MUL

Wrong, and reversed: these are the ones that use no address field.

✗ All of them

Wrong. If all of them needed one, it would not be a zero-address machine.

✗ None of them

Wrong. The stack has to be filled from somewhere, and PUSH is how.

PUSH and POP need an address to say which operand communicates with the stack. ADD and MUL take their operands from the top of the stack, so they need none.

6.41

MEDIUM

ISA S39

A stack-organized computer does not use an address field for the instructions ADD and _____.

- a. MUL
- b. PUSH
- c. POP
- d. LOAD

Answer: **MUL**

MUL. Both take their operands from the top of the stack, so there is nothing to name.

6.42

MEDIUM

ISA S40

In the zero-address program, _____ stands for top of stack.

- a. TOS
- b. AC
- c. SP
- d. PC

Answer: **TOS**

TOS. The deck's comments track it line by line, so you can see the operands stack up and then collapse.

6.43

MEDIUM

ISA S41

A stack computer can evaluate an arithmetic expression directly, without converting it to reverse Polish notation first.

Answer: **false**

The deck says the conversion is necessary. The stack has no way of holding a pending operator, so the operands must arrive before the operator does.

6.44

MEDIUM

ISA S41

Why is this type of computer called zero-address?

- a. Because of the absence of an address field in the computational instructions
- b. Because it has no memory addresses at all
- c. Because the stack always starts at address zero
- d. Because every operand is at address zero

✓ Because of the absence of an address field in the computational instructions

Correct, and computational is the word that keeps the claim true.

✗ Because it has no memory addresses at all

Wrong. PUSH A names an address.

✗ Because the stack always starts at address zero

Wrong. Nothing in the deck fixes the stack at address zero.

✗ Because every operand is at address zero

Wrong. The operands are at their own addresses; they are simply not named in ADD or MUL.

The computational instructions carry no address field. PUSH and POP still do, which is why the name is qualified in exactly that way.

6.45

MEDIUM

ISA S41

The name zero-address is given because of the absence of an address field in the _____ instructions.

- a. computational
- b. all
- c. branch
- d. transfer

Answer: **computational**

Computational. PUSH and POP are not computational, and they keep their address fields.

6.46

MEDIUM

Rev S15

The revision exercise asks for zero, one, two and three address formats for which machine?

- a. A Pentium 4
- b. An MC68000
- c. An IBM 801
- d. A DLX

✓ A Pentium 4

Correct. A Pentium 4 assembly language program segment.

✗ An MC68000

Wrong. The MC68000 supplies instruction examples in the ISA deck.

✗ An IBM 801

Wrong. The IBM 801 is an early RISC machine.

✗ A DLX

Wrong. DLX appears in the data hazard classification.

A Pentium 4 assembly language program segment. The other three machines all appear elsewhere in the course, which is what makes them plausible distractors.

6.47

MEDIUM

Rev S15

The revision zero-address version ends with ____ X.

- a. POP
- b. PUSH
- c. STORE
- d. MOV

Answer: **POP**

POP X, taking the single remaining value off the stack and writing it to X.

A processor designer wants to increase the number of available opcodes without changing instruction length. Which modification is most appropriate?

- a. Increase RAM capacity
- b. Increase opcode bits while reducing operand bits
- c. Add pipeline stages
- d. Increase cache size

X Increase RAM capacity

Wrong. RAM capacity is outside the processor and has no effect on how an instruction word is divided.

✓ Increase opcode bits while reducing operand bits

Correct. A fixed word, so one field grows only at another's expense.

X Add pipeline stages

Wrong. Pipeline stages change how instructions are PROCESSED, not how many distinct ones can be encoded.

X Increase cache size

Wrong. Cache size affects speed, not the encoding space of the instruction set.

The instruction word is a fixed budget of bits. If the total length may not grow, the only way to widen one field is to narrow another. An extra opcode bit doubles the opcode count, since k bits give 2 to the power k of them.

Which instruction format generally results in fewer memory accesses?

- a. Two-address
- b. Three-address
- c. Zero-address
- d. One-address

✗ Two-address

Wrong. Two-address must overwrite one source, so a value that is needed again has to be copied first.

✓ Three-address

Correct. Everything is named at once, so the expression completes in the fewest instructions.

✗ Zero-address

Wrong, and the furthest from it. A stack machine must PUSH each operand and POP the result, which is the most traffic of the four.

✗ One-address

Wrong. One-address routes everything through the accumulator, so each extra operand costs a separate load or store.

A three-address instruction names both sources and the destination, so a whole operation is done in one go. The shorter formats need extra loads, stores and pushes to move operands into place, and each of those is another access.

6.50**HARD**

ISA S26

The binary instruction 1010 0011 11001010 is interpreted as:

- a. ADD R3, [202]
- b. ADD R3, [201]
- c. SUB R3, [202]
- d. ADD R10, [202]

✓ ADD R3, [202]

Correct. Opcode 1010 is ADD, register 0011 is R3, and address 11001010 is 202.

✗ ADD R3, [201]

Wrong. 11001010 is 128 plus 64 plus 8 plus 2, which is 202.

✗ SUB R3, [202]

Wrong. The opcode 1010 stands for ADD in this example.

✗ ADD R10, [202]

Wrong. The register field 0011 is 3, not 10; 1010 is the opcode.

| Opcode 1010 is ADD, register 0011 is R3, and address 11001010 is 202.

6.51**HARD**

ISA S26

In the binary example the address field 11001010 is memory location ____ .

- a. 202
- b. 201
- c. 212
- d. 192

Answer: **202**

| 202. Check it: 128 plus 64 plus 8 plus 2.

6.52

HARD

ISA S27

The 32-bit instruction 000001 00001 00010 00011 00000000000 is interpreted as:

- a. ADD R1, R2, R3
- b. ADD R2, R1, R3
- c. SUB R1, R2, R3
- d. ADD R1, R2

✓ ADD R1, R2, R3

Correct, and it is a three-address instruction, hence three register fields.

✗ ADD R2, R1, R3

Wrong. The first register field after the opcode is the destination, and it holds 00001, which is R1.

✗ SUB R1, R2, R3

Wrong. The opcode given for ADD is 000001.

✗ ADD R1, R2

Wrong. Three register fields are present, so it is a three-operand instruction.

| Opcode 000001 is ADD, then R1 as the destination, R2 as operand 1 and R3 as operand 2.

6.53

HARD

ISA S27

In the 32-bit example the field 00001 is R1, the _____, and the field 00011 is R3, _____.

BLANK 1

- a. destination
- b. opcode
- c. operand 1
- d. address

BLANK 2

- a. operand 2
- b. operand 1
- c. destination
- d. the mode field

Answer: **destination, operand 2**

| Destination first, then operand 1, then operand 2. The eleven trailing zeros are unused in this instruction.

6.54

HARD

ISA S35

How many instructions does the two-address program for $X \text{ equals } A \text{ plus } B, \text{ times } C \text{ plus } D$ take?

- a. 6
- b. 3
- c. 7
- d. 8

✓ 6

Correct. Six: MOV R1, A then ADD R1, B then MOV R2, C then ADD R2, D then MUL R1, R2 then MOV X, R1.

✗ 3

Wrong. Three is the three-address program.

✗ 7

Wrong. Seven is the one-address program.

✗ 8

Wrong. Eight is the zero-address program.

Six: MOV R1, A then ADD R1, B then MOV R2, C then ADD R2, D then MUL R1, R2 then MOV X, R1.

6.55

HARD

ISA S35

The two-address program opens with MOV R1, A then ____ R1, B, and closes with MOV X, ____ .

BLANK 1

- a. ADD
- b. MOV
- c. MUL
- d. STORE

BLANK 2

- a. R1
- b. R2
- c. R3
- d. AC

Answer: **ADD, R1**

ADD R1, B, and the last instruction is MOV X, R1. R1 accumulates the whole result, which is why it is the register written out at the end.

In a two-address instruction, the first symbol listed is assumed to be:

- a. Both a source and the destination where the result is transferred
- b. The source only
- c. The destination only
- d. The opcode

✓ Both a source and the destination where the result is transferred

Correct, and it is the reason `ADD R1, B` overwrites `R1`.

✗ The source only

Wrong. If it were only a source, the result would have nowhere to go.

✗ The destination only

Wrong. It is read as well as written.

✗ The opcode

Wrong. The opcode is the mnemonic in front of the operands.

Both at once. That convention is what makes two addresses enough for an operation that logically needs three.

6.57

HARD

ISA S38

How many instructions does the one-address program for $X \text{ equals } A \text{ plus } B, \text{ times } C \text{ plus } D$ take?

- a. 7
- b. 6
- c. 3
- d. 8

✓ 7

Correct. Seven: LOAD A, ADD B, STORE T, LOAD C, ADD D, MOL T, STORE X.

✗ 6

Wrong. Six is the two-address program.

✗ 3

Wrong. Three is the three-address program.

✗ 8

Wrong. Eight is the zero-address program.

Seven: LOAD A, ADD B, STORE T, LOAD C, ADD D, MOL T, STORE X.

In the one-address program, what is T?

- a. The address of a temporary memory location for storing the intermediate result
- b. A second accumulator
- c. The top of the stack
- d. The destination of the final result

✓ The address of a temporary memory location for storing the intermediate result

Correct. A temporary memory location.

✗ A second accumulator

Wrong. The deck explicitly neglects the second register in this example.

✗ The top of the stack

Wrong. TOS, the top of stack, belongs to the zero-address program.

✗ The destination of the final result

Wrong. The final result goes to X.

A temporary memory location. With only one accumulator, the first sub-result has to be parked somewhere while the second is computed.

6.59

HARD

ISA S40

How many instructions does the zero-address program for $X \text{ equals } A \text{ plus } B, \text{ times } C \text{ plus } D$ take?

- a. 8
- b. 7
- c. 6
- d. 3

✓ 8

Correct. Eight: PUSH A, PUSH B, ADD, PUSH C, PUSH D, ADD, MOL, POP X.

✗ 7

Wrong. Seven is the one-address program.

✗ 6

Wrong. Six is the two-address program.

✗ 3

Wrong. Three is the three-address program.

Eight: PUSH A, PUSH B, ADD, PUSH C, PUSH D, ADD, MOL, POP X. It is the longest of the four programs, which is the cost of carrying no addresses.

6.60

HARD

ISA S41

To evaluate arithmetic expressions in a stack computer, it is necessary to convert the expression into:

- a. Reverse Polish notation
- b. Scientific notation
- c. Two's complement
- d. Binary coded decimal

✓ Reverse Polish notation

Correct. Reverse Polish notation, where the operator follows its operands.

✗ Scientific notation

Wrong. Scientific notation belongs to floating point.

✗ Two's complement

Wrong. Two's complement is a way of writing negative integers.

✗ Binary coded decimal

Wrong. BCD codes decimal digits.

Reverse Polish notation, where the operator follows its operands. That is exactly the order the PUSH, PUSH, ADD sequence takes.

6.61

HARD

Rev S15

The revision exercise evaluates $X \text{ equals } A \text{ times } B, \text{ plus } C \text{ over } D, \text{ all divided by } F \text{ times } G \text{ plus } J \text{ times } \underline{\hspace{1cm}} .$

- a. K
- b. L
- c. H
- d. E

Answer: **K**

K. The expression uses A, B, C, D, F, G, J and K, eight operands, which is why every version of the program is long.

6.62

HARD

Rev S15

In the revision worked answer, which instruction opens the three-address version?

- a. MUL R1, A, B
- b. DIV R2, C, D
- c. ADD R1, R1, R2
- d. PUSH A

✓ MUL R1, A, B

Correct. MUL R1, A, B, because A times B is the innermost operation of the expression.

✗ DIV R2, C, D

Wrong. It is the second instruction, not the first.

✗ ADD R1, R1, R2

Wrong. It is the third, combining the first two results.

✗ PUSH A

Wrong. PUSH belongs to the zero-address version.

MUL R1, A, B, because A times B is the innermost operation of the expression. DIV R2, C, D follows it.

6.63

HARD

Rev S15

The revision three-address answer ends with ____ X, R2, R3.

- a. DIV
- b. MUL
- c. ADD
- d. MOV

Answer: **DIV**

DIV, since the whole expression is one quantity divided by another, so the division is the last thing done.

How does the revision zero-address version begin?

- a. PUSH A, PUSH B, MUL
- b. PUSH A, MUL, PUSH B
- c. MUL, PUSH A, PUSH B
- d. LOAD A, MUL B

✓ PUSH A, PUSH B, MUL

Correct. Both operands are pushed before the operator, which is reverse Polish order.

✗ PUSH A, MUL, PUSH B

Wrong. MUL would find only one operand on the stack.

✗ MUL, PUSH A, PUSH B

Wrong. MUL would find an empty stack.

✗ LOAD A, MUL B

Wrong. LOAD and a one-operand MUL belong to the one-address version.

Both operands are pushed before the operator, which is reverse Polish order. The program closes with POP X.

7 Addressing Modes

7.1

EASY

The real test 1, question 38

A compiler embeds a constant value directly within an instruction instead of storing it in memory. Which addressing mode is being used?

- a. Immediate Addressing
- b. Direct Addressing
- c. Indirect Addressing
- d. Register Addressing

✓ Immediate Addressing

Correct. The value travels with the instruction, which is what "embedded directly" says.

✗ Direct Addressing

Wrong. Direct puts the ADDRESS of the operand in the instruction, so one memory read is still required.

✗ Indirect Addressing

Wrong. Indirect puts the address OF the address in the instruction, needing two reads.

✗ Register Addressing

Wrong. Register addressing names a register that holds the operand, so the value is in the register file and not in the instruction.

Immediate means the operand itself is inside the instruction word, so no memory access is needed to fetch it.

Suppose the instruction address = 500 and displacement = +30. Using PC-relative addressing, the effective address becomes:

- a. 30
- b. 500
- c. 470
- d. 530

✗ 30

Wrong. 30 is the displacement alone, with the base forgotten.

✗ 500

Wrong. 500 is the base alone, with the displacement forgotten.

✗ 470

Wrong sign. 500 minus 30 gives 470, but the displacement is stated as +30.

✓ 530

Correct. $500 + 30 = 530$.

PC-relative adds the displacement to the current instruction address: $500 + 30 = 530$. This is how branches are encoded, and why code can be relocated freely.

An instruction contains the operand itself within the instruction. Which addressing mode is used?

- a. Indexed
- b. Register
- c. Immediate
- d. Direct

✗ Indexed

Wrong. Indexed carries a displacement to be added to an index register, not the operand.

✗ Register

Wrong. Register addressing carries a register NAME, and the operand lives in that register.

✓ Immediate

Correct. The value is in the instruction word itself.

✗ Direct

Wrong. Direct carries the operand's ADDRESS. The distinction between direct and immediate is exactly what this question tests.

The operand itself, not its address, sits in the instruction. That is the definition of immediate addressing.

Suppose the base register = 4000 and displacement = 120. Using Base Register Addressing, the effective address is:

- a. 3880
- b. 120
- c. 4000
- d. 4120

✗ 3880

Wrong sign. 4000 minus 120 gives 3880, but the displacement is added.

✗ 120

Wrong. 120 is the displacement alone, with no base.

✗ 4000

Wrong. 4000 is the base alone, with no displacement.

✓ 4120

Correct. $4000 + 120 = 4120$.

Base plus displacement: $4000 + 120 = 4120$. Base register addressing is how a program is relocated, since moving the base moves everything at once.

7.5

EASY

ISA S47

How does the deck describe immediate addressing?

- a. One of the simplest and fastest addressing modes in the ISA
- b. The slowest addressing mode, because it needs two memory reads
- c. The only mode that requires an effective address
- d. A mode used only for branch instructions

✓ One of the simplest and fastest addressing modes in the ISA

Correct. Simplest and fastest, because the operand travels inside the instruction and no memory access is needed to fetch it.

✗ The slowest addressing mode, because it needs two memory reads

Wrong. Two memory reads is indirect addressing, the slowest of these modes.

✗ The only mode that requires an effective address

Wrong, and reversed: immediate is the one mode where no effective address is required at all.

✗ A mode used only for branch instructions

Wrong. Relative addressing is the branch mode; immediate supplies constants.

Simplest and fastest, because the operand travels inside the instruction and no memory access is needed to fetch it.

7.6

EASY

ISA S47

Immediate addressing is one of the simplest and _____ addressing modes in the Instruction Set Architecture.

- a. fastest
- b. slowest
- c. widest
- d. rarest

Answer: **fastest**

Fastest. The operand arrives with the instruction, so there is nothing left to fetch.

In immediate addressing, where is the actual operand?

- a. Directly embedded inside the instruction itself
- b. In a register named by the instruction
- c. At the memory location named by the instruction
- d. At the memory location pointed to by another memory location

✓ Directly embedded inside the instruction itself

Correct. Inside the instruction, rather than in a register or a memory location.

✗ In a register named by the instruction

Wrong. That is register addressing.

✗ At the memory location named by the instruction

Wrong. That is direct addressing.

✗ At the memory location pointed to by another memory location

Wrong. That is indirect addressing.

Inside the instruction, rather than in a register or a memory location. The other three options are register, direct and indirect addressing in turn.

7.8

EASY

ISA S49

In direct addressing, the address field of the instruction contains:

- a. The actual memory location of the operand
- b. The operand itself
- c. The address of another memory location that holds the operand's address
- d. An offset to be added to the program counter

✓ The actual memory location of the operand

Correct. The actual location.

✗ The operand itself

Wrong. That is immediate addressing.

✗ The address of another memory location that holds the operand's address

Wrong. That is indirect addressing, which needs two accesses.

✗ An offset to be added to the program counter

Wrong. That is relative addressing.

| The actual location. One memory access gets you the operand.

7.9

EASY

ISA S49

Direct addressing is a technique in which the address field of the instruction contains the _____ memory location of the operand.

- a. actual
- b. relative
- c. indirect
- d. implied

Answer: **actual**

| Actual. The word rules out both the pointer of indirect addressing and the offset of relative addressing.

7.10**EASY**

ISA S49

The deck breaks direct addressing into ____ steps.

- a. 4
- b. 3
- c. 5
- d. 2

Answer: **4**

Four. Compare it with indirect addressing, which takes five because of the extra memory lookup.

7.11**EASY**

ISA S51

The basic format of an indirect instruction is given as:

- a. Opcode then Address
- b. Opcode then Mode then Address
- c. Tag then Index then Offset
- d. Address then Opcode

✓ Opcode then Address

Correct. Opcode and Address, the same two-field shape as a direct instruction.

✗ Opcode then Mode then Address

Wrong. The deck's basic format on this slide shows two fields.

✗ Tag then Index then Offset

Wrong. That is the direct mapping cache address format.

✗ Address then Opcode

Wrong. The opcode comes first in every format in this course.

Opcode and Address, the same two-field shape as a direct instruction. What changes is how the address field is used, not the layout.

7.12**EASY**

ISA S54

The relative addressing formula is Effective Address equals _____ plus Offset.

- a. PC
- b. IR
- c. MAR
- d. AC

Answer: **PC**

PC. This formula has appeared on a past paper, so it is worth being able to write down cold.

7.13**EASY**

ISA S58

BEQ R1, R2, plus 30 jumps to 230 if R1 _____ R2.

- a. equals
- b. exceeds
- c. differs from
- d. is less than

Answer: **equals**

Equals, since BEQ is branch if equal. Its companion BNE branches when the two differ.

7.14

EASY

Rev S17

In the revision addressing mode example, what are the contents of the PC and R1?

- a. PC is 7000 and R1 is 1500
- b. PC is 1500 and R1 is 7000
- c. PC is 1200 and R1 is 4500
- d. PC is 7000 and R1 is 1200

✓ PC is 7000 and R1 is 1500

Correct. PC equals 7000, R1 equals 1500.

✗ PC is 1500 and R1 is 7000

Wrong, and swapping them would change the relative and register indirect answers.

✗ PC is 1200 and R1 is 4500

Wrong. 1200 is the instruction's address field and 4500 is a memory content.

✗ PC is 7000 and R1 is 1200

Wrong. R1 holds 1500.

PC equals 7000, R1 equals 1500. Every part of the six-way answer depends on those two values plus the memory table.

7.15

EASY

Rev S17

In the revision addressing example, R1 contains ____ .

- a. 1500
- b. 7000
- c. 1200
- d. 6000

Answer: **1500**

1500. Under register addressing that value is the operand; under register indirect it is the effective address.

7.16

EASY

Rev S19

Immediate addressing requires no effective address.

Answer: **true**

None is required, because there is nothing to locate: the operand is already in the instruction.

The introduction deck says addressing modes determine:

- a. How operands are accessed during instruction execution
- b. Which operations the processor supports
- c. How instructions are stored in memory
- d. How blocks are mapped into cache lines

✓ How operands are accessed during instruction execution

Correct. How operands are accessed during instruction execution, which matches the ISA deck's phrase, methods used to access operands at runtime.

✗ Which operations the processor supports

Wrong. That is the instruction set.

✗ How instructions are stored in memory

Wrong. That is the instruction format.

✗ How blocks are mapped into cache lines

Wrong. That is a cache mapping function.

How operands are accessed during instruction execution, which matches the ISA deck's phrase, methods used to access operands at runtime.

Select the addressing mode examples the introduction deck gives.

- a. Immediate addressing
- b. Direct addressing
- c. Indirect addressing
- d. Register addressing
- e. Relative addressing

✓ Immediate addressing

Correct, immediate heads the list.

✓ Direct addressing

Correct, direct is second.

✓ Indirect addressing

Correct, indirect is third.

✓ Register addressing

Correct, register closes the list of four.

✗ Relative addressing

Wrong on this slide. Relative addressing gets its own treatment in the ISA deck, with $EA = PC + \text{Offset}$.

Four examples in the introduction deck. The ISA deck's own list adds indexed, and relative is taught in full later, but neither is on this slide.

In which application does the deck say fast memory access improves frame rates and responsiveness?

- a. Gaming systems
- b. Embedded systems
- c. Financial systems
- d. Medical devices

✓ Gaming systems

Correct. Gaming systems.

✗ Embedded systems

Wrong. Embedded systems are named for efficient memory access in devices with limited memory.

✗ Financial systems

Wrong. Financial systems appear under data types and floating point.

✗ Medical devices

Wrong. Medical devices appear under input/output systems.

Gaming systems. Embedded systems get the limited memory reason, and financial systems belong to the data types slide.

Assume $R1 = 2000$ and $\text{Memory}[2000] = 500$. Using Register Indirect Addressing, the operand is:

- a. Memory address only
- b. 2000
- c. 500
- d. R1

✗ Memory address only

Wrong. The question asks for the operand, the value being worked on, and "memory address only" names no value.

✗ 2000

Wrong. 2000 is the effective ADDRESS, one step short of the answer. This is the commonest slip on this question.

✓ 500

Correct. Follow the pointer once: R1 gives 2000, and $\text{Memory}[2000]$ gives 500.

✗ R1

Wrong. R1 is the register's name, not its contents nor the value it points at.

In register indirect, the register holds an ADDRESS, not the value. R1 gives 2000, and the operand is the contents of location 2000, which is 500.

Which addressing mode generally requires the fewest memory accesses during instruction execution?

- a. Immediate
- b. Indexed
- c. Direct
- d. Indirect

✓ Immediate

Correct. Zero further accesses, because the operand arrived inside the instruction.

✗ Indexed

Wrong. Indexed computes an address, then makes one access to fetch the operand.

✗ Direct

Wrong. Direct also makes one access, using the address in the instruction.

✗ Indirect

Wrong, and the most expensive of the four. Indirect reads the pointer, then reads the operand, so two accesses.

Count the trips to memory after the instruction is fetched. Immediate needs zero, direct and indexed need one, indirect needs two.

Consider $R2 = 300$, $\text{Memory}[320] = 900$, and instruction displacement = 20. Using Indexed Addressing with R2 as the index register, the effective address is:

- a. 300
- b. 320
- c. 280
- d. 900

✗ 300

Wrong. 300 is the index register alone, with the displacement not yet added.

✓ 320

Correct. $300 + 20 = 320$.

✗ 280

Wrong sign. That subtracts the displacement instead of adding it.

✗ 900

Wrong, and the trap the stem was built for. 900 is the CONTENTS of address 320, the operand, but the question asked for the effective address.

Indexed addressing adds the index register to the displacement: $300 + 20 = 320$. The question asks for the ADDRESS, so stop there.

7.23

MEDIUM

The real test 2, question 53

A processor designer aims to improve the efficiency of accessing dynamically allocated data structures such as linked lists. Which addressing mode is generally the most suitable?

- a. Immediate addressing
- b. Implied addressing
- c. Register addressing
- d. Indirect addressing

✗ Immediate addressing

Wrong. Immediate holds a constant fixed at compile time, and a dynamically allocated address is not known then.

✗ Implied addressing

Wrong. Implied names no operand at all, as in a stack machine, so it cannot chase a pointer.

✗ Register addressing

Wrong. Register addressing holds the VALUE in a register. It is a useful helper, but it does not itself follow a pointer to memory.

✓ Indirect addressing

Correct. Following the address held at a location is exactly what traversing a linked list requires.

A linked list IS a chain of pointers, and indirect addressing is the mode that follows a pointer to reach the data.

7.24

MEDIUM

ISA S47

In immediate addressing the data value is stored in a register or memory location rather than in the instruction.

Answer: **false**

The opposite. The deck says the actual operand is directly embedded inside the instruction itself, rather than being stored in a register or memory location.

In the deck's four steps for direct addressing, what happens immediately after the CPU extracts the memory address?

- a. It accesses that memory location
- b. It reads the instruction
- c. It retrieves the effective address stored there
- d. It adds the offset to the program counter

✓ It accesses that memory location

Correct. Read the instruction, extract the address, access the location, retrieve or store data.

✗ It reads the instruction

Wrong. That is step one, before the address is extracted.

✗ It retrieves the effective address stored there

Wrong. Retrieving an effective address from memory is the indirect procedure.

✗ It adds the offset to the program counter

Wrong. That is relative addressing.

Read the instruction, extract the address, access the location, retrieve or store data. Four steps, and the third is the memory access.

7.26

MEDIUM

ISA S51

In indirect addressing, the address field of the instruction contains:

- a. The address of another memory location that stores the effective address of the operand
- b. The actual memory location of the operand
- c. The operand itself
- d. A register number

✓ The address of another memory location that stores the effective address of the operand

Correct. A pointer to a pointer.

✗ The actual memory location of the operand

Wrong. That is direct addressing.

✗ The operand itself

Wrong. That is immediate addressing.

✗ A register number

Wrong. That is register or register indirect addressing.

A pointer to a pointer. The address field names a location, and that location holds the address where the operand really is.

7.27

MEDIUM

ISA S51

In indirect addressing, the address field contains a pointer to the operand's address rather than the operand's location itself.

Answer: **true**

That is the deck's own interpretation line: the address field does not contain the actual operand location, it contains a pointer to the operand's address.

7.28

MEDIUM

ISA S51

In an indirect instruction the address field does not contain the actual operand location; it contains a _____ to the operand's address.

- a. pointer
- b. copy
- c. mask
- d. offset

Answer: **pointer**

Pointer. This is why Test 2 tied indirect addressing to pointers and linked lists.

7.29

MEDIUM

ISA S52

In the step-by-step account of indirect addressing, what does the CPU do after it goes to memory location A?

- a. Retrieves the effective address stored there
- b. Retrieves the operand
- c. Adds the offset to the program counter
- d. Writes the result back

✓ Retrieves the effective address stored there

Correct. It collects the effective address, then makes a second access using it to reach the operand.

✗ Retrieves the operand

Wrong. The operand comes on the next step, after the effective address is known.

✗ Adds the offset to the program counter

Wrong. That is relative addressing.

✗ Writes the result back

Wrong. Writing back is not part of the addressing procedure.

It collects the effective address, then makes a second access using it to reach the operand. Two memory accesses in all.

7.30

MEDIUM

ISA S52

Reaching the operand under indirect addressing takes _____ memory accesses.

- a. 2
- b. 1
- c. 3
- d. 0

Answer: **2**

Two, which is why indirect addressing is the slowest of these modes and why direct addressing needs only one.

7.31

MEDIUM

ISA S52

For LOAD R1, (200) where Memory[200] contains 500, what is the final operation?

- a. R1 equals Memory[500]
- b. R1 equals 500
- c. R1 equals Memory[200]
- d. R1 equals 200

✓ R1 equals Memory[500]

Correct. 200 is the address field, Memory[200] is 500 which is the effective address, and the operand is Memory[500].

✗ R1 equals 500

Wrong. 500 is the effective address, not the operand; stopping here is the classic one-step-short error.

✗ R1 equals Memory[200]

Wrong. That is what a direct LOAD would give.

✗ R1 equals 200

Wrong. 200 is what an immediate LOAD would give.

200 is the address field, Memory[200] is 500 which is the effective address, and the operand is Memory[500].

7.32

MEDIUM

ISA S52

In LOAD R1, (200) the instruction contains 200, Memory[200] contains _____, and Memory[_____] contains the actual data.

BLANK 1

- a. 500
- b. 200
- c. 999
- d. 1200

BLANK 2

- a. 500
- b. 200
- c. 999
- d. 300

Answer: **500, 500**

500 both times: the content of the first location becomes the address of the second.

7.33

MEDIUM

ISA S53

For LOAD R2, (300) with Memory[300] equal to 400 and Memory[400] equal to 50, what is R2?

- a. 50
- b. 400
- c. 300
- d. 450

✓ 50

Correct. Memory[300] is 400, so the effective address is 400, and Memory[400] is 50.

✗ 400

Wrong. 400 is the effective address, one step short of the operand.

✗ 300

Wrong. 300 is the address field in the instruction.

✗ 450

Wrong. Nothing in the example is added.

Memory[300] is 400, so the effective address is 400, and Memory[400] is 50. R2 becomes 50.

7.34

MEDIUM

ISA S53

In LOAD R2, (300) the effective address is ____ .

- a. 400
- b. 300
- c. 50
- d. 350

Answer: **400**

400, which is the content of Memory[300]. The operand, 50, then comes from Memory[400].

7.35

MEDIUM

ISA S53

In the indirect STORE example, the value 30 is written to the location whose address was found in Memory[500].

Answer: **true**

It is. Indirection works the same way for a store as for a load: find the effective address first, then act on it.

7.36

MEDIUM

ISA S54

In relative addressing the effective address is obtained by:

- a. Adding a displacement to the current value of the Program Counter
- b. Adding a displacement to the contents of a base register
- c. Reading the address field directly
- d. Reading a second memory location

✓ Adding a displacement to the current value of the Program Counter

Correct. EA equals PC plus Offset.

✗ Adding a displacement to the contents of a base register

Wrong. That is base register addressing, which Test 2 asks about separately.

✗ Reading the address field directly

Wrong. That is direct addressing.

✗ Reading a second memory location

Wrong. That is indirect addressing.

EA equals PC plus Offset. Because the address is measured from where the program already is, the same code can run anywhere in memory.

7.37

MEDIUM

ISA S55

The offset is extracted from the instruction and added to the PC, and the result gives the _____ address.

- a. target
- b. return
- c. base
- d. physical

Answer: **target**

Target address, which the deck also calls the effective address on the previous slide.

7.38

MEDIUM

ISA S58

For BEQ R1, R2, plus 30 with PC equal to 200, what is the effective address?

- a. 230
- b. 170
- c. 30
- d. 200

✓ 230

Correct. 200 plus 30 is 230, and the machine jumps to 230 if R1 equals R2.

✗ 170

Wrong. The offset is positive, so it is added.

✗ 30

Wrong. 30 is the offset alone, with no PC.

✗ 200

Wrong. 200 is the PC alone, with no offset.

200 plus 30 is 230, and the machine jumps to 230 if R1 equals R2.

7.39

MEDIUM

ISA S58

For BNE R3, R4, minus 40 with PC equal to 1000, what is the effective address?

- a. 960
- b. 1040
- c. 40
- d. 1000

✓ 960

Correct. 1000 minus 40 is 960.

✗ 1040

Wrong sign. The offset is minus 40.

✗ 40

Wrong. That is the offset alone.

✗ 1000

Wrong. That is the PC alone.

1000 minus 40 is 960. A negative offset is what makes a backward branch, and therefore a loop, possible.

7.40

MEDIUM

ISA S58

A relative addressing offset can be negative, which is how a branch jumps backwards.

Answer: **true**

The deck's second example uses minus 40 and lands at 960, below the PC. Loops depend on exactly this.

In the revision memory table, what does address 4500 hold?

- a. 999
- b. 4500
- c. 200
- d. 400

✓ 999

Correct. 999. 999. It is the operand under indirect addressing, reached through Memory[1200] equal to 4500.

✗ 4500

Wrong. 4500 is the address, and it is what Memory[1200] holds.

✗ 200

Wrong. 200 is held at 6000.

✗ 400

Wrong. 400 is held at 8100.

| 999. It is the operand under indirect addressing, reached through Memory[1200] equal to 4500.

The revision exercise asks for the effective address and operand under which addressing modes?

- a. Immediate
- b. Direct
- c. Indirect
- d. Register
- e. Register Indirect using R1
- f. Relative
- g. Indexed

✓ Immediate

Correct, immediate is part i.

✓ Direct

Correct, direct is part ii.

✓ Indirect

Correct, indirect is part iii.

✓ Register

Correct, register is part iv.

✓ Register Indirect using R1

Correct, register indirect using R1 is part v.

✓ Relative

Correct, relative is part vi.

✗ Indexed

Wrong. Indexed addressing is set as an assignment in the ISA deck and does not appear here.

Six modes, numbered i to vi. Indexed addressing was left as an assignment for students in the ISA deck and is not part of this exercise.

Under immediate addressing with an address field of 1200, what are the effective address and operand?

- a. The operand is 1200 and no effective address is required
- b. The effective address is 1200 and the operand is 4500
- c. The effective address is 4500 and the operand is 999
- d. The operand is 1500 and no effective address is required

✓ The operand is 1200 and no effective address is required

Correct. Immediate takes the address field as the value itself, so the operand is 1200 and there is nothing to compute.

✗ The effective address is 1200 and the operand is 4500

Wrong. That is the direct answer.

✗ The effective address is 4500 and the operand is 999

Wrong. That is the indirect answer.

✗ The operand is 1500 and no effective address is required

Wrong. 1500 is the register addressing operand.

Immediate takes the address field as the value itself, so the operand is 1200 and there is nothing to compute.

7.44

MEDIUM

Rev S19

Under direct addressing with an address field of 1200, what is the operand?

- a. 4500
- b. 1200
- c. 999
- d. 6000

✓ 4500

Correct. EA is 1200 and the operand is the content of Memory[1200], which is 4500.

✗ 1200

Wrong. 1200 is the immediate answer.

✗ 999

Wrong. 999 is the indirect answer, one lookup further on.

✗ 6000

Wrong. 6000 is the register indirect answer.

EA is 1200 and the operand is the content of Memory[1200], which is 4500.

7.45

MEDIUM

Rev S19

Under direct addressing the effective address is _____ and the operand is the content of Memory[1200], which is _____ .

BLANK 1

- a. 1200
- b. 4500
- c. 1500
- d. 7000

BLANK 2

- a. 4500
- b. 1200
- c. 999
- d. 6000

Answer: **1200, 4500**

EA 1200, operand 4500. One lookup and you are done.

7.46

MEDIUM

Rev S19

Under indirect addressing the operand is the content of Memory[4500], which is ____ .

- a. 999
- b. 4500
- c. 400
- d. 200

Answer: **999**

999. Direct stops at 4500; indirect goes one hop further and finds 999.

7.47

MEDIUM

Rev S19

Under register addressing, supposing the operand is stored in R1, what is the operand?

- a. 1500
- b. 6000
- c. 1200
- d. 7000

✓ 1500

Correct. 1500, the content of R1 itself.

✗ 6000

Wrong. 6000 is what register indirect gives, after using 1500 as an address.

✗ 1200

Wrong. 1200 is the immediate answer.

✗ 7000

Wrong. 7000 is the PC.

1500, the content of R1 itself. No memory is touched at all.

7.48

MEDIUM

Rev S19

Under register addressing the operand is the contents of the register itself, so no memory access is needed.

Answer: **true**

That is why register addressing is fast, and why the revision answer is simply the value 1500 held in R1.

7.49

MEDIUM

Intro S36

Match each real-life application of addressing modes to the reason the deck gives.

Efficient program execution

Compilers choose addressing modes to optimize speed

Embedded systems

Efficient memory access is crucial in devices with limited memory

Gaming systems

Fast memory access improves frame rates and responsiveness

Three applications. All three come back to the same idea: the mode you choose decides how many memory accesses an instruction costs.

7.50

HARD

The real test 2, question 37

Given that the instruction specifies address = 700, $\text{Memory}[700] = 1500$, and $\text{Memory}[1500] = 60$. Under Indirect Addressing, the operand equals:

- a. 700
- b. 60
- c. Undefined
- d. 1500

✗ 700

Wrong. 700 is the address written in the instruction, which is only the start of the chain.

✓ 60

Correct. Follow both hops: 700 gives 1500, and 1500 gives 60.

✗ Undefined

Wrong. Every value needed is supplied, so nothing here is undefined.

✗ 1500

Wrong, and the commonest error. 1500 is the EFFECTIVE ADDRESS, one hop short of the operand. Under DIRECT addressing 1500 would have been the answer.

Indirect means two hops. The instruction gives 700, location 700 holds the real address 1500, and location 1500 holds the operand 60.

7.51

HARD

ISA S53

For ADD R1, (150) with R1 equal to 10, Memory[150] equal to 250 and Memory[250] equal to 5, what is R1 afterwards?

- a. 15
- b. 260
- c. 160
- d. 5

✓ 15

Correct. The effective address is 250, so the operand is 5, and 10 plus 5 is 15.

✗ 260

Wrong. That adds the effective address, 250, instead of the operand.

✗ 160

Wrong. That adds the address field, 150.

✗ 5

Wrong. That replaces R1 with the operand instead of adding to it.

| The effective address is 250, so the operand is 5, and 10 plus 5 is 15.

7.52**HARD**

ISA S53

In ADD R1, (150) the operand fetched is _____ , so R1 becomes 10 plus that, which is _____ .

BLANK 1

- a. 5
- b. 250
- c. 150
- d. 10

BLANK 2

- a. 15
- b. 260
- c. 160
- d. 5

Answer: **5, 15**

The operand is 5 and the result is 15. Two lookups before a single addition.

For STORE R3, (500) with R3 equal to 30 and Memory[500] equal to 800, what happens?

- a. Memory[800] becomes 30
- b. Memory[500] becomes 30
- c. R3 becomes 800
- d. Memory[800] becomes 500

✓ Memory[800] becomes 30

Correct. The effective address is 800, taken from Memory[500], and the store writes R3's value, 30, there.

✗ Memory[500] becomes 30

Wrong. That is what a direct STORE would do, and it would destroy the pointer.

✗ R3 becomes 800

Wrong. A store writes memory from a register; the register is unchanged.

✗ Memory[800] becomes 500

Wrong. The value written is R3's contents, 30.

| The effective address is 800, taken from Memory[500], and the store writes R3's value, 30, there.

7.54

HARD

ISA S55

In the step-by-step account of relative addressing, what is true of the PC when the offset is added?

- a. It already points to the next instruction
- b. It still points to the current instruction
- c. It has been cleared to zero
- d. It holds the operand

✓ It already points to the next instruction

Correct, and the deck states it explicitly as step two.

✗ It still points to the current instruction

Wrong. The fetch has already advanced it.

✗ It has been cleared to zero

Wrong. Nothing clears the PC.

✗ It holds the operand

Wrong. The PC holds an address, never an operand.

The PC has already been incremented during the fetch, so the offset is measured from the following instruction. That detail is what makes branch offsets come out right.

7.55

HARD

Rev S17

Match each memory address in the revision example to the value it holds.

7000	1200
1200	4500
1500	6000
3200	8100
4500	999
6000	200
8100	400

Seven entries. Notice that 8200 is not among them, which is what makes the relative addressing operand unknown.

Under indirect addressing with an address field of 1200, what are the effective address and operand?

- a. EA is 4500 and the operand is 999
- b. EA is 1200 and the operand is 4500
- c. EA is 999 and the operand is unknown
- d. EA is 1500 and the operand is 6000

✓ EA is 4500 and the operand is 999

Correct. Memory[1200] is 4500, so the effective address is 4500, and Memory[4500] is 999.

✗ EA is 1200 and the operand is 4500

Wrong. That is the direct answer, one lookup short.

✗ EA is 999 and the operand is unknown

Wrong. 999 is the operand, not the effective address.

✗ EA is 1500 and the operand is 6000

Wrong. That is the register indirect answer.

Memory[1200] is 4500, so the effective address is 4500, and Memory[4500] is 999.

Under register indirect addressing using R1, what are the effective address and operand?

- a. EA is 1500 and the operand is 6000
- b. EA is 6000 and the operand is 200
- c. The operand is 1500 and no EA is required
- d. EA is 4500 and the operand is 999

✓ EA is 1500 and the operand is 6000

Correct. R1 holds 1500, which becomes the effective address, and Memory[1500] is 6000.

✗ EA is 6000 and the operand is 200

Wrong, and it takes one hop too many; 6000 is the operand, and 200 is what lies beyond it.

✗ The operand is 1500 and no EA is required

Wrong. That is plain register addressing.

✗ EA is 4500 and the operand is 999

Wrong. That is memory indirect through 1200.

| R1 holds 1500, which becomes the effective address, and Memory[1500] is 6000.

7.58

HARD

Rev S20

Under register indirect addressing the effective address is R1, which is _____, and the operand is the content of Memory[1500], which is _____.

BLANK 1

- a. 1500
- b. 6000
- c. 1200
- d. 200

BLANK 2

- a. 6000
- b. 1500
- c. 200
- d. 999

Answer: **1500, 6000**

EA 1500, operand 6000. Register addressing and register indirect differ by exactly one memory access.

7.59

HARD

Rev S20

Under relative addressing with PC equal to 7000 and an address field of 1200, what is the effective address?

- a. 8200
- b. 5800
- c. 1200
- d. 7000

✓ 8200

Correct. 7000 plus 1200 is 8200.

✗ 5800

Wrong. The offset is positive here, so it is added.

✗ 1200

Wrong. That would be direct addressing.

✗ 7000

Wrong. That is the PC alone.

7000 plus 1200 is 8200. The address field is treated as an offset from the program counter.

Under relative addressing the effective address works out as 8200. What is the operand?

- a. Not specified or unknown, because 8200 does not appear in the memory table
- b. 400, because 8100 is the nearest listed address
- c. 0, because unlisted memory is empty
- d. 8200, because the effective address is the operand

✓ Not specified or unknown, because 8200 does not appear in the memory table

Correct, and the deck says so in exactly those words.

✗ 400, because 8100 is the nearest listed address

Wrong. 8100 is a different address; addresses are not rounded to the nearest listed one.

✗ 0, because unlisted memory is empty

Wrong. Nothing in the question says unlisted locations hold zero.

✗ 8200, because the effective address is the operand

Wrong. The effective address is where the operand lives, not the operand.

The memory table lists 7000, 1200, 1500, 3200, 4500, 6000 and 8100, but not 8200, so the revision answer says the operand is not specified or unknown.

7.61

HARD

Rev S20

Under relative addressing the effective address is 7000 plus 1200, which is _____, and the operand is _____.

BLANK 1

- a. 8200
- b. 8100
- c. 5800
- d. 1200

BLANK 2

- a. not specified or unknown
- b. 400
- c. 999
- d. 200

Answer: **8200, not specified or unknown**

8200, and the operand is not specified or unknown. Do not be pulled towards 8100, which is listed and holds 400: it is a different address.

8 Memory, Buses and CPU Components

8.1

EASY

The real test 1, question 4

A processor's control unit is responsible for:

- a. Performing arithmetic calculations
- b. Storing application files
- c. Coordinating instruction execution through control signals
- d. Managing peripheral power supply

X Performing arithmetic calculations

Wrong. Arithmetic is the ALU's job. Control unit and ALU are the two halves of the CPU, and the exam separates them constantly.

X Storing application files

Wrong. Files are held in secondary storage, managed by the operating system.

✓ Coordinating instruction execution through control signals

Correct. Decode, then sequence the rest of the machine through control signals.

X Managing peripheral power supply

Wrong. Power management belongs to the power supply and chipset, not to the instruction control path.

The control unit is the conductor. It decodes each instruction and issues the control signals that tell every other unit what to do and when.

A software engineer optimizes a program by storing frequently used variables in CPU registers instead of RAM. The primary benefit is:

- a. Improved memory capacity
- b. Lower bus traffic only
- c. Reduced storage cost
- d. Faster execution speed

✗ Improved memory capacity

Wrong, and backwards. There are only a few dozen registers, so capacity falls rather than rises.

✗ Lower bus traffic only

Wrong because of the word ONLY. Less bus traffic is a real side effect, but the PRIMARY benefit is speed.

✗ Reduced storage cost

Wrong. Registers are the most expensive storage per bit in the whole machine.

✓ Faster execution speed

Correct. The variable is already where the ALU needs it, so no memory trip is made.

Registers sit inside the CPU and are reached in a single cycle, while RAM costs tens or hundreds of cycles.

Which bus carries information identifying memory locations and I/O device locations?

- a. Control Bus
- b. Data Bus
- c. Memory Bus
- d. Address Bus

✗ Control Bus

Wrong. The control bus carries timing and command signals such as read and write.

✗ Data Bus

Wrong. The data bus carries the value itself, once the location has been named.

✗ Memory Bus

Wrong. "Memory bus" is a loose collective term for the whole path, not the line that carries locations.

✓ Address Bus

Correct. Location information travels on the address bus, and its width sets how much memory can be addressed.

Three buses, three jobs: the address bus says WHERE, the data bus carries WHAT, and the control bus says WHEN and WHICH WAY.

A motherboard manufacturer introduces a faster bus system, enabling quicker communication among components. This enhancement belongs to:

- a. Computer Architecture
- b. Instruction Design
- c. Data Representation
- d. Computer Organization

X Computer Architecture

Wrong. Architecture would mean a change to the instruction set or the programmer-visible model, and there is none here.

X Instruction Design

Wrong. No instruction was added, removed or re-encoded.

X Data Representation

Wrong. Data representation is about encoding numbers and characters, untouched by bus speed.

✓ Computer Organization

Correct. Interconnection and speed are organization.

A faster bus changes how components are interconnected and how quickly they talk. No instruction changes, so nothing programmer-visible changes.

Which statement regarding ROM is correct?

- a. ROM is mainly used for temporary program execution.
- b. ROM stores firmware permanently.
- c. ROM requires periodic refreshing.
- d. ROM loses contents after power failure.

X ROM is mainly used for temporary program execution.

Wrong. Temporary working storage is RAM's job. ROM holds code that must persist.

✓ ROM stores firmware permanently.

Correct. Firmware such as the BIOS is held permanently in ROM.

X ROM requires periodic refreshing.

Wrong. Refreshing is a DRAM requirement, because DRAM stores charge in capacitors that leak.

X ROM loses contents after power failure.

Wrong, and the direct opposite. ROM is non-volatile, so its contents survive power loss.

ROM is read-only and non-volatile, which is why the boot firmware lives there and survives a power cut.

Which of the following correctly orders memory from fastest to slowest?

- a. Cache → Registers → SSD → RAM
- b. Registers → Cache → Main Memory → Secondary Storage
- c. Registers → Main Memory → Cache → SSD
- d. Cache → Registers → Main Memory → Hard Disk

✗ Cache → Registers → SSD → RAM

Wrong twice. Registers must come before cache, and RAM is faster than an SSD, not slower.

✓ Registers → Cache → Main Memory → Secondary Storage

Correct. The standard hierarchy, top to bottom.

✗ Registers → Main Memory → Cache → SSD

Wrong. Cache sits between registers and main memory, not below main memory.

✗ Cache → Registers → Main Memory → Hard Disk

Wrong. The last two are in order, but registers are faster than cache, so the first pair is swapped.

Registers, cache, main memory, secondary storage. Learn this ladder once and it answers a whole family of questions: speed and cost per bit fall together as capacity rises.

Which combination correctly matches memory type with volatility?

- a. Flash Memory, Non-volatile
- b. DRAM, Non-volatile
- c. SRAM, Non-volatile
- d. Cache Memory, Non-volatile

✓ Flash Memory, Non-volatile

Correct. Flash holds charge on a floating gate, so a USB drive still has your files tomorrow.

✗ DRAM, Non-volatile

Wrong. DRAM is volatile, and it does not even hold data for a full second without refreshing.

✗ SRAM, Non-volatile

Wrong. SRAM is volatile too. It needs no refresh, but it does need power.

✗ Cache Memory, Non-volatile

Wrong. Cache is built from SRAM, so it is volatile as well.

Only flash keeps its contents without power. DRAM, SRAM and the cache built from SRAM are all volatile.

Which component stores the address of the memory location currently being accessed?

- a. Memory Address Register
- b. Memory Data Register
- c. Program Counter
- d. Instruction Register

✓ Memory Address Register

Correct. The Memory Address Register drives the address bus.

✗ Memory Data Register

Wrong. The MDR holds the DATA travelling to or from that address.

✗ Program Counter

Wrong, and the near miss. The Program Counter holds the address of the NEXT INSTRUCTION, not of the location being accessed right now.

✗ Instruction Register

Wrong. The Instruction Register holds the instruction currently being decoded and executed.

MAR holds WHERE, MDR holds WHAT. The pair is worth memorising together, because the paper sets both.

A memory technology provides higher density but lower speed than SRAM. Which technology is being described?

- a. ROM
- b. EEPROM
- c. Register memory
- d. DRAM

✗ ROM

Wrong. ROM is read-only, so it is not the read-write main memory the comparison implies.

✗ EEPROM

Wrong. EEPROM is non-volatile and much slower to write, used for firmware rather than main memory.

✗ Register memory

Wrong. Registers are the fastest and least dense storage in the machine, the opposite of this description.

✓ DRAM

Correct. Denser and slower than SRAM, which is why main memory is DRAM and cache is SRAM.

One transistor and one capacitor per bit against SRAM's six transistors. That is why DRAM packs more bits per chip and runs slower.

Which memory characteristic primarily determines how much information can be stored in a memory device?

- a. Access time
- b. Capacity
- c. Transfer rate
- d. Volatility

✗ Access time

Wrong. Access time is the delay before data arrives, a speed measure.

✓ Capacity

Correct. Capacity is the total amount that can be held.

✗ Transfer rate

Wrong. Transfer rate is bits per second, another speed measure.

✗ Volatility

Wrong. Volatility says whether contents survive without power, not how much fits.

Capacity is HOW MUCH. Access time and transfer rate are HOW FAST, and volatility is WHETHER IT SURVIVES power loss.

8.11

EASY

Mem S2

Which access method reads locations one after the other in sequence, and what is its example?

- a. Sequential, and its example is magnetic tape
- b. Random, and its example is RAM
- c. Direct, and its example is the hard disk drive
- d. Associative, and its example is cache

✓ Sequential, and its example is magnetic tape

Correct. Sequential access reads A, B, C, D, E one after the other, and the deck's example is magnetic tape.

✗ Random, and its example is RAM

Wrong. Random access reaches all locations in constant time.

✗ Direct, and its example is the hard disk drive

Wrong. Direct access finds the vicinity or block first.

✗ Associative, and its example is cache

Wrong. Associative access holds address and content together.

Sequential access reads A, B, C, D, E one after the other, and the deck's example is magnetic tape.

8.12

EASY

Mem S2

The deck's example of a sequential access device is magnetic ____ .

- a. tape
- b. disk
- c. drum
- d. core

Answer: **tape**

Tape. You cannot reach the middle of a tape without winding through everything before it, which is what sequential means.

8.13

EASY

Mem S2

Random access means that:

- a. All locations can be accessed in constant time
- b. Locations are accessed in an unpredictable order
- c. The address and the content are stored together
- d. The vicinity is located first, then the exact item

✓ All locations can be accessed in constant time

Correct. Constant time for every location, which is why RAM is its example.

✗ Locations are accessed in an unpredictable order

Wrong, and it is the everyday sense of random rather than the technical one.

✗ The address and the content are stored together

Wrong. That is associative access.

✗ The vicinity is located first, then the exact item

Wrong. That is direct access.

Constant time for every location, which is why RAM is its example. Random here means uniform, not unpredictable.

8.14

EASY

Mem S2

Under random access all locations can be accessed at constant time, and the deck's example is ____ .

- a. RAM
- b. ROM
- c. HDD
- d. cache

Answer: **RAM**

RAM, and the name says it: Random Access Memory.

8.15

EASY

Mem S2

Computer memory exhibits _____ characteristics: location, access methods, performance and capacity.

- a. 4
- b. 3
- c. 5
- d. 6

Answer: **4**

Four: location, access methods, performance and capacity or size.

8.16

EASY

Mem S4

Memory hierarchy is defined as a structured arrangement of different storage types designed to optimize:

- a. Performance and cost
- b. Capacity alone
- c. Reliability and security
- d. Power consumption

✓ Performance and cost

Correct. Performance and cost together.

✗ Capacity alone

Wrong. Capacity is one of the three questions, but the aim is the balance.

✗ Reliability and security

Wrong. Reliability and security are not discussed here.

✗ Power consumption

Wrong. Power is not one of the design constraints given.

Performance and cost together. Neither alone would need a hierarchy: one fast memory or one cheap one would do.

8.17

EASY

Mem S4

Memory hierarchy is a _____ arrangement of different storage types in a computer system.

- a. structured
- b. random
- c. flat
- d. temporary

Answer: **structured**

Structured. The structure is the ordering by speed, cost and capacity that the hierarchy table sets out.

8.18

EASY

Mem S4

A memory with a smaller cost per bit is generally slower in terms of access.

Answer: **true**

Cheap and slow travel together, which is why bulk storage sits at the bottom of the hierarchy.

8.19

EASY

Mem S5

Design constraints on memory can be summed up by _____ questions.

- a. 3
- b. 2
- c. 4
- d. 5

Answer: **3**

Three: how much space, how fast, how expensive.

8.20

EASY

Mem S5

The deck argues that if memory exists, applications will likely be developed to use it.

Answer: **true**

It does, as its answer to the how much space question.

8.21

EASY

Mem S5

The cost of memory must be reasonable in relationship to other components.

Answer: **true**

That is the deck's answer to the third design question. Memory competes for the same budget as everything else in the machine.

8.22

EASY

Mem S8

In the hierarchy table, registers are described as:

- a. Fastest, highest in cost, smallest in capacity
- b. Fastest, lowest in cost, smallest in capacity
- c. Slowest, highest in cost, largest in capacity
- d. Very fast, high in cost, small in capacity

✓ Fastest, highest in cost, smallest in capacity

Correct. Fastest, highest, smallest.

✗ Fastest, lowest in cost, smallest in capacity

Wrong. Registers are the highest cost, not the lowest.

✗ Slowest, highest in cost, largest in capacity

Wrong. That describes the bottom of the table, except for the cost.

✗ Very fast, high in cost, small in capacity

Wrong, and this is the cache row, one level below registers.

Fastest, highest, smallest. Note the superlatives: cache is very fast, high and small, which is the next row down.

The in-class question in the memory deck asks:

- a. Why do we not use only cache memory in a computer system?
- b. Why were CISC architectures initially popular?
- c. Why is pipelining more difficult in CISC processors?
- d. What is the primary focus of computer architecture?

✓ Why do we not use only cache memory in a computer system?

Correct. That is the memory deck's in-class question, and its answer slide follows immediately.

✗ Why were CISC architectures initially popular?

Wrong. That question belongs to the RISC and CISC deck.

✗ Why is pipelining more difficult in CISC processors?

Wrong. That question also belongs to the RISC and CISC deck.

✗ What is the primary focus of computer architecture?

Wrong. That question belongs to the introduction deck.

| That is the memory deck's in-class question, and its answer slide follows immediately.

Why do we not use only cache memory in a computer system?

- a. Because cache is very expensive and has limited capacity
- b. Because cache is volatile and main memory is not
- c. Because cache cannot be addressed by the processor
- d. Because cache is slower than main memory for large transfers

✓ Because cache is very expensive and has limited capacity

Correct. Very expensive and limited in capacity.

✗ Because cache is volatile and main memory is not

Wrong. Both cache and main memory are volatile.

✗ Because cache cannot be addressed by the processor

Wrong. The cache is not usually visible to the processor, but that is not the reason given.

✗ Because cache is slower than main memory for large transfers

Wrong. Cache is faster than main memory, which is the whole point of it.

Very expensive and limited in capacity. Both halves matter, and both come straight from the trade-off slide.

8.25

EASY

Mem S10

We do not use only cache because cache is very _____ and has _____ capacity.

BLANK 1

- a. expensive
- b. slow
- c. volatile
- d. large

BLANK 2

- a. limited
- b. unlimited
- c. variable
- d. shared

Answer: **expensive, limited**

Expensive and limited. The two reasons are the first two columns of the hierarchy table, read off the cache row.

8.26

EASY

Mem S12

Match each kind of locality to what it predicts.

Space, that is spatial

Close by addresses will very likely be accessed

Time, that is temporal

We will very likely access the same location again

Space against time. Both tests have asked about this pair, so the two words are worth being certain of.

8.27

EASY

Mem S13

Because of iterative loops and subroutines, instructions and data will be accessed _____ times.

- a. multiple
- b. single
- c. random
- d. fewer

Answer: **multiple**

Multiple times, which is exactly what makes caching worthwhile.

8.28

EASY

Mem S19

Processor registers are described as:

- a. The fastest, smallest and most expensive type of memory
- b. The fastest, largest and cheapest type of memory
- c. Slower than cache but larger
- d. The principal internal memory system of the computer

✓ The fastest, smallest and most expensive type of memory

Correct. Fastest, smallest, most expensive, all three at once.

✗ The fastest, largest and cheapest type of memory

Wrong on two counts: registers are the smallest and the dearest.

✗ Slower than cache but larger

Wrong. Registers sit above cache, so they are faster and smaller.

✗ The principal internal memory system of the computer

Wrong. That phrase describes main memory.

Fastest, smallest, most expensive, all three at once. That is the register row of the hierarchy table in a sentence.

8.29

EASY

Mem S19

Processor registers are the fastest, smallest and most _____ type of memory.

- a. expensive
- b. volatile
- c. reliable
- d. durable

Answer: **expensive**

Most expensive, per bit. That is why there are so few of them.

8.30

EASY

Mem S19

Main memory is described as:

- a. The principal internal memory system of the computer, each location having a unique address
- b. The fastest and smallest memory in the machine
- c. A copy of portions of the cache
- d. Memory that survives a power cut

✓ The principal internal memory system of the computer, each location having a unique address

Correct. Principal internal memory, with a unique address per location.

✗ The fastest and smallest memory in the machine

Wrong. Those are registers.

✗ A copy of portions of the cache

Wrong, and reversed: the cache contains a copy of portions of main memory.

✗ Memory that survives a power cut

Wrong. Main memory is volatile; ROM and secondary storage are not.

Principal internal memory, with a unique address per location. That uniqueness is what makes an address field meaningful in the first place.

8.31

EASY

Mem S19

Main memory is the principal internal memory system of the computer, and each location has a _____ address.

- a. unique
- b. shared
- c. virtual
- d. relative

Answer: **unique**

Unique. In the cache organization slides this becomes each word has a unique n-bit address.

8.32

EASY

Mem S20

The cache contains a copy of portions of main memory.

Answer: **true**

A copy of portions, not of the whole thing. That is why a tag is needed: the cache has to record which portions it is currently holding.

8.33

EASY

Mem S21

How many bits are in a nibble and in a byte?

- a. 4 and 8
- b. 8 and 16
- c. 2 and 8
- d. 4 and 16

✓ 4 and 8

Correct. A nibble is 4 bits and a byte is 8.

✗ 8 and 16

Wrong. A byte is 8 bits, not 16.

✗ 2 and 8

Wrong. A nibble is 4 bits.

✗ 4 and 16

Wrong on the byte.

A nibble is 4 bits and a byte is 8. A nibble is also exactly one hexadecimal digit, which is why hex groups bits in fours.

8.34

EASY

Mem S21

A nibble is _____ bits.

- a. 4
- b. 8
- c. 2
- d. 16

Answer: **4**

4 bits, half a byte, and the origin of the name.

8.35

EASY

Mem S22

When the processor attempts to read a word and the word is in the cache, this is called a cache hit. What happens?

- a. The word is delivered to the processor
- b. The block containing it is read into the cache first
- c. The address is loaded onto the system bus
- d. The cache line is updated with a new tag

✓ The word is delivered to the processor

Correct. Straight delivery, with no main memory traffic at all.

✗ The block containing it is read into the cache first

Wrong. That is the miss path.

✗ The address is loaded onto the system bus

Wrong. On a hit the data and address buffers are disabled and there is no bus traffic.

✗ The cache line is updated with a new tag

Wrong. Updating the line with a new tag happens on a miss.

Straight delivery, with no main memory traffic at all. The other three options all describe what happens on a miss.

8.36

EASY

Mem S22

When the processor attempts to read a word of memory, a check is made to determine if the word is in the ____ .

- a. cache
- b. register
- c. disk
- d. buffer

Answer: **cache**

The cache. That check happens on every read, which is why the deck later notes it is being ignored for simplicity in the two-level calculation.

8.37

EASY

Mem S22

The justification for reading a whole block into the cache is the locality of reference principle, in both its time and space forms.

Answer: **true**

The deck cites locality of reference, time and space, at exactly this point. Space justifies fetching the neighbours, time justifies keeping them.

8.38

EASY

Mem S26

If a word in a block of memory is read, the whole _____ is transferred to a cache line.

- a. block
- b. word
- c. page
- d. line

Answer: **block**

Block. Main memory is divided into blocks and the cache into lines, and a block occupies a line.

8.39

EASY

Mem S28

The processor generates the read address, abbreviated _____, of the word to be read.

- a. RA
- b. MAR
- c. PC
- d. EA

Answer: **RA**

RA. Do not confuse it with EA, the effective address of the addressing modes topic, or with MAR, the register that holds an address.

8.40

EASY

Mem S34

Match each type of cache address to what it is.

Physical addresses

Actual memory addresses

Logical addresses

Virtual-memory addresses

Physical means real, logical means virtual. The mapping between them is what virtual memory performs.

8.41

EASY

Intro S37

The introduction deck names which two types of register?

- a. General-purpose registers and special-purpose registers such as the Program Counter and Stack Pointer
- b. Cache registers and memory registers
- c. Input registers and output registers
- d. Physical registers and logical registers

✓ General-purpose registers and special-purpose registers such as the Program Counter and Stack Pointer

Correct. General-purpose and special-purpose, with the Program Counter and the Stack Pointer as the two named special ones.

✗ Cache registers and memory registers

Wrong. The cache is a level of memory, not a class of register.

✗ Input registers and output registers

Wrong. Input and output are handled by I/O systems.

✗ Physical registers and logical registers

Wrong. Physical and logical describe addresses, not registers.

General-purpose and special-purpose, with the Program Counter and the Stack Pointer as the two named special ones.

8.42

EASY

Intro S37

Registers are small, fast storage locations _____ the CPU.

- a. inside
- b. beside
- c. outside
- d. below

Answer: **inside**

Inside the CPU. That word is what separates a register from the cache, which is close to the processor but not within it.

8.43

EASY

Intro S46

Computer organization deals with:

- a. How architectural features are physically implemented
- b. Which instructions the processor supports
- c. How data types are represented
- d. Which addressing modes are available

✓ How architectural features are physically implemented

Correct. The physical implementation of architectural features.

✗ Which instructions the processor supports

Wrong. The instruction set is architectural.

✗ How data types are represented

Wrong. Data types and representation is architectural.

✗ Which addressing modes are available

Wrong. Addressing modes are architectural.

The physical implementation of architectural features. The other three options are all architectural components.

8.44

EASY

Intro S46

The first component of computer organization listed in the introduction deck is the _____ unit.

- a. control
- b. arithmetic logic
- c. memory
- d. input/output

Answer: **control**

The control unit, and the ALU follows it. The revision class lists the same six components in the same order.

8.45

EASY

Intro S46

Which component of the CPU coordinates all its operations?

- a. The control unit
- b. The arithmetic logic unit
- c. The register set
- d. The system bus

✓ The control unit

Correct. The control unit, by fetching, decoding and generating control signals.

✗ The arithmetic logic unit

Wrong. The ALU computes; it does not coordinate.

✗ The register set

Wrong. Registers hold values.

✗ The system bus

Wrong. Buses carry data, addresses and control signals, but the signals come from the control unit.

The control unit, by fetching, decoding and generating control signals. Coordination is the word to look for.

Select the operations the Arithmetic Logic Unit performs, according to the deck.

- a. Addition
- b. Subtraction
- c. AND
- d. OR
- e. NOT
- f. Instruction fetch

✓ Addition

Correct, addition is given as an arithmetic operation.

✓ Subtraction

Correct, subtraction is the other.

✓ AND

Correct, AND is a logical operation.

✓ OR

Correct, OR is a logical operation.

✓ NOT

Correct, NOT closes the logical list here, though the ISA deck names exclusive-OR instead.

✗ Instruction fetch

Wrong. Fetching instructions is done by the control unit.

Two arithmetic operations and three logical ones. Fetching is the control unit's job.

The ALU performs _____ operations such as addition and subtraction, and _____ operations such as AND, OR and NOT.

BLANK 1

- a. arithmetic
- b. logical
- c. control
- d. transfer

BLANK 2

- a. logical
- b. arithmetic
- c. branch
- d. memory

Answer: **arithmetic, logical**

Arithmetic and logical, which is where the unit's name comes from.

Which application does the deck list under the ALU, saying its neural network computations involve massive arithmetic operations?

- a. Artificial intelligence systems
- b. Robotics systems
- c. Cloud systems
- d. Smart homes

✓ Artificial intelligence systems

Correct. Artificial intelligence systems, alongside digital calculators and gaming systems.

✗ Robotics systems

Wrong. Robotics is listed under the control unit.

✗ Cloud systems

Wrong. Cloud systems are listed under memory organization.

✗ Smart homes

Wrong. Smart homes are listed under I/O systems.

Artificial intelligence systems, alongside digital calculators and gaming systems.

8.49

EASY

Intro S49

In the introduction deck's memory organization component, which level is described as fastest?

- a. Cache
- b. Main memory, RAM
- c. Secondary storage
- d. Tertiary storage

✓ Cache

Correct on this slide.

✗ Main memory, RAM

Wrong. Main memory is the middle of the three named here.

✗ Secondary storage

Wrong. Secondary storage is the slowest of the three.

✗ Tertiary storage

Wrong. Tertiary storage is not on this slide at all.

Cache is marked fastest on this slide, which lists cache, main memory and secondary storage. The memory deck's fuller table puts registers above cache.

8.50

EASY

Intro S49

The introduction deck says memory is organized ____ : cache, main memory, secondary storage.

- a. hierarchically
- b. sequentially
- c. randomly
- d. associatively

Answer: **hierarchically**

Hierarchically, which is the idea the memory deck then develops into a five-level table.

Which application does the deck attach to cache memory speeding up page loading?

- a. Web browsers
- b. Smartphones
- c. Cloud systems
- d. Gaming computers

✓ Web browsers

Correct. Web browsers.

✗ Smartphones

Wrong. Smartphones are named for faster app performance.

✗ Cloud systems

Wrong. Cloud systems are named for supporting large-scale computing.

✗ Gaming computers

Wrong. Gaming computers appear under system buses, for graphics performance.

Web browsers. Smartphones get the app performance reason and cloud systems get large-scale computing.

8.52

EASY

Intro S50

Input/output systems handle:

- a. Communication between the computer and external devices
- b. Communication between the ALU and the registers
- c. Translation of logical to physical addresses
- d. The overlapping of instruction execution

✓ Communication between the computer and external devices

Correct. The computer and the outside world.

✗ Communication between the ALU and the registers

Wrong. That traffic is internal to the processor.

✗ Translation of logical to physical addresses

Wrong. Address translation is virtual memory's job, through the MMU.

✗ The overlapping of instruction execution

Wrong. That is pipelining.

| The computer and the outside world. Everything internal belongs to some other component.

8.53

EASY

Intro S50

Input/output systems handle communication between the computer and _____ devices.

- a. external
- b. internal
- c. virtual
- d. logical

Answer: **external**

| External devices, which is why keyboards, monitors and sensors are its examples.

8.54

EASY

Intro S51

Select the three types of bus the deck names.

- a. Data bus
- b. Address bus
- c. Control bus
- d. Instruction bus

✓ Data bus

Correct, the data bus carries the data.

✓ Address bus

Correct, the address bus carries the addresses.

✓ Control bus

Correct, the control bus carries the control signals.

✗ Instruction bus

Wrong. There is no instruction bus here; instructions travel on the data bus. A separate instruction path is the Harvard architecture, a different idea.

Data, address, control. They match exactly what buses transfer: data, addresses and control signals.

8.55

EASY

Intro S51

Buses transfer data, _____ and _____ signals.

BLANK 1

- a. addresses
- b. instructions
- c. operands
- d. interrupts

BLANK 2

- a. control
- b. clock
- c. timing
- d. status

Answer: **addresses, control**

Data, addresses, control signals, which is why there are exactly three buses.

8.56

EASY

Intro S52

In the real-life applications of system buses, which system does the deck say uses high-speed buses to improve graphics performance?

- a. Gaming computers
- b. Servers
- c. Smartphones
- d. Medical devices

✓ Gaming computers

Correct. Gaming computers.

✗ Servers

Wrong. Servers are named for supporting multiple users.

✗ Smartphones

Wrong. Smartphones appear under memory organization.

✗ Medical devices

Wrong. Medical devices appear under I/O systems.

Gaming computers. Servers get efficient data transfer supporting multiple users, and motherboard communication heads the list.

8.57

EASY

Intro S53

Pipelining is the _____ of instruction execution.

- a. overlapping
- b. repeating
- c. reordering
- d. duplicating

Answer: **overlapping**

Overlapping, which is the same word the pipelining deck uses throughout.

An autonomous robot continuously performs sensor processing and actuator control. Which organizational component primarily coordinates these activities?

- a. Control Unit
- b. ALU
- c. Register Set
- d. Cache Memory

✓ Control Unit

Correct. Coordination and sequencing is the control unit's defining role.

✗ ALU

Wrong. The ALU computes results when told to, it does not decide the order of work.

✗ Register Set

Wrong. Registers hold operands and results. Storage is not coordination.

✗ Cache Memory

Wrong. Cache speeds up access to recently used data, which is a performance aid and not a controller.

The keyword is COORDINATES. Whenever a stem asks which component sequences or directs activity, the answer is the control unit.

Why is cache memory generally constructed using SRAM rather than DRAM?

- a. SRAM offers faster access without refresh operations.
- b. SRAM is cheaper.
- c. SRAM is non-volatile.
- d. SRAM has larger storage capacity.

✓ SRAM offers faster access without refresh operations.

Correct. Speed with no refresh cycles stealing access time.

✗ SRAM is cheaper.

Wrong, and backwards. SRAM uses about six transistors per bit against DRAM's one transistor and one capacitor, so it costs far more.

✗ SRAM is non-volatile.

Wrong. SRAM is volatile, like DRAM. It loses everything when power goes.

✗ SRAM has larger storage capacity.

Wrong. DRAM is the denser technology, which is exactly why main memory is DRAM.

SRAM holds each bit in a flip-flop, so it is fast and needs no refresh. Cache exists to be fast, so it pays SRAM's price.

Which architectural feature most directly contributes to reducing the average instruction execution time without increasing clock frequency?

- a. Increasing monitor resolution
- b. Increasing hard disk capacity
- c. Expanding ROM size
- d. Increasing cache hit rate

✗ Increasing monitor resolution

Wrong. Monitor resolution is a display property and does not touch instruction timing.

✗ Increasing hard disk capacity

Wrong. Disk capacity is about how much you can store, not how fast instructions execute.

✗ Expanding ROM size

Wrong. A bigger ROM holds more firmware. It does not speed up running code.

✓ Increasing cache hit rate

Correct. Fewer misses means fewer long trips to main memory.

Average access time is hit time plus miss rate times miss penalty. Raise the hit rate and the average falls, with the clock untouched.

Which statement correctly explains memory hierarchy?

- a. Main memory is faster than registers.
- b. Every memory level has identical speed.
- c. Higher levels are faster, smaller, and more expensive per bit.
- d. Lower levels are faster than upper levels.

X Main memory is faster than registers.

Wrong, and inverted. Registers are the fastest storage in the machine.

X Every memory level has identical speed.

Wrong. If every level were equally fast there would be no reason to have a hierarchy at all.

✓ Higher levels are faster, smaller, and more expensive per bit.

Correct, and all three properties are stated together.

X Lower levels are faster than upper levels.

Wrong, and inverted again. Lower means larger, cheaper and slower.

Climbing the hierarchy, speed rises, size falls and cost per bit rises. All three move together, which is why the pyramid has its shape.

Which memory characteristic is most critical for real-time control systems?

- a. Low manufacturing cost
- b. Predictable low access time
- c. Storage capacity
- d. High density

X Low manufacturing cost

Wrong. Cost is a business constraint, not a timing guarantee.

✓ Predictable low access time

Correct. Note the word PREDICTABLE. This is why some real-time designs prefer tightly coupled memory over an unpredictable cache.

X Storage capacity

Wrong. A control system's working set is usually small. Capacity is not the binding constraint.

X High density

Wrong. Density is a manufacturing property, and it says nothing about worst-case timing.

Real-time means a deadline must be met EVERY time. Predictability matters as much as raw speed, since a fast average with an occasional long stall still misses deadlines.

Why is secondary storage unsuitable for direct CPU execution of instructions?

- a. It stores binary data.
- b. It has significantly higher access latency than primary memory.
- c. It is non-volatile.
- d. It uses logical addressing.

✗ It stores binary data.

Wrong. Every level of the hierarchy stores binary data, so this cannot be the distinguishing reason.

✓ It has significantly higher access latency than primary memory.

Correct. The latency gap is the whole reason for loading before executing.

✗ It is non-volatile.

Wrong. Non-volatility is an ADVANTAGE of secondary storage, not an obstacle to execution.

✗ It uses logical addressing.

Wrong. Logical addressing is a translation scheme, and it does not prevent execution.

Disk latency is measured in milliseconds against nanoseconds for RAM, a gap of about a million to one. That is why programs are loaded into main memory before they run.

Which statement best summarizes the fundamental trade-off underlying modern computer memory hierarchy?

- a. All memory technologies have similar speed and cost.
- b. Slower memories always consume more power than faster memories.
- c. Faster memories generally provide larger capacities at lower cost.
- d. Faster memories are typically smaller and more expensive per bit, while slower memories provide larger capacities at lower cost.

✗ All memory technologies have similar speed and cost.

Wrong. If speed and cost were similar everywhere, one flat memory would do and there would be no hierarchy.

✗ Slower memories always consume more power than faster memories.

Wrong. "Always" overstates it, and slower technologies such as disk and DRAM are generally cheaper per bit in power terms too.

✗ Faster memories generally provide larger capacities at lower cost.

Wrong, and exactly inverted. Fast memory is small and expensive.

✓ Faster memories are typically smaller and more expensive per bit, while slower memories provide larger capacities at lower cost.

Correct, and it states both halves of the trade-off.

You cannot have fast, large and cheap at once. The hierarchy exists to buy the ILLUSION of all three by keeping hot data high and cold data low.

Which statement best distinguishes SRAM from DRAM?

- a. SRAM stores data using flip-flops whereas DRAM stores data using capacitors.
- b. DRAM is faster than SRAM.
- c. SRAM requires periodic refreshing while DRAM does not.
- d. DRAM is non-volatile.

✓ SRAM stores data using flip-flops whereas DRAM stores data using capacitors.

Correct, and it is the root fact from which the rest follows.

✗ DRAM is faster than SRAM.

Wrong, and inverted. SRAM is the faster of the two, which is why cache uses it.

✗ SRAM requires periodic refreshing while DRAM does not.

Wrong, and inverted. DRAM is the one that must be refreshed, and the D stands for dynamic.

✗ DRAM is non-volatile.

Wrong. DRAM is volatile, and it loses data in milliseconds without refreshing.

One structural fact explains everything else: flip-flops hold their state, so SRAM is fast and needs no refresh but costs many transistors. Capacitors leak, so DRAM must be refreshed but packs far more bits per chip.

A processor repeatedly accesses the same instruction during program execution. The improved execution speed primarily results from which memory hierarchy principle?

- a. Spatial locality only
- b. Temporal locality
- c. Virtual addressing
- d. Sequential consistency

✗ Spatial locality only

Wrong. Spatial locality concerns neighbouring addresses, and the word ONLY makes it worse.

✓ Temporal locality

Correct. The same item reused shortly after, which is a cache hit.

✗ Virtual addressing

Wrong. Virtual addressing is an address translation mechanism, not a locality principle.

✗ Sequential consistency

Wrong. Sequential consistency is a memory ordering model for multiprocessors.

Temporal is about TIME: the SAME location used again soon. Spatial is about PLACE: NEARBY locations used next. The word "repeatedly" settles it.

Which memory level has the highest cost per bit?

- a. SSD
- b. Registers
- c. Main memory
- d. Hard disk

✗ SSD

Wrong. An SSD is near the bottom, cheap per bit and large.

✓ Registers

Correct. Highest in the hierarchy, therefore highest cost per bit.

✗ Main memory

Wrong. Main memory is DRAM, far cheaper per bit than the SRAM and logic above it.

✗ Hard disk

Wrong. Hard disk is the cheapest per bit of the four.

Cost per bit rises as you climb the hierarchy, so the top level is the dearest. Registers are built from the fastest logic and sit inside the processor die.

Which statement correctly describes virtual memory?

- a. It replaces cache memory.
- b. It physically increases RAM capacity.
- c. It eliminates page faults.
- d. It allows programs to execute using an address space larger than physical memory.

✗ It replaces cache memory.

Wrong. Cache and virtual memory sit at different levels and coexist.

✗ It physically increases RAM capacity.

Wrong. No RAM is added. The illusion of more is created using disk, and the word PHYSICALLY gives this away.

✗ It eliminates page faults.

Wrong, and inverted. Page faults are the MECHANISM of virtual memory, not something it removes.

✓ It allows programs to execute using an address space larger than physical memory.

Correct. An address space larger than the physical memory installed.

Virtual memory gives each program a large logical address space, with pages moved between RAM and disk as needed.

A workload frequently accesses neighboring memory locations after accessing a particular location. Which locality principle is being demonstrated?

- a. Virtual locality
- b. Random locality
- c. Temporal locality
- d. Spatial locality

✗ Virtual locality

Wrong. There is no such principle. It is invented to sound like virtual memory.

✗ Random locality

Wrong. Random access is the ABSENCE of locality, which is the worst case for a cache.

✗ Temporal locality

Wrong, and the near miss. Temporal locality is the same location reused over TIME, while this stem says neighbouring.

✓ Spatial locality

Correct. Nearby addresses accessed next.

NEIGHBOURING locations means space, so this is spatial locality. It is why cache fetches a whole block rather than a single word, and why array traversal is fast.

8.70

MEDIUM

Mem S2

The deck compares direct access to locating a street, that is finding the vicinity or block first. Its example is:

- a. The hard disk drive
- b. Magnetic tape
- c. RAM
- d. Cache

✓ The hard disk drive

Correct. The hard disk drive.

✗ Magnetic tape

Wrong. Magnetic tape is the sequential example.

✗ RAM

Wrong. RAM is the random access example.

✗ Cache

Wrong. Cache is the associative example.

The hard disk drive. The head moves to the right track, which is the vicinity, and then waits for the item to come round.

8.71

MEDIUM

Mem S2

Direct access is compared to locating a street: you find the _____ first.

- a. vicinity
- b. address
- c. content
- d. sequence

Answer: **vicinity**

The vicinity, which the deck glosses as the block. Direct access is therefore a halfway house between sequential and random.

Associative access is described as address plus content together. Its example is:

- a. Cache
- b. RAM
- c. Magnetic tape
- d. Secondary storage

✓ Cache

Correct, and the tag field is exactly the address stored with the content.

✗ RAM

Wrong. RAM is random access.

✗ Magnetic tape

Wrong. Magnetic tape is sequential.

✗ Secondary storage

Wrong. Secondary storage such as a hard disk is direct access.

Cache. It stores a tag beside each line, so a search compares content rather than counting through addresses.

Select the characteristics of computer memory the deck lists.

- a. Location
- b. Access methods
- c. Performance, such as transfer rate
- d. Capacity or size
- e. Manufacturer

✓ Location

Correct, location heads the list.

✓ Access methods

Correct, access methods is second, and the four are set out on the same slide.

✓ Performance, such as transfer rate

Correct, with transfer rate given as the example measure.

✓ Capacity or size

Correct, capacity or size closes the list.

✗ Manufacturer

Wrong. Manufacturer is not a characteristic of memory in this course.

Four characteristics: location, access methods, performance, capacity or size.

Select the trade-offs the deck states about memory.

- a. Faster access time means greater cost per bit
- b. Greater capacity means slower access
- c. Smaller cost per bit means slower access
- d. Faster access time means greater capacity

✓ Faster access time means greater cost per bit

Correct, faster costs more per bit.

✓ Greater capacity means slower access

Correct, more capacity is slower.

✓ Smaller cost per bit means slower access

Correct, cheaper per bit is slower.

✗ Faster access time means greater capacity

Wrong, and it is the opposite of the second trade-off: fast memories are small, not large.

Three trade-offs, and the deck calls them a sad part of reality. Every one of them says that you cannot have two good things at once.

Design constraints on memory can be summed up by which three questions?

- a. How much space?
- b. How fast, that is, what access time?
- c. How expensive?
- d. How reliable?

✓ How much space?

Correct, capacity is the first question.

✓ How fast, that is, what access time?

Correct, access time is the second.

✓ How expensive?

Correct, cost is the third.

✗ How reliable?

Wrong. Reliability is not one of the three.

Three questions: how much, how fast, how expensive. They map exactly onto the three columns of the hierarchy table.

8.76

MEDIUM

Mem S5

On the question of how much memory, the deck observes that:

- a. If memory exists, applications will likely be developed to use it
- b. Applications rarely use the memory available to them
- c. Memory should always be doubled every two years
- d. Memory size is fixed by the instruction set

✓ If memory exists, applications will likely be developed to use it

Correct. Demand follows supply.

✗ Applications rarely use the memory available to them

Wrong, and it inverts the observation.

✗ Memory should always be doubled every two years

Wrong. No such rule is given in this course.

✗ Memory size is fixed by the instruction set

Wrong. The address width bounds what can be addressed, but that is not the point being made here.

Demand follows supply. Whatever you provide will eventually be consumed, which is why the capacity question never settles.

8.77

MEDIUM

Mem S5

Best performance is achieved when:

- a. Memory keeps up with the processor, minimizing pausing or waiting for instructions or operands
- b. The processor keeps up with the memory
- c. Memory and processor run on separate clocks
- d. The cache is as large as main memory

✓ Memory keeps up with the processor, minimizing pausing or waiting for instructions or operands

Correct. Memory keeps up with the processor.

✗ The processor keeps up with the memory

Wrong, and reversed: the processor is not the component being asked to wait.

✗ Memory and processor run on separate clocks

Wrong. Separate clocks are not discussed.

✗ The cache is as large as main memory

Wrong, and a cache the size of main memory would be ruinously expensive, which is the point of the whole hierarchy.

Memory keeps up with the processor. The processor is the one setting the pace, and every pause it takes is wasted time.

8.78

MEDIUM

Mem S5

As the processor executes instructions, memory should minimize pausing or waiting for instructions or ____ .

- a. operands
- b. addresses
- c. interrupts
- d. registers

Answer: **operands**

Operands. Instructions and operands are the two things a processor fetches, and either can stall it.

The deck's way out of the memory dilemma is to:

- a. Employ a memory hierarchy, supplementing smaller, more expensive, faster memories with larger, cheaper, slower ones
- b. Rely on a single fast memory
- c. Rely on a single large memory
- d. Remove the cache and use main memory only

✓ Employ a memory hierarchy, supplementing smaller, more expensive, faster memories with larger, cheaper, slower ones

Correct. Do not rely on a single memory: use a hierarchy, and let each level cover the other's weakness.

✗ Rely on a single fast memory

Wrong. The deck's first instruction is not to rely on a single memory.

✗ Rely on a single large memory

Wrong, for the same reason.

✗ Remove the cache and use main memory only

Wrong. Removing the cache would give up the whole benefit.

Do not rely on a single memory: use a hierarchy, and let each level cover the other's weakness.

8.80

MEDIUM

Mem S6

The way out is to supplement smaller, more expensive, _____ memories with larger, cheaper, _____ memories.

BLANK 1

- a. faster
- b. slower
- c. denser
- d. volatile

BLANK 2

- a. slower
- b. faster
- c. smaller
- d. costlier

Answer: **faster, slower**

Faster at the top, slower below. Each of the three adjectives moves in the opposite direction as you descend.

8.81

MEDIUM

Mem S7

As one goes down the memory hierarchy, which of these happen?

- a. Cost per bit decreases
- b. Capacity increases
- c. Access time increases
- d. Access time decreases

✓ Cost per bit decreases

Correct, decreasing cost per bit.

✓ Capacity increases

Correct, increasing capacity.

✓ Access time increases

Correct, increasing access time, which means slower.

✗ Access time decreases

Wrong. Access time increases going down; it decreases going up.

Cost per bit down, capacity up, access time up. Access time increasing means it gets slower, which is the wording most easily misread.

8.82

MEDIUM

Mem S7

As one goes down the hierarchy, cost per bit _____ while capacity increases.

- a. decreases
- b. increases
- c. stays the same
- d. doubles

Answer: **decreases**

Decreases. It is the only one of the four that goes down as you descend.

8.83

MEDIUM

Mem S7

Going down the memory hierarchy, the frequency with which the processor accesses each level decreases.

Answer: **true**

It does, and it has to. A hierarchy only works if the slow levels are touched rarely.

8.84

MEDIUM

Mem S8

In the typical memory hierarchy from top to bottom, which level is Level 3?

- a. Main Memory, that is RAM
- b. Cache Memory
- c. Registers
- d. Secondary Storage

✓ Main Memory, that is RAM

Correct. Level 1 Registers, Level 2 Cache, Level 3 Main Memory, Level 4 Secondary Storage, Level 5 Tertiary Storage.

✗ Cache Memory

Wrong. Cache is Level 2.

✗ Registers

Wrong. Registers are Level 1.

✗ Secondary Storage

Wrong. Secondary storage is Level 4.

Level 1 Registers, Level 2 Cache, Level 3 Main Memory, Level 4 Secondary Storage, Level 5 Tertiary Storage.

8.85

MEDIUM

Mem S8

Match each level number to its memory type in the hierarchy table.

Level 1	Registers
Level 2	Cache Memory
Level 3	Main Memory (RAM)
Level 4	Secondary Storage (SSD or HDD)
Level 5	Tertiary Storage

Five levels, top to bottom. Tertiary storage at Level 5 is the one most often forgotten, since the hierarchy is usually drawn with only four.

8.86

MEDIUM

Mem S8

In the hierarchy table tertiary storage is _____ in speed, lowest in cost and _____ in capacity.

BLANK 1

- a. slowest
- b. fastest
- c. moderate
- d. very fast

BLANK 2

- a. very large
- b. large
- c. medium
- d. smallest

Answer: **slowest, very large**

| Slowest, lowest cost, very large. It is the exact opposite of the register row in all three columns.

8.87

MEDIUM

Mem S8

In the hierarchy table, how is main memory described?

- a. Moderate in speed, medium in cost, medium in capacity
- b. Very fast, high in cost, small in capacity
- c. Slow, low in cost, large in capacity
- d. Fastest, highest in cost, smallest in capacity

✓ Moderate in speed, medium in cost, medium in capacity

Correct. Moderate, medium, medium.

✗ Very fast, high in cost, small in capacity

Wrong. That is the cache row.

✗ Slow, low in cost, large in capacity

Wrong. That is the secondary storage row.

✗ Fastest, highest in cost, smallest in capacity

Wrong. That is the register row.

Moderate, medium, medium. Main memory sits in the middle of all three columns, which is what makes it the reference point.

8.88

MEDIUM

Mem S8

In the hierarchy table cache memory is described as _____ fast, high in cost and small in capacity.

- a. very
- b. moderately
- c. extremely
- d. reasonably

Answer: **very**

Very fast. Registers alone get the superlative fastest; cache is very fast, one step below.

8.89

MEDIUM

Mem S12

Spatial locality says that if we access a memory location:

- a. Close by addresses will very likely be accessed
- b. We will very likely access the same location again
- c. We will not access that region again
- d. The next access will be random

✓ Close by addresses will very likely be accessed

Correct. Nearby addresses, and the deck's example is a loop operation.

✗ We will very likely access the same location again

Wrong. That is temporal locality.

✗ We will not access that region again

Wrong, and the opposite of what locality claims.

✗ The next access will be random

Wrong. Randomness is exactly what locality says does not happen.

Nearby addresses, and the deck's example is a loop operation. It is why a whole block is fetched into the cache rather than a single word.

8.90

MEDIUM

Mem S12

The deck's example of spatial locality in action is a ____ operation.

- a. loop
- b. branch
- c. store
- d. interrupt

Answer: **loop**

A loop, whose instructions sit next to one another in memory and are stepped through in order.

Temporal locality says that if we access a memory location:

- a. We will very likely access it again
- b. Close by addresses will very likely be accessed
- c. It will soon be swapped out to disk
- d. It will be written back immediately

✓ We will very likely access it again

Correct. The same location again.

✗ Close by addresses will very likely be accessed

Wrong. That is spatial locality.

✗ It will soon be swapped out to disk

Wrong. Swapping is a virtual memory operation and is not what locality predicts.

✗ It will be written back immediately

Wrong. Write-back policy is not part of this claim.

| The same location again. Time, not place, which is what the word temporal signals.

Locality of reference is described as a consequence of using:

- a. Iterative loops and subroutines
- b. Virtual memory
- c. Pipelining
- d. Cache mapping functions

✓ Iterative loops and subroutines

Correct. Loops and subroutines, because instructions and data get accessed multiple times.

✗ Virtual memory

Wrong. Virtual memory exploits locality; it does not cause it.

✗ Pipelining

Wrong. Pipelining is a processor technique.

✗ Cache mapping functions

Wrong. Mapping functions are how a cache is organized, not why locality exists.

Loops and subroutines, because instructions and data get accessed multiple times. Locality is a property of how programs are written, not of the hardware.

8.93

MEDIUM

Mem S14

In the two-level worked example, what are the sizes and access times of L1 and L2?

- a. L1 is 1000 words with an access time of 0.01 microseconds; L2 is 100,000 words with an access time of 0.1 microseconds
- b. L1 is 100 words at 0.01 microseconds; L2 is 1000 words at 0.1 microseconds
- c. L1 is 1000 words at 0.1 microseconds; L2 is 100,000 words at 0.01 microseconds
- d. L1 is 10,000 words at 0.01 microseconds; L2 is 100,000 words at 1.0 microseconds

✓ L1 is 1000 words with an access time of 0.01 microseconds; L2 is 100,000 words with an access time of 0.1 microseconds

Correct. L1 is a hundred times smaller and ten times faster than L2.

✗ L1 is 100 words at 0.01 microseconds; L2 is 1000 words at 0.1 microseconds

Wrong sizes. L1 is a hundred times smaller and ten times faster than L2.

✗ L1 is 1000 words at 0.1 microseconds; L2 is 100,000 words at 0.01 microseconds

Wrong, and it reverses the access times, which would make the hierarchy pointless.

✗ L1 is 10,000 words at 0.01 microseconds; L2 is 100,000 words at 1.0 microseconds

Wrong on both the L1 size and the L2 access time.

L1 is a hundred times smaller and ten times faster than L2. Those two ratios drive the whole calculation.

8.94

MEDIUM

Mem S14

In the two-level example, L2 contains _____ words or bytes and has an access time of 0.1 microseconds.

- a. 100000
- b. 1000
- c. 10000
- d. 1000000

Answer: **100000**

100,000, against L1's 1000.

8.95

MEDIUM

Mem S14

In the two-level model, if a word is in L1 the processor accesses it directly.

Answer: **true**

It does, at 0.01 microseconds. Only an L2 word takes the two-step route.

8.96

MEDIUM

Mem S15

In the two-level example, what does H stand for?

- a. The fraction of all memory accesses that are found in L1
- b. The access time to L1
- c. The access time to L2
- d. The number of levels in the hierarchy

✓ The fraction of all memory accesses that are found in L1

Correct. H is the hit fraction for L1.

✗ The access time to L1

Wrong. That is T1.

✗ The access time to L2

Wrong. That is T2.

✗ The number of levels in the hierarchy

Wrong. The number of levels is not given a symbol here.

H is the hit fraction for L1. T1 and T2 are the access times to L1 and L2 respectively.

8.97

MEDIUM

Mem S15

For simplicity, the example ignores the time required for the processor to determine whether a word is in L1 or L2.

Answer: **true**

It is ignored, which is worth noticing: a real machine pays something for the lookup itself.

8.98

MEDIUM

Mem S17

For high percentages of L1 access, the average total access time is:

- a. Much closer to that of L1 than that of L2
- b. Much closer to that of L2 than that of L1
- c. Exactly halfway between the two
- d. Equal to the sum of the two

✓ Much closer to that of L1 than that of L2

Correct. Closer to L1, which is the whole promise of the hierarchy: most of the speed of the fast level, most of the size of the slow one.

✗ Much closer to that of L2 than that of L1

Wrong. That would only happen if most accesses missed.

✗ Exactly halfway between the two

Wrong. It would be halfway only at a hit rate of about 50 per cent, and even then not exactly.

✗ Equal to the sum of the two

Wrong. The sum is the cost of a single miss, not the average.

Closer to L1, which is the whole promise of the hierarchy: most of the speed of the fast level, most of the size of the slow one.

8.99

MEDIUM

Mem S17

For high percentages of L1 access, the average total access time is much closer to that of

_____ .

- a. L1
- b. L2
- c. the disk
- d. the register file

Answer: **L1**

L1, as desired. The worked figure that follows, 0.015 microseconds against L1's 0.01, shows exactly how close.

8.100

MEDIUM

Mem S18

The strategy to minimize accesses is to organize data across the hierarchy such that:

- a. The percentage of accesses to lower levels is substantially less than that to upper levels
- b. Every level is accessed equally often
- c. Lower levels are accessed more often than upper levels
- d. Only the lowest level is ever accessed

✓ The percentage of accesses to lower levels is substantially less than that to upper levels
Correct. Far fewer accesses to the slow levels.

✗ Every level is accessed equally often
Wrong. Equal access would let the slowest level dominate the average.

✗ Lower levels are accessed more often than upper levels
Wrong, and it is exactly the wrong way round.

✗ Only the lowest level is ever accessed
Wrong. The upper levels are the ones meant to serve most accesses.

Far fewer accesses to the slow levels. It restates the frequency of access point from the hierarchy slide as a design instruction.

8.101

MEDIUM

Mem S18

Organize data so that the percentage of accesses to lower levels is _____ less than that of upper levels.

- a. substantially
- b. slightly
- c. occasionally
- d. marginally

Answer: **substantially**

Substantially. The 95 per cent example shows what substantially looks like in practice.

8.102

MEDIUM

Mem S18

In the strategy, what is held in L2 and what is held in L1?

- a. L2 contains all program instructions and data, while data currently being used is in L1
- b. L1 contains all program instructions and data, while L2 holds only the current data
- c. Both hold identical copies of everything
- d. L1 holds instructions and L2 holds data

✓ L2 contains all program instructions and data, while data currently being used is in L1
Correct. Everything in L2, the working set in L1.

✗ L1 contains all program instructions and data, while L2 holds only the current data
Wrong, and reversed. L1 is the small fast level, so it cannot hold everything.

✗ Both hold identical copies of everything
Wrong. L1 is far too small for that.

✗ L1 holds instructions and L2 holds data
Wrong. Splitting instructions from data is the Harvard architecture, a different idea entirely.

Everything in L2, the working set in L1. Eventually L1 data is swapped back to L2 to make room for new data.

8.103

MEDIUM

Mem S18

Data in L1 is eventually swapped to L2 to make room for new data, so that on average most references are to data in L1.

Answer: **true**

That is the mechanism. L1 is kept stocked with what is being used now, and what falls out of use goes back down.

In the hierarchy, the cache is described as staging data movement between:

- a. Registers and main memory
- b. Main memory and secondary storage
- c. The ALU and the control unit
- d. Secondary and tertiary storage

✓ Registers and main memory

Correct. The cache sits between the registers above it and main memory below it, and staging the traffic between them is its whole job.

✗ Main memory and secondary storage

Wrong. That gap is bridged by virtual memory, not by the cache.

✗ The ALU and the control unit

Wrong. Both are processor units and neither is a level of the hierarchy.

✗ Secondary and tertiary storage

Wrong. The cache is at the top of the hierarchy, not the bottom.

The cache sits between the registers above it and main memory below it, and staging the traffic between them is its whole job.

Cache memory is designed to combine:

- a. The access time of expensive, high-speed memory with the size of less expensive, lower-speed memory
- b. The access time of cheap memory with the size of expensive memory
- c. The reliability of ROM with the speed of RAM
- d. The capacity of secondary storage with the volatility of main memory

✓ The access time of expensive, high-speed memory with the size of less expensive, lower-speed memory

Correct. The speed of the fast memory with the size of the slow one.

✗ The access time of cheap memory with the size of expensive memory

Wrong, and it takes the worst of each rather than the best.

✗ The reliability of ROM with the speed of RAM

Wrong. Reliability is not part of this claim.

✗ The capacity of secondary storage with the volatility of main memory

Wrong. Volatility is not something anyone sets out to combine.

The speed of the fast memory with the size of the slow one. It is the hierarchy's promise stated for one particular pair of levels.

8.106

MEDIUM

Mem S20

The deck gives typical sizes as:

- a. Cache 1 to 2 MB, main memory up to 64 GB
- b. Cache 1 to 2 GB, main memory up to 64 MB
- c. Cache 16 KB, main memory 128 KB
- d. Cache 64 MB, main memory 1 to 2 GB

✓ Cache 1 to 2 MB, main memory up to 64 GB

Correct. 1 to 2 MB against up to 64 GB, a ratio of tens of thousands to one.

✗ Cache 1 to 2 GB, main memory up to 64 MB

Wrong, and reversed, which would make the cache larger than main memory.

✗ Cache 16 KB, main memory 128 KB

Wrong. Those are figures from the worked mapping examples, not the typical sizes.

✗ Cache 64 MB, main memory 1 to 2 GB

Wrong on both numbers.

1 to 2 MB against up to 64 GB, a ratio of tens of thousands to one. That gap is why the mapping functions matter.

8.107

MEDIUM

Mem S20

Cache can be 1 to 2 MB while main memory can be up to _____ GB.

- a. 64
- b. 16
- c. 32
- d. 128

Answer: **64**

64 GB, against a cache of 1 to 2 MB, a ratio of tens of thousands to one.

8.108

MEDIUM

Mem S21

In the deck's memory terms, a block is:

- a. A collection of words, typically 2, 4 or 8
- b. A single byte
- c. Four bits
- d. One complete row of memory

✓ A collection of words, typically 2, 4 or 8

Correct. A collection of words.

✗ A single byte

Wrong. A byte is 8 bits.

✗ Four bits

Wrong. Four bits is a nibble.

✗ One complete row of memory

Wrong. That is the deck's definition of a word.

A collection of words. The word itself is one or more complete rows of memory, and the deck's example counts 4 words to a block.

8.109

MEDIUM

Mem S21

A word is one or more complete _____ of memory.

- a. rows
- b. columns
- c. blocks
- d. pages

Answer: **rows**

Rows. The slide draws memory as a grid, so a row is the natural unit and a block is several of them.

On a cache miss, what is read into the cache?

- a. The block of main memory that contains the word
- b. The single word only
- c. The whole page containing the word
- d. Nothing, the word is read straight from main memory

✓ The block of main memory that contains the word

Correct, and the cache then delivers the word to the processor.

✗ The single word only

Wrong, and it would throw away the benefit of spatial locality.

✗ The whole page containing the word

Wrong. Pages belong to virtual memory, blocks to the cache.

✗ Nothing, the word is read straight from main memory

Wrong. The deck's sequence loads the block first.

The whole block, not just the word. Spatial locality says the neighbours will be wanted soon, so fetching them is not waste.

8.111

MEDIUM

Mem S22

On a cache _____, the block of main memory containing the word is read into the cache, and the cache then delivers the _____ to the processor.

BLANK 1

- a. miss
- b. hit
- c. flush
- d. stall

BLANK 2

- a. word
- b. block
- c. tag
- d. line

Answer: **miss, word**

| A miss brings in a block, but only a word is handed to the processor.

Because of the locality of reference principle, when a block of data is fetched into the cache:

- a. It is likely that there will be future references to that same memory location
- b. It is unlikely to be referenced again
- c. It must be written back immediately
- d. It replaces the entire cache

✓ It is likely that there will be future references to that same memory location

Correct. Future references are likely, by both temporal and spatial locality.

✗ It is unlikely to be referenced again

Wrong, and it is the exact opposite of the locality principle.

✗ It must be written back immediately

Wrong. Write-back timing is not discussed here.

✗ It replaces the entire cache

Wrong. A block replaces a line, not the whole cache.

Future references are likely, by both temporal and spatial locality. Without that likelihood, caching would be a pure cost.

8.113

MEDIUM

Mem S23

How does the deck describe L2 cache relative to L1?

- a. Slower and typically larger
- b. Faster and typically smaller
- c. The same speed but larger
- d. Slower and typically smaller

✓ Slower and typically larger

Correct. Slower and larger, and L3 stands in the same relation to L2.

✗ Faster and typically smaller

Wrong, and reversed.

✗ The same speed but larger

Wrong. It is slower as well as larger.

✗ Slower and typically smaller

Wrong. It is larger, not smaller.

Slower and larger, and L3 stands in the same relation to L2. The whole hierarchy repeats inside the cache.

8.114

MEDIUM

Mem S23

L3 cache is _____ and typically _____ than the L2 cache.

BLANK 1

- a. slower
- b. faster
- c. cheaper
- d. narrower

BLANK 2

- a. larger
- b. smaller
- c. denser
- d. simpler

Answer: **slower, larger**

Slower and larger, exactly as L2 is to L1.

8.115

MEDIUM

Mem S24

Main memory consists of 2 to the power n addressable words. What follows about each word's address?

- a. Each word has a unique n -bit address
- b. Each word has a unique $2n$ -bit address
- c. Addresses are shared between blocks
- d. Each block, not each word, has an address

✓ Each word has a unique n -bit address

Correct. N bits address 2 to the power n words, one address each.

✗ Each word has a unique $2n$ -bit address

Wrong. $2n$ bits would address 2 to the power $2n$ words.

✗ Addresses are shared between blocks

Wrong. The addresses are unique.

✗ Each block, not each word, has an address

Wrong. Words are addressable; blocks are groups of words.

n bits address 2 to the power n words, one address each. It is the same counting argument as the unsigned range.

8.116

MEDIUM

Mem S24

Memory consists of a number of fixed-length blocks of _____ words each.

- a. K
- b. M
- c. n
- d. m

Answer: **K**

K words per block. M is the number of blocks, m is the number of cache lines, and n is the address width, so the four letters are worth keeping straight.

8.117

MEDIUM

Mem S24

If main memory holds 2^n words in blocks of K words, the number of blocks is 2^n divided by K .

Answer: **true**

It is, and the deck writes it as $M = 2^n / K$.

8.118

MEDIUM

Mem S25

Cache memory consists of m blocks, which are called:

- a. Lines
- b. Pages
- c. Frames
- d. Segments

✓ Lines

Correct, and line is the word the mapping functions use throughout.

✗ Pages

Wrong. Pages belong to virtual memory.

✗ Frames

Wrong here, although the worked examples note that frame size means the same as line size.

✗ Segments

Wrong. Segment is the pipelining deck's word for a stage.

Lines, each containing K words, the same K as a main memory block. The worked examples treat block size, frame size and line size as the same quantity.

8.119

MEDIUM

Mem S25

Each cache line contains K words and also includes _____ bits.

- a. control
- b. parity
- c. index
- d. offset

Answer: **control**

Control bits, which the deck notes are not shown in its figure. The tag is held alongside them.

8.120

MEDIUM

Mem S26

If a word in a block of memory is read, what is transferred to a cache line?

- a. The whole block
- b. The word alone
- c. The tag alone
- d. The whole page

✓ The whole block

Correct. The block.

✗ The word alone

Wrong. Transferring one word would waste the fetch and ignore spatial locality.

✗ The tag alone

Wrong. The tag identifies the block; it is not the payload.

✗ The whole page

Wrong. Pages are the unit of virtual memory, not of cache transfer.

The block. That is the unit of transfer between main memory and the cache, and it is why the block size fixes the offset field width.

8.121

MEDIUM

Mem S26

The information identifying which block a cache line is holding is stored in the _____ field.

- a. tag
- b. index
- c. offset
- d. control

Answer: **tag**

The tag field. Every mapping function in the next topic is really a question about how wide that field has to be.

8.122

MEDIUM

Mem S28

In the cache read operation, what does the processor generate first?

- a. The read address, RA, of the word to be read
- b. The tag of the block
- c. The line number
- d. The block offset

✓ The read address, RA, of the word to be read

Correct. The read address RA.

✗ The tag of the block

Wrong. The tag is a part of the address, derived from it afterwards.

✗ The line number

Wrong. The line number is also a field within the address.

✗ The block offset

Wrong. The offset is likewise a field within the address.

| The read address RA. The tag, index and offset are then extracted from it by the cache hardware.

8.123

MEDIUM

Mem S28

On a cache miss, loading the block into the cache and delivering the word to the processor happen one after the other, never at the same time.

Answer: **false**

| They occur in parallel. The deck says so explicitly at the end of the cache read operation.

8.124

MEDIUM

Mem S30

The cache connects to the processor via which lines?

- a. Data
- b. Control
- c. Address
- d. Interrupt

✓ Data

Correct, data lines.

✓ Control

Correct, control lines.

✓ Address

Correct, address lines.

✗ Interrupt

Wrong. Interrupt lines are not mentioned in this course.

| Data, control and address, the same three kinds of line the system buses carry.

8.125

MEDIUM

Mem S30

The data and address lines also attach to data and address buffers, which attach to:

- a. A system bus, from which main memory is reached
- b. The register file
- c. The control unit
- d. Secondary storage directly

✓ A system bus, from which main memory is reached

Correct. The system bus, and main memory lies beyond it.

✗ The register file

Wrong. The register file sits inside the processor.

✗ The control unit

Wrong. The control unit generates control signals rather than receiving the buffers.

✗ Secondary storage directly

Wrong. Secondary storage is not part of this diagram.

| The system bus, and main memory lies beyond it. That path is what a cache hit avoids using.

8.126

MEDIUM

Mem S31

On a cache hit there is no system _____ traffic.

- a. bus
- b. clock
- c. memory
- d. cache

Answer: **bus**

| No system bus traffic. Communication stays between the processor and the cache.

8.127

MEDIUM

Mem S32

On a cache miss the desired address is loaded onto the system bus.

Answer: **true**

| It is, because main memory can only be reached across the bus. On a hit the buffers stay disabled and the bus stays quiet.

The two types of cache address are:

- a. Physical addresses, the actual memory addresses, and logical addresses, the virtual-memory addresses
- b. Tag addresses and index addresses
- c. Absolute addresses and relative addresses
- d. Read addresses and write addresses

✓ Physical addresses, the actual memory addresses, and logical addresses, the virtual-memory addresses

Correct. Physical and logical.

✗ Tag addresses and index addresses

Wrong. Tag and index are fields within an address, not kinds of address.

✗ Absolute addresses and relative addresses

Wrong. Relative addressing is an addressing mode.

✗ Read addresses and write addresses

Wrong. Direction of access is not what distinguishes them.

Physical and logical. A cache using logical addresses is a virtual cache, which the deck then compares against the alternative.

8.129

MEDIUM

Mem S35

Virtual memory performs mapping between:

- a. The logical addresses used by a program and physical addresses
- b. Cache lines and main memory blocks
- c. Registers and cache lines
- d. Instructions and data

✓ The logical addresses used by a program and physical addresses

Correct. Logical to physical.

✗ Cache lines and main memory blocks

Wrong. That mapping is the cache mapping function.

✗ Registers and cache lines

Wrong. No such mapping is described in this course.

✗ Instructions and data

Wrong. Separating instructions from data is the Harvard architecture.

Logical to physical. Its main idea is to let a program disregard the amount of main memory available.

8.130

MEDIUM

Mem S35

The main idea behind virtual memory is to _____ the amount of main memory available.

- a. disregard
- b. double
- c. measure
- d. reserve

Answer: **disregard**

Disregard it. The program is written as though memory were unlimited, and the mapping deals with the truth.

8.131

MEDIUM

Mem S37

A virtual cache stores data using logical addresses. What follows for the processor?

- a. It accesses the cache directly, without going through the MMU
- b. It must go through the MMU on every access
- c. It cannot use the cache for writes
- d. It must flush the cache on every access

✓ It accesses the cache directly, without going through the MMU

Correct. Straight to the cache, with no address translation first.

✗ It must go through the MMU on every access

Wrong, and it would remove the advantage.

✗ It cannot use the cache for writes

Wrong. Nothing here restricts writes.

✗ It must flush the cache on every access

Wrong. Flushing is needed at a context switch, not on every access.

Straight to the cache, with no address translation first. Skipping the MMU is exactly where the speed comes from.

8.132

MEDIUM

Mem S37

A virtual cache stores data using _____ addresses.

- a. logical
- b. physical
- c. absolute
- d. effective

Answer: **logical**

Logical, which is the same thing as virtual-memory addresses.

8.133

MEDIUM

Mem S37

The advantage of a virtual cache is:

- a. Faster access speed, since the cache can respond without an MMU address translation
- b. Lower cost per bit
- c. Greater capacity
- d. No need to flush on a context switch

✓ Faster access speed, since the cache can respond without an MMU address translation
Correct. Speed, because a translation step is removed from the path.

✗ Lower cost per bit
Wrong. The addressing scheme does not change the cost of the memory.

✗ Greater capacity
Wrong. Capacity is unaffected.

✗ No need to flush on a context switch
Wrong, and precisely backwards: flushing on a context switch is its disadvantage.

| Speed, because a translation step is removed from the path.

8.134

MEDIUM

Mem S37

A virtual cache is faster because it can respond without needing an MMU address translation.

Answer: **true**

| That is the stated advantage. The translation is real work, and skipping it saves time on every hit.

8.135

MEDIUM

Mem S38

Besides the addressing logic, available _____ and board area also limits cache size.

- a. chip
- b. bus
- c. power
- d. clock

Answer: **chip**

| Chip and board area. A cache has to physically fit, which is a hard limit no design cleverness removes.

Select the real-life applications of register sets the deck gives.

- a. High-speed computation: programs execute faster when data is in registers instead of memory
- b. Real-time systems such as air traffic control rely on fast register operations
- c. Mobile devices: efficient register usage improves battery life and performance
- d. Cloud systems: efficient register usage supports large-scale computing

✓ High-speed computation: programs execute faster when data is in registers instead of memory

Correct, high-speed computation heads the list.

✓ Real-time systems such as air traffic control rely on fast register operations

Correct, with air traffic control as the named example.

✓ Mobile devices: efficient register usage improves battery life and performance

Correct, mobile devices close the list.

✗ Cloud systems: efficient register usage supports large-scale computing

Wrong. Cloud systems are listed under memory organization.

Three applications. Cloud systems appear on the memory organization slide, not the register one.

The control unit coordinates all operations of the CPU by doing what?

- a. Fetching instructions
- b. Decoding instructions
- c. Generating control signals
- d. Performing arithmetic

✓ Fetching instructions

Correct, fetching instructions is first.

✓ Decoding instructions

Correct, decoding is second.

✓ Generating control signals

Correct, generating control signals is third.

✗ Performing arithmetic

Wrong. Arithmetic and logic operations are the ALU's work.

Fetch, decode, generate control signals. Arithmetic belongs to the ALU, which is the next organizational component.

8.138

MEDIUM

Intro S47

Select the real-life applications of the control unit the deck gives.

- a. Processor execution: every program relies on the control unit
- b. Industrial automation: machines operate based on control signals
- c. Robotics systems: it controls sensors and actuators
- d. Digital calculators: they perform arithmetic operations

✓ Processor execution: every program relies on the control unit

Correct, processor execution heads the list.

✓ Industrial automation: machines operate based on control signals

Correct, industrial automation is second.

✓ Robotics systems: it controls sensors and actuators

Correct, robotics systems close the list.

✗ Digital calculators: they perform arithmetic operations

Wrong. Digital calculators appear under the arithmetic logic unit.

Three applications. Digital calculators are listed under the ALU, since arithmetic is what they do.

8.139

MEDIUM

Intro S50

Match each real-life application of I/O systems to the deck's description.

User interaction	Keyboard input and monitor output
-------------------------	-----------------------------------

Medical devices	Sensors provide input, displays provide output
------------------------	--

Smart homes	Devices communicate with environmental sensors
--------------------	--

Three applications, and each names an input and an output, which is the point of the component.

How does the introduction deck distinguish pipelining from parallelism?

- a. Pipelining is overlapping instruction execution, parallelism is executing multiple tasks simultaneously
- b. Pipelining is executing multiple tasks simultaneously, parallelism is overlapping instruction execution
- c. They are two words for the same technique
- d. Pipelining applies to memory, parallelism to the processor

✓ Pipelining is overlapping instruction execution, parallelism is executing multiple tasks simultaneously

Correct. Overlapping against simultaneous.

✗ Pipelining is executing multiple tasks simultaneously, parallelism is overlapping instruction execution

Wrong, and reversed.

✗ They are two words for the same technique

Wrong. The deck defines them separately on the same line.

✗ Pipelining applies to memory, parallelism to the processor

Wrong. Both are processor techniques.

Overlapping against simultaneous. Pipelining shares one datapath among instructions at different stages; parallelism uses more than one unit at once.

Select the real-life applications of pipelining and parallelism the deck gives.

- a. Modern CPUs execute multiple instructions at once
- b. Multicore processors enable multitasking
- c. AI and big data systems handle massive computations efficiently
- d. Web browsers cache pages to load faster

✓ Modern CPUs execute multiple instructions at once

Correct, modern CPUs head the list.

✓ Multicore processors enable multitasking

Correct, multicore processors are second.

✓ AI and big data systems handle massive computations efficiently

Correct, AI and big data systems close it.

✗ Web browsers cache pages to load faster

Wrong. That is listed under memory organization.

Three applications. Web browser caching belongs to memory organization.

A computer architect argues that registers should be larger than cache memory. Why is this argument incorrect?

- a. Registers are virtual memory components.
- b. Registers are intended for only the most immediate processor operations.
- c. Registers belong to secondary storage.
- d. Registers are slower.

✗ Registers are virtual memory components.

Wrong. Virtual memory is a main memory and disk mechanism. Registers are physical CPU storage.

✓ Registers are intended for only the most immediate processor operations.

Correct. Their job is the immediate working set, so a small, very fast file is the right design.

✗ Registers belong to secondary storage.

Wrong. Secondary storage is disk and SSD, at the very bottom of the hierarchy.

✗ Registers are slower.

Wrong, and plainly false. Registers are the FASTEST storage there is.

Registers exist to hold the handful of values in use RIGHT NOW. Adding more would need wider instruction fields to name them and would slow the register file down.

A computer engineer claims that increasing cache memory always eliminates processor-memory performance bottlenecks. Which statement best evaluates this claim?

- a. It is correct only when DRAM is removed.
- b. It is incorrect because cache cannot eliminate all memory access delays.
- c. It is correct because cache replaces main memory.
- d. It is always correct because cache operates at processor speed.

✗ It is correct only when DRAM is removed.

Wrong. Removing DRAM would leave nowhere to back the cache from, so the machine could not run at all.

✓ It is incorrect because cache cannot eliminate all memory access delays.

Correct. Misses are inevitable, so some delay always remains.

✗ It is correct because cache replaces main memory.

Wrong. Cache SUPPLEMENTS main memory, holding copies. It does not replace it.

✗ It is always correct because cache operates at processor speed.

Wrong. Cache is fast, but the claim fails on "always": a miss still costs a full trip to main memory.

Compulsory misses still happen, large working sets still overflow, and a bigger cache is itself slower to search. Cache narrows the gap, it never closes it.

A processor spends considerable time waiting for data despite having a large cache. Which phenomenon is primarily responsible?

- a. Address aliasing
- b. Register spilling
- c. Memory wall
- d. Bus inversion

X Address aliasing

Wrong. Address aliasing is two addresses referring to one location, a correctness issue rather than a speed one.

X Register spilling

Wrong. Register spilling is the compiler running out of registers and pushing values to memory. It is a contributor at most, not the phenomenon named here.

✓ Memory wall

Correct. The processor-memory speed gap, which is exactly what the stem describes.

X Bus inversion

Wrong. Bus inversion is a signalling trick for cutting switching power, unrelated to waiting for data.

Processor speed has grown far faster than memory speed for decades. That widening gap is the memory wall, and cache only softens it.

8.145

HARD

The real test 2, question 58

A memory module has an access time of 60 ns and a cycle time of 90 ns. Which interpretation is correct?

- a. Access time is longer than cycle time.
- b. Cycle time has no practical significance.
- c. Consecutive accesses must be separated by at least 90 ns.
- d. A new memory operation can begin every 60 ns.

✗ Access time is longer than cycle time.

Wrong, and contradicted by the figures given: 60 is less than 90.

✗ Cycle time has no practical significance.

Wrong. Cycle time sets the maximum sustainable rate, which is very practical indeed.

✓ Consecutive accesses must be separated by at least 90 ns.

Correct. The next access can only begin after the full 90 ns cycle.

✗ A new memory operation can begin every 60 ns.

Wrong, and the trap. 60 ns is when the DATA ARRIVES, not when the next operation may start.

Access time is how long until the data arrives, 60 ns here. Cycle time is how long until the memory can START AGAIN, 90 ns here, because it needs 30 ns of recovery. Cycle time is always the larger of the two.

8.146

HARD

Mem S2

Match each access method to the example the deck gives.

Sequential	Magnetic tape
Random	RAM
Direct	Hard disk drive
Associative	Cache

Four methods, four examples. Learn them as pairs, because an exam question usually gives you one side and asks for the other.

Which change going down the hierarchy does the deck single out as very key to the performance of the CPU?

- a. Decreasing frequency of access of memory by the processor
- b. Decreasing cost per bit
- c. Increasing capacity
- d. Increasing access time

✓ Decreasing frequency of access of memory by the processor

Correct, and the deck marks it as very key to CPU performance.

✗ Decreasing cost per bit

Wrong. Cost matters to the budget, not to the speed the CPU sees.

✗ Increasing capacity

Wrong. Capacity is a benefit, not the performance lever named here.

✗ Increasing access time

Wrong. Access time increasing is the problem; falling access frequency is what makes it bearable.

The processor must touch the lower levels less often as they get slower. If it did not, the slow levels would set the pace.

In the two-level example, what does it cost to access a word that is in L2 but not L1?

- a. 0.01 plus 0.1 microseconds, because the word is transferred to L1 and then accessed
- b. 0.1 microseconds only, because it is read straight from L2
- c. 0.01 microseconds, the same as an L1 access
- d. 0.1 minus 0.01 microseconds

✓ 0.01 plus 0.1 microseconds, because the word is transferred to L1 and then accessed
Correct, and the deck writes it as 0.01 plus 0.1.

✗ 0.1 microseconds only, because it is read straight from L2
Wrong. The transfer into L1 still has to happen.

✗ 0.01 microseconds, the same as an L1 access
Wrong. That is the cost when the word is already in L1.

✗ 0.1 minus 0.01 microseconds
Wrong. The times add; they do not subtract.

| The processor never reads L2 directly. The word is transferred to L1 first, so both times are paid.

If 95 per cent of memory accesses are found in L1, what is the average time to access a word?

- a. 0.015 microseconds
- b. 0.011 microseconds
- c. 0.055 microseconds
- d. 0.11 microseconds

✓ 0.015 microseconds

Correct. 0.95 times 0.01 is 0.0095, and 0.05 times 0.11 is 0.0055. Adding them gives 0.015 microseconds, much closer to 0.01 than to 0.1.

✗ 0.011 microseconds

Wrong. That would come from costing a miss at 0.1 instead of 0.11 and mishandling the weights.

✗ 0.055 microseconds

Wrong. That is the miss contribution scaled wrongly.

✗ 0.11 microseconds

Wrong. 0.11 is the cost of a single miss, not the average over all accesses.

0.95 times 0.01 is 0.0095, and 0.05 times 0.11 is 0.0055. Adding them gives 0.015 microseconds, much closer to 0.01 than to 0.1.

8.150**HARD**

Mem S17

The calculation is 0.95 times 0.01, which is _____ , plus 0.05 times 0.11, which is _____ .

BLANK 1

- a. 0.0095
- b. 0.0055
- c. 0.0100
- d. 0.0150

BLANK 2

- a. 0.0055
- b. 0.0095
- c. 0.0050
- d. 0.0110

Answer: **0.0095, 0.0055**

| 0.0095 and 0.0055, summing to 0.015. Note the miss term uses 0.01 plus 0.1, not 0.1 alone.

Select what the deck says about the cache in the hierarchy.

- a. It stages data movement between registers and main memory
- b. It improves performance
- c. It is not usually visible to the processor
- d. It is not usually visible to the programmer
- e. It must be managed explicitly by the program

✓ It stages data movement between registers and main memory

Correct, and it is the cache's position in the hierarchy.

✓ It improves performance

Correct, improving performance is the reason it exists.

✓ It is not usually visible to the processor

Correct, not usually visible to the processor.

✓ It is not usually visible to the programmer

Correct, not usually visible to the programmer either.

✗ It must be managed explicitly by the program

Wrong, and it contradicts the two invisibility claims: a cache the program had to manage would be visible to it.

Four claims, and the last two are the important pair: the cache works without anyone asking it to. That is what makes it organizational rather than architectural.

8.152**HARD**

Mem S21

A memory size of 4 MB works out as how many bits?

- a. 33,554,432
- b. 4,194,304
- c. 4,000,000
- d. 81,920

✓ 33,554,432

Correct. 4 times 1024 times 1024 gives 4,194,304 bytes, and multiplying by 8 gives 33,554,432 bits.

✗ 4,194,304

Wrong. That is the answer in bytes, before multiplying by 8.

✗ 4,000,000

Wrong. It uses 1000 rather than 1024 and skips the bits conversion.

✗ 81,920

Wrong. 81,920 bits is the deck's figure for 10 KB.

4 times 1024 times 1024 gives 4,194,304 bytes, and multiplying by 8 gives 33,554,432 bits.

8.153**HARD**

Mem S21

10 KB works out as 10 times 1024 times 8, which is _____ bits.

- a. 81920
- b. 10240
- c. 80000
- d. 8192

Answer: **81920**

81,920 bits. The two steps are always the same: to bytes with 1024, then to bits with 8.

How many blocks does main memory contain?

- a. M equals 2 to the power n , divided by K
- b. M equals 2 to the power n , times K
- c. M equals K divided by 2 to the power n
- d. M equals 2 to the power K , divided by n

✓ M equals 2 to the power n , divided by K

Correct. Total words over words per block.

✗ M equals 2 to the power n , times K

Wrong. Multiplying would give more blocks than words.

✗ M equals K divided by 2 to the power n

Wrong, and it inverts the division.

✗ M equals 2 to the power K , divided by n

Wrong. The exponent is n , and K is the divisor.

Total words over words per block. The same shape appears in the worked examples as cache size over line size.

Because m is much less than M , the cache cannot permanently store a block. What follows?

- a. It needs to identify the block currently stored, and that information is held in the tag field
- b. It must store every block in turn
- c. It must be flushed after each instruction
- d. Blocks must be smaller than lines

✓ It needs to identify the block currently stored, and that information is held in the tag field

Correct, and it is the reason the tag field exists at all.

✗ It must store every block in turn

Wrong. Storing every block would need a cache the size of main memory.

✗ It must be flushed after each instruction

Wrong. Flushing on a context switch is a virtual cache issue, not this one.

✗ Blocks must be smaller than lines

Wrong. A block occupies a line, so they are the same size.

Far more blocks than lines, so any line could be holding one of many blocks. The tag is what says which.

On a miss, the block is loaded into the cache and the word is delivered to the processor.
How do these two operations relate?

- a. They occur in parallel
- b. The load must finish before the delivery starts
- c. The delivery must finish before the load starts
- d. Only one of the two happens

✓ They occur in parallel

Correct, and the deck says these last two operations occur in parallel.

✗ The load must finish before the delivery starts

Wrong, and it would waste time.

✗ The delivery must finish before the load starts

Wrong, and it reverses the order as well.

✗ Only one of the two happens

Wrong. Both happen.

In parallel, which is what stops the processor waiting for the whole block to arrive before it can use the word it asked for.

When a cache hit occurs, what happens to the data and address buffers?

- a. They are disabled, so communication is only between processor and cache and there is no system bus traffic
- b. They are enabled, so the address goes onto the system bus
- c. They are flushed
- d. They forward the request to main memory anyway

✓ They are disabled, so communication is only between processor and cache and there is no system bus traffic

Correct. Disabled, and no system bus traffic.

✗ They are enabled, so the address goes onto the system bus

Wrong. That is what happens on a miss.

✗ They are flushed

Wrong. Flushing is what a virtual cache does on a context switch.

✗ They forward the request to main memory anyway

Wrong. There would be no point in a cache that always asked main memory too.

Disabled, and no system bus traffic. That silence on the bus is the real saving of a cache hit.

When a cache miss occurs, the desired address is loaded onto the system bus and the data are returned through the data buffer to:

- a. Both the cache and the processor
- b. The cache only
- c. The processor only
- d. Main memory

✓ Both the cache and the processor

Correct. Both at once, which is the parallel loading and delivering described in the cache read operation.

✗ The cache only

Wrong. The processor gets its word at the same time.

✗ The processor only

Wrong. The cache keeps a copy, otherwise the next reference would miss again.

✗ Main memory

Wrong. Main memory is where the data came from.

| Both at once, which is the parallel loading and delivering described in the cache read operation.

What is the disadvantage of a virtual cache?

- a. The same virtual address in two applications refers to two different physical addresses, so the cache must be flushed at each context switch, or extra bits added to each line
- b. It cannot hold instructions, only data
- c. It is slower than a physical cache
- d. It requires an MMU translation on every access

✓ The same virtual address in two applications refers to two different physical addresses, so the cache must be flushed at each context switch, or extra bits added to each line
Correct, and the deck gives both remedies.

✗ It cannot hold instructions, only data

Wrong. There is no such restriction.

✗ It is slower than a physical cache

Wrong. Speed is its advantage.

✗ It requires an MMU translation on every access

Wrong, and that is exactly what it avoids.

Two programs can use the same virtual address for different real data, so the cache has to be emptied between them, or each line must record which address space it belongs to.

Either the virtual cache must be _____ with each application context switch, or extra bits must be added to each cache line to identify which virtual address _____ the address refers to.

BLANK 1

- a. flushed
- b. doubled
- c. locked
- d. widened

BLANK 2

- a. space
- b. range
- c. region
- d. mode

Answer: **flushed, space**

Flushed, and virtual address space. Two remedies for one problem: throw the contents away, or label them.

Cache size should be small enough that, and large enough that:

- a. The overall average cost per bit is close to that of main memory alone, and the overall average access time is close to that of the cache alone
- b. The average cost per bit is close to that of the cache alone, and the average access time is close to that of main memory alone
- c. Both cost and access time match main memory
- d. Both cost and access time match the cache

✓ The overall average cost per bit is close to that of main memory alone, and the overall average access time is close to that of the cache alone

Correct. Pay like main memory, perform like cache.

✗ The average cost per bit is close to that of the cache alone, and the average access time is close to that of main memory alone

Wrong, and it swaps the two, which would give the worst of both.

✗ Both cost and access time match main memory

Wrong. Matching main memory's access time would make the cache pointless.

✗ Both cost and access time match the cache

Wrong. Matching the cache's cost per bit would be unaffordable.

Pay like main memory, perform like cache. Which side each of the two words attaches to is the whole trick of this sentence.

The cache should be small enough that the average cost per bit is close to that of _____ alone, and large enough that the average access time is close to that of _____ alone.

BLANK 1

- a. main memory
- b. the cache
- c. the registers
- d. secondary storage

BLANK 2

- a. the cache
- b. main memory
- c. the registers
- d. secondary storage

Answer: **main memory, the cache**

Cost like main memory, speed like the cache. Cache is dear and fast, so being small keeps the cost down and being large keeps the hit rate up.

Why do large caches tend to be slightly slower than small ones?

- a. The larger the cache, the more complex the addressing logic
- b. Larger caches use cheaper memory technology
- c. Larger caches must be flushed more often
- d. Larger caches have a lower hit rate

✓ The larger the cache, the more complex the addressing logic

Correct. More complex addressing logic.

✗ Larger caches use cheaper memory technology

Wrong. The technology is not what changes with size here.

✗ Larger caches must be flushed more often

Wrong. Flushing relates to virtual caches and context switches.

✗ Larger caches have a lower hit rate

Wrong. A larger cache generally has a better hit rate, which is why anyone would want one.

More complex addressing logic. It is a real limit on how far the size cure can be taken, and available chip and board area limits it further.

9 Performance, RISC and CISC

9.1

EASY

The real test 2, question 23

Variable-length instruction formats are commonly associated with:

- a. CISC architectures
- b. SIMD processors
- c. RISC architectures
- d. Harvard architecture

✓ CISC architectures

Correct. Variable length is a defining CISC trait, and x86 is the standing example.

✗ SIMD processors

Wrong. SIMD is about operating on many data elements at once, which is independent of instruction length.

✗ RISC architectures

Wrong, and the direct opposite. RISC uses FIXED-length instructions.

✗ Harvard architecture

Wrong. Harvard is about separate instruction and data memories, not about encoding.

CISC packs many operations of differing complexity into one instruction set, so instructions vary in length. RISC fixes the length to make decoding and pipelining simple.

9.2

EASY

RISC S3

A processor is defined as:

- a. The logic circuitry that responds to and processes the basic instructions that drive a computer
- b. The set of instructions a programmer can give to a machine
- c. The memory that holds the instructions
- d. The interface between hardware and software

✓ The logic circuitry that responds to and processes the basic instructions that drive a computer

Correct. Logic circuitry that responds to and processes the basic instructions.

✗ The set of instructions a programmer can give to a machine

Wrong. That is the instruction set, defined two lines later on the same slide.

✗ The memory that holds the instructions

Wrong. Instructions are stored in memory, but memory is not the processor.

✗ The interface between hardware and software

Wrong. That is the ISA.

Logic circuitry that responds to and processes the basic instructions. It is hardware, not a set of instructions and not an interface.

9.3

EASY

RISC S3

A processor is the _____ circuitry that responds to and processes the basic instructions that drive a computer.

- a. logic
- b. memory
- c. control
- d. arithmetic

Answer: **logic**

Logic circuitry. The control unit and the ALU are both parts of it.

9.4**EASY**

RISC S3

CPU instructions are numbers stored in memory.

Answer: **true**

They are, which is the stored program idea again: an instruction is just a bit pattern, and the instruction format slide shows one.

9.5**EASY**

RISC S3

The basic operations are: read instruction from memory, decode, execute, ____ .

- a. write back
- b. branch
- c. stall
- d. forward

Answer: **write back**

Write back. In the five-stage pipeline a memory access stage sits between execute and write back.

9.6

EASY

RISC S4

What do CISC and RISC stand for?

- a. Complex Instruction Set Computer and Reduced Instruction Set Computer
- b. Compact Instruction Set Computer and Rapid Instruction Set Computer
- c. Complex Integrated System Computer and Reduced Integrated System Computer
- d. Controlled Instruction Sequence Computer and Registered Instruction Sequence Computer

✓ Complex Instruction Set Computer and Reduced Instruction Set Computer

Correct. Complex and Reduced.

✗ Compact Instruction Set Computer and Rapid Instruction Set Computer

Wrong. Compact and Rapid are not the words.

✗ Complex Integrated System Computer and Reduced Integrated System Computer

Wrong. The middle words are Instruction Set.

✗ Controlled Instruction Sequence Computer and Registered Instruction Sequence Computer

Wrong on every word but Computer.

Complex and Reduced. The words describe the instruction set, and the deck later stresses that reduced means reduced work per instruction rather than merely fewer instructions.

9.7

EASY

RISC S4

Over the years _____ major processor design philosophies emerged.

- a. 2
- b. 3
- c. 4
- d. 5

Answer: 2

Two: CISC and RISC. They were developed to address different challenges in processor design and performance optimization.

9.8

EASY

RISC S5

Engineers attempted to reduce program size by creating processors with powerful instructions capable of performing complex tasks in a single instruction, and this led to CISC.

Answer: **true**

That is the deck's account of CISC's origin: expensive memory made small programs valuable, so instructions were made to do more.

9.9

EASY

RISC S6

The observation that simpler instructions could be executed much faster led to the development of RISC architectures.

Answer: **true**

It did, together with the finding that most programs use only a small subset of instructions.

9.10

EASY

RISC S7

The first in-class question in the RISC and CISC deck asks:

- a. Why were CISC architectures initially popular?
- b. Why is pipelining more difficult in CISC processors?
- c. Why do we not use only cache memory?
- d. What is the primary focus of computer architecture?

✓ Why were CISC architectures initially popular?

Correct. It follows the historical background, and its answer slide comes next.

✗ Why is pipelining more difficult in CISC processors?

Wrong. That is the second in-class question in this deck, much later.

✗ Why do we not use only cache memory?

Wrong. That belongs to the memory deck.

✗ What is the primary focus of computer architecture?

Wrong. That belongs to the introduction deck.

It follows the historical background, and its answer slide comes next.

9.11

EASY

RISC S9

CISC existed close to the beginning of general computing, when the earliest machines were programmed in assembly language and memory was slow and expensive.

Answer: **true**

That is the deck's framing. Assembly programming by hand is what made a hundred powerful instructions preferable to thousands of simple ones.

9.12

EASY

RISC S9

The deck's example of what a CISC processor has built in is:

- a. The ability to calculate a square root
- b. The ability to sort an array
- c. The ability to draw a rectangle
- d. The ability to translate an address

✓ The ability to calculate a square root

Correct. A square root, so the compiler does not have to emit a long sequence of machine instructions for it.

✗ The ability to sort an array

Wrong. Sorting is not mentioned.

✗ The ability to draw a rectangle

Wrong. Rectangles belong to the vector graphics section.

✗ The ability to translate an address

Wrong. Address translation is the MMU's job.

A square root, so the compiler does not have to emit a long sequence of machine instructions for it.

9.13

EASY

RISC S10

In CISC, some instructions take more time than others.

Answer: **true**

They do, and the deck flags it as the problem. It comes back later as the reason CISC instructions are harder to pipeline.

9.14

EASY

RISC S11

Match each kind of device to the architecture the deck says it typically uses.

Computers

CISC

Tablets and smartphones

RISC

It lines up with the ISA example in the introduction deck, where mobile apps are built for the ARM-based processors in smartphones.

9.15

EASY

RISC S11

In CISC, instructions are executed by microcode.

Answer: **true**

They are, and the deck elsewhere adds that microcode control makes the CISC instruction set possible and flexible.

9.16

EASY

RISC S12

A CISC processor includes some instructions that perform specialized tasks and are used infrequently.

Answer: **true**

It does, and that observation is what the 1980s researchers seized on: most programs use only a small subset of the available instructions.

9.17

EASY

RISC S13

CISC has a _____ instruction set, some simple, some very complex.

- a. richer
- b. reduced
- c. shorter
- d. uniform

Answer: **richer**

Richer. Reduced is its opposite, and the R in RISC.

9.18

EASY

RISC S13

CISC instructions generally take:

- a. More than 1 clock to execute
- b. Exactly 1 clock to execute
- c. Less than 1 clock to execute
- d. A fixed 5 clocks to execute

✓ More than 1 clock to execute

Correct. More than one clock, against RISC's one instruction per clock cycle.

✗ Exactly 1 clock to execute

Wrong, and that is the RISC property.

✗ Less than 1 clock to execute

Wrong. No instruction takes less than a clock.

✗ A fixed 5 clocks to execute

Wrong. The point is that CISC timing is not fixed.

More than one clock, against RISC's one instruction per clock cycle. That single number is behind most of the comparison table.

9.19

EASY

RISC S13

CISC instructions generally take more than 1 clock to execute.

Answer: **true**

They do, which is why the comparison table says CISC is slower since an instruction can take more than 1 cycle.

9.20

EASY

RISC S15

Complex instructions are _____ used by programmers and compilers.

- a. infrequently
- b. frequently
- c. exclusively
- d. always

Answer: **infrequently**

Infrequently, which is the disadvantage that gave RISC its opening.

9.21

EASY

RISC S15

The deck lists as a CISC disadvantage that the instruction set and chip become more complex with each generation of computers.

Answer: **true**

It does, and it is a compounding problem: every generation must keep the old instructions and add new ones.

9.22

EASY

RISC S18

The deck concedes that RISC requires more instructions, but adds that:

- a. Each executes very quickly
- b. Each takes more than one clock
- c. Memory usage falls
- d. The compiler becomes simpler

✓ Each executes very quickly

Correct. More instructions, each much faster.

✗ Each takes more than one clock

Wrong, and that is the CISC property.

✗ Memory usage falls

Wrong. More instructions means more memory, which is the code expansion disadvantage.

✗ The compiler becomes simpler

Wrong. RISC pushes complexity to the compiler, so it becomes harder.

More instructions, each much faster. That is the whole RISC bargain, and code expansion is the price it pays.

9.23

EASY

RISC S18

RISC needs fewer instructions than CISC for the same operation.

Answer: **false**

More instructions, not fewer. The deck says although more instructions are required, each executes very quickly.

9.24

EASY

RISC S19

RISC utilizes a small, highly-optimized set of instructions rather than a more specialized set.

Answer: **true**

Small and optimized. The specialized instructions CISC accumulated are exactly the ones that turned out to be rarely used.

9.25

EASY

RISC S20

The RISC concept is stated as:

- a. Break an operation into simpler sub operations
- b. Encode the intention directly
- c. Match the instruction set to high-level language constructs
- d. Use microcode to interpret complex instructions

✓ Break an operation into simpler sub operations

Correct. Break the operation down, as in load X, load Y, add X and Y, store Z.

✗ Encode the intention directly

Wrong, and that is the CISC concept on the earlier slide.

✗ Match the instruction set to high-level language constructs

Wrong. That is a CISC advantage about micro-program instruction sets.

✗ Use microcode to interpret complex instructions

Wrong. Microcode is the CISC mechanism.

Break the operation down, as in load X, load Y, add X and Y, store Z. Encoding the intention directly is the CISC concept.

9.26

EASY

RISC S21

RISC uses shorter instructions, breaking a complex instruction into several short simpler instructions.

Answer: **true**

It does, and the MULT A, B example shows exactly that breaking down into four instructions.

9.27

EASY

RISC S22

Registers can transfer information to other registers much faster than information can be transferred to and from memory.

Answer: **true**

They can, and it is the reason a load-store architecture pays: get the data into registers once, then work there.

9.28

EASY

RISC S23

Among the listed properties of a RISC processor, how long do instructions take to execute?

- a. One clock cycle
- b. More than one clock cycle
- c. Five clock cycles
- d. It varies with the instruction

✓ One clock cycle

Correct. One clock cycle, property 2.

✗ More than one clock cycle

Wrong, and that is the CISC property.

✗ Five clock cycles

Wrong. Five is the number of pipeline stages, not the cycles per instruction at steady state.

✗ It varies with the instruction

Wrong. Uniform timing is the RISC property; variation is CISC's.

One clock cycle, property 2. It contrasts directly with CISC's more than 1 clock.

9.29

EASY

RISC S23

The first listed property of a RISC processor is simple ____ instructions and addressing modes.

- a. primitive
- b. complex
- c. variable
- d. microcoded

Answer: **primitive**

Simple primitive instructions. Primitive here means basic, not crude.

9.30

EASY

RISC S24

RISC processors can be designed more quickly than corresponding CISC processors.

Answer: **true**

They can, which the deck calls a shorter design cycle and lists as the third advantage of RISC.

9.31

EASY

RISC S29

The second in-class question in the RISC and CISC deck asks:

- a. Why is pipelining more difficult in CISC processors?
- b. Why were CISC architectures initially popular?
- c. Why do we not use only cache memory?
- d. What does computer organization describe?

✓ Why is pipelining more difficult in CISC processors?

Correct. It comes after the five-stage pipeline slides, and its answer follows immediately.

✗ Why were CISC architectures initially popular?

Wrong. That is the first in-class question in this deck.

✗ Why do we not use only cache memory?

Wrong. That belongs to the memory deck.

✗ What does computer organization describe?

Wrong. That belongs to the introduction deck.

It comes after the five-stage pipeline slides, and its answer follows immediately.

9.32

EASY

RISC S32

CISC has more complex instructions than RISC.

Answer: **true**

More complex, and also fewer of them, which is the pair of claims that trips people up: more complex but fewer in number.

9.33

EASY

RISC S34

In the comparison table, where does each architecture place its emphasis?

- a. CISC on hardware, RISC on software
- b. CISC on software, RISC on hardware
- c. Both on hardware
- d. Both on software

✓ CISC on hardware, RISC on software

Correct. CISC hardware, RISC software.

✗ CISC on software, RISC on hardware

Wrong, and reversed.

✗ Both on hardware

Wrong. The table contrasts them.

✗ Both on software

Wrong, for the same reason.

CISC hardware, RISC software. It is the first row of the table, and it summarises where the complexity is made to live.

9.34

EASY

RISC S34

In the comparison table, why is RISC faster?

- a. Because its instructions usually take 1 instruction cycle, while a CISC instruction can take more than 1
- b. Because it has a higher clock frequency
- c. Because it has fewer instructions in its set
- d. Because it uses microcode

✓ Because its instructions usually take 1 instruction cycle, while a CISC instruction can take more than 1

Correct. One cycle against more than one.

✗ Because it has a higher clock frequency

Wrong. The performance comparison assumes the same clock rates.

✗ Because it has fewer instructions in its set

Wrong. Set size is a separate matter and not the reason given here.

✗ Because it uses microcode

Wrong. RISC uses hardwired control.

| One cycle against more than one. The table states the reason in the same row as the claim.

9.35

EASY

RISC S34

The comparison table says CISC is slower since an instruction can take more than 1 cycle.

Answer: **true**

| It does, and RISC's entry on the same row is faster since instructions usually take 1 instruction cycle.

9.36

EASY

RISC S34

The main objective of RISC is ____ .

- a. speed
- b. less code
- c. compatibility
- d. flexibility

Answer: **speed**

Speed. CISC's is less code.

9.37

EASY

RISC S35

The second comparison table describes CISC as more hardware oriented.

Answer: **true**

It does, matching the first table's emphasis on hardware for CISC.

9.38

EASY

RISC S35

In the second comparison table, how is instruction size described for each?

- a. CISC mostly varied in size, RISC always a set size
- b. CISC always a set size, RISC mostly varied
- c. Both a set size
- d. Both varied

✓ CISC mostly varied in size, RISC always a set size

Correct. Varied against set.

✗ CISC always a set size, RISC mostly varied

Wrong, and reversed.

✗ Both a set size

Wrong. CISC uses variable-length instruction formats.

✗ Both varied

Wrong. RISC instruction size is always a set size.

Varied against set. A set size is what makes the RISC instruction easy to decode and easy to pipeline.

9.39

EASY

RISC S35

In the table, RISC instruction size is always a _____ size.

- a. set
- b. varied
- c. short
- d. long

Answer: **set**

A set size, which the properties list calls uniform length and fixed instruction format.

9.40

EASY

RISC S35

In the second comparison table, addressing modes are described as:

- a. Complex in CISC, simple in RISC
- b. Simple in CISC, complex in RISC
- c. Complex in both
- d. Simple in both

✓ Complex in CISC, simple in RISC

Correct. Complex against simple, which agrees with the earlier figures: CISC typically has 5 to 20 modes, RISC relatively few.

✗ Simple in CISC, complex in RISC

Wrong, and reversed.

✗ Complex in both

Wrong. RISC uses only a few simple modes.

✗ Simple in both

Wrong. CISC modes can be complex.

Complex against simple, which agrees with the earlier figures: CISC typically has 5 to 20 modes, RISC relatively few.

Suppose Processor X executes fewer instructions than Processor Y for the same program, but Processor Y completes execution faster due to better pipelining. Which statement is most accurate?

- a. Processor X has a superior ISA.
- b. Microarchitecture significantly affects performance.
- c. ISA alone determines performance.
- d. Instruction count is irrelevant.

X Processor X has a superior ISA.

Wrong. A superior ISA that finishes LAST is a strange claim, and instruction count alone does not make an ISA better.

✓ Microarchitecture significantly affects performance.

Correct. Pipelining is microarchitecture, and here it outweighed the instruction count advantage.

X ISA alone determines performance.

Wrong, and directly contradicted: the two differ in pipelining, which is not the ISA.

X Instruction count is irrelevant.

Wrong, too strong. Instruction count is a real term in the performance equation, it simply was not decisive here.

Fewer instructions did not win, because Y's better pipelining cut its CPI. Instruction count and CPI trade against each other, and this is the classic RISC against CISC comparison.

A processor implements speculative execution and branch prediction. The primary objective of these techniques is to:

- a. Increase storage capacity.
- b. Reduce pipeline stalls caused by control hazards.
- c. Reduce transistor count.
- d. Replace cache memory.

✗ Increase storage capacity.

Wrong. Neither technique stores anything for the program. They are about timing, not capacity.

✓ Reduce pipeline stalls caused by control hazards.

Correct, and note the exact phrase CONTROL HAZARD, which is the hazard branches cause.

✗ Reduce transistor count.

Wrong, and backwards. Prediction tables and speculation logic ADD transistors.

✗ Replace cache memory.

Wrong. Cache addresses data delay, branch prediction addresses control delay. They are different problems.

A branch is a control hazard: the pipeline does not know which instruction follows until the branch resolves. Predicting and running ahead speculatively keeps the pipeline full.

An embedded processor minimizes silicon area and power consumption by reducing instruction complexity and relying heavily on compiler optimization. This design reflects:

- a. SIMD architecture
- b. RISC philosophy
- c. VLIW architecture
- d. CISC philosophy

✗ SIMD architecture

Wrong. SIMD adds wide vector units for data parallelism, which costs area rather than saving it.

✓ RISC philosophy

Correct. Reduced instruction complexity plus compiler reliance is RISC by definition.

✗ VLIW architecture

Wrong, though it is the closest decoy: VLIW also leans on the compiler, but it does so to issue several operations at once, not to minimise area and power.

✗ CISC philosophy

Wrong, and the opposite. CISC puts the complexity in the HARDWARE to make the compiler's job easier.

Simple instructions, simple hardware, and complexity pushed into the compiler. That trade is the RISC philosophy in one sentence.

A computer architect wants to reduce compiler complexity by designing instructions that closely resemble high-level programming constructs. Which ISA philosophy best supports this objective?

- a. Stack-based ISA
- b. Harvard Architecture
- c. RISC
- d. CISC

✗ Stack-based ISA

Wrong. A stack ISA simplifies operand addressing, but it does not aim to mirror high-level constructs.

✗ Harvard Architecture

Wrong. Harvard describes the memory organization, not the instruction philosophy.

✗ RISC

Wrong, and the opposite. RISC deliberately keeps instructions simple and makes the COMPILER work harder.

✓ CISC

Correct. Complex instructions resembling high-level constructs, which is where the semantic gap argument comes from.

Closing the semantic gap between high-level languages and machine instructions was the original CISC goal, so one instruction could do the work of a whole statement.

9.45

MEDIUM

RISC S3

According to the RISC deck, an instruction set is:

- a. A set of instructions a programmer can give to a machine to perform operations, specific to the CPU architecture
- b. The logic circuitry that executes instructions
- c. The numbers stored in memory during execution
- d. The four basic operations of a processor

✓ A set of instructions a programmer can give to a machine to perform operations, specific to the CPU architecture

Correct. A set of instructions a programmer can give, and the deck adds that the instructions are specific to the CPU architecture, which is why software is tied to an ISA.

✗ The logic circuitry that executes instructions

Wrong. That is the processor.

✗ The numbers stored in memory during execution

Wrong. Those are the instructions themselves, once loaded.

✗ The four basic operations of a processor

Wrong. Read, decode, execute, write back are the basic operations.

A set of instructions a programmer can give, and the deck adds that the instructions are specific to the CPU architecture, which is why software is tied to an ISA.

9.46

MEDIUM

RISC S3

The instructions of a machine are specific to the CPU ____ .

- a. architecture
- b. organization
- c. clock
- d. manufacturer

Answer: **architecture**

Architecture. That is why an application compiled for x86 will not run on ARM.

Select the basic operations of a processor the deck lists.

- a. Read instruction from memory
- b. Decode
- c. Execute
- d. Write back
- e. Branch

✓ Read instruction from memory

Correct, reading the instruction from memory is first.

✓ Decode

Correct, decode is second.

✓ Execute

Correct, execute is third.

✓ Write back

Correct, write back closes the list.

✗ Branch

Wrong. Branching is a kind of instruction, not one of the basic operations of executing any instruction.

Four operations, and they line up with four of the five stages of the RISC pipeline: fetch, decode, execute, write back.

During the 1960s and 1970s, what was the relationship between memory and processor hardware costs?

- a. Memory was expensive while processor hardware was relatively cheap
- b. Memory was cheap while processor hardware was expensive
- c. Both were expensive
- d. Both were cheap

✓ Memory was expensive while processor hardware was relatively cheap

Correct. Expensive memory, cheap hardware.

✗ Memory was cheap while processor hardware was expensive

Wrong, and reversed, which would have argued for RISC instead.

✗ Both were expensive

Wrong. The contrast between the two is the point.

✗ Both were cheap

Wrong, for the same reason.

Expensive memory, cheap hardware. That imbalance is the whole reason CISC made sense: spend transistors to save memory.

9.49

MEDIUM

RISC S5

During the _____ and 1970s memory was expensive while processor hardware was relatively _____ .

BLANK 1

- a. 1960s
- b. 1950s
- c. 1970s
- d. 1980s

BLANK 2

- a. cheap
- b. expensive
- c. slow
- d. scarce

Answer: **1960s, cheap**

The 1960s, and cheap hardware. By the early 1980s the reasoning had reversed and RISC followed.

9.50

MEDIUM

RISC S6

By the early 1980s researchers discovered that most programs:

- a. Used only a small subset of the available instructions
- b. Used almost every available instruction
- c. Were limited by memory size
- d. Spent most of their time in input and output

✓ Used only a small subset of the available instructions

Correct. A small subset, which made the elaborate rest of a CISC instruction set look like wasted silicon.

✗ Used almost every available instruction

Wrong, and it would have been an argument for keeping CISC.

✗ Were limited by memory size

Wrong. Memory cost was the 1960s and 1970s problem.

✗ Spent most of their time in input and output

Wrong. I/O is not what the observation was about.

A small subset, which made the elaborate rest of a CISC instruction set look like wasted silicon.

9.51

MEDIUM

RISC S6

By the early _____ researchers discovered that most programs used only a small subset of the available instructions.

- a. 1980s
- b. 1970s
- c. 1960s
- d. 1990s

Answer: **1980s**

The early 1980s, which is also when the first RISC projects were running at IBM, Stanford and UC-Berkeley.

Which observation about instruction timing led to RISC?

- a. Complex instructions often required multiple clock cycles while simpler instructions executed much faster
- b. All instructions took the same time
- c. Simple instructions required multiple clock cycles
- d. Clock speed could not be increased further

✓ Complex instructions often required multiple clock cycles while simpler instructions executed much faster

Correct. Complex instructions cost many cycles, simple ones few.

✗ All instructions took the same time

Wrong, and uniform timing is a RISC goal rather than a 1980s observation.

✗ Simple instructions required multiple clock cycles

Wrong, and reversed.

✗ Clock speed could not be increased further

Wrong. Clock speed is one of three performance factors but is not this observation.

Complex instructions cost many cycles, simple ones few. Combined with the small subset finding, that is the entire case for RISC.

Why were CISC architectures initially popular?

- a. Because memory was expensive, and CISC instructions could perform complex operations using fewer instructions, reducing program size
- b. Because they were faster than RISC at the same clock rate
- c. Because they pipelined more easily
- d. Because they used fewer transistors

✓ Because memory was expensive, and CISC instructions could perform complex operations using fewer instructions, reducing program size

Correct. Expensive memory plus fewer instructions equals smaller programs.

✗ Because they were faster than RISC at the same clock rate

Wrong. RISC achieves 2 to 4 times the performance at comparable clock rates.

✗ Because they pipelined more easily

Wrong, and the opposite: varying lengths and execution times make CISC harder to pipeline.

✗ Because they used fewer transistors

Wrong. Simple hardware and less chip space are RISC advantages.

Expensive memory plus fewer instructions equals smaller programs. Every clause of the answer slide is doing work.

9.54

MEDIUM

RISC S8

CISC was initially popular because memory was _____, and CISC instructions could perform complex operations using fewer instructions, thereby reducing program _____.

BLANK 1

- a. expensive
- b. cheap
- c. slow
- d. volatile

BLANK 2

- a. size
- b. time
- c. complexity
- d. cost

Answer: **expensive, size**

Expensive memory, reduced program size. The cost of memory is what made size worth paying for.

9.55

MEDIUM

RISC S9

IBM designed an instruction set to allow early programmers to easily program:

- a. A hundred complex instructions rather than thousands of individual instructions
- b. Thousands of complex instructions rather than a hundred individual ones
- c. A hundred individual instructions rather than a thousand complex ones
- d. Ten complex instructions rather than a hundred individual ones

✓ A hundred complex instructions rather than thousands of individual instructions

Correct. A hundred complex against thousands of individual.

✗ Thousands of complex instructions rather than a hundred individual ones

Wrong, and reversed.

✗ A hundred individual instructions rather than a thousand complex ones

Wrong. The hundred are the complex ones.

✗ Ten complex instructions rather than a hundred individual ones

Wrong figures. A hundred complex against thousands of individual.

A hundred complex against thousands of individual. The trade is fewer, richer instructions for the human writing them.

9.56

MEDIUM

RISC S9

_____ designed an instruction set to let early programmers write a hundred complex instructions rather than thousands of individual ones.

- a. IBM
- b. Intel
- c. AMD
- d. Motorola

Answer: **IBM**

IBM, which also appears later on the other side of the argument, as the home of the IBM 801, one of the first RISC projects.

9.57

MEDIUM

RISC S9

In CISC, most of the burden of generating machine instructions is shifted to the compiler.

Answer: **false**

The other way round. CISC shifts the burden to the processor, which is what makes compiler development simpler.

9.58

MEDIUM

RISC S10

Complex instruction set computing is a CPU design where single instructions can:

- a. Execute several low-level operations, such as a load, an arithmetic operation and a store
- b. Execute only one low-level operation each
- c. Execute only register-to-register operations
- d. Be decoded in a single cycle

✓ Execute several low-level operations, such as a load, an arithmetic operation and a store

Correct, and the deck names load, arithmetic and store as the example trio.

✗ Execute only one low-level operation each

Wrong. That is closer to the RISC philosophy.

✗ Execute only register-to-register operations

Wrong. Register-to-register only is the RISC restriction.

✗ Be decoded in a single cycle

Wrong. Easy decoding is an important aspect of the RISC instruction format.

Several low-level operations in one instruction, or multi-step operations, or addressing modes within single instructions.

9.59

MEDIUM

RISC S10

A CISC instruction may execute several low-level operations, such as a _____ from memory, an arithmetic operation, and a memory _____ .

BLANK 1

- a. load
- b. store
- c. branch
- d. shift

BLANK 2

- a. store
- b. load
- c. fetch
- d. compare

Answer: **load, store**

Load, then arithmetic, then store, all inside one instruction. In RISC those would be three separate instructions.

9.60

MEDIUM

RISC S10

CISC is called complex because of:

- a. The complex work performed per instruction
- b. The complexity of its compiler
- c. The number of registers it has
- d. The difficulty of manufacturing it

✓ The complex work performed per instruction

Correct. Work per instruction.

✗ The complexity of its compiler

Wrong. The CISC compiler is the simpler one.

✗ The number of registers it has

Wrong. CISC has a small number of general purpose registers, but that is not where the name comes from.

✗ The difficulty of manufacturing it

Wrong. Manufacturing difficulty is not the reason given.

Work per instruction. It matches RISC being called reduced because of the reduction of work performed by an instruction.

9.61

MEDIUM

RISC S10

The CISC concept is to encode the _____ directly, as in add X and Y and put the result in Z.

- a. intention
- b. address
- c. operand
- d. opcode

Answer: **intention**

The intention. You say what you want done, and the processor works out the steps.

The problem the deck identifies with CISC is that:

- a. Some instructions take more time than others
- b. All instructions take the same time
- c. It has too many registers
- d. Its instructions are too short

✓ Some instructions take more time than others

Correct. Uneven timing, which is exactly what makes pipelining hard: a pipeline wants every stage to take the same time.

✗ All instructions take the same time

Wrong. Uniform timing is a RISC property and an advantage.

✗ It has too many registers

Wrong. CISC has a small number of general purpose registers.

✗ Its instructions are too short

Wrong. CISC instructions are variable in length, and often long.

Uneven timing, which is exactly what makes pipelining hard: a pipeline wants every stage to take the same time.

9.63

MEDIUM

RISC S10

Which architectures does the deck give as examples of CISC?

- a. x86 and s390
- b. ARM and MIPS
- c. IBM 801 and Berkeley RISC 1
- d. DLX and MIPS

✓ x86 and s390

Correct. X86 and s390.

✗ ARM and MIPS

Wrong. ARM is the smartphone RISC example and MIPS came out of Stanford's RISC project.

✗ IBM 801 and Berkeley RISC 1

Wrong. Both are named as early RISC projects.

✗ DLX and MIPS

Wrong. Both are RISC.

| x86 and s390. All the other options are RISC machines.

9.64

MEDIUM

RISC S10

CISC has a _____ number of general purpose registers.

- a. small
- b. large
- c. variable
- d. fixed

Answer: **small**

| Small. RISC leans the other way, since almost all its instructions work register to register and it needs plenty of them.

According to the deck, which devices typically use CISC and which use RISC?

- a. Computers typically use CISC, while tablets, smartphones and other devices use RISC
- b. Computers typically use RISC, while tablets and smartphones use CISC
- c. Both use CISC
- d. Both use RISC

✓ Computers typically use CISC, while tablets, smartphones and other devices use RISC
Correct. Computers CISC, mobile devices RISC.

✗ Computers typically use RISC, while tablets and smartphones use CISC
Wrong, and reversed.

✗ Both use CISC
Wrong. The slide draws a contrast.

✗ Both use RISC
Wrong, for the same reason.

Computers CISC, mobile devices RISC. The reason follows immediately: RISC is more efficient where cycles and power are in short supply.

9.66

MEDIUM

RISC S11

Why does the higher efficiency of RISC make it desirable in tablets and smartphones?

- a. Because cycles and power are usually in short supply in these applications
- b. Because memory is expensive in these applications
- c. Because their compilers are simpler
- d. Because they need variable-length instructions

✓ Because cycles and power are usually in short supply in these applications

Correct. Cycles and power.

✗ Because memory is expensive in these applications

Wrong. Expensive memory is the 1960s and 1970s argument for CISC.

✗ Because their compilers are simpler

Wrong. RISC pushes complexity to the compiler, making it harder rather than simpler.

✗ Because they need variable-length instructions

Wrong. Variable-length instructions are a CISC characteristic.

| Cycles and power. It is the same idea as battery life in the smartphone architecture example.

9.67

MEDIUM

RISC S11

RISC is desirable where _____ and _____ are usually in short supply.

BLANK 1

- a. cycles
- b. registers
- c. instructions
- d. addresses

BLANK 2

- a. power
- b. memory
- c. cache
- d. bandwidth

Answer: **cycles, power**

| Cycles and power, the two things a battery-driven device cannot spare.

In CISC, instructions are executed by:

- a. Microcode
- b. Hardwired control
- c. The compiler
- d. The operating system

✓ Microcode

Correct. Microcode, and RISC uses hardwired control instead.

✗ Hardwired control

Wrong, and that is the RISC property.

✗ The compiler

Wrong. The compiler produces instructions; it does not execute them.

✗ The operating system

Wrong. The operating system is software running on the machine.

Microcode, and RISC uses hardwired control instead. That single contrast explains a great deal about both designs.

A CISC instruction set typically includes many instructions with different _____ and execution _____, which makes CISC instructions harder to pipeline.

BLANK 1

- a. sizes
- b. names
- c. opcodes
- d. operands

BLANK 2

- a. cycles
- b. stages
- c. modes
- d. formats

Answer: **sizes, cycles**

Sizes and cycles. The answer slide to the second in-class question says the same thing in other words: varying lengths and execution times.

9.70

MEDIUM

RISC S12

A CISC processor has a large number of instructions, typically:

- a. From 100 to 250
- b. From 5 to 20
- c. From 250 to 500
- d. From 50 to 100

✓ From 100 to 250

Correct. 100 to 250 instructions.

✗ From 5 to 20

Wrong, and that is the addressing mode figure.

✗ From 250 to 500

Wrong. The upper bound given is 250.

✗ From 50 to 100

Wrong. The lower bound given is 100.

100 to 250 instructions. Do not confuse it with 5 to 20, which is the number of addressing modes on the very next line.

9.71

MEDIUM

RISC S12

A CISC processor typically has from 100 to _____ instructions.

- a. 250
- b. 200
- c. 300
- d. 150

Answer: **250**

250 is the top of the range, and 100 is the bottom.

A CISC processor has a large variety of addressing modes, typically:

- a. 5 to 20 different modes
- b. 100 to 250 different modes
- c. 2 to 5 different modes
- d. 20 to 50 different modes

✓ 5 to 20 different modes

Correct. 5 to 20.

✗ 100 to 250 different modes

Wrong, and that is the instruction count.

✗ 2 to 5 different modes

Wrong. The range given starts at 5 and ends at 20.

✗ 20 to 50 different modes

Wrong. 20 is the top of the range, not the bottom.

5 to 20. The pair of figures on this slide, 100 to 250 instructions and 5 to 20 modes, are easily swapped under pressure.

9.73

MEDIUM

RISC S12

Select the characteristics of CISC processors from this list.

- a. Variable-length instruction formats
- b. Instructions that manipulate operands in memory
- c. Uniform length instructions
- d. Memory access limited to load and store

✓ Variable-length instruction formats

Correct, variable-length instruction formats.

✓ Instructions that manipulate operands in memory

Correct, instructions that manipulate operands in memory.

✗ Uniform length instructions

Wrong. Uniform length and fixed format are RISC properties.

✗ Memory access limited to load and store

Wrong. Restricting memory access to load and store is the RISC rule.

Variable length and memory operands are CISC. The other two are RISC properties, and they are the exact opposites.

9.74

MEDIUM

RISC S12

CISC uses _____ -length instruction formats.

- a. variable
- b. fixed
- c. uniform
- d. word

Answer: **variable**

Variable. It saves memory, since a simple instruction need not be padded to the size of a complex one, but it complicates decoding and pipelining.

Among the properties of a CISC processor, how is its instruction set described?

- a. Richer, some simple and some very complex, with instructions of a variable size
- b. Simple and primitive, with instructions of a fixed size
- c. Uniform in length and easy to decode
- d. Limited to register-to-register operations

✓ Richer, some simple and some very complex, with instructions of a variable size

Correct. Richer and mixed, with variable size.

✗ Simple and primitive, with instructions of a fixed size

Wrong. Simple primitive instructions is the first RISC property.

✗ Uniform in length and easy to decode

Wrong. Easy decoding is an important aspect of RISC.

✗ Limited to register-to-register operations

Wrong. Register-to-register is the RISC restriction.

Richer and mixed, with variable size. Everything else in the list follows from that mixture.

9.76

MEDIUM

RISC S13

Among the listed properties of a CISC processor, what does the deck say about pipelining?

- a. No pipelining
- b. Pipelining, in five stages
- c. Pipelining, but only for load and store
- d. Pipelining is optional

✓ No pipelining

Correct. No pipelining, listed as property 5.

✗ Pipelining, in five stages

Wrong. The five-stage pipeline is the RISC one.

✗ Pipelining, but only for load and store

Wrong. No such qualification is given.

✗ Pipelining is optional

Wrong. The property is stated flatly as no pipelining.

No pipelining, listed as property 5. RISC lists pipelining as one of its own properties, so the two lists are mirror images.

9.77

MEDIUM

RISC S13

A CISC processor interfaces with memory in multiple mechanisms with ____ addressing modes.

- a. complex
- b. simple
- c. fixed
- d. relative

Answer: **complex**

Complex addressing modes. RISC has simple ones, and the comparison table states that contrast directly.

9.78

MEDIUM

RISC S13

According to the listed properties, microcode control makes the CISC instruction set:

- a. Possible and flexible
- b. Faster and smaller
- c. Fixed and uniform
- d. Easier to pipeline

✓ Possible and flexible

Correct. Possible and flexible.

✗ Faster and smaller

Wrong. Microcode adds a layer of interpretation and does not make things faster.

✗ Fixed and uniform

Wrong. Flexibility is the opposite of fixed.

✗ Easier to pipeline

Wrong. CISC is harder to pipeline, not easier.

Possible and flexible. Without microcode, every complex instruction would have to be wired in by hand.

9.79

MEDIUM

RISC S13

CISC works well with a simpler compiler.

Answer: **true**

It does, which is the seventh listed property and matches the earlier claim that CISC was developed to make compiler development simpler.

9.80

MEDIUM

RISC S14

The deck says microprogramming is as easy as assembly language to implement, and:

- a. Much less expensive than hardwiring a control unit
- b. Much more expensive than hardwiring a control unit
- c. About the same cost as hardwiring
- d. Only possible on RISC machines

✓ Much less expensive than hardwiring a control unit

Correct. Much less expensive than hardwiring.

✗ Much more expensive than hardwiring a control unit

Wrong, and reversed.

✗ About the same cost as hardwiring

Wrong. The deck asserts a clear cost advantage.

✗ Only possible on RISC machines

Wrong. Hardwired control is the RISC property; microprogramming belongs to CISC.

| Much less expensive than hardwiring. Cost, not speed, is the advantage claimed.

9.81

MEDIUM

RISC S14

Microprogramming is much less expensive than _____ a control unit.

- a. hardwiring
- b. pipelining
- c. compiling
- d. microcoding

Answer: **hardwiring**

| Hardwiring. RISC accepts that expense in exchange for speed.

9.82

MEDIUM

RISC S14

Making each CISC instruction more capable meant fewer instructions were needed to implement a given task.

Answer: **true**

| Fewer, more powerful instructions, which is the same trade the historical background describes as reducing program size.

9.83

MEDIUM

RISC S14

Micro-program instruction sets can be written to match the constructs of _____ languages.

- a. high-level
- b. low-level
- c. assembly
- d. machine

Answer: **high-level**

High-level languages. The closer the instruction set is to the language, the less translation the compiler has to do.

9.84

MEDIUM

RISC S15

The first listed disadvantage of CISC is that complex instructions are:

- a. Infrequently used by programmers and compilers
- b. Frequently used but slow
- c. Impossible to microcode
- d. Too short to be useful

✓ Infrequently used by programmers and compilers

Correct. Infrequently used.

✗ Frequently used but slow

Wrong. The complaint is that they are not used, not that they are used and slow.

✗ Impossible to microcode

Wrong. Microcode is what makes them possible.

✗ Too short to be useful

Wrong. They are long, not short.

Infrequently used. All that silicon sits idle most of the time, which is precisely the 1980s finding.

9.85

MEDIUM

RISC S15

What does the deck say about memory references, loads and stores, in CISC?

- a. They are slow and account for a significant fraction of all instructions
- b. They are fast and rare
- c. They are forbidden except in load and store instructions
- d. They are handled entirely by microcode without touching memory

✓ They are slow and account for a significant fraction of all instructions

Correct. Slow and common, which is a bad combination.

✗ They are fast and rare

Wrong on both counts.

✗ They are forbidden except in load and store instructions

Wrong, and that is the RISC restriction, not a CISC disadvantage.

✗ They are handled entirely by microcode without touching memory

Wrong. Microcode sequences the work; it does not remove the memory access.

| Slow and common, which is a bad combination. RISC's answer is to make them explicit and few.

9.86

MEDIUM

RISC S15

In CISC, memory references such as loads and stores account for a significant fraction of all instructions.

Answer: **true**

| They do, and they are slow, which is why the deck lists this as a disadvantage.

9.87

MEDIUM

RISC S15

Procedure and function calls are a major bottleneck: passing _____, storing and retrieving values in _____.

BLANK 1

- a. arguments
- b. addresses
- c. opcodes
- d. results

BLANK 2

- a. registers
- b. memory
- c. cache
- d. the stack

Answer: **arguments, registers**

Arguments, and registers. Both halves are pure overhead around the useful work of the call.

9.88

MEDIUM

RISC S16

How does the deck's CISC example multiply 5 by 10?

- a. Mov ax,10 then Mov bx,5 then Mul bx
- b. Mov ax,0 then Mov bx,10 then Mov cx,5 then a loop of Add ax,bx
- c. LOAD R1, A then LOAD R2, B then MUL R3, R1, R2 then STORE R3, C
- d. PUSH 5 then PUSH 10 then MUL

✓ Mov ax,10 then Mov bx,5 then Mul bx

Correct. Three instructions, and one of them is a multiply.

✗ Mov ax,0 then Mov bx,10 then Mov cx,5 then a loop of Add ax,bx

Wrong. That is the RISC example, which multiplies by repeated addition in a loop.

✗ LOAD R1, A then LOAD R2, B then MUL R3, R1, R2 then STORE R3, C

Wrong. That is the RISC philosophy example, breaking MULT A, B into four instructions.

✗ PUSH 5 then PUSH 10 then MUL

Wrong. That is a zero-address stack sequence from the instruction formats topic.

Three instructions, and one of them is a multiply. The processor knows how to multiply, so the program does not have to.

9.89

MEDIUM

RISC S16

The CISC example takes _____ instructions to multiply 5 by 10.

- a. 3
- b. 4
- c. 5
- d. 6

Answer: **3**

Three: two moves and a multiply. The RISC version needs a loop instead.

9.90

MEDIUM

RISC S17

Where did the first RISC projects come from, and when?

- a. IBM, Stanford and UC-Berkeley, in the late 70s and early 80s
- b. Intel, AMD and Motorola, in the late 80s
- c. IBM alone, in the 1960s
- d. Stanford and MIT, in the early 90s

✓ IBM, Stanford and UC-Berkeley, in the late 70s and early 80s

Correct. Three institutions, late 70s and early 80s.

✗ Intel, AMD and Motorola, in the late 80s

Wrong. Intel and AMD appear in the pipelining deck, as users of pipelining.

✗ IBM alone, in the 1960s

Wrong. IBM was one of three, and the period is the late 70s and early 80s.

✗ Stanford and MIT, in the early 90s

Wrong. MIT is not named, and the period is wrong.

Three institutions, late 70s and early 80s. IBM appears on both sides of the story, having designed the CISC instruction set earlier.

9.91

MEDIUM

RISC S17

The first RISC projects came from IBM, Stanford and UC- ____ .

- a. Berkeley
- b. Davis
- c. Irvine
- d. Stanford

Answer: **Berkeley**

UC-Berkeley, home of Berkeley RISC 1 and 2.

Select the machines the deck names as designed with the philosophy that became known as RISC.

- a. The IBM 801
- b. Stanford MIPS
- c. Berkeley RISC 1 and 2
- d. The PDP-1

✓ The IBM 801

Correct, from IBM.

✓ Stanford MIPS

Correct, from Stanford.

✓ Berkeley RISC 1 and 2

Correct, from UC-Berkeley.

✗ The PDP-1

Wrong. The PDP-1 is named among the old one's complement computers.

Three machines, one from each institution. The PDP-1 appears in the data representation deck, as a one's complement machine.

9.93

MEDIUM

RISC S17

Select the three main factors that affect processor performance.

- a. How fast you can crank up the clock
- b. How much work you can do per cycle
- c. How many instructions you need to perform a task
- d. How much memory the machine has

✓ How fast you can crank up the clock

Correct, clock speed is the first factor.

✓ How much work you can do per cycle

Correct, work per cycle is the second.

✓ How many instructions you need to perform a task

Correct, instruction count is the third, and it is the one CISC targets.

✗ How much memory the machine has

Wrong. Memory size is not one of the three factors named.

Clock rate, work per cycle, instruction count. They are the same three terms as the CPU time equation, and CISC and RISC attack different ones.

9.94

MEDIUM

RISC S17

The deck names _____ main factors that affect processor performance.

- a. 3
- b. 2
- c. 4
- d. 5

Answer: **3**

Three. RISC improves the second and third; CISC concentrates on the third alone.

9.95

MEDIUM

RISC S18

Instead of the single complex instruction **MULT A, B**, how does RISC break the operation down?

- a. LOAD R1, A then LOAD R2, B then MUL R3, R1, R2 then STORE R3, C
- b. Mov ax,10 then Mov bx,5 then Mul bx
- c. PUSH A then PUSH B then MUL then POP C
- d. LOAD A then MUL B then STORE C

✓ **LOAD R1, A then LOAD R2, B then MUL R3, R1, R2 then STORE R3, C**
Correct. Four instructions: two loads, a register-to-register multiply, and a store.

✗ **Mov ax,10 then Mov bx,5 then Mul bx**
Wrong. That is the CISC example of multiplying 5 by 10.

✗ **PUSH A then PUSH B then MUL then POP C**
Wrong. That is a zero-address stack sequence.

✗ **LOAD A then MUL B then STORE C**
Wrong. That is a one-address accumulator sequence.

Four instructions: two loads, a register-to-register multiply, and a store. Memory is touched only by the load and store.

9.96

MEDIUM

RISC S18

RISC breaks **MULT A, B** into _____ simple instructions.

- a. 4
- b. 3
- c. 2
- d. 5

Answer: **4**

Four: LOAD, LOAD, MUL, STORE.

9.97

MEDIUM

RISC S19

RISC is called reduced because of:

- a. The reduction of work performed by an instruction
- b. The reduction in the number of registers
- c. The reduction in memory size
- d. The reduction in clock speed

✓ The reduction of work performed by an instruction

Correct, and this is the distinction most often got wrong.

✗ The reduction in the number of registers

Wrong. RISC tends to have more registers, not fewer.

✗ The reduction in memory size

Wrong. RISC programs are larger, not smaller.

✗ The reduction in clock speed

Wrong. Nothing about the clock is reduced.

Work per instruction, not count of instructions. It mirrors CISC being called complex because of the complex work performed per instruction.

9.98

MEDIUM

RISC S19

RISC is called reduced because of the reduction of _____ performed by an instruction.

- a. work
- b. instructions
- c. registers
- d. cycles

Answer: **work**

Work. A common error is to answer instructions, which describes a consequence rather than the reason for the name.

RISC is a microprocessor architecture that utilizes:

- a. A small, highly-optimized set of instructions
- b. A large, specialized set of instructions
- c. A variable-length instruction format
- d. Microcoded complex instructions

✓ A small, highly-optimized set of instructions

Correct. Small and highly-optimized, rather than the more specialized set found in other architectures.

✗ A large, specialized set of instructions

Wrong, and that describes CISC.

✗ A variable-length instruction format

Wrong. RISC uses a fixed instruction length.

✗ Microcoded complex instructions

Wrong. RISC uses hardwired control.

Small and highly-optimized, rather than the more specialized set found in other architectures.

9.100

MEDIUM

RISC S19

The small instruction set of a typical RISC processor consists mostly of:

- a. Register-to-register operations, with only simple load and store operations for memory access
- b. Memory-to-memory operations
- c. Operations on the top of a stack
- d. Accumulator operations against memory

✓ Register-to-register operations, with only simple load and store operations for memory access

Correct. Register to register, with load and store as the only memory instructions.

✗ Memory-to-memory operations

Wrong, and memory-to-memory is the CISC pattern in the comparison table.

✗ Operations on the top of a stack

Wrong. Stack operations belong to the zero-address format.

✗ Accumulator operations against memory

Wrong. That is the one-address, accumulator machine.

Register to register, with load and store as the only memory instructions. That is the load-store architecture in a sentence.

9.101

MEDIUM

RISC S19

A RISC instruction set consists mostly of _____ operations, with only simple load and store for memory access.

- a. register-to-register
- b. memory-to-memory
- c. stack-to-stack
- d. cache-to-cache

Answer: **register-to-register**

Register-to-register. Only load and store touch memory at all.

9.102

MEDIUM

RISC S20

Select what the deck says about operands and results in RISC.

- a. Each operand is brought into a processor register with a load instruction
- b. All computations are done among the data stored in processor registers
- c. Results are transferred to memory by means of store instructions
- d. Arithmetic instructions may read operands directly from memory

✓ Each operand is brought into a processor register with a load instruction

Correct, load brings operands in.

✓ All computations are done among the data stored in processor registers

Correct, computation happens only among registers.

✓ Results are transferred to memory by means of store instructions

Correct, store writes results back.

✗ Arithmetic instructions may read operands directly from memory

Wrong. That is a CISC characteristic: instructions that manipulate operands in memory.

Load in, compute in registers, store out. Reading operands directly from memory is exactly what a load-store architecture forbids.

9.103

MEDIUM

RISC S20

In RISC, results are transferred to memory by means of ____ instructions.

- a. store
- b. load
- c. move
- d. push

Answer: **store**

Store. Load brings values in and store sends them out, and nothing else touches memory.

9.104

MEDIUM

RISC S21

Why does RISC use only a few addressing modes?

- a. Because almost all its instructions have simple register addressing
- b. Because its instructions are variable in length
- c. Because it has no memory instructions
- d. Because microcode restricts them

✓ Because almost all its instructions have simple register addressing

Correct. Almost everything is register addressed, so there is little for other modes to do.

✗ Because its instructions are variable in length

Wrong. RISC instructions are of fixed length.

✗ Because it has no memory instructions

Wrong. Load and store are memory instructions.

✗ Because microcode restricts them

Wrong. RISC uses hardwired control, not microcode.

Almost everything is register addressed, so there is little for other modes to do. The deck adds that other modes such as immediate operands may still be included.

9.105

MEDIUM

RISC S21

RISC may still include other addressing modes, such as immediate operands.

Answer: **true**

It may. Few does not mean one, and the deck says other addressing modes may be included.

9.106

MEDIUM

RISC S21

By using a relatively simple instruction format, what becomes possible in RISC?

- a. The instruction length can be fixed and aligned on word boundaries, and the format is easy to decode
- b. The instruction length can vary to save memory
- c. Instructions can carry three memory addresses
- d. Instructions can be interpreted by microcode

✓ The instruction length can be fixed and aligned on word boundaries, and the format is easy to decode

Correct. Fixed length, word aligned, easy to decode.

✗ The instruction length can vary to save memory

Wrong. Variable length is a CISC characteristic.

✗ Instructions can carry three memory addresses

Wrong. Three-address formats exist, but memory addresses in arithmetic instructions are what RISC avoids.

✗ Instructions can be interpreted by microcode

Wrong. Microcode belongs to CISC.

Fixed length, word aligned, easy to decode. Easy decoding is what the deck calls an important aspect of the RISC instruction format.

9.107

MEDIUM

RISC S21

An important aspect of the RISC instruction format is that it is easy to ____ .

- a. decode
- b. write
- c. compress
- d. extend

Answer: **decode**

Easy to decode, which keeps the ID stage of the pipeline short and predictable.

9.108

MEDIUM

RISC S22

Pipelining overlaps the fetch, decode and execute phases of _____ or three instructions.

- a. 2
- b. 3
- c. 4
- d. 5

Answer: **2**

Two or three, as the deck puts it. The full five-stage RISC pipeline can hold five at once.

9.109

MEDIUM

RISC S22

The advantage of register storage over memory storage is that:

- a. Registers can transfer information to other registers much faster than to and from memory
- b. Registers hold more data than memory
- c. Registers are cheaper per bit
- d. Registers are non-volatile

✓ Registers can transfer information to other registers much faster than to and from memory

Correct. Speed of register-to-register transfer, which is why RISC computes only among registers.

✗ Registers hold more data than memory

Wrong. Registers are the smallest storage in the machine.

✗ Registers are cheaper per bit

Wrong. They are the most expensive per bit.

✗ Registers are non-volatile

Wrong. Registers lose their contents when power is removed.

Speed of register-to-register transfer, which is why RISC computes only among registers.

9.110

MEDIUM

RISC S22

Select the further characteristics of RISC processors listed on this slide.

- a. Relatively few instructions
- b. Relatively few addressing modes
- c. Memory access limited to load and store instructions
- d. Can run several instructions simultaneously
- e. Variable-length instruction formats

✓ Relatively few instructions

Correct, relatively few instructions.

✓ Relatively few addressing modes

Correct, relatively few addressing modes.

✓ Memory access limited to load and store instructions

Correct, memory access limited to load and store.

✓ Can run several instructions simultaneously

Correct, and it follows from pipelining.

✗ Variable-length instruction formats

Wrong. RISC instruction size is always a set size.

Four characteristics. Variable-length formats are a CISC characteristic and the opposite of RISC's fixed length.

9.111

MEDIUM

RISC S22

In RISC, memory access is limited to _____ and store instructions.

- a. load
- b. move
- c. push
- d. add

Answer: **load**

Load and store. This is the property that gives the load-store architecture its name.

Among the RISC properties, how do instructions interface with memory?

- a. Via fixed mechanisms, load and store
- b. Via multiple mechanisms with complex addressing modes
- c. Only through the cache
- d. Only through microcode

✓ Via fixed mechanisms, load and store

Correct. Fixed mechanisms, meaning load and store.

✗ Via multiple mechanisms with complex addressing modes

Wrong, and that is CISC property 4.

✗ Only through the cache

Wrong. The cache is transparent and is not an instruction-level mechanism.

✗ Only through microcode

Wrong. RISC uses hardwired control.

Fixed mechanisms, meaning load and store. The alternative offered is word for word the CISC property.

9.113

MEDIUM

RISC S23

RISC uses _____ length instructions and a _____ instruction format.

BLANK 1

- a. uniformed
- b. variable
- c. compressed
- d. extended

BLANK 2

- a. fixed
- b. variable
- c. flexible
- d. microcoded

Answer: **uniformed, fixed**

Uniform length and fixed format, exactly as the property list has it. Both make decoding and pipelining straightforward.

9.114

MEDIUM

RISC S23

Which pair of properties closes the RISC list?

- a. Hardwired control, and complexity pushed to the compiler
- b. Microcode control, and complexity pushed to the processor
- c. No pipelining, and a simpler compiler
- d. Variable-length instructions, and complex addressing modes

✓ Hardwired control, and complexity pushed to the compiler

Correct. Hardwired control and complexity in the compiler.

✗ Microcode control, and complexity pushed to the processor

Wrong, and both halves are the CISC positions.

✗ No pipelining, and a simpler compiler

Wrong. RISC lists pipelining as one of its properties.

✗ Variable-length instructions, and complex addressing modes

Wrong. Both are CISC characteristics.

Hardwired control and complexity in the compiler. Every clause is the mirror image of a CISC property.

9.115

MEDIUM

RISC S23

In RISC, complexity is pushed to the ____ .

- a. compiler
- b. processor
- c. microcode
- d. programmer

Answer: **compiler**

The compiler. CISC does the reverse, shifting the burden of generating machine instructions to the processor.

9.116

MEDIUM

RISC S24

Because the instruction set of a RISC processor is so simple:

- a. It uses up much less chip space and has simple hardware requirements
- b. It uses more chip space but runs cooler
- c. It needs a larger microcode store
- d. It needs fewer registers

✓ It uses up much less chip space and has simple hardware requirements

Correct. Less chip space and simpler hardware.

✗ It uses more chip space but runs cooler

Wrong. It uses less space, not more.

✗ It needs a larger microcode store

Wrong. RISC is hardwired and has no microcode store.

✗ It needs fewer registers

Wrong. RISC leans on registers heavily.

Less chip space and simpler hardware. The space saved is what pays for the transistors each instruction gets.

9.117

MEDIUM

RISC S24

Because the RISC instruction set is so simple, it uses up much less _____ space.

- a. chip
- b. memory
- c. cache
- d. register

Answer: **chip**

Chip space. Available chip and board area also turns up as a limit on cache size in the memory deck.

9.118

MEDIUM

RISC S24

The third advantage of RISC is a shorter design cycle, because:

- a. Being simpler than corresponding CISC processors, they can be designed more quickly
- b. They are designed by compilers
- c. They reuse CISC designs
- d. They have no control unit

✓ Being simpler than corresponding CISC processors, they can be designed more quickly
Correct. Simpler to design, so quicker to design, and the deck adds that they can complete their work in 1 clock cycle.

✗ They are designed by compilers
Wrong. Compilers generate code; they do not design chips.

✗ They reuse CISC designs
Wrong. Nothing about reuse is claimed.

✗ They have no control unit
Wrong. Every processor needs a control unit; RISC's is hardwired.

Simpler to design, so quicker to design, and the deck adds that they can complete their work in 1 clock cycle.

9.119

MEDIUM

RISC S25

The performance of a RISC processor depends greatly on the _____ that it is executing.

- a. code
- b. clock
- c. cache
- d. data

Answer: **code**

The code. The deck's heading for this disadvantage is Code Quality.

9.120

MEDIUM

RISC S25

Why can code expansion be a problem for RISC?

- a. Because CISC machines perform complex actions with a single instruction where RISC may require multiple instructions
- b. Because RISC instructions are longer than CISC instructions
- c. Because RISC uses variable-length instructions
- d. Because RISC has too few registers

✓ Because CISC machines perform complex actions with a single instruction where RISC may require multiple instructions

Correct. More instructions for the same action means a bigger program.

✗ Because RISC instructions are longer than CISC instructions

Wrong. Individual RISC instructions are of a fixed, ordinary size; it is the count that grows.

✗ Because RISC uses variable-length instructions

Wrong. Variable length is CISC.

✗ Because RISC has too few registers

Wrong. Register count is not the issue here.

More instructions for the same action means a bigger program. It is the direct cost of the philosophy of breaking operations down.

9.121

MEDIUM

RISC S25

Code expansion is listed as a disadvantage of RISC.

Answer: **true**

It is the second of the three RISC disadvantages, after code quality and before system design.

9.122

MEDIUM

RISC S25

The large memory cache on a RISC chip itself is known as a _____ -level cache.

- a. first
- b. second
- c. third
- d. virtual

Answer: **first**

First-level cache, which the memory deck calls L1.

9.123

MEDIUM

RISC S26

In the RISC instruction example, how could a compiler multiply a by b?

- a. By adding a to itself b times, in a loop
- b. By calling a built-in multiply instruction
- c. By shifting a left b times
- d. By pushing both onto a stack and multiplying

✓ By adding a to itself b times, in a loop

Correct. Repeated addition in a loop, because the designers guaranteed that add executes in one clock.

✗ By calling a built-in multiply instruction

Wrong. A built-in multiply is what the CISC example uses.

✗ By shifting a left b times

Wrong. Shifting multiplies by powers of 2, not by an arbitrary b.

✗ By pushing both onto a stack and multiplying

Wrong. That is the zero-address stack approach.

Repeated addition in a loop, because the designers guaranteed that add executes in one clock. The program is Mov ax,0 then Mov bx,10 then Mov cx,5 then a Begin label with Add ax,bx and Loop Begin.

9.124

MEDIUM

RISC S26

In the RISC example the designers make sure that _____ executes in one clock.

- a. add
- b. mul
- c. load
- d. loop

Answer: **add**

Add. Guaranteeing one clock for the simple operation is what makes building the complex one out of it worthwhile.

9.125

MEDIUM

RISC S30

Why is pipelining more difficult in CISC processors?

- a. Because instructions have varying lengths and execution times, making it harder to process them uniformly through the pipeline stages
- b. Because CISC has too few instructions
- c. Because CISC uses hardwired control
- d. Because CISC has too many registers

✓ Because instructions have varying lengths and execution times, making it harder to process them uniformly through the pipeline stages

Correct. Varying lengths and execution times.

✗ Because CISC has too few instructions

Wrong. CISC has more instructions, typically 100 to 250.

✗ Because CISC uses hardwired control

Wrong. Hardwired control is RISC's; CISC uses microcode.

✗ Because CISC has too many registers

Wrong. CISC has a small number of general purpose registers.

Varying lengths and execution times. A pipeline advances every stage together, so instructions that do not take the same time do not fit it.

9.126

MEDIUM

RISC S30

Pipelining is harder in CISC because instructions have varying _____ and execution _____ .

BLANK 1

- a. lengths
- b. opcodes
- c. operands
- d. names

BLANK 2

- a. times
- b. modes
- c. stages
- d. orders

Answer: **lengths, times**

Lengths and times. It is the same claim as the earlier slide about different sizes and execution cycles.

9.127

MEDIUM

RISC S32

In the comparison, instructions utilize how many cycles in CISC compared with RISC?

- a. More cycles in CISC than in RISC
- b. Fewer cycles in CISC than in RISC
- c. The same number in both
- d. It cannot be compared

✓ More cycles in CISC than in RISC

Correct. More cycles per instruction in CISC, and CISC also has more complex instructions than RISC.

✗ Fewer cycles in CISC than in RISC

Wrong. CISC instructions generally take more than 1 clock; RISC aims at exactly 1.

✗ The same number in both

Wrong. The whole comparison rests on the difference.

✗ It cannot be compared

Wrong. The deck compares them directly.

More cycles per instruction in CISC, and CISC also has more complex instructions than RISC.

9.128

MEDIUM

RISC S32

CISC implementations tend to be _____ than RISC implementations.

- a. slower
- b. faster
- c. cheaper
- d. smaller

Answer: **slower**

Slower, which follows from taking more than one cycle per instruction.

9.129

MEDIUM

RISC S32

The deck says RISC design is approximately how cost-effective compared with CISC?

- a. Approximately twice as cost-effective
- b. Approximately four times as cost-effective
- c. About the same
- d. Half as cost-effective

✓ Approximately twice as cost-effective

Correct. Approximately twice.

✗ Approximately four times as cost-effective

Wrong. Four is the top of the performance range, not the cost-effectiveness figure.

✗ About the same

Wrong. A clear advantage is claimed.

✗ Half as cost-effective

Wrong direction.

Approximately twice. Do not confuse it with the 2 to 4 times performance figure, which is a different claim.

9.130

MEDIUM

RISC S32

RISC design is approximately _____ as cost-effective as CISC.

- a. twice
- b. three times
- c. four times
- d. half

Answer: **twice**

Twice. The performance claim on the same topic is 2 to 4 times, so keep the two apart.

9.131

MEDIUM

RISC S34

From the comparison table, match each architecture to where it places its emphasis.

CISC

Emphasis on hardware

RISC

Emphasis on software

The first row of the table. It pairs with the last row of the second table, where CISC is more hardware oriented and RISC more software oriented since the compiler deals with translations.

9.132

MEDIUM

RISC S34

In the comparison table, RISC has:

- a. Single-clock, reduced instruction only
- b. Multi-clock, complex instructions
- c. Multi-clock, reduced instructions
- d. Single-clock, complex instructions

✓ Single-clock, reduced instruction only

Correct. Single-clock and reduced only, against CISC's multi-clock complex instructions.

✗ Multi-clock, complex instructions

Wrong. That is the CISC entry.

✗ Multi-clock, reduced instructions

Wrong. RISC is single-clock.

✗ Single-clock, complex instructions

Wrong. RISC instructions are reduced, not complex.

Single-clock and reduced only, against CISC's multi-clock complex instructions.

9.133

MEDIUM

RISC S34

In the comparison table, CISC includes _____ -clock, complex instructions.

- a. multi
- b. single
- c. half
- d. dual

Answer: **multi**

Multi-clock. RISC's entry is single-clock, reduced instruction only.

9.134

MEDIUM

RISC S34

What are the main objectives of CISC and RISC in the comparison table?

- a. CISC: less code. RISC: speed
- b. CISC: speed. RISC: less code
- c. Both: speed
- d. Both: less code

✓ CISC: less code. RISC: speed
Correct. Less code against speed.

✗ CISC: speed. RISC: less code
Wrong, and reversed.

✗ Both: speed
Wrong. Less code is CISC's objective.

✗ Both: less code
Wrong. RISC accepts code expansion in exchange for speed.

Less code against speed. It is the single line that best summarises the whole topic, and it traces straight back to the cost of memory in the 1960s.

In the second comparison table, why is RISC described as more software oriented?

- a. Because the compiler deals with translations
- b. Because it runs more software
- c. Because its instructions are written in a high-level language
- d. Because it has no hardware control unit

✓ Because the compiler deals with translations

Correct. The compiler does the translating, which is the same point as complexity pushed to the compiler.

✗ Because it runs more software

Wrong. The amount of software it runs is not the criterion.

✗ Because its instructions are written in a high-level language

Wrong. Instructions are machine instructions in either architecture.

✗ Because it has no hardware control unit

Wrong. RISC has a hardwired control unit.

| The compiler does the translating, which is the same point as complexity pushed to the compiler.

A computer architect argues that processor performance should no longer be evaluated solely by clock frequency. Which of the following provides the strongest justification for this argument?

- a. Modern processors execute exactly one instruction per clock cycle.
- b. Processor performance also depends on CPI (Cycles Per Instruction), Instruction-Level Parallelism (ILP), cache performance, branch prediction accuracy, and memory latency.
- c. Clock frequency completely determines instruction execution time.
- d. Higher clock frequency eliminates cache misses.

X Modern processors execute exactly one instruction per clock cycle.

Wrong. Superscalar processors issue several instructions per cycle, and stalls can push it below one. "Exactly one" is false.

✓ Processor performance also depends on CPI (Cycles Per Instruction), Instruction-Level Parallelism (ILP), cache performance, branch prediction accuracy, and memory latency.
Correct. It names the other terms in the performance equation and the factors that drive CPI.

X Clock frequency completely determines instruction execution time.

Wrong, and it argues the OPPOSITE of the architect's case.

X Higher clock frequency eliminates cache misses.

Wrong. A faster clock does not change how often data is found in cache. If anything a miss costs MORE cycles at a higher frequency.

Execution time is instruction count times CPI divided by clock rate. Frequency is one term of three, and CPI itself depends on cache, branches and parallelism.

A processor executes a benchmark requiring 12 billion instructions, operating at 4 GHz, with an average CPI of 1.5. What is the approximate execution time?

- a. 2.5 seconds
- b. 4.5 seconds
- c. 5.5 seconds
- d. 3.5 seconds

✗ 2.5 seconds

Wrong. No correct combination of the three figures gives 2.5.

✓ 4.5 seconds

Correct. 18 billion cycles at 4 billion cycles per second is 4.5 seconds.

✗ 5.5 seconds

Wrong. Too high. Recheck the division by 4, not by 3.

✗ 3.5 seconds

Wrong. Note that dropping the CPI gives $12 / 4 = 3$ seconds, which is not offered, so this option catches a half-remembered formula rather than that specific slip.

CPU time = (instruction count x CPI) / clock rate. Cycles needed = $12 \times 10^9 \times 1.5 = 18 \times 10^9$. Divide by 4×10^9 cycles per second to get 4.5 seconds.

CISC was developed to make compiler development simpler. How?

- a. It shifts most of the burden of generating machine instructions to the processor
- b. It shifts most of the burden to the compiler
- c. It removes the need for a compiler
- d. It requires the compiler to schedule every instruction

✓ It shifts most of the burden of generating machine instructions to the processor

Correct. The burden moves to the processor.

✗ It shifts most of the burden to the compiler

Wrong, and that is the RISC position.

✗ It removes the need for a compiler

Wrong. A compiler is still needed; it is simply simpler.

✗ It requires the compiler to schedule every instruction

Wrong. Instruction scheduling is a RISC compiler's job, and poor scheduling is a RISC disadvantage.

The burden moves to the processor. RISC does the reverse, which is why the deck later says complexity is pushed to the compiler in RISC.

What about a CISC instruction set makes its instructions harder to pipeline?

- a. It includes many instructions with different sizes and execution cycles
- b. It includes too few instructions
- c. Its instructions are all the same length
- d. It has too many registers

✓ It includes many instructions with different sizes and execution cycles

Correct. Different sizes and different cycle counts.

✗ It includes too few instructions

Wrong. CISC has more instructions, typically 100 to 250.

✗ Its instructions are all the same length

Wrong, and uniform length is a RISC property that helps pipelining.

✗ It has too many registers

Wrong. CISC has a small number of general purpose registers.

Different sizes and different cycle counts. A pipeline works best when every instruction is the same shape and takes the same time.

9.140**HARD**

RISC S12

A CISC processor typically has from _____ to 250 instructions and from 5 to _____ addressing modes.

BLANK 1

- a. 100
- b. 5
- c. 50
- d. 20

BLANK 2

- a. 20
- b. 250
- c. 50
- d. 10

Answer: **100, 20**

100 to 250 instructions, 5 to 20 modes. Both ranges are worth memorising as a pair, because a question about one usually offers the other as a distractor.

As each CISC instruction became more capable, fewer instructions were needed for a task. What did that achieve?

- a. More efficient use of the relatively slow main memory
- b. More efficient use of the register file
- c. Simpler pipelining
- d. Fewer clock cycles per instruction

✓ More efficient use of the relatively slow main memory

Correct. Fewer instructions means fewer instruction fetches, and fetches from slow memory were the bottleneck of the era.

✗ More efficient use of the register file

Wrong. CISC has few general purpose registers.

✗ Simpler pipelining

Wrong. Capable, variable instructions make pipelining harder.

✗ Fewer clock cycles per instruction

Wrong. Complex instructions take more cycles each, not fewer.

Fewer instructions means fewer instruction fetches, and fetches from slow memory were the bottleneck of the era.

Because micro-program instruction sets can be written to match the constructs of high-level languages:

- a. The compiler does not have to be as complicated
- b. The compiler has to be more complicated
- c. No compiler is needed
- d. The processor can be simpler

✓ The compiler does not have to be as complicated

Correct. A simpler compiler, because the gap between the language and the machine is narrower.

✗ The compiler has to be more complicated

Wrong, and that is the RISC position.

✗ No compiler is needed

Wrong. A compiler is still required.

✗ The processor can be simpler

Wrong. The complexity moves into the processor, not out of it.

| A simpler compiler, because the gap between the language and the machine is narrower.

9.143

HARD

RISC S15

The deck calls procedure and function calls a major CISC bottleneck. Why?

- a. Because of passing arguments and storing and retrieving values in registers
- b. Because they cannot be microcoded
- c. Because they always miss in the cache
- d. Because they require variable-length instructions

✓ Because of passing arguments and storing and retrieving values in registers

Correct. Argument passing and the saving and restoring of registers around the call.

✗ Because they cannot be microcoded

Wrong. Microcode is not the issue here.

✗ Because they always miss in the cache

Wrong. Cache behaviour is not what the slide names.

✗ Because they require variable-length instructions

Wrong. Variable-length instructions are a separate characteristic.

Argument passing and the saving and restoring of registers around the call. All of it is overhead that computes nothing.

9.144

HARD

RISC S17

Match each early RISC machine to where it came from.

IBM 801

IBM

MIPS

Stanford

RISC 1 and 2

UC-Berkeley

Three projects, three homes. Berkeley's machines are the ones that gave the philosophy its name.

What was RISC's original goal?

- a. To limit the number of instructions on the chip so that each could be allocated enough transistors to execute in one cycle
- b. To fit as many instructions on the chip as possible
- c. To reduce the size of compiled programs
- d. To allow instructions to access memory directly

✓ To limit the number of instructions on the chip so that each could be allocated enough transistors to execute in one cycle

Correct. Fewer instructions so each gets more transistors, and therefore one-cycle execution.

✗ To fit as many instructions on the chip as possible

Wrong, and that is closer to the CISC habit.

✗ To reduce the size of compiled programs

Wrong. Smaller programs is the CISC objective; RISC accepts code expansion.

✗ To allow instructions to access memory directly

Wrong. RISC restricts memory access to load and store.

Fewer instructions so each gets more transistors, and therefore one-cycle execution. The transistor budget is the hinge of the whole argument.

9.146

HARD

RISC S19

RISC's original goal was to limit the number of instructions on the chip so that each could be allocated enough _____ to make it execute in _____ cycle.

BLANK 1

- a. transistors
- b. registers
- c. bits
- d. cycles

BLANK 2

- a. one
- b. two
- c. five
- d. four

Answer: **transistors, one**

Transistors, and one cycle. Fewer instructions each get a bigger share of a fixed chip.

How does RISC achieve the ability to execute one instruction per clock cycle?

- a. By overlapping the fetch, decode and execute phases of two or three instructions, using pipelining
- b. By raising the clock frequency
- c. By using microcode
- d. By reducing the number of registers

✓ By overlapping the fetch, decode and execute phases of two or three instructions, using pipelining

Correct, and the deck names the procedure as pipelining.

✗ By raising the clock frequency

Wrong. Clock frequency is a separate performance factor.

✗ By using microcode

Wrong. Microcode is the CISC mechanism.

✗ By reducing the number of registers

Wrong. RISC benefits from having plenty of registers.

Pipelining. Each instruction still takes several cycles from start to finish, but one completes every cycle.

RISC processors often achieve how much of the performance of CISC processors, using comparable semiconductor technology and the same clock rates?

- a. 2 to 4 times
- b. 1.5 times
- c. 10 times
- d. Half

✓ 2 to 4 times

Correct. 2 to 4 times.

✗ 1.5 times

Wrong. The deck's figure starts at 2.

✗ 10 times

Wrong. The deck's figure stops at 4.

✗ Half

Wrong direction: RISC is the faster of the two.

2 to 4 times. The qualifications matter: comparable semiconductor technology and the same clock rates, so the gain is architectural.

9.149

HARD

RISC S24

RISC processors often achieve _____ to _____ times the performance of CISC processors at comparable technology and clock rates.

BLANK 1

- a. 2
- b. 1
- c. 3
- d. 4

BLANK 2

- a. 4
- b. 3
- c. 5
- d. 10

Answer: **2, 4**

2 to 4. Note the separate claim that RISC design is approximately twice as cost-effective as CISC, which is a different figure about a different thing.

The code quality disadvantage of RISC is that if the programmer or compiler schedules instructions poorly:

- a. The processor can spend time stalling, waiting for the result of one instruction before proceeding with a subsequent one
- b. The processor runs the instructions out of order automatically
- c. The program will not compile
- d. The cache will be flushed

✓ The processor can spend time stalling, waiting for the result of one instruction before proceeding with a subsequent one

Correct. Stalling, which is exactly the data hazard of the pipelining topic.

✗ The processor runs the instructions out of order automatically

Wrong. Nothing in this course gives the processor that power; the pipelining deck rearranges instructions by hand.

✗ The program will not compile

Wrong. Poor scheduling is legal, merely slow.

✗ The cache will be flushed

Wrong. Flushing relates to virtual caches and context switches.

Stalling, which is exactly the data hazard of the pipelining topic. Because complexity was pushed to the compiler, a poor compiler costs real performance.

RISC machines require very fast memory systems to feed their instructions. What do RISC-based systems therefore typically contain?

- a. Large memory caches on the chip itself, known as a first-level cache
- b. Larger main memory than CISC systems
- c. A microcode store
- d. Separate instruction and data memories

✓ Large memory caches on the chip itself, known as a first-level cache

Correct. Large on-chip caches, and the deck names them the first-level cache.

✗ Larger main memory than CISC systems

Wrong. Main memory size is not what the deck says changes.

✗ A microcode store

Wrong. RISC is hardwired and has no microcode.

✗ Separate instruction and data memories

Wrong. That is the Harvard architecture, a separate idea.

Large on-chip caches, and the deck names them the first-level cache. A machine that retires an instruction every cycle must be fed one every cycle.

In the comparison list, CISC typically has how many instructions relative to RISC?

- a. Fewer instructions than RISC
- b. More instructions than RISC
- c. The same number
- d. Exactly twice as many

✓ Fewer instructions than RISC

Correct as the deck states it: CISC typically has fewer instructions than RISC.

✗ More instructions than RISC

Wrong here, though it is true of the size of the instruction set, which is the distinction to keep clear.

✗ The same number

Wrong. A difference is asserted.

✗ Exactly twice as many

Wrong. No such ratio is given.

Fewer, according to this comparison slide, since each CISC instruction does more and a task therefore needs fewer of them. It is a claim about instructions executed, and it sits beside the separate fact that a CISC instruction SET holds 100 to 250 instructions.

RISC architectures are designed for a good cost to performance ratio, whereas CISC architectures are designed for:

- a. Good performance on slow memories
- b. Good performance on fast memories
- c. Low power consumption
- d. Easy pipelining

✓ Good performance on slow memories

Correct. Slow memories, which is the whole 1960s and 1970s setting: expensive, slow memory made small programs worth having.

✗ Good performance on fast memories

Wrong. It is RISC that needs very fast memory systems to feed its instructions.

✗ Low power consumption

Wrong. Low power favours RISC, which is why smartphones use it.

✗ Easy pipelining

Wrong. CISC is harder to pipeline, not designed for it.

Slow memories, which is the whole 1960s and 1970s setting: expensive, slow memory made small programs worth having.

9.154

HARD

RISC S32

RISC architectures are designed for a good _____ ratio, whereas CISC architectures are designed for good performance on _____ memories.

BLANK 1

- a. cost/performance
- b. power/weight
- c. size/speed
- d. hit/miss

BLANK 2

- a. slow
- b. fast
- c. cheap
- d. large

Answer: **cost/performance, slow**

Cost against performance for RISC, slow memories for CISC. Each design answers the constraint of its own era.

In the comparison table, how do the two architectures treat LOAD and STORE?

- a. CISC is memory-to-memory with LOAD and STORE incorporated in instructions; RISC is register-to-register with LOAD and STORE as independent instructions
- b. CISC has them as independent instructions; RISC incorporates them
- c. Both incorporate them into other instructions
- d. Both keep them independent

✓ CISC is memory-to-memory with LOAD and STORE incorporated in instructions; RISC is register-to-register with LOAD and STORE as independent instructions

Correct. Incorporated in CISC, independent in RISC.

✗ CISC has them as independent instructions; RISC incorporates them

Wrong, and reversed.

✗ Both incorporate them into other instructions

Wrong. RISC keeps them separate on purpose.

✗ Both keep them independent

Wrong. CISC folds them into other instructions.

Incorporated in CISC, independent in RISC. It is the clearest statement in the course of what a load-store architecture means.

9.156

HARD

RISC S34

The table describes CISC as _____ and RISC as _____ .

BLANK 1

- a. memory-to-memory
- b. register-to-register
- c. stack-to-stack
- d. load-to-store

BLANK 2

- a. register-to-register
- b. memory-to-memory
- c. stack-to-stack
- d. load-to-store

Answer: **memory-to-memory, register-to-register**

Memory-to-memory against register-to-register, which is the same distinction as where LOAD and STORE sit.

9.157

HARD

RISC S35

Match each entry of the second comparison table to the architecture it belongs to.

More hardware oriented	CISC
More software oriented, since the compiler deals with translations	RISC
Instruction size mostly varied	CISC
Instruction size always a set size	RISC
Addressing modes can be complex	CISC
Addressing modes are simple	RISC

Three rows, six entries. Read the table as three questions, and RISC is the simpler, more uniform answer to every one of them.

10 Von Neumann, Harvard and Image Data

10.1

EASY

The real test 1, question 18

For a monochrome bitmap image, each pixel requires:

- a. 2 bits
- b. 8 bits
- c. 1 bit
- d. 4 bits

✗ 2 bits

Wrong. 2 bits give 4 levels, which is a four-shade image rather than a monochrome one.

✗ 8 bits

Wrong. 8 bits give 256 levels, the usual greyscale depth.

✓ 1 bit

Correct. One bit, two states, so image size in bits equals the number of pixels.

✗ 4 bits

Wrong. 4 bits give 16 levels, the old 16-colour standard.

Monochrome means two states, black or white, and one bit gives exactly two states. The general rule is that k bits per pixel give 2^k colours.

Photographs are generally better represented using:

- a. Vector graphics
- b. ASCII encoding
- c. Excess-3 encoding
- d. Bitmap graphics

✗ Vector graphics

Wrong. Vectors describe shapes with equations, which suits logos and diagrams, not the texture of a real scene.

✗ ASCII encoding

Wrong. ASCII encodes text characters.

✗ Excess-3 encoding

Wrong. Excess-3 encodes single decimal digits.

✓ Bitmap graphics

Correct. A pixel grid captures continuous tone, which is what a photograph is.

A photograph is continuous tone with no clean edges to describe as shapes, so it is stored as a grid of pixel values. Pair this with the logo question, where the answer was vector.

An embedded system uses separate memory modules for instructions and data to improve throughput. Which architectural design is being used?

- a. Von Neumann Architecture
- b. Stack Architecture
- c. Pipeline Architecture
- d. Harvard Architecture

X Von Neumann Architecture

Wrong. Von Neumann is the SINGLE shared memory design, the opposite of what the stem describes.

X Stack Architecture

Wrong. Stack architecture is about operand addressing, not memory separation.

X Pipeline Architecture

Wrong. Pipelining overlaps the stages of instruction processing. It is a technique, not a memory arrangement, though Harvard helps it along.

✓ Harvard Architecture

Correct. Separate instruction and data memories.

The phrase SEPARATE MEMORY for instructions and data is the definition of Harvard. It is common in embedded and signal processing designs for exactly the throughput reason given.

10.4

EASY

Intro S39

In the architecture component called system design, what distinguishes the Von Neumann architecture from the Harvard architecture?

- a. Von Neumann shares memory for data and instructions, Harvard separates them
- b. Von Neumann separates memory for data and instructions, Harvard shares it
- c. Von Neumann has a control unit, Harvard does not
- d. Von Neumann is pipelined, Harvard is not

✓ Von Neumann shares memory for data and instructions, Harvard separates them

Correct. Von Neumann: shared memory for data and instructions. Harvard: separate memory for each.

✗ Von Neumann separates memory for data and instructions, Harvard shares it

Wrong, and exactly reversed. Harvard is the one with two memories.

✗ Von Neumann has a control unit, Harvard does not

Wrong. Both have control units; the Von Neumann configuration is even described as containing one.

✗ Von Neumann is pipelined, Harvard is not

Wrong. Pipelining is a separate technique and is not what defines either architecture.

Shared against separate. Everything else about the two designs follows from this one difference.

10.5

EASY

Intro S39

Match each architecture to its memory arrangement.

Von Neumann Architecture

Shared memory for data and instructions

Harvard Architecture

Separate memory for data and instructions

System design defines how memory and processing units are structured, and these two are the choices the course gives.

10.6

EASY

Intro S40

In the Von Neumann architecture, program instructions and the data acted upon by those instructions live in the same area of memory.

Answer: **true**

They do, and that is the stored program idea. It is also the origin of the bottleneck, because instructions and data then compete for one path.

10.7

EASY

Intro S43

Which figure in the introduction deck shows the layout of a processor implementing the Harvard architecture?

- a. Figure 2
- b. Figure 1
- c. Figure 3
- d. Figure 4

✓ Figure 2

Correct. Figure 2 is the Harvard diagram.

✗ Figure 1

Wrong. Figure 1 shows the elements of Von Neumann architecture.

✗ Figure 3

Wrong. There is no Figure 3 in this deck.

✗ Figure 4

Wrong. There is no Figure 4 in this deck.

Figure 1 shows the elements of Von Neumann architecture and Figure 2 shows the Harvard layout, in that order.

The deck's example of a Harvard system is:

- a. The microcontroller in a washing machine or a car
- b. A desktop computer
- c. A laptop computer
- d. A cloud server

✓ The microcontroller in a washing machine or a car

Correct. These are the deck's two named Harvard examples.

✗ A desktop computer

Wrong. Desktops are given as Von Neumann.

✗ A laptop computer

Wrong. Laptops are given as Von Neumann.

✗ A cloud server

Wrong. Cloud servers are not mentioned in this deck.

Microcontrollers in washing machines or cars. Harvard systems are used in microcontrollers and embedded systems for faster access.

10.9**EASY**

Data S65

Which two types of graphics can be represented?

- a. Vector graphics and bitmap graphics
- b. Raster graphics and pixel graphics
- c. Analogue graphics and digital graphics
- d. Compressed graphics and uncompressed graphics

✓ Vector graphics and bitmap graphics

Correct. These are the two the deck names.

✗ Raster graphics and pixel graphics

Wrong. Raster is another word for bitmap, but pixel graphics is not a category the deck uses.

✗ Analogue graphics and digital graphics

Wrong. All computer graphics here are digital.

✗ Compressed graphics and uncompressed graphics

Wrong. Compression is not a category in this deck.

Vector and bitmap. Everything in this section is a comparison between those two.

10.10**EASY**

Data S65

The deck says _____ types of graphics can be represented.

- a. 2
- b. 3
- c. 4
- d. 5

Answer: **2**

Two: vector and bitmap.

10.11

EASY

Data S65

What does the word pixel stand for?

- a. Picture element
- b. Pictorial cell
- c. Pixel element
- d. Picture excerpt

✓ Picture element

Correct. The deck spells it out in brackets: Picture Elements.

✗ Pictorial cell

Wrong. Cell is not the word used.

✗ Pixel element

Wrong, and circular, since it uses the word it is meant to expand.

✗ Picture excerpt

Wrong. Excerpt is not the word used.

Picture element. Any graphic is made up from a series of pixels, and each pixel is an individual point on the screen.

10.12

EASY

Data S65

Any graphic is made up from a series of pixels, and each pixel is an individual point on the screen.

Answer: **true**

The deck says any graphic, which covers vector images too: a vector image is described by objects but is still drawn onto pixels.

10.13**EASY**

Data S66

In the bitmap illustration, assuming only black and white, how is each pixel stored?

- a. As a single 1 or 0
- b. As four bits
- c. As one byte
- d. As three values for red, green and blue

✓ As a single 1 or 0

Correct. Black and white means two states, so one bit each.

✗ As four bits

Wrong. Four bits would allow sixteen shades, which the illustration does not use.

✗ As one byte

Wrong. A byte per pixel would allow 256 levels, again more than needed here.

✗ As three values for red, green and blue

Wrong. RGB encoding is mentioned in the introduction deck for multimedia, but this illustration is black and white only.

One bit per pixel. That assumption is what makes the storage calculation later come out at exactly one bit per pixel.

10.14**EASY**

Data S66

Assuming only black and white for each pixel, a pixel is stored as a single _____ or 0.

- a. 1
- b. 2
- c. 8
- d. 16

Answer: **1**

1 or 0. Two states, one bit.

10.15

EASY

Data S67

The quality of a bitmap image depends on:

- a. The number of pixels
- b. The number of objects
- c. The number of attributes per object
- d. The file format

✓ The number of pixels

Correct. Quality follows pixel count in a bitmap.

✗ The number of objects

Wrong. Objects are how a vector image is built.

✗ The number of attributes per object

Wrong. Attributes define vector objects, not bitmap quality.

✗ The file format

Wrong. File format is not discussed in this deck.

The number of pixels: more pixels means higher resolution and a clearer image. Objects and attributes belong to vector graphics.

10.16

EASY

Data S67

More pixels means higher resolution and a clearer image.

Answer: **true**

Straight from the resolution slide, and it is the reason a bitmap loses quality when it is enlarged: the pixel count does not grow with it.

10.17

EASY

Data S69

In vector graphics, each image is made from:

- a. Objects such as a line, a rectangle or a circle
- b. Individual pixels, each stored separately
- c. Rows of bits read from top to bottom
- d. Compressed blocks of colour

✓ Objects such as a line, a rectangle or a circle

Correct. Line, rect and circle are the deck's examples of objects.

✗ Individual pixels, each stored separately

Wrong. Storing pixels individually is what a bitmap does.

✗ Rows of bits read from top to bottom

Wrong. That describes reading a bitmap.

✗ Compressed blocks of colour

Wrong. Compression is not part of this deck.

Objects, and every object has attributes which define it. That is the whole difference from a bitmap, which stores the picture point by point.

10.18

EASY

Data S69

In vector graphics every object has _____ which define it.

- a. attributes
- b. pixels
- c. layers
- d. coordinates

Answer: **attributes**

Attributes. Coordinates are one attribute among several, so the general word is the one the deck uses.

The deck says vector graphics are ideal for:

- a. Drawing plans, using a library of objects
- b. Storing photographs
- c. Real life images
- d. Altering individual pixels

✓ Drawing plans, using a library of objects

Correct. Plans, with a library of reusable objects.

✗ Storing photographs

Wrong. Photographs suit bit-mapped graphics, which are more realistic for that purpose.

✗ Real life images

Wrong. More realistic when used for photos and real life is a bitmap advantage.

✗ Altering individual pixels

Wrong. Altering individual pixels is a bitmap advantage.

Ideal for drawing plans, using a library of objects. Photographs and real life are the bitmap side of the comparison.

A processor fetches an instruction while simultaneously reading data from memory. Which architecture makes this possible?

- a. Stack Architecture
- b. CISC Architecture
- c. Von Neumann Architecture
- d. Harvard Architecture

✗ Stack Architecture

Wrong. Stack architecture describes how operands are addressed, not how memory is wired.

✗ CISC Architecture

Wrong. CISC is an instruction set philosophy, unrelated to the number of memory paths.

✗ Von Neumann Architecture

Wrong, and this is the contrast being tested. Von Neumann shares ONE memory and one bus for both, which forces the two accesses to take turns. That queue is the Von Neumann bottleneck.

✓ Harvard Architecture

Correct. Separate paths mean genuinely simultaneous access.

Harvard keeps instructions and data in separate memories with separate buses, so both can be read in the same cycle.

Which characteristic is a major advantage of the Harvard Architecture?

- a. Single memory space for all data
- b. Simpler hardware implementation
- c. Simultaneous access to instructions and data
- d. Reduced memory requirements

✗ Single memory space for all data

Wrong, and it describes VON NEUMANN. Harvard deliberately splits the memory space in two.

✗ Simpler hardware implementation

Wrong, and backwards. Two memory systems and two bus sets are MORE hardware, which is the price Harvard pays.

✓ Simultaneous access to instructions and data

Correct. Both accesses in the same cycle.

✗ Reduced memory requirements

Wrong. Splitting memory does not reduce how much is needed, and fixed separate pools can waste space when one fills before the other.

Two memories with two buses means an instruction fetch and a data read can happen at once, which raises throughput and removes the Von Neumann bottleneck.

10.22

MEDIUM

Intro S40

Select the elements the Von Neumann processor configuration consists of.

- a. A control unit
- b. An arithmetic logic unit
- c. A register set
- d. A memory region containing program instructions and data
- e. A separate instruction memory

✓ A control unit

Correct, the control unit is first.

✓ An arithmetic logic unit

Correct, the arithmetic logic unit is second.

✓ A register set

Correct, the register set is third.

✓ A memory region containing program instructions and data

Correct, and the single memory region holding both is the defining feature.

✗ A separate instruction memory

Wrong. A separate instruction memory would make it a Harvard machine.

Four elements. A separate instruction memory is the Harvard arrangement, and it is what Von Neumann does not have.

10.23

MEDIUM

Intro S40

The Von Neumann processor configuration consists of a control unit, an arithmetic logic unit, a _____ set, and a memory region containing program instructions and data.

- a. register
- b. instruction
- c. control
- d. buffer

Answer: **register**

Register set. Note that the register set is also one of the six components of computer architecture in the revision listing.

The key feature distinguishing the Von Neumann architecture from the Harvard architecture is the use of:

- a. A single area of memory for program instructions and the data acted upon by those instructions
- b. A single bus for all transfers
- c. A single control unit
- d. A single arithmetic logic unit

✓ A single area of memory for program instructions and the data acted upon by those instructions

Correct, and the deck calls it the key feature in exactly these words.

✗ A single bus for all transfers

Wrong as the stated key feature. Harvard does duplicate buses, but the deck names the memory area first and treats the extra lines as the cost.

✗ A single control unit

Wrong. Both designs have one control unit.

✗ A single arithmetic logic unit

Wrong. Both designs have one ALU.

A single area of memory for both. The single bus follows from that, but the memory is what the deck names as the key feature.

10.25

MEDIUM

Intro S41

According to the deck, locating all of the code and data a program requires in a single memory region is:

- a. Conceptually straightforward for programmers and relatively easier for circuit designers
- b. Harder for programmers but easier for circuit designers
- c. Easier for programmers but harder for circuit designers
- d. Harder for both, and done only for cost reasons

✓ Conceptually straightforward for programmers and relatively easier for circuit designers

Correct. Conceptually straightforward for programmers, relatively easier for circuit designers.

✗ Harder for programmers but easier for circuit designers

Wrong. The deck says it is straightforward for programmers too.

✗ Easier for programmers but harder for circuit designers

Wrong. The deck says it is easier for circuit designers as well.

✗ Harder for both, and done only for cost reasons

Wrong. Cost is not the reason given; simplicity on both sides is.

Easier for both. That is why Von Neumann became the general-purpose default, despite the bottleneck.

10.26

MEDIUM

Intro S41

It is conceptually _____ for programmers to locate all of the code and data a program requires in a single memory region.

- a. straightforward
- b. difficult
- c. expensive
- d. inefficient

Answer: **straightforward**

Straightforward. The Von Neumann design trades a performance limit for simplicity at both the programming and the circuit level.

A strict Harvard architecture uses:

- a. One address space and memory bus for program instructions and a separate address space and memory bus for data
- b. One address space with two memory buses
- c. Two address spaces sharing one memory bus
- d. One address space and one memory bus, with instructions cached separately

✓ One address space and memory bus for program instructions and a separate address space and memory bus for data

Correct. Separate address space and separate memory bus for each of instructions and data.

✗ One address space with two memory buses

Wrong. The address spaces are separate as well, not just the buses.

✗ Two address spaces sharing one memory bus

Wrong. Sharing one bus would put the two back into competition, which is what Harvard exists to avoid.

✗ One address space and one memory bus, with instructions cached separately

Wrong. A separate instruction cache on a shared memory is a different arrangement and is not what strict Harvard means.

Both the address space AND the memory bus are duplicated. That is what strict means here, and it is why the address lines, data lines and control signals all have to be duplicated too.

A strict Harvard architecture uses one _____ and memory bus for program instructions and a separate one for _____ .

BLANK 1

- a. address space
- b. register set
- c. control unit
- d. cache line

BLANK 2

- a. data
- b. instructions
- c. operands
- d. results

Answer: **address space, data**

| Address space, and data. Two address spaces, two memory buses, one for each kind of content.

10.29

MEDIUM

Intro S42

What immediate benefit does the Harvard configuration have?

- a. It enables simultaneous access to instructions and data, implementing a form of parallelism
- b. It halves the amount of memory needed
- c. It removes the need for a control unit
- d. It makes the instruction set smaller

✓ It enables simultaneous access to instructions and data, implementing a form of parallelism

Correct, and the phrase a form of parallelism is the deck's own.

✗ It halves the amount of memory needed

Wrong. It uses more hardware, not less, and the memory is not reduced.

✗ It removes the need for a control unit

Wrong. The control unit is required either way.

✗ It makes the instruction set smaller

Wrong. Instruction set size is an ISA matter, unrelated to how many memories there are.

Simultaneous access, which the deck explicitly calls a form of parallelism. In a Von Neumann machine that access has to be taken in turns.

10.30

MEDIUM

Intro S42

The Harvard configuration enables _____ access to instructions and data, implementing a form of parallelism.

- a. simultaneous
- b. sequential
- c. random
- d. direct

Answer: **simultaneous**

Simultaneous. The word to reach for in any exam question about what Harvard buys you.

10.31

MEDIUM

Intro S42

The Harvard architecture gives simultaneous access to instructions and data at no additional hardware cost.

Answer: **false**

There is a cost, and the deck names it: essentially duplicating the address lines, data lines and control signals the processor must implement.

10.32

MEDIUM

Intro S44

Von Neumann systems are used in:

- a. General-purpose computers such as laptops and desktops
- b. Microcontrollers in washing machines
- c. Microcontrollers in cars
- d. Embedded systems needing faster access

✓ General-purpose computers such as laptops and desktops

Correct. Laptops and desktops are the Von Neumann examples.

✗ Microcontrollers in washing machines

Wrong. Washing machine microcontrollers are given as a Harvard example.

✗ Microcontrollers in cars

Wrong. Car microcontrollers are given as a Harvard example.

✗ Embedded systems needing faster access

Wrong. Embedded systems needing faster access are exactly why Harvard is chosen there.

General-purpose computers. The other three options are all the deck's Harvard examples.

10.33

MEDIUM

Intro S44

Harvard systems are used in _____ and embedded systems for _____ access.

BLANK 1

- a. microcontrollers
- b. mainframes
- c. laptops
- d. servers

BLANK 2

- a. faster
- b. cheaper
- c. safer
- d. simpler

Answer: **microcontrollers, faster**

Microcontrollers, and faster access. Speed is the reason a design that costs more in wiring is still worth it in an embedded part.

10.34

MEDIUM

Data S68

In the memory storage worked example, what are the image size and resolution?

- a. 4 inches by 6 inches at 300 dots per linear inch
- b. 4 inches by 6 inches at 600 dots per linear inch
- c. 6 inches by 8 inches at 300 dots per linear inch
- d. 3 inches by 5 inches at 300 dots per linear inch

✓ 4 inches by 6 inches at 300 dots per linear inch

Correct. These are the figures given.

✗ 4 inches by 6 inches at 600 dots per linear inch

Wrong. Doubling the resolution would quadruple the pixel count and give a different answer.

✗ 6 inches by 8 inches at 300 dots per linear inch

Wrong. The dimensions are 4 by 6.

✗ 3 inches by 5 inches at 300 dots per linear inch

Wrong. The dimensions are 4 by 6.

4 by 6 inches, 300 d.p.i., black and white. Those three facts fix the whole calculation.

10.35

MEDIUM

Data S68

The worked image is 4 inches by 6 inches, the resolution is _____ d.p.i., and the image is black and white.

- a. 300
- b. 150
- c. 600
- d. 1200

Answer: **300**

300 dots per linear inch. The words linear inch matter: the count applies along each edge, not to the area.

10.36

MEDIUM

Data S68

How many pixels does the 4 inch by 6 inch image at 300 d.p.i. contain?

- a. 2,160,000
- b. 216,000
- c. 21,600,000
- d. 7,200

✓ 2,160,000

Correct. 1800 by 1200 gives 2,160,000.

✗ 216,000

Wrong by a factor of ten, a decimal slip.

✗ 21,600,000

Wrong by a factor of ten in the other direction.

✗ 7,200

Wrong. That is 1800 plus 1200 times a factor, not the product; the pixels form an area, so the two edge counts multiply.

Length 6 times 300 is 1800, breadth 4 times 300 is 1200, and 1800 times 1200 is 2,160,000 pixels.

10.37

MEDIUM

Data S68

Length is 6 times 300, which is _____ pixels, and breadth is 4 times 300, which is _____ pixels.

BLANK 1

- a. 1800
- b. 1200
- c. 2400
- d. 900

BLANK 2

- a. 1200
- b. 1800
- c. 800
- d. 1500

Answer: **1800, 1200**

1800 by 1200. The 6 inch edge is the length and the 4 inch edge is the breadth, exactly as the deck assigns them.

10.38

MEDIUM

Data S69

The deck lists _____ attributes needed to draw its rectangle.

- a. 6
- b. 4
- c. 5
- d. 7

Answer: **6**

Six. The thickness and colour of the lines are given as one item, which is where a miscount usually comes from.

Select the advantages of vector graphics, the draw packages.

- a. Images can be enlarged without losing resolution
- b. Objects can be edited by changing their attributes
- c. Objects can be layered on top or behind
- d. Images take up less disc space
- e. Each pixel can be altered

✓ Images can be enlarged without losing resolution

Correct, and it is the headline advantage.

✓ Objects can be edited by changing their attributes

Correct, editing by attribute is second.

✓ Objects can be layered on top or behind

Correct, layering is third.

✓ Images take up less disc space

Correct, less disc space is fourth.

✗ Each pixel can be altered

Wrong. Each pixel can be altered is listed under bit-mapped graphics, the paint packages.

Four advantages, plus a fifth on the slide: ideal for drawing plans, using a library of objects.
Altering each pixel is a bitmap advantage.

10.40

MEDIUM

Data S71

Why can a vector image be enlarged without losing resolution?

- a. Because it is stored as objects with attributes rather than as a fixed grid of pixels
- b. Because it stores more pixels than a bitmap
- c. Because it is compressed before being stored
- d. Because it always uses 1 bit per pixel

✓ Because it is stored as objects with attributes rather than as a fixed grid of pixels
Correct, and it follows directly from the attribute list for the rectangle.

✗ Because it stores more pixels than a bitmap
Wrong. A vector image stores no pixels at all, only object descriptions.

✗ Because it is compressed before being stored
Wrong. Compression is not part of this deck, and it would not help with enlargement anyway.

✗ Because it always uses 1 bit per pixel
Wrong. One bit per pixel is the black and white bitmap assumption.

Nothing in a vector object fixes a pixel count, so the objects can simply be redrawn larger. A bitmap has only the pixels it was captured with.

10.41

MEDIUM

Data S71

Vector graphics are ideal for drawing plans, using a _____ of objects.

- a. library
- b. grid
- c. layer
- d. palette

Answer: **library**

Library. Because objects are described rather than drawn, they can be stored once and reused.

Select the advantages of bit-mapped graphics, the paint packages.

- a. Each pixel can be altered
- b. More realistic when used for photos and real life
- c. Images can be enlarged without losing resolution
- d. Images take up less disc space

✓ Each pixel can be altered

Correct. Pixel by pixel control is what a paint package gives you.

✓ More realistic when used for photos and real life

Correct. Realism for photographs is the second bitmap advantage.

✗ Images can be enlarged without losing resolution

Wrong. Enlarging without losing resolution is a vector advantage; a bitmap loses quality when enlarged.

✗ Images take up less disc space

Wrong. Less disc space is a vector advantage.

Just two: each pixel can be altered, and they are more realistic for photos and real life. The other two options are vector advantages.

The Harvard enhancement comes at the expense of essentially duplicating which of these?

- a. Address lines
- b. Data lines
- c. Control signals
- d. Arithmetic logic units

✓ Address lines

Correct, address lines are duplicated.

✓ Data lines

Correct, data lines are duplicated.

✓ Control signals

Correct, control signals are duplicated.

✗ Arithmetic logic units

Wrong. Nothing in the deck duplicates the ALU; the cost is all in the interconnection.

Three things are duplicated: address lines, data lines and control signals, all so the processor can reach both memory regions. The ALU is not duplicated.

Select the enhancements the deck lists alongside Von Neumann systems.

- a. Cache memory
- b. Introduction of DDR memories
- c. Pre-fetching
- d. PIM
- e. Duplicated address lines

✓ Cache memory

Correct, cache memory heads the list.

✓ Introduction of DDR memories

Correct, the introduction of DDR memories is second.

✓ Pre-fetching

Correct, pre-fetching is third.

✓ PIM

Correct, PIM closes the list.

✗ Duplicated address lines

Wrong. Duplicated address lines are the cost of going Harvard, not a Von Neumann enhancement.

Four: cache memory, DDR memories, pre-fetching and PIM. Every one of them is a way of easing the single-memory bottleneck without abandoning the single memory.

10.45**HARD**

Data S68

What is the final storage requirement in the worked example?

- a. 263.4 KBytes
- b. 270 KBytes
- c. 2,160 KBytes
- d. 2.16 MBytes

✓ 263.4 KBytes

Correct. 263.4 KBytes is the deck's final answer.

✗ 270 KBytes

Wrong, and it is the trap: 270,000 bytes is right, but a KByte is 1024 bytes, not 1000, so it becomes 263.4 KBytes.

✗ 2,160 KBytes

Wrong. That treats the bit count as a byte count.

✗ 2.16 MBytes

Wrong. That is the bit count read as bytes and then converted.

2,160,000 bits is 270,000 bytes, and 270,000 divided by 1024 is 263.4 KBytes. The last division is the step people skip.

10.46**HARD**

Data S68

Black and white means 1 bit per pixel, so the storage is 2,160,000 bits, which is _____ bytes, that is 263.4 KBytes.

- a. 270000
- b. 263400
- c. 17280000
- d. 2160000

Answer: **270000**

270,000 bytes, from dividing the bit count by 8. Then dividing by 1024 gives 263.4 KBytes.

To draw the deck's rectangle, which attributes do we need to know?

- a. Start X and Y coordinates
- b. The length
- c. The breadth
- d. The thickness and colour of the lines
- e. The type of line, for example dashed
- f. The fill colour
- g. The number of pixels in the image

✓ Start X and Y coordinates

Correct, the start coordinates come first.

✓ The length

Correct, the length is needed.

✓ The breadth

Correct, the breadth is needed.

✓ The thickness and colour of the lines

Correct, the thickness and colour of the lines are listed together.

✓ The type of line, for example dashed

Correct, the type of line, with dashed as the example.

✓ The fill colour

Correct, the fill colour closes the list.

✗ The number of pixels in the image

Wrong. A vector object carries no pixel count, which is why it scales cleanly.

Six attributes. Pixel count is nowhere in the list, and that is exactly why a vector image can be enlarged without losing resolution.

10.48

HARD

Data S71

Match each advantage to the kind of graphics it belongs to.

Images can be enlarged without losing resolution	Vector
Objects can be edited by changing their attributes	Vector
Objects can be layered on top or behind	Vector
Images take up less disc space	Vector
Each pixel can be altered	Bit-mapped
More realistic when used for photos and real life	Bit-mapped

Four against two. The pattern is worth naming: vector wins on flexibility and size, bitmap wins on fine detail and realism.

11 Cache Mapping Functions

11.1

EASY

Cache S2, Rev S38

The direct mapping address format is:

- a. Tag, Index, Offset
- b. Tag, Offset
- c. Tag, Set Index, Offset
- d. Opcode, Mode, Address

✓ Tag, Index, Offset

Correct. Three fields, in that order.

✗ Tag, Offset

Wrong. That is the fully associative format, which has no index field.

✗ Tag, Set Index, Offset

Wrong. That is the set associative format.

✗ Opcode, Mode, Address

Wrong. That is the instruction format, from a different topic.

Three fields, in that order. Fully associative drops the index and set associative renames it the set index.

11.2**EASY**

Cache S3, Rev S39

In direct mapping Example 1, what are the three given quantities?

- a. Main memory 64 KB, cache 1 KB, block 16 bytes
- b. Main memory 128 KB, cache 4 KB, block 32 bytes
- c. Main memory 1 MB, cache 8 KB, block 64 bytes
- d. Main memory 16 KB, cache 1 KB, block 64 bytes

✓ Main memory 64 KB, cache 1 KB, block 16 bytes

Correct. 64 KB, 1 KB, 16 bytes.

✗ Main memory 128 KB, cache 4 KB, block 32 bytes

Wrong. That is direct mapping Example 2.

✗ Main memory 1 MB, cache 8 KB, block 64 bytes

Wrong. That is direct mapping Example 3.

✗ Main memory 16 KB, cache 1 KB, block 64 bytes

Wrong. No example uses those figures.

64 KB, 1 KB, 16 bytes. The same three numbers return for fully associative Example 1 and 2-way set associative Example 1, so all three mapping functions can be compared on one case.

11.3**EASY**

Cache S3, Rev S39

A cache size of 1 KB is 1024 bytes, which is 2 to the power ____ .

- a. 10
- b. 8
- c. 12
- d. 16

Answer: **10**

2 to the power 10. Knowing that 1 KB is 2 to the 10 and 1 MB is 2 to the 20 makes every one of these examples quick.

11.4**EASY**

Cache S4, Rev S40

With a block size of 16 bytes, how many offset bits are needed?

- a. 4
- b. 16
- c. 5
- d. 8

✓ 4

Correct. 16 bytes is 2 to the power 4, so the offset is 4 bits, running 0000 up to 1111.

✗ 16

Wrong. 16 is the block size in bytes, not the bit count.

✗ 5

Wrong. 5 bits would index 32 bytes.

✗ 8

Wrong. 8 bits would index 256 bytes, which is the worked example in the memory deck.

16 bytes is 2 to the power 4, so the offset is 4 bits, running 0000 up to 1111.

11.5**EASY**

Cache S4, Rev S40

The offset bits come from the block size: 16 bytes is 2 to the power ____ .

- a. 4
- b. 5
- c. 8
- d. 16

Answer: **4**

2 to the 4. The offset always comes from the block size, in every one of the three mapping functions.

What does direct mapping Example 2 ask you to find?

- a. Offset
- b. Index
- c. Tag bits
- d. Tag Directory Size
- e. Set bits

✓ Offset

Correct, offset is first.

✓ Index

Correct, index is second.

✓ Tag bits

Correct, tag bits is third.

✓ Tag Directory Size

Correct, tag directory size closes the list.

✗ Set bits

Wrong. Direct mapping has no sets; each block maps to exactly one line.

Four quantities. Set bits only appear in the set associative examples, where a set index replaces the plain index.

11.7

EASY

Cache S8, Rev S44

In fully associative mapping:

- a. Any memory block can be placed in any cache line
- b. Each block maps to exactly one line
- c. Each block maps to any line within one set
- d. Blocks are placed in order of arrival

✓ Any memory block can be placed in any cache line

Correct. Any block, any line.

✗ Each block maps to exactly one line

Wrong, and that is direct mapping, which the slide contrasts on the same line.

✗ Each block maps to any line within one set

Wrong. That is set associative mapping.

✗ Blocks are placed in order of arrival

Wrong. Arrival order is not a mapping function.

Any block, any line. Direct mapping is the opposite extreme, one line only, and set associative sits between them.

11.8

EASY

Cache S8, Rev S44

The slide contrasts fully associative with direct mapping, where each block goes to exactly _____ line.

- a. 1
- b. 2
- c. 4
- d. 8

Answer: **1**

Exactly 1 line, which is what causes thrashing when two hot blocks collide.

What do the fully associative examples ask you to find?

- a. Offset bits
- b. Tag bits
- c. Total cache lines
- d. Tag directory size
- e. Index bits

✓ Offset bits

Correct, offset bits.

✓ Tag bits

Correct, tag bits.

✓ Total cache lines

Correct, total cache lines, needed for the tag directory.

✓ Tag directory size

Correct, tag directory size.

✗ Index bits

Wrong. There is no index field in fully associative mapping.

Four quantities, and index is not among them because a fully associative cache has no index field.

11.10

EASY

Cache S11, Rev S47

The set associative address format is:

- a. Tag, Set Index, Offset
- b. Tag, Index, Offset
- c. Tag, Offset
- d. Set Index, Tag, Offset

✓ Tag, Set Index, Offset

Correct. Tag, Set Index, Offset.

✗ Tag, Index, Offset

Wrong. In set associative mapping the middle field is a set index, not a line index.

✗ Tag, Offset

Wrong. That is the fully associative format.

✗ Set Index, Tag, Offset

Wrong. The tag always comes first.

Tag, Set Index, Offset. It is the direct mapping format with the index selecting a set rather than a single line.

11.11

EASY

Cache S11, Rev S47

The set associative address format is Tag, then _____ Index, then Offset.

- a. Set
- b. Line
- c. Block
- d. Word

Answer: **Set**

Set Index. The name changes because the field now picks a group of lines rather than one line.

What do the set associative examples ask you to find?

- a. Address bits
- b. Offset
- c. Cache lines
- d. Set bits
- e. Tag bits
- f. Tag directory size
- g. Tag directory size in bytes rather than bits

✓ Address bits

Correct, address bits come from the main memory size.

✓ Offset

Correct, the offset comes from the block size.

✓ Cache lines

Correct, cache lines come from cache size over block size.

✓ Set bits

Correct, set bits come from lines over n.

✓ Tag bits

Correct, tag bits are what is left of the address.

✓ Tag directory size

Correct, and it is lines times tag bits.

✗ Tag directory size in bytes rather than bits

Wrong. The examples ask for the tag directory size, and the worked answers give it in bits first, converting to bytes only at the end. It is not a separate quantity the question asks you to find.

Six quantities, in the order you have to compute them: address, offset, lines, sets, tag, directory. Each step feeds the next, and the directory is asked for in bits.

11.13

EASY

Mem S39

There are _____ cache lines than main memory blocks, so a mapping function is needed.

- a. fewer
- b. more
- c. larger
- d. faster

Answer: **fewer**

Fewer. The memory deck writes it as m much less than M.

11.14

EASY

Mem S39

Select the three techniques for mapping blocks into cache lines.

- a. Direct
- b. Associative
- c. Set associative
- d. Sequential

✓ Direct

Correct, and it is the simplest.

✓ Associative

Correct, also called fully associative.

✓ Set associative

Correct, and it combines the best of the other two.

✗ Sequential

Wrong. Sequential access is what magnetic tape does.

Direct, associative, set associative. Sequential is an access method, from the memory characteristics slide, not a mapping function.

11.15**EASY**

Mem S39

_____ techniques can be used for mapping blocks into cache lines.

- a. 3
- b. 2
- c. 4
- d. 5

Answer: **3**

Three: direct, associative and set associative.

11.16**EASY**

Mem S40

Direct mapping, the simplest technique, maps each block of main memory into:

- a. Only one possible cache line
- b. Any cache line
- c. Any line within one set
- d. Two possible cache lines

✓ Only one possible cache line

Correct. Exactly one line, which is what makes it simple and what makes thrashing possible.

✗ Any cache line

Wrong. That is associative mapping.

✗ Any line within one set

Wrong. That is set associative mapping.

✗ Two possible cache lines

Wrong. Two lines per block would be 2-way set associative.

Exactly one line, which is what makes it simple and what makes thrashing possible.

11.17

EASY

Mem S40

Direct mapping is described as the _____ technique.

- a. simplest
- b. fastest
- c. newest
- d. most flexible

Answer: **simplest**

Simplest. Its advantage is that it is simple and inexpensive to implement.

11.18

EASY

Mem S43

If the tag from the address matches the line tag, what follows?

- a. A cache hit, and the corresponding word is retrieved from the indexed line
- b. A cache miss, and the line is updated
- c. The next line is checked
- d. The cache is flushed

✓ A cache hit, and the corresponding word is retrieved from the indexed line

Correct. A hit, and the word comes from the line the index already selected.

✗ A cache miss, and the line is updated

Wrong. That happens when the tags do not match.

✗ The next line is checked

Wrong. In direct mapping there is only one line to check.

✗ The cache is flushed

Wrong. Flushing belongs to virtual caches at a context switch.

A hit, and the word comes from the line the index already selected.

11.19

EASY

Mem S43

In a direct mapped cache, only one line ever has to be checked for a given address.

Answer: **true**

One only, because the index picks it out. That is the difference from associative mapping, which has to examine every line's tag.

11.20

EASY

Mem S44

The advantage of direct mapping is that it is:

- a. Simple and inexpensive to implement
- b. Free of thrashing
- c. Able to place any block anywhere
- d. The fastest at high hit rates

✓ Simple and inexpensive to implement

Correct. Simple and inexpensive.

✗ Free of thrashing

Wrong, and thrashing is precisely its disadvantage.

✗ Able to place any block anywhere

Wrong. That freedom belongs to associative mapping.

✗ The fastest at high hit rates

Wrong. No such claim is made.

| Simple and inexpensive. Only one tag comparison is ever needed.

11.21

EASY

Mem S44

Direct mapping is simple and _____ to implement.

- a. inexpensive
- b. expensive
- c. slow
- d. flexible

Answer: **inexpensive**

| Inexpensive. Fully associative mapping is the costly one, since every tag must be compared at once.

11.22**EASY**

Mem S45

The memory deck's worked example asks for the number of bits in the tag and the _____ directory size.

- a. tag
- b. line
- c. block
- d. set

Answer: **tag**

The tag directory size. Both worked examples in that deck ask for the same two quantities.

11.23**EASY**

Mem S50

Associative mapping permits each main memory block to be loaded into _____ line of the cache.

- a. any
- b. one
- c. every
- d. the first free

Answer: **any**

Any line. Direct mapping allows exactly one.

11.24**EASY**

Mem S54

Worked associative Example 1 considers a fully associative mapped cache of size _____ KB.

- a. 16
- b. 4
- c. 8
- d. 32

Answer: **16**

16 KB, with a 256 byte block and 128 KB of main memory.

11.25**EASY**

Mem S60

The solution to associative Example 2 appears twice in the memory deck, both times giving a 34-bit address, a 12-bit block offset and a 22-bit tag.

Answer: **true**

Slides 59 and 60 carry the same working. Repetition in a lecture deck is usually a signal about what will be asked.

11.26**EASY**

Mem S62

In worked set associative Example 1 the set size is ____ .

- a. 2
- b. 4
- c. 8
- d. 16

Answer: **2**

2, because the cache is 2-way. Set size and the n of n-way are the same thing.

11.27

MEDIUM

Cache S2, Rev S38

The direct mapping formula is:

- a. Cache Line equals Main Memory Block Number mod Number of Cache Addresses
- b. Cache Line equals Main Memory Block Number times Number of Cache Addresses
- c. Cache Line equals Number of Cache Addresses mod Main Memory Block Number
- d. Cache Line equals Main Memory Block Number divided by Block Size

✓ Cache Line equals Main Memory Block Number mod Number of Cache Addresses
Correct. Block number mod number of lines.

✗ Cache Line equals Main Memory Block Number times Number of Cache Addresses
Wrong. Multiplying would give a line number far beyond the cache.

✗ Cache Line equals Number of Cache Addresses mod Main Memory Block Number
Wrong, and modulo is not symmetric, so the order matters.

✗ Cache Line equals Main Memory Block Number divided by Block Size
Wrong. Dividing by the block size converts an address to a block number, which is a different step.

Block number mod number of lines. The memory deck writes the same rule as $i \text{ equals } j \text{ modulo } m$.

11.28

MEDIUM

Cache S2, Rev S38

For direct mapping, Cache Line equals the Main Memory Block Number _____ the Number of Cache Addresses.

- a. mod
- b. times
- c. plus
- d. minus

Answer: **mod**

Mod, that is the remainder after division. It guarantees the answer falls inside the cache however large the block number is.

How many address bits does a main memory of 64 KB need?

- a. 16
- b. 14
- c. 17
- d. 10

✓ 16

Correct. 64 KB is 2 to the power 6 times 2 to the power 10, which is 2 to the power 16 bytes, so the address is 16 bits.

✗ 14

Wrong. 64 KB is 2 to the 16, not 2 to the 14.

✗ 17

Wrong. 17 bits is 128 KB, which is Example 2.

✗ 10

Wrong. 10 bits would address only 1 KB.

64 KB is 2 to the power 6 times 2 to the power 10, which is 2 to the power 16 bytes, so the address is 16 bits.

11.30

MEDIUM

Cache S3, Rev S39

64 KB is 2 to the power 6 times 2 to the power 10, which is 2 to the power ____ bytes, so the address is ____ bits.

BLANK 1

- a. 16
- b. 14
- c. 60
- d. 17

BLANK 2

- a. 16
- b. 14
- c. 10
- d. 6

Answer: **16, 16**

2 to the 16, so 16 address bits. The exponent and the bit count are always the same number, which is the shortcut worth carrying.

11.31

MEDIUM

Cache S4, Rev S40

How is the number of cache lines found?

- a. Cache size divided by block size, $\frac{\text{so } 1024}{16}$, which is 64 lines
- b. Cache size times block size
- c. Main memory size divided by cache size
- d. Block size divided by cache size

✓ Cache size divided by block size, $\frac{\text{so } 1024}{16}$, which is 64 lines

Correct. Cache size over block size.

✗ Cache size times block size

Wrong. Multiplying gives a number far larger than the cache.

✗ Main memory size divided by cache size

Wrong. That ratio tells you how many caches would fit in main memory.

✗ Block size divided by cache size

Wrong, and it inverts the division.

Cache size over block size. 1024 over 16 gives 64 lines of 16 bytes each.

11.32

MEDIUM

Cache S4, Rev S40

1024 bytes over 16 bytes is _____ lines.

- a. 64
- b. 32
- c. 128
- d. 16

Answer: **64**

64 lines, each of 16 bytes. This figure returns in the tag directory calculation and in the set count.

11.33

MEDIUM

Cache S4, Rev S40

With 64 cache lines, how many index bits are needed?

- a. 6
- b. 64
- c. 5
- d. 4

✓ 6

Correct. 64 is 2 to the power 6, so the index is 6 bits: the number of bits used to identify each line.

✗ 64

Wrong. 64 is the line count, not the bit count.

✗ 5

Wrong. 5 bits identify 32 items, which is the set count in the 2-way example.

✗ 4

Wrong. 4 bits is the offset here.

64 is 2 to the power 6, so the index is 6 bits: the number of bits used to identify each line.

11.34

MEDIUM

Cache S4, Rev S40

Because 64 lines is 2 to the power _____, the index is _____ bits.

BLANK 1

- a. 6
- b. 5
- c. 7
- d. 4

BLANK 2

- a. 6
- b. 5
- c. 7
- d. 4

Answer: **6, 6**

2 to the 6, so 6 index bits. The index is defined as the number of bits used to identify each line.

11.35

MEDIUM

Cache S5, Rev S41

In direct mapping, the tag bits are found as:

- a. Total main memory bits minus the sum of the index and the offset
- b. Total main memory bits minus the offset only
- c. The index plus the offset
- d. Total main memory bits divided by the number of lines

✓ Total main memory bits minus the sum of the index and the offset

Correct. Subtract both other fields.

✗ Total main memory bits minus the offset only

Wrong. That is the fully associative tag formula.

✗ The index plus the offset

Wrong. Adding the other two fields does not give the third.

✗ Total main memory bits divided by the number of lines

Wrong. Division has no role in the field widths.

Subtract both other fields. In fully associative there is no index, so only the offset comes off, and the tag is correspondingly larger.

11.36

MEDIUM

Cache S5, Rev S41

Tag bits equal the total number of main memory bits minus the sum of the _____ and the offset.

- a. index
- b. tag
- c. block
- d. line size

Answer: **index**

The index. The three fields together always account for the whole address.

In direct mapping Example 1, what is the complete address format?

- a. Tag 6 bits, Index 6 bits, Offset 4 bits
- b. Tag 4 bits, Index 6 bits, Offset 6 bits
- c. Tag 12 bits, Offset 4 bits
- d. Tag 7 bits, Set Index 5 bits, Offset 4 bits

✓ Tag 6 bits, Index 6 bits, Offset 4 bits

Correct. 16 minus 6 minus 4 gives a 6-bit tag, so the format is 6, 6, 4, which adds back to 16.

✗ Tag 4 bits, Index 6 bits, Offset 6 bits

Wrong. The offset is 4 and the tag is 6.

✗ Tag 12 bits, Offset 4 bits

Wrong. That is the fully associative answer for the same machine.

✗ Tag 7 bits, Set Index 5 bits, Offset 4 bits

Wrong. That is the 2-way set associative answer for the same machine.

| 16 minus 6 minus 4 gives a 6-bit tag, so the format is 6, 6, 4, which adds back to 16.

11.38

MEDIUM

Cache S5, Rev S41

In direct mapping Example 1 the tag is 16 minus 6 minus 4, which is _____ bits, and the three fields add back to _____ bits.

BLANK 1

- a. 6
- b. 4
- c. 7
- d. 12

BLANK 2

- a. 16
- b. 14
- c. 17
- d. 10

Answer: **6, 16**

A 6-bit tag, and the fields sum to the 16-bit address. Adding them back is the quickest check on any of these answers.

11.39

MEDIUM

Cache S6, Rev S42

Tag Directory Size equals:

- a. Total cache lines times tag bits
- b. Total cache lines times block size
- c. Tag bits times offset bits
- d. Cache size times tag bits

✓ Total cache lines times tag bits

Correct. Lines times tag bits, because every line stores a tag of its own.

✗ Total cache lines times block size

Wrong. That would give the cache's data capacity in bytes.

✗ Tag bits times offset bits

Wrong. The offset plays no part in the tag directory.

✗ Cache size times tag bits

Wrong. It is the number of lines, not the size in bytes, that multiplies the tag.

Lines times tag bits, because every line stores a tag of its own. The memory deck writes it as number of tags times tag size.

11.40

MEDIUM

Cache S6, Rev S42

Tag Directory Size equals total cache _____ times tag bits.

- a. lines
- b. blocks
- c. bytes
- d. sets

Answer: **lines**

Lines. Each line needs one tag, so the count of lines is the multiplier even in a set associative cache.

11.41

MEDIUM

Cache S6, Rev S42

In direct mapping Example 1, what is the tag directory size?

- a. 384 bits, which is 48 bytes
- b. 768 bits, which is 96 bytes
- c. 448 bits, which is 56 bytes
- d. 192 bits, which is 24 bytes

✓ 384 bits, which is 48 bytes

Correct. 64 lines times 6 tag bits is 384 bits, and 384 over 8 is 48 bytes.

✗ 768 bits, which is 96 bytes

Wrong. 768 bits is the fully associative answer for the same machine, where the tag is 12 bits.

✗ 448 bits, which is 56 bytes

Wrong. 448 bits is the 2-way set associative answer, where the tag is 7 bits.

✗ 192 bits, which is 24 bytes

Wrong. 192 bits is the memory deck's worked example, with 3 tag bits.

64 lines times 6 tag bits is 384 bits, and 384 over 8 is 48 bytes.

11.42

MEDIUM

Cache S6, Rev S42

The tag directory size is 64 times 6, which is _____ bits, or _____ bytes when divided by 8.

BLANK 1

- a. 384
- b. 768
- c. 448
- d. 192

BLANK 2

- a. 48
- b. 96
- c. 56
- d. 24

Answer: **384, 48**

384 bits, 48 bytes. Dividing by 8 at the end is the step most often forgotten.

11.43

MEDIUM

Cache S6, Rev S42

The full answer to direct mapping Example 1 gives offset 4 bits, index 6 bits, tag _____ bits, and a tag directory of 384 bits.

- a. 6
- b. 4
- c. 12
- d. 7

Answer: **6**

6 bits. In this example the tag and the index happen to be the same width, which is a coincidence of the numbers rather than a rule.

11.44

MEDIUM

Cache S7, Rev S43

Direct mapping Example 2 gives which figures?

- a. Main memory 128 KB, cache 4 KB, block 32 B
- b. Main memory 64 KB, cache 1 KB, block 16 B
- c. Main memory 1 MB, cache 8 KB, block 64 B
- d. Main memory 256 KB, cache 4 KB, block 32 B

✓ Main memory 128 KB, cache 4 KB, block 32 B
Correct. 128 KB, 4 KB, 32 B.

✗ Main memory 64 KB, cache 1 KB, block 16 B
Wrong. That is direct mapping Example 1.

✗ Main memory 1 MB, cache 8 KB, block 64 B
Wrong. That is direct mapping Example 3.

✗ Main memory 256 KB, cache 4 KB, block 32 B
Wrong. 256 KB belongs to fully associative Example 2.

128 KB, 4 KB, 32 B. Note how close it is to fully associative Example 2, which uses 256 KB with the same cache and block.

11.45

MEDIUM

Cache S7, Rev S43

Direct mapping Example 3 gives which figures?

- a. Main memory 1 MB, cache 8 KB, block 64 B
- b. Main memory 2 MB, cache 16 KB, block 64 B
- c. Main memory 4 MB, cache 32 KB, block 64 B
- d. Main memory 128 KB, cache 4 KB, block 32 B

✓ Main memory 1 MB, cache 8 KB, block 64 B

Correct. 1 MB, 8 KB, 64 B.

✗ Main memory 2 MB, cache 16 KB, block 64 B

Wrong. 2 MB with 16 KB is fully associative Example 3.

✗ Main memory 4 MB, cache 32 KB, block 64 B

Wrong. 4 MB with 32 KB is the 8-way set associative Example 3.

✗ Main memory 128 KB, cache 4 KB, block 32 B

Wrong. That is direct mapping Example 2.

1 MB, 8 KB, 64 B. The other three MB-sized cases belong to fully associative Example 3 and set associative Example 3.

11.46

MEDIUM

Cache S7, Rev S43

In direct mapping Example 3 the block size is _____ B.

- a. 64
- b. 32
- c. 16
- d. 128

Answer: **64**

64 bytes, which is 2 to the power 6, so the offset would be 6 bits.

11.47

MEDIUM

Cache S8, Rev S44

The fully associative address format is Tag then Offset. What is missing compared with direct mapping?

- a. There is no index field
- b. There is no tag field
- c. There is no offset field
- d. There is no set index field

✓ There is no index field

Correct, and the slide says so explicitly.

✗ There is no tag field

Wrong. The tag is present and larger than in direct mapping.

✗ There is no offset field

Wrong. The offset is always needed to pick a byte inside a block.

✗ There is no set index field

Wrong. A set index exists only in set associative mapping.

No index. There is nothing to index, because a block may be anywhere, so those bits go to the tag instead.

11.48

MEDIUM

Cache S8, Rev S44

A fully associative cache address has no index field.

Answer: **true**

None. Its format is Tag then Offset, and the bits that would have been an index become part of the tag.

11.49

MEDIUM

Cache S9, Rev S45

In fully associative Example 1, with main memory 64 KB, cache 1 KB and block 16 B, what are the address and offset widths?

- a. Address 16 bits, offset 4 bits
- b. Address 16 bits, offset 6 bits
- c. Address 17 bits, offset 4 bits
- d. Address 10 bits, offset 4 bits

✓ Address 16 bits, offset 4 bits

Correct. The same 16-bit address and 4-bit offset as the direct mapped case, because those two depend only on the memory size and the block size.

✗ Address 16 bits, offset 6 bits

Wrong. 6 bits was the index in the direct mapped example, not the offset.

✗ Address 17 bits, offset 4 bits

Wrong. 17 bits is 128 KB.

✗ Address 10 bits, offset 4 bits

Wrong. 10 bits would address only the 1 KB cache.

The same 16-bit address and 4-bit offset as the direct mapped case, because those two depend only on the memory size and the block size.

11.50

MEDIUM

Cache S9, Rev S45

In fully associative Example 1 the address is _____ bits, exactly as in the direct mapped case.

- a. 16
- b. 12
- c. 17
- d. 10

Answer: **16**

16 bits. Only the split of those bits changes between mapping functions, never the total.

11.51

MEDIUM

Cache S9, Rev S45

The fully associative tag directory is 64 times 12, which is 768 bits, that is ____ bytes.

- a. 96
- b. 48
- c. 56
- d. 72

Answer: **96**

96 bytes. Full associativity buys placement freedom and pays for it in tag storage.

11.52

MEDIUM

Cache S10, Rev S46

Fully associative Example 2 gives which figures?

- a. Main memory 256 KB, cache 4 KB, block 32 B
- b. Main memory 128 KB, cache 4 KB, block 32 B
- c. Main memory 512 KB, cache 8 KB, block 32 B
- d. Main memory 2 MB, cache 16 KB, block 64 B

✓ Main memory 256 KB, cache 4 KB, block 32 B

Correct. 256 KB, 4 KB, 32 B.

✗ Main memory 128 KB, cache 4 KB, block 32 B

Wrong. 128 KB is direct mapping Example 2.

✗ Main memory 512 KB, cache 8 KB, block 32 B

Wrong. 512 KB is the 4-way set associative Example 2.

✗ Main memory 2 MB, cache 16 KB, block 64 B

Wrong. That is fully associative Example 3.

256 KB, 4 KB, 32 B. It shares its cache and block sizes with direct mapping Example 2, which uses 128 KB of main memory instead.

11.53

MEDIUM

Cache S10, Rev S46

Fully associative Example 3 gives which figures?

- a. Main memory 2 MB, cache 16 KB, block 64 B
- b. Main memory 1 MB, cache 8 KB, block 64 B
- c. Main memory 4 MB, cache 32 KB, block 64 B
- d. Main memory 256 KB, cache 4 KB, block 32 B

✓ Main memory 2 MB, cache 16 KB, block 64 B

Correct. 2 MB, 16 KB, 64 B.

✗ Main memory 1 MB, cache 8 KB, block 64 B

Wrong. That is direct mapping Example 3.

✗ Main memory 4 MB, cache 32 KB, block 64 B

Wrong. That is the 8-way set associative Example 3.

✗ Main memory 256 KB, cache 4 KB, block 32 B

Wrong. That is fully associative Example 2.

2 MB, 16 KB, 64 B. All three Example 3 cases use a 64 B block, so the offset is 6 bits in every one of them.

11.54

MEDIUM

Cache S10, Rev S46

In fully associative Example 3 the main memory is _____ MB.

- a. 2
- b. 1
- c. 4
- d. 8

Answer: 2

2 MB, which is 2 to the power 21 bytes, so the address would be 21 bits.

11.55

MEDIUM

Cache S11, Rev S47

For an n-way set associative cache, the number of sets is:

- a. Cache Lines divided by n
- b. Cache Lines times n
- c. n divided by Cache Lines
- d. Cache size divided by n

✓ Cache Lines divided by n

Correct. $\frac{\text{Lines}}{n}$.

✗ Cache Lines times n

Wrong. Multiplying would give more sets than lines.

✗ n divided by Cache Lines

Wrong, and it inverts the division.

✗ Cache size divided by n

Wrong. It is the number of lines, not the size in bytes, that is divided.

$\frac{\text{Lines}}{n}$. Each set holds n lines, so dividing the lines among the sets gives the count.

11.56

MEDIUM

Cache S11, Rev S47

For an n-way set associative cache, Number of Sets equals Cache Lines divided by ____ .

- a. n
- b. 2
- c. the block size
- d. the tag

Answer: **n**

By n, the associativity. With n equal to 1 it reduces to direct mapping, and with n equal to the line count it becomes fully associative.

In the 2-way set associative Example 1, with main memory 64 KB, cache 1 KB and block 16 B, how many cache lines are there?

- a. 64
- b. 32
- c. 128
- d. 16

✓ 64

Correct. $\frac{1024}{16}$ is 64 lines, the same as in the direct mapped and fully associative versions.

✗ 32

Wrong. 32 is the number of SETS, once the 64 lines are paired up.

✗ 128

Wrong. That would need a smaller block or a larger cache.

✗ 16

Wrong. 16 is the block size in bytes.

$\frac{1024}{16}$ is 64 lines, the same as in the direct mapped and fully associative versions. Associativity groups the lines; it does not change how many there are.

11.58

MEDIUM

Cache S12, Rev S48

In the 2-way set associative Example 1 the address is _____ bits and the offset is _____ bits.

BLANK 1

- a. 16
- b. 10
- c. 17
- d. 12

BLANK 2

- a. 4
- b. 5
- c. 6
- d. 7

Answer: **16, 4**

16 and 4, unchanged from the other two mapping functions on the same machine. Only the middle field and the tag differ.

11.59

MEDIUM

Cache S12, Rev S48

In the 2-way example the number of sets is 64 over 2, which is _____ .

- a. 32
- b. 64
- c. 16
- d. 128

Answer: **32**

32 sets, so 5 set index bits.

Set associative Example 2 is which configuration?

- a. 4-way, main memory 512 KB, cache 8 KB, block 32 B
- b. 2-way, main memory 64 KB, cache 1 KB, block 16 B
- c. 8-way, main memory 4 MB, cache 32 KB, block 64 B
- d. 4-way, main memory 256 KB, cache 4 KB, block 32 B

✓ 4-way, main memory 512 KB, cache 8 KB, block 32 B

Correct. 4-way, 512 KB, 8 KB, 32 B.

✗ 2-way, main memory 64 KB, cache 1 KB, block 16 B

Wrong. That is set associative Example 1.

✗ 8-way, main memory 4 MB, cache 32 KB, block 64 B

Wrong. That is set associative Example 3.

✗ 4-way, main memory 256 KB, cache 4 KB, block 32 B

Wrong. 256 KB with 4 KB is fully associative Example 2.

4-way, 512 KB, 8 KB, 32 B. The three set associative examples step up the associativity: 2-way, 4-way, 8-way.

11.61

MEDIUM

Cache S13, Rev S49

Set associative Example 3 is which configuration?

- a. 8-way, main memory 4 MB, cache 32 KB, block 64 B
- b. 8-way, main memory 2 MB, cache 16 KB, block 64 B
- c. 4-way, main memory 512 KB, cache 8 KB, block 32 B
- d. 2-way, main memory 4 MB, cache 32 KB, block 64 B

✓ 8-way, main memory 4 MB, cache 32 KB, block 64 B

Correct. 8-way, 4 MB, 32 KB, 64 B.

✗ 8-way, main memory 2 MB, cache 16 KB, block 64 B

Wrong. 2 MB with 16 KB is fully associative Example 3.

✗ 4-way, main memory 512 KB, cache 8 KB, block 32 B

Wrong. That is set associative Example 2.

✗ 2-way, main memory 4 MB, cache 32 KB, block 64 B

Wrong associativity: Example 3 is 8-way.

8-way, 4 MB, 32 KB, 64 B. It is the largest case in the deck and the one with the deepest associativity.

11.62

MEDIUM

Cache S13, Rev S49

Set associative Example 3 is _____ -way, with a main memory of 4 MB.

- a. 8
- b. 2
- c. 4
- d. 16

Answer: **8**

8-way. The three examples run 2-way, 4-way, 8-way, and each doubling moves one bit from the set index into the tag.

Why is a cache mapping function needed at all?

- a. Because there are fewer cache lines than main memory blocks
- b. Because there are more cache lines than main memory blocks
- c. Because blocks and lines are different sizes
- d. Because the cache is volatile

✓ Because there are fewer cache lines than main memory blocks

Correct. Far fewer lines than blocks, so some rule has to decide which blocks may occupy which lines.

✗ Because there are more cache lines than main memory blocks

Wrong, and if it were true no mapping would be needed at all.

✗ Because blocks and lines are different sizes

Wrong. A block occupies a line, so they are the same size.

✗ Because the cache is volatile

Wrong. Volatility has nothing to do with placement.

Far fewer lines than blocks, so some rule has to decide which blocks may occupy which lines.

In the direct mapping expression $i \text{ equals } j \text{ modulo } m$, what do the letters mean?

- a. i is the cache line number, j is the main memory block number, m is the number of lines in the cache
- b. i is the block number, j is the line number, m is the number of blocks
- c. i is the tag, j is the index, m is the offset
- d. i is the set number, j is the line number, m is the associativity

✓ i is the cache line number, j is the main memory block number, m is the number of lines in the cache

Correct. Line equals block mod line count.

✗ i is the block number, j is the line number, m is the number of blocks

Wrong, and it swaps i and j .

✗ i is the tag, j is the index, m is the offset

Wrong. Those are the address fields, not the formula's letters.

✗ i is the set number, j is the line number, m is the associativity

Wrong. Sets belong to set associative mapping.

Line equals block mod line count. It is the same rule as the Cache deck's Cache Line equals Main Memory Block Number mod Number of Cache Addresses.

11.65

MEDIUM

Mem S40

Direct mapping is expressed as $i \text{ equals } j \text{ modulo } m$, where i is the cache _____ number and m is the number of _____ in the cache.

BLANK 1

- a. line
- b. block
- c. set
- d. word

BLANK 2

- a. lines
- b. blocks
- c. sets
- d. words

Answer: **line, lines**

| Line, and lines. Only j refers to main memory.

11.66

MEDIUM

Mem S43

To determine whether a block is in a direct mapped cache, you:

- a. Use the line field of the memory address to index the cache line, then compare the tag from the address with the line tag
- b. Simultaneously compare every line's tag
- c. Search the sets in order until a match is found
- d. Ask main memory whether the block is cached

✓ Use the line field of the memory address to index the cache line, then compare the tag from the address with the line tag

Correct. Index first, then compare one tag.

✗ Simultaneously compare every line's tag

Wrong. Comparing every tag at once is the associative method.

✗ Search the sets in order until a match is found

Wrong. Nothing is searched in order; the index goes straight to the line.

✗ Ask main memory whether the block is cached

Wrong. The cache answers for itself, using its tags.

| Index first, then compare one tag. That single comparison is why direct mapping is inexpensive.

11.67

MEDIUM

Mem S43

Use the _____ field of the memory address to index the cache line, then compare the tag from the address with the line tag.

- a. line
- b. tag
- c. offset
- d. block

Answer: **line**

| The line field, which the Cache deck calls the index.

If the tags do not match, what happens?

- a. A cache miss, and the indexed cache line is updated with the word and the tag
- b. A cache hit, and the word is delivered
- c. Another line is chosen to hold the block
- d. The address is discarded

✓ A cache miss, and the indexed cache line is updated with the word and the tag

Correct. A miss, and the line is overwritten with the new word and its tag.

✗ A cache hit, and the word is delivered

Wrong. A hit needs the tags to match.

✗ Another line is chosen to hold the block

Wrong. Choosing a line is what associative mapping has to do.

✗ The address is discarded

Wrong. The access still has to be served.

A miss, and the line is overwritten with the new word and its tag. There is no choice of line: the index already fixed it.

11.69

MEDIUM

Mem S43

If both do not match, then it is a cache _____, and the cache line is updated with the word and the _____.

BLANK 1

- a. miss
- b. hit
- c. stall
- d. flush

BLANK 2

- a. tag
- b. index
- c. offset
- d. address

Answer: **miss, tag**

A miss, and both the word and the tag are written. Without the new tag the line would claim to hold the wrong block.

11.70

MEDIUM

Mem S44

The disadvantage of direct mapping is that there is a _____ cache location for any given block, so two blocks mapping to the same line are continually swapped and the _____ ratio is low.

BLANK 1

- a. fixed
- b. random
- c. shared
- d. temporary

BLANK 2

- a. hit
- b. miss
- c. swap
- d. tag

Answer: **fixed, hit**

A fixed location, and a low hit ratio. Set associative mapping was invented to address exactly this.

11.71

MEDIUM

Mem S45

The memory deck's worked direct mapping example uses which figures?

- a. Cache 16 KB, block 256 bytes, main memory 128 KB
- b. Cache 1 KB, block 16 bytes, main memory 64 KB
- c. Cache 4 KB, block 32 bytes, main memory 128 KB
- d. Cache 8 KB, block 64 bytes, main memory 1 MB

✓ Cache 16 KB, block 256 bytes, main memory 128 KB

Correct. 16 KB, 256 bytes, 128 KB.

✗ Cache 1 KB, block 16 bytes, main memory 64 KB

Wrong. Those are the Cache deck's Example 1 figures.

✗ Cache 4 KB, block 32 bytes, main memory 128 KB

Wrong. Those are the Cache deck's direct mapping Example 2 figures.

✗ Cache 8 KB, block 64 bytes, main memory 1 MB

Wrong. Those are the Cache deck's direct mapping Example 3 figures.

16 KB, 256 bytes, 128 KB. The same three figures are then reused for the associative and 2-way set associative worked examples, so all three can be compared.

11.72

MEDIUM

Mem S45

In the worked cache examples, block size, frame size and line size all mean the same thing.

Answer: **true**

The deck writes Block size equals Frame size equals Line size on the given line of every worked example, so the three words are interchangeable here.

11.73

MEDIUM

Mem S46

With a main memory of 128 KB and byte addressable memory, how wide is the physical address?

- a. 17 bits
- b. 16 bits
- c. 18 bits
- d. 14 bits

✓ 17 bits

Correct. 128 KB is 2 to the power 17 bytes, so 17 bits.

✗ 16 bits

Wrong. 16 bits is 64 KB.

✗ 18 bits

Wrong. 18 bits is 256 KB.

✗ 14 bits

Wrong. 14 bits is 16 KB, which is the cache size here, not the memory.

128 KB is 2 to the power 17 bytes, so 17 bits. The assumption of byte addressability is stated explicitly and matters.

11.74

MEDIUM

Mem S46

In the worked examples the memory is taken to be ____ addressable.

- a. byte
- b. word
- c. block
- d. bit

Answer: **byte**

Byte addressable. Without that assumption the address width could not be read off the size in bytes.

11.75

MEDIUM

Mem S46

With a block size of 256 bytes, how many bits are in the block offset?

- a. 8
- b. 4
- c. 256
- d. 6

✓ 8

Correct. 256 bytes is 2 to the power 8, so 8 offset bits.

✗ 4

Wrong. 4 bits indexes 16 bytes, the Cache deck's block size.

✗ 256

Wrong. 256 is the size in bytes, not the bit count.

✗ 6

Wrong. 6 bits indexes 64 bytes.

256 bytes is 2 to the power 8, so 8 offset bits.

11.76

MEDIUM

Mem S46

A block size of 256 bytes is 2 to the power _____ bytes, so the block offset is _____ bits.

BLANK 1

- a. 8
- b. 4
- c. 6
- d. 16

BLANK 2

- a. 8
- b. 4
- c. 6
- d. 16

Answer: **8, 8**

2 to the 8, so 8 bits. The exponent is always the bit count.

11.77

MEDIUM

Mem S47

With a 16 KB cache and a 256 byte line, how many lines are there and how wide is the line number?

- a. 2 to the power 6 lines, so 6 bits
- b. 2 to the power 8 lines, so 8 bits
- c. 2 to the power 14 lines, so 14 bits
- d. 2 to the power 5 lines, so 5 bits

✓ 2 to the power 6 lines, so 6 bits

Correct. 2 to the 14 over 2 to the 8 is 2 to the 6, that is 64 lines, so the line number is 6 bits.

✗ 2 to the power 8 lines, so 8 bits

Wrong. 8 is the offset width, from the block size.

✗ 2 to the power 14 lines, so 14 bits

Wrong. 14 is the cache size exponent, before the block size is divided out.

✗ 2 to the power 5 lines, so 5 bits

Wrong. 5 bits is the set number in the 2-way example.

| 2 to the 14 over 2 to the 8 is 2 to the 6, that is 64 lines, so the line number is 6 bits.

11.78

MEDIUM

Mem S47

Total lines equals cache size over line size, so 2 to the 14 over 2 to the 8 is 2 to the power

_____ .

- a. 6
- b. 8
- c. 14
- d. 22

Answer: **6**

| 2 to the 6. Dividing powers subtracts the exponents, which is the quickest way through all of these examples.

11.79

MEDIUM

Mem S49

With 2 to the power 6 lines and a 3-bit tag, what is the tag directory size?

- a. 192 bits, that is 24 bytes
- b. 384 bits, that is 48 bytes
- c. 576 bits, that is 72 bytes
- d. 64 bits, that is 8 bytes

✓ 192 bits, that is 24 bytes

Correct. 64 times 3 is 192 bits, which is 24 bytes.

✗ 384 bits, that is 48 bytes

Wrong. 384 bits is the Cache deck's Example 1, with a 6-bit tag.

✗ 576 bits, that is 72 bytes

Wrong. 576 bits is the fully associative version of this same example, with a 9-bit tag.

✗ 64 bits, that is 8 bytes

Wrong. That forgets to multiply by the tag width.

| 64 times 3 is 192 bits, which is 24 bytes.

11.80

MEDIUM

Mem S49

Tag directory size is the number of tags times the tag size, giving 192 bits, that is _____ bytes.

- a. 24
- b. 48
- c. 72
- d. 8

Answer: **24**

| 24 bytes, from 192 over 8.

Associative mapping overcomes the disadvantage of direct mapping by:

- a. Permitting each main memory block to be loaded into any line of the cache
- b. Making the cache larger
- c. Making the block size smaller
- d. Grouping lines into sets

✓ Permitting each main memory block to be loaded into any line of the cache

Correct. Any line, so two hot blocks need never collide.

✗ Making the cache larger

Wrong. Size is not the mechanism.

✗ Making the block size smaller

Wrong. Block size does not address the collision problem.

✗ Grouping lines into sets

Wrong. That is set associative mapping.

Any line, so two hot blocks need never collide. Grouping lines into sets is the set associative compromise, not this one.

11.82

MEDIUM

Mem S50

In associative mapping, the cache control logic interprets a memory address simply as:

- a. A Tag and a Word field
- b. A Tag, an Index and a Word field
- c. A Tag and a Set field
- d. An Opcode and an Address field

✓ A Tag and a Word field

Correct. Tag and Word, which the Cache deck calls Tag and Offset.

✗ A Tag, an Index and a Word field

Wrong. An index is what associative mapping does without.

✗ A Tag and a Set field

Wrong. A set field belongs to set associative mapping.

✗ An Opcode and an Address field

Wrong. That is an instruction format.

Tag and Word, which the Cache deck calls Tag and Offset. Two fields, because there is no line to select.

11.83

MEDIUM

Mem S50

In associative mapping the Tag field _____ identifies a block of main memory.

- a. uniquely
- b. partly
- c. loosely
- d. temporarily

Answer: **uniquely**

Uniquely. With no index to narrow the search, the tag alone has to name the block.

11.84

MEDIUM

Mem S50

A fully associative cache can find a block by checking a single line, as a direct mapped cache does.

Answer: **false**

It must examine every line's tag simultaneously, because the block could be anywhere. That is the hardware cost of full associativity.

11.85

MEDIUM

Mem S52

In associative mapping the address splits into a Tag of s bits and a Word field of w bits. What does each identify?

- a. The Tag uniquely identifies a block of main memory, and the Word uniquely identifies a word within a block
- b. The Tag identifies a word, and the Word identifies a block
- c. The Tag identifies a line, and the Word identifies a set
- d. Both identify blocks

✓ The Tag uniquely identifies a block of main memory, and the Word uniquely identifies a word within a block

Correct. Tag names the block, Word names the position inside it.

✗ The Tag identifies a word, and the Word identifies a block

Wrong, and reversed.

✗ The Tag identifies a line, and the Word identifies a set

Wrong. Lines and sets do not appear in the fully associative address.

✗ Both identify blocks

Wrong. Only the tag identifies a block.

Tag names the block, Word names the position inside it. The Word field is the same thing the Cache deck calls the offset.

11.86

MEDIUM

Mem S52

The Tag is _____ bits and the Word is _____ bits.

BLANK 1

- a. s
- b. w
- c. n
- d. m

BLANK 2

- a. w
- b. s
- c. n
- d. k

Answer: **s, w**

s for the tag, w for the word. Together they make up the whole address.

11.87

MEDIUM

Mem S53

If a match exists in the simultaneous tag comparison, what happens?

- a. A cache hit: the tag field is used to index the cache line and the word is retrieved
- b. A cache miss: a line is chosen and updated
- c. The set is searched in order
- d. The block is fetched from main memory anyway

✓ A cache hit: the tag field is used to index the cache line and the word is retrieved

Correct. A hit, and the matching tag identifies which line to read from.

✗ A cache miss: a line is chosen and updated

Wrong. That is what happens when no match exists.

✗ The set is searched in order

Wrong. All tags are compared at once, not in order.

✗ The block is fetched from main memory anyway

Wrong. A hit is served from the cache.

A hit, and the matching tag identifies which line to read from.

11.88

MEDIUM

Mem S53

If a match exists it is a cache _____, and the tag field of the memory address is used to index the cache line.

- a. hit
- b. miss
- c. fault
- d. stall

Answer: **hit**

A hit. Note that here the TAG does the indexing, since there is no index field of its own.

11.89

MEDIUM

Mem S53

If a match does not exist it is a cache miss: choose a cache line, then update it with the word and the _____.

- a. tag
- b. index
- c. offset
- d. set

Answer: **tag**

The tag, exactly as in the direct mapped case. What differs is that here the line had to be chosen first.

Worked associative Example 1 uses which configuration?

- a. A fully associative cache of 16 KB, block 256 bytes, main memory 128 KB
- b. A direct mapped cache of 16 KB, block 256 bytes, main memory 128 KB
- c. A fully associative cache of 4 KB, block 32 bytes, main memory 256 KB
- d. A 2-way cache of 16 KB, block 256 bytes, main memory 128 KB

✓ A fully associative cache of 16 KB, block 256 bytes, main memory 128 KB
Correct. The same figures as the direct mapping worked example, but fully associative.

✗ A direct mapped cache of 16 KB, block 256 bytes, main memory 128 KB
Wrong. Those are the same numbers but the earlier, direct mapped example.

✗ A fully associative cache of 4 KB, block 32 bytes, main memory 256 KB
Wrong. Those are Cache deck figures.

✗ A 2-way cache of 16 KB, block 256 bytes, main memory 128 KB
Wrong. The 2-way version of these numbers is the set associative worked example.

The same figures as the direct mapping worked example, but fully associative. Holding the numbers fixed is what makes the comparison instructive: a 3-bit tag becomes a 9-bit one.

11.91

MEDIUM

Mem S55

In worked associative Example 1, what is the physical address width?

- a. 17 bits, because 128 KB is 2 to the power 17 bytes
- b. 14 bits, because 16 KB is 2 to the power 14 bytes
- c. 8 bits, because the block is 256 bytes
- d. 34 bits

✓ 17 bits, because 128 KB is 2 to the power 17 bytes

Correct. The address width always comes from the main memory size, never from the cache size.

✗ 14 bits, because 16 KB is 2 to the power 14 bytes

Wrong. 14 bits is the cache size exponent, which sets the line count, not the address.

✗ 8 bits, because the block is 256 bytes

Wrong. 8 bits is the block offset.

✗ 34 bits

Wrong. 34 bits is associative Example 2, whose memory is 16 GB.

| The address width always comes from the main memory size, never from the cache size.

11.92

MEDIUM

Mem S55

128 KB is 2 to the power _____ bytes.

- a. 17
- b. 16
- c. 18
- d. 14

Answer: **17**

| 2 to the 17, since 128 is 2 to the 7 and a KB is 2 to the 10.

11.93

MEDIUM

Mem S56

The tag is 17 minus 8, giving _____ bits, which is _____ bits more than the direct mapped tag for the same machine.

BLANK 1

- a. 9
- b. 3
- c. 11
- d. 6

BLANK 2

- a. 6
- b. 3
- c. 9
- d. 8

Answer: **9, 6**

9 bits, which is 6 more than 3. Those 6 bits are exactly the line number field that full associativity gives up.

11.94

MEDIUM

Mem S56

In worked associative Example 1, how many lines are in the cache?

- a. 2 to the power 6, that is 64
- b. 2 to the power 8, that is 256
- c. 2 to the power 14
- d. 2 to the power 9

✓ 2 to the power 6, that is 64

Correct. 16 KB over 256 bytes is 2 to the 14 over 2 to the 8, which is 2 to the 6.

✗ 2 to the power 8, that is 256

Wrong. 2 to the 8 is the block size in bytes.

✗ 2 to the power 14

Wrong. That is the cache size in bytes.

✗ 2 to the power 9

Wrong. 9 is the tag width.

16 KB over 256 bytes is 2 to the 14 over 2 to the 8, which is 2 to the 6. The line count does not depend on the mapping function.

11.95

MEDIUM

Mem S56

Total lines equals cache size over line size, so 16 KB over 256 bytes gives 2 to the power _____ lines.

- a. 6
- b. 8
- c. 14
- d. 9

Answer: **6**

2 to the 6, that is 64 lines, the same as in the direct mapped version.

11.96

MEDIUM

Mem S57

The tag directory is 2 to the 6 times 9 bits, which is 576 bits, that is _____ bytes.

- a. 72
- b. 24
- c. 96
- d. 56

Answer: **72**

72 bytes, three times the direct mapped figure for the same cache, because the tag is three times as wide.

11.97

MEDIUM

Mem S58

Worked associative Example 2 has a block size of 4 KB and a main memory of 16 GB. How many bits is 16 GB?

- a. 2 to the power 34 bytes, so 34 address bits
- b. 2 to the power 32 bytes, so 32 address bits
- c. 2 to the power 30 bytes, so 30 address bits
- d. 2 to the power 24 bytes, so 24 address bits

✓ 2 to the power 34 bytes, so 34 address bits

Correct. 16 is 2 to the 4 and a GB is 2 to the 30, so 16 GB is 2 to the 34 bytes.

✗ 2 to the power 32 bytes, so 32 address bits

Wrong. 2 to the 32 is 4 GB.

✗ 2 to the power 30 bytes, so 30 address bits

Wrong. 2 to the 30 is 1 GB.

✗ 2 to the power 24 bytes, so 24 address bits

Wrong. 2 to the 24 is 16 MB.

16 is 2 to the 4 and a GB is 2 to the 30, so 16 GB is 2 to the 34 bytes.

11.98

MEDIUM

Mem S58

In worked associative Example 2 the block size is ____ KB.

- a. 4
- b. 16
- c. 256
- d. 8

Answer: **4**

4 KB, which is 2 to the power 12 bytes, so the block offset is 12 bits.

11.99

MEDIUM

Mem S59

In associative Example 2 the block offset is ____ bits and the tag is 34 minus that, which is ____ bits.

BLANK 1

- a. 12
- b. 8
- c. 22
- d. 10

BLANK 2

- a. 22
- b. 26
- c. 12
- d. 9

Answer: **12, 22**

12 and 22. In fully associative mapping the tag is always the address minus the offset, nothing else.

11.100

MEDIUM

Mem S61

Set associative mapping is described as:

- a. An enhanced form of direct mapping in which the drawbacks of direct mapping are removed
- b. An enhanced form of associative mapping
- c. A form of sequential access
- d. The same as direct mapping with a larger cache

✓ An enhanced form of direct mapping in which the drawbacks of direct mapping are removed

Correct. An enhanced direct mapping, and specifically one that addresses the problem of possible thrashing.

✗ An enhanced form of associative mapping

Wrong. The deck derives it from direct mapping, not from associative.

✗ A form of sequential access

Wrong. Sequential access is an access method for tape.

✗ The same as direct mapping with a larger cache

Wrong. Enlarging the cache would not remove the fixed-location drawback.

An enhanced direct mapping, and specifically one that addresses the problem of possible thrashing.

11.101

MEDIUM

Mem S61

Set associative mapping addresses the problem of possible _____ in the direct mapping method.

- a. thrashing
- b. flushing
- c. stalling
- d. overflow

Answer: **thrashing**

Thrashing, the continual swapping of two blocks that map to the same line.

11.102

MEDIUM

Mem S61

How does set associative mapping change the placement rule?

- a. A few lines are grouped into a set, and a block in memory can map to any one of the lines of a specific set
- b. A block can map to any line in the cache
- c. A block can map to exactly one line
- d. Blocks are placed in the least recently used line of the whole cache

✓ A few lines are grouped into a set, and a block in memory can map to any one of the lines of a specific set

Correct. Any line within one set.

✗ A block can map to any line in the cache

Wrong. That is fully associative mapping.

✗ A block can map to exactly one line

Wrong. That is direct mapping.

✗ Blocks are placed in the least recently used line of the whole cache

Wrong. The set restricts the choice; a replacement policy then picks within it.

Any line within one set. It is the midpoint between one line, which is direct, and every line, which is fully associative.

11.103

MEDIUM

Mem S61

Instead of exactly one line that a block can map to, a few lines are grouped together creating a ____ .

- a. set
- b. block
- c. frame
- d. page

Answer: **set**

A set. The number of lines in each set is the n of n-way.

Worked set associative Example 1 uses which configuration?

- a. 2-way, cache 16 KB, block 256 bytes, main memory 128 KB, so the set size is 2
- b. 4-way, cache 8 KB, block 32 bytes, main memory 512 KB
- c. 2-way, cache 1 KB, block 16 bytes, main memory 64 KB
- d. 8-way, cache 32 KB, block 64 bytes, main memory 4 MB

✓ 2-way, cache 16 KB, block 256 bytes, main memory 128 KB, so the set size is 2
Correct. The same 16 KB, 256 byte, 128 KB machine as the other two worked examples, now 2-way.

✗ 4-way, cache 8 KB, block 32 bytes, main memory 512 KB
Wrong. That is the Cache deck's set associative Example 2.

✗ 2-way, cache 1 KB, block 16 bytes, main memory 64 KB
Wrong. That is the Cache deck's set associative Example 1.

✗ 8-way, cache 32 KB, block 64 bytes, main memory 4 MB
Wrong. That is the Cache deck's set associative Example 3.

The same 16 KB, 256 byte, 128 KB machine as the other two worked examples, now 2-way. Three mapping functions, one machine.

In worked set associative Example 1, what are the physical address and block offset widths?

- a. 17 bits and 8 bits
- b. 16 bits and 4 bits
- c. 17 bits and 6 bits
- d. 14 bits and 8 bits

✓ 17 bits and 8 bits

Correct. The same 17 and 8 as the other two worked examples, since the memory and block sizes are unchanged.

✗ 16 bits and 4 bits

Wrong. Those are the Cache deck's figures for a different machine.

✗ 17 bits and 6 bits

Wrong. 6 bits is the line number here, not the offset.

✗ 14 bits and 8 bits

Wrong. 14 bits is the cache size exponent.

The same 17 and 8 as the other two worked examples, since the memory and block sizes are unchanged.

11.106

MEDIUM

Mem S63

In worked set associative Example 1, 128 KB gives a _____ -bit physical address and a 256 byte block gives an _____ -bit block offset.

BLANK 1

- a. 17
- b. 14
- c. 16
- d. 18

BLANK 2

- a. 8
- b. 6
- c. 4
- d. 12

Answer: **17, 8**

17 and 8. Only the middle field and the tag differ between the three mapping functions on this machine.

11.107

MEDIUM

Mem S64

In worked set associative Example 1, how many lines are in the cache?

- a. 64
- b. 32
- c. 128
- d. 256

✓ 64

Correct. 16 KB over 256 bytes is 64 lines, the same as in the direct mapped and fully associative versions.

✗ 32

Wrong. 32 is the number of sets, once the lines are paired.

✗ 128

Wrong. That would need a 128 byte block.

✗ 256

Wrong. 256 is the block size in bytes.

16 KB over 256 bytes is 64 lines, the same as in the direct mapped and fully associative versions.

11.108

MEDIUM

Mem S64

Total number of lines in cache equals cache size over line size, so 16 KB over 256 bytes is _____ lines.

- a. 64
- b. 32
- c. 128
- d. 16

Answer: **64**

64 lines. The line count never depends on the associativity.

11.109

MEDIUM

Mem S64

Total sets equals total lines over set size, so 64 over 2 is ____ sets, which is 2 to the power ____ .

BLANK 1

- a. 32
- b. 64
- c. 16
- d. 128

BLANK 2

- a. 5
- b. 6
- c. 4
- d. 7

Answer: **32, 5**

32 sets, 2 to the 5, so a 5-bit set number and therefore a 4-bit tag on this machine.

11.110

HARD

Cache S2, Cache S8, Cache S11, Rev S38

Match each mapping function to its address format.

Direct

Tag, Index, Offset

Fully associative

Tag, Offset

Set associative

Tag, Set Index, Offset

The offset is always there because a block always has to be indexed into. What changes is the middle field: an index, nothing, or a set index.

The worked address from the CPU is 000010 000011 1001. How does the deck read its three fields?

- a. Tag 2, index 3, offset 9
- b. Tag 3, index 2, offset 9
- c. Tag 2, index 9, offset 3
- d. Tag 9, index 3, offset 2

✓ Tag 2, index 3, offset 9

Correct. 000010 is 2, 000011 is 3, and 1001 is 9, in the field order tag, index, offset.

✗ Tag 3, index 2, offset 9

Wrong. 000010 is 2 and comes first.

✗ Tag 2, index 9, offset 3

Wrong. The 4-bit group at the end is the offset.

✗ Tag 9, index 3, offset 2

Wrong. The tag is the leading 6 bits.

| 000010 is 2, 000011 is 3, and 1001 is 9, in the field order tag, index, offset.

Select the correct values in the full answer to direct mapping Example 1.

- a. Offset 4 bits
- b. Index 6 bits
- c. Tag 6 bits
- d. Tag directory size 384 bits, that is 48 bytes
- e. Tag 12 bits

✓ Offset 4 bits

Correct, from the 16-byte block.

✓ Index 6 bits

Correct, from the 64 lines.

✓ Tag 6 bits

Correct, from 16 minus 6 minus 4.

✓ Tag directory size 384 bits, that is 48 bytes

Correct, from 64 times 6.

✗ Tag 12 bits

Wrong. 12 bits is the tag when there is no index field, that is under fully associative mapping.

The four-line answer the deck gives. A 12-bit tag is the fully associative result for the same machine, which is what changes when the index field disappears.

In fully associative Example 1, what is the tag width?

- a. 12 bits, from 16 minus 4
- b. 6 bits, from 16 minus 6 minus 4
- c. 7 bits, from 16 minus 5 minus 4
- d. 4 bits

✓ 12 bits, from 16 minus 4

Correct. 16 minus 4 is 12, because there is no index field to subtract.

✗ 6 bits, from 16 minus 6 minus 4

Wrong. That subtraction includes an index, which fully associative does not have.

✗ 7 bits, from 16 minus 5 minus 4

Wrong. That is the 2-way set associative calculation.

✗ 4 bits

Wrong. 4 bits is the offset.

16 minus 4 is 12, because there is no index field to subtract. That is 6 bits more tag than the direct mapped case on the same machine.

11.114**HARD**

Cache S9, Rev S45

In fully associative mapping the tag is the address width minus the _____ only, giving 16 minus 4, which is _____ bits.

BLANK 1

- a. offset
- b. index
- c. set index
- d. line number

BLANK 2

- a. 12
- b. 6
- c. 7
- d. 10

Answer: **offset, 12**

Minus the offset only, giving 12 bits. Every bit not spent on an index has to be spent on the tag.

In fully associative Example 1, what is the tag directory size?

- a. 768 bits, which is 96 bytes
- b. 384 bits, which is 48 bytes
- c. 448 bits, which is 56 bytes
- d. 576 bits, which is 72 bytes

✓ 768 bits, which is 96 bytes

Correct. 64 lines times 12 tag bits is 768 bits, which is 96 bytes.

✗ 384 bits, which is 48 bytes

Wrong. 384 bits is the direct mapped answer for the same machine.

✗ 448 bits, which is 56 bytes

Wrong. 448 bits is the 2-way set associative answer.

✗ 576 bits, which is 72 bytes

Wrong. 576 bits is the memory deck's associative example, with 9 tag bits.

64 lines times 12 tag bits is 768 bits, which is 96 bytes. It is exactly double the direct mapped figure, because the tag is double the width.

11.116**HARD**

Cache S12, Rev S48

With 64 cache lines in a 2-way set associative cache, how many sets are there and how wide is the set index?

- a. 32 sets, set index 5 bits
- b. 64 sets, set index 6 bits
- c. 32 sets, set index 6 bits
- d. 16 sets, set index 4 bits

✓ 32 sets, set index 5 bits

Correct. $\frac{64}{2}$ is 32 sets, and 32 is 2 to the power 5, so the set index is 5 bits.

✗ 64 sets, set index 6 bits

Wrong. 64 sets would be direct mapping, one line per set.

✗ 32 sets, set index 6 bits

Wrong. 32 needs 5 bits, not 6.

✗ 16 sets, set index 4 bits

Wrong. That would be a 4-way cache on this machine.

$\frac{64}{2}$ is 32 sets, and 32 is 2 to the power 5, so the set index is 5 bits. Doubling the associativity always costs one bit of index.

In the 2-way set associative Example 1, what are the tag width and the tag directory size?

- a. Tag 7 bits, directory 448 bits, that is 56 bytes
- b. Tag 6 bits, directory 384 bits, that is 48 bytes
- c. Tag 12 bits, directory 768 bits, that is 96 bytes
- d. Tag 7 bits, directory 224 bits, that is 28 bytes

✓ Tag 7 bits, directory 448 bits, that is 56 bytes

Correct. 16 minus 5 minus 4 gives a 7-bit tag, and 7 times 64 is 448 bits, which is 56 bytes.

✗ Tag 6 bits, directory 384 bits, that is 48 bytes

Wrong. That is the direct mapped answer, where the index is 6 bits.

✗ Tag 12 bits, directory 768 bits, that is 96 bytes

Wrong. That is the fully associative answer, with no index at all.

✗ Tag 7 bits, directory 224 bits, that is 28 bytes

Wrong. Multiplying by the 32 sets instead of the 64 lines gives this; every line needs its own tag.

16 minus 5 minus 4 gives a 7-bit tag, and 7 times 64 is 448 bits, which is 56 bytes. The multiplier is the line count, 64, not the set count.

11.118**HARD**

Cache S12, Rev S48

The 2-way tag is 16 minus 5 minus 4, which is _____ bits, and the tag directory is 7 times 64, which is _____ bits.

BLANK 1

- a. 7
- b. 6
- c. 12
- d. 5

BLANK 2

- a. 448
- b. 384
- c. 768
- d. 224

Answer: **7, 448**

7 bits and 448 bits, that is 56 bytes. Set it beside the other two answers for the same machine: 384 bits direct, 448 bits 2-way, 768 bits fully associative.

What is thrashing, in the direct mapping disadvantage?

- a. Two blocks that map to the same line being continually swapped, so the hit ratio is low
- b. The cache being flushed at every context switch
- c. The pipeline stalling on a data hazard
- d. A block being written to two lines at once

✓ Two blocks that map to the same line being continually swapped, so the hit ratio is low
Correct, and the deck names the effect thrashing.

✗ The cache being flushed at every context switch
Wrong. That is the virtual cache problem.

✗ The pipeline stalling on a data hazard
Wrong. Stalling belongs to the pipelining topic.

✗ A block being written to two lines at once
Wrong. A block occupies one line at a time.

Repeated references to two different blocks that map into the same line evict each other in turn, so neither is ever there when wanted.

11.120

HARD

Mem S48

In the memory deck's direct mapping worked example, what is the number of tag bits?

- a. 3
- b. 6
- c. 9
- d. 8

✓ 3

Correct. 17 minus 6 plus 8, that is 17 minus 14, giving 3 bits.

✗ 6

Wrong. 6 is the line number width.

✗ 9

Wrong. 9 is the tag in the fully associative version of the same example.

✗ 8

Wrong. 8 is the block offset.

17 minus 6 plus 8, that is 17 minus 14, giving 3 bits. It is a very small tag, because the cache is large relative to the memory.

11.121

HARD

Mem S48

The tag is 17 minus the sum of 6 and 8, that is 17 minus _____, giving _____ bits.

BLANK 1

- a. 14
- b. 6
- c. 8
- d. 3

BLANK 2

- a. 3
- b. 6
- c. 8
- d. 9

Answer: **14, 3**

17 minus 14 is 3. Adding the three fields back gives 17, which is the check.

To determine whether a block is in a fully associative cache, the cache control logic must:

- a. Simultaneously examine every line's tag for a match
- b. Examine one line's tag
- c. Examine the tags of one set
- d. Ask main memory

✓ Simultaneously examine every line's tag for a match

Correct, and simultaneously is the deck's word.

✗ Examine one line's tag

Wrong. One line is enough only in direct mapping.

✗ Examine the tags of one set

Wrong. Searching one set is the set associative method.

✗ Ask main memory

Wrong. The cache decides for itself.

Every tag at once. That is expensive in hardware, and it is the price of placement freedom.

If no match exists in a fully associative cache, what extra problem arises that direct mapping does not have?

- a. A cache line has to be chosen, since the block could go in any of them
- b. The tag has to be recomputed
- c. The block offset has to be widened
- d. The whole cache must be flushed

✓ A cache line has to be chosen, since the block could go in any of them

Correct, and it is why replacement policies exist.

✗ The tag has to be recomputed

Wrong. The tag comes from the address.

✗ The block offset has to be widened

Wrong. Field widths are fixed by the design.

✗ The whole cache must be flushed

Wrong. Only one line is replaced.

Choosing a victim line, which the deck marks with its own question, How? In direct mapping the index has already decided.

In fully associative mapping the number of tag bits is:

- a. Physical address bits minus block offset bits, so 17 minus 8, which is 9
- b. Physical address bits minus line number bits, so 17 minus 6, which is 11
- c. 17 minus 6 minus 8, which is 3
- d. 8

✓ Physical address bits minus block offset bits, so 17 minus 8, which is 9

Correct. Only the offset is subtracted, giving 9 bits against the 3 bits of the direct mapped version of the same machine.

✗ Physical address bits minus line number bits, so 17 minus 6, which is 11

Wrong. There is no line number field to subtract in a fully associative cache.

✗ 17 minus 6 minus 8, which is 3

Wrong. That is the direct mapped calculation.

✗ 8

Wrong. 8 is the offset.

Only the offset is subtracted, giving 9 bits against the 3 bits of the direct mapped version of the same machine.

In worked associative Example 1, what is the tag directory size?

- a. 576 bits, that is 72 bytes
- b. 192 bits, that is 24 bytes
- c. 768 bits, that is 96 bytes
- d. 448 bits, that is 56 bytes

✓ 576 bits, that is 72 bytes

Correct. $2^6 \times 9$ bits is 576 bits, which is 72 bytes.

✗ 192 bits, that is 24 bytes

Wrong. 192 bits is the direct mapped answer, with a 3-bit tag.

✗ 768 bits, that is 96 bytes

Wrong. 768 bits belongs to the Cache deck's fully associative Example 1.

✗ 448 bits, that is 56 bytes

Wrong. 448 bits belongs to the Cache deck's 2-way example.

2 to the 6 times 9 bits is 576 bits, which is 72 bytes. Compare it with 192 bits for the direct mapped version of the very same machine.

11.126**HARD**

Mem S59

In worked associative Example 2, what is the number of tag bits?

- a. 22
- b. 34
- c. 12
- d. 9

✓ 22

Correct. 34 minus 12 is 22 bits.

✗ 34

Wrong. 34 is the whole address.

✗ 12

Wrong. 12 is the block offset.

✗ 9

Wrong. 9 is the tag in Example 1.

34 minus 12 is 22 bits. The physical address is 34 bits and the 4 KB block gives a 12-bit offset.

11.127

HARD

Mem S61

Set associative cache mapping is said to combine:

- a. The best of direct and associative cache mapping techniques
- b. The worst of both techniques
- c. Sequential and random access
- d. Physical and logical addressing

✓ The best of direct and associative cache mapping techniques

Correct. The best of both: some placement freedom, but only a set's worth of tags to compare.

✗ The worst of both techniques

Wrong, and it inverts the claim.

✗ Sequential and random access

Wrong. Those are access methods, not mapping functions.

✗ Physical and logical addressing

Wrong. That distinction belongs to cache addresses.

| The best of both: some placement freedom, but only a set's worth of tags to compare.

11.128

HARD

Mem S61

Set-associative mapping allows each word present in the cache to have two or more words in main memory for the same index address.

Answer: **true**

| That is the deck's own phrasing, and it is the point: several memory blocks sharing an index no longer have to fight over a single line.

With 64 lines and a set size of 2, how many sets are there and how wide is the set number?

- a. 32 sets, that is 2 to the power 5, so the set number is 5 bits
- b. 64 sets, that is 2 to the power 6, so the set number is 6 bits
- c. 128 sets, so the set number is 7 bits
- d. 16 sets, so the set number is 4 bits

✓ 32 sets, that is 2 to the power 5, so the set number is 5 bits

Correct. $\frac{64}{2}$ is 32, and 32 is 2 to the 5, so 5 bits.

✗ 64 sets, that is 2 to the power 6, so the set number is 6 bits

Wrong. 64 sets would mean one line per set, which is direct mapping.

✗ 128 sets, so the set number is 7 bits

Wrong. There cannot be more sets than lines.

✗ 16 sets, so the set number is 4 bits

Wrong. 16 sets would be a 4-way cache on this machine.

$\frac{64}{2}$ is 32, and 32 is 2 to the 5, so 5 bits. Halving the number of index-like entries always costs one bit.

12 Pipelining and Hazards

12.1 EASY Pipe S3

One of the major goals of Computer Architecture is to improve the speed and overall performance of a computer system:

- a. Without excessively increasing hardware cost
- b. Whatever the hardware cost
- c. By reducing the instruction set
- d. By increasing memory size

✓ Without excessively increasing hardware cost

Correct. Speed without excessive cost.

✗ Whatever the hardware cost

Wrong. Cost is explicitly constrained in the deck's phrasing.

✗ By reducing the instruction set

Wrong. That is the RISC philosophy, a different topic.

✗ By increasing memory size

Wrong. Memory size is not the lever here.

Speed without excessive cost. The qualification matters: pipelining is attractive precisely because it costs little extra hardware.

12.2 EASY Pipe S3

The goal is to improve speed and overall performance without _____ increasing hardware cost.

- a. excessively
- b. slightly
- c. ever
- d. measurably

Answer: **excessively**

Excessively. Some extra hardware is accepted, notably the pipeline registers.

12.3

EASY

Pipe S3

Pipelining is defined as a performance enhancement technique in which:

- a. Multiple instructions are processed simultaneously by dividing instruction execution into several stages
- b. One instruction is completed before the next begins
- c. Instructions are reordered by the compiler
- d. Multiple processors run the same instruction

✓ Multiple instructions are processed simultaneously by dividing instruction execution into several stages

Correct. Simultaneous processing through staged execution.

✗ One instruction is completed before the next begins

Wrong, and that is precisely the non-pipelined behaviour pipelining replaces.

✗ Instructions are reordered by the compiler

Wrong. Reordering is one way to avoid stalls, not the definition of pipelining.

✗ Multiple processors run the same instruction

Wrong. That is parallelism across processors.

Simultaneous processing through staged execution. Dividing into stages is what makes the simultaneity possible.

12.4

EASY

Pipe S3

Pipelining processes multiple instructions simultaneously by dividing instruction execution into several ____ .

- a. stages
- b. programs
- c. registers
- d. blocks

Answer: **stages**

Stages, which the deck also calls steps or segments.

12.5

EASY

Pipe S4

Instead of waiting for one instruction to complete entirely before starting another, the pipelined processor:

- a. Overlaps the execution of multiple instructions, like an assembly line in a factory
- b. Executes instructions out of order
- c. Duplicates the whole processor
- d. Skips instructions that are not needed

✓ Overlaps the execution of multiple instructions, like an assembly line in a factory

Correct. Overlapping, and the assembly line is the deck's own comparison.

✗ Executes instructions out of order

Wrong. Out-of-order execution is not part of this course.

✗ Duplicates the whole processor

Wrong. Duplicating functional units is a different technique, though the deck notes a 4-stage pipeline resembles 4 identical units.

✗ Skips instructions that are not needed

Wrong. Nothing is skipped.

Overlapping, and the assembly line is the deck's own comparison. It returns twice more, as the laundry and the exam grading.

12.6

EASY

Pipe S4

The pipelining approach is similar to an _____ line in a factory, where different stages of production occur at the same time.

- a. assembly
- b. production
- c. queue
- d. delivery

Answer: **assembly**

Assembly line. The deck later repeats it as an automobile assembly line.

12.7

EASY

Pipe S4

A typical instruction pipeline consists of ____ stages.

- a. 5
- b. 4
- c. 6
- d. 3

Answer: **5**

Five. The revision example uses six, and the performance example uses four, so the number is not fixed by nature.

12.8

EASY

Pipe S4

Which stage stores the result back into a register?

- a. Write Back
- b. Memory Access
- c. Execute
- d. Instruction Decode

✓ Write Back

Correct. Write Back, the last stage.

✗ Memory Access

Wrong. MEM accesses memory if required.

✗ Execute

Wrong. EX performs the operation but does not commit the result.

✗ Instruction Decode

Wrong. ID interprets the instruction and reads registers.

Write Back, the last stage. Memory Access writes memory, not registers.

12.9**EASY**

Pipe S5

Pipelining increases the _____ of the processor, enabling more instructions to be completed in less time.

- a. throughput
- b. latency
- c. frequency
- d. capacity

Answer: **throughput**

Throughput, defined later as the number of items that exit the pipeline per unit time.

12.10**EASY**

Pipe S5

Pipelining introduces challenges such as pipeline _____, data dependencies, branch instructions and resource conflicts.

- a. hazards
- b. faults
- c. misses
- d. interrupts

Answer: **hazards**

Hazards, the word the whole second half of the deck turns on.

The deck says pipelining is widely used in modern processors, including those developed by:

- a. Intel and AMD
- b. IBM and Stanford
- c. ARM and MIPS
- d. UNIVAC and CDC

✓ Intel and AMD

Correct. Intel and AMD.

✗ IBM and Stanford

Wrong. IBM and Stanford are named among the first RISC projects.

✗ ARM and MIPS

Wrong. ARM and MIPS are RISC architectures.

✗ UNIVAC and CDC

Wrong. UNIVAC and CDC are named as old one's complement machines.

Intel and AMD. The other names all appear elsewhere in the course, which makes them plausible distractors.

12.12**EASY**

Pipe S10

Pipelining is described on this slide as a technique for:

- a. Overlapping operations during execution
- b. Duplicating operations across processors
- c. Reordering operations before execution
- d. Removing unnecessary operations

✓ Overlapping operations during execution

Correct. Overlapping operations during execution, and the deck adds that today this is a key feature that makes fast CPUs.

✗ Duplicating operations across processors

Wrong. Duplication across processors is multi-processing.

✗ Reordering operations before execution

Wrong. Reordering is a scheduling technique.

✗ Removing unnecessary operations

Wrong. Nothing is removed.

Overlapping operations during execution, and the deck adds that today this is a key feature that makes fast CPUs.

12.13**EASY**

Pipe S10

The deck says pipelining is today a key feature that makes fast CPUs.

Answer: **true**

It does, which is why the topic carries so much weight in the course.

12.14**EASY**

Pipe S10

The deck names _____ different types of pipeline.

- a. 3
- b. 2
- c. 4
- d. 5

Answer: **3**

Three: instruction, operation, and multi-issue.

12.15

EASY

Pipe S12

What does the deck compare a pipeline to on this slide?

- a. An automobile assembly line
- b. A telephone exchange
- c. A postal sorting office
- d. A railway timetable

✓ An automobile assembly line

Correct. An automobile assembly line, with many steps, stages or segments connected to form a pipe.

✗ A telephone exchange

Wrong. Not used in this course.

✗ A postal sorting office

Wrong. Not used in this course.

✗ A railway timetable

Wrong. Not used in this course.

| An automobile assembly line, with many steps, stages or segments connected to form a pipe.

12.16

EASY

Pipe S12

An instruction enters the pipeline through one end and progresses through the stages to exit at the other end.

Answer: **true**

| That is the deck's picture, and it is why the pipeline has a fill time at the start and a drain time at the end.

12.17

EASY

Pipe S13

A pipeline whose stage lengths are equal is called a _____ pipeline.

- a. balanced
- b. uniform
- c. regular
- d. static

Answer: **balanced**

Balanced. Unbalanced lengths of pipe stages reduce speedup.

12.18

EASY

Pipe S14

The four people in the laundry analogy are Ann, Brian, Cathy and _____ .

- a. Dave
- b. David
- c. Dan
- d. Derek

Answer: **Dave**

Dave. Four loads, labelled A, B, C and D on the timing diagram.

12.19

EASY

Pipe S15

Run sequentially, how long does the laundry analogy take for 4 loads?

- a. 8 hours, that is 480 minutes
- b. 3.5 hours
- c. 2 hours
- d. 4 hours

✓ 8 hours, that is 480 minutes

Correct. 8 hours, from 6 PM to 2 AM on the deck's timeline.

✗ 3.5 hours

Wrong. 3.5 hours is the overlapped version.

✗ 2 hours

Wrong. 2 hours is one person's load.

✗ 4 hours

Wrong. That figure does not appear.

| 8 hours, from 6 PM to 2 AM on the deck's timeline.

12.20

EASY

Pipe S15

Sequential operation of the laundry takes _____ hours for 4 loads.

- a. 8
- b. 3.5
- c. 4
- d. 2

Answer: **8**

| 8 hours, which is the 480 minutes.

12.21

EASY

Pipe S17

Pipeline rate is limited by the _____ pipeline stage.

- a. slowest
- b. fastest
- c. first
- d. last

Answer: **slowest**

Slowest. It is the same reason the cycle time is computed as the maximum stage delay plus the latch delay.

12.22

EASY

Pipe S18

What are a nanosecond and a picosecond in seconds?

- a. 10 to the power minus 9 and 10 to the power minus 12
- b. 10 to the power minus 6 and 10 to the power minus 9
- c. 10 to the power minus 12 and 10 to the power minus 9
- d. 10 to the power minus 3 and 10 to the power minus 6

✓ 10 to the power minus 9 and 10 to the power minus 12

Correct. 1 ns is 10 to the minus 9 s and 1 ps is 10 to the minus 12 s, so a picosecond is a thousand times shorter.

✗ 10 to the power minus 6 and 10 to the power minus 9

Wrong. 10 to the minus 6 is a microsecond.

✗ 10 to the power minus 12 and 10 to the power minus 9

Wrong, and reversed.

✗ 10 to the power minus 3 and 10 to the power minus 6

Wrong. Those are milliseconds and microseconds.

1 ns is 10 to the minus 9 s and 1 ps is 10 to the minus 12 s, so a picosecond is a thousand times shorter.

12.23

EASY

Pipe S20

Under pipelining each load still takes five cycles to complete.

Answer: **true**

It does. The instruction must still pass through all five stages; it simply shares the pipeline with four others while doing so.

12.24

EASY

Pipe S32

Select the three types of pipeline hazard.

- a. Structural
- b. Data
- c. Control
- d. Timing

✓ Structural

Correct, a resource conflict.

✓ Data

Correct, a dependence on a previous result.

✓ Control

Correct, caused by branches.

✗ Timing

Wrong. There is no timing hazard in this course.

Structural, data, control. Hazards reduce the performance from the ideal speedup gained by pipelines.

12.25

EASY

Pipe S32

Hazards reduce the performance from the ____ speedup gained by pipelines.

- a. ideal
- b. measured
- c. average
- d. minimum

Answer: **ideal**

Ideal speedup, which is the stage count k .

A data hazard occurs when:

- a. An instruction depends on the results of the previous instruction
- b. Two instructions need the same hardware resource
- c. A branch changes the PC
- d. The cache misses

✓ An instruction depends on the results of the previous instruction

Correct. A dependence on a previous result.

✗ Two instructions need the same hardware resource

Wrong. That is a structural hazard.

✗ A branch changes the PC

Wrong. That is a control hazard.

✗ The cache misses

Wrong. A cache miss is a memory event, not a pipeline hazard in this taxonomy.

| A dependence on a previous result. The ADD then SUB example shows it plainly.

12.27

EASY

Pipe S33

A stall is:

- a. The delay in cycles caused by any of the hazards
- b. The time taken to fill the pipeline
- c. The number of stages in the pipeline
- d. A branch that is not taken

✓ The delay in cycles caused by any of the hazards

Correct. A delay in cycles, caused by a hazard.

✗ The time taken to fill the pipeline

Wrong. Fill time is a separate loss, though the deck notes it can be folded into the average stall figure.

✗ The number of stages in the pipeline

Wrong. That is k or m.

✗ A branch that is not taken

Wrong. A branch not taken causes no delay.

| A delay in cycles, caused by a hazard. Stalling the pipeline for a cycle leaves a bubble behind it.

12.28

EASY

Pipe S33

A stall is the delay in _____ caused by any of the hazards.

- a. cycles
- b. stages
- c. instructions
- d. seconds

Answer: **cycles**

| Cycles. Measuring it in cycles is what lets it appear in the speedup formula.

12.29

EASY

Pipe S34

Stalling the pipeline for one clock cycle results in a pipeline ____ .

- a. bubble
- b. gap
- c. break
- d. flush

Answer: **bubble**

A bubble, an empty slot travelling down the pipeline behind the stalled instruction.

12.30

EASY

Pipe S47

RAW stands for Read After Write. What else does the deck say about it?

- a. It is the most common data hazard and is solved by data forwarding
- b. It is the rarest data hazard
- c. It cannot occur in DLX
- d. It is solved by branch prediction

✓ It is the most common data hazard and is solved by data forwarding

Correct. Most common, solved by forwarding.

✗ It is the rarest data hazard

Wrong. It is explicitly the most common.

✗ It cannot occur in DLX

Wrong. WAW and WAR are the ones DLX avoids; RAW is real and must be handled.

✗ It is solved by branch prediction

Wrong. Branch prediction addresses control hazards.

Most common, solved by forwarding. WAR and WAW are the ones DLX avoids.

12.31

EASY

Pipe S47

RAW means Read After ____ .

- a. Write
- b. Read
- c. Wait
- d. Word

Answer: **Write**

Write. Read the name as the order of the two operations, with the second letter naming what happens first.

12.32

EASY

Pipe S54

Besides shared memory, what other resource constraint does the deck name?

- a. A limited resource such as an execution unit, which may be busy at the required time
- b. A full cache
- c. An empty register file
- d. A slow clock

✓ A limited resource such as an execution unit, which may be busy at the required time
Correct. A busy execution unit.

✗ A full cache
Wrong. A full cache is normal operation.

✗ An empty register file
Wrong. An empty register file is not a described condition.

✗ A slow clock
Wrong. Clock speed is not a resource constraint here.

A busy execution unit. It is the same idea as the ALU appearing among the contended resources in the structural hazard list.

12.33

EASY

Pipe S55

Pipelining increases processor throughput and makes better use of _____ resources.

- a. CPU
- b. memory
- c. cache
- d. bus

Answer: **CPU**

CPU resources. No functional unit sits idle once the pipeline is full.

12.34

EASY

Pipe S55

Despite the challenges of hazards and synchronization, modern computer systems employ sophisticated hardware and software techniques to reduce pipeline delays.

Answer: **true**

Both hardware and software: forwarding is hardware, instruction scheduling is software, and both appear in this deck.

12.35

EASY

Pipe S56

As processor technologies evolve, the deck says pipelining remains:

- a. A foundational concept in high-performance computing and advanced processor architecture
- b. A historical curiosity
- c. Limited to RISC machines
- d. Useful only for floating point

✓ A foundational concept in high-performance computing and advanced processor architecture

Correct. Foundational.

✗ A historical curiosity

Wrong. It is presented as current and central.

✗ Limited to RISC machines

Wrong. CISC machines pipeline too, only with more difficulty.

✗ Useful only for floating point

Wrong. No such restriction is made.

Foundational. The deck's closing claim, and it matches the earlier note that Intel and AMD processors use it.

12.36

EASY

RISC S27

The RISC and CISC deck calls it a five stage RISC _____-store architecture.

- a. load
- b. register
- c. memory
- d. stack

Answer: **load**

Load-store, the name for an architecture whose only memory instructions are load and store.

12.37

EASY

Rev S21

What does the revision pipelining example ask you to calculate?

- a. Time required for 50 instructions without pipelining
- b. Time required for 50 instructions with pipelining
- c. Speedup obtained
- d. Efficiency obtained

✓ Time required for 50 instructions without pipelining

Correct, part a.

✓ Time required for 50 instructions with pipelining

Correct, part b.

✓ Speedup obtained

Correct, part c.

✗ Efficiency obtained

Wrong. Efficiency is not among the three parts.

Three parts, a, b and c. Efficiency is defined in the Pipe deck but is not asked for here.

12.38

EASY

Rev S22

In the revision example the pipeline cycle time is _____ ns.

- a. 12
- b. 60
- c. 6
- d. 50

Answer: **12**

12 ns. Every part of the answer is built from that figure and the 6 stages.

What does revision Example 2 ask you to calculate?

- a. Pipeline cycle time
- b. Non-pipeline execution time
- c. Speed up ratio
- d. Pipeline time for 500 tasks
- e. Sequential time for 500 tasks
- f. Pipeline efficiency

✓ Pipeline cycle time

Correct, part 1.

✓ Non-pipeline execution time

Correct, part 2.

✓ Speed up ratio

Correct, part 3.

✓ Pipeline time for 500 tasks

Correct, part 4.

✓ Sequential time for 500 tasks

Correct, part 5.

✗ Pipeline efficiency

Wrong. Efficiency is not asked for.

Five parts. Efficiency is defined in the Pipe deck but is not one of the five asked for here.

The revision class repeats the three types of hazard as:

- a. Structural, data and control
- b. RAW, WAR and WAW
- c. Direct, associative and set associative
- d. Fill, drain and stall

✓ Structural, data and control

Correct. The same three as the Pipe deck.

✗ RAW, WAR and WAW

Wrong. Those classify data hazards, not hazards in general.

✗ Direct, associative and set associative

Wrong. Those are cache mapping functions.

✗ Fill, drain and stall

Wrong. Those are pipeline phases and a delay, not hazard types.

The same three as the Pipe deck. RAW, WAR and WAW are the sub-classification of data hazards only.

The revision class repeats that stalling the pipeline for one clock cycle when a conflict is detected results in:

- a. A pipeline bubble
- b. A cache miss
- c. A branch misprediction
- d. A forwarded result

✓ A pipeline bubble

Correct. A bubble, exactly as in the Pipe deck.

✗ A cache miss

Wrong. Cache misses belong to the memory topic.

✗ A branch misprediction

Wrong. Misprediction is not discussed in this course.

✗ A forwarded result

Wrong. Forwarding avoids the stall rather than resulting from it.

A bubble, exactly as in the Pipe deck. The revision slide also repeats that a stall is the delay in cycles caused by any hazard.

The revision class repeats an exercise asking students to describe RAW, WAR and WAW, and for three given snippets to:

- a. State which is which and indicate how each occurs using an arrow
- b. Compute the speedup of each
- c. Rewrite them without hazards
- d. Convert them to reverse Polish notation

✓ State which is which and indicate how each occurs using an arrow

Correct. The same exercise as the Pipe deck, repeated in the revision class.

✗ Compute the speedup of each

Wrong. No speedup is asked for.

✗ Rewrite them without hazards

Wrong. Rewriting is Example 2's task, elsewhere.

✗ Convert them to reverse Polish notation

Wrong. Reverse Polish notation belongs to instruction formats.

The same exercise as the Pipe deck, repeated in the revision class. Repetition across two decks is a fair signal about what will be asked.

Select the stages of a typical instruction pipeline.

- a. Instruction Fetch
- b. Instruction Decode
- c. Execute
- d. Memory Access
- e. Write Back
- f. Branch Predict

✓ Instruction Fetch

Correct, IF. Five stages: IF, ID, EX, MEM, WB.

✓ Instruction Decode

Correct, ID. Five stages: IF, ID, EX, MEM, WB.

✓ Execute

Correct, EX. Five stages: IF, ID, EX, MEM, WB.

✓ Memory Access

Correct, MEM. Five stages: IF, ID, EX, MEM, WB.

✓ Write Back

Correct, WB. Five stages: IF, ID, EX, MEM, WB.

✗ Branch Predict

Wrong. Branch prediction is listed among the techniques architects design to reduce hazards.

Five stages: IF, ID, EX, MEM, WB. Branch prediction is a technique for handling control hazards, not a stage.

12.44

MEDIUM

Pipe S4

Match each pipeline stage to what it does.

Instruction Fetch	Retrieving the instruction from memory
Instruction Decode	Interpreting the instruction
Execute	Performing the decoded operation
Memory Access	Accessing memory if required
Write Back	Storing the result back into a register

Five stages, five jobs. Note the if required against Memory Access: not every instruction touches memory, but every one passes through the stage.

12.45

MEDIUM

Pipe S5

By allowing different instructions to occupy different stages simultaneously, pipelining increases:

- a. The throughput of the processor, so more instructions complete in less time
- b. The latency of each instruction
- c. The clock frequency
- d. The size of the instruction set

✓ The throughput of the processor, so more instructions complete in less time
Correct. Throughput.

✗ The latency of each instruction
Wrong. Latency per instruction is unchanged, or slightly worse because of the pipeline registers.

✗ The clock frequency
Wrong. The clock is a separate matter.

✗ The size of the instruction set
Wrong. The instruction set is untouched.

Throughput. It does not shorten any single instruction, which the deck states later as pipelining not helping the latency of a single task.

Select the challenges pipelining introduces.

- a. Pipeline hazards
- b. Data dependencies
- c. Branch instructions
- d. Resource conflicts
- e. Cache thrashing

✓ Pipeline hazards

Correct, and it is the umbrella term.

✓ Data dependencies

Correct, and they cause data hazards.

✓ Branch instructions

Correct, and they cause control hazards.

✓ Resource conflicts

Correct, and they cause structural hazards.

✗ Cache thrashing

Wrong. Thrashing is a direct mapped cache problem, from a different topic.

Four challenges, and each maps onto a hazard type: data dependencies to data hazards, branches to control hazards, resource conflicts to structural hazards.

Select the techniques computer architects design to reduce pipelining problems.

- a. Forwarding
- b. Stalling
- c. Branch prediction
- d. Caching

✓ Forwarding

Correct, forwarding solves most data hazards.

✓ Stalling

Correct, stalling handles structural hazards and the data hazards forwarding cannot fix.

✓ Branch prediction

Correct, branch prediction addresses control hazards.

✗ Caching

Wrong. Caching belongs to the memory hierarchy topic.

Three techniques, one for each hazard family. Caching is a memory technique, not a pipeline remedy.

Select what pipelining does, according to the deck's summary of why it is widely used.

- a. Improves CPU utilization
- b. Enhances instruction throughput
- c. Contributes to faster system performance
- d. Reduces the number of instructions in a program

✓ Improves CPU utilization

Correct, CPU utilization improves because no unit sits idle.

✓ Enhances instruction throughput

Correct, instruction throughput rises.

✓ Contributes to faster system performance

Correct, and it closes the list.

✗ Reduces the number of instructions in a program

Wrong. The program is unchanged; only the rate at which it retires changes.

Three benefits. Reducing the instruction count is the CISC objective, and pipelining does not change how many instructions a program has.

Select the levels of parallelism through which speedup is achieved in a typical system.

- a. Multi-user
- b. Multitasking
- c. Multi-processing
- d. Multi-programming
- e. Multi-threading
- f. Compiler optimizations
- g. Multi-mapping

✓ Multi-user

Correct. Six items, and the last one is not a hardware level at all: compiler optimizations are on the list too.

✓ Multitasking

Correct. Six items, and the last one is not a hardware level at all: compiler optimizations are on the list too.

✓ Multi-processing

Correct. Six items, and the last one is not a hardware level at all: compiler optimizations are on the list too.

✓ Multi-programming

Correct, and it is easily confused with multi-processing.

✓ Multi-threading

Correct. Six items, and the last one is not a hardware level at all: compiler optimizations are on the list too.

✓ Compiler optimizations

Correct, and it is the only software item on the list.

✗ Multi-mapping

Wrong. No such term appears in this course.

Six items, and the last one is not a hardware level at all: compiler optimizations are on the list too.

12.50

MEDIUM

Pipe S10

The list of levels of parallelism ends with _____ optimizations.

- a. compiler
- b. hardware
- c. memory
- d. cache

Answer: **compiler**

Compiler optimizations, the only software entry among six.

12.51

MEDIUM

Pipe S10

Select the different types of pipeline the deck names.

- a. Instruction pipeline
- b. Operation pipeline
- c. Multi-issue pipelines
- d. Memory pipeline

✓ Instruction pipeline

Correct. Three types.

✓ Operation pipeline

Correct, and it matches the definition mentioning arithmetic sub-operations.

✓ Multi-issue pipelines

Correct. Three types.

✗ Memory pipeline

Wrong. No memory pipeline is named in this course.

Three types. The instruction pipeline is the one the rest of the deck works with.

12.52

MEDIUM

Pipe S11

Pipeline processing is an implementation technique where what overlaps in execution?

- a. Arithmetic sub-operations, or the phases of a computer instruction cycle
- b. Whole programs
- c. Memory accesses only
- d. Processor cores

✓ Arithmetic sub-operations, or the phases of a computer instruction cycle

Correct. Either arithmetic sub-operations or instruction cycle phases, which is why both an operation pipeline and an instruction pipeline exist.

✗ Whole programs

Wrong. Overlapping whole programs is multi-programming.

✗ Memory accesses only

Wrong. Memory access is only one of the five stages.

✗ Processor cores

Wrong. Overlapping cores is multi-processing.

Either arithmetic sub-operations or instruction cycle phases, which is why both an operation pipeline and an instruction pipeline exist.

12.53

MEDIUM

Pipe S11

Pipeline processing is an _____ technique in which sub-operations or instruction cycle phases overlap in execution.

- a. implementation
- b. architectural
- c. compilation
- d. encoding

Answer: **implementation**

Implementation technique, which places pipelining on the organization side of the architecture and organization divide.

12.54

MEDIUM

Pipe S11

Pipelining exploits parallelism among instructions by overlapping them. What is this called?

- a. Instruction Level Parallelism, or ILP
- b. Multi-threading
- c. Multi-processing
- d. Data Level Parallelism

✓ Instruction Level Parallelism, or ILP

Correct. Instruction Level Parallelism, abbreviated ILP.

✗ Multi-threading

Wrong. Multi-threading is one of the levels listed on the earlier slide.

✗ Multi-processing

Wrong. Multi-processing uses several processors.

✗ Data Level Parallelism

Wrong. That term does not appear in this course.

| Instruction Level Parallelism, abbreviated ILP. It is parallelism inside a single instruction stream.

12.55

MEDIUM

Pipe S11

Overlapping instructions to exploit parallelism among them is called Instruction Level Parallelism, abbreviated ____ .

- a. ILP
- b. ISA
- c. CPI
- d. TLP

Answer: **ILP**

| ILP. Do not confuse it with ISA, the instruction set architecture, or CPI, cycles per instruction.

12.56

MEDIUM

Pipe S11

Each sub-process is executed in a special dedicated segment that operates _____ with all other segments.

- a. concurrently
- b. sequentially
- c. randomly
- d. intermittently

Answer: **concurrently**

Concurrently. Every segment is busy at once, on a different instruction.

12.57

MEDIUM

Pipe S12

Select what the deck says about the structure of a pipeline.

- a. It has many steps, stages or segments
- b. Each stage carries out a different part of an instruction or operation
- c. The stages are connected to form a pipe
- d. Each stage carries out the whole instruction

✓ It has many steps, stages or segments

Correct, and the three words are used interchangeably.

✓ Each stage carries out a different part of an instruction or operation

Correct, a different part each.

✓ The stages are connected to form a pipe

Correct, connected to form a pipe.

✗ Each stage carries out the whole instruction

Wrong. If each stage did the whole instruction there would be no division of labour.

Three claims. The fourth would describe several independent processors rather than a pipeline.

12.58

MEDIUM

Pipe S13

Throughput is defined as:

- a. The number of items that exit the pipeline per unit time
- b. The time a single item spends in the pipeline
- c. The number of stages in the pipeline
- d. The delay caused by a hazard

✓ The number of items that exit the pipeline per unit time

Correct. Items out per unit time, with examples of 1 instruction per clock cycle, 10 cars per hour and 10 floating point operations per cycle.

✗ The time a single item spends in the pipeline

Wrong. That is latency.

✗ The number of stages in the pipeline

Wrong. That is the stage count, usually written k or m.

✗ The delay caused by a hazard

Wrong. That is a stall.

Items out per unit time, with examples of 1 instruction per clock cycle, 10 cars per hour and 10 floating point operations per cycle.

12.59

MEDIUM

Pipe S13

The deck's examples of throughput include 1 instruction per clock cycle and ____ cars per hour.

- a. 10
- b. 5
- c. 20
- d. 100

Answer: **10**

10 cars per hour, and 10 floating point operations per cycle is the third example.

The pipeline designer's goal for stage time is to:

- a. Balance the length of each pipeline stage, giving a balanced pipeline
- b. Make the first stage as short as possible
- c. Make the last stage as long as possible
- d. Vary the stage lengths to suit each instruction

✓ Balance the length of each pipeline stage, giving a balanced pipeline

Correct. Balance them.

✗ Make the first stage as short as possible

Wrong. A single short stage does not help if another is long.

✗ Make the last stage as long as possible

Wrong, and a long stage is exactly what limits the pipeline.

✗ Vary the stage lengths to suit each instruction

Wrong. Varying stage lengths is what makes CISC hard to pipeline.

Balance them. Unbalanced stages reduce speedup, because the pipeline rate is set by the slowest one.

12.61

MEDIUM

Pipe S13

What does the deck say about CPI in a pipeline?

- a. A pipeline yields a reduction in cycles per instruction, and CPI is approximately equal to the stage time
- b. A pipeline increases cycles per instruction
- c. CPI is always exactly 1
- d. CPI equals the number of stages

✓ A pipeline yields a reduction in cycles per instruction, and CPI is approximately equal to the stage time

Correct. A reduction in CPI, with CPI approximately the stage time.

✗ A pipeline increases cycles per instruction

Wrong. Reducing CPI is the point of pipelining.

✗ CPI is always exactly 1

Wrong. Exactly 1 is the ideal; hazards and stalls push it above.

✗ CPI equals the number of stages

Wrong. The stage count sets the latency, not the CPI.

A reduction in CPI, with CPI approximately the stage time. In an ideal pipeline one instruction completes per stage time.

12.62

MEDIUM

Pipe S13

A pipeline yields a reduction in cycles per instruction.

Answer: **true**

It does, and CPI is approximately equal to the stage time.

In the laundry analogy, what are the four stages and how long does each take?

- a. Wash, dry, stash and fold, each taking 30 minutes
- b. Wash, dry and fold, each taking 30 minutes
- c. Wash, dry, stash and fold, each taking 20 minutes
- d. Wash and dry, each taking 60 minutes

✓ Wash, dry, stash and fold, each taking 30 minutes

Correct. Four stages of 30 minutes each: washer, dryer, folder and stasher.

✗ Wash, dry and fold, each taking 30 minutes

Wrong. There are four stages, not three.

✗ Wash, dry, stash and fold, each taking 20 minutes

Wrong. Each takes 30 minutes.

✗ Wash and dry, each taking 60 minutes

Wrong on both count and duration.

Four stages of 30 minutes each: washer, dryer, folder and stasher. Four people each have one load.

In the laundry analogy, how long does one person's load take, and how long do four loads take without overlap?

- a. 120 minutes each, and 480 minutes in total
- b. 30 minutes each, and 120 minutes in total
- c. 120 minutes each, and 210 minutes in total
- d. 60 minutes each, and 240 minutes in total

✓ 120 minutes each, and 480 minutes in total

Correct. 30 minutes times 4 stages is 120 minutes per load, and 4 loads gives 480 minutes.

✗ 30 minutes each, and 120 minutes in total

Wrong. 30 minutes is one stage, not a whole load.

✗ 120 minutes each, and 210 minutes in total

Wrong. 210 minutes is the 3.5 hours the overlapped version takes.

✗ 60 minutes each, and 240 minutes in total

Wrong. Each stage is 30 minutes and there are four of them.

30 minutes times 4 stages is 120 minutes per load, and 4 loads gives 480 minutes.

12.65

MEDIUM

Pipe S14

Each load takes 30 minutes times 4, which is _____ minutes, and four loads take _____ minutes.

BLANK 1

- a. 120
- b. 480
- c. 210
- d. 90

BLANK 2

- a. 480
- b. 120
- c. 210
- d. 360

Answer: **120, 480**

120 and 480. 480 minutes is 8 hours.

12.66

MEDIUM

Pipe S16

Overlapping brings the laundry down to _____ hours for 4 loads, giving a speedup of 8 over that, which is _____ .

BLANK 1

- a. 3.5
- b. 4
- c. 2
- d. 8

BLANK 2

- a. 2.1
- b. 2.8
- c. 4
- d. 3.88

Answer: **3.5, 2.1**

3.5 hours, speedup 2.1. With only four loads the pipeline never reaches its ideal speedup of 4.

12.67

MEDIUM

Pipe S17

Pipelining shortens the time taken by a single instruction from start to finish.

Answer: **false**

It does not. Pipelining does not help the latency of a single task; it helps the throughput of the entire workload.

12.68

MEDIUM

Pipe S17

The pipeline rate is limited by:

- a. The slowest pipeline stage
- b. The fastest pipeline stage
- c. The average of the stage times
- d. The number of tasks

✓ The slowest pipeline stage

Correct. The slowest stage sets the clock for all of them, which is why balancing stages matters and why stage time is often taken as the maximum.

✗ The fastest pipeline stage

Wrong. A fast stage simply waits.

✗ The average of the stage times

Wrong. Every stage has to advance together, so the average is not enough.

✗ The number of tasks

Wrong. The task count affects total time, not the rate.

The slowest stage sets the clock for all of them, which is why balancing stages matters and why stage time is often taken as the maximum.

12.69

MEDIUM

Pipe S17

As the number of tasks increases, the potential speedup of a pipeline equals:

- a. The number of pipe stages
- b. The number of tasks
- c. The stage time
- d. The number of stages times the number of tasks

✓ The number of pipe stages

Correct. The stage count, k .

✗ The number of tasks

Wrong. More tasks bring you closer to the limit but do not raise it.

✗ The stage time

Wrong. Stage time is a duration, not a ratio.

✗ The number of stages times the number of tasks

Wrong. Speedup is bounded by the stage count alone.

The stage count, k . It matches the formula: speedup is $k \times n$ over $k + n - 1$, which tends to k as n grows.

12.70

MEDIUM

Pipe S17

Potential speedup equals the number of pipe _____ as the number of tasks increases.

- a. stages
- b. tasks
- c. registers
- d. cycles

Answer: **stages**

Stages. A 5-stage pipeline can at best go 5 times faster.

12.71

MEDIUM

Pipe S17

Select what the deck says reduces speedup.

- a. Unbalanced lengths of pipe stages
- b. Time to fill the pipeline
- c. Time to drain the pipeline
- d. Increasing the number of tasks

✓ Unbalanced lengths of pipe stages

Correct, unbalanced stages waste time in the fast ones.

✓ Time to fill the pipeline

Correct, filling costs cycles before the first result appears.

✓ Time to drain the pipeline

Correct, draining costs cycles after the last one enters.

✗ Increasing the number of tasks

Wrong. More tasks amortise the fill and drain, improving the speedup.

Three losses. Increasing the number of tasks does the opposite: it brings the achieved speedup closer to the ideal.

12.72

MEDIUM

Pipe S17

Time to _____ the pipeline and time to _____ it both reduce speedup.

BLANK 1

- a. fill
- b. drain
- c. clear
- d. reset

BLANK 2

- a. drain
- b. fill
- c. flush
- d. stall

Answer: **fill, drain**

Fill and drain. In the formula they are the k minus 1 extra cycles beyond the n tasks.

The key idea is to break a big computation into pieces. What separates each piece?

- a. A pipeline register
- b. A cache line
- c. A control signal
- d. A stall

✓ A pipeline register

Correct. A pipeline register, which holds the partial result between stages.

✗ A cache line

Wrong. Cache lines belong to the memory topic.

✗ A control signal

Wrong. Control signals come from the control unit.

✗ A stall

Wrong. A stall is a delay, not a component.

A pipeline register, which holds the partial result between stages. It is the extra hardware pipelining costs.

12.74

MEDIUM

Pipe S18

In the deck's illustration a 1 ns computation is broken into five stages of _____ ps each, separated by pipeline _____ .

BLANK 1

- a. 200
- b. 100
- c. 500
- d. 250

BLANK 2

- a. registers
- b. buffers
- c. latches
- d. gates

Answer: **200, registers**

200 ps each, and pipeline registers between them. Five times 200 ps is the original 1 ns.

12.75

MEDIUM

Pipe S18

1 picosecond is 10 to the power minus _____ second.

- a. 12
- b. 9
- c. 6
- d. 15

Answer: **12**

Minus 12. The memory hierarchy example uses microseconds, so the course spans several scales.

12.76

MEDIUM

Pipe S19

In the exam-grading analogy, how many students, problems and graders are there?

- a. 100 students, 5 problems, five people grading
- b. 5 students, 100 problems, five people grading
- c. 100 students, 4 problems, four people grading
- d. 50 students, 5 problems, five people grading

✓ 100 students, 5 problems, five people grading

Correct. A class of 100, 5 problems, 5 graders, one problem each, passing the exam on as soon as they finish.

✗ 5 students, 100 problems, five people grading

Wrong, and it swaps the two numbers.

✗ 100 students, 4 problems, four people grading

Wrong. Five problems and five graders.

✗ 50 students, 5 problems, five people grading

Wrong. The class is 100 students.

A class of 100, 5 problems, 5 graders, one problem each, passing the exam on as soon as they finish.

12.77

MEDIUM

Pipe S19

In the grading analogy each person grades _____ problem, and passes the exam to the next person as soon as they finish.

- a. one
- b. two
- c. five
- d. all

Answer: **one**

Only one problem each, which is what makes each grader a dedicated stage.

12.78

MEDIUM

Pipe S19

Each problem takes _____ minutes to grade, so each individual exam still takes _____ hour.

BLANK 1

- a. 12
- b. 5
- c. 60
- d. 10

BLANK 2

- a. 1
- b. 5
- c. 2
- d. 12

Answer: **12, 1**

12 minutes each, 1 hour per exam. Five problems at twelve minutes.

12.79

MEDIUM

Pipe S20

In the pipelined load example, when can the 2nd load start?

- a. As soon as the 1st finishes its fetch stage
- b. As soon as the 1st finishes completely
- c. At the same instant as the 1st
- d. After the 1st reaches write back

✓ As soon as the 1st finishes its fetch stage

Correct. As soon as the fetch stage is free.

✗ As soon as the 1st finishes completely

Wrong. Waiting for full completion is the non-pipelined case.

✗ At the same instant as the 1st

Wrong. Both would need the fetch unit at once, which is a structural hazard.

✗ After the 1st reaches write back

Wrong. Write back is the last stage; waiting for it wastes almost all the benefit.

As soon as the fetch stage is free. That is the whole mechanism: a stage is reusable the moment its current occupant moves on.

12.80

MEDIUM

Pipe S20

The load instruction has 5 stages with five independent functional units, and each functional unit is used only ____ .

- a. once
- b. twice
- c. continuously
- d. rarely

Answer: **once**

Only once per instruction, which is what leaves it free for the next one.

Under pipelining, what happens to the time each load takes and to the throughput?

- a. Each load still takes five cycles, but the throughput is much higher
- b. Each load takes one cycle, and the throughput is unchanged
- c. Each load takes five cycles, and the throughput is unchanged
- d. Each load takes one cycle, and the throughput falls

✓ Each load still takes five cycles, but the throughput is much higher

Correct. Five cycles each, much higher throughput.

✗ Each load takes one cycle, and the throughput is unchanged

Wrong. A five-stage instruction cannot finish in one cycle.

✗ Each load takes five cycles, and the throughput is unchanged

Wrong. Throughput is exactly what improves.

✗ Each load takes one cycle, and the throughput falls

Wrong on both counts.

Five cycles each, much higher throughput. It is latency against throughput once again, now in cycles.

12.82

MEDIUM

Pipe S21

With n tasks, k stages and T time per stage, the total time for the non-pipelined solution is:

- a. T times k times n
- b. T times k plus n
- c. T times k plus T times n minus 1
- d. T times n

✓ T times k times n

Correct. Time per task is T times k , and there are n of them with no overlap at all.

✗ T times k plus n

Wrong. The task count multiplies rather than adds.

✗ T times k plus T times n minus 1

Wrong, and that is the PIPELINED total.

✗ T times n

Wrong. That omits the k stages within each task.

| Time per task is T times k , and there are n of them with no overlap at all.

12.83

MEDIUM

Pipe S21

Time per task equals T times ____ .

- a. k
- b. n
- c. m
- d. c

Answer: **k**

| T times k , the time per stage times the number of stages.

12.84

MEDIUM

Pipe S21

Speedup is the total non-pipelined time divided by the total _____ time.

- a. pipelined
- b. stage
- c. stall
- d. fill

Answer: **pipelined**

Divided by the pipelined time. Every speedup calculation in this course is that same ratio.

12.85

MEDIUM

Pipe S22

In the comparison of three lw instructions, what are the two timings?

- a. 800 ps per instruction non-pipelined, and 200 ps per step pipelined
- b. 200 ps per instruction non-pipelined, and 800 ps per step pipelined
- c. 1000 ps and 200 ps
- d. 800 ps and 400 ps

✓ 800 ps per instruction non-pipelined, and 200 ps per step pipelined

Correct. 800 ps per whole instruction against 200 ps per pipeline step.

✗ 200 ps per instruction non-pipelined, and 800 ps per step pipelined

Wrong, and reversed.

✗ 1000 ps and 200 ps

Wrong. The non-pipelined figure is 800 ps.

✗ 800 ps and 400 ps

Wrong. The pipelined step is 200 ps.

800 ps per whole instruction against 200 ps per pipeline step. The pipelined machine starts a new instruction every 200 ps.

12.86

MEDIUM

Pipe S22

In the single-cycle against pipelined comparison the non-pipelined machine takes _____ ps per instruction.

- a. 800
- b. 200
- c. 1000
- d. 400

Answer: **800**

800 ps, which is four times the 200 ps pipeline step.

12.87

MEDIUM

Pipe S23

For a conventional non-pipelined machine, the time to complete n tasks is:

- a. t_1 equals n times t_n , where t_n is the clock cycle
- b. t_1 equals n plus t_n
- c. t_1 equals t_n divided by n
- d. t_1 equals k plus n minus 1, times t_p

✓ t_1 equals n times t_n , where t_n is the clock cycle

Correct. N times t_n .

✗ t_1 equals n plus t_n

Wrong. The two quantities multiply.

✗ t_1 equals t_n divided by n

Wrong, and it inverts the relationship.

✗ t_1 equals k plus n minus 1, times t_p

Wrong, and that is the pipelined formula.

n times t_n . The pipelined counterpart, t_k , has the k plus n minus 1 shape instead.

12.88

MEDIUM

Pipe S23

For the conventional machine, _____ is the number of tasks to be performed and t_n is the clock cycle.

- a. n
- b. k
- c. m
- d. c

Answer: **n**

n is the task count throughout this topic; k is the number of stages.

12.89

MEDIUM

Pipe S24

For the pipelined machine, _____ is the clock cycle, that is the time to complete each sub operation.

- a. t_p
- b. t_n
- c. t_k
- d. t_1

Answer: **t_p**

t_p for the pipelined clock cycle, t_n for the non-pipelined one. Keeping the two apart is what makes the speedup formula readable.

12.90

MEDIUM

Pipe S24

In the speedup formula the numerator is n times _____, the total non-pipelined time.

- a. t_n
- b. t_p
- c. t_k
- d. k

Answer: **t_n**

n times t_n . The denominator uses t_p , the pipelined cycle.

12.91

MEDIUM

Pipe S25

In the 4-stage worked example with t_p of 20 ns, how long does one task take in a non-pipelined system?

- a. 80 ns, from 20 times 4
- b. 20 ns
- c. 2060 ns
- d. 8000 ns

✓ 80 ns, from 20 times 4

Correct. 20 ns per sub-operation times 4 stages is 80 ns for one whole task.

✗ 20 ns

Wrong. 20 ns is one stage.

✗ 2060 ns

Wrong. 2060 ns is the pipelined total for 100 tasks.

✗ 8000 ns

Wrong. 8000 ns is the non-pipelined total for 100 tasks.

20 ns per sub-operation times 4 stages is 80 ns for one whole task.

12.92

MEDIUM

Pipe S25

The example uses a _____ -stage pipeline with tp of 20 ns and _____ tasks to be executed.

BLANK 1

- a. 4
- b. 5
- c. 6
- d. 3

BLANK 2

- a. 100
- b. 50
- c. 500
- d. 10

Answer: **4, 100**

4 stages, 100 tasks. With n far larger than k, the speedup should come close to the ideal 4.

12.93

MEDIUM

Pipe S25

The deck notes that a 4-stage pipeline is basically identical to a system with 4 identical function units.

Answer: **true**

It does, as a way of seeing where the fourfold ideal speedup comes from: four things are in progress at once either way.

12.94

MEDIUM

Pipe S28

In Design 2 the 1 ns added to the clock cycle is for setup and clock _____ .

- a. skew
- b. drift
- c. jitter
- d. delay

Answer: **skew**

Clock skew. In Example 3 the same kind of cost appears as an overhead of 1 ns per stage.

12.95

MEDIUM

Pipe S29

In Example 3 the stage time is MAX of the five unit times, which is _____, plus the 1 ns overhead.

- a. 10
- b. 8
- c. 7
- d. 45

Answer: **10**

10 ns, the maximum. Three of the five units happen to take 10 ns.

12.96

MEDIUM

Pipe S30

The efficiency of a pipeline is measured as the ratio of:

- a. The busy time span to the total time span including the idle time
- b. The idle time span to the busy time span
- c. The number of stages to the number of tasks
- d. The throughput to the clock period

✓ The busy time span to the total time span including the idle time

Correct. Busy over total.

✗ The idle time span to the busy time span

Wrong, and inverted.

✗ The number of stages to the number of tasks

Wrong. That ratio has no name here.

✗ The throughput to the clock period

Wrong. Throughput is efficiency divided by the clock period, which is the other way round.

Busy over total. Efficiency asks what fraction of the pipeline's capacity is actually being used.

12.97

MEDIUM

Pipe S30

Pipeline efficiency is the ratio of the _____ time span to the total time span including the idle time.

- a. busy
- b. idle
- c. stage
- d. total

Answer: **busy**

Busy. The idle time comes from the fill and drain phases.

12.98

MEDIUM

Pipe S31

Throughput is the number of _____ that have been achieved per unit time.

- a. results
- b. stages
- c. stalls
- d. instructions issued

Answer: **results**

Results, that is completed items leaving the pipeline.

12.99

MEDIUM

Pipe S31

According to the deck, throughput denotes:

- a. The computing power of the pipeline
- b. The latency of the pipeline
- c. The cost of the pipeline
- d. The number of hazards

✓ The computing power of the pipeline

Correct. The computing power.

✗ The latency of the pipeline

Wrong. Latency is the time one item takes, and pipelining does not improve it.

✗ The cost of the pipeline

Wrong. Cost is not measured by throughput.

✗ The number of hazards

Wrong. Hazards reduce throughput but are not what it denotes.

The computing power. The deck adds that maximum speedup, efficiency and throughput are all ideal cases.

12.100

MEDIUM

Pipe S31

Maximum speedup, efficiency and throughput are ideal cases rather than what a real pipeline achieves.

Answer: **true**

The deck says so directly, and the very next slide explains why: hazards reduce performance from the ideal speedup.

12.101

MEDIUM

Pipe S32

A structural hazard is:

- a. A resource conflict, where hardware cannot support all possible combinations of instructions in simultaneous overlapped execution
- b. A dependence on the result of a previous instruction
- c. A consequence of branches affecting the PC
- d. A mismatch between stage lengths

✓ A resource conflict, where hardware cannot support all possible combinations of instructions in simultaneous overlapped execution

Correct. A resource conflict.

✗ A dependence on the result of a previous instruction

Wrong. That is a data hazard.

✗ A consequence of branches affecting the PC

Wrong. That is a control hazard.

✗ A mismatch between stage lengths

Wrong. Unbalanced stages reduce speedup but are not called a hazard.

| A resource conflict. Two instructions want the same piece of hardware in the same cycle.

12.102

MEDIUM

Pipe S32

A structural hazard is a _____ conflict.

- a. resource
- b. data
- c. control
- d. timing

Answer: **resource**

| A resource conflict. Examples of the resources involved are the register file, memory and the ALU.

12.103

MEDIUM

Pipe S32

Match each hazard to its cause.

Structural hazard	Resource conflict
Data hazard	An instruction depends on the results of the previous instruction
Control hazard	Branches and other instructions that affect the PC

Three hazards, three causes. Every remedy in the deck attaches to one of these three rows.

12.104

MEDIUM

Pipe S32

A control hazard is due to:

- a. Branches and other instructions that affect the PC
- b. Two instructions needing the ALU at once
- c. An instruction waiting for a previous result
- d. An unbalanced pipeline

✓ Branches and other instructions that affect the PC

Correct. Anything that changes the PC, because the pipeline has already begun fetching from the wrong place.

✗ Two instructions needing the ALU at once

Wrong. That is a structural hazard.

✗ An instruction waiting for a previous result

Wrong. That is a data hazard.

✗ An unbalanced pipeline

Wrong. Imbalance reduces speedup but is not a hazard.

Anything that changes the PC, because the pipeline has already begun fetching from the wrong place.

12.105

MEDIUM

Pipe S32

A control hazard arises from branches and other instructions that affect the ____ .

- a. PC
- b. ALU
- c. MAR
- d. IR

Answer: **PC**

The PC. Branch prediction is the remedy the deck lists for this family.

12.106

MEDIUM

Pipe S33

The number of cycles needed to initially fill up the pipeline could be included in the computation of the average stall per instruction.

Answer: **true**

The deck allows it, which is a neat way of folding the fill cost into one figure rather than tracking it separately.

12.107

MEDIUM

Pipe S34

Select the examples of resources that can cause a structural hazard.

- a. Register file
- b. Memory
- c. ALU
- d. Program counter

✓ Register file

Correct, the register file is named.

✓ Memory

Correct, memory is named, and the deck's illustration is a memory access conflict.

✓ ALU

Correct, the ALU is named.

✗ Program counter

Wrong. Instructions affecting the PC give control hazards.

Three named resources. The program counter causes control hazards rather than resource conflicts.

12.108

MEDIUM

Pipe S34

When more than one instruction in the pipeline needs to access a resource, the _____ is said to have a structural hazard.

- a. datapath
- b. compiler
- c. register
- d. cache

Answer: **datapath**

The datapath. It is a property of the hardware, not of the program.

12.109

MEDIUM

Pipe S34

What is the solution to a structural hazard, and what does it produce?

- a. Stall the pipeline for one clock cycle when the conflict is detected, which results in a pipeline bubble
- b. Forward the result from the EX/MEM buffer
- c. Predict the branch
- d. Reorder the instructions at compile time

✓ Stall the pipeline for one clock cycle when the conflict is detected, which results in a pipeline bubble

Correct. A one-cycle stall, leaving a bubble.

✗ Forward the result from the EX/MEM buffer

Wrong. Forwarding is the data hazard remedy.

✗ Predict the branch

Wrong. Prediction is the control hazard remedy.

✗ Reorder the instructions at compile time

Wrong. Reordering helps with load-use data hazards, as in Example 2.

A one-cycle stall, leaving a bubble. Forwarding fixes data hazards and prediction fixes control hazards, so each family has its own remedy.

12.110

MEDIUM

Pipe S34

The underlying problem in the deck's memory access conflict illustration is that there is only one memory port.

Answer: **true**

One port, so an instruction fetch and a data access cannot both happen in the same cycle. A Harvard arrangement would remove exactly this conflict.

12.111

MEDIUM

Pipe S38

In the data hazard example beginning ADD R1,R2,R3, what makes the following instructions dependent?

- a. All of them use R1 after the first instruction writes it
- b. All of them write R1
- c. They all use the same ALU
- d. They all branch on the same condition

✓ All of them use R1 after the first instruction writes it

Correct. R1 is the result of the ADD, and SUB, AND, OR and XOR all read it.

✗ All of them write R1

Wrong. Only the ADD writes R1 in this sequence.

✗ They all use the same ALU

Wrong. Sharing the ALU would be a structural hazard.

✗ They all branch on the same condition

Wrong. No branches appear in the sequence.

R1 is the result of the ADD, and SUB, AND, OR and XOR all read it. Four dependent instructions in a row.

12.112

MEDIUM

Pipe S38

In the data hazard example the last of the five instructions is ____ R10,R1,R11.

- a. XOR
- b. OR
- c. AND
- d. SUB

Answer: **XOR**

XOR. The sequence runs ADD, SUB, AND, OR, XOR, and every one after the ADD reads R1.

12.113

MEDIUM

Pipe S39

Data hazards are usually solved by data or register forwarding, which is also called:

- a. Bypassing or short-circuiting
- b. Stalling or bubbling
- c. Prediction or speculation
- d. Reordering or scheduling

✓ Bypassing or short-circuiting

Correct. Bypassing or short-circuiting.

✗ Stalling or bubbling

Wrong. Stalling is the fallback when forwarding cannot work.

✗ Prediction or speculation

Wrong. Prediction addresses control hazards.

✗ Reordering or scheduling

Wrong. Scheduling is what Example 2 does by hand.

Bypassing or short-circuiting. All three names describe the same trick: send the result straight to where it is needed rather than through the register file.

12.114

MEDIUM

Pipe S39

Data or register forwarding is also called bypassing or ____ .

- a. short-circuiting
- b. stalling
- c. buffering
- d. scheduling

Answer: **short-circuiting**

Short-circuiting, because the value takes a short cut back to the ALU.

12.115

MEDIUM

Pipe S39

If the forwarding hardware detects that its source operand has a new value, what does the logic do?

- a. Selects the newer result rather than the value read from the register file
- b. Selects the value read from the register file
- c. Stalls until the register file is updated
- d. Discards the instruction

✓ Selects the newer result rather than the value read from the register file

Correct. The newer value wins.

✗ Selects the value read from the register file

Wrong. That stale value is what causes the wrong answer.

✗ Stalls until the register file is updated

Wrong. Stalling is what forwarding avoids.

✗ Discards the instruction

Wrong. Nothing is discarded.

The newer value wins. The register file copy is stale at that moment, which is exactly the hazard.

12.116

MEDIUM

Pipe S39

When forwarding is active, the value taken from the register file is preferred over the newer forwarded result.

Answer: **false**

The newer result is selected. Preferring the register file would reproduce the very stale value the hazard is about.

12.117

MEDIUM

Pipe S40

Results are forwarded from any instruction that started up to ____ cycles before.

- a. 3
- b. 2
- c. 1
- d. 5

Answer: **3**

Three cycles.

12.118

MEDIUM

Pipe S40

The results from EX/MEM and MEM/WB are forwarded to:

- a. Both ALU inputs
- b. One ALU input only
- c. The register file only
- d. The memory port

✓ Both ALU inputs

Correct. Both inputs, because either operand of the next instruction might be the stale one.

✗ One ALU input only

Wrong. Restricting it to one input would leave half the hazards unfixed.

✗ The register file only

Wrong. Going through the register file is what forwarding bypasses.

✗ The memory port

Wrong. The memory port is not involved.

Both inputs, because either operand of the next instruction might be the stale one.

12.119

MEDIUM

Pipe S41

In the example add \$s0, \$t0, \$t1 followed by sub \$t2, \$s0, \$t3, with \$s0 equal to minus 5 before the add and 8 after it, which value should the sub use?

- a. 8
- b. minus 5
- c. 3
- d. Either, since both are valid

✓ 8

Correct. 8, the value the add produced.

✗ minus 5

Wrong, and that is precisely the stale value the hazard would deliver.

✗ 3

Wrong. 3 is neither of the two values in the example.

✗ Either, since both are valid

Wrong. Only the newer value is correct.

8, the value the add produced. The hazard is that without forwarding the sub would read minus 5 instead.

12.120

MEDIUM

Pipe S41

In the example, the second instruction is dependent on the result in _____ of the first.

- a. \$s0
- b. \$t0
- c. \$t2
- d. \$t3

Answer: **\$s0**

\$s0, the destination of the add and a source of the sub.

12.121

MEDIUM

Pipe S42

The correct value is only available in \$s0 at the end of clock ____ .

- a. 5
- b. 3
- c. 4
- d. 6

Answer: **5**

| Clock 5, the write back stage of the add.

12.122

MEDIUM

Pipe S43

Because the data is not available until MEM/WB, the subtract ALU cycle has to be stalled, introducing a ____ bubble.

- a. vertical
- b. horizontal
- c. double
- d. silent

Answer: **vertical**

| A vertical bubble, as the deck names it on the timing diagram.

12.123

MEDIUM

Pipe S47

In the WAW description, instruction i is a ____ and instruction j is an add, and both write to the same register.

- a. load
- b. store
- c. branch
- d. shift

Answer: **load**

| A load. It is slower to produce its result, which is what would let the later add finish first.

12.124

MEDIUM

Pipe S48

The data hazard classification exercise asks students to do what with the three code snippets?

- a. State which is RAW, WAR or WAW, and indicate how each occurs using an arrow
- b. Rewrite them to remove all hazards
- c. Compute the speedup of each
- d. Convert them to reverse Polish notation

✓ State which is RAW, WAR or WAW, and indicate how each occurs using an arrow
Correct. Classify each and draw the arrow.

✗ Rewrite them to remove all hazards
Wrong. Rewriting is what Example 2 asks for, separately.

✗ Compute the speedup of each
Wrong. No speedup is asked for here.

✗ Convert them to reverse Polish notation
Wrong. Reverse Polish notation belongs to the instruction formats topic.

Classify each and draw the arrow. The arrow is asked for because it shows which instruction depends on which.

12.125

MEDIUM

Pipe S49

Between I1 and I2, I2 reads R1 after I1 writes R1, which is a _____ hazard.

- a. RAW
- b. WAW
- c. WAR
- d. structural

Answer: **RAW**

RAW, the most common kind and the one forwarding fixes.

12.126

MEDIUM

Pipe S49

Between I1 and I3, I3 writes R1 after I1 writes R1, which is a ____ hazard.

- a. WAW
- b. RAW
- c. WAR
- d. control

Answer: **WAW**

WAW. In DLX this could not arise, because every write happens in WB, but the exercise asks you to classify it regardless.

12.127

MEDIUM

Pipe S51

Placing a dependent ADD immediately after a LOAD creates a ____ hazard.

- a. load-use
- b. control
- c. structural
- d. write-write

Answer: **load-use**

A load-use hazard, the case forwarding cannot fix.

12.128

MEDIUM

Pipe S52

The SUB instruction fills the ____ slot and reduces stalling.

- a. delay
- b. memory
- c. branch
- d. write

Answer: **delay**

The delay slot. This is instruction scheduling, and it is exactly the compiler work RISC relies on.

12.129

MEDIUM

Pipe S53

Which two situations does the deck describe as data hazards?

- a. One instruction cannot start until the previous returns its results, and both instructions try to modify the same data object
- b. Two instructions need the same functional unit, and a branch changes the PC
- c. The pipeline is unbalanced, and the clock is too slow
- d. A cache miss occurs, and memory is slow

✓ One instruction cannot start until the previous returns its results, and both instructions try to modify the same data object

Correct. Waiting for a result, and two writers of the same object.

✗ Two instructions need the same functional unit, and a branch changes the PC

Wrong. Those are structural and control hazards.

✗ The pipeline is unbalanced, and the clock is too slow

Wrong. Those reduce speedup but are not hazards.

✗ A cache miss occurs, and memory is slow

Wrong. Cache behaviour is a memory topic.

Waiting for a result, and two writers of the same object. Both turn on data rather than on hardware or on control flow.

12.130

MEDIUM

Pipe S53

One instruction cannot be started until the previous instruction returns the results, as both are ____ .

- a. interdependent
- b. independent
- c. identical
- d. sequential

Answer: **interdependent**

Interdependent, which is the deck's word for the dependency between successive tasks.

12.131

MEDIUM

Pipe S54

The deck's example of a resource constraint is:

- a. One common memory used for both data and instructions, so a read or write and an instruction fetch cannot happen at the same time
- b. Two registers holding the same value
- c. A branch whose target is unknown
- d. A cache line holding the wrong block

✓ One common memory used for both data and instructions, so a read or write and an instruction fetch cannot happen at the same time

Correct. One memory, two demands, so one has to wait.

✗ Two registers holding the same value

Wrong. Duplicate values cause no conflict.

✗ A branch whose target is unknown

Wrong. That is a control hazard.

✗ A cache line holding the wrong block

Wrong. That is a cache miss.

One memory, two demands, so one has to wait. This is the same conflict a Harvard architecture removes by giving instructions and data separate memories.

12.132

MEDIUM

Pipe S54

When resources are not available at the time of execution, delays are caused in pipelining.

Answer: **true**

That is the definition of a resource constraint, and it is the structural hazard seen from the resource's side.

The conclusion says that by dividing instruction execution into stages and allowing overlapping operations, pipelining:

- a. Increases processor throughput and makes better use of CPU resources
- b. Reduces the latency of each instruction
- c. Removes the need for a cache
- d. Simplifies the instruction set

✓ Increases processor throughput and makes better use of CPU resources

Correct. Throughput and utilization, the same two benefits given at the start of the deck.

✗ Reduces the latency of each instruction

Wrong. Latency per instruction is not improved.

✗ Removes the need for a cache

Wrong, and RISC machines need large caches precisely to feed their pipelines.

✗ Simplifies the instruction set

Wrong. The instruction set is a separate matter.

Throughput and utilization, the same two benefits given at the start of the deck.

Select the five stages of the RISC load-store pipeline.

- a. Instruction fetch, IF
- b. Instruction Decode, ID
- c. Execute, EX
- d. Memory, MEM
- e. Writeback, WB
- f. Commit, CM

✓ Instruction fetch, IF

Correct. The same five as the pipelining deck: IF, ID, EX, MEM, WB.

✓ Instruction Decode, ID

Correct. The same five as the pipelining deck: IF, ID, EX, MEM, WB.

✓ Execute, EX

Correct. The same five as the pipelining deck: IF, ID, EX, MEM, WB.

✓ Memory, MEM

Correct. The same five as the pipelining deck: IF, ID, EX, MEM, WB.

✓ Writeback, WB

Correct. The same five as the pipelining deck: IF, ID, EX, MEM, WB.

✗ Commit, CM

Wrong. No commit stage is named anywhere in this course.

The same five as the pipelining deck: IF, ID, EX, MEM, WB. There is no commit stage in this course.

In the five-stage RISC pipeline, what happens in IF and ID?

- a. IF gets the instruction from memory and increments the PC; ID translates the opcode into control signals and reads registers
- b. IF translates the opcode; ID gets the instruction from memory
- c. IF reads registers; ID increments the PC
- d. IF performs the ALU operation; ID accesses memory

✓ IF gets the instruction from memory and increments the PC; ID translates the opcode into control signals and reads registers

Correct. Fetch and increment, then decode and read registers.

✗ IF translates the opcode; ID gets the instruction from memory

Wrong, and reversed.

✗ IF reads registers; ID increments the PC

Wrong on both. Fetch and increment, then decode and read registers.

✗ IF performs the ALU operation; ID accesses memory

Wrong. Those are EX and MEM.

Fetch and increment, then decode and read registers. Reading registers in ID is why the data hazard shows up there.

12.136

MEDIUM

RISC S27

IF gets the instruction from memory and increments the _____, while ID translates the opcode into _____ signals and reads registers.

BLANK 1

- a. PC
- b. IR
- c. MAR
- d. SP

BLANK 2

- a. control
- b. data
- c. address
- d. clock

Answer: **PC, control**

The PC, and control signals. Generating control signals is the control unit's job, as the introduction deck has it.

12.137

MEDIUM

RISC S27

In the five-stage RISC pipeline, what does EX do besides performing the ALU operation?

- a. Computes the jump or branch target
- b. Reads the registers
- c. Updates the register file
- d. Increments the PC

✓ Computes the jump or branch target

Correct. The branch target is computed in EX, which is why a branch is not resolved until then and why control hazards exist.

✗ Reads the registers

Wrong. Registers are read in ID.

✗ Updates the register file

Wrong. The register file is updated in WB.

✗ Increments the PC

Wrong. The PC is incremented in IF.

The branch target is computed in EX, which is why a branch is not resolved until then and why control hazards exist.

12.138

MEDIUM

RISC S27

Match each of the last three RISC pipeline stages to its job.

Execute	Perform the ALU operation and compute the jump or branch target
Memory	Access memory if needed
Writeback	Update the register file

Three stages, three jobs. Note if needed against Memory: only loads and stores use it.

In the revision pipelining example, what are the given figures?

- a. 60 ns per instruction non-pipelined, a 6-stage pipeline, a 12 ns cycle time, and 50 instructions
- b. 60 ns per instruction, a 5-stage pipeline, a 12 ns cycle time, and 50 instructions
- c. 20 ns per stage, a 4-stage pipeline, and 100 instructions
- d. 60, 50, 90 and 80 ns stages with a 10 ns latch delay

✓ 60 ns per instruction non-pipelined, a 6-stage pipeline, a 12 ns cycle time, and 50 instructions

Correct. 60 ns, 6 stages, 12 ns, 50 instructions.

✗ 60 ns per instruction, a 5-stage pipeline, a 12 ns cycle time, and 50 instructions

Wrong. The pipeline has 6 stages.

✗ 20 ns per stage, a 4-stage pipeline, and 100 instructions

Wrong. Those are the Pipe deck's Example figures.

✗ 60, 50, 90 and 80 ns stages with a 10 ns latch delay

Wrong. Those are revision Example 2's figures.

60 ns, 6 stages, 12 ns, 50 instructions. Note that 6 times 12 is 72, not 60, so the pipelined version is not simply the old instruction cut into six.

12.140

MEDIUM

Rev S23

How long do 50 instructions take without pipelining?

- a. 3000 ns, that is 3.0 microseconds
- b. 660 ns
- c. 600 ns
- d. 72 ns

✓ 3000 ns, that is 3.0 microseconds

Correct. 50 times 60 ns is 3000 ns, which is 3.

✗ 660 ns

Wrong. 660 ns is the pipelined answer.

✗ 600 ns

Wrong by a factor of five.

✗ 72 ns

Wrong. 72 ns is the time for the first instruction to emerge from the pipeline.

50 times 60 ns is 3000 ns, which is 3.0 microseconds. Each instruction must complete before the next begins.

12.141

MEDIUM

Rev S23

T non-pipeline equals n times t instruction, so 50 times 60 is _____ ns.

- a. 3000
- b. 660
- c. 600
- d. 300

Answer: **3000**

3000 ns, which the deck also writes as 3.0 microseconds.

12.142

MEDIUM

Rev S24

How long does the first instruction take to complete in the 6-stage pipeline?

- a. 72 ns, from 6 times 12
- b. 60 ns
- c. 12 ns
- d. 588 ns

✓ 72 ns, from 6 times 12

Correct. 6 clock cycles at 12 ns each is 72 ns.

✗ 60 ns

Wrong. 60 ns is the non-pipelined instruction time.

✗ 12 ns

Wrong. 12 ns is one cycle.

✗ 588 ns

Wrong. 588 ns is the time for the remaining 49 instructions.

6 clock cycles at 12 ns each is 72 ns. This is the fill time, and it is longer than the 60 ns the non-pipelined machine takes per instruction.

12.143

MEDIUM

Rev S24

A 6-stage pipeline requires _____ clock cycles for the first instruction to complete.

- a. 6
- b. 12
- c. 50
- d. 5

Answer: **6**

6 cycles, one per stage.

12.144

MEDIUM

Rev S25

After the pipeline is filled, how much time do the remaining instructions add?

- a. 49 times 12, which is 588 ns
- b. 50 times 12, which is 600 ns
- c. 49 times 60, which is 2940 ns
- d. 6 times 12, which is 72 ns

✓ 49 times 12, which is 588 ns

Correct. 49 remaining instructions, one completing every cycle, at 12 ns each.

✗ 50 times 12, which is 600 ns

Wrong. The first instruction is already counted in the 72 ns.

✗ 49 times 60, which is 2940 ns

Wrong. Once filled, the pipeline advances at the cycle time, not the whole instruction time.

✗ 6 times 12, which is 72 ns

Wrong. That is the fill time.

| 49 remaining instructions, one completing every cycle, at 12 ns each.

12.145

MEDIUM

Rev S25

The general formula is $T_{\text{pipeline}} = k + n \text{ minus } \underline{\hspace{1cm}}, \text{ times } \underline{\hspace{1cm}}.$

BLANK 1

- a. 1
- b. k
- c. n
- d. 2

BLANK 2

- a. t_c
- b. t_n
- c. k
- d. n

Answer: **1, t_c**

| n minus 1, times t_c . Same shape as the Pipe deck's formula, with t_c in place of t_p .

12.146

MEDIUM

Rev S26

Substituting k of 6, n of 50 and t_c of 12 ns, what is the total pipelined time?

- a. 660 ns, that is 0.66 microseconds
- b. 672 ns
- c. 600 ns
- d. 3000 ns

✓ 660 ns, that is 0.66 microseconds

Correct. 6 plus 50 minus 1 is 55, and 55 times 12 is 660 ns, which is 0.

✗ 672 ns

Wrong. That would be 56 cycles; the minus 1 avoids counting the first instruction twice.

✗ 600 ns

Wrong. That is 50 cycles, with no fill time at all.

✗ 3000 ns

Wrong. 3000 ns is the non-pipelined answer.

6 plus 50 minus 1 is 55, and 55 times 12 is 660 ns, which is 0.66 microseconds.

12.147

MEDIUM

Rev S26

6 plus 50 minus 1 is _____ cycles, and at 12 ns each that is _____ ns.

BLANK 1

- a. 55
- b. 56
- c. 50
- d. 44

BLANK 2

- a. 660
- b. 672
- c. 600
- d. 528

Answer: **55, 660**

55 cycles, 660 ns. Check it against the two parts: 72 ns to fill plus 588 ns for the rest is also 660.

12.148

MEDIUM

Rev S27

What is the speedup in the revision pipelining example?

- a. 4.545, approximately 4.55
- b. 6.00
- c. 5.00
- d. 2.80

✓ 4.545, approximately 4.55

Correct. $\frac{3000}{660}$ is 4.

✗ 6.00

Wrong. 6 is the ideal, which the fill time and the cycle overhead prevent.

✗ 5.00

Wrong. That figure does not arise.

✗ 2.80

Wrong. 2.8 is the speedup in revision Example 2.

$\frac{3000}{660}$ is 4.545, approximately 4.55. It falls short of the ideal 6 partly because of the fill time and partly because 6 times 12 is more than 60.

12.149

MEDIUM

Rev S27

Speedup equals T non-pipeline over T pipeline, so 3000 over ____ .

- a. 660
- b. 600
- c. 672
- d. 3000

Answer: **660**

3000 over 660. The direction is always non-pipelined over pipelined.

In revision Example 2, what are the four phase durations and the latch delay?

- a. 60, 50, 90 and 80 ns, with a latch delay of 10 ns
- b. 10, 8, 10, 10 and 7 ns, with an overhead of 1 ns
- c. 60 ns per instruction and a 12 ns cycle
- d. 20 ns per stage over 4 stages

✓ 60, 50, 90 and 80 ns, with a latch delay of 10 ns

Correct. 60, 50, 90, 80, latch 10.

✗ 10, 8, 10, 10 and 7 ns, with an overhead of 1 ns

Wrong. Those are the Pipe deck's Example 3 figures.

✗ 60 ns per instruction and a 12 ns cycle

Wrong. Those are revision Example 1's figures.

✗ 20 ns per stage over 4 stages

Wrong. Those are the Pipe deck's 4-stage worked example.

60, 50, 90, 80, latch 10. The 90 is the one that matters, because the cycle time is the maximum plus the latch delay.

12.151

MEDIUM

Rev S29

Cycle time equals the maximum delay due to any stage, which is _____ ns, plus the delay due to its register, which is _____ ns.

BLANK 1

- a. 90
- b. 80
- c. 60
- d. 100

BLANK 2

- a. 10
- b. 1
- c. 20
- d. 12

Answer: **90, 10**

| 90 plus 10 is 100 ns.

12.152

MEDIUM

Rev S30

In revision Example 2, what are the non-pipeline execution time for one instruction and the speed up ratio?

- a. 280 ns and 2.8
- b. 290 ns and 2.9
- c. 280 ns and 4.0
- d. 100 ns and 2.8

✓ 280 ns and 2.8

Correct. 60 plus 50 plus 90 plus 80 is 280 ns, and 280 over the 100 ns cycle time is 2.8.

✗ 290 ns and 2.9

Wrong. The latch delay is not added to the non-pipelined time.

✗ 280 ns and 4.0

Wrong. $\frac{280}{100}$ is 2.8, not 4.

✗ 100 ns and 2.8

Wrong. 100 ns is the cycle time, not the non-pipelined instruction time.

60 plus 50 plus 90 plus 80 is 280 ns, and 280 over the 100 ns cycle time is 2.8.

12.153

MEDIUM

Rev S30

Non-pipeline execution time for one instruction is 60 plus 50 plus 90 plus 80, which is _____ ns.

- a. 280
- b. 290
- c. 100
- d. 240

Answer: **280**

280 ns. Latch delays do not apply when there are no pipeline registers.

12.154

MEDIUM

Rev S31

What is the sequential, non-pipeline time for 500 tasks?

- a. 140,000 ns, from 500 times 280 ns
- b. 50,300 ns
- c. 50,000 ns, from 500 times 100 ns
- d. 280,000 ns

✓ 140,000 ns, from 500 times 280 ns

Correct. 500 times the 280 ns each task takes on its own, giving 140,000 ns.

✗ 50,300 ns

Wrong. That is the pipelined figure.

✗ 50,000 ns, from 500 times 100 ns

Wrong. 100 ns is the pipelined cycle time, not the sequential task time.

✗ 280,000 ns

Wrong by a factor of two.

500 times the 280 ns each task takes on its own, giving 140,000 ns. Against 50,300 ns pipelined, that is close to the 2.8 speedup.

12.155

MEDIUM

Rev S31

Non-pipeline time for 500 tasks is 500 times 280 ns, which is _____ ns.

- a. 140000
- b. 50300
- c. 50000
- d. 280000

Answer: **140000**

140,000 ns.

12.156

MEDIUM

Rev S35

In the revision class repeat of the data hazard classification, which hazard is named as the most common?

- a. RAW, solved by data forwarding
- b. WAW, solved by stalling
- c. WAR, solved by reordering
- d. All three equally

✓ RAW, solved by data forwarding

Correct. RAW, most common, solved by data forwarding.

✗ WAW, solved by stalling

Wrong. WAW cannot even occur in DLX.

✗ WAR, solved by reordering

Wrong. WAR also cannot occur in DLX.

✗ All three equally

Wrong. RAW is singled out as the most common.

RAW, most common, solved by data forwarding. The revision deck repeats the Pipe deck's classification word for word.

12.157

HARD

Pipe S5

Match each pipelining remedy to the problem it chiefly addresses.

Forwarding	Data hazards
Stalling	Structural hazards, and data hazards forwarding cannot fix
Branch prediction	Control hazards

Three remedies, three hazard families. Stalling is the general fallback: it works everywhere but always costs cycles.

Pipelining is described as decomposing a sequential process into sub-operations, with each sub-process executed in:

- a. A special dedicated segment that operates concurrently with all other segments
- b. A shared segment used by all sub-processes in turn
- c. A separate processor
- d. The same functional unit, one after the other

✓ A special dedicated segment that operates concurrently with all other segments

Correct. A dedicated segment, running concurrently with the others.

✗ A shared segment used by all sub-processes in turn

Wrong. Sharing would serialise them again.

✗ A separate processor

Wrong. All the segments are within one processor.

✗ The same functional unit, one after the other

Wrong. One after the other is the non-pipelined case.

A dedicated segment, running concurrently with the others. Dedicated is what allows concurrent: if the segments shared hardware there would be a structural hazard.

12.159

HARD

Pipe S13

In general, stage time equals:

- a. Time per instruction on a non-pipelined machine, divided by the number of stages
- b. Time per instruction on a non-pipelined machine, times the number of stages
- c. The number of stages divided by the time per instruction
- d. The sum of all the stage times

✓ Time per instruction on a non-pipelined machine, divided by the number of stages

Correct. Divide the whole instruction time among the stages.

✗ Time per instruction on a non-pipelined machine, times the number of stages

Wrong. Multiplying would make each stage longer than the whole instruction.

✗ The number of stages divided by the time per instruction

Wrong, and it inverts the division.

✗ The sum of all the stage times

Wrong. The sum is the non-pipelined instruction time.

Divide the whole instruction time among the stages. The deck adds that in many instances stage time is instead the maximum of the individual stage times.

12.160

HARD

Pipe S13

In many instances, stage time equals the _____ of the times for all stages.

- a. max
- b. min
- c. sum
- d. average

Answer: **max**

The maximum. It is the rule the worked Example 3 uses: MAX of 10, 8, 10, 10 and 7, plus overhead.

12.161

HARD

Pipe S16

With overlapping tasks the laundry takes 3.5 hours instead of 8. What is the speedup?

- a. 2.1
- b. 4
- c. 2.8
- d. 3.88

✓ 2.1

Correct. $\frac{8}{3}$.

✗ 4

Wrong. 4 is the speedup in pipeline performance Example 2.

✗ 2.8

Wrong. 2.8 is the speedup in the revision Example 2.

✗ 3.88

Wrong. 3.88 is the speedup in the 4-stage, 100 task example.

| 8 over 3.5 is 2.1. Speedup is always the non-pipelined time over the pipelined time.

Pipelining does not help what, and does help what?

- a. It does not help the latency of a single task; it helps the throughput of the entire workload
- b. It helps the latency of a single task but not the throughput
- c. It helps both equally
- d. It helps neither, only the clock rate

✓ It does not help the latency of a single task; it helps the throughput of the entire workload

Correct. Latency unchanged, throughput improved.

✗ It helps the latency of a single task but not the throughput

Wrong, and exactly reversed.

✗ It helps both equally

Wrong. Latency is not improved at all.

✗ It helps neither, only the clock rate

Wrong. The clock rate is a separate matter.

Latency unchanged, throughput improved. In the laundry, Ann's own washing still takes two hours; it is everyone else who finishes sooner.

Each problem takes 12 minutes to grade. How long does one exam take, and what does having 5 graders achieve?

- a. Each exam still takes 1 hour, but with 5 people all the exams are graded five times quicker
- b. Each exam takes 12 minutes, and all the exams are graded five times quicker
- c. Each exam takes 1 hour, and there is no overall gain
- d. Each exam takes 12 minutes, and there is no overall gain

✓ Each exam still takes 1 hour, but with 5 people all the exams are graded five times quicker

Correct. 5 problems times 12 minutes is 60 minutes per exam, unchanged.

✗ Each exam takes 12 minutes, and all the exams are graded five times quicker

Wrong. 12 minutes is one problem, not one exam.

✗ Each exam takes 1 hour, and there is no overall gain

Wrong. There is a fivefold gain in throughput.

✗ Each exam takes 12 minutes, and there is no overall gain

Wrong on both counts.

5 problems times 12 minutes is 60 minutes per exam, unchanged. The gain is entirely in throughput, which is the same lesson as latency against throughput.

The total time for n tasks in a pipelined solution is:

- a. T times k , plus T times n minus 1
- b. T times k times n
- c. T times n minus k
- d. T times k minus n

✓ T times k , plus T times n minus 1

Correct. The first task costs a full k stages, and every one after it costs a single stage time.

✗ T times k times n

Wrong, and that is the non-pipelined total.

✗ T times n minus k

Wrong. Nothing is subtracted from n times T at that level.

✗ T times k minus n

Wrong, and it could go negative.

The first task costs a full k stages, and every one after it costs a single stage time. That is the fill cost plus one per task.

12.165**HARD**

Pipe S21

The pipelined total is T times k , plus T times n minus _____, because the first task pays the full pipeline _____.

BLANK 1

- a. 1
- b. k
- c. n
- d. 2

BLANK 2

- a. fill
- b. drain
- c. stall
- d. hazard

Answer: **1, fill**

n minus 1, because the first task is already counted in the T times k fill.

12.166

HARD

Pipe S21

Speedup works out as $k \times n$ over $k + n - 1$. What does it approach when n is much greater than k ?

- a. k
- b. n
- c. 1
- d. $k \times n$

✓ k

Correct. It approaches k , the number of stages, which is the ideal speedup.

✗ n

Wrong. Speedup cannot exceed the stage count.

✗ 1

Wrong. That would mean no gain at all.

✗ $k \times n$

Wrong. The product is far larger than any achievable speedup.

It approaches k , the number of stages, which is the ideal speedup. The fill cost becomes negligible against a long run of tasks.

For a pipelined machine with k stages and clock cycle t_p , the time to complete n tasks is:

- a. t_k equals k plus n minus 1, all times t_p
- b. t_k equals k times n times t_p
- c. t_k equals k minus n plus 1, all times t_p
- d. t_k equals n times t_p

✓ t_k equals k plus n minus 1, all times t_p

Correct. K cycles to fill, then one per remaining task.

✗ t_k equals k times n times t_p

Wrong, and that is the non-pipelined shape.

✗ t_k equals k minus n plus 1, all times t_p

Wrong. The signs are reversed and it could go negative.

✗ t_k equals n times t_p

Wrong. That ignores the fill time entirely.

| k cycles to fill, then one per remaining task. The bracket k plus n minus 1 counts the total cycles.

12.168

HARD

Pipe S24

Pipeline speedup S_k is given as:

- a. $n \text{ times } t_n$, divided by $k \text{ plus } n \text{ minus } 1$, all times t_p
- b. $k \text{ plus } n \text{ minus } 1$, all times t_p , divided by $n \text{ times } t_n$
- c. $n \text{ times } t_p$, divided by $k \text{ times } t_n$
- d. $k \text{ times } n$, divided by t_p

✓ $n \text{ times } t_n$, divided by $k \text{ plus } n \text{ minus } 1$, all times t_p

Correct. Non-pipelined over pipelined, which is always the direction of a speedup.

✗ $k \text{ plus } n \text{ minus } 1$, all times t_p , divided by $n \text{ times } t_n$

Wrong, and it is upside down, which would give a number less than 1.

✗ $n \text{ times } t_p$, divided by $k \text{ times } t_n$

Wrong. The two clock cycles are in the wrong places.

✗ $k \text{ times } n$, divided by t_p

Wrong. Neither total time appears.

| Non-pipelined over pipelined, which is always the direction of a speedup.

In the 4-stage example, what are the pipelined and non-pipelined totals, and the speedup?

- a. 2060 ns and 8000 ns, giving 3.88
- b. 8000 ns and 2060 ns, giving 3.88
- c. 2060 ns and 8000 ns, giving 4.00
- d. 2000 ns and 8000 ns, giving 4.00

✓ 2060 ns and 8000 ns, giving 3.88

Correct. 4 plus 99, times 20, is 2060 ns pipelined; 100 times 80 is 8000 ns non-pipelined; and 8000 over 2060 is 3.88.

✗ 8000 ns and 2060 ns, giving 3.88

Wrong, and it labels the two totals the wrong way round.

✗ 2060 ns and 8000 ns, giving 4.00

Wrong. The ideal 4 is never quite reached because of the fill time.

✗ 2000 ns and 8000 ns, giving 4.00

Wrong. The pipelined total is 2060, not 2000; the extra 60 ns is the fill.

4 plus 99, times 20, is 2060 ns pipelined; 100 times 80 is 8000 ns non-pipelined; and 8000 over 2060 is 3.88, just short of the ideal 4.

12.170**HARD**

Pipe S25

The pipelined system takes 4 plus 99, times 20, which is _____ ns, and the speedup is 8000 over that, which is _____ .

BLANK 1

- a. 2060
- b. 2000
- c. 8000
- d. 2080

BLANK 2

- a. 3.88
- b. 4.00
- c. 2.80
- d. 4.55

Answer: **2060, 3.88**

2060 ns and 3.88. The 99 is n minus 1, the tasks after the first.

12.171

HARD

Pipe S28

In Design 1 of pipeline performance Example 2, the average instruction execution time is clock cycle time times CPI. What does it come to?

- a. 44 ns
- b. 11 ns
- c. 10 ns
- d. 4 ns

✓ 44 ns

Correct. 10 ns times the bracket 4 times 0.

✗ 11 ns

Wrong. 11 ns is Design 2.

✗ 10 ns

Wrong. 10 ns is the clock cycle time alone.

✗ 4 ns

Wrong. 4 is the speedup, not a time.

10 ns times the bracket 4 times 0.4 plus 4 times 0.2 plus 5 times 0.4, that is 10 times 1.6 plus 0.8 plus 2.0, which is 10 times 4.4, giving 44 ns.

12.172**HARD**

Pipe S28

In Design 1 the three CPI terms are 1.6, _____ and 2.0, and multiplying their sum by the 10 ns clock cycle gives _____ ns.

BLANK 1

- a. 0.8
- b. 1.6
- c. 2.0
- d. 0.4

BLANK 2

- a. 44
- b. 11
- c. 40
- d. 4

Answer: **0.8, 44**

0.8, and 44 ns. Average instruction execution time equals clock cycle time times CPI.

In Design 2 the average instruction time at steady state is the clock cycle time. What is it, and what is the speedup?

- a. 11 ns, being 10 ns plus 1 ns for setup and clock skew, giving a speedup of 4
- b. 10 ns, giving a speedup of 4.4
- c. 11 ns, giving a speedup of 4.4
- d. 12 ns, giving a speedup of 3.67

✓ 11 ns, being 10 ns plus 1 ns for setup and clock skew, giving a speedup of 4

Correct. 10 plus 1 is 11 ns, and 44 over 11 is exactly 4.

✗ 10 ns, giving a speedup of 4.4

Wrong. The 1 ns overhead has to be added.

✗ 11 ns, giving a speedup of 4.4

Wrong. $\frac{44}{11}$ is 4, not 4.4.

✗ 12 ns, giving a speedup of 3.67

Wrong. The overhead is 1 ns, not 2.

10 plus 1 is 11 ns, and 44 over 11 is exactly 4. The extra nanosecond is the price of the pipeline registers.

12.174

HARD

Pipe S29

With functional unit times of 10, 8, 10, 10 and 7 ns and an overhead of 1 ns per stage, what is the pipelined stage time?

- a. 11 ns, being the maximum of the five times plus the overhead
- b. 46 ns, being the sum plus the overhead
- c. 9 ns, being the average plus the overhead
- d. 8 ns, being the minimum plus the overhead

✓ 11 ns, being the maximum of the five times plus the overhead

Correct, and this is the average instruction execution time at steady state.

✗ 46 ns, being the sum plus the overhead

Wrong. The sum is the non-pipelined time, 45 ns.

✗ 9 ns, being the average plus the overhead

Wrong. The average is not what limits a pipeline.

✗ 8 ns, being the minimum plus the overhead

Wrong. The fastest stage waits for the others.

MAX of 10, 8, 10, 10, 7 is 10, plus 1 ns overhead, giving 11 ns. The slowest stage sets the rate for all of them.

In Example 3, what are the non-pipelined time and the speedup?

- a. 45 ns, giving a speedup of 4.1 times
- b. 45 ns, giving a speedup of 4.5 times
- c. 50 ns, giving a speedup of 4.55 times
- d. 44 ns, giving a speedup of 4 times

✓ 45 ns, giving a speedup of 4.1 times

Correct. 10 plus 8 plus 10 plus 10 plus 7 is 45 ns, and 45 over 11 is 4.1.

✗ 45 ns, giving a speedup of 4.5 times

Wrong. $\frac{45}{11}$ is 4.1, not 4.5.

✗ 50 ns, giving a speedup of 4.55 times

Wrong. The sum is 45, and 4.55 is the revision Example 1 answer.

✗ 44 ns, giving a speedup of 4 times

Wrong. 44 ns and a speedup of 4 belong to Example 2.

| 10 plus 8 plus 10 plus 10 plus 7 is 45 ns, and 45 over 11 is 4.1 times.

12.176**HARD**

Pipe S29

The non-pipelined time is 10 plus 8 plus 10 plus 10 plus 7, which is _____ ns, so the speedup is that over 11, which is _____ times.

BLANK 1

- a. 45
- b. 44
- c. 46
- d. 50

BLANK 2

- a. 4.1
- b. 4.0
- c. 4.5
- d. 3.88

Answer: **45, 4.1**

45 ns and 4.1 times. Note the pattern: with five stages the ideal is 5, and the unbalanced stage times plus the overhead cost the rest.

12.177

HARD

Pipe S30

The efficiency E simplifies to n divided by m plus n minus 1. What does E become as n tends to infinity?

- a. 1
- b. 0
- c. m
- d. n

✓ 1

Correct. E becomes 1, that is 100 per cent.

✗ 0

Wrong. Efficiency rises with n , it does not fall.

✗ m

Wrong. m is the stage count, and efficiency is a ratio bounded by 1.

✗ n

Wrong. Efficiency cannot exceed 1.

E becomes 1, that is 100 per cent. With enough tasks the fill and drain become negligible and no stage is ever idle.

12.178

HARD

Pipe S30

Efficiency E equals n divided by m plus n minus ____ .

- a. 1
- b. m
- c. n
- d. 2

Answer: 1

Minus 1. The denominator m plus n minus 1 is the same cycle count as in the speedup formula.

Throughput of a pipeline is defined as the number of results achieved per unit time, and is denoted:

- a. T equals E over c, where c is the clock period
- b. T equals E times c
- c. T equals c over E
- d. T equals E plus c

✓ T equals E over c, where c is the clock period

Correct. $\frac{E}{c}$.

✗ T equals E times c

Wrong. Multiplying by a time would not give a rate.

✗ T equals c over E

Wrong, and inverted.

✗ T equals E plus c

Wrong. The two quantities do not add.

$\frac{E}{c}$. Since c is a time, dividing by it turns a dimensionless efficiency into a rate.

12.180

HARD

Pipe S33

With stalls, speedup is given as:

- a. 1 divided by 1 plus pipeline stall per instruction, times the number of stages
- b. 1 plus pipeline stall per instruction, times the number of stages
- c. The number of stages divided by the stall per instruction
- d. The number of stages minus the stall per instruction

✓ 1 divided by 1 plus pipeline stall per instruction, times the number of stages

Correct. The ideal speedup, the stage count, scaled down by the factor 1 over 1 plus the stalls.

✗ 1 plus pipeline stall per instruction, times the number of stages

Wrong. Stalls would then increase the speedup, which is backwards.

✗ The number of stages divided by the stall per instruction

Wrong. It divides by zero when there are no stalls.

✗ The number of stages minus the stall per instruction

Wrong. The relationship is multiplicative, not subtractive.

The ideal speedup, the stage count, scaled down by the factor 1 over 1 plus the stalls. With no stalls the factor is 1 and the ideal is reached.

12.181

HARD

Pipe S33

Speedup with stalls is 1 over 1 plus the pipeline stall per instruction, times the number of _____ .

- a. stages
- b. instructions
- c. cycles
- d. hazards

Answer: **stages**

Stages, the ideal speedup that the stall factor then reduces.

12.182

HARD

Pipe S39

Why does forwarding work at all?

- a. Because the data selected is not really used in ID but in the next stage, the ALU, so the result from the EX/MEM buffer can be fed back to the ALU input latches
- b. Because the register file is written before it is read
- c. Because the pipeline is stalled first
- d. Because the compiler reorders the instructions

✓ Because the data selected is not really used in ID but in the next stage, the ALU, so the result from the EX/MEM buffer can be fed back to the ALU input latches

Correct. The value is not needed until EX, which buys a cycle.

✗ Because the register file is written before it is read

Wrong. Writing early and reading late is a separate trick, for the instruction three cycles back.

✗ Because the pipeline is stalled first

Wrong. Forwarding exists precisely to avoid stalling.

✗ Because the compiler reorders the instructions

Wrong. Forwarding is hardware; reordering is software.

The value is not needed until EX, which buys a cycle. That is enough time for the previous instruction's ALU result to be routed back.

12.183

HARD

Pipe S39

The ALU result from the _____ buffer is always fed back to the ALU input latches.

- a. EX/MEM
- b. MEM/WB
- c. IF/ID
- d. ID/EX

Answer: **EX/MEM**

EX/MEM, the buffer one cycle behind. MEM/WB, two cycles behind, is forwarded as well.

12.184

HARD

Pipe S40

Results need to be forwarded not only from the immediately previous instruction but from any instruction that started:

- a. Up to three cycles before
- b. Up to two cycles before
- c. Up to five cycles before
- d. Only one cycle before

✓ Up to three cycles before

Correct. Three cycles back.

✗ Up to two cycles before

Wrong. Two cycles is the MEM/WB case alone.

✗ Up to five cycles before

Wrong. By five cycles the earlier instruction has left the pipeline.

✗ Only one cycle before

Wrong. The deck is explicit that it is not only the immediately previous instruction.

Three cycles back. One and two cycles are covered by the EX/MEM and MEM/WB buffers, and the third by the register file's split-cycle write and read.

12.185

HARD

Pipe S40

Match each buffer to how far back the instruction it holds started.

EX/MEM

1 cycle before

MEM/WB

2 cycles before

Both are forwarded to both ALU inputs. The third cycle back is handled differently, by writing the register file early in the cycle and reading it late.

How is the instruction three cycles before handled?

- a. Writing into the register file is done in the first half of the cycle and reading in the second half
- b. By forwarding from a third buffer
- c. By stalling for one cycle
- d. By reordering the instructions

✓ Writing into the register file is done in the first half of the cycle and reading in the second half

Correct. Write early, read late, within the same cycle.

✗ By forwarding from a third buffer

Wrong. Only EX/MEM and MEM/WB are named.

✗ By stalling for one cycle

Wrong. No stall is needed at three cycles' distance.

✗ By reordering the instructions

Wrong. Reordering is a software remedy for a different case.

Write early, read late, within the same cycle. That single trick removes the need for a third forwarding path.

12.187

HARD

Pipe S40

Writing into the register file is done in the _____ half of the cycle and reading is done in the _____ half.

BLANK 1

- a. first
- b. second
- c. third
- d. last

BLANK 2

- a. second
- b. first
- c. same
- d. next

Answer: **first, second**

Write first, read second, so the read sees the value written moments earlier.

12.188

HARD

Pipe S42

In the multiple clock cycle diagram, in which clock does the add write to \$s0 and in which does the sub read it?

- a. The add writes in clock 5 and the sub reads in clock 3
- b. The add writes in clock 3 and the sub reads in clock 5
- c. Both happen in clock 4
- d. The add writes in clock 5 and the sub reads in clock 6

✓ The add writes in clock 5 and the sub reads in clock 3

Correct. Write in CC5, read in CC3.

✗ The add writes in clock 3 and the sub reads in clock 5

Wrong, and reversed, which would give no hazard at all.

✗ Both happen in clock 4

Wrong. The two events are two cycles apart.

✗ The add writes in clock 5 and the sub reads in clock 6

Wrong. Reading after the write would also be safe.

Write in CC5, read in CC3. The read comes first, which is why the value found there is still minus 5.

12.189

HARD

Pipe S42

The value in \$s0 at CC3 is still _____ and not the correct value _____.

BLANK 1

- a. -5
- b. 8
- c. 5
- d. 3

BLANK 2

- a. 8
- b. -5
- c. 5
- d. 13

Answer: **-5, 8**

Minus 5 instead of 8. The sub reads in its ID stage, which falls in CC3.

In which direction does the dependency in this example run?

- a. From CC5 back to CC3, that is backward
- b. From CC3 forward to CC5
- c. Within CC3 only
- d. Within CC5 only

✓ From CC5 back to CC3, that is backward

Correct, and the deck marks it backward.

✗ From CC3 forward to CC5

Wrong. A forward dependency would be harmless.

✗ Within CC3 only

Wrong. Two different cycles are involved.

✗ Within CC5 only

Wrong, for the same reason.

Backward, from CC5 to CC3. A dependency that points backwards in time is exactly what cannot be satisfied without forwarding.

Why can forwarding not fix the hazard after LW R1,0(R2) followed by SUB R4,R1,R5?

- a. Because the data is not available until MEM/WB, so the subtract ALU cycle has to be stalled, introducing a vertical bubble
- b. Because the two instructions use different registers
- c. Because the load writes memory rather than a register
- d. Because the ALU is busy

✓ Because the data is not available until MEM/WB, so the subtract ALU cycle has to be stalled, introducing a vertical bubble

Correct, and the deck calls the result a vertical bubble.

✗ Because the two instructions use different registers

Wrong. Both use R1, which is the dependence.

✗ Because the load writes memory rather than a register

Wrong. A load writes a register; a store writes memory.

✗ Because the ALU is busy

Wrong. A busy ALU would be a structural hazard.

A load gets its value at MEM, one stage later than an ALU instruction, so there is no earlier point to forward from. A stall is unavoidable.

WAW means Write After Write. Why is there no WAW hazard in DLX?

- a. Because DLX waits for WB to write to registers
- b. Because DLX has no write instructions
- c. Because DLX reads early in ID
- d. Because DLX forwards from EX/MEM

✓ Because DLX waits for WB to write to registers

Correct. All writes happen in WB, so two writes to the same register cannot get out of order.

✗ Because DLX has no write instructions

Wrong. Of course it has writes.

✗ Because DLX reads early in ID

Wrong, and that is the reason WAR cannot happen, not WAW.

✗ Because DLX forwards from EX/MEM

Wrong. Forwarding addresses RAW.

| All writes happen in WB, so two writes to the same register cannot get out of order.

12.193

HARD

Pipe S47

WAR means Write After Read. Why can it not happen in DLX?

- a. Because all instructions read early, in ID, but write late, in WB
- b. Because all instructions write early and read late
- c. Because DLX has no read instructions
- d. Because DLX stalls on every dependence

✓ Because all instructions read early, in ID, but write late, in WB

Correct. Read early, write late.

✗ Because all instructions write early and read late

Wrong, and reversed, which would create the very hazard.

✗ Because DLX has no read instructions

Wrong. Every instruction reads its operands.

✗ Because DLX stalls on every dependence

Wrong. Stalling on everything would remove the point of pipelining.

Read early, write late. Every read has already happened by the time any later instruction writes.

12.194

HARD

Pipe S47

In WAR, instruction j tries to _____ a destination before it is read by instruction i, so i incorrectly gets its value.

- a. write
- b. read
- c. fetch
- d. decode

Answer: **write**

Write. The later instruction overwrites a value the earlier one had not yet read.

12.195

HARD

Pipe S47

The deck says WAW happens in complex instruction sets that have auto-increment mode and require operands to be read in a late cycle.

Answer: **true**

It does. Reading late is what breaks the read-early, write-late discipline that keeps DLX safe.

In hazard Example 1, with I1 ADD R1,R2,R3 and I2 SUB R4,R1,R5, what kind of hazard is there between them?

- a. RAW, because I2 reads R1 after I1 writes R1
- b. WAW, because both write R1
- c. WAR, because I2 writes before I1 reads
- d. No hazard

✓ RAW, because I2 reads R1 after I1 writes R1
Correct. I1 writes R1 and I2 reads it, so read after write.

✗ WAW, because both write R1
Wrong. I2 writes R4, not R1.

✗ WAR, because I2 writes before I1 reads
Wrong. I2 does not write R1 at all.

✗ No hazard
Wrong. R1 is written then read, which is the classic dependence.

I1 writes R1 and I2 reads it, so read after write.

In hazard Example 1, with I1 ADD R1,R2,R3 and I3 MUL R1,R6,R7, what kind of hazard is there between them?

- a. WAW, because I3 writes R1 after I1 writes R1
- b. RAW, because I3 reads R1 after I1 writes it
- c. WAR, because I3 writes R1 before I1 reads it
- d. No hazard, since they are not adjacent

✓ WAW, because I3 writes R1 after I1 writes R1

Correct. Both write R1, so it is write after write.

✗ RAW, because I3 reads R1 after I1 writes it

Wrong. I3's sources are R6 and R7, not R1.

✗ WAR, because I3 writes R1 before I1 reads it

Wrong. I1 reads R2 and R3, not R1.

✗ No hazard, since they are not adjacent

Wrong. Hazards do not require adjacency.

Both write R1, so it is write after write. I3 does not read R1 at all: it reads R6 and R7.

12.198

HARD

Pipe S50

What is the final answer to hazard Example 1?

- a. I1 to I2 RAW, I1 to I3 WAW, I3 to I4 RAW
- b. I1 to I2 WAW, I1 to I3 RAW, I3 to I4 WAR
- c. I1 to I2 RAW, I1 to I3 RAW, I3 to I4 WAW
- d. All three are RAW

✓ I1 to I2 RAW, I1 to I3 WAW, I3 to I4 RAW

Correct. Two RAWs and one WAW.

✗ I1 to I2 WAW, I1 to I3 RAW, I3 to I4 WAR

Wrong on all three.

✗ I1 to I2 RAW, I1 to I3 RAW, I3 to I4 WAW

Wrong. I1 to I3 is a double write, so WAW.

✗ All three are RAW

Wrong. The I1 to I3 pair is WAW.

Two RAWs and one WAW. The pattern is worth reading off the code: a read of a written register gives RAW, a second write of the same register gives WAW.

12.199

HARD

Pipe S50

Between I3 and I4, I4 reads R1 after I3 writes R1, which is a _____ hazard.

- a. RAW
- b. WAW
- c. WAR
- d. structural

Answer: **RAW**

RAW, the second of the two in this example.

In Example 2, why does the original sequence LOAD, ADD, SUB, MUL create a problem?

- a. Because a dependent ADD immediately follows the LOAD, which creates a load-use hazard
- b. Because the SUB writes a register the MUL reads
- c. Because all four instructions use the ALU
- d. Because the LOAD writes memory

✓ Because a dependent ADD immediately follows the LOAD, which creates a load-use hazard

Correct, and the deck names it a load-use hazard.

✗ Because the SUB writes a register the MUL reads

Wrong. The SUB writes R5, which nothing else reads.

✗ Because all four instructions use the ALU

Wrong. Sharing the ALU across stages is not the issue here.

✗ Because the LOAD writes memory

Wrong. A load reads memory and writes a register.

The ADD needs R1 straight after the LOAD produces it, and a load's value is not ready in time for forwarding to help.

What is the optimized instruction order in Example 2?

- a. LOAD, SUB, ADD, MUL
- b. LOAD, ADD, SUB, MUL
- c. SUB, LOAD, ADD, MUL
- d. LOAD, ADD, MUL, SUB

✓ LOAD, SUB, ADD, MUL

Correct. Move the independent SUB between the LOAD and the dependent ADD.

✗ LOAD, ADD, SUB, MUL

Wrong. That is the original order, the one that stalls.

✗ SUB, LOAD, ADD, MUL

Wrong. Moving the SUB before the LOAD does not separate the LOAD from the ADD.

✗ LOAD, ADD, MUL, SUB

Wrong. The ADD still follows the LOAD immediately.

Move the independent SUB between the LOAD and the dependent ADD. The SUB fills the delay slot and reduces stalling.

With stage delays of 60, 50, 90 and 80 ns and a latch delay of 10 ns, what is the pipeline cycle time?

- a. 100 ns, being the maximum delay plus the register delay
- b. 290 ns, being the sum plus the latch delay
- c. 80 ns
- d. 70 ns, being the average plus the latch delay

✓ 100 ns, being the maximum delay plus the register delay

Correct. MAX of 60, 50, 90 and 80 is 90, plus 10 ns for its register, giving 100 ns.

✗ 290 ns, being the sum plus the latch delay

Wrong. The sum, 280 ns, is the non-pipeline execution time.

✗ 80 ns

Wrong. 80 is not the maximum, and the latch delay is missing.

✗ 70 ns, being the average plus the latch delay

Wrong. The pipeline is limited by the slowest stage, not the average.

| MAX of 60, 50, 90 and 80 is 90, plus 10 ns for its register, giving 100 ns.

What is the pipeline time for 500 tasks in revision Example 2?

- a. 50,300 ns
- b. 50,000 ns
- c. 140,000 ns
- d. 49,900 ns

✓ 50,300 ns

Correct. 4 clock cycles for the first task and 1 each for the remaining 499, so 4 times 100 plus 499 times 100, that is 400 plus 49,900, giving 50,300 ns.

✗ 50,000 ns

Wrong. That omits the fill time.

✗ 140,000 ns

Wrong. 140,000 ns is the sequential time.

✗ 49,900 ns

Wrong. That is the 499 tasks alone, without the first.

4 clock cycles for the first task and 1 each for the remaining 499, so 4 times 100 plus 499 times 100, that is 400 plus 49,900, giving 50,300 ns.

Pipeline time for 500 tasks is 4 times 100 ns plus _____ times 100 ns, which is 400 plus _____ ns.

BLANK 1

- a. 499
- b. 500
- c. 496
- d. 501

BLANK 2

- a. 49900
- b. 50000
- c. 50300
- d. 40000

Answer: **499, 49900**

499 and 49,900, giving 50,300 ns in all.

Every question, solved and explained

How this is arranged

All **204 questions** from the lecture slides, the reference sheets and both real tests. Grouped by topic, and within each topic ordered **easy first**, so the definitions come before the two-step calculations that assume them.

Every question carries the source it came from. Where the same question was set twice, it appears once and says where else it was asked. The answer is marked, and every other option has a line saying why it fails.

1 Introduction to Python Programming

1.1

EASY

Manual O.1 Q2

A Python script contains a syntax error on its last line. What happens when you run it?

- a. The first lines run, then the error is reported when that line is reached
- b. Nothing runs: the whole file is checked before any of it executes
- c. The error is ignored and the script finishes normally
- d. The line is skipped and the rest of the script continues

✗ The first lines run, then the error is reported when that line is reached

Wrong, and it is the trap. This is true of a NAME error or a type error, which are found only when the line runs, but a SYNTAX error is found while the file is being parsed, before anything executes.

✓ Nothing runs: the whole file is checked before any of it executes

Correct. Python parses the whole file first. A syntax error anywhere stops it before the first line runs, which is why a stray colon can produce no output at all.

✗ The error is ignored and the script finishes normally

Wrong. Python never ignores an error. Something is always reported.

✗ The line is skipped and the rest of the script continues

Wrong. Python does not skip statements it cannot parse. There is no "carry on regardless" mode.

The answer is Nothing runs: the whole file is checked before any of it executes. Syntax errors are found at parse time, before execution. Runtime errors are found when their line runs. Knowing which is which explains why some faulty scripts print nothing at all.

What is the correct way to run a saved file called `grades.py` from the command line?

- a. `python grades.py`
- b. `run grades.py`
- c. `grades.py run`
- d. `execute python grades.py`

✓ `python grades.py`

Correct. You launch the interpreter, `python`, and give it the file to run as its argument.

✗ `run grades.py`

Wrong. There is no `run` command. The interpreter is the program you launch, and the script is what you hand it.

✗ `grades.py run`

Wrong. The order is wrong and `run` is not a Python word.

✗ `execute python grades.py`

Wrong. There is no `execute` command wrapping it.

The answer is `python grades.py`. `python <filename>`. The interpreter is the program; your file is the input to it.

Which line is a valid single-line comment in Python?

- a. `// this is a comment`
- b. `# this is a comment`
- c. `/* this is a comment */`
- d. `-- this is a comment`

X `// this is a comment`

Wrong. Two slashes are the C, Java and JavaScript comment. In Python two slashes are the floor division operator, so this line is an error.

✓ `# this is a comment`

Correct. The hash starts a comment that runs to the end of the line.

X `/* this is a comment */`

Wrong. Slash-star is the C block comment. Python has no block comment syntax at all.

X `-- this is a comment`

Wrong. Two hyphens are the SQL comment, which matters here because this course also writes SQL through Python. The two languages do not share it.

The answer is `# this is a comment`. Hash to end of line. Python has no block comment: a run of hashes, or a string used as a docstring, is how several lines are done.

In Python, what does indentation do?

- a. Nothing: it is style only, as in C or Java
- b. It sets how far output is printed from the left edge
- c. It marks which statements belong to a block, and is part of the syntax
- d. It is required only inside functions

✗ Nothing: it is style only, as in C or Java

Wrong, and it is the single biggest habit to break coming from another language. In Python the indentation IS the block structure, so getting it wrong changes the meaning of the program or stops it running.

✗ It sets how far output is printed from the left edge

Wrong. Output position is decided by what you print, not by how the source is laid out.

✓ It marks which statements belong to a block, and is part of the syntax

Correct. Where C uses braces, Python uses the indentation itself. An inconsistent indent raises `IndentationError`.

✗ It is required only inside functions

Wrong. Every block needs it: `if`, `else`, `for`, `while`, `def`, `class`, `try`, `with`.

The answer is It marks which statements belong to a block, and is part of the syntax. Indentation is syntax, not decoration. It replaces the braces that other languages use to group statements.

Which of these is a **VALID** Python variable name?

- a. 2nd_score
- b. class
- c. total score
- d. _total2

X 2nd_score

Wrong. A name may not start with a digit. Letters and the underscore only, then digits are allowed after the first character.

X class

Wrong. class is a reserved keyword, so it cannot be used as a name. The same applies to if, for, def, and the rest.

X total score

Wrong. A space separates two names, so Python reads this as two things and reports a syntax error. Use an underscore.

✓ _total2

Correct. It starts with an underscore, which is allowed, and the digit is not in first position.

The answer is _total2. Start with a letter or underscore, then letters, digits or underscores, and never a keyword. Names are case sensitive, so Total and total are two different variables.

Asked twice. This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.4.

What does this print?

```
print("A", "B")  
print("A" + "B")
```

a. A B

AB

b. AB

AB

c. A B

A B

d. A,B

AB

✓ A B

AB

Correct. A comma in print puts one space between the items; + joins two strings with nothing between.

✗ AB

AB

Wrong. The comma form adds a space between the two items.

✗ A B

A B

Wrong. + joins the strings end to end with no space.

✗ A,B

AB

Wrong. The comma separates the arguments; it is not printed.

`print('A', 'B')` prints its items separated by a single space. `'A' + 'B'` is one string, `AB`, before `print` ever sees it.

Asked twice. This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.4.

What does this print?

```
t = 15
if t > 20:
    print("hot")
    print("stay inside")
print("done")
```

- a. hot
 stay inside
 done
- b. stay inside
 done
- c. done
- d. hot
 done

X hot
 stay inside
 done

Wrong. The indented lines only run when the condition is True, and $15 > 20$ is False.

X stay inside
 done

Wrong. Both indented lines belong to the if, so they are skipped together.

✓ done

Correct. $t > 20$ is False, so both indented lines are skipped; `print('done')` is back at the margin and always runs.

X hot
done

Wrong. Neither indented line runs; they belong to the same block.

Indentation decides which lines belong to the if. The two indented prints are the block; the unindented print is outside it.

1.8

EASY

Lesson 1.1

What does it mean that Python is INTERPRETED?

- a. An interpreter reads and carries out the code line by line, rather than translating the whole program into machine code first
- b. The whole program must be compiled into machine code before any of it runs
- c. Every variable must be declared with its type before use
- d. Python code can only be run on the computer it was written on

✓ An interpreter reads and carries out the code line by line, rather than translating the whole program into machine code first

Correct. That is the course's definition of an interpreted language.

X The whole program must be compiled into machine code before any of it runs

Wrong. That describes a COMPILED language.

X Every variable must be declared with its type before use

Wrong. Python is dynamically typed; no declarations are needed.

X Python code can only be run on the computer it was written on

Wrong. Being interpreted says nothing of that kind.

Interpreted means the interpreter executes the source line by line. That is why a runtime error on line 40 appears only after lines 1 to 39 have run.

1.9

EASY

Lesson 1.1

Which two of these lines must end with a colon?

- a. `if score >= 50`
- b. `total = total + 1`
- c. `def greet(name)`
- d. `print("done")`

✓ `if score >= 50`

Correct. An `if` opens a block, so its line ends with a colon.

✗ `total = total + 1`

Wrong. An assignment does not open a block.

✓ `def greet(name)`

Correct. A `def` opens a block, so its line ends with a colon.

✗ `print("done")`

Wrong. A `print` call does not open a block.

Every line that opens a block, `if`, `elif`, `else`, `for`, `while`, `def`, `try`, `except`, ends with a colon, and the block under it is indented.

1.10

EASY

Lesson 1.1

True or false: Python is case sensitive, so `Print("hi")` works exactly like `print("hi")`.

Answer: **false**

Case sensitive means `name`, `Name` and `NAME` are three different names. `print` exists; `Print` does not.

What are the three parts of almost every program on this paper, in order?

- a. Input, processing, output
- b. Output, input, processing
- c. Processing, output, input
- d. Compile, link, run

✓ Input, processing, output

Correct. Read something in, work something out, print the answer.

✗ Output, input, processing

Wrong. The output comes last.

✗ Processing, output, input

Wrong. You cannot process before you have the input.

✗ Compile, link, run

Wrong. Python is interpreted; there is no separate compile and link step to write.

Every long question is input, then processing, then output, however elaborate the story around it.

Python is best described as which kind of language?

- a. An interpreted, high-level, general-purpose language
- b. A compiled language, translated to machine code before it runs
- c. A markup language for describing documents
- d. An assembly language for a specific processor family

✓ An interpreted, high-level, general-purpose language

Correct. Interpreted means a program runs it line by line; high-level means it hides the machine; general-purpose means it is not tied to one kind of task.

✗ A compiled language, translated to machine code before it runs

Wrong. Compilation to machine code ahead of running is the C and Java model. Python translates and runs statement by statement through an interpreter, which is why an error can appear only when that line runs.

✗ A markup language for describing documents

Wrong. A markup language describes how content is structured, like HTML. Python computes, it does not mark up.

✗ An assembly language for a specific processor family

Wrong. Assembly is tied to one processor and its instruction set. Python code runs unchanged wherever the interpreter runs.

The answer is An interpreted, high-level, general-purpose language. Interpreted, high-level, general-purpose. Those three words are the definition the paper wants back.

Which of these is NOT a stated advantage of Python?

- a. It is easy to read, with a syntax close to English
- b. It is portable: the same code runs on different platforms
- c. It is the fastest executing language available
- d. It has a large standard library and a large community

✗ It is easy to read, with a syntax close to English

Wrong choice, because readability IS an advantage. The whitespace rules exist to force it.

✗ It is portable: the same code runs on different platforms

Wrong choice, because portability IS an advantage. The interpreter hides the platform.

✓ It is the fastest executing language available

Correct, and this is the one to remember. Interpretation costs speed. Python is chosen for programmer time, not for execution time, and a paper listing slow execution as a disadvantage of Python is common.

✗ It has a large standard library and a large community

Wrong choice, because the standard library IS an advantage. It is the "batteries included" claim.

The answer is It is the fastest executing language available. Speed of execution is a DISadvantage of Python, not an advantage. Everything else on that list is real.

At the interactive prompt you type `2 + 3` and press enter. Why does 5 appear even though you never wrote `print`?

- a. Python adds a `print` automatically to every line of every program
- b. The plus operator prints its result
- c. It does not appear; the prompt shows nothing without `print`
- d. The prompt echoes the value of any expression you type

✗ Python adds a `print` automatically to every line of every program

Wrong, and this is the trap. Inside a `SCRIPT`, `2 + 3` on its own line computes 5 and throws it away silently. Only the prompt echoes.

✗ The plus operator prints its result

Wrong. The plus operator produces a value and does nothing else with it.

✗ It does not appear; the prompt shows nothing without `print`

Wrong. The prompt does echo. Try it: this is the difference between the three kinds of code block this manual sets out.

✓ The prompt echoes the value of any expression you type

Correct. The interactive prompt evaluates what you type and displays the result. That is a feature of the prompt, not of the language.

The answer is The prompt echoes the value of any expression you type. The prompt echoes; a script does not. This is why the manual marks its three kinds of block differently, and why copying prompt examples into a file makes the output vanish.

Asked twice. This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.4.

What does this print?

```
x = 4
y = x * 3
print(x)
x = 10
print(y)
```

a. 12

4

b. 4

12

c. 4

10

d. 4

30

X 12

4

Wrong. Lines run top to bottom, so x is printed before y.

✓ 4

12

Correct. y is worked out as $4 * 3 = 12$ when that line runs; changing x later does not go back and change y.

X 4

10

Wrong. The second print shows y, not the new x.

X 4
30

Wrong. y was calculated when x was 4. Reassigning x afterwards does not recalculate y.

Python runs one line at a time, top to bottom, and each line uses the values that exist at that moment. y became 12 on line 2 and stays 12.

1.16

MEDIUM

Lesson 1.1

The variable name on line 3 is misspelt. What happens when this runs?

```
print("start")
total = 5
print(totl)
print("end")
```

- a. start is printed, then a NameError stops the program
- b. Nothing is printed; the NameError is found before anything runs
- c. start and end are both printed, with a warning
- d. start is printed, then end, and the misspelt line is skipped

✓ start is printed, then a NameError stops the program

Correct. Python runs line by line: line 1 prints, line 3 raises NameError, and the program stops before line 4.

X Nothing is printed; the NameError is found before anything runs

Wrong. A misspelt NAME is only discovered when that line runs, so line 1 has already printed.

X start and end are both printed, with a warning

Wrong. An unhandled error stops the program; line 4 never runs.

X start is printed, then end, and the misspelt line is skipped

Wrong. Python does not skip a failing line; it stops there.

Python interprets as it goes, so everything before the faulty line runs normally. A name error is found only when the line using the name is reached.

2 Python Basics, Syntax, Operators and Strings

2.1

EASY

Manual O.2 Q1

What does it mean to say Python is dynamically typed?

- a. A variable must be declared with its type before use
- b. Types are checked by the compiler before the program runs
- c. Every value in Python is a string until converted
- d. The type belongs to the value, and a name may be rebound to a value of another type

✗ A variable must be declared with its type before use

Wrong, and this is the static-typing model of C and Java. Python needs no declaration at all: assignment creates the name.

✗ Types are checked by the compiler before the program runs

Wrong. There is no compiler pass checking types ahead of time. A type error surfaces when the offending line runs.

✗ Every value in Python is a string until converted

Wrong. This describes what `input()` returns, not what Python does generally. 5 is an int the moment you write it.

✓ The type belongs to the value, and a name may be rebound to a value of another type

Correct. `x = 5` then `x = "five"` is legal, because the type travels with the value, not with the name.

The answer is The type belongs to the value, and a name may be rebound to a value of another type. The value carries the type; the name is only a label. That is why `type()` reports on the value and why a name can change type mid-program.

What is meant by saying that Python strings are immutable?

- a. A string cannot be printed twice
- b. Once created, a string object cannot be changed; operations return a new string
- c. A string cannot be assigned to a variable more than once
- d. A string cannot contain numbers

✗ A string cannot be printed twice

Wrong. Printing does not change anything, and there is no limit on how often you may print.

✓ Once created, a string object cannot be changed; operations return a new string

Correct. `s.upper()` hands back a NEW string and leaves `s` untouched, and `s[0] = "X"` raises `TypeError`. To "change" a string you must rebind the name to a new one.

✗ A string cannot be assigned to a variable more than once

Wrong. Rebinding a name is always allowed. That changes what the name points to, not the string it used to point at.

✗ A string cannot contain numbers

Wrong. `"abc123"` is a perfectly ordinary string. The characters in it are irrelevant to mutability.

The answer is Once created, a string object cannot be changed; operations return a new string. Immutable means the object cannot be modified in place. This is a question the paper asks in words ("Strings are Immutable. Explain and with reasons."), so have the sentence ready and an example with it.

Which error does Python raise for `int("abc")`?

- a. `TypeError`
- b. `ValueError`
- c. `SyntaxError`
- d. `NameError`

X `TypeError`

Wrong. `TypeError` is for the wrong KIND of thing, such as `"5" + 5`. Here the kind is right, a string, and `int()` does accept strings.

✓ `ValueError`

Correct. The type is acceptable but the VALUE is not: `"abc"` is a string that does not spell a number. That is exactly what `ValueError` means.

X `SyntaxError`

Wrong. The line parses perfectly. A syntax error would mean Python could not read the line at all.

X `NameError`

Wrong. `NameError` means a name was used before it was assigned. Every name here exists.

The answer is `ValueError`. `TypeError` is the wrong type, `ValueError` is the right type carrying an impossible value. `int("abc")` is the standard example of the second.

Asked twice. This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.1 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.4.

What does this print?

```
print(int("12") + float("0.5"))
```

- a. 12.5
- b. 12
- c. an error
- d. "12"0.5

✓ 12.5

Correct. `int("12")` is 12 and `float("0.5")` is 0.5; an int plus a float gives a float.

✗ 12

Wrong. Adding 0.5 cannot give a whole number.

✗ an error

Wrong. Both conversions succeed.

✗ "12"0.5

Wrong. Both strings were converted to numbers, so `+` adds rather than joins.

Convert first, then calculate. Mixing an int and a float gives a float.

2.5

EASY

Lesson 1.2

A program reads a person's height in metres, such as 1.75. Which line is correct?

- a. `height = int(input("Height: "))`
- b. `height = input("Height: ")`
- c. `height = str(input("Height: "))`
- d. `height = float(input("Height: "))`

✗ `height = int(input("Height: "))`

Wrong. `int` would make 1.75 into 1, silently wrong.

✗ `height = input("Height: ")`

Wrong. `input` alone gives a string, which cannot be used in arithmetic.

✗ `height = str(input("Height: "))`

Wrong. `input` already returns a string; `str` changes nothing.

✓ `height = float(input("Height: "))`

Correct. A height has a fractional part, so it needs `float`.

| `int` for counts, `float` for measurements, `str` to turn a number back into text.

2.6

EASY

Lesson 1.2

True or false: `input()` returns an `int` when the user types only digits.

Answer: **false**

| The keyboard has no number key that Python believes: `input()` returns text, always.

Asked twice. This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.1 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.4.

What does this print?

```
print(23 // 4, 23 % 4)
```

a. 5.75 3

b. 5 3

c. 5 0.75

d. 6 3

✗ 5.75 3

Wrong. // throws the fraction away, so it is 5, not 5.75.

✓ 5 3

Correct. 23 divided by 4 is 5 remainder 3: // gives 5 and % gives 3.

✗ 5 0.75

Wrong. % gives the remainder as a whole number, 3.

✗ 6 3

Wrong. // does not round up.

| // is floor division, the whole part; % is modulus, the remainder.

Asked twice. This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.1 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.4.

What does this print?

```
print(9 / 3)
```

- a. 3
- b. 3.0
- c. 3.00
- d. an error

✗ 3

Wrong. That would be `9 // 3`.

✓ 3.0

Correct. `/` always returns a float, even when the division is exact.

✗ 3.00

Wrong. Python prints the float as `3.0`.

✗ an error

Wrong. `9 / 3` is perfectly valid.

One slash, one point: `/` always gives a float.

2.9

EASY

Lesson 1.3

Asked twice. This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.1 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.4.

What does this print?

```
n = 38
print(n % 10, n // 10)
```

- a. 3 8
- b. 8 3
- c. 0 38
- d. 3.8 8

✗ 3 8

Wrong. The first value is the remainder, 8.

✓ 8 3

Correct. $38 \% 10$ is the last digit, 8, and $38 // 10$ is 3.

✗ 0 38

Wrong. $38 \% 10$ is 8, not 0.

✗ 3.8 8

Wrong. `//` gives a whole number.

| $n \% 10$ gives the last digit and $n // 10$ drops it: a pair that turns up in digit questions.

2.10

EASY

Lesson 1.3

True or false: = compares two values and == assigns a value.

Answer: false

| = puts a value into a name; == asks whether two values are equal. Using = inside an if is a syntax error.

Which fault stops this program from running at all?

```
total = 0
for n in range(3)
    total += n
print(total)
```

- a. range(3) should be range(1, 3)
- b. total += n should be total = n
- c. The for line has no colon at the end
- d. print(total) should be indented

✗ range(3) should be range(1, 3)

Wrong. range(3) is valid; it gives 0, 1, 2.

✗ total += n should be total = n

Wrong. += is the correct way to accumulate.

✓ The for line has no colon at the end

Correct. A missing colon is a SyntaxError, found before anything runs.

✗ print(total) should be indented

Wrong. The print belongs after the loop, so it is correctly unindented.

! Sweep one is syntax: quotes, colons, commas, brackets. Every block-opening line needs its colon.

Asked twice. This same question also appears as: Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 2.3 and Lesson 2.4 and Lesson 3.2 and Lesson 3.4 and Lesson 3.5.

What happens when this runs?

```
student_name = "Ada"  
print("Hi", Student_name)
```

- a. It prints Hi Ada
- b. `SyntaxError`, because of the capital letter
- c. It prints Hi `Student_name`
- d. `NameError`, because `Student_name` is not the same name as `student_name`

✗ It prints Hi Ada

Wrong. The two names differ in case, so Python treats them as different names.

✗ `SyntaxError`, because of the capital letter

Wrong. A capital letter in a name is legal; the name simply does not exist.

✗ It prints Hi `Student_name`

Wrong. Without quotes it is a name, not text.

✓ `NameError`, because `Student_name` is not the same name as `student_name`

Correct. Python is case sensitive, so `Student_name` was never defined.

Sweep two is names: check every name letter for letter and capital for capital against where it was defined.

What is wrong with this program?

```
x = 5
if x = 5:
    print("five")
```

- a. = should be == in the if, and that is a syntax error
- b. print must not be indented
- c. x must be declared as an int first
- d. The if needs brackets around the condition

✓ = should be == in the if, and that is a syntax error

Correct. = assigns; a condition needs == to compare.

✗ print must not be indented

Wrong. The indented print is the if's block, as it should be.

✗ x must be declared as an int first

Wrong. Python needs no declarations.

✗ The if needs brackets around the condition

Wrong. Brackets are optional in Python.

Using = where == was meant is a planted fault; inside an if it will not even run.

Asked twice. This same question also appears as: Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 2.3 and Lesson 2.4 and Lesson 3.2 and Lesson 3.4 and Lesson 3.5.

What happens when this runs?

```
msg = "Hello'  
print(msg)
```

- a. It prints Hello'
- b. A NameError, because msg is undefined
- c. A SyntaxError, because the string opens with a double quote and closes with a single quote
- d. It prints Hello

X It prints Hello'

Wrong. The single quote does not close a double-quoted string.

X A NameError, because msg is undefined

Wrong. The fault is found before any name is used.

✓ A SyntaxError, because the string opens with a double quote and closes with a single quote

Correct. The string never closes, so Python cannot read the line.

X It prints Hello

Wrong. The line cannot be parsed, so nothing runs.

Line 1 of the paper's broken program had exactly this: "Serve Jesus Now'.

In what order does the checklist sweep a broken listing?

- a. All at once, in a single careful read
- b. Meaning, then names, then syntax
- c. Names, then syntax, then meaning
- d. Syntax, then names, then meaning

✗ All at once, in a single careful read

Wrong. One pass looking for everything finds one or two errors, not all of them.

✗ Meaning, then names, then syntax

Wrong. Syntax comes first, because nothing runs until it is fixed.

✗ Names, then syntax, then meaning

Wrong. Syntax comes before names.

✓ Syntax, then names, then meaning

Correct. First what stops it running, then names checked against their definitions, then indexes, types and logic.

Three sweeps, and count your findings against the marks before you stop.

Asked twice. This same question also appears as: Manual O.2 Q3 and Manual O.2 Q5 and Manual O.2 Q6 and Manual O.2 Q7 and Manual O.2 Q8 and Manual O.2 Q9 and Manual O.2 Q11 and Manual O.2 Q12 and Manual O.2 Q13 and Manual O.2 Q14 and Manual O.2 Q15 and Manual O.2 Q16 and Manual O.2 Q17 and Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.3 Q12 and Manual O.3 Q13 and Manual O.3 Q14 and Manual O.3 Q15 and Manual O.3 Q16 and Manual O.3 Q17 and Manual O.3 Q18 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q5 and Manual O.4 Q9 and Manual O.4 Q12 and Manual O.4 Q14.

What is printed?

```
print(2 + 3 * 4 ** 2)
```

- a. 80
- b. 400
- c. 50
- d. 100

X 80

Wrong. That is $(2 + 3) * 4 ** 2$, which adds before multiplying. Addition is last, not first.

X 400

Wrong. That is $((2 + 3) * 4) ** 2$, which applies the exponent to everything.

✓ 50

Correct. Exponent first: $4 ** 2$ is 16. Then multiply: $3 * 16$ is 48. Then add: $2 + 48$ is 50.

X 100

Wrong. That is $(2 + 3 * 4) ** 2$, exponent applied last.

The answer is 50. Exponent, then multiply and divide, then add and subtract. When in doubt in the exam, write the brackets in yourself: they cost nothing and remove the question.

Asked twice. This same question also appears as: Manual O.2 Q3 and Manual O.2 Q4 and Manual O.2 Q6 and Manual O.2 Q7 and Manual O.2 Q8 and Manual O.2 Q9 and Manual O.2 Q11 and Manual O.2 Q12 and Manual O.2 Q13 and Manual O.2 Q14 and Manual O.2 Q15 and Manual O.2 Q16 and Manual O.2 Q17 and Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.3 Q12 and Manual O.3 Q13 and Manual O.3 Q14 and Manual O.3 Q15 and Manual O.3 Q16 and Manual O.3 Q17 and Manual O.3 Q18 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q5 and Manual O.4 Q9 and Manual O.4 Q12 and Manual O.4 Q14.

What is printed?

```
print(2 ** 3 ** 2)
```

- a. 64
- b. 18
- c. 12
- d. 512

X 64

Wrong. 64 is $(2 ** 3) ** 2$, grouping from the left. The exponent operator is the exception: it groups from the RIGHT.

X 18

Wrong. 18 would be $2 * 3 * 3$, which is not what any part of this expression does.

X 12

Wrong. There is no reading of this expression that gives 12.

✓ 512

Correct. It reads as $2 ** (3 ** 2)$, which is $2 ** 9$, which is 512.

The answer is 512. Exponent is right-associative, and it is the only common operator that is. Everything else in this course groups left to right.

Asked twice. This same question also appears as: Manual O.2 Q3 and Manual O.2 Q4 and Manual O.2 Q5 and Manual O.2 Q7 and Manual O.2 Q8 and Manual O.2 Q9 and Manual O.2 Q11 and Manual O.2 Q12 and Manual O.2 Q13 and Manual O.2 Q14 and Manual O.2 Q15 and Manual O.2 Q16 and Manual O.2 Q17 and Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.3 Q12 and Manual O.3 Q13 and Manual O.3 Q14 and Manual O.3 Q15 and Manual O.3 Q16 and Manual O.3 Q17 and Manual O.3 Q18 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q5 and Manual O.4 Q9 and Manual O.4 Q12 and Manual O.4 Q14.

What is printed?

```
x = 10
x += 5
x -= 3
x *= 2
print(x)
```

a. 24

b. 12

c. 30

d. 17

✓ 24

Correct. 10 plus 5 is 15, minus 3 is 12, times 2 is 24. Each line uses the value the line before it left behind.

✗ 12

Wrong. 12 is the value after the subtraction. The multiplication still has to happen.

✗ 30

Wrong. 30 would be 15 times 2, which skips the subtraction.

✗ 17

Wrong. 17 would be 10 plus 5 plus 2, which misreads two of the three operators.

The answer is 24. $x += 5$ is exactly $x = x + 5$. Work these line by line and write the running value in the margin; the exam asks this shape often.

Asked twice. This same question also appears as: Manual O.2 Q3 and Manual O.2 Q4 and Manual O.2 Q5 and Manual O.2 Q6 and Manual O.2 Q7 and Manual O.2 Q8 and Manual O.2 Q9 and Manual O.2 Q12 and Manual O.2 Q13 and Manual O.2 Q14 and Manual O.2 Q15 and Manual O.2 Q16 and Manual O.2 Q17 and Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.3 Q12 and Manual O.3 Q13 and Manual O.3 Q14 and Manual O.3 Q15 and Manual O.3 Q16 and Manual O.3 Q17 and Manual O.3 Q18 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q5 and Manual O.4 Q9 and Manual O.4 Q12 and Manual O.4 Q14.

What is printed?

```
s = "PYTHON"
print(s[0], s[-1], s[1:4], s[::-1])
```

- a. P N YTHO NOHTYP
- b. Y N YTH NOHTYP
- c. P N YTH NOHTYP
- d. P O YTH NOHTYP

X P N YTHO NOHTYP

Wrong. The slice `s[1:4]` does not include index 4, so it is three characters, not four.

X Y N YTH NOHTYP

Wrong. `s[0]` is P. Python indexes from 0, not from 1.

✓ P N YTH NOHTYP

Correct. Index 0 is the first character, minus 1 is the last, a slice stops BEFORE its end index so `1:4` gives characters 1, 2 and 3, and a step of minus 1 walks the string backwards.

X P O YTH NOHTYP

Wrong. `s[-1]` is the last character, N, not the second to last.

The answer is P N YTH NOHTYP. Index from 0, negative counts from the end, a slice excludes its stop, and `[::-1]` reverses. Those four facts answer most string questions on this paper.

Asked twice. This same question also appears as: Manual O.2 Q3 and Manual O.2 Q4 and Manual O.2 Q5 and Manual O.2 Q6 and Manual O.2 Q7 and Manual O.2 Q8 and Manual O.2 Q9 and Manual O.2 Q11 and Manual O.2 Q12 and Manual O.2 Q13 and Manual O.2 Q15 and Manual O.2 Q16 and Manual O.2 Q17 and Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.3 Q12 and Manual O.3 Q13 and Manual O.3 Q14 and Manual O.3 Q15 and Manual O.3 Q16 and Manual O.3 Q17 and Manual O.3 Q18 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q5 and Manual O.4 Q9 and Manual O.4 Q12 and Manual O.4 Q14.

What is printed?

```
name = "Ada"  
score = 87.456  
print(f"{name} scored {score:.1f}")
```

- a. Ada scored 87.456
- b. Ada scored 87.5
- c. Ada scored 87.4
- d. {name} scored {score:.1f}

✗ Ada scored 87.456

Wrong. The .1f asks for one decimal place, so the full value is not printed.

✓ Ada scored 87.5

Correct. The f before the quote turns it into a formatted string, the braces are replaced by the values, and .1f rounds to one decimal place, giving 87.5.

✗ Ada scored 87.4

Wrong. Formatting ROUNDS, it does not truncate, and 87.456 rounds up to 87.5.

✗ {name} scored {score:.1f}

Wrong. That is what you get WITHOUT the f prefix, which is a real and common slip: the braces then print literally.

The answer is Ada scored 87.5. f"..." substitutes; :.2f fixes the decimal places. Forgetting the f is the classic error and the output tells you at once.

Asked twice. This same question also appears as: Manual O.2 Q3 and Manual O.2 Q4 and Manual O.2 Q5 and Manual O.2 Q6 and Manual O.2 Q7 and Manual O.2 Q8 and Manual O.2 Q9 and Manual O.2 Q11 and Manual O.2 Q12 and Manual O.2 Q13 and Manual O.2 Q14 and Manual O.2 Q15 and Manual O.2 Q16 and Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.3 Q12 and Manual O.3 Q13 and Manual O.3 Q14 and Manual O.3 Q15 and Manual O.3 Q16 and Manual O.3 Q17 and Manual O.3 Q18 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q5 and Manual O.4 Q9 and Manual O.4 Q12 and Manual O.4 Q14.

What is printed?

```
a, b = 1, 2
a, b = b, a
print(a, b)
```

a. 2 1

b. 1 2

c. 2 2

d. 1 1

✓ 2 1

Correct. The right-hand side is evaluated first, into the pair (2, 1), and only then unpacked into a and b. So the swap needs no temporary variable.

✗ 1 2

Wrong. That is the value before the swap.

✗ 2 2

Wrong. That is what happens in a language where `a = b` runs first and destroys the old a. Python does not: it builds the whole right side first.

✗ 1 1

Wrong, for the same reason: nothing is overwritten before it is read.

The answer is 2 1. `a, b = b, a` swaps in one line because the right side is built completely before anything is assigned.

Asked twice. This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.1 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.4.

What does this print?

```
print(type(3.0).__name__, type("3").__name__, type(3 > 2).__name__)
```

- a. float float bool
- b. float str str
- c. int str bool
- d. float str bool

X float float bool

Wrong. "3" is in quotes, so it is a str.

X float str str

Wrong. $3 > 2$ is a comparison, which gives True, a bool.

X int str bool

Wrong. 3.0 is a float, not an int, because of its decimal point.

✓ float str bool

Correct. 3.0 has a decimal point, "3" is in quotes, and a comparison gives a bool.

The decimal point makes a float; quotes make a str; a comparison makes a bool. `type(x).__name__` gives the plain name.

The user types 5 at the prompt, so a holds "5". What does this print?

```
a = "5"      # what input() handed back  
b = 2  
print(a * b)
```

- a. 7
- b. 10
- c. 55
- d. an error, because text cannot be multiplied

✗ 7

Wrong. Nothing here adds 5 and 2.

✗ 10

Wrong. That would need `int(a) * 2`. a is text.

✓ 55

Correct. a is the string "5", and * on a string repeats it: "5" twice is 55.

✗ an error, because text cannot be multiplied

Wrong. A string times a whole number is allowed; it repeats the string.

input() always returns a string. "5" * 2 repeats the text, which is the silent version of the conversion mistake: it runs and gives the wrong answer.

Asked twice. This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.1 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.4.

What does this print?

```
x = 7
print(type(x).__name__)
x = "seven"
print(type(x).__name__)
```

- a. int
str
- b. str
str
- c. int
int
- d. an error, because x changed type

✓ int
str

Correct. The type belongs to the value. x first holds an int, then is rebound to a str.

✗ str
str

Wrong. At the first print x holds the whole number 7.

✗ int
int

Wrong. After the reassignment x holds text.

✗ an error, because x changed type

Wrong. Python is dynamically typed, so a name may be rebound to any type.

Dynamically typed means the type belongs to the value, not the name, so no declaration is needed and a name can be rebound to a different type.

2.26

MEDIUM

Lesson 1.2

Asked twice. This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.1 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.4.

What does this print?

```
n = int("3") * 2
s = str(n) + "0"
print(s, type(s).__name__)
```

- a. 6 str
- b. 60 str
- c. 60 int
- d. 12 str

X 6 str

Wrong. The "0" is joined on the end.

✓ 60 str

Correct. n is $3 * 2 = 6$; `str(6) + "0"` is the text "60", a str.

X 60 int

Wrong. Joining with `+` on strings gives a str, not an int.

X 12 str

Wrong. n is 6, not 12.

`str()` turns a number into text so it can be joined; the result is text, even when it looks like a number.

Asked twice. This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.1 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.4.

What does this print?

```
print(3 + 2 ** 2 * 2)
```

- a. 11
- b. 14
- c. 18
- d. 50

✓ 11

Correct. Power first, $2 ** 2 = 4$; then $4 * 2 = 8$; then $3 + 8 = 11$.

✗ 14

Wrong. That squares $2 * 2$ in the wrong order.

✗ 18

Wrong. That adds $3 + 2$ first.

✗ 50

Wrong. That treats the whole thing as $(3 + 2) ** 2 * 2$.

■ ****** is done first of all, then *****, then **+**.

Asked twice. This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.1 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.4.

What does this print?

```
x = 3
print(x > 1 and x < 3, x > 1 or x > 5, not x == 3)
```

- a. False True False
- b. True False True
- c. True True False
- d. False False True

✓ False True False

Correct. $x < 3$ is False so the and is False; $x > 1$ is True so the or is True; $x == 3$ is True so not gives False.

✗ True False True

Wrong. Check each part: and needs both, or needs one, not flips.

✗ True True False

Wrong. $3 < 3$ is False, so the and is False.

✗ False False True

Wrong. The or needs only one True side, and $x > 1$ is True.

and is True only when both sides are True; or when at least one is; not flips it.

[illegible]

What does this print?

```
total = 4
total *= 3
total -= 2
total /= 2
print(total)
```

- 5
- 10.0
- 5.0
- 4.0

X 5

Wrong. `/=` uses true division, which always gives a float.

X 10.0

Wrong. The last step halves 10.

✓ 5.0

Correct. $4 * 3 = 12$, minus $2 = 10$, then $/ = 2$ gives 5.0 , a float because of the single slash.

X 4.0

Wrong. Work each line in order: 12, 10, 5.0.

total += 5 means total = total + 5, and likewise for the others. /= divides with /, so the result is a float.

Asked twice. This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.1 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.4.

What does this print?

```
s = "banana"
print(s.count("a"), s.find("n"), s.find("z"))
```

- a. 3 3 -1
- b. 2 2 -1
- c. 3 2 None
- d. 3 2 -1

✗ 3 3 -1

Wrong. find gives the FIRST n, at index 2.

✗ 2 2 -1

Wrong. Count them: b-a-n-a-n-a has three a's.

✗ 3 2 None

Wrong. find returns -1, not None, when nothing is found.

✓ 3 2 -1

Correct. banana has three a's; the first n is at index 2; find gives -1 when the text is absent.

count(x) counts occurrences; find(x) gives the index of the first match, or -1.

Asked twice. This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.1 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.4.

What does this print?

```
s = "hello"  
s.upper()  
print(s)
```

- a. None
- b. Hello
- c. HELLO
- d. hello

X None

Wrong. s still holds its original string.

X Hello

Wrong. Nothing here capitalises one letter.

X HELLO

Wrong. The upper-case copy was never assigned to anything.

✓ hello

Correct. upper() returns a new string, which was thrown away; s itself never changes.

Strings are immutable. A string method hands back a NEW string; to keep it you must assign it, s = s.upper().

Asked twice. This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.1 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.4.

What does this print?

```
w = " data "  
print(len(w), len(w.strip()))
```

a. 4 4

b. 8 8

c. 8 4

d. 6 4

X 4 4

Wrong. len counts the spaces too.

X 8 8

Wrong. strip() removes the spaces at both ends.

✓ 8 4

Correct. The string has two spaces, four letters and two spaces: 8 characters; strip() removes the spaces, leaving 4.

X 6 4

Wrong. Count the spaces on both sides: two and two.

len counts every character, spaces included. strip() returns a copy without the whitespace at the ends.

Asked twice. This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.1 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.4.

What does this print?

```
words = "red green blue".split()
print(len(words))
print("-".join(words))
```

- a. 14
red-green-blue
- b. 3
red green blue
- c. 1
red-green-blue
- d. 3
red-green-blue

X 14
red-green-blue

Wrong. `len(words)` counts the list's items, not the characters.

X 3
red green blue

Wrong. `join` puts the separator it is called on, the dash, between the words.

X 1
red-green-blue

Wrong. `split()` gives a list of three separate words.

✓ 3
red-green-blue

Correct. `split()` cuts on spaces into a list of three words, and `"-".join` glues them back with dashes.

`split()` returns a list; `sep.join(list)` returns a string, with `sep` between the items.

2.35

MEDIUM

Lesson 1.4

Asked twice. This same question also appears as: Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 2.3 and Lesson 2.4 and Lesson 3.2 and Lesson 3.4 and Lesson 3.5.

What happens when this runs?

```
print("abc".index("d"))
```

- a. It prints 0
- b. It prints -1
- c. It prints None
- d. It raises `ValueError`, because `index()` raises when the text is absent

✗ It prints 0

Wrong. "d" is not in "abc" at all.

✗ It prints -1

Wrong. That is what `find()` would return.

✗ It prints None

Wrong. Neither method returns None.

✓ It raises `ValueError`, because `index()` raises when the text is absent

Correct. `index()` does the same job as `find()` but raises `ValueError` on a miss.

Use `find()` when a miss is expected: it returns -1. `index()` raises `ValueError` instead.

2.36

MEDIUM

Lesson 1.4

True or false: `s[-0]` gives the last character of `s`.

Answer: **false**

The paper's broken program used `my_string[-0]` for the last character. It is the first; the last is -1.

Asked twice. This same question also appears as: Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 2.3 and Lesson 2.4 and Lesson 3.2 and Lesson 3.4 and Lesson 3.5.

What happens when this runs?

```
person = {"name": "Ada"}  
print(person["Name"])
```

- a. It raises `KeyError: 'Name'`, and the program stops
- b. It prints Name
- c. It prints None
- d. It prints Ada

✓ It raises `KeyError: 'Name'`, and the program stops

Correct. Keys are case sensitive; 'Name' is not a key, so the lookup raises `KeyError`.

✗ It prints Name

Wrong. The lookup fails; nothing is printed.

✗ It prints None

Wrong. `d[key]` raises; only `d.get(key)` returns None.

✗ It prints Ada

Wrong. The key stored is 'name', lower case.

A dictionary key is a string, so it is case sensitive, and a missing key stops the program with `KeyError`.

Which fault is in this line, and what kind of error is it?

```
total = 5  
print("Total:" total)
```

- a. A missing comma between the two items: a `SyntaxError`
- b. A missing colon: a `SyntaxError`
- c. Text cannot be printed with a number: a `TypeError`
- d. total is undefined: a `NameError`

✓ A missing comma between the two items: a `SyntaxError`

Correct. The two things being printed need a comma between them.

✗ A missing colon: a `SyntaxError`

Wrong. A print call does not open a block, so it needs no colon.

✗ Text cannot be printed with a number: a `TypeError`

Wrong. print can show text and numbers together, separated by a comma.

✗ total is undefined: a `NameError`

Wrong. total is defined on the first line; the problem is the missing comma.

Missing commas and colons are the syntax faults the examiner plants most often.

The programmer wanted the LAST character. What does this print?

```
s = "Python"  
last = s[-0]  
print(last)
```

- a. an error
- b. o
- c. n
- d. P

X an error

Wrong. It runs; it just gives the wrong character.

X o

Wrong. o is at index 4.

X n

Wrong. That is what s[-1] would give.

✓ P

Correct. -0 is 0, so s[-0] is the first character, P.

Sweep three is meaning: the program runs and quietly gives the wrong answer. The last character is s[-1].

The user typed 20, so age holds "20". What happens next?

```
age = "20"      # what input() handed back  
print(age + 1)
```

- a. It prints 20 1
- b. TypeError, because text and a number cannot be added
- c. It prints 21
- d. It prints 201

✗ It prints 20 1

Wrong. + does not insert a space; it fails here.

✓ TypeError, because text and a number cannot be added

Correct. "20" + 1 mixes a str and an int, which + refuses.

✗ It prints 21

Wrong. age was never converted with int().

✗ It prints 201

Wrong. That would need str(1); + will not join text to a number.

The fix is `age = int(input("Age: "))`. Unconverted input is the commonest planted type error.

A program does `age = input()` and the user types 20. What is printed?

```
age = "20"          # this is exactly what input() hands back
print(type(age).__name__)
print(age * 2)
```

- a. str
2020
- b. int
40
- c. str
40
- d. int
2020

✓ str
2020

Correct. `input()` always returns a str, so `type` reports str, and `"20" * 2` repeats the characters to give 2020.

✗ int
40

Wrong on both lines. `input()` never returns an int, and star on a string repeats rather than multiplies.

✗ str
40

Wrong on the second line. 40 would need age to be a number; it is a string, so star repeats it.

✗ int
2020

Wrong on the first line. The value came from `input()`, so it is a string however numeric it looks.

The answer is str then 2020. `input()` always returns a string. That is why `age + 5` raises `TypeError` and `age * 2` gives 2020, and it is the single most common first-program bug in this course. Wrap it in `int()` or `float()`.

Asked twice. This same question also appears as: Manual O.2 Q4 and Manual O.2 Q5 and Manual O.2 Q6 and Manual O.2 Q7 and Manual O.2 Q8 and Manual O.2 Q9 and Manual O.2 Q11 and Manual O.2 Q12 and Manual O.2 Q13 and Manual O.2 Q14 and Manual O.2 Q15 and Manual O.2 Q16 and Manual O.2 Q17 and Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.3 Q12 and Manual O.3 Q13 and Manual O.3 Q14 and Manual O.3 Q15 and Manual O.3 Q16 and Manual O.3 Q17 and Manual O.3 Q18 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q5 and Manual O.4 Q9 and Manual O.4 Q12 and Manual O.4 Q14.

What is printed?

```
print(7 / 2)
print(7 // 2)
print(7 % 2)
```

a. 3

3

1

b. 3.5

3

1

c. 3.5

3.5

1

d. 3.5

3

3.5

X 3

3

1

Wrong. A single slash never truncates in Python 3. $7 / 2$ is 3.5, not 3. That was Python 2 behaviour, and the change is a favourite exam point.

✓ 3.5

3

1

Correct. A single slash is true division and always gives a float, even when it divides exactly. Double slash floors the result to an integer. The percent sign gives the remainder.

X 3.5

3.5

1

Wrong. Double slash is floor division, so it produces 3, not 3.5.

X 3.5

3

3.5

Wrong. The percent sign is remainder, not division, so $7 \% 2$ is 1.

The answer is 3.5 then 3 then 1. Slash gives a float, double slash floors, percent is the remainder.

Note $7 // 2$ is 3 but $-7 // 2$ is -4, because flooring goes DOWN, not towards zero.

Asked twice. This same question also appears as: Manual O.2 Q3 and Manual O.2 Q4 and Manual O.2 Q5 and Manual O.2 Q6 and Manual O.2 Q8 and Manual O.2 Q9 and Manual O.2 Q11 and Manual O.2 Q12 and Manual O.2 Q13 and Manual O.2 Q14 and Manual O.2 Q15 and Manual O.2 Q16 and Manual O.2 Q17 and Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.3 Q12 and Manual O.3 Q13 and Manual O.3 Q14 and Manual O.3 Q15 and Manual O.3 Q16 and Manual O.3 Q17 and Manual O.3 Q18 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q5 and Manual O.4 Q9 and Manual O.4 Q12 and Manual O.4 Q14.

What is printed?

```
x = 5
print(1 < x < 10)
print(x == 5 and x != 5)
```

- a. True
True
- b. True
False
- c. False
False
- d. TypeError

X True
True

Wrong. The second line cannot be True: $x == 5$ and $x != 5$ contradict each other by construction.

✓ True
False

Correct. Python allows a chained comparison, and $1 < 5 < 10$ is True. The second line asks for something and its own opposite at once, which can never be True.

X False
False

Wrong. The first line is True, because chaining is legal in Python and 5 does lie between 1 and 10.

X TypeError

Wrong. Nothing here is a type error; all the operands are numbers.

The answer is True then False. Chained comparison is real Python and reads exactly as it does in mathematics. Most languages do not have it, so it is worth a mark when the paper shows one.

Asked twice. This same question also appears as: Manual O.2 Q3 and Manual O.2 Q4 and Manual O.2 Q5 and Manual O.2 Q6 and Manual O.2 Q7 and Manual O.2 Q9 and Manual O.2 Q11 and Manual O.2 Q12 and Manual O.2 Q13 and Manual O.2 Q14 and Manual O.2 Q15 and Manual O.2 Q16 and Manual O.2 Q17 and Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.3 Q12 and Manual O.3 Q13 and Manual O.3 Q14 and Manual O.3 Q15 and Manual O.3 Q16 and Manual O.3 Q17 and Manual O.3 Q18 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q5 and Manual O.4 Q9 and Manual O.4 Q12 and Manual O.4 Q14.

What is printed?

```
print(True and False or True)
print(not True and False)
```

a. False

False

b. True

True

c. True

False

d. False

True

✗ False

False

Wrong. The first line is True, because the or rescues it: only one side of an or needs to be true.

✗ True

True

Wrong. The second line is False. not True is False, and False and anything is False.

✓ True

False

Correct. and binds tighter than or, so the first line is (True and False) or True, which is False or True, which is True. On the second, not binds tightest, so it is (not True) and False, which is False and False.

✗ False

True

Wrong on both counts, and it has the precedence backwards.

The answer is True then False. Precedence among the three: not, then and, then or. Read them in that order and these questions become mechanical.

Asked twice. This same question also appears as: Manual O.2 Q3 and Manual O.2 Q4 and Manual O.2 Q5 and Manual O.2 Q6 and Manual O.2 Q7 and Manual O.2 Q8 and Manual O.2 Q11 and Manual O.2 Q12 and Manual O.2 Q13 and Manual O.2 Q14 and Manual O.2 Q15 and Manual O.2 Q16 and Manual O.2 Q17 and Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.3 Q12 and Manual O.3 Q13 and Manual O.3 Q14 and Manual O.3 Q15 and Manual O.3 Q16 and Manual O.3 Q17 and Manual O.3 Q18 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q5 and Manual O.4 Q9 and Manual O.4 Q12 and Manual O.4 Q14.

What is printed?

```
print(int("7") + int(3.9))  
print(float("2.5") * 2)  
print(str(4) + "2")
```

a. 10.9

5.0

6

b. 11

5.0

42

c. 10

5

42

d. 10

5.0

42

X 10.9

5.0

6

Wrong. `int()` applied to a float does not keep the fraction. It truncates towards zero.

X 11

5.0

42

Wrong. `int(3.9)` is 3, not 4. `int()` truncates, it does not round; `round()` is the function that rounds.

X 10

5

42

Wrong. Multiplying a float by an int gives a float, so it prints 5.0, with the point.

✓ 10
5.0
42

Correct. `int("7")` is 7 and `int(3.9)` TRUNCATES to 3, so the sum is 10. `float("2.5") * 2` is 5.0, a float because one operand is. `str(4) + "2"` joins two strings into "42".

The answer is 10 then 5.0 then 42. `int()` truncates towards zero and `round()` rounds. Mixing an int with a float gives a float. Plus on two strings joins them.

Asked twice. This same question also appears as: Manual O.2 Q3 and Manual O.2 Q4 and Manual O.2 Q5 and Manual O.2 Q6 and Manual O.2 Q7 and Manual O.2 Q8 and Manual O.2 Q9 and Manual O.2 Q11 and Manual O.2 Q13 and Manual O.2 Q14 and Manual O.2 Q15 and Manual O.2 Q16 and Manual O.2 Q17 and Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.3 Q12 and Manual O.3 Q13 and Manual O.3 Q14 and Manual O.3 Q15 and Manual O.3 Q16 and Manual O.3 Q17 and Manual O.3 Q18 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q5 and Manual O.4 Q9 and Manual O.4 Q12 and Manual O.4 Q14.

What is printed?

```
s = "  Hello World  "
print(s.strip().upper())
print(s.strip().split())
print("-".join(["a", "b", "c"]))
```

a. HELLO WORLD

['Hello World']

a-b-c

b. HELLO WORLD

['Hello', 'World']

a-b-c

c. Hello World

['Hello', 'World']

abc

d. HELLO WORLD

['Hello', 'World']

a-b-c

X HELLO WORLD

['Hello World']

a-b-c

Wrong. `split()` with no argument splits on EVERY run of whitespace, so two words come back as two items.

X HELLO WORLD

['Hello', 'World']

a-b-c

Wrong. `strip()` really does remove the outer spaces. That is its only job.

X Hello World

['Hello', 'World']

abc

Wrong on two counts: `upper()` does uppercase, and `join()` really does put the hyphen between the items.

✓ HELLO WORLD

`['Hello', 'World']`

a-b-c

Correct. `strip()` removes the leading and trailing spaces, `upper()` uppercases what is left, `split()` with no argument splits on whitespace into a list of words, and `join()` glues a list together with the string it is called on as the separator.

The answer is HELLO WORLD then `['Hello', 'World']` then a-b-c. `strip`, `upper`, `lower`, `split`, `join`, `replace`, `find`. Those seven cover nearly every string question this course has ever set.

Asked twice. This same question also appears as: Manual O.2 Q3 and Manual O.2 Q4 and Manual O.2 Q5 and Manual O.2 Q6 and Manual O.2 Q7 and Manual O.2 Q8 and Manual O.2 Q9 and Manual O.2 Q11 and Manual O.2 Q12 and Manual O.2 Q14 and Manual O.2 Q15 and Manual O.2 Q16 and Manual O.2 Q17 and Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.3 Q12 and Manual O.3 Q13 and Manual O.3 Q14 and Manual O.3 Q15 and Manual O.3 Q16 and Manual O.3 Q17 and Manual O.3 Q18 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q5 and Manual O.4 Q9 and Manual O.4 Q12 and Manual O.4 Q14.

What is printed?

```
print("ab" * 3)
print("ab" + "3")
```

- a. ababab
ab3
- b. ab3
ababab
- c. abababab
ab3
- d. TypeError
ab3

✓ ababab
ab3

Correct. A string times an integer repeats it. A string plus a string joins them. Both operators are overloaded for strings.

✗ ab3
ababab

Wrong, and it has the two operators swapped. Star repeats, plus joins.

✗ abababab
ab3

Wrong. Times 3 gives three copies, not four.

✗ TypeError
ab3

Wrong. String times int is perfectly legal. It is string PLUS int that raises TypeError.

The answer is ababab then ab3. Star repeats, plus joins, and "ab" + 3 is the error. The operator means different things for numbers and for strings, which is what operator overloading is.

Asked twice. This same question also appears as: Manual O.2 Q3 and Manual O.2 Q4 and Manual O.2 Q5 and Manual O.2 Q6 and Manual O.2 Q7 and Manual O.2 Q8 and Manual O.2 Q9 and Manual O.2 Q11 and Manual O.2 Q12 and Manual O.2 Q13 and Manual O.2 Q14 and Manual O.2 Q16 and Manual O.2 Q17 and Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.3 Q12 and Manual O.3 Q13 and Manual O.3 Q14 and Manual O.3 Q15 and Manual O.3 Q16 and Manual O.3 Q17 and Manual O.3 Q18 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q5 and Manual O.4 Q9 and Manual O.4 Q12 and Manual O.4 Q14.

What is printed?

```
print("a\tb")
print("c\nnd")
print("e\\f")
```

a. a\tb
c\nnd
e\\f

b. ab
cd
ef

c. a b
c
d
e\f

d. a b
cd
ef

X a\tb
c\nnd
e\\f

Wrong. That is what a RAW string would print, written `r"a\tb"`. Without the `r`, the escapes are interpreted.

X ab
cd
ef

Wrong. The escapes produce real whitespace: a tab and a line break, not nothing.

✓ a b
c

```
d
e\f
```

Correct. Backslash t is a tab, backslash n is a newline, and two backslashes print one literal backslash.

```
X a b
  cd
  ef
```

Wrong. Backslash t is a tab character, which is wider than a single space, and the last line keeps its backslash.

The answer is a b then c then d then e\f. Backslash n newline, backslash t tab, double backslash for a real backslash. This matters for file paths, where a Windows path needs doubling or a raw string.

Asked twice. This same question also appears as: Manual O.2 Q3 and Manual O.2 Q4 and Manual O.2 Q5 and Manual O.2 Q6 and Manual O.2 Q7 and Manual O.2 Q8 and Manual O.2 Q9 and Manual O.2 Q11 and Manual O.2 Q12 and Manual O.2 Q13 and Manual O.2 Q14 and Manual O.2 Q15 and Manual O.2 Q17 and Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.3 Q12 and Manual O.3 Q13 and Manual O.3 Q14 and Manual O.3 Q15 and Manual O.3 Q16 and Manual O.3 Q17 and Manual O.3 Q18 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q5 and Manual O.4 Q9 and Manual O.4 Q12 and Manual O.4 Q14.

What is printed?

```
a = [1, 2]
b = [1, 2]
print(a == b)
print(a is b)
```

a. True

True

b. False

False

c. False

True

d. True

False

✗ True

True

Wrong. `is` asks whether they are the SAME object, and two separately written lists are two objects however alike they look.

✗ False

False

Wrong. `==` on two lists compares them item by item, and these match.

✗ False

True

Wrong, and it has the two operators exactly backwards.

✓ True

False

Correct. `==` compares contents, so it is True. `is` compares identity, so it is False: `a` and `b` name two different list objects.

The answer is True then False. == is equal in value, is is the very same object. Use is only for None, and use == for everything else.

HARD

Lesson 1.2

[illegible]

What does this print?

```
print(int(-3.7))
```

- a. 4
b. -3
c. -4
d. -3.7

X 4

Wrong. The sign is kept.

✓ -3

Correct. `int()` chops off the decimal part, towards zero, so -3.7 becomes -3.

X -4

Wrong. `int()` does not round down; it simply drops the fraction.

~~X~~ -3.7

Wrong. `int()` returns a whole number.

`int()` truncates: it removes the fractional part and does not round. `int(7.9)` is 7 and `int(-3.7)` is -3.

Asked twice. This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.1 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.4.

What does this print?

```
print("9" > "10")
```

- a. an error, because strings cannot be compared
- b. 10
- c. True
- d. False

X an error, because strings cannot be compared

Wrong. Strings can be compared; they compare in dictionary order.

X 10

Wrong. A comparison gives a bool.

✓ True

Correct. Text compares character by character, and "9" comes after "1".

X False

Wrong. That would be true of the NUMBERS 9 and 10, but these are strings.

| This is the silent trap: comparing unconverted input compares text, so "9" > "10" is True.

Asked twice. This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.1 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.4.

What does this print?

```
print(10 - 4 / 2 * 3)
```

a. 1.0

b. 4.0

c. 9.0

d. 4

✗ 1.0

Wrong. That multiplies $2 * 3$ before dividing, but $/$ and $*$ go left to right.

✓ 4.0

Correct. $/$ and $*$ first, left to right: $4 / 2 = 2.0$, then $2.0 * 3 = 6.0$, then $10 - 6.0 = 4.0$.

✗ 9.0

Wrong. That does the subtraction first, which is the wrong order.

✗ 4

Wrong. $/$ made a float, so the answer is 4.0.

Precedence: $**$ first, then $*$ $/$ $//$ $\%$, left to right, then $+$ and $-$.

Asked twice. This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.1 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.4.

What does this print?

```
def check():  
    print("checked")  
    return True  
  
print(False and check())  
print(True or check())
```

a. checked

False

checked

True

b. False

checked

True

c. checked

False

True

d. False

True

X checked

False

checked

True

Wrong. Neither call runs, because both expressions short circuit.

X False

checked

True

Wrong. True or ... already knows it is True, so the right side is skipped.

✗ checked

False

True

Wrong. False and ... already knows it is False, so check() is never called.

✓ False

True

Correct. and stops at a False left side and or stops at a True left side, so check() is never called.

and and or short circuit: if the left side settles the answer, the right side is never evaluated.

Asked twice. This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.1 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.4.

What does this print?

```
s = "Python"
print(s[::-1], s[:-2])
```

- a. nohtyP Pyth
- b. Python Pyth
- c. nohtyP thon
- d. nohtyP Pytho

✓ nohtyP Pyth

Correct. `[::-1]` reverses the string, and `[:-2]` stops two characters before the end.

✗ Python Pyth

Wrong. `[::-1]` reverses it.

✗ nohtyP thon

Wrong. `[:-2]` keeps the START and drops the last two.

✗ nohtyP Pytho

Wrong. `[:-2]` drops two characters, not one.

■ A negative step reverses; a negative stop counts from the end and, as always, is excluded.

2.55

HARD

Lesson 1.4

Asked twice. This same question also appears as: Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 2.3 and Lesson 2.4 and Lesson 3.2 and Lesson 3.4 and Lesson 3.5.

What happens when this runs?

```
print("Hello"[5])
```

- a. It raises IndexError, because the last index of a five-letter string is 4
- b. It prints o
- c. It prints H
- d. It prints an empty string

✓ It raises IndexError, because the last index of a five-letter string is 4

Correct. Indexes run 0 to 4, so index 5 is out of range.

✗ It prints o

Wrong. o is at index 4.

✗ It prints H

Wrong. H is at index 0.

✗ It prints an empty string

Wrong. A SLICE past the end gives an empty string; a single INDEX past the end raises.

■ A slice that starts past the end quietly gives "", but a single index past the end is an IndexError.

3 Control Flow, Lists, Tuples, Sets and Dictionaries

3.1

EASY

Manual O.3 Q10

Which statement about tuples is TRUE?

- a. A tuple is ordered and immutable, and can be used as a dictionary key
- b. A tuple can be changed after creation, like a list
- c. A tuple cannot contain more than two items
- d. A tuple is written with square brackets

✓ A tuple is ordered and immutable, and can be used as a dictionary key

Correct. Ordered like a list, immutable unlike one, and because it cannot change it can be hashed, so it may be a dictionary key where a list may not.

✗ A tuple can be changed after creation, like a list

Wrong. That is a list. Assigning to `t[0]` on a tuple raises `TypeError`.

✗ A tuple cannot contain more than two items

Wrong. The name comes from the general mathematical term, not from two. A tuple can hold any number of items, including none.

✗ A tuple is written with square brackets

Wrong. Round brackets, or in fact just commas: `1, 2` is already a tuple. Square brackets make a list.

The answer is A tuple is ordered and immutable, and can be used as a dictionary key. List against tuple is a guaranteed question. Ordered both, mutable only the list, and only the tuple can be a dictionary key.

Asked twice. This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.1 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.4.

What does this print?

```
score = 50
if score >= 70:
    grade = "A"
elif score >= 60:
    grade = "B"
elif score >= 50:
    grade = "C"
elif score >= 45:
    grade = "D"
else:
    grade = "F"
print("Grade:", grade)
```

- a. Grade: D
- b. Grade: F
- c. Grade: C
- d. Grade: B

X Grade: D

Wrong. The chain stops at the first True test, which is ≥ 50 .

X Grade: F

Wrong. else is reached only if every test fails.

✓ Grade: C

Correct. 50 fails ≥ 70 and ≥ 60 , then matches ≥ 50 , and the chain stops.

X Grade: B

Wrong. 50 is not ≥ 60 .

In an if / elif chain exactly one block runs: the first whose condition is True.

3.3

EASY

Lesson 2.1

The same test written with elif. What does this print?

```
x = 15
if x > 10:
    print("big")
elif x > 5:
    print("medium")
else:
    print("small")
```

a. medium

b. big

c. big

medium

d. small

✗ medium

Wrong. $x > 10$ is tested first and is True.

✓ big

Correct. The first True branch runs and the rest of the chain is skipped.

✗ big

medium

Wrong. elif is part of one chain, so only one branch runs.

✗ small

Wrong. else runs only when every test fails.

elif is a queue, if is a crowd: in a chain, only the first True branch runs.

Asked twice. This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.1 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.4.

What does this print?

```
score = 105
if 0 <= score <= 100:
    print("ok")
else:
    print("invalid")
```

- a. ok
invalid
- b. invalid
- c. an error, because comparisons cannot be chained
- d. ok

✗ ok

invalid

Wrong. if and else never both run.

✓ invalid

Correct. $0 \leq 105 \leq 100$ is False because 105 is above 100.

✗ an error, because comparisons cannot be chained

Wrong. Python allows $0 \leq \text{score} \leq 100$.

✗ ok

Wrong. 105 is outside the range.

! A chained comparison is the neatest way to test a range: good when at least 0 AND at most 100.

3.5

EASY

Lesson 2.1

for and while loops belong to which of the three control structures?

- a. Decomposition
- b. Iteration
- c. Selection
- d. Sequence

✗ Decomposition

Wrong. That is not one of the three control structures.

✓ Iteration

Correct. Iteration repeats a block while a condition holds or for each item.

✗ Selection

Wrong. Selection is if, elif and else.

✗ Sequence

Wrong. Sequence is statements one after another.

Sequence, selection, iteration: one after another, choose one, do it again.

3.6

EASY

Lesson 2.1

True or false: an else can carry its own condition, as in `else score < 50`:

Answer: **false**

Writing a condition after else is a planted syntax error.

Asked twice. This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.1 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.4.

What does this print?

```
for i in range(2, 10, 3):  
    print(i, end=" ")
```

a. 3 6 9

b. 2 5 8 11

c. 2 3 4 5 6 7 8 9

d. 2 5 8

✗ 3 6 9

Wrong. The sequence starts at 2.

✗ 2 5 8 11

Wrong. 11 is beyond the stop of 10.

✗ 2 3 4 5 6 7 8 9

Wrong. The third number is the step, 3.

✓ 2 5 8

Correct. Start at 2, step 3, stop before 10: 2, 5, 8.

`range(start, stop, step)` starts at start and stops BEFORE stop. `end=" "` keeps the output on one line.

Asked twice. This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.1 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.4.

What does this print?

```
total = 0
for n in [4, 7, 1]:
    total += n
print(total)
```

- a. 4
- b. 12
- c. 1
- d. 0

X 4

Wrong. The loop adds all three numbers.

✓ 12

Correct. total starts at 0 before the loop and adds 4, 7 and 1.

X 1

Wrong. That is what happens if total is reset inside the loop.

X 0

Wrong. The additions happen inside the loop.

| Zero it before, add inside, print after: the accumulator pattern.

Asked twice. This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.1 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.4.

What does this print?

```
n = 3
while n > 0:
    print(n)
    n -= 1
print("go")
```

a. 3
2
1
go

b. 3
2
1
0
go

c. go

d. 2
1
0
go

✓ 3
2
1
go

Correct. n prints then drops by 1 each pass; when n reaches 0 the test fails and the loop ends.

X 3
2
1
0
go

Wrong. When n is 0 the condition $n > 0$ is False, so 0 is not printed.

X go

Wrong. $n > 0$ is True at the start, so the body runs.

X 2
1
0
go

Wrong. The print comes before the decrease.

A while loop repeats while its condition is True; something in the body must eventually make it False.

What is wrong with this loop?

```
n = 1
while n < 5:
    print(n)
```

- a. n never changes, so $n < 5$ stays True and it loops forever
- b. The print should come after the loop
- c. while cannot compare numbers
- d. range() is missing

✓ n never changes, so $n < 5$ stays True and it loops forever

Correct. Nothing in the body updates n, so the condition never becomes False.

✗ The print should come after the loop

Wrong. Moving the print does not fix the missing update.

✗ while cannot compare numbers

Wrong. while works with any condition.

✗ range() is missing

Wrong. A while loop does not use range.

Forgetting to update the variable in the condition is how an infinite loop is written by accident.

What is a count controlled loop?

- a. A loop that never ends
- b. A loop that counts how many errors occurred
- c. A loop that repeats a fixed, known number of times, decided before the loop begins
- d. A loop that runs until the user types a sentinel

✗ A loop that never ends

Wrong. That is an infinite loop.

✗ A loop that counts how many errors occurred

Wrong. Not the definition.

✓ A loop that repeats a fixed, known number of times, decided before the loop begins

Correct. That is the definition, written in Python with `for` and usually `range()`.

✗ A loop that runs until the user types a sentinel

Wrong. That is a sentinel, or condition, controlled loop.

for when you know how many times; while when you are waiting for something to happen.

Asked twice. This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.1 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.4.

What does this print?

```
d = {"a": 1}
print(d.get("b"), d.get("a"))
```

- a. b 1
- b. 0 1
- c. None 1
- d. KeyError

X b 1

Wrong. `get()` returns the value, not the key.

X 0 1

Wrong. `get()` returns `None`, not `0`, unless told otherwise.

✓ None 1

Correct. `get()` returns `None` for a missing key instead of raising, and the value for a present one.

X KeyError

Wrong. `d["b"]` would raise; `get()` does not.

| `d.get(key)` is the safe lookup: the value, or `None`. `d[key]` raises `KeyError`.

Each student's matric number is paired with that student's name. Which container suits this?

- a. A tuple, because the data is fixed
- b. A list, because it is ordered
- c. A dictionary, because each name is paired with its number and can be looked up by it
- d. A set, because the numbers are unique

✗ A tuple, because the data is fixed

Wrong. A tuple is one fixed record, not a lookup of many pairs.

✗ A list, because it is ordered

Wrong. A list would make you search for each pair.

✓ A dictionary, because each name is paired with its number and can be looked up by it

Correct. Paired data is key to value, which is exactly a dictionary.

✗ A set, because the numbers are unique

Wrong. A set holds single values, not pairs.

List grows, tuple is fixed, set is unique, dictionary pairs. Give the reason: the reason carries the mark.

Asked twice. This same question also appears as: Manual O.2 Q3 and Manual O.2 Q4 and Manual O.2 Q5 and Manual O.2 Q6 and Manual O.2 Q7 and Manual O.2 Q8 and Manual O.2 Q9 and Manual O.2 Q11 and Manual O.2 Q12 and Manual O.2 Q13 and Manual O.2 Q14 and Manual O.2 Q15 and Manual O.2 Q16 and Manual O.2 Q17 and Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.3 Q12 and Manual O.3 Q13 and Manual O.3 Q14 and Manual O.3 Q15 and Manual O.3 Q16 and Manual O.3 Q17 and Manual O.3 Q18 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q5 and Manual O.4 Q9 and Manual O.4 Q12 and Manual O.4 Q14.

What is printed?

```
xs = [3, 1, 2]
ys = xs.sort()
print(ys)
```

- a. None
- b. [1, 2, 3]
- c. [3, 1, 2]
- d. TypeError

✓ None

Correct. `sort()` returns `None`. If you want a new sorted list without touching the original, use `sorted(xs)`.

✗ [1, 2, 3]

Wrong, and this is the trap. `sort()` changes `xs` and hands back nothing, so the sorted list is in `xs`, not in `ys`.

✗ [3, 1, 2]

Wrong. `xs` itself IS sorted now; it is `ys` that is empty of meaning.

✗ TypeError

Wrong. Nothing raises here. It runs and quietly gives you `None`, which is what makes the bug hard to see.

The answer is `None`. Methods that change a list in place return `None`: `sort`, `append`, `reverse`, `extend`. The functions that return a NEW value are `sorted()` and `reversed()`.

Asked twice. This same question also appears as: Manual O.2 Q3 and Manual O.2 Q4 and Manual O.2 Q5 and Manual O.2 Q6 and Manual O.2 Q7 and Manual O.2 Q8 and Manual O.2 Q9 and Manual O.2 Q11 and Manual O.2 Q12 and Manual O.2 Q13 and Manual O.2 Q14 and Manual O.2 Q15 and Manual O.2 Q16 and Manual O.2 Q17 and Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.3 Q12 and Manual O.3 Q13 and Manual O.3 Q15 and Manual O.3 Q16 and Manual O.3 Q17 and Manual O.3 Q18 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q5 and Manual O.4 Q9 and Manual O.4 Q12 and Manual O.4 Q14.

What is printed?

```
months = {"Jan": 1, "Feb": 2}
try:
    print(months[2])
except KeyError as e:
    print("KeyError", e)
```

- a. KeyError 2
- b. 2
- c. Feb
- d. None

✓ **KeyError 2**

Correct. The subscript is a KEY lookup, the key 2 is absent, KeyError is raised and caught, and the missing key is what the exception carries.

✗ **2**

Wrong. Looking up 2 asks for a KEY equal to 2, and there is no such key. The values are 1 and 2, but you cannot look a dictionary up by its value.

✗ **Feb**

Wrong. Nothing here searches the values to find the key that maps to 2. That would need a loop or a reversed dictionary.

✗ **None**

Wrong. A missing key raises rather than returning None. It is `get()` that returns None instead of raising.

The answer is **KeyError 2**. `months[2]` asks for the KEY 2. Use `months.get(2)` to get None instead of a KeyError. This exact confusion is on a past paper.

Asked twice. This same question also appears as: Manual O.2 Q3 and Manual O.2 Q4 and Manual O.2 Q5 and Manual O.2 Q6 and Manual O.2 Q7 and Manual O.2 Q8 and Manual O.2 Q9 and Manual O.2 Q11 and Manual O.2 Q12 and Manual O.2 Q13 and Manual O.2 Q14 and Manual O.2 Q15 and Manual O.2 Q16 and Manual O.2 Q17 and Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.3 Q12 and Manual O.3 Q13 and Manual O.3 Q14 and Manual O.3 Q15 and Manual O.3 Q17 and Manual O.3 Q18 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q5 and Manual O.4 Q9 and Manual O.4 Q12 and Manual O.4 Q14.

What is printed?

```
a = [1, 2, 3]
b = a
b.append(4)
print(a)
```

- a. [1, 2, 3]
- b. [4]
- c. [1, 2, 3, 4]
- d. [[1, 2, 3], 4]

X [1, 2, 3]

Wrong. `b = a` does not copy anything. It gives the same list a second name, so a change through one name is visible through the other.

X [4]

Wrong. `append` adds to the existing contents; it does not replace them.

✓ [1, 2, 3, 4]

Correct. There is only ONE list here with two names, so appending through `b` shows up in `a`.

X [[1, 2, 3], 4]

Wrong. `append` adds an item to the list, it does not nest the old list inside a new one.

The answer is [1, 2, 3, 4]. Assignment binds a name, it does not copy. To copy, use `a[:]` or `list(a)` or `a.copy()`. This is the most expensive misunderstanding in the whole course.

The chain below is in the wrong order. What does it print?

```
score = 80
if score >= 40:
    grade = "D"
elif score >= 50:
    grade = "C"
elif score >= 70:
    grade = "A"
print("Grade:", grade)
```

- a. Grade: D
- b. Grade: C
- c. Grade: A
- d. Grade: A then Grade: D

✓ Grade: D

Correct. $80 \geq 40$ is True at the first test, so the chain stops there and gives D.

✗ Grade: C

Wrong. The chain stopped at the first test.

✗ Grade: A

Wrong. The A test is never reached; the first True test wins.

✗ Grade: A then Grade: D

Wrong. An elif chain runs at most one block.

Order a chain from the most demanding test to the least, or a high score is caught by a low band.

Asked twice. This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.1 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.4.

What does this print?

```
x = 15
if x > 10:
    print("big")
if x > 5:
    print("medium")
else:
    print("small")
```

- a. big
medium
small
- b. medium
- c. big
- d. big
medium

X big
medium
small

Wrong. The else belongs to the second if, which was True.

X medium

Wrong. $x > 10$ is also True, so big prints first.

X big

Wrong. That would be true with elif; separate ifs are each tested.

✓ big
medium

Correct. These are two SEPARATE if statements, so both conditions are tested and both are True.

Separate ifs are all tested, so more than one can run. An elif chain runs at most one.

Asked twice. This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.1 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.4.

What does this print?

```
age = 16
member = True
if age >= 18:
    if member:
        print("Full")
    else:
        print("Guest")
else:
    print("Too young")
```

- a. Guest
- b. Too young
- c. Full
Too young
- d. Full

X Guest

Wrong. The inner else belongs to the inner if, which is never reached.

✓ Too young

Correct. `age >= 18` is False, so the outer else runs and the inner if is never reached.

X Full

Too young

Wrong. Exactly one outer branch runs.

X Full

Wrong. The inner if is only reached when age is at least 18.

An else belongs to the if at the same indentation. The inner decision only happens inside the outer True branch.

3.20

MEDIUM

Lesson 2.1

Which condition correctly catches a score that is NOT between 0 and 100?

- a. $0 \leq \text{score} \leq 100$
- b. $\text{score} > 0$ or $\text{score} < 100$
- c. $\text{score} < 0$ and $\text{score} > 100$
- d. $\text{score} < 0$ or $\text{score} > 100$

✗ $0 \leq \text{score} \leq 100$

Wrong. That tests for a GOOD score.

✗ $\text{score} > 0$ or $\text{score} < 100$

Wrong. This is True for almost every number, good or bad.

✗ $\text{score} < 0$ and $\text{score} > 100$

Wrong. No number is below 0 and above 100 at once, so this is never True.

✓ $\text{score} < 0$ or $\text{score} > 100$

Correct. A score is bad if it is below 0 OR above 100.

To catch a bad value use or; to confirm a good value use and.

Asked twice. This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.1 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.4.

What does this print?

```
print(list(range(5, 0, -2)))
```

- a. [5, 3, 1, -1]
- b. [5, 4, 3, 2, 1]
- c. []
- d. [5, 3, 1]

✗ [5, 3, 1, -1]

Wrong. The stop, 0, is excluded, and -1 is past it.

✗ [5, 4, 3, 2, 1]

Wrong. The step is -2, not -1.

✗ []

Wrong. A negative step makes a descending range work.

✓ [5, 3, 1]

Correct. A negative step counts down, stopping before 0.

■ A negative step counts down, and the stop is still excluded.

total is reset **INSIDE** the loop. What does this print?

```
for n in [4, 7, 1]:  
    total = 0  
    total += n  
print(total)
```

- a. 12
- b. 1
- c. an error
- d. 0

✗ 12

Wrong. The reset throws away the earlier additions.

✓ 1

Correct. Each pass sets total back to 0 and adds one number, so only the last, 1, survives.

✗ an error

Wrong. It runs; it just gives the wrong answer.

✗ 0

Wrong. The last pass adds 1 after resetting.

Initialising the total inside the loop resets it every pass, so the answer is always the last number.

Asked twice. This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.1 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.4.

What does this print?

```
count = 0
for i in range(3):
    for j in range(4):
        count += 1
print(count)
```

- a. 7
- b. 3
- c. 4
- d. 12

✗ 7

Wrong. The counts multiply; they do not add.

✗ 3

Wrong. That counts only the outer passes.

✗ 4

Wrong. That counts only one pass of the inner loop.

✓ 12

Correct. The inner loop runs 4 times for each of the 3 outer passes: $3 \times 4 = 12$.

The inner loop runs completely for every pass of the outer one, so multiply the counts.

Asked twice. This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.1 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.4.

What does this print?

```
xs = [5, 3, 8]
xs.append(1)
xs.insert(1, 9)
print(xs)
```

- a. [5, 9, 3, 8]
- b. [5, 9, 3, 8, 1]
- c. [9, 5, 3, 8, 1]
- d. [5, 3, 8, 1, 9]

✗ [5, 9, 3, 8]

Wrong. append added 1 at the end.

✓ [5, 9, 3, 8, 1]

Correct. append puts 1 at the end, then insert(1, 9) puts 9 at index 1.

✗ [9, 5, 3, 8, 1]

Wrong. insert(1, ...) places the item at index 1, the second position.

✗ [5, 3, 8, 1, 9]

Wrong. insert places 9 at index 1, not at the end.

append(x) adds at the end; insert(i, x) adds at position i.

Asked twice. This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.1 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.4.

What does this print?

```
xs = [4, 2, 4, 1]
xs.remove(4)
print(xs)
```

- a. [2, 4, 1]
- b. [4, 2, 4]
- c. [4, 2, 1]
- d. [2, 1]

✓ [2, 4, 1]

Correct. remove deletes the FIRST item equal to 4 and leaves the second.

✗ [4, 2, 4]

Wrong. remove takes a value, not an index.

✗ [4, 2, 1]

Wrong. The first 4 is the one removed.

✗ [2, 1]

Wrong. remove deletes only one matching item.

remove(x) deletes the first item equal to x, and raises ValueError if there is none.

Asked twice. This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.1 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.4.

What does this print?

```
xs = [3, 1, 2]
ys = sorted(xs)
print(xs, ys)
```

- a. [1, 2, 3] [1, 2, 3]
- b. [3, 1, 2] [1, 2, 3]
- c. [1, 2, 3] None
- d. [3, 1, 2] None

✗ [1, 2, 3] [1, 2, 3]

Wrong. sorted() does not change xs.

✓ [3, 1, 2] [1, 2, 3]

Correct. sorted() returns a NEW sorted list and leaves xs alone.

✗ [1, 2, 3] None

Wrong. xs is unchanged and ys holds the sorted copy.

✗ [3, 1, 2] None

Wrong. sorted() returns a list; it is .sort() that returns None.

sorted(xs) gives a new list; xs.sort() sorts in place and returns None.

Asked twice. This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.1 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.4.

What does this print?

```
xs = [10, 20, 30]
print(xs.pop(), xs)
```

- a. 30 [10, 20, 30]
- b. 10 [20, 30]
- c. 30 [10, 20]
- d. None [10, 20]

✗ 30 [10, 20, 30]

Wrong. pop removes the item as well as returning it.

✗ 10 [20, 30]

Wrong. With no index, pop takes the LAST item.

✓ 30 [10, 20]

Correct. pop() with no argument removes and returns the last item.

✗ None [10, 20]

Wrong. Unlike append and sort, pop returns the item it removed.

pop(i) removes and returns; with no index it takes the last item.

Asked twice. This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.1 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.4.

What does this print?

```
print([0] * 3 + [1])
```

- a. [3, 1]
- b. [0, 0, 0, 1]
- c. [0, 0, 0, [1]]
- d. [0, 1, 0, 1, 0, 1]

✗ [3, 1]

Wrong. * repeats the list; it does not multiply the item.

✓ [0, 0, 0, 1]

Correct. [0] * 3 repeats to [0, 0, 0], and + joins [1] on the end.

✗ [0, 0, 0, [1]]

Wrong. + joins the items of [1], not the list itself.

✗ [0, 1, 0, 1, 0, 1]

Wrong. Only [0] is repeated, before [1] is joined.

| * on a list repeats it; + joins two lists into one.

Asked twice. This same question also appears as: Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 2.4 and Lesson 3.2 and Lesson 3.4 and Lesson 3.5.

What happens when this runs?

```
t = (1, 2, 3)
t[1] = 9
print(t)
```

- a. It raises `TypeError`, because a tuple cannot be changed
- b. It prints (1, 9, 3)
- c. It raises `IndexError`
- d. It prints (1, 2, 3)

✓ It raises `TypeError`, because a tuple cannot be changed

Correct. A tuple is immutable, so item assignment is refused.

✗ It prints (1, 9, 3)

Wrong. The assignment is refused.

✗ It raises `IndexError`

Wrong. Index 1 exists; the problem is changing it.

✗ It prints (1, 2, 3)

Wrong. The error stops the program before the print.

Round brackets are fixed: assigning to a tuple item raises `TypeError`, as with a string.

Asked twice. This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.1 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.4.

What does this print?

```
print(len(set("hello")))
```

- a. 3
- b. an error, because a string cannot become a set
- c. 5
- d. 4

✗ 3

Wrong. Count the distinct letters: h, e, l, o.

✗ an error, because a string cannot become a set

Wrong. set() accepts any sequence, including a string.

✗ 5

Wrong. Duplicates vanish, so the two l's count once.

✓ 4

Correct. The set of "hello" keeps h, e, l and o, with the repeated l stored once.

A set holds at most one copy of any value, so len() counts distinct items.

Asked twice. This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.1 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.4.

What does this print?

```
d = {"x": 5, "y": 7}
d["x"] = 6
d["z"] = 1
print(len(d), d["x"])
```

a. 3 6

b. 4 6

c. 2 6

d. 3 5

✓ 3 6

Correct. Assigning to an existing key replaces its value; assigning to a new key adds a pair.

✗ 4 6

Wrong. x was replaced, not duplicated; keys are unique.

✗ 2 6

Wrong. z is a new key, so there are three pairs.

✗ 3 5

Wrong. d["x"] = 6 replaced the 5.

| d[key] = value adds a new pair or changes an existing one. Keys are unique.

Asked twice. This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.1 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.4.

What does this print?

```
marks = {"Ada": 58, "Ben": 71, "Cy": 60}
for name, m in marks.items():
    if m >= 60:
        print(name)
```

a. Ada

Ben

Cy

b. Cy

Ben

c. Ben

d. Ben

Cy

✗ Ada

Ben

Cy

Wrong. Ada's 58 fails the test.

✗ Cy

Ben

Wrong. A dictionary keeps insertion order, Ben before Cy.

✗ Ben

Wrong. $60 \geq 60$ is True, so Cy prints too.

✓ Ben

Cy

Correct. `items()` walks each pair in insertion order, and only marks of 60 and above print.

Walk the pairs, test the value, print the ones that pass: the shape of every 'display those which' requirement.

Asked twice. This same question also appears as: Manual O.2 Q3 and Manual O.2 Q4 and Manual O.2 Q5 and Manual O.2 Q6 and Manual O.2 Q7 and Manual O.2 Q8 and Manual O.2 Q9 and Manual O.2 Q11 and Manual O.2 Q12 and Manual O.2 Q13 and Manual O.2 Q14 and Manual O.2 Q15 and Manual O.2 Q16 and Manual O.2 Q17 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.3 Q12 and Manual O.3 Q13 and Manual O.3 Q14 and Manual O.3 Q15 and Manual O.3 Q16 and Manual O.3 Q17 and Manual O.3 Q18 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q5 and Manual O.4 Q9 and Manual O.4 Q12 and Manual O.4 Q14.

What is printed?

```
score = 75
if score >= 70:
    print("A")
elif score >= 60:
    print("B")
else:
    print("C")
```

- a. B
- b. A
- B
- c. A
- d. C

X B

Wrong. 75 is at or above 60, but the chain never reaches that test: it stopped at the first true one.

X A

B

Wrong. An if/elif chain runs AT MOST one block. Separate if statements would print both, which is exactly the trap the paper sets.

✓ A

Correct. The first condition that is true wins, its block runs, and the whole chain is then finished.

X C

Wrong. else runs only when every test above it failed.

The answer is A. elif tests only if everything above it failed. Replacing elif with a second if changes the answer, and the paper has asked for exactly that comparison.

Asked twice. This same question also appears as: Manual O.2 Q3 and Manual O.2 Q4 and Manual O.2 Q5 and Manual O.2 Q6 and Manual O.2 Q7 and Manual O.2 Q8 and Manual O.2 Q9 and Manual O.2 Q11 and Manual O.2 Q12 and Manual O.2 Q13 and Manual O.2 Q14 and Manual O.2 Q15 and Manual O.2 Q16 and Manual O.2 Q17 and Manual O.3 Q1 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.3 Q12 and Manual O.3 Q13 and Manual O.3 Q14 and Manual O.3 Q15 and Manual O.3 Q16 and Manual O.3 Q17 and Manual O.3 Q18 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q5 and Manual O.4 Q9 and Manual O.4 Q12 and Manual O.4 Q14.

What is printed?

```
for i in range(1, 6, 2):  
    print(i, end=" ")
```

a. 1 2 3 4 5

b. 1 3 5 7

c. 2 4

d. 1 3 5

✗ 1 2 3 4 5

Wrong. That ignores the third argument, the step of 2.

✗ 1 3 5 7

Wrong. 7 is past the stop value, so it is never produced.

✗ 2 4

Wrong. The start is 1, not 2. range starts where you tell it to.

✓ 1 3 5

Correct. range(start, stop, step) begins at 1, adds 2 each time, and stops BEFORE 6. So 1, 3, 5.

The answer is 1 3 5. range(stop), range(start, stop), range(start, stop, step), and the stop is never included. That last part is the one people lose marks on.

Asked twice. This same question also appears as: Manual O.2 Q3 and Manual O.2 Q4 and Manual O.2 Q5 and Manual O.2 Q6 and Manual O.2 Q7 and Manual O.2 Q8 and Manual O.2 Q9 and Manual O.2 Q11 and Manual O.2 Q12 and Manual O.2 Q13 and Manual O.2 Q14 and Manual O.2 Q15 and Manual O.2 Q16 and Manual O.2 Q17 and Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.3 Q12 and Manual O.3 Q13 and Manual O.3 Q14 and Manual O.3 Q15 and Manual O.3 Q16 and Manual O.3 Q17 and Manual O.3 Q18 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q5 and Manual O.4 Q9 and Manual O.4 Q12 and Manual O.4 Q14.

What is printed?

```
n = 0
while True:
    n += 1
    if n == 3:
        break
print(n)
```

- a. 3
- b. 0
- c. 2
- d. the loop never ends

✓ 3

Correct. n reaches 3, the if fires, break leaves the loop immediately, and print sees 3.

✗ 0

Wrong. n is incremented before anything else in the body, so it can never still be 0 at the end.

✗ 2

Wrong. break happens AFTER the increment that made n equal to 3, so 3 is the value that survives.

✗ the loop never ends

Wrong. while True does loop forever unless something leaves it, and break is exactly that something.

The answer is 3. break leaves the loop at once; continue jumps to the next iteration. while True with a break inside is the standard way to write a menu loop.

Asked twice. This same question also appears as: Manual O.2 Q3 and Manual O.2 Q4 and Manual O.2 Q5 and Manual O.2 Q6 and Manual O.2 Q7 and Manual O.2 Q8 and Manual O.2 Q9 and Manual O.2 Q11 and Manual O.2 Q12 and Manual O.2 Q13 and Manual O.2 Q14 and Manual O.2 Q15 and Manual O.2 Q16 and Manual O.2 Q17 and Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.3 Q12 and Manual O.3 Q13 and Manual O.3 Q14 and Manual O.3 Q15 and Manual O.3 Q16 and Manual O.3 Q17 and Manual O.3 Q18 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q5 and Manual O.4 Q9 and Manual O.4 Q12 and Manual O.4 Q14.

What is printed?

```
for i in range(5):  
    if i % 2 == 0:  
        continue  
    print(i, end=" ")
```

a. 0 2 4

b. 1 3

c. 0 1 2 3 4

d. 1 3 5

X 0 2 4

Wrong, and it is backwards: continue SKIPS the even numbers, so they are the ones that never print.

✓ 1 3

Correct. When i is even the continue skips the print, so only the odd values 1 and 3 appear. range(5) stops at 4.

X 0 1 2 3 4

Wrong. The continue means not every value reaches the print.

X 1 3 5

Wrong. range(5) produces 0 to 4, so 5 is never reached.

The answer is 1 3. continue abandons this pass and starts the next. Reading it as "skip the rest of the body" is the reliable way to trace it.

Asked twice. This same question also appears as: Manual O.2 Q3 and Manual O.2 Q4 and Manual O.2 Q5 and Manual O.2 Q6 and Manual O.2 Q7 and Manual O.2 Q8 and Manual O.2 Q9 and Manual O.2 Q11 and Manual O.2 Q12 and Manual O.2 Q13 and Manual O.2 Q14 and Manual O.2 Q15 and Manual O.2 Q16 and Manual O.2 Q17 and Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.3 Q12 and Manual O.3 Q13 and Manual O.3 Q14 and Manual O.3 Q15 and Manual O.3 Q16 and Manual O.3 Q17 and Manual O.3 Q18 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q5 and Manual O.4 Q9 and Manual O.4 Q12 and Manual O.4 Q14.

What is printed?

```
for i in range(3):  
    print(i, end=" ")  
else:  
    print("done")
```

- a. 0 1 2
- b. done
- c. 0 1 2 done
- d. 0 1 2 3 done

X 0 1 2

Wrong. The else does run, because there is no break in this loop.

X done

Wrong. The loop body runs first, three times.

✓ 0 1 2 done

Correct. A loop else runs when the loop finished normally, that is without a break. Nothing breaks here, so it runs.

X 0 1 2 3 done

Wrong. range(3) gives 0, 1 and 2. It never reaches 3.

The answer is 0 1 2 done. A loop else means "no break happened". It is Python-specific and a favourite for a "what is the output" question because it looks like an if else and is not.

Asked twice. This same question also appears as: Manual O.2 Q3 and Manual O.2 Q4 and Manual O.2 Q5 and Manual O.2 Q6 and Manual O.2 Q7 and Manual O.2 Q8 and Manual O.2 Q9 and Manual O.2 Q11 and Manual O.2 Q12 and Manual O.2 Q13 and Manual O.2 Q14 and Manual O.2 Q15 and Manual O.2 Q16 and Manual O.2 Q17 and Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.3 Q12 and Manual O.3 Q13 and Manual O.3 Q14 and Manual O.3 Q15 and Manual O.3 Q16 and Manual O.3 Q17 and Manual O.3 Q18 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q5 and Manual O.4 Q9 and Manual O.4 Q12 and Manual O.4 Q14.

What is printed?

```
xs = [3, 1, 2]
xs.append(4)
xs.sort()
print(xs)
print(xs[0], xs[-1])
```

a. [3, 1, 2, 4]

3 4

b. [4, 3, 2, 1]

4 1

c. [1, 2, 3]

1 3

d. [1, 2, 3, 4]

1 4

✗ [3, 1, 2, 4]

3 4

Wrong. sort() really does reorder the list. It changes it and returns None.

✗ [4, 3, 2, 1]

4 1

Wrong. sort() ascends by default. Descending needs reverse=True.

✗ [1, 2, 3]

1 3

Wrong. The 4 that append added is still there. append modifies the list itself.

✓ [1, 2, 3, 4]

1 4

Correct. append adds to the end, sort orders the list IN PLACE, and the first and last items are then 1 and 4.

The answer is [1, 2, 3, 4] then 1 4. Lists are ORDERED, so position has meaning, and MUTABLE, so they can be changed in place. The paper asks for both properties by name.

Asked twice. This same question also appears as: Manual O.2 Q3 and Manual O.2 Q4 and Manual O.2 Q5 and Manual O.2 Q6 and Manual O.2 Q7 and Manual O.2 Q8 and Manual O.2 Q9 and Manual O.2 Q11 and Manual O.2 Q12 and Manual O.2 Q13 and Manual O.2 Q14 and Manual O.2 Q15 and Manual O.2 Q16 and Manual O.2 Q17 and Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.3 Q12 and Manual O.3 Q13 and Manual O.3 Q14 and Manual O.3 Q15 and Manual O.3 Q16 and Manual O.3 Q17 and Manual O.3 Q18 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q5 and Manual O.4 Q9 and Manual O.4 Q12 and Manual O.4 Q14.

What is printed?

```
numbers = [1, 2, 3, 4, 5]
print(numbers[-3:])
print(numbers[:2])
print(numbers[-3:] * 2)
```

a. [2, 3, 4]

[1, 2]

[6, 8, 10]

b. [3, 4, 5]

[1, 2]

[3, 4, 5, 3, 4, 5]

c. [3, 4, 5]

[1, 2, 3]

[6, 8, 10]

d. [3, 4, 5]

[1, 2]

[6, 8, 10]

✗ [2, 3, 4]

[1, 2]

[6, 8, 10]

Wrong. numbers[-3:] counts three back from the end, giving 3, 4 and 5.

✓ [3, 4, 5]

[1, 2]

[3, 4, 5, 3, 4, 5]

Correct. Minus 3 to the end is the last three items. Up to 2 stops before index 2, so two items. A list times 2 REPEATS the list, it does not multiply the numbers in it.

✗ [3, 4, 5]

[1, 2, 3]

[6, 8, 10]

Wrong. `numbers[:2]` excludes index 2, so it is two items, not three.

✗ [3, 4, 5]

[1, 2]

[6, 8, 10]

Wrong on the last line. Star on a list repeats the list; to multiply each number you would need a loop or a comprehension.

The answer is [3, 4, 5] then [1, 2] then [3, 4, 5, 3, 4, 5]. This exact shape, `numbers[-3:] * 2`, is on a past paper. A slice excludes its stop, and star repeats rather than scales.

3.41

HARD

Manual O.3 Q9

Asked twice. This same question also appears as: Manual O.2 Q3 and Manual O.2 Q4 and Manual O.2 Q5 and Manual O.2 Q6 and Manual O.2 Q7 and Manual O.2 Q8 and Manual O.2 Q9 and Manual O.2 Q11 and Manual O.2 Q12 and Manual O.2 Q13 and Manual O.2 Q14 and Manual O.2 Q15 and Manual O.2 Q16 and Manual O.2 Q17 and Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q11 and Manual O.3 Q12 and Manual O.3 Q13 and Manual O.3 Q14 and Manual O.3 Q15 and Manual O.3 Q16 and Manual O.3 Q17 and Manual O.3 Q18 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q5 and Manual O.4 Q9 and Manual O.4 Q12 and Manual O.4 Q14.

What is printed?

```
m = [[1, 2], [3, 4, 5], [6]]
print(m[1][2])
print(len(m))
```

a. 4

3

b. 5

6

c. 5

3

d. 3

3

X 4

3

Wrong. Index 2 of [3, 4, 5] is 5. Index 1 would be 4.

X 5

6

Wrong. len() of a nested list counts the sublists, not the numbers inside them.

✓ 5

3

Correct. m[1] is the middle list, [3, 4, 5], and its index 2 is 5. len(m) counts the OUTER items, of which there are three.

X 3

3

Wrong. m[1][2] reaches into the second list and takes its third item.

The answer is 5 then 3. Read a nested index left to right: first pick the sublist, then index into it.
The paper prints a three-deep version of this, so do it one bracket at a time on paper.

Asked twice. This same question also appears as: Manual O.2 Q3 and Manual O.2 Q4 and Manual O.2 Q5 and Manual O.2 Q6 and Manual O.2 Q7 and Manual O.2 Q8 and Manual O.2 Q9 and Manual O.2 Q11 and Manual O.2 Q12 and Manual O.2 Q13 and Manual O.2 Q14 and Manual O.2 Q15 and Manual O.2 Q16 and Manual O.2 Q17 and Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q12 and Manual O.3 Q13 and Manual O.3 Q14 and Manual O.3 Q15 and Manual O.3 Q16 and Manual O.3 Q17 and Manual O.3 Q18 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q5 and Manual O.4 Q9 and Manual O.4 Q12 and Manual O.4 Q14.

What is printed?

```
s = {3, 1, 2, 3, 1}
print(len(s))
print(sorted(s))
```

a. 5

[1, 1, 2, 3, 3]

b. 3

[1, 2, 3]

c. 3

[3, 1, 2]

d. 5

[1, 2, 3]

✗ 5

[1, 1, 2, 3, 3]

Wrong. A set discards duplicates the moment it is built, so it never held five items.

✓ 3

[1, 2, 3]

Correct. The duplicates collapse, leaving three items, and `sorted()` puts them in order for printing, which is how you show a set reliably.

✗ 3

[3, 1, 2]

Wrong. `sorted()` returns an ordered LIST, so it prints in ascending order whatever order the set happened to be in.

✗ 5

[1, 2, 3]

Wrong. `len()` of that set is 3, not 5, because uniqueness is enforced.

The answer is 3 then [1, 2, 3]. Sets are unique and unordered. That is why the manual compares set answers as sets, never by printed order: Python randomises string hashing per run.

Asked twice. This same question also appears as: Manual O.2 Q3 and Manual O.2 Q4 and Manual O.2 Q5 and Manual O.2 Q6 and Manual O.2 Q7 and Manual O.2 Q8 and Manual O.2 Q9 and Manual O.2 Q11 and Manual O.2 Q12 and Manual O.2 Q13 and Manual O.2 Q14 and Manual O.2 Q15 and Manual O.2 Q16 and Manual O.2 Q17 and Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.3 Q13 and Manual O.3 Q14 and Manual O.3 Q15 and Manual O.3 Q16 and Manual O.3 Q17 and Manual O.3 Q18 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q5 and Manual O.4 Q9 and Manual O.4 Q12 and Manual O.4 Q14.

What is printed?

```
a = {1, 2, 3, 4}
b = {3, 4, 5}
print(sorted(a & b))
print(sorted(a | b))
print(sorted(a - b))
```

- a. [1, 2]
[1, 2, 3, 4, 5]
[3, 4]
- b. [3, 4]
[1, 2, 5]
[1, 2]
- c. [3, 4]
[1, 2, 3, 4, 5]
[1, 2]
- d. [3, 4]
[1, 2, 3, 3, 4, 4, 5]
[1, 2]

✗ [1, 2]
[1, 2, 3, 4, 5]
[3, 4]
Wrong, and it has intersection and difference swapped.

✗ [3, 4]
[1, 2, 5]
[1, 2]
Wrong. Union is EVERYTHING from both, not only the items unique to one.

✓ [3, 4]
[1, 2, 3, 4, 5]

[1, 2]

Correct. The ampersand is intersection, what is in both. The bar is union, everything once. The minus is difference, what is in a and not in b.

✗ [3, 4]

[1, 2, 3, 3, 4, 4, 5]

[1, 2]

Wrong. A union is still a set, so nothing appears twice in it.

The answer is [3, 4] then [1, 2, 3, 4, 5] then [1, 2]. Ampersand intersection, bar union, minus difference, caret symmetric difference. The named methods are intersection(), union(), difference() and they mean the same thing.

Asked twice. This same question also appears as: Manual O.2 Q3 and Manual O.2 Q4 and Manual O.2 Q5 and Manual O.2 Q6 and Manual O.2 Q7 and Manual O.2 Q8 and Manual O.2 Q9 and Manual O.2 Q11 and Manual O.2 Q12 and Manual O.2 Q13 and Manual O.2 Q14 and Manual O.2 Q15 and Manual O.2 Q16 and Manual O.2 Q17 and Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.3 Q12 and Manual O.3 Q14 and Manual O.3 Q15 and Manual O.3 Q16 and Manual O.3 Q17 and Manual O.3 Q18 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q5 and Manual O.4 Q9 and Manual O.4 Q12 and Manual O.4 Q14.

What is printed?

```
d = {"a": 1, "b": 2}
d["c"] = 3
d["a"] = 9
print(d)
print(len(d))
```

a. {'a': 1, 'b': 2, 'c': 3}

3

b. {'a': 9, 'b': 2, 'c': 3}

4

c. {'a': 1, 'a': 9, 'b': 2, 'c': 3}

4

d. {'a': 9, 'b': 2, 'c': 3}

3

✗ {'a': 1, 'b': 2, 'c': 3}

3

Wrong. `d["a"] = 9` really does overwrite the 1.

✗ {'a': 9, 'b': 2, 'c': 3}

4

Wrong. There are three keys, not four. Overwriting a key does not add one.

✗ {'a': 1, 'a': 9, 'b': 2, 'c': 3}

4

Wrong. A dictionary cannot hold the same key twice. That is what makes it a mapping.

✓ {'a': 9, 'b': 2, 'c': 3}

3

Correct. Assigning to a key that does not exist ADDS it; assigning to one that does REPLACES its value. Three keys remain.

The answer is {'a': 9, 'b': 2, 'c': 3} then 3. Keys are unique; assigning to an existing key replaces its value. Since Python 3.7 a dictionary also keeps insertion order, which is why the printed order is predictable.

Asked twice. This same question also appears as: Manual O.2 Q3 and Manual O.2 Q4 and Manual O.2 Q5 and Manual O.2 Q6 and Manual O.2 Q7 and Manual O.2 Q8 and Manual O.2 Q9 and Manual O.2 Q11 and Manual O.2 Q12 and Manual O.2 Q13 and Manual O.2 Q14 and Manual O.2 Q15 and Manual O.2 Q16 and Manual O.2 Q17 and Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.3 Q12 and Manual O.3 Q13 and Manual O.3 Q14 and Manual O.3 Q16 and Manual O.3 Q17 and Manual O.3 Q18 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q5 and Manual O.4 Q9 and Manual O.4 Q12 and Manual O.4 Q14.

What is printed?

```
d = {"x": 1, "y": 2}
for k in d:
    print(k, end=" ")
print()
for k, v in d.items():
    print(k, v, end=" ")
```

a. 1 2

x 1 y 2

b. x y

x 1 y 2

c. ('x', 1) ('y', 2)

x 1 y 2

d. x y

1 2

X 1 2

x 1 y 2

Wrong. The plain loop gives keys, not values. For values you would write `d.values()`.

✓ x y

x 1 y 2

Correct. Iterating a dictionary directly gives its KEYS. `items()` gives key and value pairs, which unpack into the two loop variables.

X ('x', 1) ('y', 2)

x 1 y 2

Wrong. The plain loop yields keys alone, not pairs. Pairs come from `items()`.

X x y

1 2

Wrong on the second line: `items()` gives both parts, and both are printed.

The answer is `x y` then `x 1 y 2`. `for k in d` gives keys. `d.keys()`, `d.values()`, `d.items()` give what they say. Forgetting `items()` and then trying to unpack a key is a common exam slip.

Asked twice. This same question also appears as: Manual O.2 Q3 and Manual O.2 Q4 and Manual O.2 Q5 and Manual O.2 Q6 and Manual O.2 Q7 and Manual O.2 Q8 and Manual O.2 Q9 and Manual O.2 Q11 and Manual O.2 Q12 and Manual O.2 Q13 and Manual O.2 Q14 and Manual O.2 Q15 and Manual O.2 Q16 and Manual O.2 Q17 and Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.3 Q12 and Manual O.3 Q13 and Manual O.3 Q14 and Manual O.3 Q15 and Manual O.3 Q16 and Manual O.3 Q18 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q5 and Manual O.4 Q9 and Manual O.4 Q12 and Manual O.4 Q14.

What is printed?

```
print("a" in "cat")
print(2 in [1, 2, 3])
print("x" in {"x": 1})
```

a. True

True

False

b. False

True

True

c. True

False

True

d. True

True

True

X True

True

False

Wrong on the third line. in on a dictionary searches the keys, and "x" is a key here.

X False

True

True

Wrong on the first line. "a" is one of the characters of "cat", so it is found.

X True

False

True

Wrong on the second line. 2 is an item of that list.

✓ True

True

True

Correct. On a string, in tests for a substring. On a list, for an item. On a dictionary, for a KEY.

The answer is True then True then True. in means substring on a string, membership on a list or set, and KEY membership on a dictionary. That last one is the trap.

Asked twice. This same question also appears as: Manual O.2 Q3 and Manual O.2 Q4 and Manual O.2 Q5 and Manual O.2 Q6 and Manual O.2 Q7 and Manual O.2 Q8 and Manual O.2 Q9 and Manual O.2 Q11 and Manual O.2 Q12 and Manual O.2 Q13 and Manual O.2 Q14 and Manual O.2 Q15 and Manual O.2 Q16 and Manual O.2 Q17 and Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.3 Q12 and Manual O.3 Q13 and Manual O.3 Q14 and Manual O.3 Q15 and Manual O.3 Q16 and Manual O.3 Q17 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q5 and Manual O.4 Q9 and Manual O.4 Q12 and Manual O.4 Q14.

What is printed?

```
xs = [10, 20, 30, 40]
del xs[1]
print(xs)
d = {"a": 1, "b": 2}
del d["a"]
print(d)
```

a. [10, 30, 40]

{'b': 2}

b. [20, 30, 40]

{'b': 2}

c. [10, 20, 30]

{'a': 1}

d. [10, 30, 40]

{'a': 1}

✓ [10, 30, 40]

{'b': 2}

Correct. del removes by POSITION on a list, so index 1 (the 20) goes, and by KEY on a dictionary.

✗ [20, 30, 40]

{'b': 2}

Wrong. Index 1 is the second item. Index 0 is the 10.

✗ [10, 20, 30]

{'a': 1}

Wrong. del xs[1] takes out the second item, not the last, and del d["a"] takes out the "a" entry.

✗ [10, 30, 40]

{'a': 1}

Wrong on the dictionary: del d["a"] removes the key it names.

The answer is [10, 30, 40] then {'b': 2}. del removes by index on a list and by key on a dictionary. remove() removes by VALUE on a list, and pop() removes and gives the item back.

Asked twice. This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.1 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.4.

What does this print?

```
for n in range(1, 8):  
    if n == 5:  
        break  
    if n % 2 == 0:  
        continue  
    print(n)
```

a. 1

3

5

7

b. 1

2

3

4

c. 1

3

d. 1

3

5

X 1

3

5

7

Wrong. break stops the loop at 5.

✗ 1
2
3
4

Wrong. continue skips the even numbers.

✓ 1
3

Correct. Even numbers are skipped by continue, and at 5 break ends the loop before printing.

✗ 1
3
5

Wrong. break runs before 5 can be printed.

break leaves the loop at once; continue skips the rest of this pass.

The user types 5, then 8, then 0, then 3. The list stands in for input(). What does this print?

```
typed = [5, 8, 0, 3]    # stands in for four input() calls
i = 0
total = 0
count = 0
number = typed[i]
while number != 0:
    total += number
    count += 1
    i += 1
    number = typed[i]
print(total, count)
```

- a. 13 2
- b. 16 4
- c. 13 3
- d. 16 3

✓ 13 2

Correct. 5 and 8 are added; the 0 stops the loop before it is added or counted, and the 3 is never read.

✗ 16 4

Wrong. Neither the 0 nor the 3 is counted.

✗ 13 3

Wrong. The 0 is not counted, because the body never runs for it.

✗ 16 3

Wrong. The loop stops at 0, so 3 is never added.

A sentinel loop reads once before the loop and again at the bottom. The sentinel itself is never added or counted.

Asked twice. This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.1 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.4.

What does this print?

```
a = [1, 2]
a.append([3, 4])
b = [1, 2]
b += [3, 4]
print(len(a), len(b))
```

a. 3 3

b. 4 4

c. 4 3

d. 3 4

✗ 3 3

Wrong. += extends the list item by item.

✗ 4 4

Wrong. append adds the whole list as a single item.

✗ 4 3

Wrong. The two results are the other way round.

✓ 3 4

Correct. append adds [3, 4] as ONE item, giving 3; += adds 3 and 4 as separate items, giving 4.

| append adds one item; += and extend add the items of the other list one by one.

Asked twice. This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.1 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.4.

What does this print?

```
n = [[1, [2, 3]], [4, 5]]
print(n[0][1][0], len(n[0]))
```

- a. 2 2
- b. 1 2
- c. 2 3
- d. [2, 3] 2

✓ 2 2

Correct. `n[0]` is `[1, [2, 3]]`; `[1]` of that is `[2, 3]`; `[0]` of that is 2. `n[0]` has two items.

✗ 1 2

Wrong. Peel one bracket at a time: `n[0]`, then `[1]`, then `[0]`.

✗ 2 3

Wrong. `len` counts items: `n[0]` holds a number and a list, two items.

✗ [2, 3] 2

Wrong. The last `[0]` reaches inside `[2, 3]`.

Peel a nested index one bracket at a time and write down what you have after each; `len` counts items, not the numbers inside them.

Asked twice. This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.1 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.4.

What does this print?

```
t = (5,)
u = (5)
print(type(t).__name__, type(u).__name__)
```

- a. tuple tuple
- b. int int
- c. tuple int
- d. int tuple

✗ tuple tuple

Wrong. Without a comma the brackets are only grouping.

✗ int int

Wrong. The comma makes t a tuple.

✓ tuple int

Correct. (5,) is a tuple because of the comma; (5) is just the number 5 in brackets.

✗ int tuple

Wrong. They are the other way round.

! A tuple of one item needs a comma: (5,) is a tuple, (5) is 5.

Asked twice. This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.1 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.4.

What does this print?

```
a = [1, 2]
b = list(a)
b.append(3)
print(a, b)
```

- a. [1, 2, 3] [1, 2]
- b. [1, 2] [1, 2]
- c. [1, 2, 3] [1, 2, 3]
- d. [1, 2] [1, 2, 3]

✗ [1, 2, 3] [1, 2]

Wrong. a is the one left unchanged.

✗ [1, 2] [1, 2]

Wrong. b did get the 3.

✗ [1, 2, 3] [1, 2, 3]

Wrong. That is what `b = a` would do; `list(a)` is a copy.

✓ [1, 2] [1, 2, 3]

Correct. `list(a)` makes a NEW list, so appending to b leaves a alone.

Assignment does not copy: `b = a` gives one list two names. `list(a)` or `a[:]` makes a real copy.

Asked twice. This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.1 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.4.

What does this print?

```
print(type({}).__name__, type(set().__name__))
```

- a. set set
- b. dict dict
- c. dict set
- d. set dict

✗ set set

Wrong. {} is a dictionary.

✗ dict dict

Wrong. set() makes a set.

✓ dict set

Correct. {} on its own is an empty **DICTIONARY**; an empty set must be written set().

✗ set dict

Wrong. They are the other way round.

| {} alone is an empty dictionary, not an empty set.

Asked twice. This same question also appears as: Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 2.3 and Lesson 3.2 and Lesson 3.4 and Lesson 3.5.

What happens when this runs?

```
d = {[1, 2]: "a"}  
print(d)
```

- a. It raises `TypeError`, because a list cannot be a dictionary key
- b. It raises `KeyError`
- c. It prints `{[1, 2]: 'a'}`
- d. It prints `a`

✓ It raises `TypeError`, because a list cannot be a dictionary key

Correct. A key must be hashable, which means immutable; a list is mutable.

✗ It raises `KeyError`

Wrong. Nothing is looked up; the key itself is rejected.

✗ It prints `{[1, 2]: 'a'}`

Wrong. The dictionary cannot be built.

✗ It prints `a`

Wrong. The error happens when the dictionary is made.

Strings, numbers and tuples can be keys; lists and sets cannot, because they can change.

Asked twice. This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.1 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.4.

What does this print?

```
d = {"x": 1}
print(1 in d, "x" in d)
```

- a. True True
- b. False True
- c. True False
- d. False False

X True True

Wrong. 1 is a value, and in does not look at values.

✓ False True

Correct. in on a dictionary checks the KEYS. 1 is a value, not a key; "x" is a key.

X True False

Wrong. They are the other way round.

X False False

Wrong. "x" is a key, so that test is True.

| in means a substring on a string, an item on a list or set, and a KEY on a dictionary.

4 Functions, Modules, Files and Exceptions

4.1

EASY

Manual O.4 Q6

A file is opened with `open("data.txt", "w")` when `data.txt` already exists and contains text. What happens?

- a. The new text is added after the existing text
- b. Python raises `FileExistsError`
- c. The existing contents are erased
- d. The file is opened read-only

✗ The new text is added after the existing text

Wrong, and this is the mode people mean when they choose "w" by mistake. Appending is "a".

✗ Python raises `FileExistsError`

Wrong. "w" is perfectly happy for the file to exist. It just empties it. The mode that refuses to overwrite is "x".

✓ The existing contents are erased

Correct. "w" truncates the file to nothing the moment it is opened, before you write a single character. Data is lost even if the program then crashes.

✗ The file is opened read-only

Wrong. "w" is write-only. Reading is "r", which is also the default.

The answer is The existing contents are erased. "r" read, "w" write and TRUNCATE, "a" append, "x" create only if absent. Choosing "w" where you meant "a" destroys the file, and the paper asks about this.

A text file holds three lines. Which call gives a LIST of those three lines?

- a. `f.readlines()`
- b. `f.read()`
- c. `f.readline()`
- d. `f.line()`

✓ `f.readlines()`

Correct. `readlines()` gives a list with one string per line, each still carrying its trailing newline unless you strip it.

✗ `f.read()`

Wrong. `read()` gives the WHOLE file as one string, newlines and all.

✗ `f.readline()`

Wrong. `readline()` gives ONE line, the next one, as a string.

✗ `f.line()`

Wrong. There is no such method.

The answer is `f.readlines()`. read whole, readline one, readlines list. Note the newline stays on each item, so `print()` of one adds a blank line unless you strip.

What is a docstring?

- a. A comment starting with a hash inside a function
- b. A special variable holding the function name
- c. A string literal as the first statement of a function, module or class, describing what it does
- d. A file that documents the whole program

X A comment starting with a hash inside a function

Wrong. A hash comment is stripped and cannot be read back at run time. A docstring survives into the object.

X A special variable holding the function name

Wrong. That is `__name__`, a different attribute entirely.

✓ A string literal as the first statement of a function, module or class, describing what it does

Correct. It sits directly under the `def` line, usually in triple quotes, and becomes the object's `__doc__`, which is what `help()` shows you.

X A file that documents the whole program

Wrong. A docstring lives inside the code, not beside it.

The answer is A string literal as the first statement of a function, module or class, describing what it does. First statement, a string, becomes `__doc__`. It is the "self-documenting" part of the modularisation answer.

Asked twice. This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.1 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.4.

What does this print?

```
def area(l, w):  
    return l * w  
  
print(area(3, 4) + area(1, 2))
```

- a. 14
- b. None
- c. 12
- d. 7

✓ 14

Correct. `area(3, 4)` returns 12 and `area(1, 2)` returns 2; $12 + 2$ is 14.

✗ None

Wrong. The function returns its result, so the calls can be added.

✗ 12

Wrong. The second call adds 2 more.

✗ 7

Wrong. The function multiplies, it does not add.

Because a value returning function hands back its result, a call can be used anywhere a value can.

[illegible]

What does this print?

```
def hello():
    print("hello")

print("bye")
```

- hello
- hello
bye
- nothing at all
- bye

X hello

Wrong. `print("bye")` runs; the function body does not.

~~x~~ hello

bye

Wrong. A definition on its own produces no output.

X nothing at all

Wrong. The last line does print.

✓ bye

Correct. `hello` is defined but never called, so its body never runs.

Nothing inside a function happens until it is called.

4.6

EASY

Lesson 3.1

In `def cube(num):` and the call `cube(4)`, which is the ARGUMENT?

- a. return, the keyword
- b. cube, the function's name
- c. num, the name in the definition
- d. 4, the value supplied at the call

✗ return, the keyword

Wrong. return hands a value back.

✗ cube, the function's name

Wrong. That is the function's name.

✗ num, the name in the definition

Wrong. num is the PARAMETER.

✓ 4, the value supplied at the call

Correct. The argument is the value passed in when you call.

| The parameter is the name in the definition; the argument is the value supplied at the call.

4.7

EASY

Lesson 3.1

True or false: a function with no return statement gives back None.

Answer: **true**

| Show or give: print shows, return hands back, and no return means None.

Asked twice. This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.1 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.4.

What does this print?

```
import math
print(math.ceil(2.1), math.floor(2.9))
```

a. 3 2

b. 2 2

c. 3 3

d. 2 3

✓ 3 2

Correct. ceil always rounds UP to the next whole number and floor always rounds DOWN.

✗ 2 2

Wrong. ceil(2.1) goes up to 3.

✗ 3 3

Wrong. floor(2.9) goes down to 2.

✗ 2 3

Wrong. They are the other way round.

math.ceil rounds up, math.floor rounds down, round goes to the nearest.

Asked twice. This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.1 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.4.

What does this print?

```
from math import sqrt  
print(sqrt(49))
```

- a. an error, because sqrt needs the math. prefix
- b. 49
- c. 7.0
- d. 7

X an error, because sqrt needs the math. prefix

Wrong. The from form brings the name in, so no prefix is needed.

X 49

Wrong. sqrt is the square root, not the square.

✓ 7.0

Correct. from math import sqrt brings sqrt in directly, and sqrt always returns a float.

X 7

Wrong. sqrt returns a float, 7.0.

| from math import sqrt gives you sqrt alone; import math gives you math.sqrt.

Asked twice. This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.1 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.4.

What does this print?

```
import math
print(round(math.pi, 3))
```

- a. 3
- b. 3.1416
- c. 3.142
- d. 3.14

X 3

Wrong. round with 3 keeps three decimal places.

X 3.1416

Wrong. That is four decimal places.

✓ 3.142

Correct. round(math.pi, 3) keeps three decimal places.

X 3.14

Wrong. That is two decimal places.

| When the paper says use the math module, use math.pi, rounded as asked.

4.11

EASY

Lesson 3.2

What is a module, in the course's words?

- a. A single function inside a program
- b. A file of Python definitions and statements that other programs can import and reuse
- c. A folder of databases
- d. A window in a graphical interface

✗ A single function inside a program

Wrong. A module is a whole file, which may hold many functions.

✓ A file of Python definitions and statements that other programs can import and reuse

Correct. That is the definition.

✗ A folder of databases

Wrong. A directory of MODULES is a package.

✗ A window in a graphical interface

Wrong. That is a tkinter window.

| A module is a file of Python code that can be imported; a package is a directory of modules.

4.12

EASY

Lesson 3.2

True or false: after `import math` you can write `sqrt(16)` without the `math.` prefix.

Answer: **false**

| `import math` gives `math` dot something; `from math import sqrt` gives `sqrt` alone.

Asked twice. This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.1 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.4.

What does this print?

```
with open("t.txt", "w") as f:
    f.write("one\n")
with open("t.txt", "a") as f:
    f.write("two\n")
print(open("t.txt").read().strip())
```

a. one

two

b. two

c. one

d. two

one

✓ one

two

Correct. "a" opens at the end of the file, so the new line goes after the old one.

✗ two

Wrong. Append keeps the existing content.

✗ one

Wrong. The appended line is there too.

✗ two

one

Wrong. Appended data goes at the end.

Append adds new data at the end and leaves everything already there.

Asked twice. This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.1 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.4.

What does this print?

```
with open("t.txt", "w") as f:  
    f.write("x\ny\nz\n")  
print(len(open("t.txt").readlines()))
```

- a. 1
- b. 6
- c. 5
- d. 3

✗ 1

Wrong. That is how many strings `read()` returns.

✗ 6

Wrong. `len` counts lines here, not characters.

✗ 5

Wrong. There are three lines.

✓ 3

Correct. `readlines()` returns a list with one string per line: `x`, `y` and `z`.

| `read()` gives one string, `readline()` one line, `readlines()` a list of every line.

Asked twice. This same question also appears as: Manual O.4 Q10.

Which exception does this raise?

```
print("5" + 5)
```

- a. NameError
- b. ValueError
- c. SyntaxError
- d. TypeError

✗ NameError

Wrong. Nothing here is undefined.

✗ ValueError

Wrong. ValueError is the right kind with an impossible value, like `int("abc")`.

✗ SyntaxError

Wrong. The line is legal Python; it fails only when run.

✓ TypeError

Correct. `+` cannot join a string and a number: the wrong KIND of thing.

| TypeError is the wrong kind; ValueError is the right kind with an impossible value.

Which of these is **NOT** a benefit of breaking a program into functions?

- a. The program runs measurably faster because it is shorter
- b. The same code can be reused instead of repeated
- c. Each piece can be tested on its own
- d. The program is easier to read and to maintain

✓ The program runs measurably faster because it is shorter

Correct. Calling a function costs a little time rather than saving it. The benefits of modularising are all about the PROGRAMMER: reuse, testing, readability, maintainability, teamwork. Speed is not among them.

✗ The same code can be reused instead of repeated

Wrong choice, because reuse IS a benefit, and usually the first one listed.

✗ Each piece can be tested on its own

Wrong choice, because testing a piece in isolation IS a benefit.

✗ The program is easier to read and to maintain

Wrong choice, because readability and maintenance ARE benefits, and the paper lists them.

The answer is The program runs measurably faster because it is shorter. Every benefit of modularising is a benefit to the person reading or changing the code. Speed is not one, and a question offering it is offering the distractor.

Asked twice. This same question also appears as: Manual O.2 Q3 and Manual O.2 Q4 and Manual O.2 Q5 and Manual O.2 Q6 and Manual O.2 Q7 and Manual O.2 Q8 and Manual O.2 Q9 and Manual O.2 Q11 and Manual O.2 Q12 and Manual O.2 Q13 and Manual O.2 Q14 and Manual O.2 Q15 and Manual O.2 Q16 and Manual O.2 Q17 and Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.3 Q12 and Manual O.3 Q13 and Manual O.3 Q14 and Manual O.3 Q15 and Manual O.3 Q16 and Manual O.3 Q17 and Manual O.3 Q18 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q9 and Manual O.4 Q12 and Manual O.4 Q14.

What is printed?

```
import math
from math import sqrt
print(math.sqrt(16), sqrt(16))
```

- a. 4 4
- b. 4.0 4.0
- c. 4.0 4
- d. NameError

✗ 4 4

Wrong. `math.sqrt` returns a float even for a perfect square, so the point and zero are printed.

✓ 4.0 4.0

Correct. Both forms reach the same function. `sqrt` always returns a float, so both print 4.0.

✗ 4.0 4

Wrong. Both calls are the same function and give the same value.

✗ NameError

Wrong. Both imports are valid, and each makes its name available in its own way.

The answer is 4.0 4.0. `import math` needs the `math.` prefix; `from math import sqrt` puts the name in directly. Both are examinable and the difference is which name you must then write.

Why is `with open(...)` as `f` preferred to `f = open(...)`?

- a. It is faster to type
- b. It is the only way to read a file line by line
- c. It opens the file in a special faster mode
- d. It closes the file automatically, even if an error is raised inside the block

✗ It is faster to type

Wrong. Brevity is not the reason, and it is barely shorter anyway.

✗ It is the only way to read a file line by line

Wrong. A plain `open()` can be iterated line by line just as well. The difference is only about closing.

✗ It opens the file in a special faster mode

Wrong. The mode is whatever you pass to `open`. `with` changes nothing about how the file is opened.

✓ It closes the file automatically, even if an error is raised inside the block

Correct. The `with` block guarantees `f.close()` runs on the way out, whether the block finished normally or raised. Without it, an exception can leave the file open and its buffer unwritten.

The answer is It closes the file automatically, even if an error is raised inside the block. `with` guarantees the close. It is the same guarantee finally gives, written more briefly, and saying that in the exam earns the mark.

Asked twice. This same question also appears as: Lesson 3.4.

Which exception does this raise?

```
xs = [1, 2, 3]
print(xs[5])
```

- a. KeyError
- b. ValueError
- c. TypeError
- d. IndexError

✗ KeyError

Wrong. KeyError is for a missing DICTIONARY key. A list is indexed by position, not by key.

✗ ValueError

Wrong. ValueError is for a value of the right type that cannot be used, such as `int("abc")`.

✗ TypeError

Wrong. TypeError would be `xs["a"]`, where the index is the wrong KIND of thing entirely.

✓ IndexError

Correct. The index is outside the range of the list, which is exactly what IndexError means.

The answer is IndexError. IndexError for a sequence, KeyError for a mapping. Naming the right exception is worth marks on its own in this paper.

Why is `except:` on its own considered bad practice?

- a. It catches everything, including errors you did not anticipate, and hides real bugs
- b. It is slower than naming the exception
- c. It is not valid Python
- d. It can only be used once per program

✓ It catches everything, including errors you did not anticipate, and hides real bugs

Correct. A bare `except` swallows every exception, including a typo raising `NameError`, so the program keeps going in a broken state and the real fault is invisible.

✗ It is slower than naming the exception

Wrong. Speed is not the issue and the difference is immeasurable.

✗ It is not valid Python

Wrong. It is valid, which is exactly why it is dangerous.

✗ It can only be used once per program

Wrong. There is no such limit.

The answer is It catches everything, including errors you did not anticipate, and hides real bugs. Catch the exception you expect and no more. A bare `except` turns a loud failure into a silent wrong answer.

Asked twice. This same question also appears as: Manual O.2 Q3 and Manual O.2 Q4 and Manual O.2 Q5 and Manual O.2 Q6 and Manual O.2 Q7 and Manual O.2 Q8 and Manual O.2 Q9 and Manual O.2 Q11 and Manual O.2 Q12 and Manual O.2 Q13 and Manual O.2 Q14 and Manual O.2 Q15 and Manual O.2 Q16 and Manual O.2 Q17 and Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.3 Q12 and Manual O.3 Q13 and Manual O.3 Q14 and Manual O.3 Q15 and Manual O.3 Q16 and Manual O.3 Q17 and Manual O.3 Q18 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q5 and Manual O.4 Q9 and Manual O.4 Q14.

What is printed?

```
def check(age):  
    if age < 0:  
        raise ValueError("negative age")  
    return age  
  
try:  
    check(-1)  
except ValueError as e:  
    print(e)
```

- a. ValueError
- b. negative age
- c. -1
- d. nothing is printed

ValueError

Wrong. Printing the exception object shows its MESSAGE, not the name of its class. For the class name you would print `type(e).__name__`.

negative age

Correct. `raise` creates the exception with that message, the `except` catches it and binds it to `e`, and printing an exception prints its message.

-1

Wrong. The argument is never returned, because the `raise` happens first.

nothing is printed

Wrong. The `except` block does run and does print.

The answer is negative age. `raise` makes your own error, and validating input this way is what the paper means by defensive programming.

Asked twice. This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.1 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.4.

What does this print?

```
def power(base, exp=2):  
    return base ** exp  
  
print(power(3), power(2, 3), power(exp=1, base=9))
```

a. 6 8 9

b. 9 8 1

c. 9 6 9

d. 9 8 9

X 6 8 9

Wrong. `**` is power, so `3 ** 2` is 9.

X 9 8 1

Wrong. `exp=1, base=9` is 9 to the power 1, which is 9.

X 9 6 9

Wrong. `2 ** 3` is 8.

✓ 9 8 9

Correct. `power(3)` uses the default exponent 2; `power(2, 3)` is 8; named arguments give `9 ** 1`.

| A parameter with a default can be left out; arguments passed by name can come in any order.

Asked twice. This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.1 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.4.

What does this print?

```
def stats(xs):  
    return min(xs), max(xs)  
  
lo, hi = stats([4, 9, 2])  
print(hi - lo)
```

- a. 11
- b. 2 9
- c. (2, 9)
- d. 7

X 11

Wrong. The program subtracts, it does not add.

X 2 9

Wrong. The print shows hi - lo, one number.

X (2, 9)

Wrong. The two values were unpacked into separate names.

✓ 7

Correct. stats returns 2 and 9, which unpack into lo and hi; 9 - 2 is 7.

| return a, b hands back two values, unpacked with two names on the left.

Which two of these are benefits of modularising a program with functions?

- a. Easier maintenance: a change made in one place takes effect everywhere
- b. The program always runs faster
- c. Reusability: write once, call from many places
- d. Every variable becomes global

✓ Easier maintenance: a change made in one place takes effect everywhere

Correct. Easier maintenance is one of the five benefits.

✗ The program always runs faster

Wrong. The course says modularising does NOT make the program run faster.

✓ Reusability: write once, call from many places

Correct. Reusability is one of the five benefits.

✗ Every variable becomes global

Wrong. Names made inside a function are local.

Five benefits: reusability, readability, easier debugging and testing, easier maintenance, teamwork and simpler design.

Asked twice. This same question also appears as: Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 2.3 and Lesson 2.4 and Lesson 3.4 and Lesson 3.5.

What happens when this runs?

```
from math import pi  
print(math.pi)
```

- a. It raises `NameError`, because the name `math` was never imported
- b. It raises `ImportError`
- c. It prints `3.14`
- d. It prints `3.141592653589793`

✓ It raises `NameError`, because the name `math` was never imported
Correct. `from math import pi` brings in `pi` alone; `math` itself is not a name in the program.

✗ It raises `ImportError`
Wrong. The import itself succeeded.

✗ It prints `3.14`
Wrong. The line fails before printing anything.

✗ It prints `3.141592653589793`
Wrong. `math.pi` needs `import math`.

! The two forms are not interchangeable: after `from math import pi`, write `pi`, not `math.pi`.

Asked twice. This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.1 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.4.

What does this print?

```
import math as m
print(m.sqrt(9))
```

- a. 9
- b. 3
- c. an error, because math was renamed
- d. 3.0

X 9

Wrong. sqrt is the square root.

X 3

Wrong. sqrt returns a float.

X an error, because math was renamed

Wrong. Renaming is allowed; you then use the new name.

✓ 3.0

Correct. import math as m renames the module, so m.sqrt(9) is 3.0.

| import math as m gives the module a shorter name; everything is then reached through m.

What can `random.randint(1, 6)` return?

- a. Always 1
- b. A decimal between 1 and 6
- c. A whole number from 1 to 6, with both 1 and 6 possible
- d. A whole number from 1 to 5

✗ Always 1

Wrong. It is random.

✗ A decimal between 1 and 6

Wrong. `randint` gives whole numbers.

✓ A whole number from 1 to 6, with both 1 and 6 possible

Correct. `randint(a, b)` includes both ends.

✗ A whole number from 1 to 5

Wrong. Unlike `range`, `randint` includes its upper limit.

`random.randint(a, b)` gives a whole number from `a` to `b` inclusive; `random.choice(seq)` picks one item.

Asked twice. This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.1 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.4.

What does this print?

```
with open("t.txt", "w") as f:
    f.write("one\n")
with open("t.txt", "w") as f:
    f.write("two\n")
print(open("t.txt").read().strip())
```

- a. an error, because the file already exists
- b. two
- c. one
- d. one
two

X an error, because the file already exists

Wrong. "w" happily opens an existing file, and empties it.

✓ two

Correct. Opening with "w" a second time empties the file, so only the second line is left.

X one

Wrong. The second "w" replaced the first line.

X one

two

Wrong. That is what append mode, "a", would give.

| w wipes, a adds, r reads.

Asked twice. This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.1 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.4.

What does this print?

```
with open("t.txt", "w") as f:  
    f.write("a")  
    f.write("b")  
print(open("t.txt").read())
```

- a. a
b
- b. b
- c. ab
- d. a b

X a
b

Wrong. write() does not add a newline for you.

X b

Wrong. Both writes happen inside the same open file.

✓ ab

Correct. write() puts exactly what it is given; with no `\n` the two pieces join.

X a b

Wrong. No space was written.

| Always write the newline yourself: without `\n` every record runs into the next.

No file called gone.txt exists. What happens when this runs?

```
f = open("gone.txt", "r")  
print(f.read())
```

- a. It raises FileNotFoundError
- b. It creates an empty gone.txt
- c. It raises KeyError
- d. It prints an empty line

✓ It raises FileNotFoundError

Correct. Read mode needs the file to exist.

✗ It creates an empty gone.txt

Wrong. Only "w" and "a" create a missing file.

✗ It raises KeyError

Wrong. KeyError is for dictionaries.

✗ It prints an empty line

Wrong. The open fails before any read.

! "r" raises FileNotFoundError if the file is missing; "w" and "a" create it.

The file holds seven records. What does this print?

```
with open("r.txt", "w") as f:
    for i in range(1, 8):
        f.write(f"Student {i}\n")
with open("r.txt") as f:
    lines = f.readlines()
for line in lines[:2]:
    print(line.strip())
```

- a. Student 1
Student 2
- b. Student 1
Student 2
Student 3
- c. Student 1
- d. Student 7

✓ Student 1
Student 2

Correct. `lines[:2]` takes the first two lines, and `strip()` removes each newline.

✗ Student 1
Student 2
Student 3

Wrong. A slice stops before its stop index, 2.

✗ Student 1

Wrong. Two lines are taken.

✗ Student 7

Wrong. `[:2]` takes from the start.

Given a list of lines, `lines[:5]` is the first five: a one-line answer to the paper's requirement.

Which mode opens a file for BOTH reading and writing, at the start, without truncating it?

- a. "a"
- b. "w"
- c. "r"
- d. "r+"

✗ "a"

Wrong. a writes at the end.

✗ "w"

Wrong. w erases the file.

✗ "r"

Wrong. r reads only.

✓ "r+"

Correct. r+ reads and writes, keeps the content, and raises if the file is missing.

The four modes the paper asks for: r read, w write and truncate, a append, r+ read and write.

Asked twice. This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.1 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4.

What does this print?

```
try:
    print("a")
    x = int("b")
    print("c")
except ValueError:
    print("d")
print("e")
```

a. a

d

e

b. a

d

c. d

e

d. a

c

d

e

✓ a

d

e

Correct. a prints, int("b") raises ValueError, the rest of the try is skipped, except prints d, and the program carries on to e.

X a
d

Wrong. After a handled exception the program continues.

X d
e

Wrong. a printed before the error.

X a
c
d
e

Wrong. c is skipped: the error jumps straight to except.

When a line in try raises, the rest of the try is skipped and the matching except runs; then the program continues.

Asked twice. This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.1 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4.

What does this print?

```
try:
    n = 10 // 2
except ZeroDivisionError:
    print("zero")
else:
    print("ok", n)
finally:
    print("end")
```

- a. zero
end
- b. ok 5
end
- c. end
- d. ok 5

X zero
end

Wrong. `10 // 2` does not divide by zero.

✓ ok 5
end

Correct. Nothing is raised, so `else` runs; `finally` runs every time.

X end

Wrong. `else` runs because the `try` succeeded.

X ok 5

Wrong. `finally` always runs.

else runs when nothing was raised; finally runs either way.

4.37

MEDIUM

Lesson 3.4

Now the division is by zero. What does this print?

```
try:
    n = 10 // 0
except ZeroDivisionError:
    print("zero")
else:
    print("ok")
finally:
    print("end")
```

- a. ok
end
- b. end
- c. zero
end
- d. zero

X ok
end

Wrong. else runs only when nothing was raised.

X end

Wrong. The except block prints first.

✓ zero
end

Correct. The except runs, else is skipped because the try failed, and finally still runs.

X zero

Wrong. finally always runs.

Finally means finally: it runs whether the try worked or not.

Can try and except catch a missing colon in the same file?

- a. Yes, with a bare except
- b. Yes, with except `SyntaxError`
- c. Only inside a function
- d. No: a syntax error stops the program from running at all, so no try block ever runs

✗ Yes, with a bare except

Wrong. Nothing runs at all.

✗ Yes, with except `SyntaxError`

Wrong. The try block never gets the chance to run.

✗ Only inside a function

Wrong. A syntax error anywhere stops the whole file.

✓ No: a syntax error stops the program from running at all, so no try block ever runs

Correct. Python cannot parse the file, so nothing runs, the try included.

| Syntax errors stop it starting. Exceptions stop it finishing, and only those can be caught.

Which two of these are exceptions raised while a program RUNS, rather than syntax errors?

- a. ZeroDivisionError, from `10 / 0`
- b. A string that opens with `"` and closes with `'`
- c. KeyError, from `d["missing"]`
- d. A missing colon after an `if`

✓ ZeroDivisionError, from `10 / 0`

Correct. `10 / 0` is legal code that fails only when it runs.

✗ A string that opens with `"` and closes with `'`

Wrong. That is a syntax error too.

✓ KeyError, from `d["missing"]`

Correct. A missing key is found only when the lookup runs.

✗ A missing colon after an `if`

Wrong. That is a syntax error, found before anything runs.

■ An exception is an error detected during execution; a syntax error is bad grammar found first.

Asked twice. This same question also appears as: Manual O.2 Q3 and Manual O.2 Q4 and Manual O.2 Q5 and Manual O.2 Q6 and Manual O.2 Q7 and Manual O.2 Q8 and Manual O.2 Q9 and Manual O.2 Q11 and Manual O.2 Q12 and Manual O.2 Q13 and Manual O.2 Q14 and Manual O.2 Q15 and Manual O.2 Q16 and Manual O.2 Q17 and Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.3 Q12 and Manual O.3 Q13 and Manual O.3 Q14 and Manual O.3 Q15 and Manual O.3 Q16 and Manual O.3 Q17 and Manual O.3 Q18 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q5 and Manual O.4 Q9 and Manual O.4 Q12 and Manual O.4 Q14.

What is printed?

```
def add(a, b=10):  
    return a + b  
  
print(add(5))  
print(add(5, 1))  
print(add(b=2, a=3))
```

a. 15

6

6

b. 15

6

5

c. 5

6

5

d. TypeError

6

5

✗ 15

6

6

Wrong on the last line: $3 + 2$ is 5, not 6.

✓ 15

6

5

Correct. The default fills in for b when only one argument is given. Positional arguments fill left to right. Keyword arguments may be given in any order because they name their parameter.

X 5

6

5

Wrong on the first line. b defaults to 10, so add(5) is 15.

X TypeError

6

5

Wrong. add(5) is legal precisely BECAUSE b has a default.

The answer is 15 then 6 then 5. A parameter with a default may be omitted. Keyword arguments name their target, so order stops mattering. Defaults must come after non-defaults in the definition.

4.41

HARD

Manual O.4 Q2

Asked twice. This same question also appears as: Manual O.2 Q3 and Manual O.2 Q4 and Manual O.2 Q5 and Manual O.2 Q6 and Manual O.2 Q7 and Manual O.2 Q8 and Manual O.2 Q9 and Manual O.2 Q11 and Manual O.2 Q12 and Manual O.2 Q13 and Manual O.2 Q14 and Manual O.2 Q15 and Manual O.2 Q16 and Manual O.2 Q17 and Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.3 Q12 and Manual O.3 Q13 and Manual O.3 Q14 and Manual O.3 Q15 and Manual O.3 Q16 and Manual O.3 Q17 and Manual O.3 Q18 and Manual O.4 Q1 and Manual O.4 Q3 and Manual O.4 Q5 and Manual O.4 Q9 and Manual O.4 Q12 and Manual O.4 Q14.

What is printed?

```
def f(x):  
    print(x * 2)  
  
def g(x):  
    return x * 2  
  
a = f(3)  
b = g(3)  
print(a, b)
```

- a. 6
6 6
- b. 6 6
None 6
- c. 6
None 6
- d. None 6

X 6
6 6

Wrong. a is None. A function with no return statement gives back None.

X 6 6
None 6

Wrong. f prints once, when it is called; the value it gives back is still None.

✓ 6
None 6

Correct. f prints 6 and returns nothing, so a is None. g returns 6 silently, so b is 6. The last line then prints None 6.

X None 6

Wrong. The 6 printed inside f appears first, on its own line.

The answer is 6 then None 6. print SHOWS a value, return HANDS IT BACK. A function that only prints returns None, and using that None later is a standard exam bug.

Asked twice. This same question also appears as: Manual O.2 Q3 and Manual O.2 Q4 and Manual O.2 Q5 and Manual O.2 Q6 and Manual O.2 Q7 and Manual O.2 Q8 and Manual O.2 Q9 and Manual O.2 Q11 and Manual O.2 Q12 and Manual O.2 Q13 and Manual O.2 Q14 and Manual O.2 Q15 and Manual O.2 Q16 and Manual O.2 Q17 and Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.3 Q12 and Manual O.3 Q13 and Manual O.3 Q14 and Manual O.3 Q15 and Manual O.3 Q16 and Manual O.3 Q17 and Manual O.3 Q18 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q5 and Manual O.4 Q9 and Manual O.4 Q12 and Manual O.4 Q14.

What is printed?

```
x = 1

def f():
    x = 2
    print(x)

f()
print(x)
```

a. 2

2

b. 1

1

c. 1

2

d. 2

1

X 2

2

Wrong. Assigning inside a function does not change the global unless the function declares global x.

X 1

1

Wrong. Inside f, the local x shadows the global one, so 2 prints first.

X 1

2

Wrong, and it has the two the wrong way round.

✓ 2

1

Correct. The assignment inside `f` creates a LOCAL `x`. The global `x` is untouched, so it is still 1 afterwards.

The answer is 2 then 1. Assigning to a name inside a function makes it local. To change a global you must say `global x`, and this pair of prints is exactly how the paper tests it.

Asked twice. This same question also appears as: Manual O.2 Q3 and Manual O.2 Q4 and Manual O.2 Q5 and Manual O.2 Q6 and Manual O.2 Q7 and Manual O.2 Q8 and Manual O.2 Q9 and Manual O.2 Q11 and Manual O.2 Q12 and Manual O.2 Q13 and Manual O.2 Q14 and Manual O.2 Q15 and Manual O.2 Q16 and Manual O.2 Q17 and Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.3 Q12 and Manual O.3 Q13 and Manual O.3 Q14 and Manual O.3 Q15 and Manual O.3 Q16 and Manual O.3 Q17 and Manual O.3 Q18 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q5 and Manual O.4 Q12 and Manual O.4 Q14.

What is printed?

```
try:
    x = 1 / 0
except ZeroDivisionError:
    print("caught")
finally:
    print("always")
```

- a. always
- b. caught
always
- c. caught
- d. the program crashes

X always

Wrong. The except block ran too, and its print came first.

✓ caught
always

Correct. The division raises, the matching except handles it, and finally runs afterwards as it always does.

X caught

Wrong. finally runs whether or not anything was raised. That is its whole point.

X the program crashes

Wrong. The exception was caught, so nothing propagates and nothing crashes.

The answer is caught then always. try, except by exception type, else if nothing was raised, finally always. finally is where you put the cleanup that must happen either way.

4.44

HARD

Manual O.4 Q14

Asked twice. This same question also appears as: Manual O.2 Q3 and Manual O.2 Q4 and Manual O.2 Q5 and Manual O.2 Q6 and Manual O.2 Q7 and Manual O.2 Q8 and Manual O.2 Q9 and Manual O.2 Q11 and Manual O.2 Q12 and Manual O.2 Q13 and Manual O.2 Q14 and Manual O.2 Q15 and Manual O.2 Q16 and Manual O.2 Q17 and Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.3 Q12 and Manual O.3 Q13 and Manual O.3 Q14 and Manual O.3 Q15 and Manual O.3 Q16 and Manual O.3 Q17 and Manual O.3 Q18 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q5 and Manual O.4 Q9 and Manual O.4 Q12.

What is printed?

```
def fact(n):  
    if n <= 1:  
        return 1  
    return n * fact(n - 1)  
  
print(fact(5))
```

- a. 15
- b. 5
- c. RecursionError
- d. 120

X 15

Wrong. 15 is $1+2+3+4+5$, the SUM. This function multiplies.

X 5

Wrong. The function does not simply return its argument; it multiplies down to the base case.

X RecursionError

Wrong. There IS a base case, so the recursion terminates. Without one, RecursionError would be right.

✓ 120

Correct. $5 * 4 * 3 * 2 * 1$ is 120. The base case at $n \leq 1$ stops the recursion.

The answer is 120. Every recursion needs a base case and a step that moves towards it. Missing the base case gives RecursionError, and the paper has asked for exactly that fault to be identified.

Asked twice. This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.1 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.4.

What does this print?

```
count = 5

def reset():
    count = 0

reset()
print(count)
```

- a. 0
- b. 5
- c. None
- d. an error

X 0

Wrong. Assigning inside a function makes a new local name.

✓ 5

Correct. The count inside reset() is local to it, so the outer count is untouched.

X None

Wrong. The outer count still holds 5.

X an error

Wrong. It runs without error.

A name assigned inside a function is local and disappears when the function ends: what happens in the function stays in the function.

Asked twice. This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.1 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.4.

What does this print?

```
with open("t.txt", "w") as f:
    f.write("x\n")
with open("t.txt") as f:
    print(repr(f.readline()))
```

- a. x
- b. 'x\n'
- c. 'x'
- d. ['x\n']

X x

Wrong. repr shows the quotes and the newline marker.

✓ 'x\n'

Correct. readline() gives the next line as a string, still carrying its newline, which repr shows as \n.

X 'x'

Wrong. The newline is kept.

X ['x\n']

Wrong. readline() gives a string, not a list.

| readline() returns one line including its newline; readlines() returns a list of such strings.

Asked twice. This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.1 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.4.

What does this print?

```
with open("t.txt", "w") as f:  
    n = f.write("hello\n")  
print(n)
```

- a. hello
- b. 6
- c. 5
- d. None

X hello

Wrong. It returns a number, not the text.

✓ 6

Correct. write() returns the number of characters written: five letters and the newline.

X 5

Wrong. The newline counts as a character.

X None

Wrong. write() returns a count.

| f.write(s) writes the text and returns how many characters it wrote.

Asked twice. This same question also appears as: Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 2.3 and Lesson 2.4 and Lesson 3.2 and Lesson 3.5.

What happens when this runs?

```
try:
    d = {"a": 1}
    print(d["b"])
except ValueError:
    print("bad value")
```

- a. It prints None
- b. It prints bad value
- c. It prints nothing and carries on
- d. A KeyError stops the program, because except ValueError does not catch it

✗ It prints None

Wrong. d["b"] raises; it does not give None.

✗ It prints bad value

Wrong. The lookup raises KeyError, not ValueError.

✗ It prints nothing and carries on

Wrong. An exception that is not caught stops the program.

✓ A KeyError stops the program, because except ValueError does not catch it

Correct. except only catches the exception it names; this one is a KeyError.

■ Name the exception you expect. An exception you did not name passes straight through.

Asked twice. This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.1 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.2 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 2.4 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4.

What does this print?

```
def withdraw(balance, amount):  
    if amount > balance:  
        raise ValueError("insufficient funds")  
    return balance - amount  
  
try:  
    print(withdraw(50, 80))  
except ValueError as e:  
    print("Error:", e)
```

- a. an unhandled ValueError
- b. Error: insufficient funds
- c. -30
- d. insufficient funds

X an unhandled ValueError

Wrong. The ValueError is caught.

✓ Error: insufficient funds

Correct. raise signals the error, and the except catches it and prints its message.

X -30

Wrong. The raise stops the function before it subtracts.

X insufficient funds

Wrong. The print adds "Error:" in front.

| raise signals an error yourself; except ValueError as e gives you its message in e.

5 Databases and GUI Development

5.1

EASY

Manual O.5 Q1

What is the correct order for working with a database in Python?

- a. connect, cursor, execute, commit, close
- b. cursor, connect, commit, execute, close
- c. connect, execute, cursor, close, commit
- d. open, read, write, close

✓ connect, cursor, execute, commit, close

Correct. Connect to the database, get a cursor from the connection, execute SQL through the cursor, commit so the change is saved, then close the connection.

✗ cursor, connect, commit, execute, close

Wrong. A cursor is obtained FROM a connection, so the connection must exist first.

✗ connect, execute, cursor, close, commit

Wrong. You cannot execute before you have a cursor, and committing after closing is too late.

✗ open, read, write, close

Wrong. That is the file-handling pattern, not the database one.

The answer is connect, cursor, execute, commit, close. connect, cursor, execute, commit, close.

Forget the commit and the rows vanish when the program ends, which is the classic lost mark.

A program inserts three rows and then closes the connection without calling commit.
What is in the table?

- a. All three rows
- b. No rows
- c. The first row only
- d. An error is raised

X All three rows

Wrong. Uncommitted work is not saved. It lived only in the transaction.

✓ No rows

Correct. Without a commit the transaction is rolled back when the connection closes, so the table is unchanged.

X The first row only

Wrong. There is nothing special about the first row; all three are in the same uncommitted transaction.

X An error is raised

Wrong. Nothing raises. The program appears to work, which is what makes this bug expensive.

The answer is No rows. Commit is what makes a change permanent. No commit, no data, and no error message to tell you.

Which SQL statement retrieves only the students whose score is above 50?

- a. SELECT * FROM students IF score > 50
- b. GET * FROM students WHERE score > 50
- c. SELECT * FROM students WHERE score > 50
- d. SELECT students WHERE score > 50

✗ SELECT * FROM students IF score > 50

Wrong. SQL filters with WHERE. IF is not part of a SELECT statement.

✗ GET * FROM students WHERE score > 50

Wrong. There is no GET in SQL. Retrieval is SELECT.

✓ SELECT * FROM students WHERE score > 50

Correct. SELECT names the columns, FROM names the table, WHERE filters the rows.

✗ SELECT students WHERE score > 50

Wrong. The table must be named after FROM, and the columns before it.

The answer is SELECT * FROM students WHERE score > 50. SELECT columns FROM table WHERE condition. That one line is most of what this course asks of SQL.

After executing a **SELECT** that matches five rows, what does `fetchone()` return?

- a. All five rows as a list
- b. The first row, as a tuple
- c. The number 5
- d. None

✗ All five rows as a list

Wrong. That is `fetchall()`. `fetchone` gets one.

✓ The first row, as a tuple

Correct. `fetchone()` hands back the next row as a tuple, and returns `None` once the rows run out.

✗ The number 5

Wrong. Counting rows is done in SQL, with `COUNT(*)`.

✗ None

Wrong. `None` comes back only when there are no rows left, and here there are five.

The answer is The first row, as a tuple. `fetchone` next row or `None`, `fetchall` the rest as a list of tuples. A row is a tuple, so index it by position.

Which SQL function returns the number of rows matching a query?

- a. SUM()
- b. TOTAL()
- c. COUNT()
- d. LEN()

✗ SUM()

Wrong. SUM() adds up the values in a column. It answers how much, not how many.

✗ TOTAL()

Wrong. There is no standard TOTAL() in this course.

✓ COUNT()

Correct. COUNT(*) counts rows; COUNT(column) counts the rows where that column is not null.

✗ LEN()

Wrong. LEN() is not SQL. Python has len(); SQL has COUNT().

The answer is COUNT(). COUNT how many, SUM the total, AVG the mean, MAX and MIN the extremes. The paper has asked for all five.

What does `UPDATE students SET score = 0` do, with no `WHERE` clause?

- a. Nothing: a `WHERE` clause is required
- b. It sets score to 0 for the first row only
- c. It deletes the score column
- d. It sets score to 0 for EVERY row in the table

✗ Nothing: a `WHERE` clause is required

Wrong. `WHERE` is optional, and that is precisely the danger.

✗ It sets score to 0 for the first row only

Wrong. SQL is set-based: it acts on every row that matches, and with no `WHERE` every row matches.

✗ It deletes the score column

Wrong. Removing a column is `ALTER TABLE`. `UPDATE` changes values.

✓ It sets score to 0 for EVERY row in the table

Correct. With no `WHERE`, the statement applies to the whole table. The same is true of `DELETE FROM students`, which empties it.

The answer is It sets score to 0 for EVERY row in the table. No `WHERE` means every row. Write the `WHERE` first and the `SET` second if that helps you never forget it.

In tkinter, what does the `mainloop()` call do?

- a. It starts the event loop, which waits for and dispatches user actions
- b. It draws the window once and returns
- c. It creates the main window object
- d. It closes the application

✓ It starts the event loop, which waits for and dispatches user actions

Correct. `mainloop()` keeps the program running, watching for clicks and keystrokes and calling the handlers bound to them, until the window is closed.

✗ It draws the window once and returns

Wrong. Drawing once and returning would make the window appear and vanish immediately, which is exactly what happens if you forget `mainloop`.

✗ It creates the main window object

Wrong. Creating the window is `Tk()`. `mainloop` is what makes it live.

✗ It closes the application

Wrong. It is what keeps the application OPEN.

The answer is It starts the event loop, which waits for and dispatches user actions. `Tk()` makes the window, widgets are created and placed, `mainloop()` runs the event loop. Without the last one nothing stays on screen.

Which tkinter widget is used to collect a single line of typed text?

- a. Label
- b. Button
- c. Entry
- d. Canvas

✗ Label

Wrong. A Label DISPLAYS text. The user cannot type into it.

✗ Button

Wrong. A Button is clicked; it does not accept typing.

✓ Entry

Correct. Entry is the one-line text box, and its `get()` method reads what was typed.

✗ Canvas

Wrong. A Canvas is a drawing surface for shapes and images.

The answer is Entry. Label shows, Entry takes one line, Text takes many, Button acts. Read what was typed with `entry.get()`.

In `Button(root, text="Go", command=run)`, when is `run` called?

- a. Immediately, as the button is created
- b. When `mainloop()` starts
- c. Never, unless `bind()` is also used
- d. When the button is clicked

✗ Immediately, as the button is created

Wrong, and this is the trap. It WOULD be immediate if you wrote `command=run()`, with brackets: that calls the function and hands its return value to `command`.

✗ When `mainloop()` starts

Wrong. `mainloop` starts the waiting; it does not fire the handlers by itself.

✗ Never, unless `bind()` is also used

Wrong. `command=` is a complete way to attach a handler to a button. `bind()` is for other events.

✓ When the button is clicked

Correct. `command` is given the function ITSELF, without brackets, and `tkinter` calls it later, each time the button is pressed.

The answer is When the button is clicked. Pass the function, not a call to it. `command=run` attaches it; `command=run()` runs it once immediately and attaches its result.

Which SQL column type suits a score that can have a decimal part?

- a. REAL
- b. TEXT
- c. BLOB
- d. INTEGER

✓ REAL

Correct. REAL is SQL's floating point type.

✗ TEXT

Wrong. TEXT holds strings.

✗ BLOB

Wrong. BLOB holds raw binary data.

✗ INTEGER

Wrong. INTEGER holds whole numbers only.

| The four SQL types: INTEGER, TEXT, REAL, BLOB.

Why write CREATE TABLE IF NOT EXISTS rather than plain CREATE TABLE?

- a. So the columns get types automatically
- b. So commit() is not needed
- c. So running the program a second time does not fail because the table is already there
- d. So the table is emptied each time

X So the columns get types automatically

Wrong. You still write the column types.

X So commit() is not needed

Wrong. commit() is still needed to keep inserted rows.

✓ So running the program a second time does not fail because the table is already there

Correct. Without it, the second run fails, and the paper asks for tables created 'if they do not already exist'.

X So the table is emptied each time

Wrong. It does not empty anything.

| IF NOT EXISTS makes the create safe to run again.

You are laying out a form of labels beside entry boxes. Which geometry manager suits it?

- a. `place()`, because it uses exact pixels
- b. `grid()`, because a form is rows and columns
- c. `pack()`, because it stacks widgets
- d. `mainloop()`

✗ `place()`, because it uses exact pixels

Wrong. `place` works, but fixing every pixel is fragile.

✓ `grid()`, because a form is rows and columns

Correct. `grid` places widgets by row and column numbers, which is what a form is.

✗ `pack()`, because it stacks widgets

Wrong. `pack` stacks widgets in the order added; it does not line up columns.

✗ `mainloop()`

Wrong. `mainloop` starts the event loop; it does not arrange anything.

| `pack` stacks, `grid` uses rows and columns, `place` uses exact x and y.

Which geometry manager positions a widget at exact x and y coordinates?

- a. grid()
- b. pack()
- c. place()
- d. config()

✗ grid()

Wrong. grid uses rows and columns.

✗ pack()

Wrong. pack stacks widgets.

✓ place()

Correct. place uses exact pixel coordinates.

✗ config()

Wrong. config changes a widget's settings.

! The three geometry managers: pack, grid and place.

Which widget gives the user a MULTI-line area to type in?

- a. Button
- b. Label
- c. Text
- d. Entry

✗ Button

Wrong. A Button is clicked, not typed in.

✗ Label

Wrong. A Label displays text the user cannot edit.

✓ Text

Correct. Text is a multi line typing area.

✗ Entry

Wrong. Entry takes a single line.

Entry for one line, Text for several, Label to show, Button to click.

What is a Frame used for?

- a. Starting the event loop
- b. Reading typed text
- c. Grouping other widgets together as a container
- d. Showing a picture only

✗ Starting the event loop

Wrong. That is `mainloop()`.

✗ Reading typed text

Wrong. That is `entry.get()`.

✓ Grouping other widgets together as a container

Correct. A Frame is a container that groups other widgets.

✗ Showing a picture only

Wrong. Not its purpose in the course.

Frame groups; Label shows; Entry and Text take typing; Button clicks.

Which two of these are tkinter geometry managers?

- a. pack()
- b. grid()
- c. mainloop()
- d. Entry()

✓ pack()

Correct. pack is a geometry manager.

✓ grid()

Correct. grid is a geometry manager.

✗ mainloop()

Wrong. mainloop starts the event loop.

✗ Entry()

Wrong. Entry is a widget.

| pack, grid and place arrange widgets; mainloop keeps the window alive.

5.17**EASY**

Lesson 3.6

How does a program read what the user typed into an Entry called entry?

- a. entry.mainloop()
- b. entry.get()
- c. entry.read()
- d. input(entry)

✗ entry.mainloop()

Wrong. mainloop belongs to the window.

✓ entry.get()

Correct. get() returns the typed text.

✗ entry.read()

Wrong. read() is for files.

✗ input(entry)

Wrong. input() reads the text console, not a widget.

| entry.get() reads what the user typed.

5.18**EASY**

Lesson 3.6

True or false: without window.mainloop(), the window appears and vanishes at once.

Answer: **true**

| No loop, no window: mainloop starts the event loop that waits for clicks and typing.

Why is `cur.execute("INSERT INTO t VALUES (?, ?)", (a, b))` preferred to building the SQL string with the values in it?

- a. It runs faster on small tables
- b. It is the only form sqlite3 accepts
- c. It commits automatically
- d. It keeps the values separate from the SQL, so a value cannot be read as SQL

✗ It runs faster on small tables

Wrong. Speed is not the reason, though prepared statements can help on repeated inserts.

✗ It is the only form sqlite3 accepts

Wrong. String building is accepted. It is accepted and unsafe, which is the point.

✗ It commits automatically

Wrong. You still have to commit yourself.

✓ It keeps the values separate from the SQL, so a value cannot be read as SQL

Correct. The placeholder keeps data as data. Pasting a value into the text of a statement lets a value containing a quote change the statement, which is SQL injection.

The answer is It keeps the values separate from the SQL, so a value cannot be read as SQL. The question mark placeholder separates data from code. It is the standard answer to "how do you prevent SQL injection".

What does `cur.executemany()` do that `execute()` does not?

- a. It runs the same statement once for each item in a sequence of parameter tuples
- b. It runs several DIFFERENT statements in one call
- c. It commits after every row
- d. It returns all the rows of a table

✓ It runs the same statement once for each item in a sequence of parameter tuples

Correct. You give it one statement and a list of parameter tuples, and it runs the statement once per tuple, which is how a batch insert is done.

✗ It runs several DIFFERENT statements in one call

Wrong. The statement is the same every time. Only the parameters change.

✗ It commits after every row

Wrong. It does not commit at all. You still call `commit` yourself.

✗ It returns all the rows of a table

Wrong. Retrieving rows is `fetchall()`, after a `SELECT`.

The answer is It runs the same statement once for each item in a sequence of parameter tuples. One statement, many parameter sets. It is the tidy way to insert a list of records and this manual teaches it because the paper reached for it.

Which of these is NOT a tkinter geometry manager?

- a. align
- b. pack
- c. grid
- d. place

✓ align

Correct. There is no align manager. The three are pack, grid and place, and you should not mix pack and grid in the same container.

✗ pack

Wrong choice, because pack IS one: it stacks widgets in the space available.

✗ grid

Wrong choice, because grid IS one: it places widgets by row and column.

✗ place

Wrong choice, because place IS one: it positions by exact coordinates.

The answer is align. pack, grid, place. Three of them, and mixing pack with grid in one container makes the window hang while they argue.

Asked twice. This same question also appears as: Lesson 3.5 and Lesson 3.5 and Lesson 3.5.

The same table. What does this print?

```
import sqlite3
conn = sqlite3.connect(":memory:")
cur = conn.cursor()
cur.execute("CREATE TABLE t (name TEXT, score REAL)")
cur.executemany("INSERT INTO t VALUES (?, ?)", [("Ada", 70), ("Ben", 55), ("Cy", 82)])
conn.commit()

cur.execute("UPDATE t SET score = 50")
cur.execute("SELECT score FROM t")
print(cur.fetchall())
```

- a. [(50.0,), (50.0,), (50.0,)]
- b. [(70.0,), (55.0,), (82.0,)]
- c. [(50.0,), (55.0,), (82.0,)]
- d. [(50.0,)]

✓ [(50.0,), (50.0,), (50.0,)]

Correct. With no WHERE, UPDATE changes every row in the table.

✗ [(70.0,), (55.0,), (82.0,)]

Wrong. The UPDATE ran and changed the scores.

✗ [(50.0,), (55.0,), (82.0,)]

Wrong. Nothing limits the UPDATE to the first row.

✗ [(50.0,)]

Wrong. All three rows are still there, all changed.

WHERE or everywhere: UPDATE and DELETE without WHERE apply to every row.

Asked twice. This same question also appears as: Lesson 3.5 and Lesson 3.5 and Lesson 3.5.

The same table. What does this print?

```
import sqlite3
conn = sqlite3.connect(":memory:")
cur = conn.cursor()
cur.execute("CREATE TABLE t (name TEXT, score REAL)")
cur.executemany("INSERT INTO t VALUES (?, ?)", [("Ada", 70), ("Ben", 55), ("Cy", 82)])
conn.commit()

cur.execute("DELETE FROM t WHERE score < ?", (60,))
cur.execute("SELECT COUNT(*) FROM t")
print(cur.fetchone()[0])
```

a. 1

b. 3

c. 0

d. 2

X 1

Wrong. Ada and Cy both stay.

X 3

Wrong. The DELETE removed Ben.

X 0

Wrong. WHERE limits the DELETE to matching rows.

✓ 2

Correct. Only Ben's row has a score below 60, so one row goes and two remain.

| DELETE FROM t WHERE ... removes only the matching rows.

Asked twice. This same question also appears as: Lesson 3.5 and Lesson 3.5 and Lesson 3.5.

The same table. What does this print?

```
import sqlite3
conn = sqlite3.connect(":memory:")
cur = conn.cursor()
cur.execute("CREATE TABLE t (name TEXT, score REAL)")
cur.executemany("INSERT INTO t VALUES (?, ?)", [("Ada", 70), ("Ben", 55), ("Cy", 82)])
conn.commit()

cur.execute("SELECT name FROM t WHERE score > 100")
print(cur.fetchone())
```

- a. ()
- b. []
- c. None
- d. an error

X ()

Wrong. It returns None, not an empty tuple.

X []

Wrong. That is what fetchall() returns for no rows.

✓ None

Correct. fetchone() returns None when there is no row to give.

X an error

Wrong. An empty result is not an error.

| fetchone() gives the next row as a tuple, or None when the rows run out.

What does the course warn happens if you mix `pack()` and `grid()` in the same container?

- a. The window closes immediately
- b. The widgets are drawn twice
- c. Nothing; they combine neatly
- d. The program can freeze

✗ The window closes immediately

Wrong. That is what happens without `mainloop()`.

✗ The widgets are drawn twice

Wrong. Not what the course says.

✗ Nothing; they combine neatly

Wrong. That is exactly what the rule forbids.

✓ The program can freeze

Correct. The rule is never to mix `pack` and `grid` in one container.

Two rules: no `mainloop`, no window; and never mix `pack` with `grid` inside the same container.

The table `t` holds Ada 70, Ben 55 and Cy 82. What does this print?

```
import sqlite3
conn = sqlite3.connect(":memory:")
cur = conn.cursor()
cur.execute("CREATE TABLE t (name TEXT, score REAL)")
cur.executemany("INSERT INTO t VALUES (?, ?)", [("Ada", 70), ("Ben", 55), ("Cy", 82)])
conn.commit()

cur.execute("SELECT name FROM t WHERE score > ? ORDER BY score DESC", (60,))
print(cur.fetchall())
```

- a. [('Ada',), ('Cy',)]
- b. [('Cy',), ('Ada',)]
- c. [('Cy',), ('Ada',), ('Ben',)]
- d. ['Cy', 'Ada']

✗ [('Ada',), ('Cy',)]

Wrong. ORDER BY score DESC puts 82 before 70.

✓ [('Cy',), ('Ada',)]

Correct. Only scores above 60 match, ordered highest first, and each row comes back as a one-item tuple.

✗ [('Cy',), ('Ada',), ('Ben',)]

Wrong. Ben's 55 fails the WHERE.

✗ ['Cy', 'Ada']

Wrong. fetchall() returns a list of TUPLES.

fetchall() gives every matching row as a list of tuples, even when each row has one column.

Asked twice. This same question also appears as: Lesson 3.5 and Lesson 3.5 and Lesson 3.5.

The same table. What does this print?

```
import sqlite3
conn = sqlite3.connect(":memory:")
cur = conn.cursor()
cur.execute("CREATE TABLE t (name TEXT, score REAL)")
cur.executemany("INSERT INTO t VALUES (?, ?)", [("Ada", 70), ("Ben", 55), ("Cy", 82)])
conn.commit()

cur.execute("SELECT COUNT(*), MAX(score), AVG(score) FROM t")
print(cur.fetchone())
```

- a. 3 82.0 69.0
- b. (3, 82, 69)
- c. [(3, 82.0, 69.0)]
- d. (3, 82.0, 69.0)

✗ 3 82.0 69.0

Wrong. The row is printed as a tuple.

✗ (3, 82, 69)

Wrong. score is REAL, so MAX and AVG are floats.

✗ [(3, 82.0, 69.0)]

Wrong. fetchone() gives one row, not a list.

✓ (3, 82.0, 69.0)

Correct. The three functions come back as ONE row, a tuple, and the REAL column gives floats.

█ COUNT(*), MAX() and AVG() in one query return a single row, collected with fetchone().

Asked twice. This same question also appears as: Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 2.3 and Lesson 2.4 and Lesson 3.2 and Lesson 3.4.

What happens when this runs?

```
import sqlite3
conn = sqlite3.connect(":memory:")
cur = conn.cursor()
cur.execute("CREATE TABLE t (name TEXT, score REAL)")
cur.executemany("INSERT INTO t VALUES (?, ?)", [("Ada", 70), ("Ben", 55), ("Cy", 82)])
conn.commit()

cur.execute("SELECT * FROM t WHERE name = ?", ("Ada"))
print(cur.fetchone())
```

- a. It prints every row
- b. It prints Ada's row
- c. An error, because ("Ada") is only a string; the placeholder needs the tuple ("Ada",)
- d. It prints None

✗ It prints every row

Wrong. The query fails.

✗ It prints Ada's row

Wrong. The parameters are wrong, so the query never runs.

✓ An error, because ("Ada") is only a string; the placeholder needs the tuple ("Ada",)

Correct. Without the comma the brackets are just grouping, so SQLite receives three characters instead of one value.

✗ It prints None

Wrong. The error happens before any fetch.

| ("SS2") is a string; ("SS2",) is a tuple, and the placeholder needs the tuple.

Every question, solved and explained

How this is arranged

All **225 questions** from the lecture slides, the reference sheets and both real tests. Grouped by topic, and within each topic ordered **easy first**, so the definitions come before the two-step calculations that assume them.

Every question carries the source it came from. Where the same question was set twice, it appears once and says where else it was asked. The answer is marked, and every other option has a line saying why it fails.

1 The Language, the Toolchain and Objects

1.1

EASY

Manual O.2 Q1

What does `javac` produce, and what runs it?

- a. Machine code for your processor, run directly by the operating system
- b. A `.java` file, run by `javac` itself
- c. An executable `.exe` file, run by double clicking it
- d. Bytecode in a `.class` file, run by the Java Virtual Machine

✗ Machine code for your processor, run directly by the operating system

Wrong. Compiling straight to machine code is the C model, and it is exactly what Java avoids, because machine code runs on one kind of processor only.

✗ A `.java` file, run by `javac` itself

Wrong. The `.java` file is what you WRITE and what `javac` consumes. It is the input, not the output.

✗ An executable `.exe` file, run by double clicking it

Wrong. Java deliberately does not produce a platform executable. That is the whole point of write once, run anywhere.

✓ Bytecode in a `.class` file, run by the Java Virtual Machine

Correct. `javac` turns your `.java` source into bytecode in a `.class` file, and the `java` command starts a JVM that executes that bytecode. The JVM is what makes the same `.class` file run on any machine that has one.

The answer is Bytecode in a `.class` file, run by the Java Virtual Machine. Source to bytecode by `javac`, bytecode to behaviour by the JVM. Java is compiled AND interpreted, and saying only one of those loses half the mark.

Which signature does the JVM look for to start a program?

- a. `public static void main(String[] args)`
- b. `public void main(String[] args)`
- c. `static void Main(String[] args)`
- d. `public static int main(String args)`

✓ `public static void main(String[] args)`

Correct. `public` so the JVM can reach it, `static` so it runs with no object, `void` because it returns nothing, and a `String` array for the command line arguments.

✗ `public void main(String[] args)`

Wrong. Without `static` the JVM would need an object to call it on, and no object exists yet when the program starts.

✗ `static void Main(String[] args)`

Wrong on two counts: Java is case sensitive so `Main` is a different name, and without `public` the JVM cannot reach it.

✗ `public static int main(String args)`

Wrong on two counts: `main` returns `void`, and the parameter is an `ARRAY` of `String`, not a single `String`.

The answer is `public static void main(String[] args)`. `public static void main(String[] args)`. Write it out by hand until it is automatic; a paper answer that gets it wrong has failed before the first statement.

1.3**EASY**

Manual O.2 Q5

Asked twice. This same question also appears as: Manual O.10 Q3.

Which statement is TRUE?

- a. The JVM contains the JRE, which contains the JDK
- b. The JDK and the JRE are two names for the same thing
- c. The JRE is needed to compile, the JDK only to run
- d. The JDK contains the JRE, which contains the JVM

✗ The JVM contains the JRE, which contains the JDK

Wrong, and it inverts the nesting. The smallest of the three is the JVM.

✗ The JDK and the JRE are two names for the same thing

Wrong. The JRE can only RUN programs. Only the JDK carries javac.

✗ The JRE is needed to compile, the JDK only to run

Wrong, and it swaps them: you compile with the JDK and can run with the JRE alone.

✓ The JDK contains the JRE, which contains the JVM

Correct. The JDK is the developer kit and holds the compiler; inside it is the runtime environment, and inside that is the virtual machine that actually executes bytecode.

The answer is The JDK contains the JRE, which contains the JVM. JDK contains JRE contains JVM.
Compile needs the JDK, run needs only the JRE.

[illegible]

What does this print?

```
System.out.print("A");  
System.out.println("B");  
System.out.print("C");
```

- a. A B
C
- b. AB
C
- c. A
B
C
- d. ABC

~~X~~ A B
C

Wrong. Nothing adds a space between A and B.

✓ AB
C

Correct. `print` stays on the line, `println` ends it after B, and the last `print` starts the new line.

X A

B

C

Wrong. Only println moves to a new line; print does not.

X ABC

Wrong. println("B") ends the line before C.

println prints and then moves to a new line; print leaves the cursor where it is.

1.5

EASY

Lesson 1.1

Asked twice. This same question also appears as: Manual O.10 Q2 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.6.

What happens when this is compiled?

```
System.out.println("Hi")
```

- a. It does not compile: the println statement has no semicolon
- b. It prints Hi with a warning
- c. It does not compile: the class line needs a semicolon
- d. It prints Hi

✓ It does not compile: the println statement has no semicolon

Correct. Every statement ends in a semicolon, and this one has none.

X It prints Hi with a warning

Wrong. A missing semicolon is an error, not a warning.

X It does not compile: the class line needs a semicolon

Wrong. A line ending in an opening brace is a heading, not a statement, and takes no semicolon.

X It prints Hi

Wrong. The missing semicolon stops compilation.

A missing semicolon is the single most common error in a handwritten script.

Which of these lines needs a semicolon at the end?

- a. `public class Hello {`
- b. `}`
- c. `int total = 0`
- d. `public static void main(String[] args) {`

X `public class Hello {`

Wrong. A line ending in an opening brace opens a block.

X `}`

Wrong. A closing brace ends a block.

✓ `int total = 0`

Correct. An assignment is a statement, and every statement ends in a semicolon.

X `public static void main(String[] args) {`

Wrong. The main line opens a block.

| Semicolons go on statements only: assignments, calls and returns.

A file holds public class ReportCard. What must the file be called?

- a. ReportCard.class
- b. reportcard.java
- c. Main.java
- d. ReportCard.java

✗ ReportCard.class

Wrong. .class is the compiled bytecode, not the source you write.

✗ reportcard.java

Wrong. The case must match: ReportCard.

✗ Main.java

Wrong. The file is named after the class.

✓ ReportCard.java

Correct. A public class lives in a file of exactly its own name.

| A public class must live in a file named after it, exactly, ending .java.

Asked twice. This same question also appears as: Manual O.10 Q2 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.6.

What happens when this is compiled?

```
system.out.println("hi");
```

- a. It prints hi
- b. It prints hi and a warning
- c. It compiles, then fails when it runs
- d. It does not compile, because Java is case sensitive and the class is System

✗ It prints hi

Wrong. The lower case s stops compilation.

✗ It prints hi and a warning

Wrong. It is an error, not a warning.

✗ It compiles, then fails when it runs

Wrong. The compiler catches it before anything runs.

✓ It does not compile, because Java is case sensitive and the class is System

Correct. system with a small s is not a name Java knows.

String, System and println: case is part of every name.

[illegible]

What does this print?

```
System.out.println("5 + 3");
System.out.println(5 + 3);
```

a. 8

8

b. 8

 $5 + 3$

c. $5 + 3$

 $5 + 3$

d. $5 + 3$

8

~~X~~ 8

8

Wrong. The quotes make the first line text.

X 8

5

Wrong. The lines are in the other order.

~~X~~ $5 + 3$

 $5 + 3$

Wrong. Without quotes, $5 + 3$ is calculated.

✓ $5 + 3$

8

Correct. In quotes it is text and is printed as written; without quotes it is worked out.

Anything in double quotes is a String, printed exactly; an expression outside quotes is evaluated.

1.10 EASY Lesson 1.1

True or false: a comment written after `//` is ignored by the compiler.

Answer: **true**

`//` comments to the end of the line; `/* ... */` comments across several lines. The paper asks for comments in Sections B and C.

1.11 EASY Lesson 1.2

Which file does the JVM actually read and run?

- a. Both files, one after the other
- b. `javac.exe`
- c. `Hello.java`, the source
- d. `Hello.class`, the bytecode

✗ Both files, one after the other

Wrong. Only the bytecode is run.

✗ `javac.exe`

Wrong. `javac` is the compiler that MAKES the `.class` file.

✗ `Hello.java`, the source

Wrong. The JVM never sees your source file.

✓ `Hello.class`, the bytecode

Correct. `javac` turns the source into bytecode in a `.class` file, and the JVM runs that.

Source, compiler, bytecode, virtual machine: `Hello.java`, `javac`, `Hello.class`, the JVM.

1.12**EASY**

Lesson 1.2

What makes Java platform independent?

- a. javac produces machine code for every processor
- b. Every operating system uses the same JVM
- c. The same bytecode runs on any machine that has a JVM
- d. Java has no compiler

X javac produces machine code for every processor

Wrong. javac produces bytecode, not machine code.

X Every operating system uses the same JVM

Wrong. The JVM is the one part that differs from platform to platform.

✓ The same bytecode runs on any machine that has a JVM

Correct. Write once, run anywhere: the bytecode never changes.

X Java has no compiler

Wrong. javac is the compiler.

| The bytecode is the same everywhere, and a platform specific JVM executes it.

1.13**EASY**

Lesson 1.2

True or false: the JVM is the one part of Java that differs from platform to platform.

Answer: true

| Platform independence works because only the JVM changes between machines.

1.14**EASY**

Lesson 1.2

Which feature means memory you stop using is reclaimed for you?

- a. Platform independence
- b. Automatic garbage collection
- c. Multithreading
- d. Strong type checking

✗ Platform independence

Wrong. That is about bytecode running anywhere.

✓ Automatic garbage collection

Correct. Garbage collection frees memory nothing refers to any more.

✗ Multithreading

Wrong. Multithreading is running tasks at the same time.

✗ Strong type checking

Wrong. That is what makes Java robust about types.

Java is simple, secure, robust, multithreaded, and garbage collected.

1.15**EASY**

Lesson 1.2

True or false: `javac` executes the bytecode.

Answer: **false**

Compiler makes it, virtual machine runs it.

Which statement best distinguishes a class from an object?

- a. A class is a running instance and an object is its definition
- b. A class is a blueprint; an object is an instance of it, with its own values
- c. A class and an object are two words for the same thing
- d. A class can have only one object at a time

X A class is a running instance and an object is its definition

Wrong, and it is exactly backwards. The definition is the class; the running instance is the object.

✓ A class is a blueprint; an object is an instance of it, with its own values

Correct. The class says what every instance will have and be able to do; each object made with new has its own copy of the fields.

X A class and an object are two words for the same thing

Wrong. If they were the same, new would have nothing to do.

X A class can have only one object at a time

Wrong. A class exists so that many objects can be created from it.

The answer is A class is a blueprint; an object is an instance of it, with its own values. Class is the blueprint, object is the building. new is what turns one into the other.

Java is described as an object-oriented language. Which TWO features below are the reason? Choose the single option naming BOTH correctly.

- a. Pointers and manual memory management
- b. Preprocessor directives and macros
- c. Encapsulation and inheritance
- d. Goto statements and labels

✗ Pointers and manual memory management

Wrong. Java deliberately has neither: references replace pointers and the garbage collector replaces manual freeing.

✗ Preprocessor directives and macros

Wrong. Those are C and C++ features. Java has no preprocessor at all.

✓ Encapsulation and inheritance

Correct. Bundling data with the methods that act on it, and letting one class extend another, are two of the four pillars, alongside abstraction and polymorphism.

✗ Goto statements and labels

Wrong. goto is a reserved word in Java that is not usable, kept reserved precisely so nobody writes one.

The answer is Encapsulation and inheritance. Encapsulation, inheritance, polymorphism, abstraction. Those four are the answer to any what-makes-it-object-oriented question.

A program compiles without error but refuses to start. Which main line explains it?

- a. `static public void main(String[] args)`
- b. `public static void main(String args[])`
- c. `public static void main(String[] args)`
- d. `public static void Main(String[] args)`

✗ `static public void main(String[] args)`

Wrong. The order of public and static does not matter.

✗ `public static void main(String args[])`

Wrong. `String args[]` is another legal way to declare the same array.

✗ `public static void main(String[] args)`

Wrong. That is the correct header.

✓ `public static void Main(String[] args)`

Correct. Main with a capital M is a different name, so there is no starting point, though it compiles.

Java is case sensitive. `main` is the name the JVM looks for; `Main` compiles and never runs.

How do the JDK, the JRE and the JVM fit together?

- a. The JVM contains the JRE, which contains the JDK
- b. The JRE contains the JDK and the JVM
- c. They are three unrelated programs
- d. The JDK contains the JRE, which contains the JVM

✗ The JVM contains the JRE, which contains the JDK

Wrong. This is upside down.

✗ The JRE contains the JDK and the JVM

Wrong. The JDK is the largest.

✗ They are three unrelated programs

Wrong. They nest.

✓ The JDK contains the JRE, which contains the JVM

Correct. Each adds to the one inside it: JDK adds javac, JRE adds the class libraries, JVM executes.

Draw three nested boxes: JDK around JRE around JVM.

Which two statements are true?

- a. The JDK adds development tools, principally javac
- b. The JVM compiles your source code
- c. The JVM reads your .java file
- d. The JRE is the JVM plus the standard class libraries

✓ The JDK adds development tools, principally javac

Correct. That is the JDK.

✗ The JVM compiles your source code

Wrong. javac compiles; the JVM executes.

✗ The JVM reads your .java file

Wrong. The JVM only ever sees the .class bytecode.

✓ The JRE is the JVM plus the standard class libraries

Correct. That is the JRE.

javac compiles to bytecode, the JVM runs bytecode. Getting those two the wrong way round is the commonest lost mark here.

A Java file has a syntax error on line 40. How much of the program runs?

- a. Lines 1 to 39
- b. Everything except line 40
- c. None of it; the compiler rejects the file before anything runs
- d. Everything, with a warning at line 40

✗ Lines 1 to 39

Wrong. That is how an interpreted language behaves with a runtime error.

✗ Everything except line 40

Wrong. A file that does not compile does not run at all.

✓ None of it; the compiler rejects the file before anything runs

Correct. Java is compiled, so the whole file is checked first.

✗ Everything, with a warning at line 40

Wrong. A syntax error is not a warning.

| A syntax error means nothing runs, not even the correct lines above it.

2 Types, Operators, Input and Output

2.1

EASY

Manual O.3 Q10

A program reads an integer with a Scanner named `in`. Which line is correct?

- a. `int n = in.nextInt();`
- b. `int n = in.next();`
- c. `int n = in.nextLine();`
- d. `int n = Scanner.nextInt();`

✓ `int n = in.nextInt();`

Correct. `nextInt()` reads the next token and returns it as an `int`, which is what the variable needs.

✗ `int n = in.next();`

Wrong. `next()` returns a `String`, and a `String` cannot be assigned to an `int` without conversion.

✗ `int n = in.nextLine();`

Wrong for the same reason: `nextLine()` also returns a `String`. It would need `Integer.parseInt` around it.

✗ `int n = Scanner.nextInt();`

Wrong. `nextInt` is an instance method, called on your `Scanner` object, not on the class.

The answer is `int n = in.nextInt();`. `nextInt` for an `int`, `nextDouble` for a `double`, `nextLine` for a whole line of text. Mixing `nextInt` with `nextLine` leaves the newline behind, which is the classic `Scanner` bug.

Which assignment compiles **WITHOUT** a cast?

- a. `int n = 3.5;`
- b. `int n = 7L;`
- c. `double d = 7;`
- d. `char c = "A";`

✗ `int n = 3.5;`

Wrong. A double into an int loses information, so Java requires you to say so with an explicit cast.

✗ `int n = 7L;`

Wrong. long to int is narrowing, so it needs a cast.

✓ `double d = 7;`

Correct. int to double is a **WIDENING** conversion: nothing can be lost, so Java does it silently.

✗ `char c = "A";`

Wrong. Double quotes make a String, and a String is not a char. A char literal uses single quotes.

The answer is `double d = 7;`. Widening is automatic, narrowing needs a cast. Remember the direction: small into big is safe, big into small is your responsibility.

Asked twice. This same question also appears as: Manual O.10 Q2 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.6.

What happens when this is compiled?

```
char grade = "A";  
System.out.println(grade);
```

- a. It prints 65
- b. It compiles and prints nothing
- c. It does not compile: "A" is a String, which does not fit in a char
- d. It prints A

✗ It prints 65

Wrong. It never compiles, so nothing prints.

✗ It compiles and prints nothing

Wrong. The type mismatch stops compilation.

✓ It does not compile: "A" is a String, which does not fit in a char

Correct. A char takes single quotes, 'A'.

✗ It prints A

Wrong. The double quotes make it a String.

Double quotes for a String, single quotes for a char. A value that does not fit its box is the planted error to look for.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
double d = 7;  
System.out.println(d);
```

- a. 7.0
- b. 7
- c. 7.00
- d. It does not compile

✓ 7.0

Correct. An int widens to a double automatically, and a double prints with a decimal point.

✗ 7

Wrong. d is a double, so it prints as 7.0.

✗ 7.00

Wrong. println shows 7.0.

✗ It does not compile

Wrong. Widening is always allowed.

Widening is free: byte to short to int to long to float to double. Narrowing needs a cast.

Which two of these are legal variable names?

- a. 2price
- b. price2
- c. class
- d. _total

✗ 2price

Wrong. A name may not start with a digit.

✓ price2

Correct. Digits are allowed after the first character.

✗ class

Wrong. class is a Java keyword.

✓ _total

Correct. A name may start with an underscore.

Letters, digits, _ and \$; not starting with a digit; no spaces; not a keyword.

Asked twice. This same question also appears as: Manual O.10 Q2 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.6.

What happens when this is compiled?

```
final int MAX = 5;  
MAX = 6;  
System.out.println(MAX);
```

- a. It prints 6
- b. It prints 11
- c. It does not compile: a final variable cannot be assigned again
- d. It prints 5

✗ It prints 6

Wrong. The reassignment is refused.

✗ It prints 11

Wrong. It never runs.

✓ It does not compile: a final variable cannot be assigned again

Correct. final means the value can never change after it is set.

✗ It prints 5

Wrong. It never runs.

final makes a constant, named in capitals: MAX, VAT.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
int x = 10;  
long y = x;  
System.out.println(y + 1);
```

- a. 101
- b. 11
- c. 11.0
- d. It does not compile

X 101

Wrong. + adds numbers; nothing here is text.

✓ 11

Correct. An int widens to a long automatically, and a long plus 1 is the whole number 11.

X 11.0

Wrong. long is a whole-number type.

X It does not compile

Wrong. int to long is widening, which is always allowed.

| Moving to a bigger box is silent and safe.

2.8

EASY

Lesson 1.3

True or false: `String` is one of Java's eight primitive types.

Answer: **false**

Every primitive is lower case; `String`, with its capital `S`, is a class.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
System.out.println(29 / 4 + " " + 29 % 4);
```

- a. 7.25 1
- b. 8 1
- c. 7 0.25
- d. 7 1

X 7.25 1

Wrong. Two ints divide as ints and the remainder is dropped.

X 8 1

Wrong. Integer division does not round up.

X 7 0.25

Wrong. % gives the remainder as a whole number, 1.

✓ 7 1

Correct. 29 / 4 between ints is 7, remainder 1.

Two ints divide as ints. The remainder is what % gives you.

Asked twice. This same question also appears as: Manual O.10 Q2 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.6.

What happens when this is compiled?

```
int x = 5;
if (x = 5) {
    System.out.println("five");
}
```

- a. It prints nothing
- b. It prints five, then x is 5
- c. It prints five
- d. It does not compile: if needs a boolean, and x = 5 is an assignment

✗ It prints nothing

Wrong. It never compiles.

✗ It prints five, then x is 5

Wrong. It never compiles.

✗ It prints five

Wrong. It never compiles.

✓ It does not compile: if needs a boolean, and x = 5 is an assignment

Correct. = assigns; == compares. Java refuses an int where if needs a boolean.

Writing if (x = 5) will not compile in Java, a small mercy.

Which import does a JOptionPane program need?

- a. `import java.util.JOptionPane;`
- b. `import javax.swing.JOptionPane;`
- c. `import java.swing.JOptionPane;`
- d. `import javax.util.Scanner;`

✗ `import java.util.JOptionPane;`
Wrong. `java.util` is Scanner's package.

✓ `import javax.swing.JOptionPane;`
Correct. `javax`, with the x, then `swing`.

✗ `import java.swing.JOptionPane;`
Wrong. It is `javax`, not `java`.

✗ `import javax.util.Scanner;`
Wrong. Scanner is `java.util.Scanner`.

| `java.util.Scanner`, `javax.swing.JOptionPane`: one letter, whole import.

The user types 42 into a `JOptionPane.showInputDialog`. What does the call return?

- a. The int 42
- b. Nothing; it only shows a window
- c. The String "42"
- d. The double 42.0

✗ The int 42

Wrong. You must convert it with `Integer.parseInt`.

✗ Nothing; it only shows a window

Wrong. `showInputDialog` returns what was typed.

✓ The String "42"

Correct. A dialog only ever hands back text.

✗ The double 42.0

Wrong. It is text until you parse it.

| A dialog always gives you text, so parse it: `Integer.parseInt(t)` or `Double.parseDouble(t)`.

Which line should end a JOptionPane program, so it stops when main ends?

- a. `System.out.println();`
- b. `System.exit(0);`
- c. `JOptionPane.close();`
- d. `return 0;`

✗ `System.out.println();`

Wrong. That only prints a blank line.

✓ `System.exit(0);`

Correct. A window program does not stop on its own; `System.exit(0)` ends it.

✗ `JOptionPane.close();`

Wrong. There is no such call.

✗ `return 0;`

Wrong. `main` is void, so it returns no value.

Finish a dialog program with `System.exit(0)`.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
System.out.printf("%d items at %.2f%n", 3, 2.5);
```

- a. %d items at %.2f
- b. 3.00 items at 2.50
- c. 3 items at 2.5
- d. 3 items at 2.50

X %d items at %.2f

Wrong. The placeholders are replaced by the values.

X 3.00 items at 2.50

Wrong. %d is a whole number.

X 3 items at 2.5

Wrong. %.2f always shows two decimal places.

✓ 3 items at 2.50

Correct. %d takes the whole number 3 and %.2f shows 2.5 to two decimal places.

| %d for whole numbers, %.2f for money, %s for text, %n for the new line.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
System.out.println((int) 7.89);  
System.out.println(Math.round(7.89));
```

a. 7

7

b. 8

8

c. 7

8

d. 7.89

8

X 7

7

Wrong. Math.round(7.89) rounds up to 8.

X 8

8

Wrong. The cast does not round.

✓ 7

8

Correct. A cast to int chops off the decimal part; Math.round rounds to the nearest.

✗ 7.89

8

Wrong. (int) produces a whole number.

| (int) 7.89 is 7: a cast chops. For proper rounding use Math.round.

2.16

MEDIUM

Lesson 1.3

Asked twice. This same question also appears as: Manual O.10 Q2 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.6.

What happens when this is compiled?

```
float f = 1.5;  
System.out.println(f);
```

- a. It prints 1.5
- b. It prints 1.0
- c. It does not compile: 1.5 is a double; write 1.5f
- d. It prints 2

✗ It prints 1.5

Wrong. The literal does not fit the float box as written.

✗ It prints 1.0

Wrong. It does not compile.

✓ It does not compile: 1.5 is a double; write 1.5f

Correct. A decimal literal is a double, and a double does not fit a float without the f suffix.

✗ It prints 2

Wrong. It does not compile.

| Two suffixes to remember: a long literal ends in L, a float literal ends in f.

2.17

MEDIUM

Lesson 1.3

How many bytes does a char take in Java?

- a. 4
- b. 2
- c. 1
- d. It is not defined

✗ 4

Wrong. Four bytes is an int or a float.

✓ 2

Correct. Java characters are Unicode, two bytes.

✗ 1

Wrong. One byte is a byte.

✗ It is not defined

Wrong. It is boolean whose size is not defined.

| byte 1, short 2, int 4, long 8, float 4, double 8, char 2, boolean undefined.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
System.out.println("x" + 2 + 3);  
System.out.println(2 + 3 + "x");
```

- a. x5
5x
- b. x23
5x
- c. x5
23x
- d. x23
23x

X x5
5x

Wrong. Once text is on the left, every later + joins.

✓ x23
5x

Correct. + works left to right: "x" + 2 is already text, so 3 joins too; 2 + 3 are both numbers, so they add before meeting "x".

X x5
23x

Wrong. Both lines are the other way round.

X x23
23x

Wrong. 2 + 3 adds before the text is reached.

| If either side of + is a String, + joins. Bracket arithmetic that sits next to text.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
int n = 0;
if (n != 0 && 10 / n > 1) {
    System.out.println("big");
} else {
    System.out.println("safe");
}
```

- a. big
- b. safe
- c. an ArithmeticException
- d. It does not compile

X big

Wrong. The condition is false.

✓ safe

Correct. `n != 0` is false, so `&&` stops and the division is never attempted.

X an ArithmeticException

Wrong. The guard on the left prevents the division by zero.

X It does not compile

Wrong. It compiles.

| `&&` stops at the first false, so put the guard on the left: `n != 0 && total / n > 1`.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
System.out.println(10 - 2 * 3 + 4 / 2);
```

- a. 6
- b. -2
- c. 14
- d. 8

✓ 6

Correct. * and / first: $2 * 3 = 6$ and $4 / 2 = 2$; then $10 - 6 + 2 = 6$.

✗ -2

Wrong. The division gives 2, which is added.

✗ 14

Wrong. That works left to right ignoring precedence.

✗ 8

Wrong. Check the subtraction: $10 - 6$ is 4, plus 2 is 6.

Precedence: brackets, unary and casts, * / %, then + -, left to right.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
int x = 5;  
x += 3;  
x *= 2;  
x %= 5;  
System.out.println(x);
```

- a. 0
- b. 3
- c. 16
- d. 1

X 0

Wrong. 16 is not a multiple of 5.

X 3

Wrong. Work each line in turn: 8, 16, 1.

X 16

Wrong. The last line takes the remainder.

✓ 1

Correct. $5 + 3 = 8$, times 2 is 16, and $16 \% 5$ leaves 1.

$x += 3$ means $x = x + 3$; the same for $*=$ and $\%=$.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
String t = "42";  
int n = Integer.parseInt(t);  
System.out.println(n + 1);  
System.out.println(t + 1);
```

- a. 421
421
- b. 43
421
- c. 43
43
- d. It does not compile

X 421
421

Wrong. n is a number after parseInt.

✓ 43
421

Correct. n is the number 42, so n + 1 is 43; t is still the text "42", so t + 1 joins to 421.

✗ 43

43

Wrong. t was never converted.

✗ It does not compile

Wrong. Both lines are legal.

Integer.parseInt turns text into a whole number; the original String is unchanged.

2.23

MEDIUM

Lesson 1.5

Asked twice. This same question also appears as: Lesson 2.2 and Lesson 2.8 and Lesson 3.1 and Lesson 3.5 and Lesson 3.6.

What happens when this runs?

```
int n = Integer.parseInt("4.5");  
System.out.println(n);
```

a. It prints 5

b. It prints 4.5

c. It throws NumberFormatException, because 4.5 is not a whole number

d. It prints 4

✗ It prints 5

Wrong. parseInt does not round.

✗ It prints 4.5

Wrong. That needs Double.parseDouble.

✓ It throws NumberFormatException, because 4.5 is not a whole number

Correct. parseInt accepts only whole-number text.

✗ It prints 4

Wrong. parseInt does not chop; it refuses.

Integer.parseInt("4.5") fails; Double.parseDouble("4.5") gives 4.5.

Asked twice. This same question also appears as: Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
public class Q301 {  
    public static void main(String[] args) {  
        System.out.println(7 / 2);  
        System.out.println(7 % 2);  
        System.out.println(7.0 / 2);  
    }  
}
```

a. 3.5

1

3.5

b. 3

1

3

c. 3

1

3.5

d. 3.5

3.5

3.5

✗ 3.5

1

3.5

Wrong on the first line. Integer division truncates in Java; it does not produce a decimal. That is the single most common Java arithmetic error.

✗ 3

1

3

Wrong on the last line. 7.0 is a double, so the division is done in doubles.

✓ 3

1

3.5

Correct. Two ints divide as ints, so the fraction is thrown away and $7 / 2$ is 3. The remainder is 1. Making either operand a double promotes the whole expression, so $7.0 / 2$ is 3.5.

✗ 3.5

3.5

3.5

Wrong throughout. % is the remainder operator, not a division.

The answer is 3 then 1 then 3.5. int divided by int gives int, and the fraction is discarded, not rounded. One double operand promotes the whole expression.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
public class Q302 {  
    public static void main(String[] args) {  
        System.out.println(-7 / 2);  
        System.out.println(-7 % 2);  
    }  
}
```

a. -4

1

b. -4

-1

c. -3

1

d. -3

-1

X -4

1

Wrong. Java truncates towards ZERO, so $-7 / 2$ is -3, not -4. Flooring to -4 is what Python does, and the difference is a real exam trap.

X -4

-1

Wrong on the division for the same reason: Java truncates towards zero.

X -3

1

Wrong on the remainder. In Java the result of % has the sign of the dividend, so a negative dividend gives a negative remainder.

✓ -3

-1

Correct. Division truncates towards zero giving -3, and the remainder takes the sign of the LEFT operand, giving -1.

The answer is -3 then -1. Java truncates towards zero and % takes the sign of the left operand. Both differ from some other languages, so this is worth knowing exactly rather than by feel.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
public class Q303 {  
    public static void main(String[] args) {  
        char c = 'A';  
        System.out.println(c + 1);  
        System.out.println((char) (c + 1));  
    }  
}
```

a. 66

B

b. B

B

c. A1

B

d. 66

66

✓ 66

B

Correct. `c + 1` promotes the char to its code, 65, and adds 1 to give 66. The cast on the second line turns 66 back into a char, which prints as B.

X B

B

Wrong on the first line. A char in arithmetic is promoted to int, so the result is a number, not a letter.

X A1

B

Wrong. Plus between a char and an int is ARITHMETIC, not joining. Joining would need one side to be a String.

X 66

66

Wrong on the second line: the cast to char is what makes it print as a letter.

The answer is 66 then B. A char is a number with a costume on. Arithmetic strips the costume; a cast back to char puts it on again.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
public class Q304 {  
    public static void main(String[] args) {  
        System.out.println("Total: " + 1 + 2);  
        System.out.println("Total: " + (1 + 2));  
    }  
}
```

- a. Total: 3
Total: 3
- b. Total: 12
Total: 3
- c. Total: 12
Total: 12
- d. Total: 3
Total: 12

X Total: 3
Total: 3

Wrong on the first line. Nothing adds 1 and 2 there: by the time the 2 is reached the left side is already a String.

✓ Total: 12

Total: 3

Correct. Plus groups left to right. The first plus has a String on its left, so it JOINS, producing "Total: 1", and the next plus joins the 2 onto that. The brackets on the second line force the addition first.

✗ Total: 12

Total: 12

Wrong on the second line. The brackets change the answer, which is the whole point of the question.

✗ Total: 3

Total: 12

Wrong, and it has the two lines swapped.

The answer is Total: 12 then Total: 3. Plus is left-associative, and once a String is involved every later plus joins. Bracket the arithmetic and this whole class of bug goes away.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
public class Q305 {  
    public static void main(String[] args) {  
        int big = Integer.MAX_VALUE;  
        System.out.println(big);  
        System.out.println(big + 1);  
    }  
}
```

- a. 2147483647
2147483648
- b. 2147483647
error at run time
- c. 2147483647
-2147483648
- d. 2147483648
-2147483648

X 2147483647
2147483648

Wrong. An int cannot hold 2147483648. The type does not grow to fit.

✗ 2147483647

error at run time

Wrong. There is no exception for integer overflow in Java. That silence is exactly what makes it dangerous.

✓ 2147483647

-2147483648

Correct. int is 32 bits, so its maximum is 2147483647, and adding 1 wraps round to the most negative value. Java does not report this: it just gives the wrong number quietly.

✗ 2147483648

-2147483648

Wrong on the first line. MAX_VALUE is 2147483647, one less than 2 to the power 31.

The answer is 2147483647 then -2147483648. int holds -2147483648 to 2147483647, and overflow wraps silently. Use long when a value might exceed it.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
public class Q306 {  
    public static void main(String[] args) {  
        double a = 0.1 + 0.2;  
        System.out.println(a);  
        System.out.println(a == 0.3);  
    }  
}
```

- a. 0.3
true
- b. 0.3
false
- c. 0.30000000000000004
true
- d. 0.30000000000000004
false

X 0.3
true

Wrong on both lines. Binary floating point cannot hold 0.1 or 0.2 exactly, so their sum is not exactly 0.3.

✗ 0.3
false

Wrong on the first line: println shows the true stored value, tiny error and all.

✗ 0.30000000000000004
true

Wrong on the second line: if the value printed is not 0.3, it cannot equal 0.3.

✓ 0.30000000000000004
false

Correct. The stored sum is very slightly above 0.3 and prints in full, so the equality test is false.

The answer is 0.30000000000000004 then false. Never compare two doubles with ==. Compare the size of their difference against a small tolerance instead.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
public class Q307 {  
    public static void main(String[] args) {  
        int i = 5;  
        System.out.println(i++);  
        System.out.println(i);  
        System.out.println(++i);  
    }  
}
```

a. 5

6

7

b. 6

6

7

c. 5

5

6

d. 6

7

7

✓ 5
6
7

Correct. `i++` yields the OLD value 5 and then makes `i` 6. The second line shows that 6. `++i` increments first, to 7, and yields 7.

✗ 6
6
7

Wrong on the first line. The post-increment gives back the value BEFORE the increment.

✗ 5
5
6

Wrong on the second line. The increment did happen, so `i` is 6 by then.

✗ 6
7
7

Wrong on the first line, for the same reason as 6, 6, 7: the post-increment gives back the value BEFORE the increment, so the first line is 5.

The answer is 5 then 6 then 7. Post says use it then change it; pre says change it then use it. The variable ends up the same either way, and what the EXPRESSION gives back is what differs.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
public class Q308 {  
    public static void main(String[] args) {  
        System.out.println(2 + 3 * 4);  
        System.out.println((2 + 3) * 4);  
        System.out.println(10 - 2 - 3);  
    }  
}
```

a. 20

20

5

b. 14

20

5

c. 14

20

11

d. 14

14

5

✗ 20

20

5

Wrong on the first line. Multiplication binds tighter than addition without any brackets.

✓ 14

20

5

Correct. Multiply before add gives 14, brackets force 20, and subtraction groups left to right so $10 - 2$ is 8 and $8 - 3$ is 5.

✗ 14

20

11

Wrong on the last line. Left to right gives 5; grouping the right pair first would give 11, and no operator in Java does that.

✗ 14

14

5

Wrong on the second line: the brackets really do change it.

The answer is 14 then 20 then 5. Multiply and divide before add and subtract, then left to right. When in doubt, write the brackets yourself.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
public class Q309 {  
    static boolean shout(String s) {  
        System.out.println(s);  
        return true;  
    }  
    public static void main(String[] args) {  
        if (false && shout("left")) { }  
        if (true || shout("right")) { }  
        System.out.println("done");  
    }  
}
```

- a. left
right
done
- b. left
done
- c. done
- d. right
done

X left
right

done

Wrong. Neither call happens, which is the whole point of short circuiting.

✗ left

done

Wrong. false && anything is already decided, so the right side is skipped.

✓ done

Correct. && stops as soon as its left side is false, and || stops as soon as its left side is true, so neither shout is ever called.

✗ right

done

Wrong. true || anything is already decided, so the right side is skipped.

The answer is done. && and || evaluate the right side only when they must. That is what makes `if (s != null && s.length() > 0)` safe.

HARD

Manual O.3 Q11

[illegible]

What does this print?

```
public class Q311 {
    public static void main(String[] args) {
        System.out.printf("%.2f%n", 3.14159);
        System.out.printf("%5d|%n", 42);
    }
}
```

- a. 3.142
42 |
- b. 3.14
42|
- c. 3.14
42 |
- d. 3.1
42|

X 3.142
42 |

Wrong on both counts: .2f is two decimals, and %5d right-aligns.

✓ 3.14
42|

Correct. `%.2f` gives two decimal places, and `%5d` pads the number to five characters on the LEFT, so the spaces come first.

X 3.14
42 |

Wrong on the padding. A width without a minus sign right-aligns; `%-5d` would pad on the right.

X 3.1
42|

Wrong on the decimals: two were asked for, not one.

The answer is 3.14 then 42|. `%.2f` fixes the decimals, `%5d` fixes the width and right-aligns, `%-5d` left-aligns, and `%n` is the newline that works everywhere.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
int sum = 7, n = 2;  
System.out.println(sum / n);  
System.out.println((double) sum / n);  
System.out.println((double) (sum / n));
```

a. 3.5

3.5

3.5

b. 3

3

3

c. 3

3.5

3.5

d. 3

3.5

3.0

✗ 3.5

3.5

3.5

Wrong. The first line divides two ints.

✗ 3

3

3

Wrong. The second line casts sum before dividing.

✗ 3

3.5

3.5

Wrong. `(double)(sum / n)` divides first, in integers.

✓ 3

3.5

3.0

Correct. `sum / n` is integer division, 3; casting one side first gives 3.5; casting AFTER the division only converts the wrong 3 into 3.0.

Cast one operand, before the division: `(double) sum / n`. `(double)(sum / n)` converts an answer that is already wrong.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
int a = 3;
int b = a++ + ++a;
System.out.println(a + " " + b);
```

a. 5 7

b. 4 8

c. 5 8

d. 5 9

X 5 7

Wrong. `++a` increments to 5 before it is used.

X 4 8

Wrong. Both increments happen, so `a` ends at 5.

✓ 5 8

Correct. `a++` gives 3 then makes `a` a 4; `++a` makes `a` a 5 and gives 5; so `b` is $3 + 5 = 8$ and `a` is 5.

X 5 9

Wrong. `a++` contributes its OLD value, 3.

| Post uses the old value then adds one; pre adds one then uses the new value.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
System.out.printf("%.1f|%5d|%4s|\n", 2.46, 42, "ab");
```

a. 2.4| 42|ab |

b. 2.46|42|ab|

c. 2.5| 42|ab |

d. 2.5|42 | ab|

✗ 2.4| 42|ab |

Wrong. %.1f rounds 2.46 to 2.5.

✗ 2.46|42|ab|

Wrong. The widths and the precision change the output.

✓ 2.5| 42|ab |

Correct. %.1f rounds to one place, %5d pads 42 to five columns on the left, %-4s pads ab to four on the right.

✗ 2.5|42 | ab|

Wrong. Numbers pad on the left; the minus makes text pad on the right.

% starts a placeholder, the number sets width and decimals, the letter sets the type; %n ends the line.

The user types 5, presses Enter, then types Ada. What does this print?

```
Scanner in = new Scanner(System.in);  
int n = in.nextInt();  
String name = in.nextLine();  
System.out.println "[" + name + "];
```

- a. [5 Ada]
- b. [5]
- c. []
- d. [Ada]

✗ [5 Ada]

Wrong. `nextLine` reads only the rest of the current line.

✗ [5]

Wrong. `nextInt` already consumed the 5.

✓ []

Correct. `nextInt` reads 5 but leaves the newline behind, so `nextLine` returns the empty rest of that line.

✗ [Ada]

Wrong. `nextLine` never reaches Ada; it stops at the leftover newline.

Reading a number and then a line: `nextInt` leaves the newline, so add an extra `nextLine()` to swallow it.

3 Control Structures

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
int n = 90210, d = 0;
while (n > 0) {
    d++;
    n /= 10;
}
System.out.println(d);
```

- a. 5
- b. 90210
- c. 4
- d. 6

✓ 5

Correct. Five divisions by 10 bring 90210 to 0: 9021, 902, 90, 9, 0.

✗ 90210

Wrong. d counts; n is used up.

✗ 4

Wrong. The zeros inside the number still count.

X 6

Wrong. The loop stops as soon as n reaches 0.

A counting loop destroys what it counts, which is why a program copies n into $temp$ first.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
int a = 7, b = 9;  
int m = (a > b) ? a : b;  
System.out.println(m);
```

- a. 9
- b. 16
- c. true
- d. 7

✓ 9

Correct. $a > b$ is false, so the ternary gives the value after the colon, b.

✗ 16

Wrong. Nothing adds a and b.

✗ true

Wrong. The ternary gives one of the two values, not the test.

✗ 7

Wrong. The first value is chosen only when the test is true.

test ? valueIfTrue : valueIfFalse is an if-else squeezed into an expression.

Why does the dry-run table include the pass where the loop test FAILS?

- a. The body runs once more on that pass
- b. It prints an extra line
- c. The examiner does not want it
- d. It shows why the loop stopped, which is where off-by-one answers are caught

✗ The body runs once more on that pass

Wrong. The body does not run when the test fails.

✗ It prints an extra line

Wrong. Nothing is printed on that pass.

✗ The examiner does not want it

Wrong. Writing it is the habit that catches the error.

✓ It shows why the loop stopped, which is where off-by-one answers are caught

Correct. The failing test decides the answer between $<$ and $<=$.

■ A table, one row per pass, including the pass that fails the test.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
int score = 45;
char grade;
if (score >= 70) {
    grade = 'A';
} else if (score >= 60) {
    grade = 'B';
} else if (score >= 50) {
    grade = 'C';
} else if (score >= 45) {
    grade = 'D';
} else {
    grade = 'F';
}
System.out.println("Grade: " + grade);
```

- a. Grade: B
- b. Grade: C
- c. Grade: D
- d. Grade: F

X Grade: B

Wrong. 45 is far below 60.

X Grade: C

Wrong. 45 is not ≥ 50 .

✓ Grade: D

Correct. 45 fails 70, 60 and 50, then matches 45, and the ladder stops.

✗ Grade: F

Wrong. $45 \geq 45$ is true, so the else is never reached.

A ladder runs exactly one block: the first whose condition is true.

3.5

EASY Lesson 2.1

Which Java condition accepts a score from 0 to 100 inclusive?

- a. $0 \leq \text{score} \leq 100$
- b. $\text{score} > 0 \ \&\& \ \text{score} < 100$
- c. $\text{score} \geq 0 \ \&\& \ \text{score} \leq 100$
- d. $\text{score} \geq 0 \ || \ \text{score} \leq 100$

✗ $0 \leq \text{score} \leq 100$

Wrong. Java cannot chain comparisons; that does not compile.

✗ $\text{score} > 0 \ \&\& \ \text{score} < 100$

Wrong. That excludes 0 and 100.

✓ $\text{score} \geq 0 \ \&\& \ \text{score} \leq 100$

Correct. Both comparisons, joined with $\&\&$.

✗ $\text{score} \geq 0 \ || \ \text{score} \leq 100$

Wrong. $||$ is true for every number.

Java has no $0 \leq \text{score} \leq 100$: write both comparisons and join them.

Which control structure do if and switch belong to?

- a. Recursion
- b. Sequence
- c. Selection
- d. Iteration

✗ Recursion

Wrong. Not one of the three control structures.

✗ Sequence

Wrong. Sequence is statements one after another.

✓ Selection

Correct. Selection chooses between alternatives so that only one runs.

✗ Iteration

Wrong. Iteration is for, while and do-while.

Sequence, selection, iteration.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
int month = 1;
String season;
switch (month) {
    case 12: case 1: case 2:
        season = "Harmattan";
        break;
    case 6: case 7: case 8:
        season = "Rainy";
        break;
    default:
        season = "Other";
}
System.out.println(season);
```

- a. Other
- b. Harmattan
- c. Rainy
- d. It does not compile

X Other

Wrong. Month 1 matches a case.

✓ Harmattan

Correct. Cases 12 and 1 fall into case 2, which sets Harmattan and breaks.

✗ Rainy

Wrong. Month 1 is in the first group.

✗ It does not compile

Wrong. Empty cases sharing an answer are legal.

Deliberate fall-through is how several values share one answer.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
int day = 9;
double price;
switch (day) {
    case 1: price = 75.0; break;
    case 2: price = 60.0; break;
    case 3: price = 45.0; break;
    default: price = 0.0;
}
System.out.println("Price " + price);
```

- a. Nothing
- b. It does not compile
- c. Price 45.0
- d. Price 0.0

X Nothing

Wrong. The println after the switch always runs.

X It does not compile

Wrong. It compiles.

X Price 45.0

Wrong. Case 3 needs day to be 3.

✓ Price 0.0

Correct. 9 matches no case, so default sets the price to 0.0.

| default catches everything else. Always write one.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
String c = "B";
switch (c) {
    case "A": System.out.println("Excellent"); break;
    case "B": System.out.println("Good"); break;
    default: System.out.println("Fail");
}
```

- a. Excellent
Good
- b. It does not compile
- c. Good
- d. Fail

✗ Excellent
Good

Wrong. It jumps straight to the matching case.

✗ It does not compile

Wrong. Switching on a String is allowed.

✓ Good

Correct. A switch can take a String and matches "B" exactly.

X Fail

Wrong. "B" has its own case.

A switch compares one value against exact constants, and a String is allowed.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
int code = 7;
String label = "none";
switch (code) {
    case 1: label = "one"; break;
    case 2: label = "two"; break;
}
System.out.println(label);
```

- a. Nothing at all
- b. two
- c. one
- d. none

X Nothing at all

Wrong. The println after the switch still runs.

X two

Wrong. code is 7.

X one

Wrong. code is 7.

✓ none

Correct. Nothing matches and there is no default, so label keeps its starting value.

Without a default, an unmatched value does nothing at all.

3.11

EASY

Lesson 2.2

Grading a score into bands, 70 and above, 60 and above, and so on. Which structure?

- a. A do-while loop
- b. A switch with a case for every possible score
- c. An if-else ladder, because each band is a range
- d. A switch, because there are several grades

✗ A do-while loop

Wrong. Nothing here repeats.

✗ A switch with a case for every possible score

Wrong. Possible, but absurd, and not what the paper expects.

✓ An if-else ladder, because each band is a range

Correct. A case must be an exact value; bands are ranges.

✗ A switch, because there are several grades

Wrong. A switch cannot test a range.

Switch for exact values, ladder for ranges.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
int n = 10;
while (n < 5) {
    System.out.println(n);
    n++;
}
System.out.println("after");
```

- a. 10
after
- b. after
- c. It loops for ever
- d. Nothing

X 10
after

Wrong. A while tests first.

✓ after

Correct. $10 < 5$ is false at the start, so the while body never runs.

X It loops for ever

Wrong. The body never runs, so it cannot loop.

X Nothing

Wrong. The line after the loop still runs.

| A while may run zero times; a do-while always runs once.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
for (int i = 10; i > 0; i -= 3) {  
    System.out.print(i + " ");  
}
```

- a. 10 7 4
- b. 9 6 3
- c. 10 7 4 1 -2
- d. 10 7 4 1

X 10 7 4

Wrong. $1 > 0$, so 1 is printed.

X 9 6 3

Wrong. It starts at 10.

X 10 7 4 1 -2

Wrong. -2 fails $i > 0$.

✓ 10 7 4 1

Correct. Start 10, subtract 3 while $i > 0$: 10, 7, 4, 1.

| start; test; step: the test is made before every pass.

Which loop is guaranteed to run its body at least once?

- a. None of them
- b. do-while
- c. while
- d. for

✗ None of them

Wrong. The do-while always runs once.

✓ do-while

Correct. Its test comes after the body.

✗ while

Wrong. A while tests first and may never run.

✗ for

Wrong. A for tests first too.

| for counts, while waits, do-while waits but always goes once. A do-while ends with a semicolon.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
int count = 0;
for (int i = 3; i <= 12; i++) {
    count++;
}
System.out.println(count);
```

- a. 10
- b. 9
- c. 11
- d. 12

✓ 10

Correct. With $\leq$, a loop from 3 to 12 runs last minus first plus one times: $12 - 3 + 1 = 10$.

✗ 9

Wrong. $\leq$ includes 12.

✗ 11

Wrong. Count them: 3 to 12 is ten numbers.

✗ 12

Wrong. It starts at 3, not 1.

Count the passes before you trace: last minus first plus one, for a $\leq$ test.

3.16

EASY

Lesson 2.4

Where does the STEP of a for header go in the while version?

- a. Inside the while brackets
- b. On the last line inside the loop body
- c. Above the loop
- d. After the loop

✗ Inside the while brackets

Wrong. That is where the test goes.

✓ On the last line inside the loop body

Correct. Start above, test in the brackets, step at the bottom of the body.

✗ Above the loop

Wrong. That is where the start goes.

✗ After the loop

Wrong. After the loop it would run only once.

Start goes above. Test stays put. Step goes to the bottom of the body.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
int total = 0;
for (int i = 2; i <= 8; i += 3) {
    total += i;
}
System.out.println(total);
```

a. 10

b. 20

c. 15

d. 26

X 10

Wrong. 8 is still ≤ 8 , so it is added.

X 20

Wrong. The loop does not reach 11.

✓ 15

Correct. i takes 2, 5 and 8; at 11 the test fails. $2 + 5 + 8 = 15$.

X 26

Wrong. 11 fails the test and is never added.

Write the table: every pass, and the pass that fails the test.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
int n = 4071;
int s = 0;
while (n > 0) {
    s += n % 10;
    n /= 10;
}
System.out.println(s);
```

- a. 4071
- b. 12
- c. 11
- d. 4

X 4071

Wrong. The loop sums the digits; it does not rebuild the number.

✓ 12

Correct. The loop pulls off 1, 7, 0 and 4 with % 10 and removes each with / 10: $1 + 7 + 0 + 4 = 12$.

X 11

Wrong. Add every digit: $4 + 0 + 7 + 1$.

X 4

Wrong. That is how many digits there are.

$n \% 10$ is the last digit and $n /= 10$ removes it: together they walk a number right to left.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
int i;  
for (i = 0; i < 4; i++) {  
}  
System.out.println(i);
```

- a. 5
- b. It does not compile
- c. 3
- d. 4

X 5

Wrong. The loop stops as soon as i reaches 4.

X It does not compile

Wrong. i is declared before the loop, so it exists afterwards.

X 3

Wrong. The last step makes i 4 before the failing test.

✓ 4

Correct. i passes 0, 1, 2, 3; then i becomes 4, the test $4 < 4$ fails, and the loop ends with i at 4.

Always write the pass that fails the test: it is where off-by-one answers come from.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
int x = 1;
while (x < 50) {
    x *= 3;
}
System.out.println(x);
```

- a. 81
- b. 243
- c. 27
- d. 50

✓ 81

Correct. x goes 1, 3, 9, 27, 81; at 81 the test $x < 50$ fails.

✗ 243

Wrong. The loop stops at 81.

✗ 27

Wrong. 27 is still below 50, so it is tripled again.

✗ 50

Wrong. x only ever holds powers of 3.

The test happens before each pass; the last value is the first one that fails it.

3.21

MEDIUM

Lesson 2.1

Four separate ifs instead of a ladder. What does this print?

```
int score = 95;
char g = 'F';
if (score >= 70) g = 'A';
if (score >= 60) g = 'B';
if (score >= 50) g = 'C';
if (score >= 45) g = 'D';
System.out.println(g);
```

- a. D
- b. F
- c. ABCD
- d. A

✓ D

Correct. All four tests are made; 95 passes each, so each assignment overwrites the last and D is left.

✗ F

Wrong. At least one test matched.

✗ ABCD

Wrong. g holds one char, replaced each time.

✗ A

Wrong. A was set first, then overwritten three times.

Separate ifs are all tested and the last match wins. The paper asks for a ladder, and says so.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
String tier = "Silver";
int days = 2;
int discount;
if (tier.equals("Gold")) {
    if (days == 3) {
        discount = 25;
    } else {
        discount = 15;
    }
} else {
    if (days == 3) {
        discount = 10;
    } else {
        discount = 5;
    }
}
System.out.println("Discount " + discount + "%");
```

- a. Discount 10%
- b. Discount 5%
- c. Discount 15%
- d. Discount 25%

X Discount 10%

Wrong. 10 needs 3 days.

✓ Discount 5%

Correct. Silver is not Gold, so the outer else runs; days is not 3, so the inner else gives 5.

✗ Discount 15%

Wrong. 15 is the Gold path.

✗ Discount 25%

Wrong. 25 is Gold with 3 days.

The outer test picks a pair, the inner test picks one of that pair. Compare Strings with equals.

3.23

MEDIUM

Lesson 2.1

Asked twice. This same question also appears as: Manual O.10 Q2 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.6.

What happens when this is compiled?

```
int score = 50;
if (0 <= score <= 100) {
    System.out.println("ok");
}
```

- a. It prints true
- b. It does not compile: Java cannot chain comparisons like this
- c. It prints nothing
- d. It prints ok

✗ It prints true

Wrong. It never compiles.

✓ It does not compile: Java cannot chain comparisons like this

Correct. `0 <= score` gives a boolean, and a boolean cannot be compared with `<= 100`.

✗ It prints nothing

Wrong. It never compiles.

✗ It prints ok

Wrong. It never compiles.

Write `score >= 0 && score <= 100`.

3.24

MEDIUM

Lesson 2.1

True or false: without braces, an else attaches to the nearest unmatched if.

Answer: **true**

Braces on every if and every else, always.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
int day = 2;
switch (day) {
    case 1:
        System.out.println("one");
    case 2:
        System.out.println("two");
    case 3:
        System.out.println("three");
        break;
    case 4:
        System.out.println("four");
}
```

- a. one
two
three
- b. two
three
- c. two
- d. two
three
four

✗ one
two
three

Wrong. It jumps straight to case 2.

✓ two
three

Correct. It matches case 2, prints, falls into case 3, prints, and meets the break.

✗ two

Wrong. With no break after case 2 it falls into case 3.

✗ two
three
four

Wrong. The break after case 3 stops it.

■ A missing break falls through into the next case until a break or the closing brace.

Asked twice. This same question also appears as: Manual O.10 Q2 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 2.1 and Lesson 2.2 and Lesson 2.3 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.6.

What happens when this is compiled?

```
double price = 2.5;
switch (price) {
    case 2.5: System.out.println("cheap"); break;
    default: System.out.println("dear");
}
```

- a. It prints nothing
- b. It prints dear
- c. It does not compile: a switch cannot take a double
- d. It prints cheap

✗ It prints nothing

Wrong. It never compiles.

✗ It prints dear

Wrong. It never compiles.

✓ It does not compile: a switch cannot take a double

Correct. Switch on byte, short, int, char, String or an enum; never double or boolean.

✗ It prints cheap

Wrong. It never compiles.

■ You cannot switch on a price, only on something exactly comparable.

Asked twice. This same question also appears as: Lesson 1.5 and Lesson 2.8 and Lesson 3.1 and Lesson 3.5 and Lesson 3.6.

What happens when this runs?

```
int day = 5;
switch (day) {
    case 1: System.out.println(75); break;
    default: throw new IllegalArgumentException("Day must be 1, 2 or 3");
}
```

- a. It does not compile
- b. It prints nothing
- c. It prints 75
- d. It throws `IllegalArgumentException` from the default

✗ It does not compile

Wrong. It compiles.

✗ It prints nothing

Wrong. The throw stops the program with an exception.

✗ It prints 75

Wrong. 75 is only for day 1.

✓ It throws `IllegalArgumentException` from the default

Correct. 5 matches no case, so default throws the exception.

The paper's ticket question rejects an invalid day with throw in the default.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
int n = 10;
do {
    System.out.println(n);
    n++;
} while (n < 5);
```

- a. Nothing
- b. 10
- c. It loops for ever
- d. 10
- 11
- 12
- 13
- 14

X Nothing

Wrong. The body runs before the test.

✓ 10

Correct. A do-while runs its body once before testing, and $11 < 5$ is then false.

X It loops for ever

Wrong. The test fails after one pass.

X 10

11

12

13

14

Wrong. After the first pass the test fails.

A do-while always runs at least once: its test comes after the body.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
for (int i = 1; i <= 2; i++) {  
    for (int j = 1; j <= 3; j++) {  
        System.out.print(i * j + " ");  
    }  
    System.out.println();  
}
```

- a. 1 2 3
2 4 6
- b. 1 2 3 2 4 6
- c. 1 2
2 4
3 6
- d. 1
2
3
2
4
6

✓ 1 2 3
2 4 6

Correct. Each outer pass prints a row of three, and println ends the row.

X 1 2 3 2 4 6

Wrong. The println in the outer loop breaks the rows.

X 1 2

2 4

3 6

Wrong. The inner loop runs three times, so each row has three values.

X 1

2

3

2

4

6

Wrong. The inner loop uses print, so a row stays on one line.

Which loop is each statement inside? That decides a row against a grid.

Asked twice. This same question also appears as: Manual O.10 Q2 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.6.

What happens when this is compiled?

```
for (int i = 0; i < 3; i++) {  
}  
System.out.println(i);
```

- a. It does not compile: i exists only inside the for loop
- b. It prints 0
- c. It prints 3
- d. It prints 2

✓ It does not compile: i exists only inside the for loop

Correct. A variable declared in the for header does not exist after the loop.

✗ It prints 0

Wrong. It never compiles.

✗ It prints 3

Wrong. It never compiles.

✗ It prints 2

Wrong. It never compiles.

Declare the counter before the loop if you need it afterwards.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
int n = 1;
while (n < 100) {
    n *= 2;
}
System.out.println(n);
```

a. 256

b. 64

c. 100

d. 128

X 256

Wrong. The loop stops at 128.

X 64

Wrong. 64 is still below 100, so it doubles again.

X 100

Wrong. n only ever holds powers of 2.

✓ 128

Correct. n doubles 1, 2, 4, ..., 64, 128; at 128 the test $n < 100$ fails.

As a for loop: `for (int n = 1; n < 100; n *= 2) { }`, and the printed value is the same.

3.32

MEDIUM

Lesson 2.4

True or false: a counter declared in a for header can still be used after the loop.

Answer: **false**

That is the one real difference between the two forms.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
for (int i = 1; i <= 3; i++) {  
    for (int j = 1; j <= 3; j++) {  
        if (j == 3) break;  
        System.out.print(i + " " + j + " ");  
    }  
}
```

- a. 11 21 31
- b. 11 12 21 22 31 32
- c. 11 12 13 21 22 23 31 32 33
- d. 11 12

X 11 21 31

Wrong. The break is at 3, not 2.

✓ 11 12 21 22 31 32

Correct. break leaves only the inner loop, at $j = 3$, so each outer pass prints two values.

X 11 12 13 21 22 23 31 32 33

Wrong. $j = 3$ is never printed.

X 11 12

Wrong. The outer loop carries on after the inner break.

break and continue act on the innermost loop only.

3.34

MEDIUM

Lesson 2.5

The count is placed **ABOVE** the continue. What does this print?

```
int[] a = {5, 12, 3, 20};  
int count = 0;  
for (int i = 0; i < a.length; i++) {  
    count++;  
    if (a[i] < 10) continue;  
}  
System.out.println(count);
```

a. 4

b. 2

c. 0

d. 1

✓ 4

Correct. count++ runs for every item before the skip happens, so it counts all four.

X 2

Wrong. That would need count++ after the continue.

X 0

Wrong. count++ runs every pass.

X 1

Wrong. It runs on all four passes.

Anything above the continue runs for every item, including the ones you meant to skip.

In a FOR loop, what does continue do?

- a. Skips the rest of this pass, runs the step, and goes on to the test
- b. Leaves the loop entirely
- c. Skips the step as well
- d. Restarts the loop from its start value

✓ Skips the rest of this pass, runs the step, and goes on to the test

Correct. In a for loop the step still happens.

✗ Leaves the loop entirely

Wrong. That is break.

✗ Skips the step as well

Wrong. In a for loop the step is not skipped; in a while loop a step at the bottom would be.

✗ Restarts the loop from its start value

Wrong. The counter keeps its value and moves on.

| continue skips one, break stops them all.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
public class Q401 {  
    public static void main(String[] args) {  
        int day = 2;  
        switch (day) {  
            case 1: System.out.println("Mon");  
            case 2: System.out.println("Tue");  
            case 3: System.out.println("Wed");  
                break;  
            default: System.out.println("Other");  
        }  
    }  
}
```

- a. Tue
- b. Tue
Wed
- c. Tue
Wed
Other
- d. Mon
Tue
Wed

✗ Tue

Wrong. Without a break at the end of case 2, execution falls straight into case 3.

✓ Tue

Wed

Correct. It enters at case 2, prints Tue, falls through into case 3 and prints Wed, and the break there stops it.

✗ Tue

Wed

Other

Wrong. The break at the end of case 3 stops it before default.

✗ Mon

Tue

Wed

Wrong. It ENTERS at the matching case; the cases above it are skipped entirely.

The answer is Tue then Wed. A switch enters at the matching label and runs on until a break.
Missing breaks are the classic switch bug and a favourite exam question.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
public class Q402 {  
    public static void main(String[] args) {  
        int x = 5;  
        if (x > 10)  
            if (x > 20) System.out.println("big");  
            else System.out.println("small");  
        System.out.println("end");  
    }  
}
```

- a. small
end
- b. big
end
- c. end
- d. small

X small
end

Wrong, and it is what the INDENTATION suggests, which is why the question is here. The else does not belong to the outer if.

✗ big
end

Wrong. x is 5, so the outer condition is already false.

✓ end

Correct. An else attaches to the nearest unmatched if, which is the INNER one. Since x is not greater than 10, neither if is entered and nothing but end prints.

✗ small

Wrong. The last println is outside every if and always runs.

The answer is end. An else binds to the nearest if, whatever the indentation says. Braces on every if remove the ambiguity, and they cost nothing.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
public class Q403 {  
    public static void main(String[] args) {  
        int sum = 0;  
        for (int i = 1; i <= 5; i++) sum += i;  
        System.out.println(sum);  
        int j = 1, prod = 1;  
        while (j <= 4) { prod *= j; j++; }  
        System.out.println(prod);  
    }  
}
```

- a. 10
24
- b. 15
10
- c. 21
24
- d. 15
24

X 10
24

Wrong. The loop includes 5, because the test is $i \leq 5$, not $i < 5$.

✗ 15

10

Wrong on the second: the while loop multiplies rather than adds.

✗ 21

24

Wrong. 21 would be summing to 6, and the loop stops at 5.

✓ 15

24

Correct. $1+2+3+4+5$ is 15, and $1 \times 2 \times 3 \times 4$ is 24.

The answer is 15 then 24. The three parts of a for header, initialise, test, update, are exactly the three pieces you place by hand when converting to a while. This conversion is worth 10 marks on the 2024/25 paper.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
public class Q404 {  
    public static void main(String[] args) {  
        int n = 10;  
        while (n < 5) { System.out.println("while " + n); n++; }  
        do { System.out.println("do " + n); n++; } while (n < 5);  
    }  
}
```

- a. do 10
- b. nothing is printed
- c. while 10
do 10
- d. do 10
do 11

✓ do 10

Correct. The while loop tests first, finds 10 is not less than 5, and never runs. The do-while runs its body once, prints do 10, and only then tests.

✗ nothing is printed

Wrong. The do-while body runs before its condition is ever looked at.

X while 10

do 10

Wrong. The while loop prints nothing at all.

X do 10

do 11

Wrong. After the single pass n is 11, the condition fails, and the loop ends.

The answer is do 10. while tests first and may run zero times; do-while runs at least once. That difference is the whole reason both exist.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
public class Q405 {  
    public static void main(String[] args) {  
        for (int i = 1; i <= 5; i++) {  
            if (i == 2) continue;  
            if (i == 4) break;  
            System.out.print(i + " ");  
        }  
    }  
}
```

a. 1 2 3

b. 1 3

c. 1 3 5

d. 1 3 4

✗ 1 2 3

Wrong. The continue skips the print for 2.

✓ 1 3

Correct. 1 prints, 2 is skipped by continue, 3 prints, and at 4 the break leaves the loop before 5 is reached.

✗ 1 3 5

Wrong. The break at 4 stops the loop, so 5 never happens.

X 1 3 4

Wrong. The break happens BEFORE the print, so 4 is never printed.

The answer is 1 3 . continue abandons this pass, break abandons the loop. Read the order of the statements carefully: whether the break is before or after the print changes the answer.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
public class Q406 {  
    public static void main(String[] args) {  
        for (int i = 1; i <= 3; i++) {  
            for (int j = 1; j <= i; j++) System.out.print("*");  
            System.out.println();  
        }  
    }  
}
```

a. ***

b. *

*

*

c. *

**

d. ***

**

*

X ***

Wrong. The inner limit is i, not 3, so it grows with each outer pass.

X *

*

*

Wrong. That would need the inner loop to run exactly once each time.

✓ *

**

Correct. The inner loop runs i times, so one star, then two, then three.

X ***

**

*

Wrong, and it is upside down: i grows, so the rows get longer.

The answer is * then ** then ***. Triangle patterns are a standing favourite. Trace the inner limit for each value of the outer variable and write the row out; three rows of tracing settles any pattern question.

How many times does the body run?

```
public class Q407 {  
    public static void main(String[] args) {  
        int count = 0;  
        for (int i = 0; i < 10; i += 2) count++;  
        System.out.println(count);  
    }  
}
```

- a. 4
- b. 6
- c. 10
- d. 5

✗ 4

Wrong. Count them out: 0, 2, 4, 6, 8. That is five values, not four.

✗ 6

Wrong. 10 is not less than 10, so it never enters with i at 10.

✗ 10

Wrong. The step is 2, so it runs half as often as a step of 1 would.

✓ 5

Correct. i takes 0, 2, 4, 6 and 8, then 10 fails the test $i < 10$, so the body runs five times.

The answer is 5. When a loop steps by more than one, write the values out rather than dividing in your head. The last value is the one people get wrong.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
int n = 154, temp = n, sum = 0;
while (temp > 0) {
    int d = temp % 10;
    sum += d * d * d;
    temp /= 10;
}
System.out.println(sum + " " + (sum == n));
```

- a. 190 false
- b. 154 true
- c. 10 false
- d. 190 true

✓ 190 false

Correct. The cubes of 1, 5 and 4 are 1, 125 and 64, which sum to 190, not 154.

✗ 154 true

Wrong. The sum of the cubes is 190.

✗ 10 false

Wrong. The digits are cubed before they are added.

✗ 190 true

Wrong. 190 is not equal to 154.

Trace it digit by digit: pull off the last digit, cube it, add it, remove it.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
int x = 3;
if (x > 5); {
    System.out.println("big");
}
System.out.println("end");
```

- a. It does not compile
- b. big
end
- c. end
- d. big

X It does not compile

Wrong. It compiles, which is what makes it dangerous.

✓ big
end

Correct. `if (x > 5);` is a complete, empty if; the braces after it are an ordinary block that always runs.

X end

Wrong. The block is not controlled by the if.

✗ big

Wrong. end prints after the block.

| A semicolon straight after the condition ends the if. The block that follows runs every time.

3.45

HARD

Lesson 2.2

Asked twice. This same question also appears as: Manual O.10 Q2 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 2.1 and Lesson 2.2 and Lesson 2.3 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.6.

What happens when this is compiled?

```
int x = 2;
int n = 2;
switch (x) {
    case n: System.out.println("match"); break;
}
```

- a. It prints 2
- b. It prints nothing
- c. It prints match
- d. It does not compile: a case label must be a constant, and n is a variable

✗ It prints 2

Wrong. It never compiles.

✗ It prints nothing

Wrong. It never compiles.

✗ It prints match

Wrong. It never compiles.

✓ It does not compile: a case label must be a constant, and n is a variable

Correct. The compiler builds the jump table before the program runs, so labels must be constants.

| Every case label must be a literal or a final constant.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
int count = 0;
for (int i = 0; i < 5; i++);
{
    count++;
}
System.out.println(count);
```

- a. 1
- b. 5
- c. 0
- d. It does not compile

✓ 1

Correct. for (...); is a complete empty loop; the block after it is ordinary code that runs once.

✗ 5

Wrong. The block is not the loop's body.

✗ 0

Wrong. The block runs once.

✗ It does not compile

Wrong. It compiles, which is what makes it dangerous.

| A semicolon after the for line is a complete empty loop.

3.47

HARD

Lesson 2.3

The total is declared **INSIDE** the loop. What does this print?

```
int[] a = {4, 5, 9};  
int last = 0;  
for (int i = 0; i < a.length; i++) {  
    int total = 0;  
    total += a[i];  
    last = total;  
}  
System.out.println(last);
```

a. 0

b. It does not compile

c. 9

d. 18

✗ 0

Wrong. last holds 9.

✗ It does not compile

Wrong. The program prints last, which is declared outside.

✓ 9

Correct. The last line uses last, which holds the final item. total resets every pass and cannot be seen outside anyway.

✗ 18

Wrong. Nothing adds all the items outside the loop.

| Declaring the total inside the loop resets it every pass; declare it before, add inside, print after.

Which while loop does exactly what this for loop does?

```
int sum = 0;
for (int k = 1; k <= 4; k++) {
    sum += k;
}
System.out.println(sum);
```

- a. `int k = 1; while (k < 4) { sum += k; k++; }`
- b. `int k = 1; while (k <= 4) { sum += k; k++; }`
- c. `int k = 1; while (k <= 4) { k++; sum += k; }`
- d. `int k = 1; while (k <= 4) { sum += k; }`

✗ `int k = 1; while (k < 4) { sum += k; k++; }`

Wrong. The test changed from `<=` to `<`, so 4 is never added.

✓ `int k = 1; while (k <= 4) { sum += k; k++; }`

Correct. Start above, test unchanged, step at the bottom: the sum is 10, the same as the for.

✗ `int k = 1; while (k <= 4) { k++; sum += k; }`

Wrong. Stepping BEFORE adding adds 2, 3, 4 and 5, giving 14.

✗ `int k = 1; while (k <= 4) { sum += k; }`

Wrong. The step is missing, so k stays 1 and the loop never ends.

Start goes above, test stays put, step goes to the bottom. Then check the output is identical.

Half converted: the step is in the header AND the body. What does this print?

```
int total = 0;
for (int i = 1; i <= 10; i++) {
    total += i;
    i++;
}
System.out.println(total);
```

- a. 30
- b. 25
- c. 55
- d. It loops for ever

✗ 30

Wrong. The even numbers are skipped, not the odd.

✓ 25

Correct. i goes up by 2 each pass, 1, 3, 5, 7, 9, so only the odd numbers are added: 25.

✗ 55

Wrong. That is the sum of 1 to 10, which would need one step per pass.

✗ It loops for ever

Wrong. i still passes 10, so it stops.

Leaving the step in the body as well as the header double-steps the counter.

This for loop is converted to a while with `i++` as the last line of the body. What goes wrong?

```
int sum = 0;
for (int i = 1; i <= 5; i++) {
    if (i == 3) continue;
    sum += i;
}
System.out.println(sum);
```

- a. The while version does not compile
- b. Nothing; the while prints the same result
- c. When `i` is 3, `continue` skips the `i++`, so `i` stays 3 and the while loop never ends
- d. The while version skips 4 as well

✗ The while version does not compile

Wrong. It compiles; it just never stops.

✗ Nothing; the while prints the same result

Wrong. `continue` changes the meaning once the step is in the body.

✓ When `i` is 3, `continue` skips the `i++`, so `i` stays 3 and the while loop never ends

Correct. In a `for`, `continue` still runs the step; in the `while`, the step at the bottom is jumped over.

✗ The while version skips 4 as well

Wrong. It does not reach 4 at all.

Convert a loop containing `continue` with care, and say so in your answer.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
int[] a = {5, 12, 3, 20, 8, 30};
int count = 0, total = 0;
for (int i = 0; i < a.length; i++) {
    if (a[i] < 10) continue;
    total += a[i];
    count++;
    if (count == 2) break;
}
System.out.println(total + " " + count);
```

- a. 62 3
- b. 12 1
- c. 78 6
- d. 32 2

X 62 3

Wrong. The break at a count of 2 stops before 30.

X 12 1

Wrong. 20 is added before the break.

X 78 6

Wrong. Items under 10 are skipped, and the loop breaks early.

✓ 32 2

Correct. 5 and 3 are skipped; 12 and 20 are added and counted; at a count of 2 the loop breaks before 30.

■ Skip first, then process, then count, then decide whether to stop: the Section A pattern.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
double t = 1.1 * 3;  
System.out.println(t);  
System.out.printf("%.1f%n", t);
```

a. 3.3000000000000003

3.30

b. 3.3

3.30

c. 3.3

3.3

d. 3.3000000000000003

3.3

X 3.3000000000000003

3.30

Wrong. %.1f shows ONE decimal place.

X 3.3

3.30

Wrong. The first line shows the tail.

✗ 3.3

3.3

Wrong. println shows the exact double, tail included.

✓ 3.3000000000000003

3.3

Correct. A double cannot hold 1.1 exactly, so the raw value shows a tail; printf with %.1f shows one decimal place.

Compute money in double, print it with printf and a stated number of decimal places.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
boolean stop = false;
int found = -1;
for (int i = 1; i <= 5 && !stop; i++) {
    for (int j = 1; j <= 5; j++) {
        if (i * j == 6) {
            found = i * 10 + j;
            stop = true;
            break;
        }
    }
}
System.out.println(found);
```

- a. 16
- b. -1
- c. 23
- d. 32

X 16

Wrong. found is set as $i * 10 + j$.

X -1

Wrong. A pair is found.

✓ 23

Correct. The first pair with $i * j == 6$ is $i = 2, j = 3$; the flag then stops the outer loop too.

✗ 32

Wrong. $i = 2, j = 3$ is found before $i = 3, j = 2$.

! To leave two loops, set a boolean flag and test it in the outer condition.

4 Methods

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
static int sq(int n) {  
    return n * n;  
}  
  
// in main:  
System.out.println(sq(3) + sq(4));
```

- a. 12
- b. 25
- c. 14
- d. 7

X 12

Wrong. $9 + 16$ is 25.

✓ 25

Correct. $\text{sq}(3)$ returns 9 and $\text{sq}(4)$ returns 16; $9 + 16$ is 25.

X 14

Wrong. Squaring, not doubling.

X 7

Wrong. sq squares; it does not return its argument.

| A call to a value returning method is a value, so two can be added.

4.2

EASY

Lesson 2.6

A method is declared `area(double w, double h)` and called as `area(width, 3.0)`. Which are the arguments?

- a. width and 3.0
- b. area and double
- c. w and h
- d. w and 3.0

✓ width and 3.0

Correct. Arguments are the values supplied at the call.

X area and double

Wrong. Those are the name and a type.

X w and h

Wrong. w and h are the parameters, the names in the header.

X w and 3.0

Wrong. w is a parameter.

| The parameter is the name in the header; the argument is the value passed at the call.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
public class Q501 {  
    static void change(int n) { n = 99; }  
    public static void main(String[] args) {  
        int x = 5;  
        change(x);  
        System.out.println(x);  
    }  
}
```

- a. 5
- b. 99
- c. 0
- d. it does not compile

✓ 5

Correct. Java passes arguments BY VALUE. The parameter `n` is a separate variable holding a copy of 5, so assigning to it cannot reach `x`.

✗ 99

Wrong. The method changed its own copy of the value, not the caller's variable.

✗ 0

Wrong. `x` was assigned 5 and nothing since has touched it.

X it does not compile

Wrong. The code is perfectly legal; it simply does not do what it looks like it does.

The answer is 5. Java is pass by value, always. For an object the VALUE passed is the reference, which is why an object's contents can be changed by a method while the caller's variable still points at the same object.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
public class Q502 {
    static void fill(int[] a) { a[0] = 99; }
    static void replace(int[] a) { a = new int[]{7, 7}; }
    public static void main(String[] args) {
        int[] nums = {1, 2};
        fill(nums);
        replace(nums);
        System.out.println(nums[0] + " " + nums[1]);
    }
}
```

- a. 1 2
- b. 99 2
- c. 7 7
- d. 99 7

X 1 2

Wrong. fill really did change the array, because both names refer to the same array object.

✓ 99 2

Correct. fill reaches through the reference and changes element 0. replace only rebinds its own copy of the reference, so the caller's array is untouched.

X 7 7

Wrong. replace cannot change which array the caller's variable points at.

X 99 7

Wrong for the same reason: nothing from replace escapes the method.

The answer is 99 2. Through the reference you can change the object; assigning to the parameter only changes where that copy points. This one pair of methods is the clearest statement of what pass by value means in Java.

4.5

MEDIUM

Manual O.5 Q4

Why can a static method not use an instance field directly?

- a. Because static methods are private
- b. Because instance fields are always final
- c. Because static methods run before the class is loaded
- d. Because a static method may run with no object in existence, so there is no instance whose field it could mean

X Because static methods are private

Wrong. static and private are unrelated: a static method can be public.

X Because instance fields are always final

Wrong. Instance fields are ordinarily variable; final is a separate choice.

X Because static methods run before the class is loaded

Wrong. The class is loaded before any of its methods run.

✓ Because a static method may run with no object in existence, so there is no instance whose field it could mean

Correct. Static belongs to the class, so it can be called with no object at all. An instance field only exists inside an object, so the reference would be to nothing.

The answer is Because a static method may run with no object in existence, so there is no instance whose field it could mean. Static belongs to the class, instance belongs to the object. This is exactly why main is static: the JVM calls it before any object of your class exists.

Asked twice. This same question also appears as: Manual O.10 Q2 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.6 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.6.

What happens when this is compiled?

```
static void show(int n) {  
    System.out.println("the number is " + n);  
}  
  
// in main:  
int y = show(5);
```

- a. It does not compile: show is void, so there is no value to store
- b. It prints 5
- c. It prints the number is 5, then 0
- d. It prints the number is 5

✓ It does not compile: show is void, so there is no value to store

Correct. A void call is a statement, not a value.

✗ It prints 5

Wrong. It never compiles.

✗ It prints the number is 5, then 0

Wrong. It never compiles.

✗ It prints the number is 5

Wrong. It never compiles.

A void method performs an action and returns nothing; a value returning method hands a result back with return.

twice is not static. What happens when this is compiled?

```
int twice(int n) {  
    return 2 * n;  
}  
  
// in main:  
System.out.println(twice(4));
```

- a. It prints 8
- b. It does not compile: main is static and there is no object to call twice on
- c. It prints 4
- d. It prints 0

X It prints 8

Wrong. It never compiles.

✓ It does not compile: main is static and there is no object to call twice on

Correct. A method called from main without an object must be static.

X It prints 4

Wrong. It never compiles.

X It prints 0

Wrong. It never compiles.

| Forgetting static on a helper is one of the commonest first-exam compile errors.

What exactly is a method signature?

- a. The method's first line, including public static
- b. The method name together with the number, types and order of its parameters
- c. The return type and the method name
- d. The values passed at the call

✗ The method's first line, including public static

Wrong. Modifiers are not part of it.

✓ The method name together with the number, types and order of its parameters

Correct. And it does NOT include the return type.

✗ The return type and the method name

Wrong. The return type is not part of the signature.

✗ The values passed at the call

Wrong. Those are arguments.

Signature is name plus parameter list. Never the return type.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
class Box {
    static int made = 0;
    int id;
    Box() {
        made++;
        id = made;
    }
}

// in main:
Box a = new Box();
Box b = new Box();
Box c = new Box();
System.out.println(b.id + " " + Box.made);
```

- a. 2 3
- b. 3 3
- c. 1 3
- d. 2 1

✓ 2 3

Correct. made is one shared counter, 3 after three objects; b was the second, so its id is 2.

✗ 3 3

Wrong. Each object has its own id.

✗ 1 3

Wrong. b was made second.

✗ 2 1

Wrong. made is shared, so it counts all three.

static is one shared copy; an instance field is one copy per object.

4.10

MEDIUM

Lesson 2.7

count is an instance field. What happens when this is compiled?

```
int count = 0;

// in main:
count++;
System.out.println(count);
```

- a. It prints 1
- b. It does not compile: a static method cannot use an instance field directly
- c. It prints 0
- d. It prints nothing

✗ It prints 1

Wrong. It never compiles.

✓ It does not compile: a static method cannot use an instance field directly

Correct. main is static and may run when no object exists, so there is no count to mean.

✗ It prints 0

Wrong. It never compiles.

✗ It prints nothing

Wrong. It never compiles.

"non-static variable cannot be referenced from a static context" is the commonest first-exam compile error.

4.11

MEDIUM

Lesson 2.7

Do `void f(int a, double b)` and `void f(double a, int b)` overload legally in one class?

- a. Only if one is static
- b. Yes, because the order of the parameter types differs
- c. No, because both return void
- d. No, because they have the same types

✗ Only if one is static

Wrong. static has nothing to do with it.

✓ Yes, because the order of the parameter types differs

Correct. Same types in a different order makes a different signature.

✗ No, because both return void

Wrong. The return type plays no part.

✗ No, because they have the same types

Wrong. Order is part of the signature.

Number, types and order: change any of the three and the signature differs.

4.12

MEDIUM

Lesson 2.7

True or false: an overloaded call is resolved at run time, from the actual object.

Answer: **false**

Overloading is static, or compile time, polymorphism.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
public class Q503 {  
    static String f(int n) { return "int"; }  
    static String f(double d) { return "double"; }  
    static String f(String s) { return "String"; }  
    public static void main(String[] args) {  
        System.out.println(f(1));  
        System.out.println(f(1.0));  
        System.out.println(f("1"));  
    }  
}
```

- a. double
double
String
- b. int
int
String
- c. int
double
String
- d. it does not compile, because three methods share a name

✗ double
double
String

Wrong. 1 is an int literal and an exact int match exists, so no widening is needed.

✗ int
int
String

Wrong. 1.0 is a double literal, so the double version matches exactly.

✓ int
double
String

Correct. Overload resolution picks by the argument TYPES: 1 is an int, 1.0 is a double, "1" is a String.

✗ it does not compile, because three methods share a name

Wrong. Sharing a name with different parameter lists is exactly what overloading IS.

The answer is int then double then String. Overloading is same name, different parameter list, resolved at compile time by the argument types. Return type alone cannot distinguish two overloads.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
public class Q505 {
    static int addUp(int n) {
        int total = 0;
        for (int i = 1; i <= n; i++) {
            total += i;
            if (total > 6) return total;
        }
        return -1;
    }
    public static void main(String[] args) {
        System.out.println(addUp(5));
    }
}
```

- a. 10
- b. 15
- c. -1
- d. 6

✓ 10

Correct. The totals are 1, 3, 6, 10. The first that is greater than 6 is 10, and return hands it back immediately.

✗ 15

Wrong. 15 is the full sum to 5, but the method leaves as soon as the running total passes 6.

X -1

Wrong. The final return is only reached if the loop finishes without the condition ever being true.

X 6

Wrong. 6 is not GREATER than 6, so the loop continues one more pass.

The answer is 10. return leaves the method at once, loop and all. Note the strictness of the comparison: greater than 6 excludes 6 itself.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
static void fill(int[] a) {  
    a[0] = 42;  
}  
  
// in main:  
int[] v = {1, 2};  
fill(v);  
System.out.println(v[0] + " " + v.length);
```

- a. 1 2
- b. It does not compile
- c. 42 1
- d. 42 2

X 1 2

Wrong. A change made through the copied reference reaches the same array.

X It does not compile

Wrong. It compiles.

X 42 1

Wrong. The array still has two elements.

✓ 42 2

Correct. The method gets a copy of the REFERENCE, which points at the same array, so the change is visible.

Java is always pass by value; for an object the value copied is the reference.

4.16 **HARD** Lesson 2.6

Asked twice. This same question also appears as: Manual O.10 Q2 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.6 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.6.

What happens when this is compiled?

```
static int sign(int n) {  
    if (n > 0) {  
        return 1;  
    } else if (n < 0) {  
        return -1;  
    }  
}  
  
// in main:  
System.out.println(sign(5));
```

- a. It does not compile: there is a path ($n == 0$) with no return
- b. It prints -1
- c. It prints 0
- d. It prints 1

✓ It does not compile: there is a path ($n == 0$) with no return

Correct. A value returning method must return on every route out.

✗ It prints -1

Wrong. It never compiles.

✗ It prints 0

Wrong. Java does not invent a return value.

✗ It prints 1

Wrong. It never compiles.

Every route out of a value returning method must return, including the one nobody tested.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
static int f(int n) {
    System.out.print("f" + n + " ");
    return n + 1;
}

// in main:
int r = f(f(1));
System.out.println(r);
```

- a. f2 f1 3
- b. f1 f2 3
- c. f1 f2 2
- d. f1 3

X f2 f1 3

Wrong. The inner call happens first.

✓ f1 f2 3

Correct. The inner call f(1) runs first, printing f1 and returning 2; then f(2) prints f2 and returns 3.

X f1 f2 2

Wrong. f(2) returns 3.

X f1 3

Wrong. f is called twice.

Arguments are worked out before the method they are passed to runs.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
static String kind(int x) { return "int"; }
static String kind(double x) { return "double"; }
static String kind(String x) { return "String"; }

// in main:
System.out.println(kind(7) + " " + kind(7.5) + " " + kind('A'));
```

- a. int double String
- b. It does not compile
- c. int double char
- d. int double int

X int double String

Wrong. 'A' is a char, not a String.

X It does not compile

Wrong. Each call matches a method.

X int double char

Wrong. There is no kind(char), so 'A' widens to int.

✓ int double int

Correct. 7 matches int, 7.5 matches double, and 'A' has no char version, so it widens to int.

The compiler chooses by the signature: the number, types and order of the arguments.

4.19

HARD

Lesson 2.7

Asked twice. This same question also appears as: Manual O.10 Q2 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.6 and Lesson 2.6 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.6.

What happens when this is compiled?

```
static int f(int a) { return a; }  
static double f(int a) { return a; }
```

```
// in main:  
System.out.println(f(2));
```

- a. It does not compile: the two methods have the same signature
- b. It prints 2
- c. It prints 2.0
- d. It prints 2 and 2.0

✓ It does not compile: the two methods have the same signature

Correct. The signature is the name plus the parameter list; the return type is not part of it.

✗ It prints 2

Wrong. It never compiles.

✗ It prints 2.0

Wrong. It never compiles.

✗ It prints 2 and 2.0

Wrong. It never compiles.

Two methods differing only in return type cannot coexist: the compiler could never tell them apart.

5 Object-Oriented Programming

5.1

EASY

Manual O.6 Q2

Which pair of choices implements encapsulation?

- a. Private fields with public getters and setters
- b. Public fields and no methods
- c. Static fields with a public constructor
- d. Final fields with no constructor

✓ Private fields with public getters and setters

Correct. The data is hidden and every route to it goes through a method, which is where validation can live.

✗ Public fields and no methods

Wrong, and it is the opposite: public fields let anything change the object into an invalid state with no check possible.

✗ Static fields with a public constructor

Wrong. static concerns whether the field belongs to the class or the object, and has nothing to do with hiding it.

✗ Final fields with no constructor

Wrong. final stops reassignment; it does not hide anything, and a class with no constructor still gets the default one.

The answer is Private fields with public getters and setters. Private data, public methods. Hiding the data is only half of it: the point of the getter and setter is that they are a place to check.

Which statement is TRUE of an abstract class?

- a. It can be instantiated directly with new
- b. All of its methods must be abstract
- c. A class may extend several abstract classes at once
- d. It may contain both abstract methods and fully implemented methods

✗ It can be instantiated directly with new

Wrong. An abstract class is incomplete by definition, so new on it is a compile error. Its subclasses are what get instantiated.

✗ All of its methods must be abstract

Wrong. It may have none at all and still be marked abstract.

✗ A class may extend several abstract classes at once

Wrong. Java allows single inheritance of classes. Several INTERFACES may be implemented, which is how the restriction is worked around.

✓ It may contain both abstract methods and fully implemented methods

Correct. That is the difference from an interface in the classic form: an abstract class can carry shared state and shared behaviour alongside the methods it leaves to subclasses.

The answer is It may contain both abstract methods and fully implemented methods. Abstract class for shared state and partial implementation; interface for a contract that many unrelated classes can sign. One superclass, many interfaces.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
class Counter {
    int n;
    void inc() {
        n++;
    }
}

// in main:
Counter a = new Counter();
Counter b = new Counter();
a.inc();
a.inc();
b.inc();
System.out.println(a.n + " " + b.n);
```

- a. 2 1
- b. 1 1
- c. 3 0
- d. 3 3

✓ 2 1

Correct. Each object has its own n: a was incremented twice and b once.

✗ 1 1

Wrong. a.inc() ran twice.

✗ 3 0

Wrong. b.inc() ran once.

✗ 3 3

Wrong. n is an instance field, one per object.

■ An attribute is one copy per object; that is the difference between a class and its objects.

5.4

EASY

Lesson 3.1

What two things make a constructor different from every other method?

- a. It has the same name as the class, and no return type at all
- b. It is static and returns void
- c. It is private and takes no parameters
- d. It is called with a dot on an object

✓ It has the same name as the class, and no return type at all

Correct. Those two together are how Java recognises it.

✗ It is static and returns void

Wrong. A constructor has no return type, not even void.

✗ It is private and takes no parameters

Wrong. Constructors are usually public and often take parameters.

✗ It is called with a dot on an object

Wrong. It is run by new.

■ A constructor shares the class name and has no return type; new runs it.

Which statement correctly defines an OBJECT?

- a. A variable declared final
- b. A blueprint that defines attributes and methods
- c. An instance of a class, created with new, with its own copy of the attributes
- d. A method with no return type

✗ A variable declared final

Wrong. That is a constant.

✗ A blueprint that defines attributes and methods

Wrong. That is a class.

✓ An instance of a class, created with new, with its own copy of the attributes

Correct. That is an object.

✗ A method with no return type

Wrong. That describes a constructor.

Class is the blueprint, object is the building, new is the builder.

Asked twice. This same question also appears as: Manual O.10 Q2 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 3.1 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.6.

What happens when this is compiled?

```
class Student {  
    private int score;  
}  
  
// in main:  
Student st = new Student();  
st.score = 50;  
System.out.println(st.score);
```

- a. It throws an exception
- b. It does not compile: score is private to Student
- c. It prints 50
- d. It prints 0

X It throws an exception

Wrong. It is refused at compile time.

✓ It does not compile: score is private to Student

Correct. A private field is reachable only from inside its own class. Go through a setter.

X It prints 50

Wrong. It never compiles.

X It prints 0

Wrong. It never compiles.

| Private data, public doors.

Which two are benefits of encapsulation?

- a. The program always runs faster
- b. Validation: a setter can refuse an invalid value
- c. Control: only the class's own methods change its data
- d. A class can extend two parents

✗ The program always runs faster

Wrong. Speed is not a benefit claimed.

✓ Validation: a setter can refuse an invalid value

Correct. Validation is one benefit.

✓ Control: only the class's own methods change its data

Correct. Control is one benefit.

✗ A class can extend two parents

Wrong. Java has no multiple inheritance of classes.

Control, validation and safe change: the internal representation can change without breaking callers.

How many classes can one Java class extend?

- a. None; Java has no inheritance
- b. Two
- c. Exactly one
- d. As many as it likes

✗ None; Java has no inheritance

Wrong. extends is inheritance.

✗ Two

Wrong. One only.

✓ Exactly one

Correct. Java has no multiple inheritance of classes; a class may implement many interfaces instead.

✗ As many as it likes

Wrong. That is true of interfaces, not classes.

One parent only; use interfaces for more.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
class Animal {
    void breathe() {
        System.out.println("breathing");
    }
}
class Fish extends Animal {
    void swim() {
        System.out.println("swimming");
    }
}

// in main:
Fish f = new Fish();
f.breathe();
f.swim();
```

- a. breathing
- b. It does not compile
- c. breathing
swimming
- d. swimming

X breathing

Wrong. swim() is Fish's own method.

✗ It does not compile

Wrong. It compiles.

✓ breathing
swimming

Correct. Fish inherits breathe() from Animal and adds swim().

✗ swimming

Wrong. breathe() is inherited, so it runs too.

! The child gets the parent's public and protected members for free, and adds what is new.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
public class Q606 {
    static class Point {
        int x = 1, y = 2;
        @Override public String toString() { return "(" + x + ", " + y + ")"; }
    }
    public static void main(String[] args) {
        System.out.println(new Point());
    }
}
```

- a. (1, 2)
- b. Point@1b6d3586
- c. Point
- d. it does not compile

✓ (1, 2)

Correct. println on an object calls its toString(), and this class overrides it.

✗ Point@1b6d3586

Wrong for THIS class. That class-name-and-hash form is what the INHERITED toString gives, and it is what you see when you forget to override it.

✗ Point

Wrong. Nothing prints just the class name.

X it does not compile

Wrong. Overriding toString is ordinary and expected.

The answer is (1, 2). println on an object calls toString(). Override it and your objects become printable; forget to, and you get the class name and a hash.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
class Item {
    String name;
    double price;
    boolean sold;
}

// in main:
Item i = new Item();
System.out.println(i.name + " " + i.price + " " + i.sold);
```

- a. null 0 null
- b. "" 0.0 false
- c. null 0.0 false
- d. 0 0 false

X null 0 null

Wrong. price is a double, 0.0, and sold is a boolean, false.

X "" 0.0 false

Wrong. An unset String is null, not empty text.

✓ null 0.0 false

Correct. An unset object reference is null, a double is 0.0, a boolean is false.

✗ 0 0 false

Wrong. A String field starts as null, not 0.

Defaults for fields never set: numbers 0, boolean false, every object reference null.

5.12

MEDIUM

Lesson 3.1

Asked twice. This same question also appears as: Lesson 1.5 and Lesson 2.2 and Lesson 2.8 and Lesson 3.5 and Lesson 3.6.

What happens when this runs?

```
class Item {  
    String name;  
}  
  
// in main:  
Item i = new Item();  
System.out.println(i.name.length());
```

- a. It prints null
- b. It throws NullPointerException, because name was never set
- c. It does not compile
- d. It prints 0

✗ It prints null

Wrong. The length() call fails before anything prints.

✓ It throws NullPointerException, because name was never set

Correct. The field is null, and calling a method on null has no object to ask.

✗ It does not compile

Wrong. It compiles; the fault shows only when it runs.

✗ It prints 0

Wrong. There is no String whose length could be 0.

That is the argument for setting every field in the constructor.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
class Student {
    private int score;
    public void setScore(int s) {
        if (s >= 0 && s <= 100) {
            score = s;
        }
    }
    public int getScore() {
        return score;
    }
}

// in main:
Student st = new Student();
st.setScore(80);
st.setScore(150);
System.out.println(st.getScore());
```

- a. 150
- b. 0
- c. It does not compile
- d. 80

X 150

Wrong. The setter refuses anything above 100.

✗ 0

Wrong. The first call set it to 80.

✗ It does not compile

Wrong. It compiles.

✓ 80

Correct. 80 is accepted; 150 is refused by the check, so the score stays 80.

Encapsulation is what makes validation possible: the setter checks before it changes anything.

5.14

MEDIUM

Lesson 3.2

Which list runs from **MOST open** to **MOST closed**?

- a. private, default, protected, public
- b. public, default, protected, private
- c. protected, public, private, default
- d. public, protected, default, private

✗ private, default, protected, public

Wrong. That is most closed first.

✗ public, default, protected, private

Wrong. protected is more open than default.

✗ protected, public, private, default

Wrong. public is the most open.

✓ public, protected, default, private

Correct. Anywhere; subclasses and package; package only; the class itself only.

Fields private, methods public: private data, public behaviour.

From where can a protected member be reached?

- a. The class itself only
- b. Other packages only
- c. Anywhere at all
- d. The class itself, its subclasses, and the same package

✗ The class itself only

Wrong. That is private.

✗ Other packages only

Wrong. Not what protected means.

✗ Anywhere at all

Wrong. That is public.

✓ The class itself, its subclasses, and the same package

Correct. Mine, my children's, and my package's.

| protected earns its place when a subclass needs a parent's field.

5.16

MEDIUM

Lesson 3.2

What is the difference between encapsulation and abstraction?

- a. They are the same idea
- b. Encapsulation hides the data; abstraction hides the implementation
- c. Encapsulation hides the implementation; abstraction hides the data
- d. Abstraction is about constructors

✗ They are the same idea

Wrong. They are separate pillars.

✓ Encapsulation hides the data; abstraction hides the implementation

Correct. Two different pillars, two different answers.

✗ Encapsulation hides the implementation; abstraction hides the data

Wrong. This reverses them.

✗ Abstraction is about constructors

Wrong. Not what abstraction means.

Encapsulation: data behind methods. Abstraction: what a thing does, not how.

5.17

MEDIUM

Lesson 3.2

True or false: writing getters and setters while leaving the fields public is still encapsulation.

Answer: **false**

Private data, public doors. A setter with no check buys nothing either.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
class Account {
    double balance;
    Account(double b) {
        balance = b;
        System.out.println("Account " + b);
    }
}
class Savings extends Account {
    double rate;
    Savings(double b, double r) {
        super(b);
        rate = r;
        System.out.println("Savings " + r);
    }
}

// in main:
Savings s = new Savings(100, 0.1);
System.out.println(s.balance * s.rate);
```

a. Account 100.0

Savings 0.1

10.0

b. Account 100.0

10.0

c. Savings 0.1

Account 100.0

10.0

d. 10.0

✓ Account 100.0

Savings 0.1

10.0

Correct. super(b) runs the parent constructor first, then the rest of the child's; balance times rate is 10.0.

✗ Account 100.0

10.0

Wrong. The child's constructor prints its line too.

✗ Savings 0.1

Account 100.0

10.0

Wrong. The parent's constructor runs first, through super.

✗ 10.0

Wrong. Both constructors print.

super(...) calls the parent's constructor and must be the first statement in the child's.

Asked twice. This same question also appears as: Manual O.10 Q2 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.6.

What happens when this is compiled?

```
class Account {
    double balance;
    Account(double b) {
        balance = b;
    }
}
class Savings extends Account {
    Savings(double b) {
        System.out.println("x");
        super(b);
    }
}

// in main:
Savings s = new Savings(5);
```

- a. It prints nothing
- b. It prints x and 5.0
- c. It does not compile: super(...) must be the first statement
- d. It prints x

✗ It prints nothing

Wrong. It never compiles.

✗ It prints x and 5.0

Wrong. It never compiles.

✓ It does not compile: super(...) must be the first statement

Correct. The parent's part must exist before the child adds to it.

✗ It prints x

Wrong. It never compiles.

extends takes it, super builds it, and super goes first.

Asked twice. This same question also appears as: Manual O.10 Q2 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.6.

What happens when this is compiled?

```
class Account {
    private double balance;
}
class Savings extends Account {
    void add() {
        balance += 1;
    }
}

// in main:
new Savings().add();
```

- a. It does not compile: balance is private, so Savings cannot see it
- b. It throws an exception
- c. It prints 1.0
- d. It prints 0.0

✓ It does not compile: balance is private, so Savings cannot see it
Correct. Use protected, or go through a getter.

✗ It throws an exception
Wrong. It is a compile error.

✗ It prints 1.0
Wrong. It never compiles.

✗ It prints 0.0
Wrong. It never compiles.

private is invisible even to a child; protected means mine and my children's.

Should Car extend Engine?

- a. Yes: a car is an engine
- b. No: a car HAS an engine, so Engine should be a field of Car
- c. No: Java cannot model cars
- d. Yes: extends is the only way to reuse code

✗ Yes: a car is an engine

Wrong. A car is not an engine.

✓ No: a car HAS an engine, so Engine should be a field of Car

Correct. Inheritance is an is a; a field is a has a.

✗ No: Java cannot model cars

Wrong. Not the reason.

✗ Yes: extends is the only way to reuse code

Wrong. A field reuses the engine's code too.

| If you cannot say "a child IS A parent" with a straight face, you want a field.

Which two are NOT inherited by a subclass?

- a. Constructors
- b. Public methods
- c. Private members
- d. Protected fields

✓ Constructors

Correct. A constructor belongs to its own class; it is called with super.

✗ Public methods

Wrong. Public methods are inherited.

✓ Private members

Correct. Private members exist in the child object but the child's code cannot see them.

✗ Protected fields

Wrong. Protected fields are inherited.

| Inherited: public and protected. Not inherited: constructors and anything private.

Asked twice. This same question also appears as: Manual O.10 Q2 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.6.

What happens when this is compiled?

```
class A {  
    void show() { }  
}  
class B extends A {  
    @Override  
    void shwo() { }  
}  
  
// in main:  
new B().show();
```

- a. It compiles, and shwo is a new method
- b. It prints nothing
- c. It does not compile: shwo does not override anything, and @Override makes that an error
- d. It throws an exception

X It compiles, and shwo is a new method

Wrong. Without @Override that would happen silently; with it, it is an error.

X It prints nothing

Wrong. It never compiles.

✓ It does not compile: shwo does not override anything, and @Override makes that an error

Correct. @Override asks the compiler to check, so the typo is caught.

X It throws an exception

Wrong. It is a compile error.

| Always write @Override: a misspelt name becomes an error instead of a silent new method.

Which three facts distinguish overloading from overriding?

- a. Same class against subclass; parameters differ against identical; compile time against run time
- b. Loop against recursion; array against String; input against output
- c. Public against private; static against final; int against double
- d. There is no difference

✓ Same class against subclass; parameters differ against identical; compile time against run time

Correct. Class, parameters, timing.

✗ Loop against recursion; array against String; input against output

Wrong. Unrelated.

✗ Public against private; static against final; int against double

Wrong. Not the differences the paper wants.

✗ There is no difference

Wrong. They differ in three independent ways.

Class, parameters, timing: three words, three marks.

5.25

MEDIUM

Lesson 3.4

An Account variable holds a Savings object, and Savings overrides describe(). Whose describe() runs?

- a. Both, one after the other
- b. Savings's, decided at run time from the actual object
- c. Account's, decided at compile time from the declared type
- d. Neither; it does not compile

✗ Both, one after the other

Wrong. One version runs.

✓ Savings's, decided at run time from the actual object

Correct. The actual object decides, at run time.

✗ Account's, decided at compile time from the declared type

Wrong. That is how overloading is decided, not overriding.

✗ Neither; it does not compile

Wrong. A parent-typed variable may hold a child.

Polymorphism: one call, many forms, chosen at run time by the object.

5.26

MEDIUM

Lesson 3.4

True or false: overriding is decided at compile time from the declared type.

Answer: **false**

Overloading at compile time, overriding at run time.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
class Emp {
    double pay() {
        return 100;
    }
}
class Manager extends Emp {
    double pay() {
        return 250;
    }
}

// in main:
Emp[] staff = {new Emp(), new Manager(), new Manager()};
double total = 0;
for (int i = 0; i < staff.length; i++) {
    total += staff[i].pay();
}
System.out.println(total);
```

- a. 350.0
- b. 600.0
- c. 600
- d. 300.0

✗ 350.0

Wrong. There are two Managers.

✓ 600.0

Correct. Each object's own `pay()` runs: $100 + 250 + 250 = 600.0$.

✗ 600

Wrong. total is a double, so it prints 600.0.

✗ 300.0

Wrong. The Manager objects run Manager's `pay()`.

■ The loop never has to know the kinds: each object answers with its own version.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
public class Q601 {
    static class Dog {
        String name;
        Dog() { this("unnamed"); }
        Dog(String n) { name = n; }
    }
    public static void main(String[] args) {
        System.out.println(new Dog().name);
        System.out.println(new Dog("Rex").name);
    }
}
```

- a. null
Rex
- b. unnamed
unnamed
- c. it does not compile
- d. unnamed
Rex

X null
Rex

Wrong. The no-argument constructor does not leave name unset: it calls the other constructor with a default.

X unnamed
unnamed

Wrong. The second call passes Rex explicitly, and that is what is stored.

X it does not compile

Wrong. Constructor chaining with `this(...)` is legal, provided it is the first statement.

✓ unnamed
Rex

Correct. `this("unnamed")` chains to the other constructor, so the first object gets unnamed and the second gets Rex.

The answer is unnamed then Rex. A constructor has the class name, no return type, and runs on new. `this(...)` chains to another constructor and must be the first statement in the body.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
public class Q603 {
    static class Animal {
        void speak() { System.out.println("some sound"); }
    }
    static class Dog extends Animal {
        @Override void speak() { System.out.println("woof"); }
    }
    public static void main(String[] args) {
        Animal a = new Dog();
        a.speak();
    }
}
```

- a. some sound
- b. woof
- c. it does not compile
- d. both lines print, superclass first

X some sound

Wrong. The DECLARED type is Animal, but the method that runs is chosen by the object's ACTUAL type at run time.

✓ woof

Correct. The object is a Dog, so Dog's override runs. This is polymorphism, and it is exactly what makes inheritance useful.

X it does not compile

Wrong. Assigning a subclass object to a superclass variable is always allowed: a Dog IS an Animal.

X both lines print, superclass first

Wrong. An override replaces the superclass version. Running both would need an explicit `super.speak()` call.

The answer is woof. The variable's type decides what you may CALL; the object's type decides what actually RUNS. That sentence answers most polymorphism questions on this paper.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
public class Q604 {  
    static class A {  
        A() { System.out.println("A"); }  
    }  
    static class B extends A {  
        B() { System.out.println("B"); }  
    }  
    public static void main(String[] args) { new B(); }  
}
```

- a. B
- b. B
A
- c. A
B
- d. nothing prints

X B

Wrong. The superclass constructor always runs first, whether or not you wrote super().

X B

A

Wrong, and it is the wrong way round: the base is built before the part built on top of it.

✓ A

B

Correct. Java inserts an implicit `super()` at the top of B's constructor, so A's constructor completes before B's body starts.

✗ nothing prints

Wrong. `new B()` runs both constructors.

The answer is A then B. Constructors run from the top of the hierarchy down. A subclass object cannot exist until the superclass part of it has been built.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
class Pet {  
    String name;  
    Pet(String name) {  
        name = name;  
    }  
}  
  
// in main:  
Pet p = new Pet("Rex");  
System.out.println(p.name);
```

- a. Rex
- b. It does not compile
- c. null
- d. name

X Rex

Wrong. Without this, the field is never assigned.

X It does not compile

Wrong. It compiles, which is why it is dangerous.

✓ null

Correct. `name = name` assigns the parameter to itself; the field is never set and stays null.

✗ name

Wrong. `name` is a variable, not the text "name".

| `this.name = name`, on every line of every constructor.

5.32

HARD

Lesson 3.1

Asked twice. This same question also appears as: Manual O.10 Q2 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.6.

What happens when this is compiled?

```
class Student {  
    String name;  
    Student(String name) {  
        this.name = name;  
    }  
}  
  
// in main:  
Student s = new Student();  
System.out.println(s.name);
```

- a. It prints nothing
- b. It prints 0
- c. It prints null
- d. It does not compile: writing your own constructor removes the free no-argument one

✗ It prints nothing

Wrong. It never compiles.

✗ It prints 0

Wrong. It never compiles.

✗ It prints null

Wrong. It never compiles.

✓ It does not compile: writing your own constructor removes the free no-argument one

Correct. Once you write any constructor, `new Student()` no longer exists unless you write it too.

| Constructors can be overloaded; write both if you need both.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
class Pet {  
    String name = "unknown";  
    void Pet(String n) {  
        name = n;  
    }  
}  
  
// in main:  
Pet p = new Pet();  
System.out.println(p.name);
```

- a. unknown
- b. null
- c. It does not compile
- d. Rex

✓ unknown

Correct. `void Pet(String n)` is an ordinary method, not a constructor, so `new Pet()` uses the default and the field keeps unknown.

✗ null

Wrong. name was given the starting value unknown.

X It does not compile

Wrong. A method may share the class name; it compiles.

X Rex

Wrong. Nothing ever calls the void method.

| Giving a constructor a return type turns it into an ordinary method.

Asked twice. This same question also appears as: Manual O.10 Q2 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.6.

What happens when this is compiled?

```
class Account {
    double balance;
    Account(double b) {
        balance = b;
    }
}
class Savings extends Account {
    Savings() {
        System.out.println("new");
    }
}

// in main:
Savings s = new Savings();
```

- a. It prints new and 0.0
- b. It does not compile: Savings must call super(...) because Account has no no-argument constructor
- c. It prints new
- d. It throws an exception

✗ It prints new and 0.0

Wrong. It never compiles.

✓ It does not compile: Savings must call super(...) because Account has no no-argument constructor

Correct. Java inserts a call to super(), which does not exist here.

✗ It prints new

Wrong. It never compiles.

✗ It throws an exception

Wrong. It is a compile error.

"constructor Account in class Account cannot be applied to given types" means one thing: you forgot super.

The child's describe takes a parameter. What does this print?

```
class Account {  
    void describe() {  
        System.out.println("account");  
    }  
}  
class Savings extends Account {  
    void describe(String s) {  
        System.out.println("savings " + s);  
    }  
}  
  
// in main:  
Account a = new Savings();  
a.describe();
```

- a. account
- b. It does not compile
- c. savings null
- d. savings

✓ account

Correct. describe(String) has different parameters, so it OVERLOADS; nothing overrides describe(), and the parent's version runs.

✗ It does not compile

Wrong. It compiles.

✗ savings null

Wrong. The call passes no argument, so describe(String) is not used.

✗ savings

Wrong. Nothing overrides the no-argument describe().

Changing the parameters while meaning to override overloads instead. @Override would have caught it.

Asked twice. This same question also appears as: Manual O.10 Q2 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.6.

What happens when this is compiled?

```
class A {  
    public void show() { }  
}  
class B extends A {  
    private void show() { }  
}  
  
// in main:  
new A().show();
```

- a. It prints nothing
- b. It does not compile: an override may not be more restrictive than the method it replaces
- c. It compiles, and B.show is private
- d. It throws an exception

✗ It prints nothing

Wrong. It never compiles.

✓ It does not compile: an override may not be more restrictive than the method it replaces

Correct. A public method cannot be overridden as private.

✗ It compiles, and B.show is private

Wrong. Narrowing the access is refused.

✗ It throws an exception

Wrong. It is a compile error.

The fourth difference: an overridden method may not be made less accessible.

6 Strings

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
String a = "Gold";  
String b = "gold";  
System.out.println(a.equals(b) + " " + a.equalsIgnoreCase(b));
```

- a. false false
- b. false true
- c. true true
- d. true false

X false false

Wrong. equalsIgnoreCase ignores case.

✓ false true

Correct. equals compares characters exactly, so case matters; equalsIgnoreCase does not care about case.

X true true

Wrong. Gold and gold differ in case.

X true false

Wrong. They are the other way round.

Always compare text with equals; use equalsIgnoreCase for anything the user typed.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
StringBuilder sb = new StringBuilder();
for (int i = 1; i <= 3; i++) {
    sb.append(i).append("-");
}
System.out.println(sb.toString());
```

- a. -1-2-3
- b. 1-2-3-
- c. 1-2-3
- d. 123

X -1-2-3

Wrong. The number comes before its dash.

✓ 1-2-3-

Correct. Each pass appends the number and a dash, including after the last.

X 1-2-3

Wrong. A dash is appended after 3 too.

X 123

Wrong. Dashes are appended.

6.3

EASY

Lesson 2.9

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
System.out.println("banana".replace("a", "o"));
```

- a. bnn
- b. banana
- c. bonono
- d. bonana

X bnn

Wrong. The a's become o's; they are not removed.

X banana

Wrong. The replaced copy is what is printed.

✓ bonono

Correct. replace swaps EVERY a for an o and returns the new String.

X bonana

Wrong. Every occurrence is replaced, not just the first.

| replace(a, b) returns a copy with every a swapped for b.

The paper says to build a list "using plain String concatenation only, no StringBuilder".
What do you use?

- a. Whichever you prefer
- b. StringBuilder, because it is more efficient
- c. Plain concatenation with +, because the paper said so
- d. A char array

✗ Whichever you prefer

Wrong. The constraint decides.

✗ StringBuilder, because it is more efficient

Wrong. It is better in general, but the paper forbade it here.

✓ Plain concatenation with +, because the paper said so

Correct. Where the paper bans a tool, using it costs the marks whatever the output.

✗ A char array

Wrong. Not what the paper asked for.

Read the constraint before you choose the tool.

Why is `StringBuilder` preferred to `String` for building text in a loop?

- a. It is shorter to type
- b. `String` is immutable, so each join creates a new object, while `StringBuilder` changes one buffer in place
- c. `StringBuilder` can hold more characters than `String`
- d. `String` cannot be used inside a loop

✗ It is shorter to type

Wrong. It is longer to type, which is why the reason has to be a real one.

✓ `String` is immutable, so each join creates a new object, while `StringBuilder` changes one buffer in place

Correct. Joining strings in a loop makes a fresh object every pass and throws the previous one away. `StringBuilder` appends to a single mutable buffer.

✗ `StringBuilder` can hold more characters than `String`

Wrong. Neither has a practical limit you will meet on this course.

✗ `String` cannot be used inside a loop

Wrong. It can be used, it is just wasteful.

The answer is `String` is immutable, so each join creates a new object, while `StringBuilder` changes one buffer in place. Immutable means every join allocates. In a loop of n joins that is n objects created and discarded, which is why `StringBuilder` exists.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
String s = "java";  
String t = s.toUpperCase();  
s.concat("!");  
System.out.println(s + " " + t);
```

- a. JAVA! JAVA
- b. java JAVA
- c. java! JAVA
- d. JAVA JAVA

X JAVA! JAVA

Wrong. s is still java.

✓ java JAVA

Correct. Every String method returns a new String; s is never changed, and the concat result was thrown away.

X java! JAVA

Wrong. The concat result was never assigned.

X JAVA JAVA

Wrong. toUpperCase made a copy for t; s is unchanged.

| A String is immutable: if you did not assign the result, nothing happened.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
String s = "  hi  ";  
System.out.println "[" + s.trim() + "]" + s.length();
```

- a. [hi]6
- b. [hi]2
- c. [hi]6
- d. [hi]4

X [hi]6

Wrong. trim() removes the spaces in the copy that is printed.

X [hi]2

Wrong. s.length() measures the untrimmed s.

✓ [hi]6

Correct. trim() returns a copy without the spaces; s itself still has all six characters.

X [hi]4

Wrong. Two spaces on each side plus two letters is six.

| trim() returns a trimmed copy; the original keeps its spaces.

Asked twice. This same question also appears as: Manual O.10 Q2 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.6.

What happens when this is compiled?

```
String s = "hello";  
System.out.println(s.length);
```

- a. It does not compile: a String's length is a method, length()
- b. It prints 4
- c. It prints 0
- d. It prints 5

✓ It does not compile: a String's length is a method, length()

Correct. Strings have a length() method; arrays have a length field.

✗ It prints 4

Wrong. It never compiles.

✗ It prints 0

Wrong. It never compiles.

✗ It prints 5

Wrong. It never compiles.

| s.length() for a String, a.length for an array.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
System.out.println("banana".indexOf("x") + " " + "banana".indexOf("an"));
```

a. -1 1

b. -1 2

c. -1 3

d. 0 1

✓ -1 1

Correct. x is absent, so -1; the first an starts at index 1.

✗ -1 2

Wrong. b is index 0, a is index 1.

✗ -1 3

Wrong. indexOf finds the FIRST occurrence.

✗ 0 1

Wrong. indexOf gives -1 when the text is absent.

indexOf(x) is where x starts, or -1 if it is not there.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
public class Q701 {  
    public static void main(String[] args) {  
        String a = "hello";  
        String b = "hello";  
        String c = new String("hello");  
        System.out.println(a == b);  
        System.out.println(a == c);  
        System.out.println(a.equals(c));  
    }  
}
```

- a. true
true
true
- b. false
false
true
- c. true
false
true
- d. true
false

false

X true
true
true

Wrong on the second line. `new String()` deliberately builds a separate object, so the two references differ.

X false
false
true

Wrong on the first line: string literals are interned, so both names point at the same pooled object.

✓ true
false
true

Correct. Two identical literals share one pooled object, so `a == b` is true. `new String` makes a distinct object, so `a == c` is false. `equals` compares the CHARACTERS, so it is true.

X true
false
false

Wrong on the last line. `equals` compares contents, and the contents are the same.

The answer is true then false then true. `==` asks whether they are the same OBJECT; `equals` asks whether they read the same. For strings, always use `equals`. This is the most asked string question there is.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
public class Q702 {  
    public static void main(String[] args) {  
        String s = "java";  
        s.toUpperCase();  
        System.out.println(s);  
        System.out.println(s.toUpperCase());  
    }  
}
```

a. JAVA

JAVA

b. java

java

c. JAVA

java

d. java

JAVA

X JAVA

JAVA

Wrong on the first line. The result of the first call was thrown away, because nothing was assigned.

✗ java
java

Wrong on the second line: the value returned by the call is what is printed, and it is uppercase.

✗ JAVA
java

Wrong, and it has the two lines the wrong way round.

✓ java
JAVA

Correct. Strings are immutable, so `toUpperCase` returns a NEW string and leaves `s` alone. The first call changes nothing; the second prints its result directly.

The answer is java then JAVA. A String method never changes the string. If you do not catch what it returns, nothing happened. A statement that is only a method call on a String is almost always a bug.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
public class Q703 {  
    public static void main(String[] args) {  
        String s = "PROGRAM";  
        System.out.println(s.length());  
        System.out.println(s.charAt(0));  
        System.out.println(s.substring(1, 4));  
        System.out.println(s.indexOf("G"));  
    }  
}
```

a. 7

P

ROG

3

b. 7

P

ROGR

3

c. 7

R

ROG

4

d. 6
P
ROG
3

✓ 7
P
ROG
3

Correct. Seven characters; index 0 is P; substring(1, 4) takes indexes 1, 2 and 3, stopping BEFORE 4; and the first G is at index 3.

✗ 7
P
ROGR
3

Wrong. substring stops before its second index, so it takes three characters, not four.

✗ 7
R
ROG
4

Wrong. charAt(0) is the first character, P. Java indexes from zero.

✗ 6
P
ROG
3

Wrong. PROGRAM has seven letters.

The answer is 7 then P then ROG then 3. length() has brackets on a String and none on an array, index from zero, and substring excludes its end index. Those three answer most string questions.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
String s = "EXAMINATION";  
System.out.println(s.substring(2, 5) + " " + s.charAt(4) + " " + s.indexOf("NATION"));
```

- a. AMI I 5
- b. AMIN I 5
- c. AMI N 5
- d. AMI I 6

✓ AMI I 5

Correct. substring(2, 5) takes indexes 2 to 4; charAt(4) is I; NATION starts at index 5.

✗ AMIN I 5

Wrong. substring stops BEFORE its end index.

✗ AMI N 5

Wrong. Index 4 is I; N is at 5.

✗ AMI I 6

Wrong. Count from 0: N of NATION is at index 5.

| substring(a, b) runs from a up to but not including b; indexes start at 0.

7 Arrays

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
int[] a = {4, 8, 15, 16};  
System.out.println(a[a.length - 1] + " " + a.length);
```

- a. 16 3
- b. 16 4
- c. an `ArrayIndexOutOfBoundsException`
- d. 15 4

✗ 16 3

Wrong. length counts the elements, 4.

✓ 16 4

Correct. The last index is `length - 1`, which is 3, holding 16; length is 4.

✗ an `ArrayIndexOutOfBoundsException`

Wrong. `a.length - 1` is a valid index.

✗ 15 4

Wrong. Index 3 holds 16.

■ The last index is always `length - 1`.

Asked twice. This same question also appears as: Manual O.10 Q2 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.8 and Lesson 2.9 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.6.

What happens when this is compiled?

```
int[] a = {1.5, 2.0};  
System.out.println(a[0]);
```

- a. It prints 2
- b. It does not compile: 1.5 and 2.0 are doubles, which do not fit an int array
- c. It prints 1
- d. It prints 1.5

✗ It prints 2

Wrong. It never compiles.

✓ It does not compile: 1.5 and 2.0 are doubles, which do not fit an int array

Correct. This is the planted error from the last paper: fix it with double[] or whole values.

✗ It prints 1

Wrong. It never compiles.

✗ It prints 1.5

Wrong. It never compiles.

| An int[] filled with decimals does not compile, and it is on the 2025/2026 paper.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
public class Q802 {
    public static void main(String[] args) {
        int[] a = {1, 2, 3};
        try {
            System.out.println(a[3]);
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("caught " + e.getMessage());
        }
    }
}
```

- a. 3
- b. caught null
- c. caught Index 3 out of bounds for length 3
- d. 0

X 3

Wrong. Index 3 is the FOURTH element, and there are only three.

X caught null

Wrong. The exception carries a message describing the index and the length.

✓ caught Index 3 out of bounds for length 3

Correct. Valid indexes are 0, 1 and 2, so `a[3]` throws, and the catch prints the exception's own message.

✗ 0

Wrong. Reading past the end does not return a default; it throws.

The answer is caught Index 3 out of bounds for length 3. An array of length `n` has indexes 0 to `n` minus 1. The last valid index is always length minus one, and this off-by-one is the most common runtime error in the whole course.

7.4

MEDIUM

Lesson 2.8

Asked twice. This same question also appears as: Lesson 1.5 and Lesson 2.2 and Lesson 3.1 and Lesson 3.5 and Lesson 3.6.

What happens when this runs?

```
int[] a = {4, 8, 15, 16};  
System.out.println(a[a.length]);
```

- a. It prints 0
- b. It prints 16
- c. It throws `ArrayIndexOutOfBoundsException`
- d. It does not compile

✗ It prints 0

Wrong. Java does not give a default for a missing index.

✗ It prints 16

Wrong. 16 is at index 3.

✓ It throws `ArrayIndexOutOfBoundsException`

Correct. Valid indexes run 0 to 3; index 4 is past the end.

✗ It does not compile

Wrong. It compiles; the fault shows only when it runs.

`i <= a.length` is one step too far, every time.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
static int countAbove(int[] a, int mark) {
    int n = 0;
    for (int i = 0; i < a.length; i++) {
        if (a[i] > mark) {
            n++;
        }
    }
    return n;
}

// in main:
System.out.println(countAbove(new int[]{3, 9, 12, 9}, 9));
```

- a. 0
- b. 1
- c. 2
- d. 3

X 0

Wrong. 12 is above 9.

✓ 1

Correct. Only 12 is strictly above 9; the two 9s are not.

X 2

Wrong. Only one value is above 9.

X 3

Wrong. > does not include 9.

Count matching: $n = 0$, if the test passes $n++$, return n .

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
static int firstIndexOf(int[] a, int wanted) {
    for (int i = 0; i < a.length; i++) {
        if (a[i] == wanted) {
            return i;
        }
    }
    return -1;
}

// in main:
int[] a = {7, 3, 7, 1};
System.out.println(firstIndexOf(a, 7) + " " + firstIndexOf(a, 5));
```

- a. 0 -1
- b. 0 0
- c. 1 -1
- d. 2 -1

✓ 0 -1

Correct. 7 is found first at index 0 and the method returns at once; 5 is absent, so -1.

✗ 0 0

Wrong. -1 means not found.

X 1 -1

Wrong. Index 0 holds 7.

X 2 -1

Wrong. The method returns the FIRST match.

| Return from inside the loop on the first match; return -1 after the loop.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
int[][] m = {{1, 2}, {3, 4, 5}};  
System.out.println(m[1].length + " " + m[1][2]);
```

- a. 2 3
- b. 2 5
- c. 3 5
- d. 3 4

X 2 3

Wrong. m.length would be 2; m[1].length is 3.

X 2 5

Wrong. m[1].length is the width of row 1, which is 3.

✓ 3 5

Correct. Row 1 is {3, 4, 5}: its length is 3 and its index 2 is 5.

X 3 4

Wrong. m[1][2] is the third item of row 1.

| The first index picks the row, the second picks within it; each row has its own length.

Asked twice. This same question also appears as: Manual O.10 Q2 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.8 and Lesson 2.9 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.6.

What happens when this is compiled?

```
int[] a = {1, 2, 3};  
System.out.println(a.length());
```

- a. It does not compile: an array's length is a field, written without brackets
- b. It prints 3
- c. It prints 0
- d. It prints 2

✓ It does not compile: an array's length is a field, written without brackets

Correct. Arrays have a length field; Strings have a length() method.

✗ It prints 3

Wrong. It never compiles.

✗ It prints 0

Wrong. It never compiles.

✗ It prints 2

Wrong. It never compiles.

| a.length for an array, s.length() for a String. The examiner plants the swap.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
int[] a = {2, 3, 4, 4};
int s = 0;
for (int i = 0; i < a.length; i++) {
    s += a[i];
}
System.out.println((double) s / a.length);
```

- a. 13
- b. 3.0
- c. 3
- d. 3.25

X 13

Wrong. That is the total, not the average.

X 3.0

Wrong. The cast happens before the division, so the remainder survives.

X 3

Wrong. The cast makes this a double division.

✓ 3.25

Correct. The total is 13, and casting it to double before dividing by 4 gives 3.25.

| An average of an int array needs a cast on one side of the division.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
public class Q801 {  
    public static void main(String[] args) {  
        int[] a = new int[3];  
        a[0] = 5;  
        System.out.println(a.length);  
        System.out.println(a[1]);  
        System.out.println(a[0]);  
    }  
}
```

- a. 3
null
5
- b. 3
0
5
- c. 2
0
5
- d. 3
0
0

✗ 3
null
5

Wrong. null is the default for an OBJECT array. A numeric array defaults to zero.

✓ 3
0
5

Correct. length is a FIELD on an array, with no brackets. An int array is filled with 0 when created, and a[0] was then set to 5.

✗ 2
0
5

Wrong. new int[3] gives three elements, indexed 0, 1 and 2.

✗ 3
0
0

Wrong. a[0] was assigned 5 on the second line.

The answer is 3 then 0 then 5. a.length with no brackets for an array, s.length() with brackets for a String. Arrays are created filled with the zero of their type: 0, 0.0, false or null.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
public class Q803 {  
    public static void main(String[] args) {  
        int[][] m = {{1, 2, 3}, {4, 5, 6}};  
        System.out.println(m.length);  
        System.out.println(m[0].length);  
        System.out.println(m[1][2]);  
    }  
}
```

a. 3

2

6

b. 2

3

5

c. 6

3

6

d. 2

3

6

X 3

2

6

Wrong, and it swaps rows and columns. The outer length is always the number of rows.

X 2

3

5

Wrong. `m[1][2]` is the third element of the second row, which is 6, not 5.

X 6

3

6

Wrong. `m.length` counts rows, not the total number of elements.

✓ 2

3

6

Correct. `m.length` is the number of ROWS, two. `m[0].length` is the length of the first row, three. `m[1][2]` is row 1, column 2, which is 6.

The answer is 2 then 3 then 6. Outer index is the row, inner index is the column, and both count from zero. `m.length` is rows; `m[i].length` is the width of row `i`.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
public class Q804 {
    public static void main(String[] args) {
        int[] a = {-5, -2, -9};
        int max = 0;
        for (int x : a) if (x > max) max = x;
        System.out.println(max);
    }
}
```

- a. 0
- b. -2
- c. -9
- d. -5

✓ 0

Correct, and it is the bug. Starting max at 0 means no negative value can ever beat it, so 0 survives even though it is not in the array.

✗ -2

Wrong, and it is the answer the program was MEANT to give. It does not, because of the way max was initialised.

✗ -9

Wrong. -9 is the smallest, not the largest.

X -5

Wrong. -2 is larger than -5, so -5 could not win even with a correct start.

The answer is 0. Start max at the FIRST ELEMENT, never at zero. This exact fault is a standing exam favourite, and the fix is one line: `int max = a[0]`, then loop from index 1.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
static int countConsecutive(int[] a) {
    int n = 0;
    for (int i = 1; i < a.length; i++) {
        if (a[i] == a[i - 1]) {
            n++;
        }
    }
    return n;
}

// in main:
System.out.println(countConsecutive(new int[]{1, 1, 1, 2, 2, 3}));
```

- a. 5
- b. 3
- c. 2
- d. 4

X 5

Wrong. Each match compares a position with the one before it.

✓ 3

Correct. Indexes 1 and 2 equal the 1 before them, and index 4 equals the 2 before it.

X 2

Wrong. A run of three 1s gives two matches, not one.

X 4

Wrong. The 3 at the end has no equal neighbour.

The neighbour walk starts at 1 and compares $a[i]$ with $a[i - 1]$; a run of three gives two matches.

8 Recursion

8.1

EASY

Manual O.9 Q3

Which statement about recursion compared with iteration is TRUE?

- a. Recursion is always faster because it uses fewer instructions
- b. Any recursive method is impossible to write as a loop
- c. Recursion uses stack space for every pending call, so a deep recursion can exhaust it
- d. Recursion never needs a stopping condition

X Recursion is always faster because it uses fewer instructions

Wrong. Recursion is usually SLOWER: every call costs a stack frame and the work of calling.

X Any recursive method is impossible to write as a loop

Wrong. Anything recursive can be written iteratively, sometimes with an explicit stack of your own.

✓ Recursion uses stack space for every pending call, so a deep recursion can exhaust it

Correct. Each call in progress keeps its frame until it returns, so depth costs memory, and running out of it is `StackOverflowError`.

X Recursion never needs a stopping condition

Wrong. Without a base case it never stops, and the program dies.

The answer is Recursion uses stack space for every pending call, so a deep recursion can exhaust it. Recursion buys clarity on naturally recursive problems and pays for it in stack space and call overhead. Both sides of that trade earn marks.

What is the BASE CASE of a recursive method?

- a. The condition under which it returns without calling itself
- b. The call that passes a smaller input
- c. The first call made from main
- d. The largest value it can handle

✓ The condition under which it returns without calling itself

Correct. Every recursive method needs one, or it never stops.

✗ The call that passes a smaller input

Wrong. That is the recursive case.

✗ The first call made from main

Wrong. That is just the first call.

✗ The largest value it can handle

Wrong. Not a definition in the course.

Base case stops it, recursive case shrinks it.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
static int pow(int b, int e) {  
    if (e == 0) {  
        return 1;  
    }  
    return b * pow(b, e - 1);  
}  
  
// in main:  
System.out.println(pow(2, 5));
```

- a. 64
- b. 32
- c. 16
- d. 10

X 64

Wrong. That is 2 to the power 6.

✓ 32

Correct. $\text{pow}(2, 5)$ is $2 \times 2 \times 2 \times 2 \times 2 \times 1 = 32$.

X 16

Wrong. That is 2 to the power 4.

X 10

Wrong. That multiplies 2 by 5.

Base case $e == 0$ returns 1; each call multiplies by b and shrinks e .

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
static void down(int n) {  
    if (n < 1) {  
        return;  
    }  
    System.out.print(n + " ");  
    down(n - 1);  
}  
  
// in main:  
down(3);
```

- a. 3
- b. 3 2 1 0
- c. 1 2 3
- d. 3 2 1

X 3

Wrong. down calls itself until n is below 1.

X 3 2 1 0

Wrong. At 0 the base case returns without printing.

X 1 2 3

Wrong. That is the order when the print comes after the call.

✓ 3 2 1

Correct. Each call prints before it recurses, so the numbers come out on the way down.

Where the print sits relative to the recursive call decides the order.

8.5

MEDIUM

Lesson 3.5

Asked twice. This same question also appears as: Lesson 1.5 and Lesson 2.2 and Lesson 2.8 and Lesson 3.1 and Lesson 3.6.

What happens when this runs?

```
static int bad(int n) {  
    return bad(n - 1);  
}  
  
// in main:  
System.out.println(bad(3));
```

- a. It fails with `StackOverflowError`
- b. It loops quietly for ever
- c. It prints 0
- d. It does not compile

✓ It fails with `StackOverflowError`

Correct. With no base case every call makes another, until the call stack is full.

✗ It loops quietly for ever

Wrong. Each call uses a stack frame, so it runs out of memory quickly.

✗ It prints 0

Wrong. Nothing ever returns.

✗ It does not compile

Wrong. It compiles.

A recursion with no base case is the recursive infinite loop: `StackOverflowError`.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
static int ds(int n) {
    if (n < 10) {
        return n;
    }
    return n % 10 + ds(n / 10);
}

// in main:
System.out.println(ds(9045));
```

- a. 18
- b. 9045
- c. 4
- d. 27

✓ 18

Correct. $5 + 4 + 0 + 9 = 18$: each call peels the last digit off with `% 10`.

✗ 9045

Wrong. The method adds the digits.

✗ 4

Wrong. That counts the digits.

X 27

Wrong. Add each digit once.

| The base case is a single digit; the recursive case adds the last digit to the sum of the rest.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
static String reverse(String s) {  
    if (s.length() <= 1) {  
        return s;  
    }  
    return reverse(s.substring(1)) + s.charAt(0);  
}  
  
// in main:  
System.out.println(reverse("CODE"));
```

- a. CODE
- b. EDOC
- c. DEOC
- d. ODEC

X CODE

Wrong. The method reverses the text.

✓ EDOC

Correct. Each call puts the first character on the END of the reversed rest.

X DEOC

Wrong. Trace one level at a time.

X ODEC

Wrong. Trace it: `reverse("ODE") + 'C'`, and so on down.

| `reverse(s.substring(1)) + s.charAt(0)`: first character last.

8.8

MEDIUM

Lesson 3.5

Which uses more memory, and why?

- a. Iteration, because a loop keeps every value
- b. Recursion uses no memory at all
- c. They use exactly the same
- d. Recursion, because every call in progress holds a stack frame until the base case returns

X Iteration, because a loop keeps every value

Wrong. A loop reuses the same variables.

X Recursion uses no memory at all

Wrong. Each call occupies memory.

X They use exactly the same

Wrong. Each recursive call adds a frame.

✓ Recursion, because every call in progress holds a stack frame until the base case returns

Correct. A loop uses one set of variables however many times it goes round.

| Recursion is clearer for self-similar problems and heavier in memory; iteration is the default.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
public class Q901 {  
    static int fact(int n) {  
        if (n <= 1) return 1;  
        return n * fact(n - 1);  
    }  
    public static void main(String[] args) {  
        System.out.println(fact(5));  
    }  
}
```

- a. 120
- b. 15
- c. 5
- d. StackOverflowError

✓ 120

Correct. $5 \times 4 \times 3 \times 2 \times 1$ is 120, and the base case at $n \leq 1$ stops the recursion.

✗ 15

Wrong. 15 is the SUM $1+2+3+4+5$. This method multiplies.

✗ 5

Wrong. The method does not simply return its argument.

X StackOverflowError

Wrong. There IS a base case and each call moves towards it, so the recursion terminates.

The answer is 120. Every recursion needs a base case and a step that moves towards it. Remove either and you get StackOverflowError, which is what the paper asks you to identify.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
public class Q902 {
    static int fib(int n) {
        if (n <= 1) return n;
        return fib(n - 1) + fib(n - 2);
    }
    public static void main(String[] args) {
        for (int i = 0; i < 7; i++) System.out.print(fib(i) + " ");
    }
}
```

- a. 1 1 2 3 5 8 13
- b. 0 1 1 2 3 5 8
- c. 0 1 2 3 4 5 6
- d. 1 2 3 5 8 13 21

X 1 1 2 3 5 8 13

Wrong. This version starts at 0, because fib(0) returns 0.

✓ 0 1 1 2 3 5 8

Correct. The base case returns n itself for 0 and 1, so the sequence starts 0, 1, and each later term is the sum of the two before it.

X 0 1 2 3 4 5 6

Wrong. That is just the counting numbers, not a sum of the two previous terms.

X 1 2 3 5 8 13 21

Wrong. Seven terms from index 0 ends at 8, not 21.

The answer is 0 1 1 2 3 5 8 . Two base cases here, folded into one line: $\text{fib}(0)$ is 0 and $\text{fib}(1)$ is 1.
Trace the first four calls on paper and the pattern becomes obvious.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
static void up(int n) {
    if (n < 1) {
        return;
    }
    up(n - 1);
    System.out.print(n + " ");
}

// in main:
up(3);
```

- a. 3
- b. 0 1 2 3
- c. 1 2 3
- d. 3 2 1

X 3

Wrong. Every level prints once.

X 0 1 2 3

Wrong. At 0 nothing is printed.

✓ 1 2 3

Correct. Each call recurses FIRST and prints afterwards, so the prints happen on the way back up.

✗ 3 2 1

Wrong. That would need the print before the call.

Nothing is done on the way down that comes after the call; it all happens on the way back up.

9 Exceptions and File Input and Output

9.1

EASY

Manual O.10 Q3

Asked twice. This same question also appears as: Manual O.2 Q5.

Which statement is TRUE?

- a. A checked exception must be caught or declared with throws
- b. An unchecked exception must always be caught
- c. All exceptions in Java are checked
- d. A checked exception cannot be caught

✓ A checked exception must be caught or declared with throws

Correct. The compiler insists: handle it, or declare that your method may throw it. `IOException` is the one this course meets most.

✗ An unchecked exception must always be caught

Wrong. Unchecked exceptions, the `RuntimeException` family, need neither. `NullPointerException` is the standard example.

✗ All exceptions in Java are checked

Wrong. Everything descending from `RuntimeException` and `Error` is unchecked.

✗ A checked exception cannot be caught

Wrong. It can be caught; it simply may not be ignored.

The answer is A checked exception must be caught or declared with throws. Checked means the compiler makes you deal with it. Unchecked means it is your job to avoid it, and the commonest ones, null pointer and array index, are bugs rather than conditions.

Which class does this course use to read a text file line by line, and what must surround it?

- a. System.out, inside a for loop
- b. Scanner or BufferedReader, inside a try block that handles IOException
- c. FileWriter, inside a switch
- d. String, with no exception handling needed

✗ System.out, inside a for loop

Wrong. System.out is the output stream, not a way in.

✓ Scanner or BufferedReader, inside a try block that handles IOException

Correct. Either class can read lines, and file work throws a CHECKED exception, so it must be caught or declared.

✗ FileWriter, inside a switch

Wrong. FileWriter writes. Reading needs a reader.

✗ String, with no exception handling needed

Wrong. String is a type, not a file, and the exception handling is not optional for file access.

The answer is Scanner or BufferedReader, inside a try block that handles IOException. Reading needs a reader and reading can fail, so it needs a catch. Both halves of that sentence earn marks.

Which goes in a method HEADER to say the method may raise an exception it does not handle?

- a. finally
- b. catch
- c. throw
- d. throws

✗ finally

Wrong. finally is the cleanup block.

✗ catch

Wrong. catch handles one.

✗ throw

Wrong. throw raises one now, inside the body.

✓ throws

Correct. throws, with the s, declares it.

throw raises one, throws warns about one.

Asked twice. This same question also appears as: Lesson 3.7 and Lesson 3.7 and Lesson 3.7.

main declares throws IOException. What does this print?

```
PrintWriter out = new PrintWriter("log.txt");
out.println("first");
out.close();
PrintWriter more = new PrintWriter(new FileWriter("log.txt", true));
more.println("second");
more.close();
Scanner in = new Scanner(new File("log.txt"));
while (in.hasNextLine()) {
    System.out.println(in.nextLine());
}
in.close();
```

- a. second
first
- b. second
- c. first
- d. first
second

X second
first

Wrong. Appended lines go at the end.

X second

Wrong. The true keeps what was there.

X first

Wrong. The appended line is there too.

✓ first
second

Correct. FileWriter with true opens for append, so the new line goes after the old one.

| new PrintWriter(new FileWriter(name, true)): the true is the whole difference.

Asked twice. This same question also appears as: Lesson 3.7 and Lesson 3.7 and Lesson 3.7.

main declares throws IOException. What does this print?

```
PrintWriter out = new PrintWriter("r.txt");
out.println("a");
out.println("b");
out.println("c");
out.close();
Scanner in = new Scanner(new File("r.txt"));
int n = 0;
while (in.hasNextLine()) {
    in.nextLine();
    n++;
}
in.close();
System.out.println(n);
```

a. 0

b. 2

c. 4

d. 3

X 0

Wrong. The writer was closed, so the lines reached the file.

X 2

Wrong. All three lines are read.

X 4

Wrong. Only three lines were written.

✓ 3

Correct. hasNextLine keeps the loop going for each of the three lines written.

| while (in.hasNextLine()) reads a file of any length.

Which line opens records.txt to ADD records without destroying the existing ones?

- a. `new FileWriter("records.txt", false)`
- b. `new PrintWriter("records.txt")`
- c. `new PrintWriter(new FileWriter("records.txt", true))`
- d. `new Scanner(new File("records.txt"))`

✗ `new FileWriter("records.txt", false)`

Wrong. false means overwrite.

✗ `new PrintWriter("records.txt")`

Wrong. That wipes the file.

✓ `new PrintWriter(new FileWriter("records.txt", true))`

Correct. FileWriter with true appends.

✗ `new Scanner(new File("records.txt"))`

Wrong. A Scanner reads; it does not write.

The file question always asks you to add to the file you made earlier. That one true is the mark.

Why do the file listings write `throws IOException` on `main`?

- a. To make the program run faster
- b. File operations raise checked exceptions, which must be caught or declared
- c. Because every Java program needs it
- d. To close the file automatically

✗ To make the program run faster

Wrong. It has nothing to do with speed.

✓ File operations raise checked exceptions, which must be caught or declared

Correct. Declaring them is the short, legal way to satisfy the compiler.

✗ Because every Java program needs it

Wrong. Only methods that can raise checked exceptions need it.

✗ To close the file automatically

Wrong. You must still close the file.

✓ Catch them, or declare `throws IOException`.

Asked twice. This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.6.

What happens when this is compiled?

```
public class Q1002 {  
    public static void main(String[] args) {  
        try {  
            int x = 1 / 0;  
        } catch (Exception e) {  
            System.out.println("general");  
        } catch (ArithmeticException e) {  
            System.out.println("arithmetic");  
        }  
    }  
}
```

- a. It prints general
- b. It prints arithmetic
- c. It prints both lines
- d. It does not compile: the second catch can never be reached

X It prints general

Wrong. It never runs at all, because it does not compile.

X It prints arithmetic

Wrong for the same reason, and the general catch would win in any case.

X It prints both lines

Wrong. At most one catch clause ever runs for a given exception.

✓ It does not compile: the second catch can never be reached

Correct. Exception is the superclass of ArithmeticException, so the first catch already covers everything the second could. Java rejects unreachable catch clauses at compile time.

The answer is It does not compile: the second catch can never be reached. Catch the most specific exception FIRST and the general one last. Reversing them is a compile error, not a subtle bug, which is the helpful part.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
public class Q1005 {  
    public static void main(String[] args) {  
        String s = null;  
        try {  
            System.out.println(s.length());  
        } catch (NullPointerException e) {  
            System.out.println("null pointer");  
        }  
    }  
}
```

- a. 0
- b. null
- c. null pointer
- d. it does not compile

X 0

Wrong. There is no object to measure, so nothing returns 0.

X null

Wrong. Printing s alone would show null; calling a METHOD on it throws.

✓ null pointer

Correct. `s` refers to no object, so calling a method on it throws `NullPointerException`, which the catch handles.

X it does not compile

Wrong. It compiles perfectly. The fault only appears when it runs, which is what unchecked means.

The answer is null pointer. null means the reference points at no object. Reading it is fine; calling a method on it is not. Check for null before you dereference.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
try {  
    int[] a = new int[2];  
    a[2] = 5;  
    System.out.println("stored");  
} catch (ArithmeticException e) {  
    System.out.println("math");  
} catch (ArrayIndexOutOfBoundsException e) {  
    System.out.println("index");  
} finally {  
    System.out.println("done");  
}
```

- a. stored
done
- b. index
done
- c. math
done
- d. index

X stored
done

Wrong. The failing line stops the try before stored is printed.

✓ index
done

Correct. `a[2]` is past the end, so the `ArrayIndexOutOfBoundsException` catch runs, then finally.

✗ math
done

Wrong. Nothing divides by zero.

✗ index

Wrong. finally always runs.

✓ The matching catch runs; finally runs every time.

Asked twice. This same question also appears as: Lesson 1.5 and Lesson 2.2 and Lesson 2.8 and Lesson 3.1 and Lesson 3.5.

What happens when this runs?

```
try {  
    String s = null;  
    System.out.println(s.length());  
} catch (ArithmeticException e) {  
    System.out.println("math");  
}
```

- a. A `NullPointerException` stops the program; the `ArithmeticException` catch does not match it
- b. It prints math
- c. It does not compile
- d. It prints 0

✓ A `NullPointerException` stops the program; the `ArithmeticException` catch does not match it

Correct. A catch only catches its own type.

✗ It prints math

Wrong. Nothing divides by zero.

✗ It does not compile

Wrong. It compiles.

✗ It prints 0

Wrong. `s` is null, so `length()` fails.

■ Name the exception you expect; others pass straight through.

main does NOT declare throws. What happens when this is compiled?

```
FileReader r = new FileReader("missing.txt");  
System.out.println("opened");
```

- a. It prints nothing
- b. It prints opened
- c. It does not compile: FileNotFoundException is checked and must be caught or declared
- d. It throws FileNotFoundException when it runs

X It prints nothing

Wrong. It never compiles.

X It prints opened

Wrong. It never compiles.

✓ It does not compile: FileNotFoundException is checked and must be caught or declared

Correct. The compiler insists on checked exceptions: catch it or write throws.

X It throws FileNotFoundException when it runs

Wrong. It is stopped before it can run.

| Checked: the compiler makes you deal with it. File work is the everyday example.

Which of these is a **CHECKED** exception?

- a. IOException
- b. ArithmeticException
- c. NumberFormatException
- d. NullPointerException

✓ **IOException**

Correct. It comes from outside the program, so the compiler makes you handle it.

✗ **ArithmeticException**

Wrong. Unchecked: dividing by zero is your fault.

✗ **NumberFormatException**

Wrong. Unchecked: bad text given to parseInt.

✗ **NullPointerException**

Wrong. Unchecked: a fault in the program's own logic.

Checked extends Exception but not RuntimeException; unchecked extends RuntimeException.

Asked twice. This same question also appears as: Lesson 3.7 and Lesson 3.7 and Lesson 3.7.

main declares throws IOException. What does this print?

```
PrintWriter out = new PrintWriter("log.txt");
out.println("first");
out.close();
PrintWriter again = new PrintWriter("log.txt");
again.println("second");
again.close();
Scanner in = new Scanner(new File("log.txt"));
while (in.hasNextLine()) {
    System.out.println(in.nextLine());
}
in.close();
```

- a. Nothing
- b. first
second
- c. second
- d. first

X Nothing

Wrong. The second line was written and the writer closed.

X first
second

Wrong. That needs the append form with FileWriter and true.

✓ second

Correct. new PrintWriter on a filename starts an empty file every time, so the first line is gone.

X first

Wrong. The second writer replaced it.

| PrintWriter on a name wipes it.

Asked twice. This same question also appears as: Lesson 3.7 and Lesson 3.7 and Lesson 3.7.

main declares throws IOException. What does this print?

```
PrintWriter out = new PrintWriter("r.txt");
for (int i = 1; i <= 5; i++) {
    out.println("Rec " + i);
}
out.close();
Scanner in = new Scanner(new File("r.txt"));
int n = 0;
while (in.hasNextLine()) {
    String line = in.nextLine();
    n++;
    if (n > 2) break;
    System.out.println(line);
}
in.close();
```

a. Rec 1

Rec 2

Rec 3

b. Rec 3

c. Rec 1

d. Rec 1

Rec 2

X Rec 1

Rec 2

Rec 3

Wrong. The break comes before the third print.

X Rec 3

Wrong. The first lines print; the third is where it stops.

X Rec 1

Wrong. Two lines print before the break.

✓ Rec 1

Rec 2

Correct. The count reaches 3 on the third line, and break leaves before printing it.

Display only the first few: count as you read and break.

Which import is needed for new `File("records.txt")`, separately from `Scanner`?

- a. `import java.io.File;`
- b. No import is needed
- c. `import java.util.File;`
- d. `import javax.swing.File;`

✓ `import java.io.File;`

Correct. `File` lives in `java.io`, `Scanner` in `java.util`.

✗ No import is needed

Wrong. `File` must be imported.

✗ `import java.util.File;`

Wrong. `java.util` is `Scanner`'s package, not `File`'s.

✗ `import javax.swing.File;`

Wrong. `javax.swing` is for `JOptionPane`.

! `java.io.PrintWriter`, `java.io.FileWriter`, `java.io.File`, `java.io.IOException`, `java.util.Scanner`.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
public class Q1001 {
    static int f() {
        try {
            return 1;
        } finally {
            System.out.println("finally");
        }
    }
    public static void main(String[] args) {
        System.out.println(f());
    }
}
```

- a. 1
- b. 1
finally
- c. finally
1
- d. finally

X 1

Wrong. The finally block runs even when the try block returns.

✗ 1

finally

Wrong on the order. finally runs BEFORE the method actually returns to its caller, so its output appears first.

✓ finally

1

Correct. The return value is computed, then finally runs on the way out, and only then does the caller receive the 1 and print it.

✗ finally

Wrong. The method does return 1, and main prints it.

The answer is finally then 1. finally runs on every path out of a try, including a return. That is why it is where cleanup belongs.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
try {  
    int n = Integer.parseInt("12a");  
} catch (NumberFormatException e) {  
    System.out.println(e.getMessage());  
}
```

a. NumberFormatException

b. For input string: "12a"

c. 12a

d. 12

X NumberFormatException

Wrong. getMessage gives the detail, not the class name.

✓ For input string: "12a"

Correct. e.getMessage() gives the detail the exception carries.

X 12a

Wrong. The message says more than the text.

X 12

Wrong. parseInt refuses the whole text.

The variable e holds the exception object, and e.getMessage() gives the detail.

9.19

HARD

Lesson 3.6

Asked twice. This same question also appears as: Manual O.10 Q2 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4.

What happens when this is compiled?

```
try {  
    int x = 1;  
    System.out.println(x);  
} catch (IOException e) {  
    System.out.println("io");  
}
```

- a. It prints 1
- b. It prints nothing
- c. It does not compile: the try block cannot throw IOException
- d. It prints io

✗ It prints 1

Wrong. It never compiles.

✗ It prints nothing

Wrong. It never compiles.

✓ It does not compile: the try block cannot throw IOException

Correct. Java refuses a catch for a checked exception the try block can never throw.

✗ It prints io

Wrong. It never compiles.

Catching a checked exception you never cause is a compile error.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
static int check(int day) {
    if (day < 1 || day > 3) {
        throw new IllegalArgumentException("Day must be 1, 2 or 3");
    }
    return day * 10;
}

// in main:
try {
    System.out.println(check(2));
    System.out.println(check(4));
} catch (IllegalArgumentException e) {
    System.out.println("Error: " + e.getMessage());
}
```

- a. 20
- b. 20
Error: Day must be 1, 2 or 3
- c. Error: Day must be 1, 2 or 3
- d. 20
40

✗ 20

Wrong. The second call's exception is caught and printed.

✓ 20

Error: Day must be 1, 2 or 3

Correct. check(2) returns 20; check(4) throws, and the catch prints its message.

✗ Error: Day must be 1, 2 or 3

Wrong. The first call succeeds and prints 20.

✗ 20

40

Wrong. 4 is rejected before any multiplication.

throw new IllegalArgumentException(...) in the method; catch it where it is called.

9.21

HARD

Lesson 3.7

The writer is never closed. main declares throws IOException. What does this print?

```
PrintWriter out = new PrintWriter("f.txt");  
out.println("hello");  
Scanner in = new Scanner(new File("f.txt"));  
System.out.println(in.hasNextLine());
```

a. true

b. false

c. hello

d. It does not compile

✗ true

Wrong. Nothing reached the file without close().

✓ false

Correct. The text is still in the writer's buffer, so the file on disk is empty and has no line.

✗ hello

Wrong. The program prints whether a line exists, and none does.

✗ It does not compile

Wrong. It compiles.

Close it, or nothing was written.